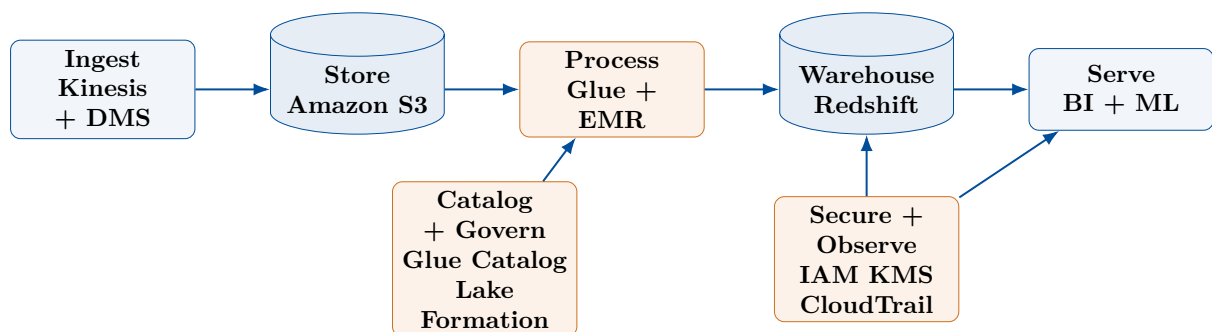
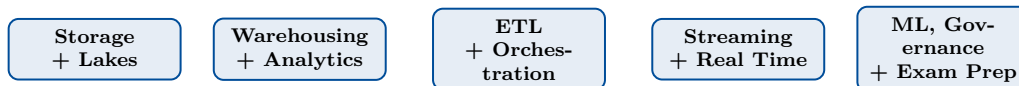

A COMPREHENSIVE, HANDS-ON ACADEMIC GUIDE

AWS Master Data Engineer

Data Engineering on Amazon Web Services



Inside this book



From durable object storage to governed lakehouse, streaming, analytics, and machine-learning data platforms.

PhD Aned Esquerra Arguelles

A 24-week, milestone-driven academic and hands-on AWS data engineering program

August 31, 2026

Preface

Welcome to *AWS Master Data Engineer*. This text is designed as an academic, hands-on, progressive program for learning how modern data platforms are built on Amazon Web Services. By working through the architecture discussions, service trade-offs, and weekly labs, you will move from foundational object storage to secure, governed, production-grade data lakehouse and analytics systems.

This book is the theory-and-architecture companion to a 24-week AWS Data Engineering training program. Where the weekly practice workspace asks you to *deploy* services and build projects, this book explains *why* each service exists, what operational failure mode it addresses, and how a data engineer reasons about scale, security, governance, cost, and reliability.

Who This Book Is For

This book is written for aspiring and practicing data engineers, cloud engineers, analytics engineers, database developers, and technical leads who want a rigorous path into AWS data engineering. It assumes basic SQL, Python, command-line comfort, and familiarity with data formats such as CSV, JSON, and Parquet. Prior AWS experience helps, but the first part establishes the storage, identity, networking, and lake foundations used throughout the rest of the program.

How This Book Is Organized

The book follows six Parts, matching six curriculum milestones, and 24 chapters, matching 24 training weeks:

- **Part I: Core Data Infrastructure and Storage** builds the base: Amazon S3, IAM and networking concepts, data lake layout, and Athena query patterns.
- **Part II: Data Warehousing and Analytics** develops Amazon Redshift, performance design, semantic analytics, QuickSight, and cost-aware analytical platforms.
- **Part III: Batch Processing and ETL Orchestration** covers the AWS Glue Data Catalog, Glue ETL with Spark, Amazon EMR, Step Functions, and managed orchestration patterns.
- **Part IV: Streaming and Real-Time Analytics** introduces Kinesis, Firehose, Amazon MSK, Lambda stream processing, and real-time architecture trade-offs.
- **Part V: Machine Learning and MLOps** connects data engineering to SageMaker, feature stores, ML data pipelines, model monitoring, and operational ML workflows.
- **Part VI: Security, Governance, and Exam Prep** consolidates Lake Formation, compliance, observability, architecture review, capstone work, and certification-oriented re-

view.

Appendices provide selected command references, design checklists, troubleshooting playbooks, cost-estimation patterns, and worked solutions for selected exercises.

Reading Paths

- **New to AWS data engineering:** read Parts I–III in order before attempting streaming, MLOps, or governance-heavy architectures.
- **Already using AWS analytics:** skim Chapter 1 for terminology, then focus on Redshift, Glue, EMR, and Kinesis chapters relevant to your current platform.
- **Building a portfolio:** complete each hands-on project and keep architecture diagrams, CLI commands, failure notes, and cost estimates as evidence.
- **Preparing for interviews or certification:** work every Key Takeaway, Interview Q&A, and Exercise section; practice explaining trade-offs rather than memorizing service names.
- **Operating a regulated platform:** read the security and governance material alongside every technical chapter, not only at the end.

Conventions

Code identifiers, AWS CLI commands, bucket names, IAM actions, and file names appear in **monospace**. Python listings use the shared style defined in the preamble; shell commands and JSON policies use listing captions to state their purpose. Callout boxes flag **objectives**, **key terms**, **definitions**, **notes**, **tips**, and **pitfalls**. Citations refer to the bibliography at the back of the book. Every architecture discussion treats cost, reliability, security, and governance as first-class design dimensions.

Contents

0	Getting Started: Your AWS Free Tier Account and Environment	1
0.1	Why Chapter 0 Exists	1
0.2	Creating an AWS Account on the Free Tier	2
0.2.1	What You Need Before Sign-up	2
0.2.2	Understanding Free Tier Categories	2
0.2.3	Sign-up Walkthrough	3
0.3	Securing the Account	3
0.3.1	Root User: Owner, Not Operator	3
0.3.2	Enable MFA on the Root User	4
0.3.3	Create an Administrative Identity	4
0.3.4	Password Policy and Account Hygiene	4
0.4	Installing and Configuring the Tooling	5
0.4.1	AWS CLI v2 on Windows	5
0.4.2	Configuration Option A: IAM Identity Center	5
0.4.3	Configuration Option B: Access Keys	5
0.4.4	Profiles, Regions, and Output Formats	6
0.4.5	AWS CloudShell as a No-install Alternative	6
0.5	Cost Guardrails to Stay Free	7
0.5.1	Turn on Billing Visibility	7
0.5.2	Create a Zero-Spend or Low-Spend Budget	7
0.5.3	Free Tier Usage Alerts and Service-specific Controls	9
0.6	Choosing a Region and Naming Conventions	9
0.6.1	Default Region for the Book	9
0.6.2	Naming Convention	9
0.6.3	Tagging Convention	10
0.7	Setup Flow Across the Six Platform Books	10
0.8	Verification: Hello World	11
0.8.1	Check the Caller Identity	11
0.8.2	Create, List, and Delete a Small S3 Bucket	11
0.8.3	Optional Setup Flow Diagram	12
0.9	Further Reading	12
0.10	Exercises	12
0.11	Curated Community Resources	13
I	Core Data Infrastructure and Storage	16
1	Amazon S3 and Storage Foundations	17
1.1	Week 1 Roadmap and Mental Model	18
1.2	Monday: S3 Architecture and Storage Classes	19

1.2.1	Object Storage, Not a Shared File System	19
1.2.2	Regional Buckets and Global Names	19
1.2.3	Durability and Availability	20
1.2.4	Consistency Model	20
1.3	Choosing a Storage Class for Data Engineering	21
1.4	Tuesday: Lifecycle, Versioning, Object Lock, and Replication	21
1.4.1	Lifecycle Policies	21
1.4.2	Versioning	21
1.4.3	Object Lock: Governance, Compliance, and Legal Hold	21
1.4.4	S3 Replication: SRR and CRR	22
1.5	Wednesday: Migration and Hybrid Storage	22
1.5.1	AWS DataSync	22
1.5.2	AWS Storage Gateway	23
1.5.3	AWS Snow Family	23
1.6	Thursday Hands-on Project: Bucket, Lifecycle, and Events	24
1.6.1	Boto3 Verification and Event Reader	26
1.7	Friday Use Case: Healthcare Disaster-Recovery Storage Architecture	27
1.7.1	Requirements and Assumptions	27
1.7.2	Architecture Decisions	27
1.8	Interview Q&A	28
1.8.1	Concepts and Trade-offs	28
1.9	Further Reading	29
1.10	Exercises	29
1.11	Capstone Project: A Compliant Multi-Region Medical Image Data Lake on S3	30
2	Relational Databases: RDS and Aurora	40
2.1	Week 2 Roadmap and Mental Model	41
2.2	Monday: Amazon RDS Architecture, Multi-AZ, and Read Replicas	42
2.2.1	Managed RDS Architecture	42
2.2.2	Backups, Maintenance, and Observability	42
2.2.3	Multi-AZ Deployments	42
2.2.4	Read Replicas	43
2.3	Tuesday: Amazon Aurora Cloud-Native Relational Architecture	44
2.3.1	Distributed Storage and Compute Separation	44
2.3.2	Aurora Global Database	44
2.3.3	Aurora Serverless v2	45
2.4	Wednesday: AWS DMS and Schema Conversion Tool	45
2.4.1	Migration Phases	45
2.4.2	Replication Instances, Endpoints, and Tasks	45
2.4.3	Validation and Cutover	45
2.5	Thursday Hands-on Project: Aurora Serverless v2 and Regional Failover	46
2.5.1	Boto3 Verification	47
2.6	Friday Use Case: Zero-Downtime Oracle ERP Migration to Aurora PostgreSQL	48
2.6.1	Architecture Narrative	48
2.6.2	Operational Decisions	49
2.7	Interview Q&A	49
2.7.1	Concepts and Trade-offs	49
2.8	Further Reading	50
2.9	Exercises	50
2.10	Capstone Project: Migrating and Operating a Cross-Region Aurora PostgreSQL Fleet for Oracle ERP Cutover	51

3	High-Scale NoSQL: Amazon DynamoDB	58
3.1	Week 3 Roadmap and Mental Model	59
3.2	Monday: DynamoDB Core Concepts, Partitions, and Capacity	60
3.2.1	Tables, Items, and Physical Partitions	60
3.2.2	Read and Write Capacity	60
3.2.3	Provisioned Versus On-Demand Capacity	61
3.3	Tuesday: Data Modeling with Primary Keys, Sort Keys, LSI, and GSI	61
3.3.1	Access-Pattern-First Modeling	61
3.3.2	Primary Keys and Sort-Key Encoding	61
3.3.3	Local and Global Secondary Indexes	62
3.4	Wednesday: Streams, TTL, and Global Tables	62
3.4.1	DynamoDB Streams	62
3.4.2	Time to Live	62
3.4.3	Global Tables	63
3.5	Thursday Hands-on Project: Forum Single-Table Schema and GSI	63
3.6	Friday Use Case: Global Gaming Leaderboard at One Million Reads and Writes Per Second	64
3.6.1	Architecture Narrative	65
3.6.2	Design Decisions	65
3.7	Interview Q&A	66
3.7.1	Concepts and Trade-offs	66
3.8	Further Reading	66
3.9	Exercises	67
3.10	Capstone Project: Building a Global Gaming-Leaderboard Data Layer	67
4	Hybrid Ingestion and Caching	73
4.1	Week 4 Roadmap and Mental Model	74
4.2	Monday: Amazon ElastiCache for Database Acceleration	75
4.2.1	Redis, Valkey, and Memcached in the Data Path	75
4.2.2	Cache-Aside, Write-Through, TTL, and Invalidation	75
4.2.3	Operational Design	76
4.3	Tuesday: DynamoDB Accelerator	76
4.3.1	Where DAX Fits	76
4.3.2	DAX Versus ElastiCache	76
4.3.3	DAX Operations	77
4.4	Wednesday: AppSync and API Gateway Data Ingestion	77
4.4.1	Managed API Front Doors	77
4.4.2	Choosing AppSync or API Gateway	78
4.4.3	Hybrid Ingestion Patterns	78
4.5	Thursday Hands-on Project: Redis Cache-Aside for RDS Queries	78
4.6	Friday Use Case: Mobile Sports API with Extreme Traffic Spikes	80
4.6.1	Architecture Narrative	80
4.6.2	Design Decisions	81
4.7	Interview Q&A	81
4.7.1	Concepts and Trade-offs	81
4.8	Further Reading	82
4.9	Exercises	82
4.10	Milestone 1 Capstone: E-Commerce Multi-Tier Data Backend	83

II	Data Warehousing and Analytics	89
5	Amazon Redshift Architecture and Dimensional Modeling	90
5.1	Week 5 Roadmap and Mental Model	91
5.2	Monday: Redshift MPP Architecture	92
5.2.1	Shared-Nothing Parallelism	92
5.2.2	Columnar Storage and Zone Maps	92
5.2.3	RA3 Nodes and Managed Storage	93
5.3	Tuesday: Redshift Serverless Versus Provisioned	93
5.3.1	Two Ways to Buy Capacity	93
5.3.2	Reading the Utilization Curve	93
5.3.3	The Broader Analytical Pricing Landscape	94
5.4	Wednesday: Loading Data and Kimball Dimensional Modeling	94
5.4.1	COPY: The Parallel Bulk Loader	94
5.4.2	Auto-Copy for Continuous Micro-Batches	95
5.4.3	Declare the Grain First	95
5.4.4	Three Fact Table Types	95
5.4.5	Conformed Dimensions and the Bus Matrix	95
5.4.6	Slowly Changing Dimensions	96
5.5	Thursday Hands-on Project: Load a Star Schema into Redshift Serverless	97
5.6	Friday Use Case: Costing a Month-End BI Workload	100
5.6.1	Requirements and Assumptions	100
5.6.2	The Arithmetic	101
5.6.3	Architecture Decisions	101
5.7	Interview Q&A	102
5.7.1	Concepts and Trade-offs	102
5.8	Further Reading	102
5.9	Exercises	103
5.10	Capstone Project: A Dimensional Retail Warehouse on Redshift Serverless	103
6	Redshift Optimization and Performance Modeling	107
6.1	Week 6 Roadmap and Mental Model	108
6.2	Monday: Distribution Styles	108
6.2.1	The Four Styles	108
6.2.2	Choosing the Distribution Key	109
6.3	Tuesday: Sort Keys and Compression Encodings	109
6.3.1	Compound Versus Interleaved	109
6.3.2	Compression Encodings	110
6.4	Wednesday: WLM, Concurrency Scaling, and Materialized Views	110
6.4.1	Workload Management	110
6.4.2	Concurrency Scaling	110
6.4.3	Materialized Views	110
6.5	Thursday Hands-on Project: Benchmarking Distribution Styles	111
6.6	Friday Use Case: Rescuing a Slow Multi-Petabyte Aggregation Table	112
6.6.1	Diagnosis First	112
6.6.2	The Re-architecture	113
6.7	Interview Q&A	113
6.7.1	Concepts and Trade-offs	113
6.8	Further Reading	114
6.9	Exercises	114
6.10	Capstone Project: A Measured Performance Benchmark Harness for Redshift	115

7	Data Lakehouse and Amazon Athena	117
7.1	Week 7 Roadmap and Mental Model	118
7.2	Monday: Amazon Athena—Serverless SQL on S3	118
7.2.1	How Athena Executes	118
7.2.2	Workgroups and Cost Guardrails	119
7.3	Tuesday: The Glue Data Catalog and Crawlers	119
7.3.1	A Shared Metastore	119
7.3.2	Crawlers: Inference and Its Limits	119
7.4	Wednesday: Redshift Spectrum and the Open Table Layer	119
7.4.1	Querying the Lake from the Warehouse	119
7.4.2	From Files to Tables: Parquet and Iceberg	120
7.5	Thursday Hands-on Project: Crawl, Query, and Federate the Same Data	120
7.6	Friday Use Case: A 50 PB Media Log Lakehouse	122
7.6.1	Architecture	122
7.6.2	Design Decisions	122
7.7	Interview Q&A	123
7.7.1	Concepts and Trade-offs	123
7.8	Further Reading	123
7.9	Exercises	124
7.10	Capstone Project: A Cost-Measured Lakehouse Tier for Web Analytics	124
8	Data Sharing and Governance with Lake Formation	127
8.1	Week 8 Roadmap and Mental Model	128
8.2	Monday: Lake Formation Centralized Governance	128
8.2.1	The Permission Stack	128
8.2.2	Registering Locations and Revoking the Default	128
8.3	Tuesday: Row-Level and Column-Level Security	129
8.3.1	Column-Level Grants	129
8.3.2	Row-Level Data Filters	129
8.3.3	LF-Tags: Policy by Attribute	129
8.4	Wednesday: Redshift Data Sharing Across Accounts	130
8.4.1	Live Shares, No Copies	130
8.5	Thursday Hands-on Project: Hiding the PII Column from Marketing	130
8.6	Friday Use Case: A Data Clean Room for Ad-Tech	132
8.6.1	Why Not CSV Exchange	132
8.6.2	Architecture with AWS Clean Rooms	132
8.6.3	Design Decisions	133
8.7	Interview Q&A	133
8.7.1	Concepts and Trade-offs	133
8.8	Further Reading	133
8.9	Exercises	134
8.10	Milestone 2 Capstone: Enterprise Data Lakehouse Build	135
III	Batch Processing and ETL Orchestration	138
9	Amazon EMR: Managed Hadoop and Spark	139
9.1	Week 9 Roadmap and Mental Model	140
9.2	Monday: Cluster Architecture and Spot Capacity	140
9.2.1	Master, Core, and Task Nodes	140
9.2.2	Spot Instances for the Task Tier	140
9.3	Tuesday: EMR on EC2 vs. EMR Serverless vs. EMR on EKS	141

9.3.1	Three Deployment Modes	141
9.4	Wednesday: EMRFS—S3 as the Storage Layer	141
9.4.1	Decoupling Compute from Storage	141
9.4.2	Transient Clusters and Bootstrap Actions	142
9.5	Thursday Hands-on Project: A PySpark Aggregation on EMR	142
9.6	Friday Use Case: Migrating a 100-Node Cloudera Cluster	144
9.6.1	Migration Strategy	144
9.6.2	Cost Framing	144
9.7	Interview Q&A	145
9.7.1	Concepts and Trade-offs	145
9.8	Further Reading	145
9.9	Exercises	146
9.10	Capstone Project: A Transient-Cluster Batch Platform	146
10	Serverless ETL with AWS Glue	148
10.1	Week 10 Roadmap and Mental Model	148
10.2	Monday: Glue ETL—DynamicFrames Versus Spark DataFrames	149
10.2.1	The DynamicFrame Abstraction	149
10.3	Tuesday: Glue Studio and DataBrew	150
10.3.1	Visual ETL for Working Teams	150
10.3.2	DataBrew: Profiling Before Pipelines	150
10.4	Wednesday: Bookmarks, DPU's, and Flex	151
10.4.1	Job Bookmarks and Idempotency	151
10.4.2	Capacity and Cost: DPU's, Worker Types, Flex	151
10.5	Thursday Hands-on Project: A Glue Studio Pipeline into Redshift	151
10.6	Friday Use Case: Glue or EMR for the Analyst Team?	152
10.6.1	The Evaluation	152
10.7	Interview Q&A	153
10.7.1	Concepts and Trade-offs	153
10.8	Further Reading	154
10.9	Exercises	154
10.10	Capstone Project: An Idempotent Nightly ETL Fleet on Glue	155
11	Change Data Capture, Event-Driven Pipelines, and Data Contracts	157
11.1	Week 11 Roadmap and Mental Model	158
11.2	Monday: AWS DMS Architecture for Continuous Replication	158
11.2.1	Full Load Plus CDC	158
11.2.2	CDC to the Lake	158
11.3	Tuesday: Event-Driven Pipelines with EventBridge	159
11.3.1	Routing Events Instead of Polling	159
11.3.2	Events as Pipeline Contracts	159
11.4	Wednesday: Lambda Transformations and Data Contracts	159
11.4.1	Lambda in the Data Path	159
11.4.2	Data Contracts with the Glue Schema Registry	159
11.5	Thursday Hands-on Project: CDC from RDS PostgreSQL to the Lake	160
11.6	Friday Use Case: Near-Real-Time Oracle-to-Redshift Synchronization	161
11.6.1	Architecture	162
11.6.2	Design Decisions	162
11.7	Interview Q&A	162
11.7.1	Concepts and Trade-offs	162
11.8	Further Reading	163
11.9	Exercises	163

11.10	Capstone Project: A Contract-Protected CDC Feed	164
12	Pipeline Orchestration with Amazon MWAA and Step Functions	166
12.1	Week 12 Roadmap and Mental Model	167
12.2	Monday: Airflow Concepts	167
12.2.1	DAGs, Operators, Tasks, and Scheduling	167
12.2.2	XComs and the Data Rule	167
12.3	Tuesday: Amazon MWAA Architecture	168
12.3.1	Managed Airflow, Honest Boundaries	168
12.4	Wednesday: AWS Step Functions	168
12.4.1	Native Serverless Orchestration	168
12.4.2	Choosing Between MWAA and Step Functions	169
12.5	Thursday Hands-on Project: A Production-Shaped MWAA DAG	169
12.6	Friday Use Case: Self-Healing Banking Reconciliation Across Fifteen Systems	170
12.6.1	Architecture	171
12.6.2	Design Decisions	171
12.7	Interview Q&A	171
12.7.1	Concepts and Trade-offs	171
12.8	Further Reading	172
12.9	Exercises	173
12.10	Milestone 3 Capstone: Automated Batch ELT Pipeline	173
IV	Streaming and Real-Time Analytics	176
13	Message Queuing and Streaming Ingestion with Amazon Kinesis	177
13.1	Week 13 Roadmap and Mental Model	178
13.2	Monday: Decoupling with SQS and SNS	178
13.2.1	SQS: The Work Queue	178
13.2.2	SNS: The Fan-Out	178
13.3	Tuesday: Kinesis Data Streams—Shards, Keys, and Scaling	179
13.3.1	The Shard Model	179
13.3.2	Sizing and Hot Shards	179
13.3.3	Consumers, Checkpointing, and Iterator Age	179
13.4	Wednesday: Amazon MSK Versus Kinesis	180
13.4.1	Same Log, Different Contract	180
13.5	Thursday Hands-on Project: An IoT Temperature Stream	180
13.6	Friday Use Case: Global Telemetry for Five Million Mobile Games	182
13.6.1	Unpacking the Requirements	182
13.6.2	Architecture	182
13.6.3	Design Decisions	183
13.7	Interview Q&A	183
13.7.1	Concepts and Trade-offs	183
13.8	Further Reading	183
13.9	Exercises	184
13.10	Capstone Project: A Resilient Telemetry Ingestion Tier	184
14	Real-Time Delivery with Amazon Data Firehose	187
14.1	Week 14 Roadmap and Mental Model	188
14.2	Monday: Firehose Architecture and Destinations	188
14.2.1	The Delivery Pipeline	188
14.2.2	Buffering Is a Latency–Efficiency Trade	189

14.3	Tuesday: Inline Transformation with Lambda	189
14.3.1	Per-Batch Record Surgery	189
14.3.2	Latency and Cost Accounting	189
14.4	Wednesday: In-Flight Format Conversion	189
14.4.1	JSON In, Parquet Out	189
14.4.2	Source-Record Backup	190
14.5	Thursday Hands-on Project: Stream to Parquet via Firehose	190
14.6	Friday Use Case: Real-Time HL7 Anonymization into Redshift	192
14.6.1	Architecture	192
14.6.2	Design Decisions	192
14.7	Interview Q&A	193
14.7.1	Concepts and Trade-offs	193
14.8	Further Reading	193
14.9	Exercises	194
14.10	Capstone Project: A Governed Streaming Delivery Layer	194
15	Stream Processing with Apache Flink on AWS	197
15.1	Week 15 Roadmap and Mental Model	198
15.2	Monday: Managed Service for Apache Flink	198
15.2.1	What the Service Manages	198
15.3	Tuesday: Event Time, Processing Time, and Watermarks	199
15.3.1	Two Clocks	199
15.3.2	Watermarks: The Lateness Contract	199
15.4	Wednesday: Windows, Joins, and State Management	199
15.4.1	Windowing Strategies	199
15.4.2	Stream-Stream Joins	199
15.4.3	State Backends, TTL, Checkpoints, and Savepoints	200
15.4.4	End-to-End Exactly-Once	200
15.5	Thursday Hands-on Project: A Flink SQL Tumbling-Window Counter	200
15.6	Friday Use Case: The Dashboard That Lied About Late Mobile Data	201
15.6.1	The Failure Chain	202
15.6.2	The Fix	202
15.6.3	Exactly-Once, End to End	202
15.7	Interview Q&A	202
15.7.1	Concepts and Trade-offs	202
15.8	Further Reading	203
15.9	Exercises	203
15.10	Capstone Project: A Late-Data-Tolerant Sessionization Pipeline	204
16	Business Intelligence with QuickSight and Reverse ETL	206
16.1	Week 16 Roadmap and Mental Model	207
16.2	Monday: QuickSight Architecture and SPICE	207
16.2.1	The Serving Layer, Managed	207
16.2.2	SPICE: The Acceleration Decision	207
16.3	Tuesday: Connecting QuickSight to Redshift, Athena, and S3	208
16.4	Wednesday: Row-Level Security, ML Insights, and Reverse ETL	208
16.4.1	Row-Level Security on Shared Datasets	208
16.4.2	ML Insights	208
16.4.3	Reverse ETL: Closing the Loop	209
16.5	Thursday Hands-on Project: A Monitored Sales Dashboard	209
16.6	Friday Use Case: Governed BI for 500 Analysts	210
16.6.1	Design Decisions	210

16.7 Interview Q&A	211
16.7.1 Concepts and Trade-offs	211
16.8 Further Reading	212
16.9 Exercises	212
16.10 Milestone 4 Capstone: Real-Time Fraud Detection	213
V Machine Learning and MLOps	216
17 In-Database Machine Learning with Redshift ML	217
17.1 Week 17 Roadmap and Mental Model	218
17.2 Monday: Redshift ML Architecture	218
17.2.1 SQL in Front, SageMaker Behind	218
17.3 Tuesday: Creating Models—Regression, Classification, K-Means	219
17.3.1 The Three Problem Types	219
17.4 Wednesday: Evaluation and Scoring in SQL	219
17.4.1 Evaluation Where the Data Lives	219
17.4.2 Scoring Live Data	219
17.5 Thursday Hands-on Project: Churn Prediction in Pure SQL	220
17.6 Friday Use Case: SQL-Only Inventory Forecasting for 1,000 Stores	221
17.6.1 Architecture Decisions	221
17.7 Interview Q&A	221
17.7.1 Concepts and Trade-offs	221
17.8 Further Reading	222
17.9 Exercises	222
17.10 Capstone Project: A Warehouse-Native Demand Forecasting Pipeline	223
18 Amazon SageMaker Foundations for Data Engineers	225
18.1 Week 18 Roadmap and Mental Model	226
18.2 Monday: Studio, Notebooks, and Data Wrangler	226
18.2.1 The Working Surfaces	226
18.2.2 Data Wrangler: Visual Prep That Exports	226
18.3 Tuesday: Built-In Algorithms, Autopilot, and Custom Containers	227
18.3.1 Three Ladders of Control	227
18.4 Wednesday: Inference Shapes—Real-Time, Serverless, Batch	227
18.4.1 Four Ways to Serve	227
18.5 Thursday Hands-on Project: Wrangler to Serverless Endpoint	228
18.6 Friday Use Case: GPU Training on a Governed Lake	229
18.6.1 Architecture	229
18.6.2 Design Decisions	229
18.7 Interview Q&A	230
18.7.1 Concepts and Trade-offs	230
18.8 Further Reading	231
18.9 Exercises	231
18.10 Capstone Project: A Governed ML Training Service	232
19 MLOps: Feature Stores, Pipelines, and Model Monitoring	234
19.1 Week 19 Roadmap and Mental Model	235
19.2 Monday: SageMaker Feature Store—Online and Offline	235
19.2.1 One Definition, Two Stores	235
19.2.2 Training/Serving Skew and Point-in-Time Correctness	235
19.3 Tuesday: SageMaker Pipelines	236

19.3.1	The Pipeline as the Product	236
19.3.2	Model Registry: Versions and Approvals	236
19.4	Wednesday: Model Monitor—Drift as an Event	236
19.4.1	Baselines and Schedules	236
19.5	Thursday Hands-on Project: A SageMaker Pipeline, End to End	237
19.6	Friday Use Case: Drift-Triggered Retraining for a Recommendation Engine	238
19.6.1	Architecture	238
19.6.2	Design Decisions	239
19.7	Interview Q&A	239
19.7.1	Concepts and Trade-offs	239
19.8	Further Reading	240
19.9	Exercises	240
19.10	Capstone Project: A Drift-Guarded Recommendation Lifecycle	241
20	Unstructured Data and Managed AI Services	243
20.1	Week 20 Roadmap and Mental Model	244
20.2	Monday: Rekognition and Textract	244
20.2.1	Vision as an API	244
20.2.2	Textract: Structure from Documents	244
20.3	Tuesday: Comprehend and Translate	245
20.3.1	NLP as Enrichment	245
20.4	Wednesday: Integrating AI APIs into Data Pipelines	245
20.4.1	The Integration Pattern	245
20.4.2	PII in OCR Output	245
20.5	Thursday Hands-on Project: S3-Triggered Textract Extraction	246
20.6	Friday Use Case: Terabyte-Scale Image Understanding with Search	247
20.6.1	Architecture	248
20.6.2	Design Decisions	248
20.7	Interview Q&A	248
20.7.1	Concepts and Trade-offs	248
20.8	Further Reading	249
20.9	Exercises	249
20.10	Milestone 5 Capstone: End-to-End MLOps Pipeline	250
VI	Security, Governance, and Exam Prep	252
21	Security, IAM, and Networking for Data Platforms	253
21.1	Week 21 Roadmap and Mental Model	254
21.2	Monday: IAM Deep Dive—Roles, Boundaries, and SCPs	254
21.2.1	Least Privilege as an Engineering Practice	254
21.2.2	Ceilings: Permissions Boundaries and SCPs	254
21.3	Tuesday: VPC Endpoints and PrivateLink	255
21.3.1	Gateway Versus Interface	255
21.3.2	Endpoint Policies: The Network Half of the Perimeter	255
21.4	Wednesday: KMS and Secrets Manager	255
21.4.1	KMS: Keys as Policy Objects	255
21.4.2	Secrets Manager: No More Static Credentials	255
21.5	Thursday Hands-on Project: Private S3 Access with No Internet Route	256
21.6	Friday Use Case: A Healthcare Data Perimeter That Survives Credential Theft	257
21.6.1	The Layered Design	257
21.6.2	Verification and Operations	258

21.7 Interview Q&A	258
21.7.1 Concepts and Trade-offs	258
21.8 Further Reading	259
21.9 Exercises	259
21.10Capstone Project: A Tested Data Perimeter for the Analytics Estate	260
22 Data Governance and Data Quality at Scale	262
22.1 Week 22 Roadmap and Mental Model	263
22.2 Monday: Cross-Account Sharing and Tag-Based Access	263
22.2.1 Sharing the Catalog, Not the Data	263
22.2.2 Why Tags Win at Scale	263
22.3 Tuesday: AWS Glue Data Quality	264
22.3.1 RULES as Code	264
22.3.2 Rule Selection: Drift Versus Degradation	264
22.4 Wednesday: Lineage—Tracing Failures Upstream	264
22.4.1 Why Lineage Is an Operational Tool	264
22.5 Thursday Hands-on Project: A Quality Gate That Halts the Pipeline	265
22.6 Friday Use Case: A Five-Domain Data Mesh	266
22.6.1 Operating Model	266
22.6.2 Architecture	267
22.6.3 Failure Modes the Design Anticipates	267
22.7 Interview Q&A	267
22.7.1 Concepts and Trade-offs	267
22.8 Further Reading	268
22.9 Exercises	268
22.10Capstone Project: A Governed Two-Domain Mesh Slice	269
23 Compliance and Data Privacy with Amazon Macie	271
23.1 Week 23 Roadmap and Mental Model	272
23.2 Monday: Amazon Macie—Automated PII Discovery	272
23.2.1 What Macie Actually Does	272
23.2.2 Custom Data Identifiers and the Noise Budget	272
23.3 Tuesday: De-Identification and the Erasure Problem	273
23.3.1 Masking, Tokenization, Hashing	273
23.3.2 Right to Erasure Against a Durable Architecture	273
23.4 Wednesday: CloudTrail and Config—Auditing Access and Posture	274
23.4.1 CloudTrail: The Access Ledger	274
23.4.2 Config: Posture as Code	274
23.5 Thursday Hands-on Project: Macie Discovery and Automated Quarantine	274
23.6 Friday Use Case: Tokenizing Third-Party Customer Data Onboarding	276
23.6.1 Architecture	276
23.6.2 Design Decisions	276
23.7 Interview Q&A	277
23.7.1 Concepts and Trade-offs	277
23.8 Further Reading	277
23.9 Exercises	278
23.10Capstone Project: An Auditable PII Onboarding and Erasure Service	278
24 Production DataOps and Certification Preparation	280
24.1 Week 24 Roadmap and Mental Model	281
24.2 Monday: Terraform for Data Infrastructure	281
24.2.1 Everything You Clicked, as Code	281

24.3	Tuesday: dbt on Redshift	282
24.3.1	Transformations as a Tested DAG	282
24.4	Wednesday: CI/CD and Zero-Downtime Schema Evolution	283
24.4.1	The Pipeline for Pipelines	283
24.4.2	Expand → Migrate → Contract	283
24.5	Thursday Hands-on Project: The Repository Is the Platform	283
24.6	Friday: Certification Preparation—Data Engineer Associate	285
24.6.1	The Review Protocol	285
24.7	Interview Q&A	285
24.7.1	Concepts and Trade-offs	285
24.8	Further Reading	286
24.9	Exercises	286
24.10	Milestone 6 Capstone: A Secure, Governed, Multi-Account Data Mesh	287
A	Cross-Cloud Service Equivalence	290
A.1	Object and Data-Lake Storage	290
A.2	Managed Relational Databases	291
A.3	Elastic Compute and Analytical Warehouses	291
A.4	Data Organization and the Medallion Pattern	292
A.5	Document Data Modeling	292
A.6	Globally Distributed and NoSQL Databases	292
A.7	Managed Apache Spark	293
A.8	Query Acceleration and Materialized Views	293
A.9	In-Memory Caching and Hybrid Ingestion	294
A.10	Wide-Column and Key-Value NoSQL	294
A.11	Data Protection: Cloning, Time Travel, and Migration	294
A.12	Account and Environment Setup	295
A.13	Cloud Data Warehouses and MPP Architecture	295
A.14	Warehouse Performance and Physical Design	296
A.15	Lakehouse Query Layer and Open Table Formats	296
A.16	Lake Governance, Sharing, and Clean Rooms	297
A.17	Managed Hadoop and Spark Clusters	297
A.18	Serverless, Catalog-Integrated ETL	298
A.19	Change Data Capture, Event Routing, and Schema Contracts	298
A.20	Orchestration: DAG Schedulers and State Machines	299
A.21	Messaging and Streaming Primitives	299
A.22	Managed Stream-to-Sink Delivery	300
A.23	Stateful Stream Processing	300
A.24	Real-Time Serving, BI, and Reverse ETL	301
A.25	In-Warehouse Machine Learning	301
A.26	Managed ML Platforms	302
A.27	MLOps: Feature Stores, Registries, and Monitoring	302
A.28	Pre-trained AI Services and Unstructured Data	303
A.29	Identity, Networking, and Key Management	303
A.30	Data Quality, Lineage, and Mesh Governance	304
A.31	Sensitive-Data Discovery and Erasure	304
A.32	DataOps: IaC, Transformation-as-Code, and CI/CD	305
B	Environment Setup Reference	306
B.1	Account and Identity	306
B.2	AWS CLI v2 and Profiles	307
B.3	SDKs: Boto3 and Friends	307

B.4	Infrastructure as Code: CDK and Terraform	307
B.5	Cost Guardrails	308
B.6	Per-Chapter Tool Requirements	308
C	AWS CLI, SDK, and SQL Quick Reference	310
C.1	Amazon S3 (Chapters 1, 7, 21)	310
C.2	Amazon Athena (Chapter 7)	311
C.3	Amazon Redshift and Redshift Serverless (Chapters 5, 6, 17)	311
C.4	AWS Glue (Chapters 7, 9, 10, 22)	312
C.5	Amazon EMR (Chapter 9)	312
C.6	Amazon Kinesis Data Streams (Chapter 13)	312
C.7	Amazon MSK (Chapter 14)	313
C.8	Amazon Data Firehose (Chapter 14)	313
C.9	AWS Lambda (Chapters 14, 15, 20)	313
C.10	Managed Flink (Chapter 15)	313
C.11	Step Functions and Amazon MWAA (Chapter 12)	314
C.12	Amazon SageMaker (Chapters 17–20)	314
C.13	Amazon Textract, Comprehend, and Macie (Chapters 20, 23)	314
C.14	Lake Formation and Governance (Chapters 8, 21, 22)	315
C.15	Cross-Cutting: STS, IAM, Budgets (Chapter 0)	315
D	Lab Solutions and Acceptance Guidance	316
D.1	Chapter 0: Environment Verification	316
D.1.1	Project: Account, Profile, Guardrails, and the S3 Smoke Test	316
D.2	Part I: Core Data Infrastructure and Storage (Chapters 1–4)	316
D.2.1	Chapter 1: Bucket, Lifecycle, and Events	316
D.2.2	Chapter 2: Aurora Serverless v2 and Regional Failover	317
D.2.3	Chapter 3: Forum Single-Table Schema and GSI	317
D.2.4	Chapter 4: Redis Cache-Aside for RDS Queries	317
D.3	Part II: Data Warehousing and Analytics (Chapters 5–8)	318
D.3.1	Chapter 5: Load a Star Schema into Redshift Serverless	318
D.3.2	Chapter 6: Benchmarking Distribution Styles	318
D.3.3	Chapter 7: Crawl, Query, and Federate the Same Data	318
D.3.4	Chapter 8: Hiding the PII Column from Marketing	318
D.4	Part III: Batch Processing and ETL Orchestration (Chapters 9–12)	319
D.4.1	Chapter 9: A PySpark Aggregation on EMR	319
D.4.2	Chapter 10: A Glue Studio Pipeline into Redshift	319
D.4.3	Chapter 11: CDC from RDS PostgreSQL to the Lake	319
D.4.4	Chapter 12: A Production-Shaped MWAA DAG	319
D.5	Part IV: Streaming and Real-Time Analytics (Chapters 13–16)	320
D.5.1	Chapter 13: An IoT Temperature Stream	320
D.5.2	Chapter 14: Stream to Parquet via Firehose	320
D.5.3	Chapter 15: A Flink SQL Tumbling-Window Counter	320
D.5.4	Chapter 16: A Monitored Sales Dashboard	320
D.6	Part V: Machine Learning and MLOps (Chapters 17–20)	321
D.6.1	Chapter 17: Churn Prediction in Pure SQL	321
D.6.2	Chapter 18: Wrangler to Serverless Endpoint	321
D.6.3	Chapter 19: A SageMaker Pipeline, End to End	321
D.6.4	Chapter 20: S3-Triggered Textract Extraction	321
D.7	Part VI: Security, Governance, and Exam Prep (Chapters 21–24)	321
D.7.1	Chapter 21: Private S3 Access with No Internet Route	321
D.7.2	Chapter 22: A Quality Gate That Halts the Pipeline	322

D.7.3	Chapter 23: Macie Discovery and Automated Quarantine	322
D.7.4	Chapter 24: The Repository Is the Platform	322
D.8	Selected End-of-Chapter Exercises	322
D.8.1	Storage and Warehousing	322
D.8.2	ETL and Streaming	323
D.8.3	ML, Security, and DataOps	323
E	Production-Readiness and Governance Checklists	324
E.1	Storage and Data Lake (S3, Chapter 1)	324
E.2	Warehousing and Databases (RDS/Aurora, DynamoDB, Redshift; Chapters 2, 3, 5, 6)	324
E.3	ETL and Orchestration (Glue, EMR, MWAA, Step Functions; Chapters 9–12)	325
E.4	Streaming (Kinesis, Firehose, MSK, Flink, Lambda; Chapters 13–15)	325
E.5	Machine Learning and MLOps (Chapters 16–20)	326
E.6	Security, Governance, and Compliance (Chapters 8, 21–23)	326
E.7	Cost and Operations (Chapters 0, 23, 24)	327
F	Exam Mapping: AWS Certified Data Engineer – Associate	328
F.1	Domain-to-Chapter Map	328
F.2	Objective-Level Breakdown	328
F.2.1	Domain 1: Data Ingestion and Transformation	328
F.2.2	Domain 2: Data Store Management	329
F.2.3	Domain 3: Data Operations and Support	329
F.2.4	Domain 4: Data Security and Governance	329
F.3	Study Strategy	329
F.4	Self-Assessment Checklist	330
G	Interview Question Bank	331
G.1	Storage and the Data Lake	331
G.2	Databases and Caching	331
G.3	Warehousing and Lakehouse Analytics	333
G.4	ETL, CDC, and Orchestration	334
G.5	Streaming	336
G.6	Machine Learning and MLOps	337
G.7	Security, Governance, and Compliance	338
G.8	DataOps and Delivery	339
	Glossary	341

List of Figures

1	Chapter 0 setup flow: account, secure identity, tooling, cost guardrail, and verification.	12
1.1	A storage-class lifecycle path for analytical and archival objects. Actual thresholds must be justified by access data, compliance needs, and retrieval objectives.	21
1.2	A multi-region compliant disaster-recovery storage architecture for 10 PB of healthcare imaging data. Replication protects regional availability; Object Lock, KMS, IAM, Lake Formation, and CloudTrail support compliance controls.	27
1.3	Capstone reference architecture for a compliant multi-region medical-image data lake on S3. The primary bucket receives encrypted, versioned, immutable objects; lifecycle rules manage cost; CRR creates a disaster-recovery copy; S3 event notifications feed SQS for downstream processing.	32
2.1	RDS Multi-AZ protects availability by synchronously maintaining a standby in another Availability Zone and moving the endpoint during failover. It is not a read-scaling feature in the traditional DB instance model.	43
2.2	Aurora separates compute instances from a distributed cluster volume. Writer and reader endpoints route application traffic while the storage service handles replication and durability.	44
2.3	Zero-downtime Oracle ERP migration pattern using SCT for schema conversion, DMS for full load and CDC, validation for business reconciliation, and Aurora PostgreSQL with cross-region recovery for the target.	49
2.4	Capstone reference architecture for an Oracle ERP migration to Aurora PostgreSQL with DMS CDC, SCT conversion, high availability, cross-region recovery, observability, and analytics offload.	53
3.1	DynamoDB hashes the partition key to route items and requests to physical partitions. Sort keys organize related items after the partition key has chosen the item collection.	60
3.2	Global gaming leaderboard architecture with regional writes to DynamoDB Global Tables, streams for derived top-N views and lake ingestion, and cache or DAX layers for hot read serving.	65
3.3	Capstone reference architecture for a global gaming-leaderboard data layer with regional APIs, DynamoDB Global Tables, Streams processors, top-N materialized items, cache acceleration, S3 export, and observability.	69
4.1	Cache-aside flow between an application, ElastiCache Redis, and an RDS or Aurora source of truth.	75
4.2	Spiky-traffic mobile sports API using API Gateway, AppSync, ElastiCache, DAX, DynamoDB, Aurora or RDS, and streaming ingestion.	81

4.3	Milestone 1 e-commerce reference architecture integrating Aurora Serverless v2, DynamoDB, S3, ElastiCache, DAX, AWS DMS, and observability across all tiers.	85
5.1	Redshift MPP architecture: the leader node plans and coordinates; compute nodes own data slices and execute query fragments in parallel.	92
5.2	Star schema for the Thursday project: fact_sales at the order-line grain with four conformed dimensions.	96
5.3	Capstone reference architecture: S3 landing, COPY into a staging schema, star-schema marts in Redshift Serverless, with RPU guardrails.	104
7.1	Media lakehouse: logs land raw, are compacted to partitioned Parquet/Iceberg, and are served through one catalog to Athena (ad-hoc) and Spectrum (joins with warehouse data).	122
8.1	Clean-room collaboration: each party contributes a configured table behind analysis rules; only rule-compliant aggregate queries can run, and only aggregates leave.	132
8.2	Milestone 2 reference architecture: governed enterprise lakehouse.	136
11.1	Near-real-time ledger sync: DMS tails Oracle redo into an S3 change log; a Glue MERGE maintains current state in Redshift; lag and contracts are monitored.	162
12.1	Two-tier orchestration: Step Functions handles per-arrival, per-system ingestion; MWAA owns the cross-system daily reconciliation with dependencies, retries, and alerting.	171
12.2	Milestone 3 reference architecture: automated batch ELT pipeline with a data-quality gate.	174
13.1	Global telemetry: edge validation, per-player partition keys, on-demand shards, independent consumers, idempotent writes.	182
14.1	HL7 anonymization pipeline: tokenization happens in the delivery path; only de-identified records reach Redshift; the restricted raw backup preserves originals under separate controls.	192
16.1	Milestone 4 reference architecture: real-time fraud detection from swipe simulation to live dashboard.	214
18.1	Governed GPU training: a catalog-scoped extraction job produces the training set; scientists' roles never touch the governed lake directly.	229
19.1	The MLOps loop: monitored endpoint traffic feeds drift detection; violations route through EventBridge to a retraining pipeline with a human approval gate before redeploy.	238
20.1	Image understanding pipeline: parallel enrichment by Rekognition and Textract, cataloged JSON metadata, OpenSearch indexing, and a confidence-gated moderation path.	248
21.1	The four perimeters: identity, network, organization, and keys. A stolen credential must defeat all four to exfiltrate data.	257
22.1	Data mesh on AWS: domains publish governed products from their own accounts; the center owns taxonomy and audit; consumers read through tag-expression grants.	267

23.1 Third-party PII onboarding: discovery, KMS-anchored tokenization, a sanitized zone, warehouse load, and verification by re-scan.	276
24.1 Milestone 6 reference architecture: the multi-account governed data mesh.	288

List of Tables

1	Minimum training tags used throughout this book.	10
1.1	Storage-class choice as a workload and risk decision, not merely a price decision.	20
1.2	S3 management features mapped to operational review questions.	22
1.3	Migration and hybrid-storage options for AWS data engineering foundations.	23
2.1	RDS deployment patterns mapped to common data engineering decisions.	43
3.1	Example single-table access patterns for a forum workload.	61
4.1	DAX and ElastiCache solve different cache problems.	77
5.1	Serverless versus provisioned Redshift as an economic and operational decision.	93
5.2	Analytical engines on AWS compared by billing model and fit. Chapter 7 covers Athena and Spectrum in depth.	94
5.3	Illustrative month-end BI cost comparison. Substitute live pricing and measured RPU consumption before committing.	101
6.1	Distribution-style selection guide.	109
6.2	From diagnosis to design: each fix maps to a measured symptom.	113
9.1	EMR deployment modes compared.	141
9.2	Cloudera-to-EMR migration mapping.	144
10.1	Glue versus EMR for the analyst reporting estate.	153
12.1	Choosing the orchestrator: MWAA versus Step Functions.	169
13.1	Kinesis versus MSK: an operating-model decision.	180
16.1	SPICE versus direct query.	208
16.2	The 500-analyst BI access layer.	211
18.1	Choosing an inference shape.	228
24.1	Exam domains mapped to the book. Weights are indicative of the published guide.	285
F.1	Exam domains mapped to the book (same mapping as Chapter 24, expanded below).	328

Getting Started: Your AWS Free Tier Account and Environment

Learning Objectives

- Create an AWS account, distinguish 12-month Free Tier offers from Always-Free offers and short trials, and identify the inputs required during sign-up.
- Secure the account by protecting the root user with MFA, avoiding root for daily work, and creating a separate administrative identity.
- Install and validate AWS CLI v2 on Windows PowerShell, and compare long-lived access-key configuration with IAM Identity Center sign-in.
- Configure profiles, default Regions, and AWS CloudShell so examples in later chapters can run reproducibly.
- Establish cost guardrails with billing alerts, Free Tier usage alerts, AWS Budgets, Cost Explorer, and routine Billing dashboard checks.
- Apply the Region, naming, and tagging conventions used throughout this book.
- Verify the environment with AWS STS and a short S3 create, list, and cleanup workflow.

Key Terms

Root user: the account owner identity created with the account email address. **IAM Identity Center:** AWS's workforce sign-in service, formerly AWS Single Sign-On. **Profile:** a named set of AWS CLI credentials and configuration. **Region:** a geographic AWS service location such as `us-east-1`. **Budget:** a billing guardrail that sends alerts when actual or forecasted spend crosses thresholds.

0.1 Why Chapter 0 Exists

Before creating buckets, Glue jobs, streams, warehouses, or lakehouse tables, you need a safe sandbox. This chapter builds that sandbox deliberately: account, identity, command-line tooling, budget alarms, naming rules, and a tiny verification exercise. The goal is not to become an AWS administrator in one sitting. The goal is to remove the avoidable problems that derail hands-on learning: using the root user every day, working in the wrong Region, losing track of profiles, creating resources without tags, or discovering charges after a lab was supposed to be cleaned up.

This book assumes a Windows workstation and PowerShell. Most AWS commands are the

same on macOS, Linux, AWS CloudShell, Git Bash, and Windows Terminal, but file paths, environment variables, quoting, and here-documents differ. Where the shell matters, listings use native PowerShell. Later chapters may show Bash-style commands when that is shorter for cloud shells or Linux-based services; when you run them on Windows, translate variable assignment and file creation to PowerShell.

Note

The examples in this chapter use a personal learning account or an organization-approved sandbox account. Do not run training commands in a production account unless your employer has explicitly approved the account, Region, budget, and cleanup process.

0.2 Creating an AWS Account on the Free Tier

0.2.1 What You Need Before Sign-up

To create a standard AWS account, prepare four items: a unique email address, a strong password, a credit or debit card for verification and payment, and a phone number that can receive verification. The email address becomes the root user sign-in name. Use an email address you can keep long term, because it is also the recovery channel for account ownership.

The payment card is required even if you plan to stay within free offers. AWS uses it for identity and billing verification, and charges can occur if you exceed Free Tier limits or create resources that are not free. The phone verification step proves that you control a reachable phone number. Depending on country, AWS may also ask for a billing address, tax information, or an additional verification step.

Definition

AWS Free Tier is a collection of offers, not a single unlimited free account. Some offers last for 12 months after account creation, some are Always Free within monthly limits, and some are short service-specific trials.

0.2.2 Understanding Free Tier Categories

AWS groups free usage into three practical categories:

- **12-month Free Tier:** introductory monthly allowances that expire 12 months after the account creation date. Common examples include limited compute, storage, database, and data-transfer allowances. When the 12-month period ends, normal pay-as-you-go pricing applies.
- **Always-Free offers:** recurring monthly allowances that do not expire as long as AWS continues the offer and you stay within limits. Examples vary by service and Region, so always check the service's current Free Tier page before a lab.
- **Trials:** service-specific promotions for a fixed time, fixed usage amount, or fixed credit. Trials may start when you first activate the service, not necessarily when you create the account.

Free Tier is measured by usage dimensions: GB-months stored, requests made, compute hours, data processed, messages sent, log ingestion, or data transferred out. A service can have one free dimension but charge for another. For example, a storage allowance may be free while requests, retrieval, replication, public IPv4 addresses, NAT gateways, or cross-Region transfer are billable.

Common Pitfall

Assuming Free Tier means no bill. Free Tier is a discount schedule with limits. Resources outside the offer, usage above the allowance, unsupported Regions, idle billable infrastructure, data egress, and forgotten resources can still create charges.

0.2.3 Sign-up Walkthrough

Start at the AWS sign-up page and choose **Create an AWS account**. Enter the email address that will own the account, an account name, and a strong password. Use an account name that is human-readable but not sensitive, such as `data-engineering-sandbox`. Avoid patient names, internal project codenames, confidential client names, or anything that would be embarrassing in a bill, screenshot, or support case.

Next, choose the account type. Individual learners usually choose a personal account. Company learners should follow their employer's cloud onboarding process instead, because many organizations require AWS Organizations, centralized identity, service control policies, centralized logging, and approved billing. Enter contact information, payment information, and the verification code sent through the phone flow. Choose the Basic support plan unless your organization requires a different plan.

After sign-up, wait for account activation email. Then sign in as the root user only long enough to secure the account and create an administrative identity. Record the account ID from the console menu; later commands and budget policies may need it. Do not create training resources yet. Security and cost guardrails come first.

Tip

Use a password manager from the beginning. Store the root email, password, account ID, MFA recovery codes or device recovery notes, and the name of the administrative profile you create later. Do not store access keys in notes, screenshots, or source files.

0.3 Securing the Account**0.3.1 Root User: Owner, Not Operator**

The root user has unrestricted authority over the account. It can change billing settings, close the account, alter account-level security settings, recover access, and perform actions that ordinary IAM users or roles may not need. That power makes root valuable for rare account ownership operations and dangerous for daily work.

Daily work should use federated identities, IAM Identity Center permission sets, IAM roles, or in a small learning account, an IAM administrator user. This separation gives you better audit trails, lets you rotate or revoke operational credentials without changing account ownership, and reduces blast radius if a laptop, browser session, or access key is compromised.

Common Pitfall

Leaving root unprotected. An AWS account with root sign-in and no MFA is a high-risk target. If an attacker obtains the root password, they can create users, disable controls, delete resources, change billing settings, and lock you out. Enable MFA on root before building anything.

0.3.2 Enable MFA on the Root User

Sign in to the AWS Management Console as the root user. Open the account menu, choose **Security credentials**, and find the MFA section. Add a virtual authenticator app, hardware security key, or another supported MFA device. Scan the QR code or register the key, enter consecutive MFA codes when prompted, and verify that the device appears as active.

A virtual authenticator is acceptable for a personal lab, but a hardware security key is stronger. If you use a phone app, think about device replacement: keep recovery information in a secure place and avoid a single point of failure. Do not share the root MFA device casually. For organizations, root MFA ownership should follow a documented break-glass process.

Note

After MFA is enabled, sign out and sign back in once to confirm that root sign-in requires the second factor. Then stop using root for routine labs.

0.3.3 Create an Administrative Identity

AWS has two common paths for a learning account. The preferred modern path is **IAM Identity Center**. The smaller, legacy-compatible path is an **IAM admin user and group**. Use IAM Identity Center if you can; use an IAM admin user only if your account or course environment does not support the Identity Center flow.

Preferred: IAM Identity Center

Open **IAM Identity Center** in the console. If prompted, enable it for the account. Create a user with your learner email address, assign that user to the AWS account, and attach an administrative permission set for the sandbox account. Accept the invitation email, set a password, register MFA for the workforce user if prompted, and sign in through the AWS access portal URL.

From the access portal, you can open the console or copy CLI configuration values. This flow avoids creating long-lived access keys on your laptop. It also resembles how many companies grant cloud access: users authenticate to a workforce identity system and receive temporary credentials for a role.

Alternative: IAM Admin User and Group

If IAM Identity Center is unavailable, create an IAM group such as **Administrators**, attach the AWS-managed **AdministratorAccess** policy, then create a user for daily work and add the user to that group. Give the user a console password only if you need console sign-in. Create access keys only for CLI work, rotate them periodically, and delete unused keys.

For a personal learning account, an admin user is acceptable as a temporary bootstrap identity. For team, enterprise, or production use, prefer federation and roles. Never create access keys for the root user.

0.3.4 Password Policy and Account Hygiene

In IAM account settings, configure a strong password policy: minimum length of at least 14 characters, uppercase, lowercase, number, symbol, expiration or rotation according to your organization's policy, and prevention of password reuse. Require MFA for privileged users. Add an alternate billing contact and security contact if the account is important enough that another human must receive urgent notices.

Finally, review the account’s security credential report or IAM access analyzer findings periodically. Delete unused users, deactivate unused access keys, and avoid embedding credentials in scripts. In later chapters, the safest pattern is to use short-lived role credentials from IAM Identity Center or CloudShell instead of long-lived local keys.

0.4 Installing and Configuring the Tooling

0.4.1 AWS CLI v2 on Windows

AWS CLI v2 is the command-line interface used throughout the book. On Windows, install the official MSI package from AWS, or install through an approved package manager if your organization manages software that way. Open a new PowerShell window after installation so `aws.exe` is on PATH. Verify the version before configuring credentials.

```
1 aws --version
2
3 # Expected shape; exact patch versions vary.
4 # aws-cli/2.x.x Python/3.x Windows/10 exe/AMD64 prompt/off
```

Listing 1: Verifying AWS CLI v2 from Windows PowerShell.

On macOS, AWS provides a signed installer package. On Linux, AWS provides a bundled installer archive. The commands in this book focus on AWS CLI behavior, not installer mechanics; use your operating system’s official AWS CLI v2 instructions when in doubt.

0.4.2 Configuration Option A: IAM Identity Center

If you created an IAM Identity Center user, configure the CLI with SSO. The command opens a browser so you can authorize the device. The profile name should be explicit because later labs may use multiple accounts or Regions.

```
1 aws configure sso --profile aws-book-admin
2
3 # Typical prompts:
4 # SSO session name: aws-book
5 # SSO start URL: https://your-start-url.awsapps.com/start
6 # SSO region: us-east-1
7 # CLI default client Region: us-east-1
8 # CLI default output format: json
9
10 aws sso login --profile aws-book-admin
11 aws sts get-caller-identity --profile aws-book-admin
```

Listing 2: Configuring an AWS CLI profile with IAM Identity Center.

SSO profiles use temporary credentials cached locally after login. When the session expires, run `aws sso login --profile aws-book-admin` again. This is the recommended path for the book because it reduces the risk of leaked static access keys.

0.4.3 Configuration Option B: Access Keys

If you use an IAM admin user, create an access key for that user and configure a named profile. Keep the secret access key private; AWS shows it only once. Do not paste it into documents, screenshots, notebooks, Git repositories, or chat messages.

```
1 aws configure --profile aws-book-admin
2
3 # AWS Access Key ID [None]: AKIA...
4 # AWS Secret Access Key [None]: paste-secret-here
```

```

5 # Default region name [None]: us-east-1
6 # Default output format [None]: json
7
8 aws sts get-caller-identity --profile aws-book-admin

```

Listing 3: Configuring an AWS CLI profile with access keys.

The command writes configuration under `%USERPROFILE%\.aws`, including the `config` and `credentials` files. Treat the credentials file as sensitive. If a key is exposed, deactivate it immediately in IAM and create a replacement only if still needed.

Common Pitfall

Using the default profile accidentally. If one terminal uses `default` and another uses `aws-book-admin`, commands may run in different accounts or Regions. Use `-profile` in examples until you are confident, or set `$Env:AWS_PROFILE = 'aws-book-admin'` for the current PowerShell session.

0.4.4 Profiles, Regions, and Output Formats

A profile combines credentials with defaults such as Region and output format. This book uses `aws-book-admin` as the learning profile and `us-east-1` as the default Region unless a chapter says otherwise. JSON output is preferred because it can be queried, stored, and compared in tests.

```

1 $Env:AWS_PROFILE = 'aws-book-admin'
2 $Env:AWS_REGION = 'us-east-1'
3 $Env:AWS_DEFAULT_REGION = 'us-east-1'
4
5 aws configure list
6 aws sts get-caller-identity

```

Listing 4: Selecting the book profile and Region for the current PowerShell session.

PowerShell environment variables set this way affect only the current terminal session. That is useful: you can keep one terminal pointed at a sandbox and avoid changing global settings. If you want persistent configuration, use `aws configure set region us-east-1 -profile aws-book-admin`.

0.4.5 AWS CloudShell as a No-install Alternative

AWS CloudShell is a browser-based shell available from the AWS console in supported Regions. It includes AWS CLI, credentials from your console session, persistent home storage within service limits, and common command-line tools. CloudShell is excellent when you cannot install software locally, need a quick verification, or want to avoid local credential files.

CloudShell is still tied to your signed-in identity and selected Region. Before running a lab, check the Region selector and run `aws sts get-caller-identity`. Store only disposable training files in CloudShell home storage. Do not treat it as a permanent development workstation.

Tip

CloudShell removes local installation friction, not security responsibility. You still need MFA, least privilege, budget alerts, cleanup, and careful Region selection.

0.5 Cost Guardrails to Stay Free

0.5.1 Turn on Billing Visibility

Open the Billing and Cost Management console as root or an identity with billing permissions. Activate IAM access to billing if needed, then use the administrative identity for routine billing checks. Enable Free Tier usage alerts where available and confirm the email address that receives notices. Billing alerts do not prevent spending; they shorten the time between activity and awareness.

Cost Explorer helps you see cost trends by service, Region, usage type, linked account, and tag. It may need to be enabled and can take time to populate. For a learning account, the most important dashboard questions are simple: which services have charges, in which Regions, and from which resources?

Tip

Measure it. At the start and end of every hands-on session, open the Billing dashboard and Cost Explorer. Record current month-to-date spend and the services with nonzero cost. If the number changed unexpectedly, stop and investigate before starting the next chapter.

0.5.2 Create a Zero-Spend or Low-Spend Budget

AWS Budgets can send email alerts when actual or forecasted spend crosses thresholds. A zero-spend budget is useful in a new account because any charge matters. A low-spend budget, such as \$5 or \$10 per month, is practical if you expect tiny unavoidable charges while learning. The budget is a notification guardrail, not a hard spending cap.

Create at least two alerts: one for actual spend and one for forecasted spend. Actual spend tells you that charges have occurred. Forecasted spend warns that current usage trends may exceed your limit by the end of the month. Use an email address you actively read.

```

1 {
2   "BudgetName": "aws-book-low-spend",
3   "BudgetLimit": {
4     "Amount": "5.00",
5     "Unit": "USD"
6   },
7   "TimeUnit": "MONTHLY",
8   "BudgetType": "COST",
9   "CostFilters": {},
10  "CostTypes": {
11    "IncludeTax": true,
12    "IncludeSubscription": true,
13    "UseBlended": false,
14    "IncludeRefund": false,
15    "IncludeCredit": true,
16    "IncludeUpfront": true,
17    "IncludeRecurring": true,
18    "IncludeOtherSubscription": true,
19    "IncludeSupport": true,
20    "IncludeDiscount": true,
21    "UseAmortized": false
22  }
23 }
```

Listing 5: Minimal monthly AWS budget definition for a low-spend learning guardrail.

The following PowerShell example creates the JSON files and calls AWS Budgets. Replace the email address and account ID. Budgets is a global billing service, but the CLI still needs a configured profile.

```

1 $AccountId = '123456789012'
2 $Email = 'you@example.com'
3
4 @'
5 {
6   "BudgetName": "aws-book-low-spend",
7   "BudgetLimit": { "Amount": "5.00", "Unit": "USD" },
8   "TimeUnit": "MONTHLY",
9   "BudgetType": "COST",
10  "CostTypes": {
11    "IncludeTax": true,
12    "IncludeSubscription": true,
13    "UseBlended": false,
14    "IncludeRefund": false,
15    "IncludeCredit": true,
16    "IncludeUpfront": true,
17    "IncludeRecurring": true,
18    "IncludeOtherSubscription": true,
19    "IncludeSupport": true,
20    "IncludeDiscount": true,
21    "UseAmortized": false
22  }
23 }
24 '@ | Set-Content -Path .\budget.json -Encoding utf8
25
26 @"
27 [
28   {
29     "Notification": {
30       "NotificationType": "ACTUAL",
31       "ComparisonOperator": "GREATER_THAN",
32       "Threshold": 1,
33       "ThresholdType": "PERCENTAGE"
34     },
35     "Subscribers": [
36       { "SubscriptionType": "EMAIL", "Address": "$Email" }
37     ]
38   },
39   {
40     "Notification": {
41       "NotificationType": "FORECASTED",
42       "ComparisonOperator": "GREATER_THAN",
43       "Threshold": 80,
44       "ThresholdType": "PERCENTAGE"
45     },
46     "Subscribers": [
47       { "SubscriptionType": "EMAIL", "Address": "$Email" }
48     ]
49   }
50 ]
51 "@ | Set-Content -Path .\notifications.json -Encoding utf8
52
53 aws budgets create-budget '
54   --account-id $AccountId '
55   --budget file://budget.json '
56   --notifications-with-subscribers file://notifications.json

```

Listing 6: Creating an AWS Budget with actual and forecasted email alerts.

After creating the budget, confirm the subscription email if AWS sends a confirmation message.

Test that the budget appears in the console. If you later close or replace the account, recreate budgets in the new account; budgets are not portable across accounts automatically.

0.5.3 Free Tier Usage Alerts and Service-specific Controls

Free Tier usage alerts warn when usage approaches Free Tier limits for eligible services. Enable them from the Billing preferences page. They complement budgets because some Free Tier dimensions are usage-based rather than cost-based. For example, a service may remain at \$0 until the allowance is exceeded, then begin charging.

Some services also have their own limits, alarms, or quotas. S3 lifecycle expiration can delete temporary objects. CloudWatch alarms can detect unexpected traffic. Service Quotas can request or view quotas, but quotas are not a substitute for cleanup. For this book, the default policy is: create only what the lab requires, tag it, verify it, and delete it when the exercise says to delete it.

Common Pitfall

Relying on budgets as a kill switch. AWS Budgets sends notifications and can trigger actions in some scenarios, but it does not universally stop resource creation or halt all charges. Design labs so cleanup is explicit and cheap resources do not depend on expensive hidden infrastructure.

0.6 Choosing a Region and Naming Conventions

0.6.1 Default Region for the Book

Unless a chapter states otherwise, use `us-east-1`. It supports broad service coverage, keeps examples consistent, and is commonly used in public documentation. If your organization mandates another Region, use that Region consistently and change names accordingly. Be aware that service availability, pricing, quotas, and Free Tier eligibility can vary by Region.

A Region is not just a performance choice. It affects data residency, regulatory posture, disaster recovery design, KMS key location, networking, and service integration. For personal learning, consistency matters more than geographic perfection. For professional environments, Region selection should be a documented architecture decision.

0.6.2 Naming Convention

Resource names should be readable, unique where required, and safe to show in screenshots. This book uses the pattern:

`aws-book-<service>-<purpose>-<region>-<suffix>`

For globally unique names such as S3 buckets, append a short suffix such as your account ID fragment, date, or random string. Avoid uppercase letters in S3 bucket names. Avoid names that reveal sensitive data. Use hyphens rather than underscores for resource names unless the service requires otherwise.

Examples:

- `aws-book-s3-landing-us-east-1-1234`
- `aws-book-glue-demo-us-east-1`
- `aws-book-budget-low-spend`

0.6.3 Tagging Convention

Tags help you find, filter, allocate cost, and clean up resources. Not every AWS resource supports tags, but when tags are available, apply a consistent minimum set:

Tag key	Example value	Purpose
Project	AWSBook	Groups all book resources for search and cost review.
Environment	Training	Distinguishes labs from production, development, or shared sandboxes.
Owner	learner	Identifies who should clean up the resource. Do not put sensitive personal data here.
Chapter	Chapter0	Makes exercise resources easy to find and delete.
DeleteAfter	2026-08-31	Documents intended cleanup date. It is not automatic unless automation enforces it.

Table 1: Minimum training tags used throughout this book.

Tip

Use tags in the Billing dashboard and Cost Explorer. If a charge appears and the resource has no project or chapter tag, you have already found a process gap.

0.7 Setup Flow Across the Six Platform Books

Cross-Cloud Equivalence

The first hour in every cloud book follows the same pattern: understand the free offer, secure the administrative identity, install or open the shell, set cost guardrails, and verify the environment.

Setup step	AWS	Azure	GCP
Free offer	Free Tier (12 mo + always free)	Free account (\$200/30 days + always free)	Free trial (\$300/90 days + Always Free)
Sign-up guard	Card + root email	Card + Microsoft account	Card + Google account
Identity model	IAM / IAM Identity Center	Microsoft Entra ID + RBAC	Cloud IAM
Admin isolation	Never use root daily	Avoid Global Admin daily	Avoid Owner; least privilege
CLI	AWS CLI v2 (aws configure)	Azure CLI (az login)	gcloud CLI (gcloud init)
Browser shell	AWS CloudShell	Azure Cloud Shell	Cloud Shell
Cost guardrail	AWS Budgets + alerts	Cost Management budgets	Budgets & alerts
Console	AWS Management Console	Azure Portal	Google Cloud Console

The three non-object-store books differ: Microsoft Fabric uses a Microsoft 365/Power BI work account plus a Fabric trial capacity (no card, org tenant); Snowflake uses a 30-day/\$400 trial (no card) guarded by Resource Monitors; MongoDB Atlas offers a forever-free M0 cluster (no card) secured by database users and an IP allowlist.

0.8 Verification: Hello World

0.8.1 Check the Caller Identity

The safest first command is read-only: `aws sts get-caller-identity`. It returns the AWS account, user or role ARN, and user ID for the active credentials. Run it before every lab, especially if you work with multiple profiles.

```

1 $Env:AWS_PROFILE = 'aws-book-admin'
2 $Env:AWS_REGION = 'us-east-1'
3
4 aws sts get-caller-identity
5
6 # Expected output shape:
7 # {
8 #     "UserId": "AROAXAMPLEID:learner",
9 #     "Account": "123456789012",
10 #     "Arn": "arn:aws:sts::123456789012:assumed-role/
    AWSReservedSSO_AdministratorAccess_abc123/learner"
11 # }
```

Listing 7: Confirming which AWS identity the CLI is using.

If the account ID is not the account you expect, stop. Do not continue a hands-on lab in the wrong account. If the ARN shows root, stop and switch to your administrative identity.

0.8.2 Create, List, and Delete a Small S3 Bucket

This smoke test proves that credentials, Region, permissions, and cleanup all work. It creates an empty S3 bucket, lists it, and deletes it. S3 bucket names are globally unique, so the example constructs a suffix from your account ID and the current date. If the name already exists, change the suffix.

```

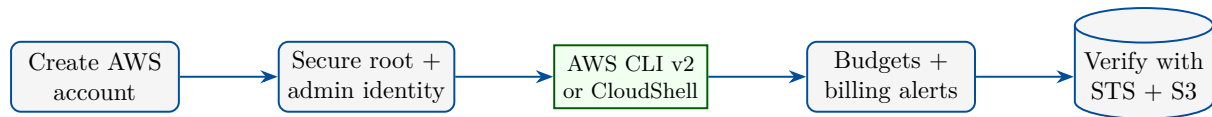
1 $Env:AWS_PROFILE = 'aws-book-admin'
2 $Env:AWS_REGION = 'us-east-1'
3 $AccountId = (aws sts get-caller-identity --query Account --output text)
4 $Suffix = Get-Date -Format 'yyyyMMddHHmmss'
5 $Bucket = "aws-book-chapter0-$AccountId-$Suffix"
6
7 aws s3api create-bucket --bucket $Bucket --region $Env:AWS_REGION
8 aws s3api put-bucket-tagging '
9     --bucket $Bucket '
10     --tagging 'TagSet=[{Key=Project,Value=AWSBook},{Key=Environment,Value=
    Training},{Key=Chapter,Value=Chapter0}]'
11
12 aws s3 ls "s3://$Bucket"
13 aws s3api list-buckets --query "Buckets[?Name=='$Bucket'].Name" --output text
14
15 aws s3api delete-bucket --bucket $Bucket
16 aws s3api list-buckets --query "Buckets[?Name=='$Bucket'].Name" --output text
```

Listing 8: S3 smoke test with cleanup from Windows PowerShell.

Expected behavior: `create-bucket` succeeds, `aws s3 ls` returns no objects because the bucket is empty, `list-buckets` prints the bucket name before deletion, and the final `list-buckets` prints nothing after deletion. If deletion fails with `BucketNotEmpty`, remove objects and versions first; this smoke test should not create any objects.

Note

S3's `create-bucket` syntax differs for `us-east-1` and other Regions. In `us-east-1`, the simple command above is valid. In most other Regions, include `-create-bucket-configuration LocationConstraint=<region>`.

0.8.3 Optional Setup Flow Diagram

Build guardrails before creating data-engineering resources.

Figure 1: Chapter 0 setup flow: account, secure identity, tooling, cost guardrail, and verification.

0.9 Further Reading

Review the AWS account creation, Free Tier, IAM, IAM Identity Center, AWS CLI v2, CloudShell, Billing, Budgets, and Cost Explorer documentation from AWS. Keep those pages close while working through the first labs because console navigation and Free Tier offers can change. For security posture, also read AWS guidance on the root user, MFA, least privilege, and the shared-responsibility model. For cost control, read the current pricing page for each service before running a chapter that creates resources.

Key Takeaways

- Free Tier is a set of limited offers; it is not a guarantee that all account activity is free.
- Protect the root user with MFA and use a separate administrative identity for daily labs.
- IAM Identity Center and `aws configure sso` are preferred because they use temporary credentials.
- Named profiles and a consistent Region prevent many accidental-account and wrong-Region mistakes.
- Budgets, Free Tier alerts, Cost Explorer, and routine Billing dashboard checks are required learning-account hygiene.
- Tags, naming conventions, and cleanup commands make hands-on work auditable and reversible.
- A successful `sts get-caller-identity` plus S3 create, list, and delete test proves that Chapter 1 can start safely.

0.10 Exercises

1. **(Checklist)** Complete this environment setup checklist before Chapter 1:
 - AWS account created and account ID recorded.
 - Root user password stored securely and root MFA enabled.

- Daily administrative identity created through IAM Identity Center or an IAM admin user and group.
 - Administrative identity can sign in to the console without using root.
 - AWS CLI v2 installed locally or AWS CloudShell selected as the working shell.
 - Named profile `aws-book-admin` configured, or CloudShell identity verified.
 - Default Region set to `us-east-1` unless your organization requires another Region.
 - Billing access verified, Free Tier usage alerts enabled where available, and an AWS Budget created.
 - Cost Explorer enabled or checked for availability.
 - `aws sts get-caller-identity` returns the expected account and non-root identity.
 - S3 smoke-test bucket created, listed, deleted, and confirmed absent.
2. **(Conceptual)** Explain the difference between the root user, an IAM user, an IAM role, and an IAM Identity Center user. Which should you use for daily labs and why?
 3. **(Cost)** Create a monthly budget with actual and forecasted alerts. Record the budget name, threshold, email recipient, and the date you verified it in the console.
 4. **(CLI)** Run `aws configure list` and `aws sts get-caller-identity`. Save the account ID and ARN in your private lab notes, not in a public repository.
 5. **(Region)** Choose a Region other than `us-east-1`. Identify one service from later chapters that is available there and one pricing or availability difference you would check before using it.
 6. **(Governance)** Propose a tag policy for a shared training account. Include owner, project, environment, chapter, and cleanup-date fields.
 7. **(Cleanup)** Search the console for resources tagged `Project=AWSBook`. Confirm that no Chapter 0 resources remain after the S3 smoke test.

0.11 Curated Community Resources

Beyond this book and the vendor documentation, the following community-curated resources are worth your time. Do not try to consume everything at once: pick one resource per category, practice it with real data, and build something small before moving on.

Practical Case Studies (Video Walkthroughs)

Fifteen end-to-end builds that show how the tools work together:

1. SQL Data Warehouse from Scratch — SQL, ETL, data modeling, star schemas (<https://lnkd.in/gcrJ2qde>)
2. End-to-End Data Analytics Project — Python, Pandas, SQL Server, data cleaning (<https://lnkd.in/gHxTN-B2>)
3. Netflix Data Engineering Project — Python, SQL, ELT, exploratory analysis (<https://lnkd.in/gMkbpWmh>)
4. DuckDB and Python Data Engineering Project — API ingestion, pipeline development (<https://lnkd.in/gEPnubGA>)

5. Airflow Podcast Data Pipeline — Airflow, APIs, scheduling (<https://lnkd.in/gmy5rS4Z>)
6. Automated Python ETL Pipeline with Airflow — SQL Server, DAG development (<https://lnkd.in/ghrBc8BE>)
7. PySpark and dbt Data Engineering Project — incremental loading, dimensional modeling (<https://lnkd.in/gVZjvyR7>)
8. Apache Spark Data Engineering Project — PySpark, Databricks (<https://lnkd.in/gVNkqk5A>)
9. Airbnb Data Engineering Project — dbt, Snowflake, AWS, testing (<https://lnkd.in/g2f4TxPW>)
10. AWS ETL Pipeline Project — S3, Lambda, Glue, Athena, serverless ETL (<https://lnkd.in/gFC3NkNs>)
11. AWS Data Warehouse Project — PySpark, Glue, Redshift (<https://lnkd.in/gyZtHMEF>)
12. End-to-End Azure Data Engineering Project — Data Factory, Databricks, ADLS (<https://lnkd.in/gUpbRjzE>)
13. Spotify Azure Data Engineering Project — streaming, Delta Lake (<https://lnkd.in/gUGKuZ8N>)
14. Uber Real-Time Data Engineering Project — Event Hubs, structured streaming (<https://lnkd.in/gu7Tpg7D>)
15. End-to-End GCP Data Engineering Project — Cloud Storage, Dataproc, BigQuery (<https://lnkd.in/gv8Yedfu>)

Free Video Courses

Twenty videos covering the essential skill path — recommended learning order: Python, SQL, Linux, Git, Docker, data modeling, data warehousing, Spark, Kafka, Airflow, dbt, cloud, portfolio projects.

- **Foundations:** Data Engineering Course for Beginners (<https://lnkd.in/gprfkqnp>); Big Data Engineer Masterclass (<https://lnkd.in/gqPkrg63>)
- **Core tools:** Python for Data Engineers (<https://lnkd.in/gZeKJEi4>); SQL for Big Data Engineering (<https://lnkd.in/gt5DZXwQ>); PostgreSQL Full Course (<https://lnkd.in/gAU5n7gM>); Linux Command Line (<https://lnkd.in/gA-7zFaQ>); Git and GitHub Tutorial (<https://lnkd.in/gs73kHau>); Docker Tutorial (<https://lnkd.in/g59TCWPD>)
- **Warehousing and modeling:** Data Warehousing Beginner's Guide (https://lnkd.in/gS_-iJAb); Data Modeling, Star Schema and Kimball (<https://lnkd.in/g4it9vpy>)
- **Batch processing:** PySpark Tutorial for Beginners (<https://lnkd.in/gre86dH6>); PySpark Full Course (<https://lnkd.in/gZXg38hk>)
- **Streaming:** Apache Kafka Full Course (https://lnkd.in/gnnXc9_d); Kafka Crash Course with Project (<https://lnkd.in/gP6DFCem>)
- **Pipelines:** Apache Airflow Tutorial (<https://lnkd.in/gbFzzReZ>); dbt Tutorial with Netflix Project (<https://lnkd.in/gHvmdjNX>); ELT with dbt, Snowflake and Airflow (https://lnkd.in/g5ZP_xC3)
- **Cloud:** AWS for Data Engineers (<https://lnkd.in/g8vRZmkp>)

- **End-to-end projects:** Airbnb Project with Snowflake, dbt and AWS (<https://lnkd.in/g2f4TxPW>); Real-Time E-Commerce with Kafka and Snowflake (<https://lnkd.in/g58V6A3P>)

GitHub Repositories

Twenty free repositories to learn from, practice with, and build upon. Choose one roadmap, complete one project, study the code structure, then break it, fix it, and document what you learned.

- **Roadmaps and guides:** Data Engineering Zoomcamp (<https://lnkd.in/gxKmbbHc>); Data Engineer Handbook (<https://lnkd.in/gstev9z7>); Data Engineering Cookbook (<https://lnkd.in/gABzJ-vE>); Complete Data Engineer Roadmap (<https://lnkd.in/gmwQrmvm>)
- **Curated lists:** Awesome Data Engineering (<https://lnkd.in/gh4Ky273>); Awesome Big Data (https://lnkd.in/g_y-n3zm); Awesome Open-Source Data Engineering (<https://lnkd.in/ggDTw66U>)
- **Portfolio and analytics engineering:** Data Engineering Project Template (<https://lnkd.in/g-QKrMXV>); Jaffle Shop dbt Practice Project (<https://lnkd.in/gmXVhYKX>); dbt Core (<https://lnkd.in/gnCXxNbB>)
- **Ingestion and orchestration:** Apache Airflow (<https://lnkd.in/gqvG5F6N>); Airbyte (<https://lnkd.in/g2B9AVwM>); dlt (<https://lnkd.in/g3A8BMp3>)
- **Processing:** Apache Spark (<https://lnkd.in/g5CPE3hh>); Apache Kafka (<https://lnkd.in/gdH5T4xX>)
- **Storage and lakehouses:** DuckDB (<https://lnkd.in/gehQd42W>); Apache Iceberg (<https://lnkd.in/gVweVV8y>)
- **Quality, lineage and governance:** Great Expectations (<https://lnkd.in/g8TPyAy8>); OpenLineage (https://lnkd.in/gz3_MiAP); DataHub (<https://lnkd.in/gEcNMRkJ>)

Part I

Core Data Infrastructure and Storage

Amazon S3 and Storage Foundations

Learning Objectives

- Explain Amazon S3’s object-storage architecture, namespace model, durability model, availability model, and strong consistency guarantees.
- Compare S3 Standard, Standard-IA, One Zone-IA, Glacier Instant Retrieval, Glacier Flexible Retrieval, and Glacier Deep Archive for data engineering workloads.
- Design lifecycle, versioning, Object Lock, legal hold, and replication policies that match retention, recovery, and compliance requirements.
- Choose among AWS DataSync, Storage Gateway, and the AWS Snow Family for online, hybrid, and offline migration scenarios.
- Deploy a bucket with the AWS CLI, configure a 30-day transition to Glacier Flexible Retrieval, and connect S3 event notifications to Amazon SQS.
- Design a multi-region healthcare disaster-recovery storage architecture for 10 PB of medical-imaging data under HIPAA-style constraints.

Cross-Cloud Equivalence

Although this book teaches **AWS**, the Week 1 storage foundations map almost one-to-one onto the other clouds—learn one mental model and carry it everywhere.

Capability	AWS	Azure	GCP
Object storage	Amazon S3	ADLS Gen2 / Blob	Cloud Storage
Hot tier	S3 Standard	Hot	Standard
Infrequent access	S3 Standard-IA	Cool / Cold	Nearline
Archive (instant)	Glacier Instant Retrieval	Cold	Coldline
Deep archive	Glacier Deep Archive	Archive	Archive
Lifecycle rules	Lifecycle policies	Lifecycle management	Object Lifecycle Mgmt
Versioning	S3 Versioning	Blob versioning	Object Versioning
Immutability (WORM)	Object Lock	Immutable blob storage	Bucket Lock
Cross-region copy	CRR / SRR	Object replication (GRS)	Dual/multi-region + Transfer
Online transfer	DataSync	AzCopy	Storage Transfer Service
Offline transfer	Snow Family	Data Box	Transfer Appliance
Design durability	11 nines	11 nines (LRS) / 16 (GRS)	11 nines
Availability SLA	99.9% (Standard)	99.9% (RA-GRS read 99.99%)	99.95% (multi-region)
Encryption at rest	SSE-S3 / SSE-KMS / SSE-C	SSE + CMK via Key Vault	Google-managed / CMEK / CSEK
Access control	IAM & bucket policies, ACLs	Azure RBAC, SAS, POSIX ACLs	IAM, ACLs, signed URLs
Pricing levers	Storage + requests + egress + retrieval	Storage + operations + egress + retrieval	Storage + operations + egress + retrieval
Consistency	Strong read-after-write	Strong read-after-write	Strong (global)

Beyond raw object storage, the same layer reappears as a managed abstraction: **Fabric OneLake** is a tenant-wide lake built on ADLS Gen2 (Delta–Parquet), **Snowflake** persists data as immutable micro-partitions on the provider’s object store, and **MongoDB’s** WiredTiger is a node-local engine rather than object storage—the contrast is itself instructive.

1.1 Week 1 Roadmap and Mental Model

Amazon Simple Storage Service (Amazon S3) is the base layer of most AWS data engineering architectures. It is the landing zone for raw extracts, the durable system of record for data lakes, the staging layer for Redshift and EMR, the archive tier for infrequently accessed records, and the integration point for event-driven pipelines. The service looks simple because its core API is small: create buckets, put objects, get objects, list keys, attach metadata, and apply policies. The engineering challenge is not memorizing API names; it is matching S3’s storage semantics to data lifecycle, governance, cost, and reliability goals [33, 3].

A useful starting model is that S3 separates the concerns usually bundled inside a file system. The *bucket* is a regional administrative container. The *object key* is a string used for lookup and organization. The *object body* is immutable for a given version. Metadata, tags, access policies, encryption settings, lifecycle rules, event notifications, and replication rules sit around those objects as managed controls. Data engineers use those controls to turn inexpensive object storage into a governed analytical platform.

Definition

Amazon S3 is a regional object storage service. It stores data as objects inside buckets, exposes API-based access rather than block-device semantics, and provides features for access control, encryption, lifecycle management, replication, retention, and event notification.

Key Terms

Object: bytes plus metadata, tags, and a key. **Bucket:** regional namespace and policy boundary. **Prefix:** leading portion of a key used for organization and request distribution. **Version:** immutable historical instance of an object when versioning is enabled. **Storage class:** economic and operational placement for an object.

1.2 Monday: S3 Architecture and Storage Classes

1.2.1 Object Storage, Not a Shared File System

S3 is not POSIX storage. It does not expose in-place mutation, directory locks, or append semantics. A key such as `raw/xray/2026/07/30/study-001.dcm` may look like a path, but S3 treats it as a key string. Console folders are a user-interface convention based on delimiters. This distinction matters for Spark, Glue, Athena, and Redshift Spectrum because those engines discover data by listing prefixes and reading objects rather than traversing a local file system.

For data engineering, the non-POSIX model is a strength. It allows S3 to store very large numbers of objects, serve high request rates, decouple compute from storage, and become the durable substrate for a lakehouse architecture [85]. It also means writers must use commit protocols and table formats carefully. A failed writer cannot rely on renaming a temporary directory atomically in the way a local file system might.

Common Pitfall

Treating S3 like a local file system. Directory renames, small-file-heavy workloads, and append-in-place assumptions can create slow listings, expensive request patterns, and broken table commits. Design for object immutability, partition-aware prefixes, and idempotent writers.

1.2.2 Regional Buckets and Global Names

Each S3 bucket has a globally unique name and belongs to one AWS Region. Objects stay in that Region unless replication, transfer, or client code moves them. This regional boundary is a compliance and architecture decision: it affects latency, data residency, KMS key choice, disaster recovery, and downstream service placement. A bucket name should be stable and descriptive, but not leak sensitive data; for example, use `acme-health-imaging-prod-us-east-1` rather than patient or project names.

Note

Strong service durability does not remove the need for architecture-level recovery design. Region choice, replication, versioning, backup policy, and access controls are still your responsibility under the AWS shared-responsibility model.

1.2.3 Durability and Availability

AWS designs S3 for extremely high durability by redundantly storing objects across multiple Availability Zones in a Region for most storage classes [33]. Durability is the probability that stored data is not lost. Availability is the probability that a request can be served when needed. These are related but different: an object can remain durably stored even during an availability impairment that prevents a client from reading it immediately.

Definition

Durability measures resistance to data loss over time. **Availability** measures readiness to serve requests at a point in time. S3 storage-class selection must consider both, plus retrieval latency, minimum storage duration, retrieval fees, and resilience to Availability Zone loss.

Storage class	Primary use	Access pattern	Design caution
S3 Standard	Active data lakes, landing zones, analytics staging	Frequent, low-latency access	Highest storage price among listed classes; still usually right for hot data
S3 Standard-IA	Durable multi-AZ storage for infrequent access	Millisecond access, lower storage cost	Retrieval fee and minimum storage duration make it poor for short-lived data
S3 One Zone-IA	Re-creatable infrequent data	Millisecond access in one AZ	Lower resilience; do not use for the only copy of critical data
S3 Glacier Instant Retrieval	Rarely accessed archive needing milliseconds	Millisecond archive access	Retrieval pricing and minimum duration require careful modeling
S3 Glacier Flexible Retrieval	Long-lived archive with minutes-to-hours retrieval	Expedited, standard, or bulk retrieval	Not appropriate for interactive analytics
S3 Glacier Deep Archive	Lowest-cost deep archive	Hours-scale retrieval	Retrieval planning and restore operations must be operationalized

Table 1.1: Storage-class choice as a workload and risk decision, not merely a price decision.

1.2.4 Consistency Model

Modern S3 provides strong read-after-write consistency for puts and deletes of objects, and strong consistency for list operations after writes [2]. If a successful PUT returns, a subsequent GET or LIST should observe that change. This simplifies data-lake pipelines compared with older patterns that inserted sleep intervals or extra metadata stores purely to compensate for eventual list behavior.

Tip

Strong consistency simplifies pipeline correctness, but it does not make multi-object transactions automatic. If a job writes 10,000 objects and fails halfway, S3 will consistently show the partial result. Table formats, manifests, job bookmarks, and idempotent orchestration still matter.

1.3 Choosing a Storage Class for Data Engineering

Storage class is a lifecycle attribute. It should follow the data’s access pattern and risk profile over time. A common analytical object might begin in S3 Standard while ingestion, validation, profiling, and downstream consumption are active. After 30 or 90 days, it might transition to Standard-IA or Glacier Instant Retrieval. After a legal or business retention threshold, it might move to Glacier Flexible Retrieval or Deep Archive. For a reproducible intermediate dataset, One Zone-IA may be acceptable; for medical records or financial records, it usually is not.



Cost decreases left-to-right, but retrieval latency, retrieval fees, and operational planning generally increase.

Figure 1.1: A storage-class lifecycle path for analytical and archival objects. Actual thresholds must be justified by access data, compliance needs, and retrieval objectives.

1.4 Tuesday: Lifecycle, Versioning, Object Lock, and Replication

1.4.1 Lifecycle Policies

Lifecycle rules automate transitions and expirations. A rule can target a bucket, prefix, tag, object size, current versions, noncurrent versions, incomplete multipart uploads, or delete markers. In a data lake, lifecycle rules are often applied by zone. Raw data might be retained longer than curated derivatives. Temporary job output might expire after a few days. Audit logs might transition to archive but never be deleted until the retention period ends.

Note

Lifecycle is not a backup strategy by itself. Expiration rules delete data intentionally. Use versioning, replication, backup policy, and retention controls for recovery from accidental deletion or malicious overwrite.

1.4.2 Versioning

S3 Versioning keeps multiple variants of an object under the same key. A normal delete creates a delete marker rather than immediately destroying prior versions. Versioning is a foundation for recovery, replication, and Object Lock. It is also a cost multiplier when pipelines rewrite large partitions repeatedly. Data engineers should pair versioning with lifecycle rules for noncurrent versions, monitoring, and explicit restore procedures.

Common Pitfall

Enabling versioning without noncurrent-version lifecycle. A job that repeatedly overwrites a 500 GB partition can silently retain every historical version. This may be exactly what compliance requires, but if it is accidental, storage cost will grow quickly.

1.4.3 Object Lock: Governance, Compliance, and Legal Hold

S3 Object Lock provides write-once-read-many controls for object versions when the bucket is created with Object Lock support and versioning is enabled [70]. In governance mode, privileged

users with specific permissions can bypass retention. In compliance mode, protected object versions cannot be overwritten or deleted by any user, including the root account, until retention expires. Legal hold is an independent hold flag that prevents overwrite or deletion until removed by an authorized principal.

Definition

Governance mode protects objects while allowing explicitly authorized bypass for administrative workflows. **Compliance mode** prevents deletion or overwrite until the retention date expires. **Legal hold** has no fixed expiration date; it remains until an authorized user removes it.

Tip

For regulated datasets, design Object Lock before ingestion. Some settings must be enabled at bucket creation, and compliance mode is intentionally difficult to undo. Test policies in a non-production account before applying retention to irreplaceable data.

1.4.4 S3 Replication: SRR and CRR

S3 Replication copies objects automatically according to rules [73]. Same-Region Replication (SRR) is useful for account separation, log aggregation, data sovereignty within a Region, and different ownership or encryption boundaries. Cross-Region Replication (CRR) is used for disaster recovery, lower-latency regional access, regulatory isolation, or multi-region analytical platforms. Replication is asynchronous, so designs must define a recovery point objective (RPO), monitor replication lag, and decide whether delete markers and existing objects are included.

Feature	Use it when	Question to ask in review
Lifecycle transition	Access declines predictably	What retrieval objective and minimum-duration charge apply?
Lifecycle expiration	Data has a clear disposal date	Who approved deletion and how is it audited?
Versioning	Accidental overwrite or delete must be recoverable	How are noncurrent versions aged and monitored?
Object Lock	WORM retention is required	Governance or compliance mode, and who can bypass?
Legal hold	Litigation or investigation requires indefinite hold	What process removes the hold and records approval?
SRR	Same-region separation is needed	Is the target account, owner, and KMS key correct?
CRR	Regional disaster recovery is needed	What RPO, RTO, encryption, and failover process are tested?

Table 1.2: S3 management features mapped to operational review questions.

1.5 Wednesday: Migration and Hybrid Storage

1.5.1 AWS DataSync

AWS DataSync is a managed online transfer service for moving data between on-premises storage, edge locations, other clouds, and AWS storage services such as S3, EFS, and FSx [43]. It is appropriate when a network path exists and ongoing or incremental synchronization is needed. DataSync handles parallelization, verification, scheduling, bandwidth limits, and task

reporting. For data engineers, it is often the cleanest path for moving existing file shares into S3 landing zones while preserving metadata required by downstream processing.

1.5.2 AWS Storage Gateway

AWS Storage Gateway is a hybrid cloud storage service. File Gateway presents file protocols backed by S3. Volume Gateway presents block storage with cloud-backed snapshots. Tape Gateway presents a virtual tape library for backup applications [58]. Storage Gateway is not merely a migration tool; it is a bridge for workloads that cannot immediately be rewritten to native object APIs.

Definition

File Gateway maps NFS or SMB file access to S3 objects. **Volume Gateway** provides iSCSI block volumes with snapshots. **Tape Gateway** emulates tape workflows for backup software while storing virtual tapes in AWS.

1.5.3 AWS Snow Family

The AWS Snow Family supports offline or edge transfer when networks are too slow, too expensive, unreliable, or unavailable [56]. Snowcone is a small rugged device for edge and limited transfer. Snowball Edge supports larger transfer and edge compute. Snowmobile is a truck-scale option for extremely large migrations. Offline migration does not remove data-engineering responsibilities: you still need inventory, chain of custody, encryption, verification, cutover planning, and reconciliation.

Migration option	Best fit	Trade-off
DataSync	Online, repeatable transfer with available bandwidth	Requires network connectivity; large first loads may still take time
File Gateway	Applications need NFS/SMB while AWS stores objects	File semantics can hide object-layout choices; not a substitute for native data-lake design
Volume Gateway	Block workloads and snapshot workflows	Less natural for analytical object processing
Tape Gateway	Existing backup software expects tapes	Good transition path; not an analytical ingestion pattern
Snowcone	Edge sites, small rugged transfers, constrained environments	Operational handling and shipping time
Snowball Edge	Large offline transfer or edge processing	Requires device logistics, job planning, validation
Snowmobile	Massive data-center-scale migration	Specialized engagement; heavy operational planning

Table 1.3: Migration and hybrid-storage options for AWS data engineering foundations.

Tip

Estimate migration time before choosing a service. A 10 PB migration over a 1 Gbps link can take years after overhead and contention. Offline transfer may be the only practical first-load path, followed by DataSync or application replication for deltas.

1.6 Thursday Hands-on Project: Bucket, Lifecycle, and Events

This lab creates an S3 bucket, enables versioning, applies a 30-day transition to S3 Glacier Flexible Retrieval, creates an SQS queue, and configures S3 event notifications. It is intentionally CLI-first so that the same actions can be automated later in CloudFormation, CDK, Terraform, or CI/CD. Replace names, account IDs, and Regions before running.

Note

The commands below create billable AWS resources. Use a sandbox account, set a unique bucket name, and delete resources after testing. Bucket names are globally unique; append your initials, account alias, or a random suffix.

```

1 # PowerShell users can run these commands from a shell with AWS CLI v2
   configured.
2 # Replace the values before running.
3 REGION="us-east-1"
4 ACCOUNT_ID="123456789012"
5 BUCKET="aws-master-data-engineer-week1-123456789012"
6 QUEUE="s3-ingest-events-week1"
7
8 aws s3api create-bucket \
9   --bucket "$BUCKET" \
10  --region "$REGION"
11
12 aws s3api put-bucket-versioning \
13   --bucket "$BUCKET" \
14   --versioning-configuration Status=Enabled
15
16 cat > lifecycle.json <<'JSON'
17 {
18   "Rules": [
19     {
20       "ID": "transition-raw-objects-to-glacier-after-30-days",
21       "Status": "Enabled",
22       "Filter": { "Prefix": "raw/" },
23       "Transitions": [
24         { "Days": 30, "StorageClass": "GLACIER" }
25       ],
26       "NoncurrentVersionTransitions": [
27         { "NoncurrentDays": 30, "StorageClass": "GLACIER" }
28       ],
29       "AbortIncompleteMultipartUpload": { "DaysAfterInitiation": 7 }
30     }
31   ]
32 }
33 JSON
34
35 aws s3api put-bucket-lifecycle-configuration \
36   --bucket "$BUCKET" \
37   --lifecycle-configuration file:///lifecycle.json
38
39 QUEUE_URL=$(aws sqs create-queue \
40   --queue-name "$QUEUE" \
41   --attributes VisibilityTimeout=60 \
42   --query QueueUrl \
43   --output text)
44
45 QUEUE_ARN=$(aws sqs get-queue-attributes \
46   --queue-url "$QUEUE_URL" \
47   --attribute-names QueueArn \
48   --query 'Attributes.QueueArn' \

```

```
49 --output text)
```

Listing 1.1: AWS CLI setup for an S3 bucket with versioning, lifecycle transition, SQS queue, and S3 event notification.

```
1 cat > sqs-policy.json <<JSON
2 {
3   "Version": "2012-10-17",
4   "Statement": [
5     {
6       "Sid": "AllowS3ToSendObjectCreatedEvents",
7       "Effect": "Allow",
8       "Principal": { "Service": "s3.amazonaws.com" },
9       "Action": "sqs:SendMessage",
10      "Resource": "$QUEUE_ARN",
11      "Condition": {
12        "ArnLike": { "aws:SourceArn": "arn:aws:s3:::$BUCKET" },
13        "StringEquals": { "aws:SourceAccount": "$ACCOUNT_ID" }
14      }
15    }
16  ]
17 }
18 JSON
19
20 aws sqs set-queue-attributes \
21   --queue-url "$QUEUE_URL" \
22   --attributes Policy="$(cat sqs-policy.json)"
23
24 cat > notification.json <<JSON
25 {
26   "QueueConfigurations": [
27     {
28       "Id": "send-raw-object-created-events-to-sqs",
29       "QueueArn": "$QUEUE_ARN",
30       "Events": ["s3:ObjectCreated:*"],
31       "Filter": {
32         "Key": {
33           "FilterRules": [
34             { "Name": "prefix", "Value": "raw/" }
35           ]
36         }
37       }
38     }
39   ]
40 }
41 JSON
42
43 aws s3api put-bucket-notification-configuration \
44   --bucket "$BUCKET" \
45   --notification-configuration file://notification.json
46
47 printf 'study_id,modality\n001,XRAY\n' > sample.csv
48 aws s3 cp sample.csv "s3://$BUCKET/raw/xray/sample.csv"
49 aws sqs receive-message --queue-url "$QUEUE_URL" --max-number-of-messages 1
```

Listing 1.2: Continuing the AWS CLI event-notification configuration.

Common Pitfall

Copying Bash syntax directly into PowerShell. The listing uses portable shell conventions for compactness. In native PowerShell, assign variables with `$REGION = 'us-east-1'` and create JSON files with here-strings. The AWS CLI calls are the important part.

The lifecycle JSON in listing 1.3 is the minimal policy extracted from the lab. It transitions current and noncurrent objects under `raw/` after 30 days and aborts incomplete multipart uploads after seven days.

```

1 {
2   "Rules": [
3     {
4       "ID": "transition-raw-objects-to-glacier-after-30-days",
5       "Status": "Enabled",
6       "Filter": { "Prefix": "raw/" },
7       "Transitions": [
8         { "Days": 30, "StorageClass": "GLACIER" }
9       ],
10      "NoncurrentVersionTransitions": [
11        { "NoncurrentDays": 30, "StorageClass": "GLACIER" }
12      ],
13      "AbortIncompleteMultipartUpload": { "DaysAfterInitiation": 7 }
14    }
15  ]
16 }

```

Listing 1.3: Lifecycle configuration for a 30-day transition to S3 Glacier Flexible Retrieval.

1.6.1 Boto3 Verification and Event Reader

The Python snippet below checks versioning, lifecycle rules, and SQS events. In production, this logic would belong in integration tests or deployment verification, not in an ad hoc notebook.

```

1 import json
2 import os
3
4 import boto3
5
6 region = os.environ.get("AWS_REGION", "us-east-1")
7 bucket = os.environ["BUCKET"]
8 queue_url = os.environ["QUEUE_URL"]
9
10 s3 = boto3.client("s3", region_name=region)
11 sqs = boto3.client("sqs", region_name=region)
12
13 versioning = s3.get_bucket_versioning(Bucket=bucket)
14 assert versioning.get("Status") == "Enabled", versioning
15
16 lifecycle = s3.get_bucket_lifecycle_configuration(Bucket=bucket)
17 rules = {rule["ID"]: rule for rule in lifecycle["Rules"]}
18 rule = rules["transition-raw-objects-to-glacier-after-30-days"]
19 assert rule["Transitions"][0]["Days"] == 30
20 assert rule["Transitions"][0]["StorageClass"] == "GLACIER"
21
22 s3.put_object(
23     Bucket=bucket,
24     Key="raw/xray/boto3-smoke-test.txt",
25     Body=b"week 1 smoke test\n",
26     ContentType="text/plain",
27 )
28
29 messages = sqs.receive_message(
30     QueueUrl=queue_url,
31     MaxNumberOfMessages=1,
32     WaitTimeSeconds=10,
33 ).get("Messages", [])
34
35 if not messages:

```



```

36     raise RuntimeError("No S3 event arrived in SQS within 10 seconds")
37
38 body = json.loads(messages[0]["Body"])
39 record = body["Records"][0]
40 print(record["eventName"], record["s3"]["object"]["key"])

```

Listing 1.4: Boto3 verification for S3 lifecycle and SQS event delivery.

1.7 Friday Use Case: Healthcare Disaster-Recovery Storage Architecture

A healthcare provider processes 10 PB of X-ray images. The platform must ingest studies from hospitals, preserve original DICOM objects, support analytics and ML labeling, survive a regional outage, and maintain HIPAA-style controls for encryption, access, audit, retention, and deletion. The storage architecture should be conservative: raw medical images are irreplaceable, regulated, and expensive to reacquire.

1.7.1 Requirements and Assumptions

Assume the primary Region is `us-east-1` and the recovery Region is `us-west-2`. Images arrive from hospitals through private network links, DataSync agents, and occasional Snowball Edge jobs for backfills. The provider requires server-side encryption with customer-managed KMS keys, least-privilege access, audit logs, Object Lock for raw images, CRR for regional recovery, lifecycle transition for cost control, and a tested failover runbook. The target RPO is measured in minutes for online ingestion and in the shipping interval for offline backfills. The RTO depends on downstream processing, not only S3 object availability.

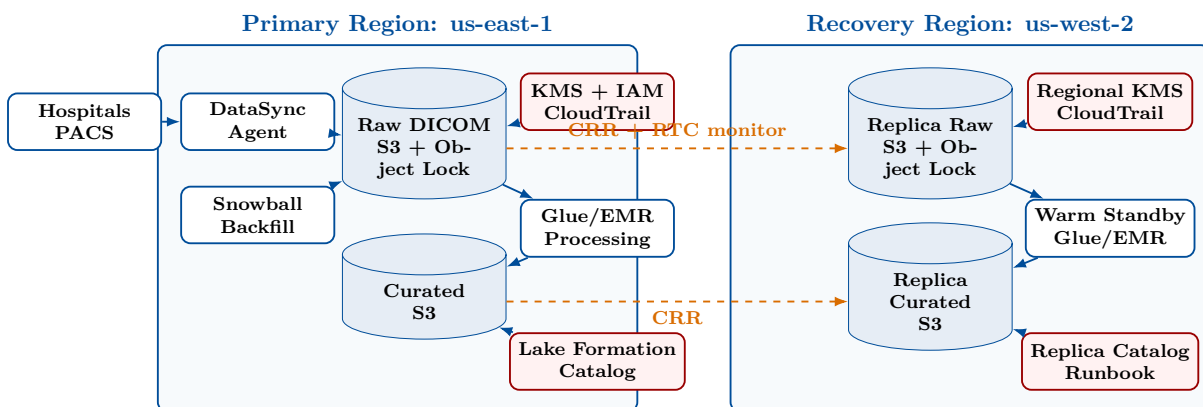


Figure 1.2: A multi-region compliant disaster-recovery storage architecture for 10 PB of healthcare imaging data. Replication protects regional availability; Object Lock, KMS, IAM, Lake Formation, and CloudTrail support compliance controls.

1.7.2 Architecture Decisions

Landing and retention. Raw DICOM images land in a primary S3 bucket with versioning and Object Lock enabled. Compliance mode may be justified for legally mandated immutable retention, but governance mode may be safer during early rollout if the organization still needs an authorized break-glass path. Legal holds are reserved for investigations or litigation.

Encryption and key management. Buckets require server-side encryption with AWS KMS customer-managed keys. Primary and replica Regions use separate regional keys. Key policies

are limited to ingestion roles, processing roles, security administrators, and replication roles. CloudTrail data events are enabled for sensitive buckets and delivered to a separate log archive account.

Replication and recovery. CRR copies raw and curated objects to the recovery Region. The replication role must be permitted to use both source and destination KMS keys. Replication metrics and alarms track lag. The failover runbook includes DNS or application endpoint changes, catalog validation, processing job activation, and a reconciliation process for objects written near the outage window.

Lifecycle and cost. Raw images remain in S3 Standard during active diagnosis, annotation, and model training windows. After an access-based threshold, they transition to Glacier Instant Retrieval or Glacier Flexible Retrieval depending on clinical retrieval requirements. Deep Archive is considered only for records with hours-scale retrieval objectives. Curated analytical derivatives may have shorter retention because they can be regenerated from locked raw data.

Ingestion at 10 PB scale. Online hospital feeds use DataSync when network connectivity and scheduling are feasible. Historical backfills use Snowball Edge jobs to avoid impractical transfer windows. All transfer paths write manifests, checksums, and reconciliation reports to S3 so missing studies can be identified before source systems are decommissioned.

Common Pitfall

Replicating data without replicating operations. A second-region bucket is not a disaster-recovery plan by itself. You must also replicate or recreate KMS access, IAM roles, Glue Catalog metadata, Lake Formation grants, monitoring, event routing, runbooks, and operator training.

1.8 Interview Q&A

1.8.1 Concepts and Trade-offs

- Q: Why is S3 a better foundation for a data lake than local disks attached to a cluster?**
A: S3 decouples durable storage from ephemeral compute, allows multiple engines to read the same data, and supports lifecycle, replication, encryption, and policy controls independently of a cluster.
- Q: When would you avoid One Zone-IA?**
A: Avoid it for the only copy of critical or regulated data because it stores data in one Availability Zone. It is better for re-creatable derivatives or secondary copies.
- Q: What does strong consistency solve, and what does it not solve?**
A: It ensures completed object writes and lists are visible immediately. It does not create multi-object transactions or repair failed jobs that produce partial datasets.
- Q: Governance mode or compliance mode for Object Lock?**
A: Governance mode allows authorized bypass and is useful when administrative exception processes are required. Compliance mode is stronger WORM retention and should be used only when the retention policy is certain.
- Q: DataSync or Snowball for 10 PB?**
A: Snowball or a specialized Snow Family approach is usually required for initial bulk transfer unless dedicated bandwidth is enormous. DataSync can handle ongoing deltas afterward.

1.9 Further Reading

Start with the Amazon S3 User Guide for bucket, object, lifecycle, encryption, event, and replication behavior [33]. Review AWS guidance on S3 storage classes before setting transitions [78], and study strong consistency semantics for pipeline correctness [2]. For retention design, read S3 Object Lock documentation [70]; for disaster-recovery copying, read S3 Replication documentation [73]. For migration planning, compare DataSync [43], Storage Gateway [58], and the Snow Family [56]. The AWS Well-Architected Analytics Lens gives broader review questions for reliability, operational excellence, security, performance, and cost [3]. For distributed-system background, Dynamo remains a classic paper on highly available storage design [88]; for the lakehouse direction of modern analytics platforms, see Armbrust et al. [85].

Key Takeaways

- S3 is the durable object-storage foundation for many AWS data engineering architectures, but it is not a POSIX file system.
- Storage-class choice must balance access frequency, retrieval latency, retrieval fees, minimum duration, resilience, and compliance.
- Strong consistency simplifies ingestion and listing logic, but multi-object correctness still requires idempotent jobs and table-aware commit patterns.
- Lifecycle rules control cost and disposal; versioning, Object Lock, legal hold, and replication control recovery and retention.
- DataSync, Storage Gateway, and Snow Family solve different migration problems: online sync, hybrid protocol bridge, and offline or edge transfer.
- A compliant disaster-recovery design includes data, keys, policies, metadata, monitoring, runbooks, and tested failover operations.

1.10 Exercises

1. **(Conceptual)** Explain the difference between durability and availability. Give one S3 design decision that improves each.
2. **(Conceptual)** A team wants all objects to transition to Glacier Deep Archive after seven days. List three questions you would ask before approving the rule.
3. **(Hands-on)** Run the Thursday lab in a sandbox account. Capture the bucket versioning status, lifecycle configuration, and one SQS event message as evidence.
4. **(Hands-on)** Modify the lifecycle rule so objects under `tmp/` expire after seven days while objects under `raw/` transition after 30 days. Validate with `aws s3api get-bucket-lifecycle-configuration`.
5. **(Design)** Design prefixes and lifecycle rules for a lake with `raw/`, `validated/`, `curated/`, and `sandbox/` zones. State retention, storage class, and deletion policy for each.
6. **(Design)** Choose DataSync, File Gateway, Snowball Edge, or a combination for a 600 TB on-premises radiology archive with a 500 Mbps link. Justify the migration timeline and cutover plan.
7. **(Governance)** Compare Object Lock governance mode, compliance mode, and legal hold for medical-image retention. Include the operational risk of choosing the wrong mode.

8. **(Architecture)** Extend fig. 1.2 with Athena, Redshift Spectrum, or SageMaker consumers. Describe how access would be governed without exposing raw images broadly.

1.11 Capstone Project: A Compliant Multi-Region Medical Image Data Lake on S3

Capstone Project

Mission. Design, implement, and verify a compliant multi-region S3 data lake for medical X-ray images. Your platform must ingest raw images, preserve immutable versions, encrypt data with customer-managed keys, transition objects through cost-optimized storage classes, replicate protected data to a disaster-recovery Region, and publish ingest events to Amazon SQS.

Estimated time. 4–6 hours in a sandbox AWS account, plus optional time for cost modeling and security review.

What you'll practice.

- Selecting S3 storage classes and lifecycle transitions for hot, warm, archive, and deep-archive medical-image data.
- Enabling versioning, S3 Object Lock in compliance mode, and legal holds for WORM retention.
- Configuring SSE-KMS encryption, least-privilege IAM, Cross-Region Replication, and S3 event notifications to SQS.
- Verifying the design with AWS CLI commands, lifecycle JSON, replication configuration, and a Python/Boto3 smoke test.

Scenario

A regional healthcare provider is consolidating 10 PB of historical and newly generated X-ray images into an S3-backed data lake. The source systems include hospital PACS archives, imaging-center uploads, radiology workflow applications, and periodic research exports. Raw files are treated as regulated medical records: they may contain protected health information, must be encrypted at rest, must be retained without tampering, and must remain recoverable after operator error or a regional service disruption.

The business also faces cost pressure. Keeping all 10 PB in S3 Standard forever is simple but wasteful once studies age out of active clinical workflows. At the same time, moving records too quickly into deep archive can violate retrieval expectations for patient care, audits, research reproducibility, or litigation. Your design must therefore combine durability, compliance, disaster recovery, observability, and lifecycle economics instead of optimizing only for the lowest storage price.

Assume the primary Region is `us-east-1`, the disaster-recovery Region is `us-west-2`, and the organization already has AWS CLI v2, Python, Boto3, and appropriate sandbox permissions. Use globally unique bucket names, separate customer-managed KMS keys per Region, and non-production retention values while testing. For a real compliance deployment, retention periods and legal-hold procedures must be approved by legal, security, privacy, and records-management teams.

Requirements

Functional requirements.

- Create a primary S3 bucket for raw X-ray images and a disaster-recovery bucket in a second Region.
- Enable versioning on both buckets because S3 Object Lock and replication depend on versioned object history.
- Enable S3 Object Lock on the primary bucket at creation time, configure default retention in compliance mode, and demonstrate a legal hold on at least one test object.
- Require server-side encryption with AWS KMS customer-managed keys for all objects. Use a primary-region key for the source bucket and a separate recovery-region key for the replica bucket.
- Apply lifecycle transitions for objects under `raw/xray/`: S3 Standard for active ingestion, S3 Standard-IA after 30 days, and S3 Glacier Deep Archive after 365 days. Include noncurrent-version transitions so overwritten versions do not remain indefinitely in hot storage.
- Configure Cross-Region Replication from the primary bucket to the disaster-recovery bucket, including encrypted objects and Object Lock metadata where applicable [73, 70].
- Configure S3 event notifications so every `s3:ObjectCreated:*` event under `raw/xray/` sends a message to an SQS queue.
- Provide a least-privilege IAM design for ingestion, verification, replication, security administration, and read-only audit roles.

Non-functional requirements.

- Durability must rely on S3 multi-AZ storage in each Region plus CRR for regional disaster recovery.
- Recovery objectives must be explicit: target a minutes-scale RPO for online ingestion, document that CRR is asynchronous, and define an RTO that includes KMS, IAM, catalog, application, and queue reconfiguration.
- Compliance controls must prevent destructive changes to protected object versions during the retention period. Compliance mode is intentionally irreversible until the retention date expires, so testing must use short-lived, non-production retention only.
- Access must follow least privilege. Ingestion can write but not delete protected records; audit can list and read metadata; replication can read source versions and write replicas; security administrators can manage policies without broad data access.
- Operations must include verification evidence: bucket versioning status, Object Lock configuration, lifecycle rules, replication status, KMS encryption, legal-hold status, and one SQS event message.
- Cost controls must explain why each lifecycle threshold exists and must identify retrieval-latency risk before using Glacier Deep Archive [78].

Reference Architecture

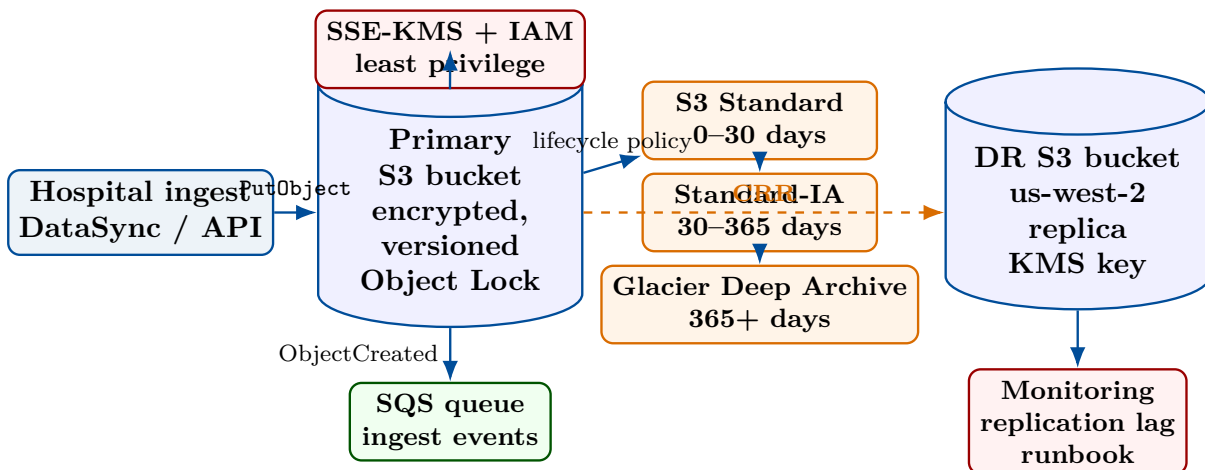


Figure 1.3: Capstone reference architecture for a compliant multi-region medical-image data lake on S3. The primary bucket receives encrypted, versioned, immutable objects; lifecycle rules manage cost; CRR creates a disaster-recovery copy; S3 event notifications feed SQS for downstream processing.

Implementation Steps

1. **Prepare names and Regions.** Choose globally unique bucket names and record your AWS account ID. In PowerShell, set variables for the primary Region, recovery Region, source bucket, destination bucket, queue name, retention days, and IAM role names. Keep all names free of patient identifiers.
2. **Create KMS keys.** Create one customer-managed KMS key in each Region. The source key must allow the ingestion and replication roles to encrypt and decrypt as required. The destination key must allow the replication role to encrypt replicas. In production, key policies should separate key administration from data use.
3. **Create Object-Lock-enabled buckets.** Object Lock must be enabled when the bucket is created. Enable versioning on both buckets immediately, then configure default retention on the primary bucket in compliance mode. Use a short test retention period in a sandbox.
4. **Apply encryption and lifecycle.** Configure default SSE-KMS encryption and apply lifecycle JSON for current and noncurrent versions under `raw/xray/`. The example transitions current objects to Standard-IA after 30 days and Glacier Deep Archive after 365 days, while moving noncurrent versions on their own schedule.
5. **Configure replication.** Create an IAM role trusted by S3, attach a scoped replication policy, and apply a replication configuration that handles encrypted objects. Verify that the destination bucket uses the recovery-region KMS key.
6. **Configure SQS ingest events.** Create an SQS queue, attach a queue policy allowing only the source bucket to send messages, and configure S3 event notifications for object-created events under `raw/xray/`.
7. **Ingest and verify.** Upload a sample object with KMS encryption, set a legal hold on that object version, confirm replication status, and consume the SQS notification. Record screenshots or command output as evidence.

```

1 # Run from Windows PowerShell with AWS CLI v2 configured.
2 $PrimaryRegion = "us-east-1"
3 $DrRegion = "us-west-2"
4 $AccountId = "123456789012"
5 $SourceBucket = "medical-xray-primary-123456789012"
6 $DestBucket = "medical-xray-dr-123456789012"
7 $QueueName = "medical-xray-ingest-events"
8 $ReplicationRoleName = "s3-medical-xray-replication-role"
9 $SourceKmsKeyArn = "arn:aws:kms:us-east-1:123456789012:key/source-key-id"
10 $DestKmsKeyArn = "arn:aws:kms:us-west-2:123456789012:key/dest-key-id"
11
12 aws s3api create-bucket '
13   --bucket $SourceBucket '
14   --region $PrimaryRegion '
15   --object-lock-enabled-for-bucket
16
17 aws s3api create-bucket '
18   --bucket $DestBucket '
19   --region $DrRegion '
20   --create-bucket-configuration LocationConstraint=$DrRegion '
21   --object-lock-enabled-for-bucket
22
23 aws s3api put-bucket-versioning '
24   --bucket $SourceBucket '
25   --versioning-configuration Status=Enabled
26
27 aws s3api put-bucket-versioning '
28   --bucket $DestBucket '
29   --region $DrRegion '
30   --versioning-configuration Status=Enabled
31
32 $ObjectLockJson = @"
33 {
34   "ObjectLockEnabled": "Enabled",
35   "Rule": {
36     "DefaultRetention": {
37       "Mode": "COMPLIANCE",
38       "Days": 30
39     }
40   }
41 }
42 "@
43 $ObjectLockJson | Set-Content -Path object-lock.json -Encoding ascii
44
45 aws s3api put-object-lock-configuration '
46   --bucket $SourceBucket '
47   --object-lock-configuration file://object-lock.json
48
49 $LifecycleJson = @"
50 {
51   "Rules": [
52     {
53       "ID": "raw-xray-standard-to-ia-to-deep-archive",
54       "Status": "Enabled",
55       "Filter": { "Prefix": "raw/xray/" },
56       "Transitions": [
57         { "Days": 30, "StorageClass": "STANDARD_IA" },
58         { "Days": 365, "StorageClass": "DEEP_ARCHIVE" }
59       ],
60       "NoncurrentVersionTransitions": [
61         { "NoncurrentDays": 30, "StorageClass": "STANDARD_IA" },
62         { "NoncurrentDays": 365, "StorageClass": "DEEP_ARCHIVE" }
63       ],

```



```

64     "AbortIncompleteMultipartUpload": { "DaysAfterInitiation": 7 }
65   }
66 ]
67 }
68 @"
69 $LifecycleJson | Set-Content -Path lifecycle-capstone.json -Encoding ascii
70
71 aws s3api put-bucket-lifecycle-configuration '
72   --bucket $SourceBucket '
73   --lifecycle-configuration file://lifecycle-capstone.json
74
75 $SourceEncryptionJson = @"
76 {
77   "Rules": [
78     {
79       "ApplyServerSideEncryptionByDefault": {
80         "SSEAlgorithm": "aws:kms",
81         "KMSEMasterKeyID": "$SourceKmsKeyArn"
82       },
83       "BucketKeyEnabled": true
84     }
85   ]
86 }
87 @"
88 $SourceEncryptionJson | Set-Content -Path source-encryption.json -Encoding
89   ascii
90
91 aws s3api put-bucket-encryption '
92   --bucket $SourceBucket '
93   --server-side-encryption-configuration file://source-encryption.json
94
95 $QueueUrl = aws sqs create-queue '
96   --queue-name $QueueName '
97   --region $PrimaryRegion '
98   --query QueueUrl '
99   --output text
100
101 $QueueArn = aws sqs get-queue-attributes '
102   --queue-url $QueueUrl '
103   --attribute-names QueueArn '
104   --query 'Attributes.QueueArn' '
105   --output text
106
107 $SqsPolicyJson = @"
108 {
109   "Version": "2012-10-17",
110   "Statement": [
111     {
112       "Sid": "AllowOnlyPrimaryBucketEvents",
113       "Effect": "Allow",
114       "Principal": { "Service": "s3.amazonaws.com" },
115       "Action": "sqs:SendMessage",
116       "Resource": "$QueueArn",
117       "Condition": {
118         "ArnLike": { "aws:SourceArn": "arn:aws:s3:::$SourceBucket" },
119         "StringEquals": { "aws:SourceAccount": "$AccountId" }
120       }
121     }
122   ]
123 }
124 @"
125 $SqsPolicyJson | Set-Content -Path sqs-policy-capstone.json -Encoding ascii

```



```

126 aws sqs set-queue-attributes '
127   --queue-url $QueueUrl '
128   --attributes Policy="$(Get-Content -Raw sqs-policy-capstone.json)"
129
130 $NotificationJson = @"
131 {
132   "QueueConfigurations": [
133     {
134       "Id": "raw-xray-object-created-to-sqs",
135       "QueueArn": "$QueueArn",
136       "Events": ["s3:ObjectCreated:*"],
137       "Filter": {
138         "Key": {
139           "FilterRules": [
140             { "Name": "prefix", "Value": "raw/xray/" }
141           ]
142         }
143       }
144     }
145   ]
146 }
147 "@
148 $NotificationJson | Set-Content -Path notification-capstone.json -Encoding
    ascii
149
150 aws s3api put-bucket-notification-configuration '
151   --bucket $SourceBucket '
152   --notification-configuration file://notification-capstone.json

```

Listing 1.5: PowerShell-oriented AWS CLI commands for the capstone S3 data lake. Replace every placeholder before running.

The replication policy is intentionally separated from the first listing because it is the easiest place to over-permission a role. The policy must allow the role to read source versions, read Object Lock metadata, decrypt source objects when required, replicate objects and tags, and encrypt destination replicas with the destination KMS key.

```

1 {
2   "Role": "arn:aws:iam::123456789012:role/s3-medical-xray-replication-role",
3   "Rules": [
4     {
5       "ID": "replicate-raw-xray-to-dr-region",
6       "Status": "Enabled",
7       "Priority": 1,
8       "DeleteMarkerReplication": { "Status": "Disabled" },
9       "Filter": { "Prefix": "raw/xray/" },
10      "SourceSelectionCriteria": {
11        "SseKmsEncryptedObjects": { "Status": "Enabled" }
12      },
13      "Destination": {
14        "Bucket": "arn:aws:s3:::medical-xray-dr-123456789012",
15        "StorageClass": "STANDARD",
16        "EncryptionConfiguration": {
17          "ReplicaKmsKeyID": "arn:aws:kms:us-west-2:123456789012:key/dest-key-
id"
18        },
19        "Metrics": {
20          "Status": "Enabled",
21          "EventThreshold": { "Minutes": 15 }
22        },
23        "ReplicationTime": {
24          "Status": "Enabled",
25          "Time": { "Minutes": 15 }

```

```

26     }
27     }
28 }
29 ]
30 }

```

Listing 1.6: Skeleton replication configuration for encrypted medical-image objects. Replace role and KMS values before use.

After creating and attaching the replication role policy, apply the replication configuration with the AWS CLI. Then run a Boto3 smoke test that uploads a sample object, records its version ID, applies a legal hold, verifies encryption and Object Lock metadata, checks for a replication status value, and reads the SQS event.

```

1  import json
2  import os
3  import time
4  from pathlib import Path
5
6  import boto3
7
8  primary_region = os.environ.get("PRIMARY_REGION", "us-east-1")
9  dr_region = os.environ.get("DR_REGION", "us-west-2")
10 source_bucket = os.environ["SOURCE_BUCKET"]
11 dest_bucket = os.environ["DEST_BUCKET"]
12 queue_url = os.environ["QUEUE_URL"]
13 kms_key_arn = os.environ["SOURCE_KMS_KEY_ARN"]
14
15 s3 = boto3.client("s3", region_name=primary_region)
16 s3_dr = boto3.client("s3", region_name=dr_region)
17 sqs = boto3.client("sqs", region_name=primary_region)
18
19 key = "raw/xray/capstone-study-0001.dcm"
20 body = b"synthetic DICOM placeholder for capstone validation\n"
21
22 put_response = s3.put_object(
23     Bucket=source_bucket,
24     Key=key,
25     Body=body,
26     ServerSideEncryption="aws:kms",
27     SSEKMSKeyId=kms_key_arn,
28     ContentType="application/dicom",
29     Metadata={"study-id": "capstone-study-0001", "modality": "xray"},
30 )
31 version_id = put_response["VersionId"]
32
33 s3.put_object_legal_hold(
34     Bucket=source_bucket,
35     Key=key,
36     VersionId=version_id,
37     LegalHold={"Status": "ON"},
38 )
39
40 head = s3.head_object(Bucket=source_bucket, Key=key, VersionId=version_id)
41 assert head["ServerSideEncryption"] == "aws:kms"
42 assert head["SSEKMSKeyId"] == kms_key_arn
43
44 lock = s3.get_object_legal_hold(
45     Bucket=source_bucket,
46     Key=key,
47     VersionId=version_id,
48 )
49 assert lock["LegalHold"]["Status"] == "ON"

```

```

50
51 versioning = s3.get_bucket_versioning(Bucket=source_bucket)
52 assert versioning.get("Status") == "Enabled"
53
54 object_lock = s3.get_object_lock_configuration(Bucket=source_bucket)
55 mode = object_lock["ObjectLockConfiguration"]["Rule"]["DefaultRetention"]["Mode"]
56 assert mode == "COMPLIANCE"
57
58 for attempt in range(12):
59     replica = s3.head_object(Bucket=source_bucket, Key=key, VersionId=
        version_id)
60     status = replica.get("ReplicationStatus", "PENDING")
61     if status in {"COMPLETED", "FAILED", "REPLICA"}:
62         print(f"replication status: {status}")
63         break
64     time.sleep(10)
65 else:
66     print("replication status still pending after smoke-test wait")
67
68 try:
69     s3_dr.head_object(Bucket=dest_bucket, Key=key)
70     print("destination object is visible")
71 except s3_dr.exceptions.ClientError as exc:
72     print(f"destination check not complete yet: {exc.response['Error']['Code']}")
73
74 messages = sqs.receive_message(
75     QueueUrl=queue_url,
76     MaxNumberOfMessages=1,
77     WaitTimeSeconds=10,
78 ).get("Messages", [])
79 if not messages:
80     raise RuntimeError("No S3 ObjectCreated notification arrived in SQS")
81
82 payload = json.loads(messages[0]["Body"])
83 record = payload["Records"][0]
84 assert record["s3"]["bucket"]["name"] == source_bucket
85 assert record["s3"]["object"]["key"] == key
86 print("verified event:", record["eventName"], record["s3"]["object"]["key"])
87
88 Path("capstone-verification.json").write_text(
89     json.dumps(
90         {
91             "bucket": source_bucket,
92             "key": key,
93             "version_id": version_id,
94             "legal_hold": "ON",
95             "object_lock_mode": mode,
96             "event_name": record["eventName"],
97         },
98         indent=2,
99     ),
100     encoding="utf-8",
101 )

```

Listing 1.7: Boto3 ingestion and verification snippet for the capstone.

Deliverables

Submit a concise engineering packet containing the following artifacts:

- A one-page architecture summary with primary Region, disaster-recovery Region, bucket

names, KMS-key strategy, RPO, RTO, and lifecycle thresholds.

- The final lifecycle JSON, Object Lock configuration, replication configuration, SQS queue policy, and S3 notification configuration.
- IAM policy excerpts for ingestion, replication, audit, and security administration roles, with a short least-privilege explanation for each role.
- Command output or screenshots showing versioning enabled, Object Lock compliance retention configured, legal hold enabled on one object version, lifecycle rules present, and default SSE-KMS encryption active.
- Evidence of successful ingestion: object key, version ID, KMS key ID, metadata, SQS event message, and replication status or replication-lag observation.
- A cleanup plan for sandbox resources, including queues, IAM roles, temporary JSON files, test objects after retention expires, and KMS key deletion scheduling where appropriate.

Acceptance Criteria

Use this checklist to self-grade the capstone before submitting:

- The source bucket was created with Object Lock support and has versioning enabled.
- The destination bucket is in a different Region and has versioning enabled before replication is configured.
- Default bucket encryption uses SSE-KMS with a customer-managed key, not unencrypted objects or ad hoc client-side encryption.
- Object Lock default retention is configured in compliance mode for the test prefix or bucket, and the submission explains why production retention values require formal approval.
- At least one uploaded object version has legal hold enabled and the evidence includes its version ID.
- Lifecycle rules transition current versions from S3 Standard to Standard-IA and then to Glacier Deep Archive, with noncurrent-version handling and incomplete multipart upload cleanup.
- CRR is configured for `raw/xray/`, includes encrypted-object handling, and uses a destination KMS key in the recovery Region.
- S3 event notifications send `s3:ObjectCreated:*` events for `raw/xray/` to SQS, and the queue policy restricts sends to the source bucket and account.
- IAM policies are scoped by bucket ARN, prefix, KMS key, and role purpose. Broad administrative wildcards are justified only for temporary sandbox setup roles, not steady-state data access.
- Verification evidence includes AWS CLI or Boto3 output for versioning, lifecycle, Object Lock, legal hold, encryption, SQS event receipt, and replication status.
- The architecture explains retrieval-latency trade-offs for Standard-IA and Glacier Deep Archive instead of treating lifecycle as a pure cost-saving feature.
- The disaster-recovery plan covers data, keys, permissions, monitoring, catalog dependencies, application cutover, and post-failover reconciliation.

Stretch Goals

- Use S3 Batch Operations to apply tags, legal holds, or retention settings to a manifest of historical objects after validating the workflow on a small sample.
- Run Amazon Macie discovery jobs on a controlled prefix to identify possible protected health information or other sensitive fields in metadata exports before broad analytics access is granted.
- Build a cost model for 10 PB across S3 Standard, Standard-IA, Glacier Instant Retrieval, Glacier Flexible Retrieval, and Glacier Deep Archive. Include request charges, retrieval charges, minimum storage duration, replication storage, KMS requests, and data transfer assumptions.
- Enable S3 Storage Lens and define dashboard metrics for object count, bytes by storage class, incomplete multipart uploads, noncurrent-version growth, encrypted-object coverage, and replication health.
- Add EventBridge or Lambda processing downstream of SQS to write an immutable ingest manifest containing object key, version ID, checksum, KMS key ID, study metadata, and replication status.
- Prototype a failover drill: disable writes to the primary application path, validate replica availability in the recovery Region, switch a read-only consumer to the DR bucket, and document the reconciliation steps required before failing back.

Relational Databases: RDS and Aurora

Learning Objectives

- Explain the managed relational database model behind Amazon RDS, including engines, instance classes, storage, backups, maintenance windows, and operational responsibility boundaries.
- Design highly available RDS deployments with Multi-AZ DB instances or Multi-AZ DB clusters, and distinguish failover capacity from horizontal read scaling.
- Compare RDS Read Replicas, Aurora Replicas, Aurora Global Database, and cross-region replica patterns for reporting, disaster recovery, and low-latency reads.
- Describe Aurora's cloud-native distributed storage architecture, quorum writes, cache-aware failover, Global Database replication, and Aurora Serverless v2 scaling model.
- Use AWS Database Migration Service and AWS Schema Conversion Tool to plan homogeneous and heterogeneous database migrations with full-load and change data capture phases.
- Deploy an Aurora Serverless v2 PostgreSQL cluster, add cross-region read capacity, simulate regional failover, and design a zero-downtime Oracle ERP cutover to Aurora PostgreSQL.

Cross-Cloud Equivalence

Managed relational platforms use different product names, but the architectural choices are familiar: engine compatibility, high availability, read scale, global reach, migration tooling, and operational automation.

Capability	AWS	Azure	GCP
Managed relational	RDS / Aurora	Azure SQL / Azure Database for PostgreSQL Hyperscale	Cloud SQL / AlloyDB
Cloud-native scale-out	Aurora		AlloyDB
High availability	Multi-AZ	Zone-redundant HA / auto-failover groups	Regional HA
Read replicas	RDS Read Replicas / Aurora Replicas	Read replicas	Read replicas
Global or geo-replication	Aurora Global Database	Active geo-replication	Cross-region replicas
Migration	DMS + SCT	Azure DMS	Database Migration Service
Operational automation	Backups, patching, monitoring, Performance Insights	Automated backups, maintenance, Query Performance Insight	Backups, maintenance, Query Insights

Adjacent services follow the same mental model. MongoDB Atlas and Amazon DocumentDB are managed document-database analogs; Microsoft Fabric and BigQuery are lakehouse or warehouse platforms rather than OLTP systems; Snowflake separates compute and storage for analytical workloads, while RDS and Aurora optimize transactional consistency and operational database availability.

2.1 Week 2 Roadmap and Mental Model

Relational databases remain central to data engineering because most enterprise facts begin as transactions: orders, invoices, shipments, journal entries, patient encounters, payments, and inventory movements. A data engineer must understand not only how to extract those records, but also how the source database preserves consistency, scales reads, fails over, and migrates safely. Week 2 therefore focuses on managed relational infrastructure: Amazon RDS for familiar engines, Amazon Aurora for cloud-native relational scale, and AWS DMS plus AWS SCT for migration and replication workflows [26, 7, 42].

Amazon RDS removes much of the undifferentiated database administration work: provisioning, backups, patching, monitoring hooks, storage management, and high-availability automation. It does not remove schema design, transaction design, index selection, query tuning, capacity planning, security boundaries, or migration accountability. Aurora goes further by redesigning the storage layer for distributed cloud infrastructure, but applications still experience it through MySQL-compatible or PostgreSQL-compatible SQL endpoints.

Definition

Amazon RDS is a managed relational database service for engines such as PostgreSQL, MySQL, MariaDB, Oracle, SQL Server, and Db2. **Amazon Aurora** is an AWS-designed, MySQL-compatible and PostgreSQL-compatible relational database that separates compute from a distributed storage volume.

Key Terms

Primary instance: the writer for a database. **Standby:** synchronous failover target in a Multi-AZ deployment. **Read replica:** asynchronous read-scaling copy. **Cluster endpoint:** Aurora endpoint that tracks the current writer. **CDC:** change data capture used to keep a migration target current.

2.2 Monday: Amazon RDS Architecture, Multi-AZ, and Read Replicas

2.2.1 Managed RDS Architecture

An RDS database is a managed combination of engine software, compute, storage, networking, backup automation, monitoring, and control-plane operations. You choose an engine, version, instance class, allocated storage, subnet group, security groups, parameter groups, option groups, backup retention, maintenance window, encryption configuration, and high-availability topology. AWS operates the underlying infrastructure and service automation; the customer owns data model, credentials, network exposure, query behavior, and application compatibility.

For a data engineering team, the most important RDS design boundary is the split between transactional workloads and analytical workloads. RDS can support operational reporting, but it is not a replacement for S3 data lakes, Redshift, Athena, or specialized analytical stores. Extracting from production RDS requires workload isolation, replica usage where appropriate, and a clear understanding of transaction logs, snapshots, and business cutover windows.

Note

RDS automation is not the same as database tuning. A poorly indexed query, chatty transaction pattern, or long-running report can still saturate CPU, memory, locks, connections, IOPS, or log volume on a fully managed database.

2.2.2 Backups, Maintenance, and Observability

RDS automated backups combine daily snapshots with transaction logs so a database can be restored to a point within the retention period. Manual snapshots persist until deleted. Maintenance windows let AWS apply engine or OS updates; some changes are online, while others require a reboot or failover. CloudWatch metrics, Enhanced Monitoring, Performance Insights, database logs, and events are the operational evidence used to decide whether a system is healthy.

Tip

For production databases, set backup retention, deletion protection, encryption, monitoring, and maintenance windows intentionally at creation time. Retrofitting these controls during an incident is slower and riskier than declaring them in infrastructure as code.

2.2.3 Multi-AZ Deployments

Multi-AZ is the core RDS high-availability pattern. In a traditional Multi-AZ DB instance deployment, RDS synchronously replicates data to a standby in a different Availability Zone and automatically fails over when the primary becomes unavailable. The standby is not used for read traffic. Newer Multi-AZ DB clusters provide a writer and readable standby instances with lower commit latency and faster failover for supported engines [66].

Definition

RDS Multi-AZ is a high-availability configuration that maintains a synchronous standby in another Availability Zone and uses automatic failover to preserve database availability during infrastructure, instance, or maintenance events.

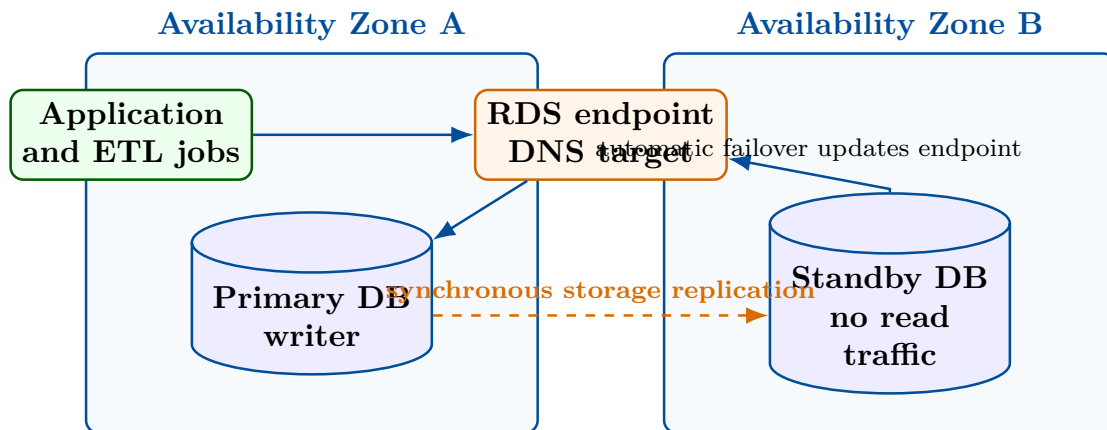


Figure 2.1: RDS Multi-AZ protects availability by synchronously maintaining a standby in another Availability Zone and moving the endpoint during failover. It is not a read-scaling feature in the traditional DB instance model.

Common Pitfall

Confusing Multi-AZ with read scaling. Multi-AZ is primarily for availability and durability of service during failures or maintenance. Use Read Replicas, Aurora Replicas, caching, or analytical offload when the problem is read throughput.

2.2.4 Read Replicas

RDS Read Replicas use asynchronous replication to offload read-heavy workloads, support reporting, test upgrades, or create cross-region disaster-recovery candidates for supported engines [79]. Because replication is asynchronous, replicas can lag. Applications must tolerate stale reads, and data engineers must not point critical reconciliation jobs at a lagging replica without checking freshness.

Pattern	Best fit	Design caution
Single-AZ RDS	Development, tests, noncritical workloads	No AZ-level database failover target
Multi-AZ DB instance	Production OLTP availability	Standby is not a reporting target
Multi-AZ DB cluster	Production engines needing faster failover and readable standbys	Engine and feature support differ from classic deployments
Read Replica	Read scaling, reporting, migration staging	Asynchronous lag and replica-specific capacity planning
Cross-region replica	Regional DR candidate or local reads	Higher cost, lag, egress, and failover runbook complexity

Table 2.1: RDS deployment patterns mapped to common data engineering decisions.

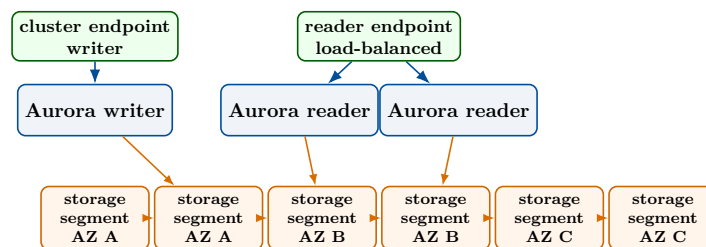
2.3 Tuesday: Amazon Aurora Cloud-Native Relational Architecture

2.3.1 Distributed Storage and Compute Separation

Aurora keeps the SQL engine familiar while changing the storage architecture. An Aurora cluster has one writer instance, zero or more reader instances, and a distributed cluster volume that automatically grows as data increases. The storage layer replicates data across multiple Availability Zones, and database instances attach to that shared volume rather than each owning a complete independent storage copy. This separation enables fast replica creation, faster failover, and storage durability that is not tied to one compute node [6].

Definition

Aurora cluster volume is a distributed, replicated storage volume shared by the DB instances in an Aurora cluster. Compute nodes can fail or be replaced while the storage service preserves the database volume independently.



The storage volume is replicated across Availability Zones; readers share the same logical volume rather than receiving separate full database copies.

Figure 2.2: Aurora separates compute instances from a distributed cluster volume. Writer and reader endpoints route application traffic while the storage service handles replication and durability.

Note

Aurora replicas can serve reads and participate in failover promotion, but they still share the cluster's operational fate more closely than independent analytical systems. Heavy reporting can still affect cache, connections, locks, and engine resources.

2.3.2 Aurora Global Database

Aurora Global Database replicates an Aurora cluster to secondary Regions using storage-level replication designed for low-latency cross-region propagation [76]. The primary Region accepts writes. Secondary Regions can serve low-latency reads and can be promoted during a regional disaster. A global database improves recovery posture, but it requires a failover plan for application endpoints, secrets, DNS, parameter differences, monitoring, and downstream integrations.

Tip

Write down the difference between *planned switchover* and *unplanned failover*. A planned switchover can coordinate writes and reduce data loss. An unplanned failover must prioritize recovery time while accepting that asynchronous replication may have an RPO window.

2.3.3 Aurora Serverless v2

Aurora Serverless v2 scales database capacity in Aurora Capacity Units without managing fixed DB instance sizes [77]. It is useful for variable workloads, development and testing, SaaS tenants with uneven activity, and operational systems that need finer-grained scaling than scheduled instance changes. Unlike a stateless compute service, database scaling still interacts with connections, transactions, locks, memory pressure, and query plans.

Common Pitfall

Assuming serverless means unlimited. Aurora Serverless v2 scales within configured minimum and maximum ACUs. Connection storms, unbounded queries, missing indexes, and long transactions can still produce throttling, poor latency, or cost surprises.

2.4 Wednesday: AWS DMS and Schema Conversion Tool

2.4.1 Migration Phases

AWS Database Migration Service moves data from supported sources to supported targets with full-load, change data capture, or combined full-load-plus-CDC tasks [42]. Homogeneous migrations, such as PostgreSQL to PostgreSQL, can often use native schema tools plus DMS for replication. Heterogeneous migrations, such as Oracle to Aurora PostgreSQL, require schema assessment, code conversion, data-type mapping, and extensive application testing. AWS Schema Conversion Tool analyzes and converts schema objects where possible, then flags objects that require manual work [54].

Definition

Full load copies existing rows from the source to the target. **Change data capture** reads ongoing changes from database logs and applies them to the target so the migration can approach cutover with a small lag window.

2.4.2 Replication Instances, Endpoints, and Tasks

A DMS migration uses a replication instance or serverless replication configuration, source and target endpoints, table mappings, task settings, and validation options. The replication resource must reach both databases over the network and must have credentials with enough rights to read source logs and apply target changes. For Oracle sources, supplemental logging, archive log retention, LOB handling, and privileges are frequent planning items.

Note

DMS is a migration and replication service, not a magic compatibility layer. Stored procedures, triggers, packages, optimizer hints, scheduler jobs, security models, and application SQL must be assessed separately.

2.4.3 Validation and Cutover

A zero-downtime or near-zero-downtime migration depends on controlling writes. The usual pattern is: assess schema, provision target, convert schema, run full load, keep CDC running, validate row counts and checksums, rehearse cutover, freeze or dual-write source transactions, let lag reach an approved threshold, switch applications, and keep rollback evidence until the business accepts the new system. DMS validation helps, but business reconciliation remains essential [59].

Tip

Always define who can approve cutover and rollback before the migration weekend. Technical success is insufficient if finance, supply chain, or operations teams cannot reconcile critical balances after the switch.

2.5 Thursday Hands-on Project: Aurora Serverless v2 and Regional Failover

This lab provisions an Aurora PostgreSQL Serverless v2 cluster, creates a secondary-region read path using Aurora Global Database, and rehearses a failover workflow. Replace every account-specific value before running. The commands are written for Windows PowerShell using AWS CLI v2; line continuations use backticks.

Note

The lab creates billable database resources. Use a sandbox account, small ACU ranges, restrictive security groups, and short-lived clusters. Clean up global clusters, DB clusters, DB instances, subnet groups, and secrets after testing.

```

1 # Run from Windows PowerShell with AWS CLI v2 configured.
2 $PrimaryRegion = "us-east-1"
3 $SecondaryRegion = "us-west-2"
4 $GlobalClusterId = "erp-aurora-global-demo"
5 $PrimaryClusterId = "erp-aurora-primary-demo"
6 $SecondaryClusterId = "erp-aurora-secondary-demo"
7 $DbName = "erpdb"
8 $MasterUser = "erp_admin"
9 $PrimarySubnetGroup = "aurora-private-us-east-1"
10 $SecondarySubnetGroup = "aurora-private-us-west-2"
11 $PrimarySg = "sg-0123456789abcdef0"
12 $SecondarySg = "sg-0fedcba9876543210"
13
14 aws rds create-global-cluster `
15     --global-cluster-identifier $GlobalClusterId `
16     --engine aurora-postgresql `
17     --engine-version 16.4 `
18     --region $PrimaryRegion
19
20 aws rds create-db-cluster `
21     --db-cluster-identifier $PrimaryClusterId `
22     --global-cluster-identifier $GlobalClusterId `
23     --engine aurora-postgresql `
24     --engine-version 16.4 `
25     --database-name $DbName `
26     --master-username $MasterUser `
27     --manage-master-user-password `
28     --serverless-v2-scaling-configuration MinCapacity=0.5,MaxCapacity=4 `
29     --db-subnet-group-name $PrimarySubnetGroup `
30     --vpc-security-group-ids $PrimarySg `
31     --backup-retention-period 7 `
32     --deletion-protection `
33     --region $PrimaryRegion
34
35 aws rds create-db-instance `
36     --db-instance-identifier "$PrimaryClusterId-writer-1" `
37     --db-cluster-identifier $PrimaryClusterId `
38     --engine aurora-postgresql `
39     --db-instance-class db.serverless `

```

```
40 --region $PrimaryRegion
```

Listing 2.1: AWS CLI commands to create an Aurora PostgreSQL Serverless v2 cluster and attach it to a global database.

```
1 aws rds wait db-cluster-available '
2   --db-cluster-identifier $PrimaryClusterId '
3   --region $PrimaryRegion
4
5 aws rds create-db-cluster '
6   --db-cluster-identifier $SecondaryClusterId '
7   --global-cluster-identifier $GlobalClusterId '
8   --engine aurora-postgresql '
9   --engine-version 16.4 '
10  --serverless-v2-scaling-configuration MinCapacity=0.5,MaxCapacity=4 '
11  --db-subnet-group-name $SecondarySubnetGroup '
12  --vpc-security-group-ids $SecondarySg '
13  --region $SecondaryRegion
14
15 aws rds create-db-instance '
16   --db-instance-identifier "$SecondaryClusterId-reader-1" '
17   --db-cluster-identifier $SecondaryClusterId '
18   --engine aurora-postgresql '
19   --db-instance-class db.serverless '
20   --region $SecondaryRegion
21
22 aws rds describe-global-clusters '
23   --global-cluster-identifier $GlobalClusterId '
24   --query "GlobalClusters[0].GlobalClusterMembers[*].[DBClusterArn,IsWriter]" '
25   --output table '
26   --region $PrimaryRegion
27
28 # Planned switchover is preferred for drills because it coordinates roles.
29 aws rds switchover-global-cluster '
30   --global-cluster-identifier $GlobalClusterId '
31   --target-db-cluster-identifier "arn:aws:rds:us-west-2:123456789012:cluster:
32     erp-aurora-secondary-demo" '
33   --region $PrimaryRegion
```

Listing 2.2: AWS CLI commands to add a secondary Aurora Global Database cluster and rehearse failover.

2.5.1 Boto3 Verification

The Boto3 snippet verifies that the global cluster has one writer, collects endpoints, and records failover evidence. In a real test, pair this with SQL-level checks using a PostgreSQL client from a host inside the VPC.

```
1 import json
2 import os
3 from pathlib import Path
4
5 import boto3
6
7 primary_region = os.environ.get("PRIMARY_REGION", "us-east-1")
8 secondary_region = os.environ.get("SECONDARY_REGION", "us-west-2")
9 global_cluster_id = os.environ["GLOBAL_CLUSTER_ID"]
10 primary_cluster_id = os.environ["PRIMARY_CLUSTER_ID"]
11 secondary_cluster_id = os.environ["SECONDARY_CLUSTER_ID"]
12
13 rds_primary = boto3.client("rds", region_name=primary_region)
14 rds_secondary = boto3.client("rds", region_name=secondary_region)
```

```

15
16 global_cluster = rds_primary.describe_global_clusters(
17     GlobalClusterIdentifier=global_cluster_id
18 )["GlobalClusters"][0]
19 members = global_cluster["GlobalClusterMembers"]
20 writers = [member for member in members if member["IsWriter"]]
21 assert len(writers) == 1, writers
22
23 primary = rds_primary.describe_db_clusters(
24     DBClusterIdentifier=primary_cluster_id
25 )["DBClusters"][0]
26 secondary = rds_secondary.describe_db_clusters(
27     DBClusterIdentifier=secondary_cluster_id
28 )["DBClusters"][0]
29
30 evidence = {
31     "global_cluster": global_cluster_id,
32     "writer_cluster_arn": writers[0]["DBClusterArn"],
33     "primary_endpoint": primary.get("Endpoint"),
34     "primary_reader_endpoint": primary.get("ReaderEndpoint"),
35     "secondary_endpoint": secondary.get("Endpoint"),
36     "secondary_reader_endpoint": secondary.get("ReaderEndpoint"),
37     "members": [
38         {"arn": member["DBClusterArn"], "is_writer": member["IsWriter"]}
39         for member in members
40     ],
41 }
42
43 Path("chapter2-aurora-global-evidence.json").write_text(
44     json.dumps(evidence, indent=2), encoding="utf-8"
45 )
46 print(json.dumps(evidence, indent=2))

```

Listing 2.3: Boto3 verification for Aurora Global Database membership and endpoints.

Common Pitfall

Failing over only the database. A regional failover also needs secrets, DNS, application configuration, security groups, IAM, monitoring alarms, DMS tasks, job schedulers, and downstream consumers to point at the promoted Region.

2.6 Friday Use Case: Zero-Downtime Oracle ERP Migration to Aurora PostgreSQL

A manufacturing company runs a monolithic on-premises Oracle ERP that supports purchasing, inventory, order management, finance, and shop-floor execution. The database is expensive to operate, hard to scale for reporting, and tightly coupled to legacy stored procedures. The target state is Aurora PostgreSQL for transactional workloads, S3 and Redshift for analytics, and DMS-based replication during the migration window.

2.6.1 Architecture Narrative

The migration begins with discovery. SCT assesses schemas, packages, triggers, sequences, materialized views, synonyms, partitioning, data types, and SQL constructs. Objects that convert cleanly become Aurora PostgreSQL DDL. Objects that do not convert become remediation backlog for database developers and application teams. Meanwhile, the platform team provisions Aurora PostgreSQL with Multi-AZ storage, at least one reader, encryption, backups, parameter groups, monitoring, and a cross-region recovery pattern.

DMS then performs a full load from Oracle to Aurora PostgreSQL and continues CDC from Oracle redo or archive logs. During rehearsals, teams compare table counts, financial balances, open orders, inventory quantities, and application smoke tests. The final cutover freezes writes or routes them through a controlled dual-write layer, waits for DMS latency to reach the approved threshold, validates business-critical queries, switches application endpoints, and keeps Oracle read-only until rollback risk has passed.

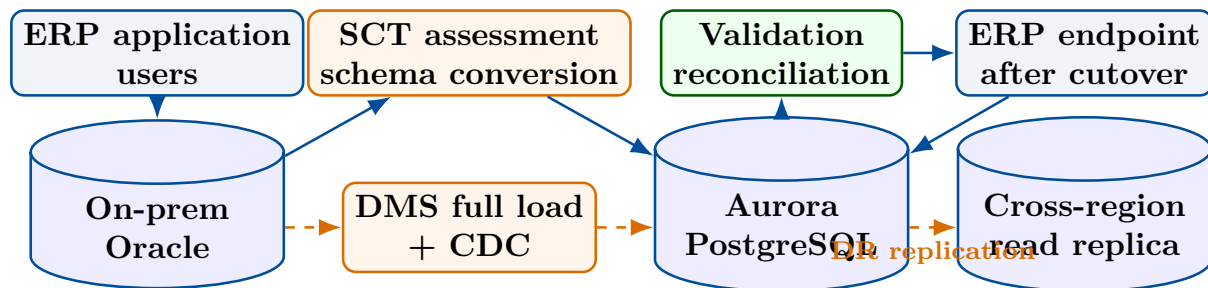


Figure 2.3: Zero-downtime Oracle ERP migration pattern using SCT for schema conversion, DMS for full load and CDC, validation for business reconciliation, and Aurora PostgreSQL with cross-region recovery for the target.

2.6.2 Operational Decisions

Source readiness. Oracle supplemental logging, archive log retention, network connectivity, and read privileges must be in place before DMS starts. The source must keep enough logs to survive DMS pauses without losing CDC continuity.

Target readiness. Aurora PostgreSQL parameter groups, extensions, schemas, roles, vacuum behavior, indexes, and connection pooling must be tested under realistic ERP concurrency. Load indexes strategically; sometimes a full load is faster before building selected secondary indexes, but constraints required for correctness cannot be skipped casually.

Cutover control. The business needs a cutover commander, rollback owner, reconciliation owners, and a communication plan. Technical DMS latency is only one metric; financial totals, order status, inventory balances, and integration queues decide whether the ERP is ready.

Common Pitfall

Underestimating stored procedures and batch jobs. Oracle ERP systems often contain business logic in packages, triggers, scheduler jobs, and ad hoc reports. Schema conversion without application and batch remediation is not a complete migration.

2.7 Interview Q&A

2.7.1 Concepts and Trade-offs

1. **Q: What is the difference between RDS Multi-AZ and a Read Replica?**

A: Multi-AZ provides synchronous standby capacity for high availability and failover. A Read Replica uses asynchronous replication for read scaling, reporting, or migration staging and can lag behind the writer.

2. **Q: Why can Aurora fail over faster than many traditional databases?**

A: Aurora separates compute from a distributed cluster volume. A replica can be promoted

without copying an entire database volume because the storage layer is already shared and replicated.

3. **Q: When would you use Aurora Global Database?**

A: Use it when a workload needs low-latency cross-region reads or a stronger regional disaster-recovery posture than backups alone. It still requires application and operations failover planning.

4. **Q: What does Aurora Serverless v2 solve, and what does it not solve?**

A: It provides fine-grained capacity scaling for variable workloads. It does not eliminate schema tuning, connection management, transaction design, or maximum-capacity planning.

5. **Q: What is the role of SCT in an Oracle to Aurora PostgreSQL migration?**

A: SCT assesses and converts schema objects where possible and identifies manual remediation for incompatible SQL, procedural code, data types, and database features.

6. **Q: Why is CDC important for near-zero-downtime migration?**

A: CDC keeps the target current while the source remains online after the initial full load. Cutover can then happen after replication lag and business validation reach approved thresholds.

2.8 Further Reading

Start with the Amazon RDS User Guide for instance, backup, monitoring, security, Multi-AZ, and replica behavior [26]. Review RDS Multi-AZ and Read Replica documentation before promising RTO or read-scaling outcomes [66, 79]. Study the Aurora User Guide, especially storage reliability, endpoints, Global Database, and Serverless v2 capacity scaling [7, 6, 76, 77]. For migrations, read the AWS DMS User Guide, AWS SCT User Guide, and DMS best-practices guidance before designing an Oracle ERP cutover [42, 54, 59].

Key Takeaways

- RDS is managed infrastructure for familiar relational engines, but customers still own schema design, query behavior, access control, and workload isolation.
- Multi-AZ protects database availability; Read Replicas and Aurora Replicas address read scaling and can introduce asynchronous lag.
- Aurora separates compute from a distributed storage layer, enabling fast replica operations, failover, and cloud-native durability patterns.
- Aurora Global Database supports cross-region reads and disaster recovery, but application failover, secrets, networking, and monitoring must be rehearsed.
- Aurora Serverless v2 scales database capacity within configured ACU limits; it is not a substitute for indexing, pooling, and transaction discipline.
- DMS and SCT can reduce migration downtime and conversion effort, but heterogeneous ERP migrations still require business reconciliation and application remediation.

2.9 Exercises

1. **(Conceptual)** Explain why a traditional RDS Multi-AZ standby is not used for reporting traffic. What problem does it solve instead?

2. **(Conceptual)** Compare RDS Read Replicas and Aurora Replicas. Include replication behavior, failover use, and reporting risk.
3. **(Hands-on)** Use AWS CLI in a sandbox to describe an existing RDS or Aurora cluster and identify engine version, backup retention, deletion protection, endpoints, and Multi-AZ or replica status.
4. **(Hands-on)** Modify listing 2.3 to write CloudWatch-style metrics for global-cluster writer Region and replica count.
5. **(Design)** Design an RDS PostgreSQL deployment for a billing system with strict RTO requirements and moderate reporting load. Choose Multi-AZ, replicas, backups, and monitoring.
6. **(Migration)** List the prerequisites for DMS CDC from Oracle to Aurora PostgreSQL. Include source logging, privileges, network, LOB handling, and validation.
7. **(Governance)** Create a cutover checklist for an ERP migration that includes technical, financial, security, and business sign-offs.
8. **(Architecture)** Extend fig. 2.3 with S3, Glue, and Redshift for analytics offload. Explain how you avoid reporting queries against the OLTP writer.

2.10 Capstone Project: Migrating and Operating a Cross-Region Aurora PostgreSQL Fleet for Oracle ERP Cutover

Capstone Project

Mission. Migrate and operate a highly available, cross-region Aurora PostgreSQL fleet for an Oracle ERP cutover. Your platform must support schema conversion, full-load migration, CDC replication, validation, planned cutover, regional failover, and post-cutover operations.

Estimated time. 6–8 hours in a sandbox AWS account for infrastructure and rehearsal artifacts, plus optional time for application SQL remediation and performance testing.

What you'll practice.

- Translating Oracle ERP migration requirements into Aurora PostgreSQL capacity, HA, backup, monitoring, security, and cross-region recovery decisions.
- Using SCT assessment outputs and DMS full-load-plus-CDC tasks to reduce downtime during heterogeneous migration.
- Verifying Aurora Global Database or cross-region read-replica readiness, failover evidence, replication lag, and application endpoint cutover.
- Producing business-facing acceptance criteria for reconciliation, rollback, and operational handoff.

Scenario

A manufacturing enterprise is replacing an on-premises Oracle ERP database with Aurora PostgreSQL. The ERP supports procurement, inventory, manufacturing execution, sales orders, shipping, accounts payable, and general ledger. Downtime must be minimized because warehouses and production lines operate across time zones. The source database contains decades of transactional history, Oracle packages, triggers, sequences, materialized views, and nightly batch jobs.

The target platform must run in a primary AWS Region with high availability across Availability

Zones and a secondary Region for disaster recovery. The migration team must prove that DMS can keep the target current, that SCT conversion gaps are remediated or accepted, that critical business totals reconcile, and that operators can promote the secondary Region if the primary Region is unavailable.

Requirements

Functional requirements.

- Produce an SCT assessment summary that classifies schema objects as automatically converted, manually remediated, retired, or deferred.
- Provision Aurora PostgreSQL with encryption, backup retention, deletion protection, at least one reader, Performance Insights, database logs, and restricted network access.
- Configure cross-region recovery using Aurora Global Database or a documented cross-region replica pattern with explicit RPO and RTO targets.
- Create DMS source and target endpoints, table mappings, and a full-load-plus-CDC task for ERP schemas selected for the cutover rehearsal.
- Validate row counts, sample checksums, sequence alignment, referential integrity, and business totals for orders, inventory, invoices, and ledger balances.
- Define cutover steps that freeze or control writes, drain replication lag, switch application endpoints, verify smoke tests, and preserve rollback evidence.

Non-functional requirements.

- The target database must be highly available within the primary Region and recoverable in the secondary Region.
- Secrets must be stored outside scripts, preferably in AWS Secrets Manager, and database access must follow least privilege.
- DMS replication lag, task errors, table errors, database CPU, memory, storage, connections, deadlocks, and slow queries must be observable.
- The migration must have a named cutover owner, rollback owner, validation owners, and business sign-off process.
- The final design must avoid analytical reporting against the Aurora writer by using replicas, S3 exports, or downstream analytical stores.

Reference Architecture

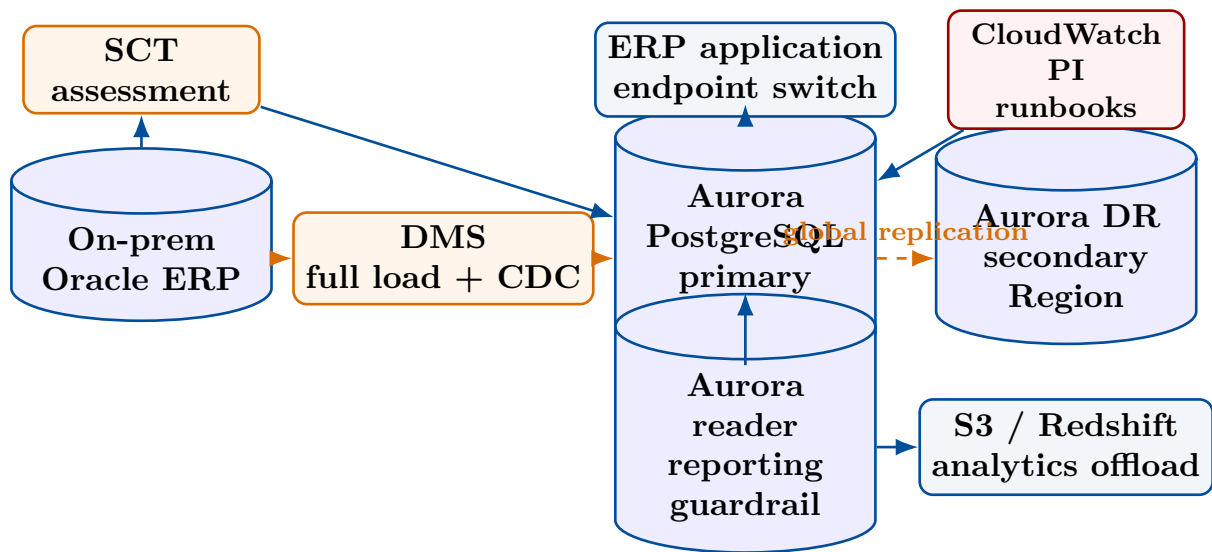


Figure 2.4: Capstone reference architecture for an Oracle ERP migration to Aurora PostgreSQL with DMS CDC, SCT conversion, high availability, cross-region recovery, observability, and analytics offload.

Implementation Steps

1. **Assess and classify.** Run SCT against the Oracle source, export the assessment, and classify incompatible objects by business criticality and remediation owner.
2. **Provision Aurora.** Create the Aurora PostgreSQL cluster, readers, subnet groups, security groups, parameter groups, Secrets Manager entries, backups, logs, Performance Insights, and deletion protection.
3. **Configure regional recovery.** Create a global database or cross-region replica, document RPO and RTO, and run a planned switchover or failover rehearsal.
4. **Create DMS migration tasks.** Configure source and target endpoints, table mappings, full-load-plus-CDC settings, validation, logging, and CloudWatch alarms.
5. **Reconcile data.** Compare row counts, checksums, sequence values, totals, and application smoke-test outputs. Record exceptions and owners.
6. **Execute cutover rehearsal.** Freeze writes or enable controlled dual-write, wait for CDC lag to drain, switch a test application endpoint, validate transactions, and document rollback steps.

```

1 $Region = "us-east-1"
2 $DrRegion = "us-west-2"
3 $ClusterId = "erp-prod-aurora-pg"
4 $GlobalClusterId = "erp-prod-global"
5 $DbName = "erp"
6 $SubnetGroup = "erp-private-db-subnets"
7 $SecurityGroup = "sg-0123456789abcdef0"
8 $KmsKeyId = "arn:aws:kms:us-east-1:123456789012:key/aurora-key"
9
10 aws rds create-global-cluster '
11   --global-cluster-identifier $GlobalClusterId '
12   --engine aurora-postgresql '

```

```

13 --engine-version 16.4 '
14 --region $Region
15
16 aws rds create-db-cluster '
17 --db-cluster-identifier $ClusterId '
18 --global-cluster-identifier $GlobalClusterId '
19 --engine aurora-postgresql '
20 --engine-version 16.4 '
21 --database-name $DbName '
22 --manage-master-user-password '
23 --storage-encrypted '
24 --kms-key-id $KmsKeyId '
25 --backup-retention-period 14 '
26 --deletion-protection '
27 --enable-cloudwatch-logs-exports postgresql '
28 --serverless-v2-scaling-configuration MinCapacity=2,MaxCapacity=32 '
29 --db-subnet-group-name $SubnetGroup '
30 --vpc-security-group-ids $SecurityGroup '
31 --region $Region
32
33 aws rds create-db-instance '
34 --db-instance-identifier "$ClusterId-writer-1" '
35 --db-cluster-identifier $ClusterId '
36 --engine aurora-postgresql '
37 --db-instance-class db.serverless '
38 --enable-performance-insights '
39 --region $Region

```

Listing 2.4: PowerShell-oriented AWS CLI skeleton for Aurora PostgreSQL fleet creation.

```

1 $DmsReplicationInstanceArn = "arn:aws:dms:us-east-1:123456789012:rep:
  ABCDEF123456"
2 $OracleSecretArn = "arn:aws:secretsmanager:us-east-1:123456789012:secret:oracle
  -erp"
3 $AuroraSecretArn = "arn:aws:secretsmanager:us-east-1:123456789012:secret:aurora
  -erp"
4 $OracleEndpointId = "oracle-erp-source"
5 $AuroraEndpointId = "aurora-pg-target"
6 $TaskId = "oracle-erp-to-aurora-cdc"
7
8 aws dms create-endpoint '
9 --endpoint-identifier $OracleEndpointId '
10 --endpoint-type source '
11 --engine-name oracle '
12 --database-name ERPDDB '
13 --secrets-manager-secret-id $OracleSecretArn '
14 --region us-east-1
15
16 aws dms create-endpoint '
17 --endpoint-identifier $AuroraEndpointId '
18 --endpoint-type target '
19 --engine-name aurora-postgresql '
20 --database-name erp '
21 --secrets-manager-secret-id $AuroraSecretArn '
22 --region us-east-1
23
24 $TableMappings = @"
25 {
26   "rules": [
27     {
28       "rule-type": "selection",
29       "rule-id": "include-erp-schema",
30       "rule-name": "include-erp-schema",
31       "object-locator": { "schema-name": "ERP", "table-name": "%" },

```

```

32     "rule-action": "include"
33   }
34 ]
35 }
36 "@
37 $TableMappings | Set-Content -Path ch2-dms-table-mappings.json -Encoding ascii
38
39 aws dms create-replication-task '
40   --replication-task-identifier $TaskId '
41   --source-endpoint-arn "arn:aws:dms:us-east-1:123456789012:endpoint:SOURCE" '
42   --target-endpoint-arn "arn:aws:dms:us-east-1:123456789012:endpoint:TARGET" '
43   --replication-instance-arn $DmsReplicationInstanceArn '
44   --migration-type full-load-and-cdc '
45   --table-mappings file://ch2-dms-table-mappings.json '
46   --replication-task-settings file://ch2-dms-task-settings.json '
47   --region us-east-1

```

Listing 2.5: PowerShell-oriented AWS CLI skeleton for DMS endpoints and a full-load-plus-CDC task.

```

1  import json
2  import os
3  from pathlib import Path
4
5  import boto3
6
7  region = os.environ.get("AWS_REGION", "us-east-1")
8  cluster_id = os.environ["AURORA_CLUSTER_ID"]
9  task_id = os.environ["DMS_TASK_ID"]
10
11 rds = boto3.client("rds", region_name=region)
12 dms = boto3.client("dms", region_name=region)
13
14 cluster = rds.describe_db_clusters(DBClusterIdentifier=cluster_id)["DBClusters"]
15         [0]
16 instances = rds.describe_db_instances()["DBInstances"]
17 cluster_instances = [
18     instance for instance in instances
19     if instance.get("DBClusterIdentifier") == cluster_id
20 ]
21 tasks = dms.describe_replication_tasks(
22     Filters=[{"Name": "replication-task-id", "Values": [task_id]}]
23 )["ReplicationTasks"]
24 assert tasks, f"DMS task {task_id} not found"
25 task = tasks[0]
26
27 summary = {
28     "cluster_id": cluster_id,
29     "engine": cluster["Engine"],
30     "engine_version": cluster["EngineVersion"],
31     "multi_az_members": cluster.get("AvailabilityZones", []),
32     "backup_retention_days": cluster["BackupRetentionPeriod"],
33     "deletion_protection": cluster.get("DeletionProtection"),
34     "cluster_endpoint": cluster.get("Endpoint"),
35     "reader_endpoint": cluster.get("ReaderEndpoint"),
36     "instance_count": len(cluster_instances),
37     "dms_task_status": task["Status"],
38     "dms_migration_type": task["MigrationType"],
39     "dms_stop_reason": task.get("StopReason"),
40 }
41
42 Path("ch2-capstone-evidence.json").write_text(
43     json.dumps(summary, indent=2), encoding="utf-8"

```

```
44 )  
45 print(json.dumps(summary, indent=2))
```

Listing 2.6: Boto3 evidence collector for DMS task status and Aurora cluster posture.

Deliverables

- Architecture summary with primary Region, secondary Region, RPO, RTO, Aurora sizing, backup policy, security posture, and failover plan.
- SCT assessment summary showing automatic conversions, manual remediations, retired objects, and owners for unresolved issues.
- Aurora configuration evidence: engine version, endpoints, readers, backup retention, encryption, deletion protection, Performance Insights, logs, and parameter groups.
- DMS artifacts: endpoint definitions, table mappings, task settings, validation settings, CloudWatch alarms, and replication-lag evidence.
- Reconciliation packet with row counts, checksums or sample comparisons, sequence validation, and business totals for critical ERP domains.
- Cutover and rollback runbook with command sequence, approval gates, communications, smoke tests, and rollback criteria.

Acceptance Criteria

- Aurora PostgreSQL is deployed with encryption, backups, deletion protection, monitoring, and at least one reader in the primary Region.
- Cross-region recovery is configured and the team has documented whether failover is planned switchover, unplanned failover, or restore from backup.
- DMS full-load-plus-CDC task reaches running or load-complete status in rehearsal, and replication lag is measured before cutover.
- SCT findings are triaged; no critical Oracle package, trigger, or scheduler dependency is left without an owner or disposition.
- Validation covers both technical counts and business reconciliation for orders, inventory, invoices, and ledger balances.
- Application endpoint switching is rehearsed without hardcoded database hostnames in application code or batch jobs.
- Rollback criteria are explicit, time-bound, and approved by business and technical owners before the cutover window.
- Analytical workloads are routed away from the Aurora writer through readers, S3 exports, Redshift, or another downstream store.

Stretch Goals

- Add AWS DMS premigration assessment and validation reports to the acceptance packet, including table-level error thresholds.
- Use AWS Secrets Manager rotation for Aurora credentials and document how applications refresh connections safely.

- Add RDS Proxy or PgBouncer-style pooling for application connection storms, then test failover behavior with pooled connections.
- Export selected Aurora tables to S3 and build a Redshift or Athena reporting path that eliminates ad hoc reporting against the writer.
- Build CloudWatch dashboards for ACU usage, CPU, database connections, deadlocks, commit latency, replica lag, DMS CDCLatencySource, and CDCLatencyTarget.
- Run a game day that simulates primary-region unavailability, promotes the secondary database, switches a read-only application, and documents failback reconciliation.

High-Scale NoSQL: Amazon DynamoDB

Learning Objectives

- Explain DynamoDB core concepts: tables, items, attributes, partitions, request routing, read capacity units, write capacity units, and capacity modes.
- Model high-scale access patterns with partition keys, sort keys, sparse indexes, local secondary indexes, and global secondary indexes.
- Describe DynamoDB Streams, time to live, and Global Tables for event-driven integration, lifecycle management, and multi-region active-active designs.
- Create a DynamoDB table, design a single-table schema for a forum workload, insert items with Boto3, and configure a GSI for query access.
- Design an ingestion and serving layer for a global gaming leaderboard that handles one million reads and writes per second with predictable hot-key controls.
- Build a capstone data layer using single-table design, GSIs, Streams, Global Tables, and on-demand scaling.

Cross-Cloud Equivalence

NoSQL services differ in APIs and consistency models, but the recurring design vocabulary is familiar: key distribution, item or document shape, secondary access paths, change feeds, expiration, and multi-region replication.

Concept	AWS DynamoDB	Dy-	Azure DB	Cosmos	GCP Firestore / Bigtable	MongoDB
Key-value / document store	Table of items with scalar, map, list, set, and document attributes		Container items with JSON documents	items	Firestore documents or Bigtable rows and column families	Collections of BSON documents
Partition key	Hash key distributes items across physical partitions		Partition key routes items to logical partitions		Firestore document path or Bigtable row-key prefix design	Shard key when sharding is enabled
Sort key	Optional range key orders related items within one partition key		Composite keys are modeled in item fields and indexed		Firestore indexed fields; Bigtable row-key suffix patterns	Compound index fields or embedded sortable attributes
Secondary indexes	GSI for alternate partition or sort keys; LSI for alternate sort key on same partition key		Automatic or defined indexes with partition-aware query design		Firestore composite indexes; Bigtable secondary indexes are limited and often modeled manually	Secondary and compound indexes
Streams / change feed	DynamoDB Streams with Lambda, Kinesis, or consumers		Change feed processor		Firestore triggers; Bigtable change streams	Change streams on replica sets or sharded clusters
TTL	Per-item TTL attribute deletes expired items asynchronously		Time to live on containers or items		Firestore TTL policies; Bigtable garbage-collection rules	TTL indexes on date fields
Global tables / multi-region	Global replicate active tables across Regions		Multi-region writes with tunable consistency		Firestore multi-region locations; Bigtable replication	Atlas global clusters and zone sharding

The transferable skill is not memorizing product names. It is learning to state access patterns first, choose keys that distribute traffic, and add only the secondary indexes needed to satisfy known queries.

3.1 Week 3 Roadmap and Mental Model

DynamoDB is AWS's managed, serverless NoSQL database for key-value and document workloads that need single-digit millisecond latency at very high scale [11]. Unlike relational systems, DynamoDB does not reward late query discovery. The data model is intentionally shaped around known access patterns, expected cardinality, item size, consistency needs, and write distribution. A data engineer uses DynamoDB when the workload needs operational serving speed, elastic scale, event-driven integration, or multi-region active-active availability more than ad hoc joins.

Definition

Amazon DynamoDB is a fully managed NoSQL key-value and document database. Applications read and write items in tables using primary keys, optional secondary indexes, capacity modes, and service-managed partitioning.

Key Terms

Item: one record in a table. **Attribute:** one named value on an item. **Partition key:** hash key used to distribute and locate data. **Sort key:** optional range key used to order related items. **GSI:** global secondary index with its own key schema. **LSI:** local secondary index with the table partition key and alternate sort key.

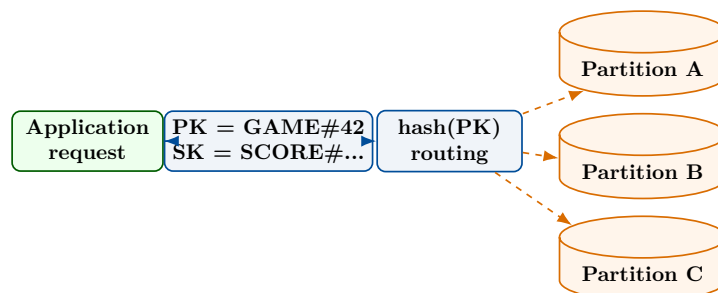
3.2 Monday: DynamoDB Core Concepts, Partitions, and Capacity

3.2.1 Tables, Items, and Physical Partitions

A DynamoDB table stores items. Each item is a set of attributes and can have a different shape from its neighbors. The required key attributes define how the service locates data. With a simple primary key, the partition key uniquely identifies an item. With a composite primary key, the partition key groups related items and the sort key uniquely orders items inside that group. Internally, DynamoDB hashes the partition key to route requests to physical partitions that hold data and serve capacity [60].

Note

DynamoDB is schema-flexible, not schema-free. The table does not enforce every attribute, but the application contract must still define entity types, required attributes, key formats, index projections, and lifecycle rules.



A good partition key spreads writes and reads across many hash values while preserving efficient item collections for known queries.

Figure 3.1: DynamoDB hashes the partition key to route items and requests to physical partitions. Sort keys organize related items after the partition key has chosen the item collection.

3.2.2 Read and Write Capacity

DynamoDB meters throughput in read capacity units and write capacity units for provisioned mode, or request units under on-demand billing. One write capacity unit supports one 1 KB write per second. Strongly consistent reads consume more capacity than eventually consistent reads, and larger items require more units. Capacity is consumed by the table and by indexes because index entries are separate write targets [64].

Definition

RCU and **WCU** are capacity abstractions used by provisioned DynamoDB tables. They make item size, read consistency, and request rate visible so teams can plan cost and throttling risk.

Tip

Estimate capacity from access patterns, not from table size alone. A small table with a celebrity user, global match, or viral post can throttle if the key design concentrates traffic on one partition key.

3.2.3 Provisioned Versus On-Demand Capacity

Provisioned capacity is appropriate when traffic is predictable, budgets require explicit limits, or autoscaling can follow a known pattern. On-demand capacity is appropriate for spiky, unknown, or early-stage workloads where operational simplicity and burst tolerance matter more than the lowest possible steady-state cost. Both modes still require key designs that avoid hot partitions.

Common Pitfall

Assuming on-demand fixes bad keys. On-demand mode removes capacity planning for the table, but it does not make one partition key able to absorb unlimited traffic. Hot keys still need write sharding, bucketing, caching, aggregation, or a different access pattern.

3.3 Tuesday: Data Modeling with Primary Keys, Sort Keys, LSI, and GSI**3.3.1 Access-Pattern-First Modeling**

Relational modeling starts from entities and normalization. DynamoDB modeling starts from questions the application must answer. For a forum, those questions might be: load a forum by slug, list threads in a forum by recent activity, load posts in a thread by creation time, list a user's posts, show moderation queues by status, and retrieve leaderboard summaries. Each access pattern becomes a key choice, index choice, or denormalized item.

Definition

Single-table design stores multiple entity types in one DynamoDB table and uses overloaded key attributes to colocate items that are read together. It trades relational normalization for predictable, low-latency access patterns.

Access pattern	PK	SK	Index
Get forum metadata	FORUM#slug	META	Table primary key
List threads in forum	FORUM#slug	THREAD#lastActivity#threadId	Table primary key
Load posts in thread	THREAD#threadId	POST#createdAt#postId	Table primary key
List posts by user	USER#userId	POST#createdAt#postId	GSI1
Moderation queue	STATUS#pending	REPORTED#createdAt#postId	GSI2, Global secondary index

Table 3.1: Example single-table access patterns for a forum workload.

3.3.2 Primary Keys and Sort-Key Encoding

Partition keys should have enough cardinality to distribute load. Sort keys should encode hierarchy and ordering in a way that supports begins-with, between, forward order, and reverse order queries. Common patterns include prefixes such as **FORUM#**, **THREAD#**, and **POST#**; ISO-8601 timestamps for lexical ordering; and inverted scores or padded numbers when descending order is needed.

Note

DynamoDB query operations require an equality condition on the partition key. Sort-key conditions refine the item collection after the partition key is chosen. A query that starts with only a non-key attribute is not a DynamoDB query; it is usually an index-design problem.

3.3.3 Local and Global Secondary Indexes

An LSI is created with the table, shares the table partition key, and provides an alternate sort key for items in the same item collection. A GSI can be created after table creation and has its own partition key and optional sort key. GSIs are the main tool for alternate access patterns, but they add write cost, storage cost, replication delay, and operational complexity [68].

Tip

Use sparse GSIs deliberately. If only items with GSI1PK and GSI1SK attributes appear in the index, the GSI can become a compact queue, lookup table, or status-specific access path.

Common Pitfall

Adding indexes after every new question. Each GSI duplicates selected attributes and receives writes. Too many indexes turn one application write into many index writes and can make hot access patterns harder to diagnose.

3.4 Wednesday: Streams, TTL, and Global Tables

3.4.1 DynamoDB Streams

DynamoDB Streams capture item-level changes in near real time. A stream record can contain keys only, new image, old image, or both images. Lambda functions, stream consumers, and Kinesis integrations use these records to update search indexes, maintain materialized views, publish events, enrich lake ingestion, or invalidate caches [65].

Definition

DynamoDB Streams is a time-ordered change log for item modifications in a table. It enables event-driven processing without polling the table for recently changed items.

Note

Stream consumers must be idempotent. Lambda retries, partial failures, and duplicate event processing are normal distributed-systems concerns, so downstream writes should use deterministic keys or conditional updates.

3.4.2 Time to Live

TTL marks items for asynchronous expiration using an epoch-time attribute. It is useful for sessions, temporary leaderboards, deduplication records, invitations, cached computations, and retention controls. TTL deletion is not immediate and should not be used when exact deletion time is a hard compliance boundary [65].

Common Pitfall

Treating TTL as a scheduler. TTL is a lifecycle hint, not a cron system. If business logic must execute exactly at expiration, use EventBridge, Step Functions, or a workflow service and treat TTL as cleanup.

3.4.3 Global Tables

Global Tables replicate DynamoDB tables across AWS Regions for multi-active applications. Applications can read and write locally in each participating Region, and DynamoDB replicates changes. This reduces latency for global users and improves regional resilience, but it also requires conflict-aware design, deterministic writes, and careful observability [63].

Tip

Global tables work best when each write has a clear owner or monotonic version strategy. For leaderboards, make the score update conditional on a higher score or newer version so replicated writes do not accidentally move a player backward.

3.5 Thursday Hands-on Project: Forum Single-Table Schema and GSI

This lab creates a DynamoDB table for a forum, inserts representative single-table items, and configures a GSI for user-centric queries. The commands assume Windows PowerShell with AWS CLI v2 and a sandbox AWS account. Replace account-specific values and clean up resources after testing.

Note

The lab uses on-demand capacity for simplicity. In production, choose on-demand or provisioned capacity from measured access patterns, expected peak traffic, budget controls, and throttling tolerance.

```

1 # Run from Windows PowerShell with AWS CLI v2 configured.
2 $Region = "us-east-1"
3 $TableName = "forum-single-table-demo"
4
5 aws dynamodb create-table '
6   --table-name $TableName '
7   --attribute-definitions '
8     AttributeName=PK,AttributeType=S '
9     AttributeName=SK,AttributeType=S '
10    AttributeName=GSI1PK,AttributeType=S '
11    AttributeName=GSI1SK,AttributeType=S '
12  --key-schema '
13    AttributeName=PK,KeyType=HASH '
14    AttributeName=SK,KeyType=RANGE '
15  --global-secondary-indexes "[
16    {
17      \"IndexName\": \"GSI1\",
18      \"KeySchema\": [
19        {\"AttributeName\": \"GSI1PK\", \"KeyType\": \"HASH\"},
20        {\"AttributeName\": \"GSI1SK\", \"KeyType\": \"RANGE\"}
21      ],
22      \"Projection\": {\"ProjectionType\": \"ALL\"}
23    }
24  ]" '

```

```

25 --billing-mode PAY_PER_REQUEST '
26 --stream-specification StreamEnabled=true,StreamViewType=NEW_AND_OLD_IMAGES '
27 --region $Region
28
29 aws dynamodb wait table-exists --table-name $TableName --region $Region
30
31 aws dynamodb update-time-to-live '
32 --table-name $TableName '
33 --time-to-live-specification "Enabled=true,AttributeName=expiresAt" '
34 --region $Region

```

Listing 3.1: PowerShell-oriented AWS CLI commands to create a DynamoDB table with a GSI.

```

1 from datetime import datetime, timedelta, timezone
2 from decimal import Decimal
3 import os
4 import boto3
5
6 region = os.environ.get("AWS_REGION", "us-east-1")
7 table_name = os.environ.get("TABLE_NAME", "forum-single-table-demo")
8 table = boto3.resource("dynamodb", region_name=region).Table(table_name)
9 now = datetime.now(timezone.utc).isoformat()
10 forum_slug, thread_id, user_id = "data-engineering", "thread-1001", "user-42"
11
12 items = [
13     {"PK": f"FORUM#{forum_slug}", "SK": "META", "entityType": "Forum", "title":
14         "Data Engineering", "createdAt": now},
15     {"PK": f"FORUM#{forum_slug}", "SK": f"THREAD#{now}#{thread_id}",
16         "entityType": "Thread", "threadId": thread_id, "title": "DynamoDB modeling
17         patterns", "GSI1PK": f"USER#{user_id}", "GSI1SK": f"THREAD#{now}#{thread_id}"
18     },
19     {"PK": f"THREAD#{thread_id}", "SK": f"POST#{now}#post-1", "entityType": "
20         Post", "postId": "post-1", "authorUserId": user_id, "body": "Start with
21         access patterns.", "score": Decimal("1"), "GSI1PK": f"USER#{user_id}", "
22         GSI1SK": f"POST#{now}#post-1", "expiresAt": int((datetime.now(timezone.utc)
23         + timedelta(days=365)).timestamp())},
24 ]
25
26 with table.batch_writer() as batch:
27     for item in items:
28         batch.put_item(Item=item)
29
30 response = table.query(IndexName="GSI1", KeyConditionExpression="GSI1PK = :user
31     ", ExpressionAttributeValues={" :user": f"USER#{user_id}"}, ScanIndexForward=
32     False)
33 for item in response["Items"]:
34     print(item["entityType"], item["SK"])

```

Listing 3.2: Boto3 script to insert forum items and query a GSI by user.

3.6 Friday Use Case: Global Gaming Leaderboard at One Million Reads and Writes Per Second

A gaming company runs tournaments in North America, Europe, Asia, and South America. Every match emits score events, profile updates, anti-cheat flags, and session telemetry. Players expect local writes, fresh regional leaderboards, and fast reads from mobile clients. The platform target is one million reads and writes per second globally, using DynamoDB Global Tables for active-active serving, Streams for derived views, and cache layers for extreme fan-out.

3.6.1 Architecture Narrative

The write path accepts score events through regional APIs. Each event uses an idempotency key so retries cannot double-count a match. The score item is written to a regional DynamoDB replica with a partition key that includes game, season, region, and bucket. A conditional expression updates the player's best score only if the new score is higher or has a newer authoritative version. Global Tables replicate the update to peer Regions. Streams publish changes to Lambda or Kinesis consumers that maintain top-N materialized leaderboard items, feed S3 analytics, and invalidate caches.

The read path separates personalized reads from hot global rankings. Player profile and rank lookups go directly to DynamoDB or DAX where appropriate [10]. Extremely hot top-N pages are precomputed from streams and served from a cache or CDN-backed API. This keeps millions of clients from repeatedly querying the same partition key during a tournament final.

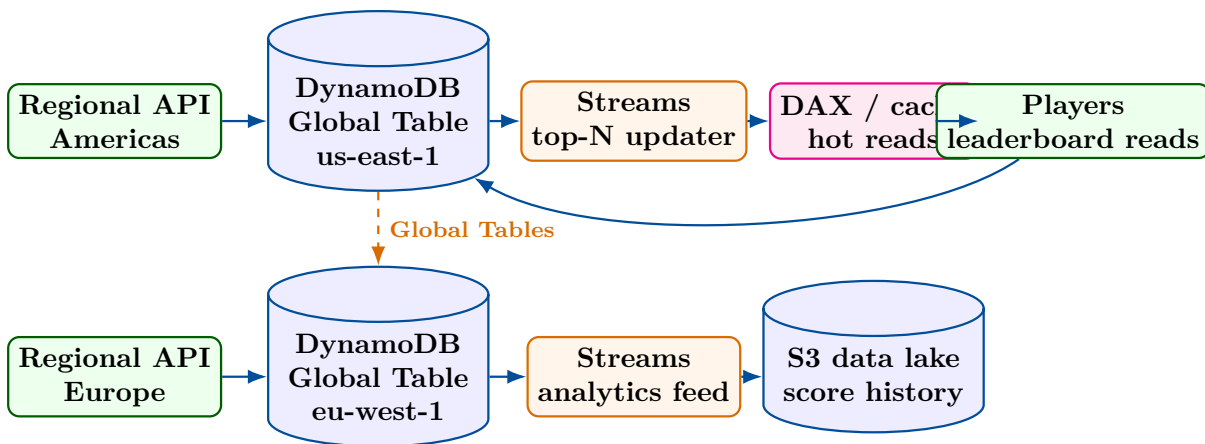


Figure 3.2: Global gaming leaderboard architecture with regional writes to DynamoDB Global Tables, streams for derived top-N views and lake ingestion, and cache or DAX layers for hot read serving.

3.6.2 Design Decisions

Write distribution. Avoid a single partition key such as `GAME#123#SEASON#2026` for all score writes. Add buckets, region segments, or match identifiers so writes spread across many partition keys, then compute aggregate top-N views asynchronously.

Correctness. Use conditional writes for best score, optimistic version attributes for profile changes, and idempotency items for score-event ingestion. Streams consumers must tolerate duplicate records and out-of-order cross-region observations.

Serving. Use DynamoDB for player-specific and partitioned rank queries. Use stream-maintained materialized top-N items plus cache for the hottest pages. DAX can reduce read latency for read-heavy DynamoDB access patterns, but it does not replace good key design.

Common Pitfall

Trying to query the whole leaderboard live. A global top-N query over every player on demand is an analytical operation. Operational leaderboards should maintain precomputed slices for each game, season, region, league, and time window.

3.7 Interview Q&A

3.7.1 Concepts and Trade-offs

1. **Q: How is DynamoDB different from a relational database?**

A: DynamoDB is a managed key-value and document database optimized for predictable access patterns and high scale. It does not provide joins or ad hoc relational querying; the table and indexes are modeled around known queries.

2. **Q: What makes a DynamoDB partition key good?**

A: It has high cardinality, distributes reads and writes evenly, and still groups items that must be queried together. It avoids hot keys created by globally popular users, games, or status values.

3. **Q: When would you choose on-demand capacity?**

A: Choose it for unknown, bursty, or early-stage traffic where operational simplicity matters. For predictable steady traffic, provisioned capacity with autoscaling may be more cost controlled.

4. **Q: What is the difference between an LSI and a GSI?**

A: An LSI shares the table partition key and provides an alternate sort key created with the table. A GSI has its own key schema, can be created later, and supports alternate access patterns.

5. **Q: Why are sparse GSIs useful?**

A: Only items containing the index key attributes appear in the index, so a sparse GSI can efficiently represent queues, status views, or selected entity types without indexing every item.

6. **Q: What are DynamoDB Streams used for?**

A: They capture item changes for event-driven processing such as cache invalidation, materialized views, search indexing, audit trails, and lake ingestion.

7. **Q: What caveat applies to TTL?**

A: TTL expiration is asynchronous. It is useful for cleanup and lifecycle management, but not for exact-time scheduling or strict deletion deadlines.

8. **Q: What extra design work do Global Tables require?**

A: They require conflict-aware write rules, regional observability, deterministic updates, idempotent consumers, and application routing that understands multi-region behavior.

3.8 Further Reading

Start with the DynamoDB Developer Guide for table design, APIs, indexes, streams, TTL, backup, and security behavior [11]. Study capacity mode, adaptive capacity, and throttling guidance before estimating costs [64, 60]. Review secondary index design and projection choices for every non-primary access pattern [68]. For event-driven and multi-region workloads, read the Streams, TTL, Global Tables, and DAX documentation [65, 63, 10].

Key Takeaways

- DynamoDB design starts with access patterns, traffic distribution, item size, consistency, and latency requirements.

- Partition keys distribute data and capacity; sort keys organize related items for efficient range and prefix queries.
- Provisioned capacity gives explicit throughput control, while on-demand capacity simplifies bursty workloads; neither mode fixes a hot key.
- GSIs support alternate access patterns, but each index adds write cost, storage cost, and replication behavior to understand.
- Streams, TTL, and Global Tables extend DynamoDB into event-driven, lifecycle-managed, and multi-region architectures.
- Large-scale leaderboards need write sharding, idempotency, materialized views, and cache-aware read serving rather than live global scans.

3.9 Exercises

1. **(Conceptual)** Explain why a high-cardinality partition key matters for both cost and availability under heavy write traffic.
2. **(Conceptual)** Compare provisioned and on-demand capacity for a workload with predictable weekday peaks and unpredictable weekend tournaments.
3. **(Modeling)** Extend table 3.1 with an access pattern for unread notifications per user.
4. **(Hands-on)** Run listing 3.1 in a sandbox account, then capture table status, stream ARN, TTL status, and GSI status.
5. **(Hands-on)** Modify listing 3.2 to insert three posts by two users and query each user's content through GSI1.
6. **(Design)** Design a sparse GSI for a moderation queue that only includes reported posts with status pending, escalated, or blocked.
7. **(Operations)** Write an alarm plan for throttled requests, system errors, replication latency, stream iterator age, and cache hit ratio.
8. **(Architecture)** Sketch a leaderboard write-sharding strategy for one million global writes per second and explain how top-N views are rebuilt.

3.10 Capstone Project: Building a Global Gaming-Leaderboard Data Layer

Capstone Project

Mission. Build the data layer for a global gaming leaderboard using DynamoDB single-table design, GSIs, Streams, Global Tables, and on-demand scaling. The platform must support regional score writes, player rank lookups, hot top-N serving, analytics export, and operational evidence.

Estimated time. 5–7 hours in a sandbox AWS account for table design, scripted inserts, stream integration stubs, global-table configuration, and acceptance evidence.

What you'll practice.

- Translating leaderboard access patterns into partition keys, sort keys, item collections, and GSIs.
- Designing write-sharded score ingestion with idempotency and conditional updates.

- Using Streams to maintain materialized top-N views and send immutable score history to analytics storage.
- Configuring Global Tables and on-demand scaling for multi-region active-active serving.

Scenario

A competitive gaming company is launching a world championship across four Regions. Players submit scores after every match, mobile clients read personal rank after each game, and spectators refresh top leaderboards during finals. The business requires low-latency regional writes, globally replicated player best scores, and hot read paths that can absorb sudden fan traffic. Data science teams also need score history in S3 for fraud detection and tournament analytics.

The data layer must use DynamoDB as the operational source of truth. It should avoid global scans, use GSIs only where access patterns require them, and produce evidence that streams, TTL, Global Tables, and capacity settings are configured correctly.

Requirements

Functional requirements.

- Design a single-table schema for games, seasons, players, score events, player best scores, regional leaderboard buckets, and top-N materialized views.
- Use a composite primary key with write-sharded leaderboard partitions for high-ingestion score events.
- Create at least one GSI for player-centric queries and one sparse GSI for tournament moderation or anti-cheat review.
- Enable Streams with new and old images so consumers can update top-N views and export score history.
- Enable TTL for temporary idempotency records, session records, and short-lived tournament cache items.
- Configure Global Tables across at least two Regions or document the exact commands and acceptance evidence required when sandbox limits prevent creation.

Non-functional requirements.

- The design must sustain hot tournaments without a single partition key receiving all writes.
- All score updates must be idempotent and conditional so retries or replicated writes do not corrupt best-score state.
- Read paths must separate personal rank lookups, regional top-N queries, and global spectator traffic.
- Monitoring must include throttling, consumed capacity, user errors, system errors, stream iterator age, replication latency, and cache hit ratio where a cache is used.
- Scripts must not contain secrets, account credentials, or hardcoded personal data.

Reference Architecture

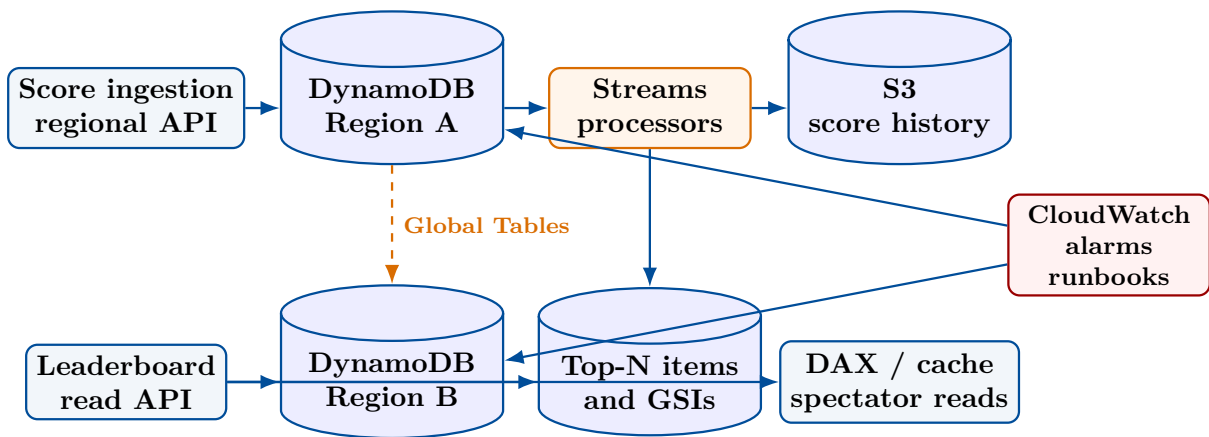


Figure 3.3: Capstone reference architecture for a global gaming-leaderboard data layer with regional APIs, DynamoDB Global Tables, Streams processors, top-N materialized items, cache acceleration, S3 export, and observability.

Implementation Steps

1. **Define access patterns.** List every read and write path: submit score, get player best score, get player rank, list top players by game and season, review flagged events, and export score history.
2. **Design keys.** Choose table PK and SK formats, write-sharded leaderboard bucket keys, GSI keys for player lookups, and sparse GSI keys for anti-cheat queues.
3. **Create the table.** Use on-demand capacity, streams with new and old images, TTL, server-side encryption, point-in-time recovery, and least-privilege IAM.
4. **Write ingestion code.** Insert score events idempotently and update best-score items with conditional expressions.
5. **Build stream consumers.** Maintain top-N materialized items and export immutable events to S3 or a delivery stream.
6. **Configure Global Tables.** Add replica Regions, test regional writes, and record replication status and latency evidence.
7. **Validate scale posture.** Demonstrate write distribution across bucketed keys, confirm hot read paths use materialized views or cache, and document alarms.

```

1 $PrimaryRegion = "us-east-1"
2 $ReplicaRegion = "eu-west-1"
3 $TableName = "global-leaderboard"
4
5 aws dynamodb create-table '
6   --table-name $TableName '
7   --attribute-definitions '
8     AttributeName=PK,AttributeType=S '
9     AttributeName=SK,AttributeType=S '
10    AttributeName=GSI1PK,AttributeType=S '
11    AttributeName=GSI1SK,AttributeType=S '
12    AttributeName=GSI2PK,AttributeType=S '
13    AttributeName=GSI2SK,AttributeType=S '
14    --key-schema AttributeName=PK,KeyType=HASH AttributeName=SK,KeyType=RANGE '

```

```

15 --global-secondary-indexes "[
16     {
17         \"IndexName\": \"GSI1PlayerLookup\",
18         \"KeySchema\": [
19             {\"AttributeName\": \"GSI1PK\", \"KeyType\": \"HASH\"},
20             {\"AttributeName\": \"GSI1SK\", \"KeyType\": \"RANGE\"}
21         ],
22         \"Projection\": {\"ProjectionType\": \"ALL\"}
23     },
24     {
25         \"IndexName\": \"GSI2ReviewQueue\",
26         \"KeySchema\": [
27             {\"AttributeName\": \"GSI2PK\", \"KeyType\": \"HASH\"},
28             {\"AttributeName\": \"GSI2SK\", \"KeyType\": \"RANGE\"}
29         ],
30         \"Projection\": {\"ProjectionType\": \"ALL\"}
31     }
32 ]" '
33 --billing-mode PAY_PER_REQUEST '
34 --stream-specification StreamEnabled=true,StreamViewType=NEW_AND_OLD_IMAGES '
35 --sse-specification Enabled=true '
36 --region $PrimaryRegion
37
38 aws dynamodb wait table-exists --table-name $TableName --region $PrimaryRegion
39
40 aws dynamodb update-continuous-backups '
41     --table-name $TableName '
42     --point-in-time-recovery-specification PointInTimeRecoveryEnabled=true '
43     --region $PrimaryRegion
44
45 aws dynamodb update-table '
46     --table-name $TableName '
47     --replica-updates "Create={RegionName=$ReplicaRegion}" '
48     --region $PrimaryRegion

```

Listing 3.3: PowerShell-oriented AWS CLI skeleton for the leaderboard table and replicas.

```

1 from datetime import datetime, timedelta, timezone
2 from decimal import Decimal
3 import os, uuid, boto3
4 from botocore.exceptions import ClientError
5
6 table = boto3.resource("dynamodb", region_name=os.environ.get("AWS_REGION", "us
    -east-1")).Table(os.environ.get("TABLE_NAME", "global-leaderboard"))
7
8 def submit_score(game_id, season_id, player_id, score, event_id=None):
9     now = datetime.now(timezone.utc)
10    event_id = event_id or str(uuid.uuid4())
11    bucket = int(player_id[-2:], 16) % 100
12    try:
13        table.put_item(Item={"PK": f"IDEMPOTENCY#{event_id}", "SK": "EVENT", "
    expiresAt": int((now + timedelta(days=2)).timestamp()), ConditionExpression
    ="attribute_not_exists(PK)"})
14    except ClientError as exc:
15        if exc.response["Error"]["Code"] == "ConditionalCheckFailedException":
16            return {"status": "duplicate", "eventId": event_id}
17        raise
18
19    table.put_item(Item={"PK": f"GAME#{game_id}#SEASON#{season_id}#BUCKET#{
    bucket:02d}", "SK": f"SCORE#{now.isoformat()}#{event_id}", "entityType": "
    ScoreEvent", "playerId": player_id, "score": Decimal(str(score)), "GSI1PK":
    f"PLAYER#{player_id}", "GSI1SK": f"SCORE#{now.isoformat()}#{game_id}"})
20    table.update_item(
21        Key={"PK": f"PLAYER#{player_id}", "SK": f"BEST#{game_id}#{season_id}"},

```

```

22     UpdateExpression="SET entityType=:t, bestScore=:s, updatedAt=:n, GSI1PK
    =:p, GSI1SK=:g",
23     ConditionExpression="attribute_not_exists(bestScore) OR bestScore < :s"
    ,
24     ExpressionAttributeValues={":t": "PlayerBestScore", ":s": Decimal(str(
    score)), ":n": now.isoformat(), ":p": f"PLAYER#{player_id}", ":g": f"BEST#{
    game_id}#{season_id}"},
25     )
26     return {"status": "accepted", "eventId": event_id, "bucket": bucket}
27
28 print(submit_score("space-race", "2026", "af42", 987650))

```

Listing 3.4: Boto3 score ingestion with idempotency and conditional best-score updates.

Deliverables

- Access-pattern matrix with table keys, sort-key formats, GSI keys, expected cardinality, and consistency choice for each query.
- DynamoDB table creation evidence showing billing mode, stream setting, TTL status, point-in-time recovery, encryption, GSIs, and replica Regions.
- Ingestion script with idempotency records, write-sharded score events, conditional best-score updates, and sample output.
- Stream-processing design that explains how top-N views are updated, how duplicate events are handled, and how score history reaches S3.
- Read-serving plan for player rank, regional leaderboard, global top-N, spectator traffic, and cache invalidation.
- Operations packet with CloudWatch alarms, dashboard metrics, failure scenarios, and cleanup steps.

Acceptance Criteria

- The table uses a composite primary key and at least two GSIs: one for player lookups and one sparse index for review or moderation workflows.
- Score writes include idempotency protection and conditional best-score updates.
- Leaderboard writes are distributed across bucketed partition keys rather than concentrated on one game-season key.
- Streams are enabled with new and old images, and the top-N update design is idempotent.
- TTL is enabled for temporary records, and the design does not rely on exact deletion timing.
- Global Tables are configured across at least two Regions or a sandbox exception documents the exact command, required IAM, and expected evidence.
- The read path avoids live global scans and uses materialized views or cache for hot top-N pages.
- Monitoring covers throttles, errors, consumed capacity, stream iterator age, global replication health, and cache behavior.

Stretch Goals

- Add a load-test script that writes synthetic scores across 100 leaderboard buckets and reports key distribution.
- Implement a Lambda stream consumer that maintains top 100 players per game, season, region, and league.
- Add DAX or another cache for player rank reads, then measure hit ratio and stale-read tolerance.

Hybrid Ingestion and Caching

Learning Objectives

- Explain when to accelerate databases with Amazon ElastiCache for Redis or Memcached, and how cache-aside, write-through, TTL, eviction, and invalidation choices affect correctness.
- Describe DynamoDB Accelerator as a managed, API-compatible, in-memory cache for read-heavy DynamoDB workloads.
- Compare AWS AppSync and Amazon API Gateway as managed ingestion front doors for GraphQL, REST, HTTP, and event-driven data APIs.
- Deploy an ElastiCache Redis cluster and write a Python cache-aside script that caches RDS query results to reduce repeated database load.
- Design a high-throughput, low-latency mobile data API that survives live sports traffic spikes with cache-aware serving paths.
- Complete the Part I milestone by integrating S3, Aurora Serverless v2, DynamoDB, ElastiCache, DAX, and AWS DMS into an e-commerce data backend.

Cross-Cloud Equivalence

Caching services vary by API, but the engineering vocabulary transfers across clouds: in-memory acceleration, TTL, eviction, cache-aside, write-through, database-integrated cache, invalidation, and hot-key protection.

Concept	AWS	Azure	GCP	Others
Managed Redis / Memcached	ElastiCache for Redis OSS, Valkey, and Memcached; MemoryDB for durable Redis-compatible workloads	Azure Cache for Redis	Memorystore for Redis and Memorystore for Memcached	Redis Enterprise Cloud, Upstash, Redis self-managed Redis or Memcached
Database-integrated cache	DynamoDB Accelerator for API-compatible DynamoDB read caching	Azure Cosmos DB integrated cache	No direct universal equivalent; combine Memorystore with Firestore, Spanner, AlloyDB, or Bigtable patterns	MongoDB Atlas query features plus external Redis caches
Cache-aside	Application checks cache, then queries RDS, Aurora, DynamoDB, or an API and stores the result	Same pattern with Azure SQL, Cosmos DB, or services behind Functions	Same pattern with Cloud SQL, Spanner, Firestore, or Bigtable	Common pattern in open-source Redis, Memcached, and application frameworks
Write-through / write-behind	Application or library writes cache and source together; requires idempotency and failure handling	Same concept with cache plus source write coordination	Same concept; often implemented in service code or stream processors	Available in some cache clients, proxies, and data-grid products
TTL and eviction	Per-key TTL, Redis policies, Memcached LRU behavior, DAX item and query cache TTL	TTL and eviction policies in Azure Cache for Redis; Cosmos DB integrated-cache staleness setting	Memorystore maxmemory policies and TTL support	Product-specific maxmemory, LRU, LFU, TTL, and persistence options
Hot-key control	Shard keys, precompute hot pages, DAX for DynamoDB reads, ElastiCache replicas, API throttling	Redis clustering, replicas, API Management throttles	Memorystore scaling, Cloud CDN, API Gateway quotas	CDN, request coalescing, client backoff, distributed locks, and key bucketing
The transferable skill is not choosing a cache by brand. It is deciding which data can be stale, how stale it may be, who invalidates it, and how the system behaves when the cache is unavailable.				

4.1 Week 4 Roadmap and Mental Model

Weeks 1–3 built the storage foundation: S3 for durable objects, RDS and Aurora for relational records, and DynamoDB for high-scale key-value and document access. Week 4 adds the serving layer that makes those systems usable during real traffic: cache acceleration, managed API ingestion, and hybrid read paths. A cache is not a database replacement. It is a latency and load-management tool placed between clients and a durable source of truth.

Definition

Hybrid ingestion and caching combines durable data stores, in-memory acceleration, and managed API front doors so applications can accept traffic quickly, serve hot reads cheaply, and preserve correct writes in systems of record.

Key Terms

Cache-aside: application reads from cache first, then loads from the database on a miss. **TTL:** time to live for cached values. **Eviction:** removal caused by capacity pressure or policy. **Invalidation:** explicit removal or refresh after source data changes. **Hot key:** cache or database key receiving disproportionate traffic.

4.2 Monday: Amazon ElastiCache for Database Acceleration

4.2.1 Redis, Valkey, and Memcached in the Data Path

Amazon ElastiCache is a managed in-memory service for Redis OSS, Valkey, and Memcached workloads [12]. Redis-compatible engines provide rich data structures, replication, clustering, persistence options, sorted sets, streams, pub/sub, and atomic operations. Memcached is simpler: it is a distributed, in-memory key-value cache optimized for ephemeral objects and horizontal simplicity. A data engineer usually places ElastiCache in front of RDS, Aurora, an API, or a derived view when repeated reads are expensive or latency-sensitive.

Definition

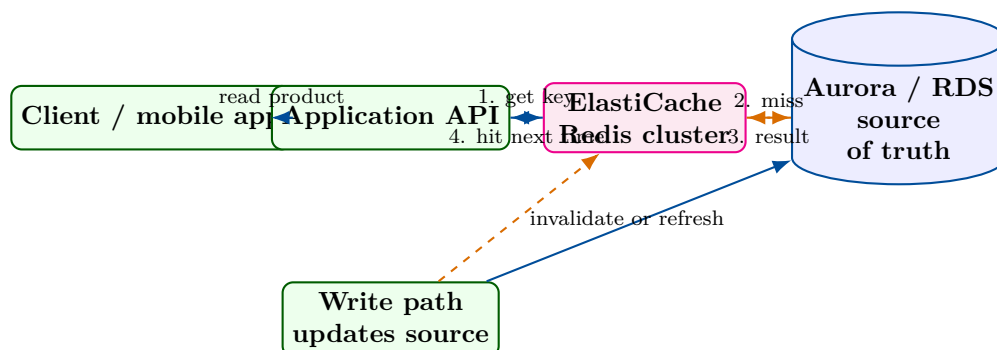
Amazon ElastiCache is a fully managed in-memory cache service. It reduces database load by serving repeated reads from memory and by storing short-lived computations, sessions, counters, and hot aggregates.

Note

ElastiCache improves read latency only when the application repeatedly asks for cacheable data. It will not fix unindexed SQL, missing DynamoDB access patterns, slow network calls, or write bottlenecks unless the architecture changes the path those requests take.

4.2.2 Cache-Aside, Write-Through, TTL, and Invalidation

The most common pattern is cache-aside. The application checks Redis or Memcached. If the key exists, the value is returned. If it misses, the application queries the database, serializes the result, stores it with a TTL, and returns it. This pattern keeps the cache disposable and easy to rebuild. The cost is application complexity: every caller must agree on key names, serialization, TTL, error handling, and invalidation.



Cache-aside keeps Redis disposable, but correctness depends on TTL selection and write-path invalidation.

Figure 4.1: Cache-aside flow between an application, ElastiCache Redis, and an RDS or Aurora source of truth.

Write-through writes data to the cache and database together. It can make read-after-write behavior more predictable, but it creates more failure modes. If the cache write succeeds and the database write fails, the cache may advertise data that is not durable. If the database write succeeds and the cache write fails, reads may stay stale until TTL or invalidation. For most operational systems, cache-aside plus precise invalidation is easier to reason about.

Tip

Start with short TTLs and explicit invalidation for data that users can change. Use longer TTLs for reference data, product catalog fragments, feature flags, and computed summaries where bounded staleness is acceptable.

Common Pitfall

Caching without an ownership rule. Every cached key needs an owner that knows when the source data changes. If multiple services write the same database row but only one service invalidates the cache, stale reads become intermittent and hard to diagnose.

4.2.3 Operational Design

ElastiCache clusters live in VPC subnets and should be reached from private application tiers, not from public clients. Production designs use security groups, encryption in transit where supported, authentication, subnet groups, Multi-AZ replication, automatic failover, metrics, and alarms. Important metrics include cache hit ratio, evictions, CPU, memory, network throughput, replication lag, current connections, and command latency.

Note

A low cache hit ratio is not automatically bad. New workloads, write-heavy workloads, or personalized one-time reads may be poor cache candidates. Measure hit ratio with database load, latency, and stale-read tolerance before declaring success.

4.3 Tuesday: DynamoDB Accelerator

4.3.1 Where DAX Fits

DynamoDB Accelerator is a managed, in-memory cache for DynamoDB. It is API-compatible with DynamoDB client patterns through the DAX SDK, so applications can accelerate eventually consistent read-heavy access patterns without building a separate Redis key naming scheme [10]. DAX keeps item and query results in memory, handles cache population, and invalidates entries for writes that pass through the DAX client.

Definition

DAX is a DynamoDB-compatible caching layer that can reduce read latency from milliseconds to microseconds for suitable read-heavy workloads. It is not a general Redis replacement and it does not support strongly consistent reads.

4.3.2 DAX Versus ElastiCache

Choose DAX when the source is DynamoDB, the application uses key-value or query access patterns, and the team wants the cache to understand DynamoDB operations. Choose ElastiCache when the source is RDS, Aurora, APIs, S3 metadata, computed responses, session data,

counters, sorted sets, rate limits, or any workload requiring custom cache structures. DAX is specialized; ElastiCache is general-purpose.

Decision point	DAX	ElastiCache
Source of truth	DynamoDB tables and indexes	RDS, Aurora, DynamoDB, APIs, files, computed values, and service responses
Client model	DAX-aware DynamoDB client	Redis, Valkey, or Memcached protocol clients
Invalidation	Managed for writes through DAX; TTL for cached items and query results	Application-managed invalidation, TTL, eviction, pub/sub, or stream-driven refresh
Data structures	DynamoDB items and query result sets	Strings, hashes, lists, sets, sorted sets, streams, counters, and arbitrary serialized objects
Consistency	Eventually consistent reads only	Application-defined; cache can be stale unless invalidated or bypassed
Best fit	Read-heavy DynamoDB hot paths	General cache-aside, sessions, rate limits, leaderboards, API response cache, SQL result cache

Table 4.1: DAX and ElastiCache solve different cache problems.

Tip

Use DAX for workloads with repeated `GetItem`, `BatchGetItem`, and predictable `Query` requests. Do not use it to hide poor partition-key design, scans, or analytical access patterns.

Common Pitfall

Expecting strong consistency from DAX. Strongly consistent reads bypass the DAX cache and go to DynamoDB. If the workflow requires read-after-write correctness for every request, design that path explicitly instead of assuming the cache can provide it.

4.3.3 DAX Operations

A DAX cluster is deployed into a VPC with subnet groups and IAM permissions. Applications should set timeouts, retries, and fallback behavior so a cache issue does not become a total outage. Monitor cache hit rate, item-cache hits, query-cache hits, CPU, memory, error counts, and request latency. Because DAX nodes are stateful caches, resizing or failover can temporarily reduce hit ratio while entries warm again.

4.4 Wednesday: AppSync and API Gateway Data Ingestion

4.4.1 Managed API Front Doors

AWS AppSync is a managed GraphQL service for building APIs that connect clients to DynamoDB, Lambda, OpenSearch, HTTP services, RDS through resolvers, and other sources [35]. Amazon API Gateway provides managed REST, HTTP, and WebSocket APIs with authorization, throttling, usage plans, transformations, stages, and integrations [4]. In data engineering systems, these services are the ingestion and serving layer that protects databases from direct client traffic.

Definition

Managed API ingestion uses services such as AppSync and API Gateway to authenticate, validate, throttle, transform, and route incoming data before it reaches storage or processing systems.

4.4.2 Choosing AppSync or API Gateway

AppSync fits applications where clients need a typed graph, flexible selection sets, subscriptions, offline synchronization, or direct resolver integration with DynamoDB. API Gateway fits RESTful ingestion, HTTP proxy patterns, serverless microservices, webhooks, partner APIs, and simple mobile backends. Both can integrate with Lambda for validation and orchestration, and both can sit in front of cached read paths.

Note

GraphQL reduces over-fetching, but it can increase resolver fan-out if each field triggers a separate backend lookup. Use batching, pipeline resolvers, authorization boundaries, and cache-aware resolver design for high-traffic graphs.

Tip

Place throttles and quotas at the API layer before traffic reaches the database. A cache absorbs repeated reads, but API Gateway usage plans, AppSync limits, WAF rules, and client backoff prevent abusive or accidental request storms.

4.4.3 Hybrid Ingestion Patterns

Common ingestion patterns include API Gateway to Lambda to Aurora, AppSync resolvers to DynamoDB, API Gateway to Kinesis for streaming telemetry, and AppSync subscriptions for near-real-time client updates. The serving side may read from DAX for DynamoDB, ElastiCache for RDS query results, or precomputed S3 and CloudFront assets for public content. The best architecture often separates write correctness from read speed: writes go to the durable source, while reads use caches, materialized views, and CDN-friendly objects.

Common Pitfall

Letting clients bypass the ingestion layer. Direct database access from mobile or browser clients exposes credentials, weakens throttling, and removes the chance to validate payloads before writes reach the system of record.

4.5 Thursday Hands-on Project: Redis Cache-Aside for RDS Queries

This lab deploys a small ElastiCache Redis-compatible cluster and uses Python to cache RDS query results. The commands assume Windows PowerShell, AWS CLI v2, a sandbox VPC with at least two private subnets, an application security group, and an existing RDS or Aurora database endpoint. Replace identifiers before running, and clean up resources after testing.

Note

ElastiCache nodes are billed while running. Use the smallest test node type available in your account and delete the replication group after the lab if it is not needed.

```

1 # Run from Windows PowerShell with AWS CLI v2 configured.
2 $Region = "us-east-1"
3 $VpcId = "vpc-0123456789abcdef0"
4 $SubnetIds = "subnet-aaaa1111 subnet-bbbb2222"
5 $AppSecurityGroupId = "sg-app1234567890"
6 $CacheSecurityGroupId = "sg-cache1234567890"
7 $SubnetGroupName = "ch4-redis-subnets"
8 $ReplicationGroupId = "ch4-rds-query-cache"
9 aws elasticache create-cache-subnet-group '
10     --cache-subnet-group-name $SubnetGroupName '
11     --cache-subnet-group-description "Private subnets for Chapter 4 Redis" '
12     --subnet-ids $SubnetIds '
13     --region $Region
14 aws elasticache create-replication-group '
15     --replication-group-id $ReplicationGroupId '
16     --replication-group-description "Chapter 4 Redis cache-aside demo" '
17     --engine redis '
18     --cache-node-type cache.t4g.micro '
19     --num-cache-clusters 2 '
20     --automatic-failover-enabled '
21     --cache-subnet-group-name $SubnetGroupName '
22     --security-group-ids $CacheSecurityGroupId '
23     --transit-encryption-enabled '
24     --at-rest-encryption-enabled '
25     --region $Region
26 aws elasticache wait replication-group-available '
27     --replication-group-id $ReplicationGroupId '
28     --region $Region

```

Listing 4.1: PowerShell-oriented AWS CLI commands to create an ElastiCache Redis replication group.

```

1 import json
2 import os
3 from decimal import Decimal
4 import psycopg
5 import redis
6 redis_client = redis.Redis(
7     host=os.environ["REDIS_HOST"],
8     port=int(os.environ.get("REDIS_PORT", "6379")),
9     ssl=os.environ.get("REDIS_SSL", "true").lower() == "true",
10    socket_timeout=1.0,
11    decode_responses=True,
12 )
13 connection_string = os.environ["RDS_POSTGRES_DSN"]
14 CACHE_SECONDS = int(os.environ.get("CACHE_SECONDS", "60"))
15 def default_json(value):
16     if isinstance(value, Decimal):
17         return float(value)
18     raise TypeError(f"Cannot serialize {type(value)}")
19 def get_product(product_id):
20     cache_key = f"product:{product_id}"
21     cached = redis_client.get(cache_key)
22     if cached:
23         return {"source": "cache", "item": json.loads(cached)}
24     with psycopg.connect(connection_string) as conn:
25         with conn.cursor() as cur:
26             cur.execute(
27                 """
28                 SELECT product_id, name, price, inventory_count, updated_at
29                 FROM products
30                 WHERE product_id = %s
31                 """,
32                 (product_id,),

```

```

33         )
34         row = cur.fetchone()
35     if row is None:
36         redis_client.setex(cache_key, 15, json.dumps({"missing": True}))
37         return {"source": "database", "item": None}
38     item = {
39         "product_id": row[0],
40         "name": row[1],
41         "price": row[2],
42         "inventory_count": row[3],
43         "updated_at": row[4].isoformat(),
44     }
45     redis_client.setex(cache_key, CACHE_SECONDS, json.dumps(item, default=
default_json))
46     return {"source": "database", "item": item}
47 if __name__ == "__main__":
48     for _ in range(2):
49         print(get_product(os.environ.get("PRODUCT_ID", "SKU-1001")))

```

Listing 4.2: Python cache-aside script using redis-py and pycpg to cache RDS query results.

Tip

Cache negative lookups briefly. A short TTL for missing products, users, or feature flags prevents repeated database hits during typos or crawler traffic while limiting stale absence after a new record appears.

4.6 Friday Use Case: Mobile Sports API with Extreme Traffic Spikes

A sports media company serves a mobile app during championship events. Before kickoff, traffic is moderate. During a goal, trade, injury, or final play, millions of clients refresh the same scorecards, player statistics, odds summaries, and social reactions. The workload combines hot public reads, personalized user reads, bursty clickstream writes, and strict correctness for purchases, subscriptions, and user settings.

4.6.1 Architecture Narrative

The write path accepts clickstream and interaction events through API Gateway and routes them to Kinesis, DynamoDB, or Lambda depending on latency and ordering needs. User profile and billing writes go to Aurora or RDS through a service boundary. Active sessions, shopping carts, live score summaries, and user-specific notification state use DynamoDB where key-value scale is required. The read path uses AppSync for GraphQL mobile screens, API Gateway for REST endpoints, DAX for repeated DynamoDB reads, and ElastiCache for live-score fragments, SQL query results, rate limits, and computed response objects.

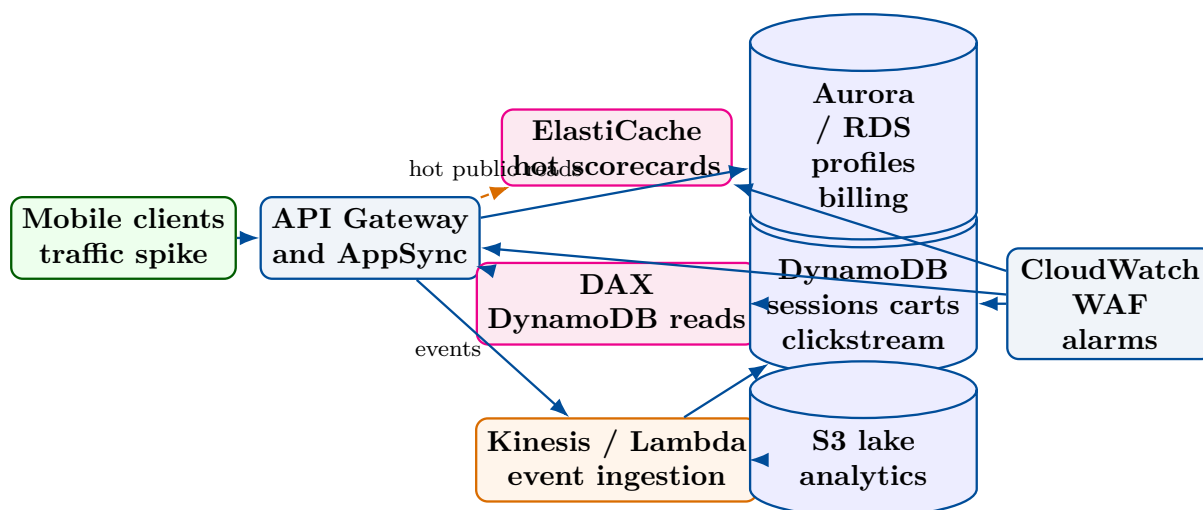


Figure 4.2: Spiky-traffic mobile sports API using API Gateway, AppSync, ElastiCache, DAX, DynamoDB, Aurora or RDS, and streaming ingestion.

4.6.2 Design Decisions

Separate hot public reads from personalized reads. Everyone asks for the same live score after a major play, so cache public scorecards and summary fragments aggressively. Personalized notifications, purchases, and account settings need user-scoped keys and stricter authorization. **Precompute the most popular shapes.** Maintain live scoreboard documents, player stat cards, and top-comment summaries with stream processors or scheduled refresh. Do not ask the database to assemble the same response millions of times during a spike. **Throttle before storage.** API Gateway, AppSync, AWS WAF, quotas, and client retry guidance protect the database from fan-out. Caches reduce load after requests arrive, but throttles prevent overload when the request itself is unnecessary or abusive.

Common Pitfall

Caching the wrong boundary. Caching a full personalized response can leak data if the cache key omits identity, locale, subscription tier, or authorization context. Public and private response fragments should use different key spaces and TTL policies.

4.7 Interview Q&A

4.7.1 Concepts and Trade-offs

1. **Q: What problem does ElastiCache solve?**

A: It serves repeated reads or transient computations from memory so durable databases and APIs handle fewer requests with lower average latency.

2. **Q: When would you choose Redis-compatible ElastiCache over Memcached?**

A: Choose Redis-compatible engines for richer data structures, replication, clustering, atomic operations, streams, sorted sets, and persistence options. Choose Memcached for simple ephemeral object caching with a minimal key-value model.

3. **Q: What is cache-aside?**

A: The application checks the cache first, loads from the database on a miss, stores the result with a TTL, and returns it. The database remains the source of truth.

4. **Q: Why is cache invalidation difficult?**

A: The system must know every write that changes a cached value, remove or refresh all affected keys, and handle races between readers, writers, retries, and TTL expiration.

5. **Q: How is DAX different from ElastiCache?**

A: DAX is a DynamoDB-specific, API-compatible cache for eventually consistent read-heavy access patterns. ElastiCache is a general Redis, Valkey, or Memcached service for many sources and data structures.

6. **Q: Can DAX provide strongly consistent reads?**

A: No. Strongly consistent reads bypass DAX and go to DynamoDB, so workflows requiring strict read-after-write behavior need explicit design.

7. **Q: When is AppSync a better fit than API Gateway?**

A: AppSync is often better for GraphQL mobile or web experiences that need typed schemas, subscriptions, flexible selection sets, and resolver integration. API Gateway is often better for REST, HTTP, webhook, and microservice ingestion patterns.

8. **Q: What metrics indicate cache health?**

A: Cache hit ratio, latency, evictions, memory pressure, CPU, connection count, replication lag, error rate, and downstream database load together show whether the cache is helping.

4.8 Further Reading

Start with the ElastiCache documentation for engine choices, cluster modes, security, scaling, and monitoring [12]. Review DAX behavior before using it with DynamoDB hot paths [10]. Study AppSync and API Gateway ingestion patterns, authorization options, quotas, and resolver or integration models [35, 4]. For the capstone, revisit S3, Aurora Serverless v2, DynamoDB, and AWS DMS documentation from Weeks 1–3 [33, 77, 11, 42].

Key Takeaways

- Caches reduce latency and database load only for cacheable access patterns with repeated reads or reusable computations.
- ElastiCache is general-purpose in-memory acceleration; DAX is specialized DynamoDB read acceleration.
- Cache-aside is simple and resilient, but it requires disciplined key naming, TTLs, serialization, fallback behavior, and invalidation.
- AppSync and API Gateway protect data stores by authenticating, throttling, validating, transforming, and routing client traffic.
- Spiky mobile APIs need precomputed hot responses, API throttles, cache-aware read paths, and durable write paths.
- Part I systems become powerful when S3, Aurora, DynamoDB, caches, and DMS are composed around clear sources of truth.

4.9 Exercises

1. **(Conceptual)** Explain why a cache should usually be treated as disposable rather than as the system of record.

2. **(Conceptual)** Compare cache-aside and write-through for product catalog reads where prices change hourly.
3. **(Design)** Choose Redis-compatible ElastiCache, Memcached, or DAX for sessions, leaderboard sorted sets, DynamoDB product lookups, and RDS report fragments. Justify each choice.
4. **(Hands-on)** Run listing 4.1 in a sandbox VPC and capture the replication group status, primary endpoint, security group rule, and subnet group.
5. **(Hands-on)** Modify listing 4.2 to cache a category query and print whether each response came from the cache or database.
6. **(Operations)** Define CloudWatch alarms for cache evictions, CPU, memory pressure, replication lag, connection count, and RDS read load.
7. **(Security)** Describe how cache key design changes when responses vary by user identity, subscription tier, locale, or authorization scope.
8. **(Architecture)** Sketch a spiky live sports API and identify which responses use AppSync, API Gateway, DAX, ElastiCache, DynamoDB, Aurora, and S3.

4.10 Milestone 1 Capstone: E-Commerce Multi-Tier Data Backend

Capstone Project

Mission. Build an e-commerce multi-tier data backend that integrates the Part I services: Aurora Serverless v2 for user profiles and billing, DynamoDB for active shopping carts and clickstream tracking, S3 for encrypted product images with lifecycle rules, ElastiCache and DAX for hot-path acceleration, and AWS DMS to continuously replicate Aurora data to S3 for analytics. **Estimated time.** 8–12 hours in a sandbox AWS account, or 4–6 hours for a documented architecture and scripted dry run when account limits prevent full deployment.

What you'll practice.

- Composing relational, NoSQL, object storage, cache, and replication services into one data platform.
- Separating sources of truth from hot read paths and analytical replicas.
- Applying encryption, lifecycle management, least privilege, TTL, capacity planning, and operational evidence.
- Producing implementation artifacts, acceptance criteria, and a cleanup plan suitable for a portfolio project.

Scenario

A fast-growing e-commerce company is replacing a monolithic database with a multi-tier AWS backend. User profiles, billing methods, orders, refunds, and account status require relational transactions and belong in Aurora Serverless v2. Active shopping carts, clickstream events, recommendation impressions, and session metadata need key-value scale and belong in DynamoDB. Product images and exports belong in S3 with encryption and lifecycle controls. Read-heavy product and cart APIs need ElastiCache and DAX. Finance and analytics teams need a continuous copy of Aurora data in S3 for lake queries without impacting production.

Requirements

Functional requirements.

- Create an Aurora Serverless v2 cluster or document the exact cluster, database, subnet group, and security configuration required for profiles and billing.
- Create DynamoDB tables or a single-table design for active carts, cart items, clickstream events, session state, and recommendation impressions.
- Store product images in S3 with server-side encryption, public access blocked, versioning where appropriate, and lifecycle transition or expiration rules.
- Use ElastiCache to cache hot product detail and inventory summary reads sourced from Aurora or API composition.
- Use DAX for repeated DynamoDB cart or session reads where eventual consistency is acceptable.
- Configure AWS DMS to continuously replicate selected Aurora tables to S3 for analytics landing files.

Non-functional requirements.

- No source code, scripts, or documentation may contain secrets, passwords, personal credentials, or private keys.
- User and billing writes must favor correctness over cache freshness. Caches must be invalidated or bypassed for sensitive updates.
- Product image buckets must block public access unless a separate CloudFront or signed URL design is explicitly documented.
- Cart and clickstream keys must distribute traffic across users and time buckets rather than concentrate all writes on one global key.
- DMS replication must have monitoring for task status, latency, errors, and S3 object delivery.

Reference Architecture

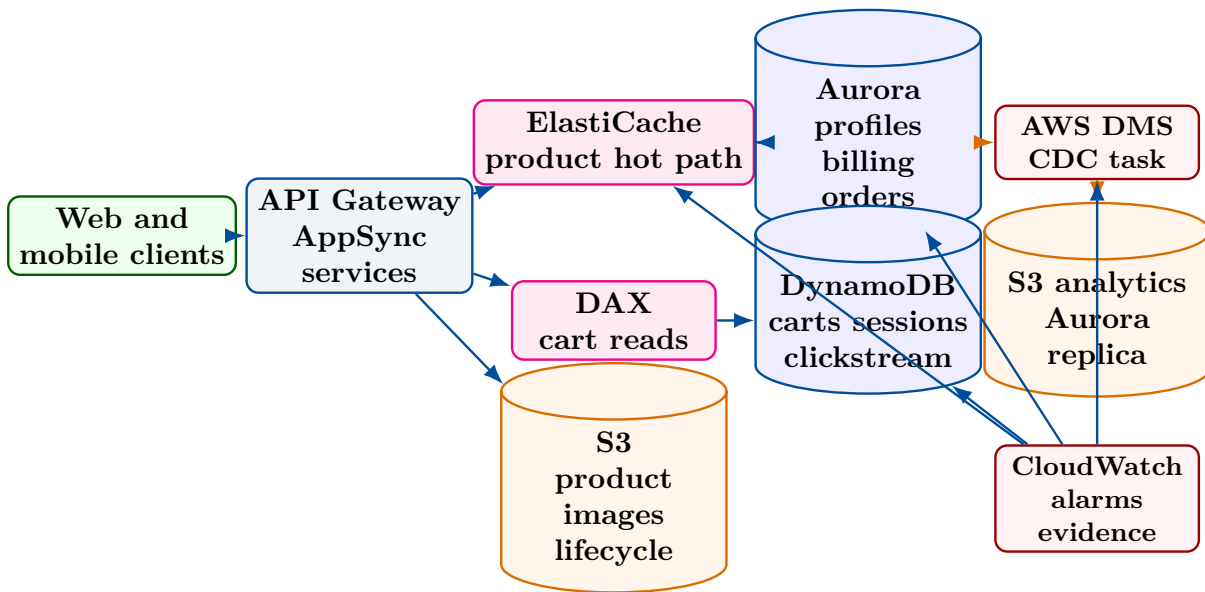


Figure 4.3: Milestone 1 e-commerce reference architecture integrating Aurora Serverless v2, DynamoDB, S3, ElastiCache, DAX, AWS DMS, and observability across all tiers.

Implementation Steps

1. **Define domains and ownership.** Mark Aurora as the source of truth for profiles, billing, orders, and refunds. Mark DynamoDB as the source for active carts, sessions, and clickstream events. Mark S3 as the source for product image objects and analytics landing files.
2. **Create storage foundations.** Configure S3 buckets with encryption, public access block, versioning, and lifecycle rules. Create Aurora Serverless v2 and DynamoDB resources using private networking and least-privilege IAM.
3. **Add acceleration.** Place ElastiCache in private subnets for product detail and inventory summary reads. Add DAX for repeated cart reads if eventual consistency is acceptable.
4. **Implement ingestion APIs.** Use API Gateway or AppSync to validate requests, authorize users, throttle clients, and route writes to Aurora, DynamoDB, or S3 pre-signed URL workflows.
5. **Replicate to analytics.** Configure DMS source and target endpoints, a replication instance or serverless task where available, table mappings, and continuous CDC from Aurora to S3.
6. **Validate failure modes.** Test cache miss behavior, stale cache invalidation, DAX bypass for strict reads, DynamoDB TTL behavior, S3 lifecycle policy syntax, and DMS task recovery.
7. **Capture evidence.** Save architecture diagrams, command output, resource descriptions, sample API calls, metrics screenshots, and cleanup commands.

```

1 $Region = "us-east-1"
2 $AccountSuffix = "123456789012"
3 $ImageBucket = "m1-ecommerce-product-images-$AccountSuffix"

```

```

4 $AnalyticsBucket = "m1-ecommerce-analytics-$AccountSuffix"
5 $CartTable = "m1-active-carts"
6 $SubnetGroupName = "m1-cache-subnets"
7 $ReplicationGroupId = "m1-product-cache"
8 aws s3api create-bucket --bucket $ImageBucket --region $Region
9 aws s3api put-public-access-block '
10   --bucket $ImageBucket '
11   --public-access-block-configuration '
12     BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,
13     RestrictPublicBuckets=true
14 aws s3api put-bucket-encryption '
15   --bucket $ImageBucket '
16   --server-side-encryption-configuration '
17     '{"Rules":[{"ApplyServerSideEncryptionByDefault":{"SSEAlgorithm":"AES256"
18     }}}'
19 aws s3api put-bucket-lifecycle-configuration '
20   --bucket $ImageBucket '
21   --lifecycle-configuration '
22     '{"Rules":[{"ID":"transition-old-images","Status":"Enabled","Filter":{"
23     Prefix":"products/"},"Transitions":[{"Days":90,"StorageClass":"STANDARD_IA
24     }]}]}'
25 aws s3api create-bucket --bucket $AnalyticsBucket --region $Region
26 aws dynamodb create-table '
27   --table-name $CartTable '
28   --attribute-definitions AttributeName=PK,AttributeType=S AttributeName=SK,
29   AttributeType=S '
30   --key-schema AttributeName=PK,KeyType=HASH AttributeName=SK,KeyType=RANGE '
31   --billing-mode PAY_PER_REQUEST '
32   --stream-specification StreamEnabled=true,StreamViewType=NEW_AND_OLD_IMAGES '
33   --sse-specification Enabled=true '
34   --region $Region
35 aws dynamodb wait table-exists --table-name $CartTable --region $Region
36 aws dynamodb update-time-to-live '
37   --table-name $CartTable '
38   --time-to-live-specification "Enabled=true,AttributeName=expiresAt" '
39   --region $Region
40 aws elasticache create-cache-subnet-group '
41   --cache-subnet-group-name $SubnetGroupName '
42   --cache-subnet-group-description "Milestone 1 cache subnets" '
43   --subnet-ids subnet-aaaa1111 subnet-bbbb2222 '
44   --region $Region
45 aws elasticache create-replication-group '
46   --replication-group-id $ReplicationGroupId '
47   --replication-group-description "Milestone 1 product cache" '
48   --engine redis '
49   --cache-node-type cache.t4g.micro '
50   --num-cache-clusters 2 '
51   --automatic-failover-enabled '
52   --cache-subnet-group-name $SubnetGroupName '
53   --security-group-ids sg-cache1234567890 '
54   --transit-encryption-enabled '
55   --at-rest-encryption-enabled '
56   --region $Region

```

Listing 4.3: PowerShell-oriented AWS CLI skeleton for S3, DynamoDB, and cache resources.

```

1 from datetime import datetime, timedelta, timezone
2 from decimal import Decimal
3 import os
4 import uuid
5 import boto3
6 region = os.environ.get("AWS_REGION", "us-east-1")
7 table = boto3.resource("dynamodb", region_name=region).Table(
8     os.environ.get("CART_TABLE", "m1-active-carts")

```

```

9 )
10 s3 = boto3.client("s3", region_name=region)
11 image_bucket = os.environ["IMAGE_BUCKET"]
12 def put_cart_item(user_id, sku, quantity, price):
13     now = datetime.now(timezone.utc)
14     ttl = int((now + timedelta(days=30)).timestamp())
15     table.put_item(
16         Item={
17             "PK": f"USER#{user_id}",
18             "SK": f"CART#{sku}",
19             "entityType": "CartItem",
20             "sku": sku,
21             "quantity": quantity,
22             "price": Decimal(str(price)),
23             "updatedAt": now.isoformat(),
24             "expiresAt": ttl,
25         }
26     )
27     return {"cacheInvalidationKey": f"cart:{user_id}", "sku": sku}
28 def record_click(user_id, sku, action):
29     now = datetime.now(timezone.utc)
30     event_id = str(uuid.uuid4())
31     day = now.strftime("%Y-%m-%d")
32     table.put_item(
33         Item={
34             "PK": f"CLICK#{day}#{user_id[-2:]}",
35             "SK": f"{now.isoformat()}#{event_id}",
36             "entityType": "ClickstreamEvent",
37             "userId": user_id,
38             "sku": sku,
39             "action": action,
40             "expiresAt": int((now + timedelta(days=7)).timestamp()),
41         }
42     )
43     return event_id
44 def create_product_image_upload(sku, content_type="image/jpeg"):
45     key = f"products/{sku}/{uuid.uuid4()}.jpg"
46     return s3.generate_presigned_url(
47         "put_object",
48         Params={"Bucket": image_bucket, "Key": key, "ContentType": content_type},
49         ExpiresIn=900,
50     )

```

Listing 4.4: Boto3 sketch for carts, clickstream events, and cache invalidation metadata.

Deliverables

- Architecture diagram showing clients, API layer, Aurora Serverless v2, DynamoDB, S3 product images, ElastiCache, DAX, DMS, S3 analytics, and monitoring.
- Data ownership matrix that identifies the source of truth, cache policy, TTL, encryption setting, backup or retention rule, and access pattern for each entity.
- AWS CLI or infrastructure scripts for S3 buckets, DynamoDB tables, ElastiCache, DAX or documented DAX plan, Aurora configuration, and DMS replication.
- Boto3 or service code demonstrating cart writes, clickstream ingestion, product image upload workflow, cache invalidation keys, and safe serialization.
- Evidence packet with resource descriptions, table schemas, bucket policies, lifecycle output, cache endpoints, DMS task status, CloudWatch metrics, and cleanup commands.

Acceptance Criteria

- Aurora Serverless v2 is the documented transactional source for profiles, billing, orders, and refunds.
- DynamoDB cart and clickstream keys distribute traffic by user, date, or bucket and include TTL where temporary data is expected.
- S3 product image storage blocks public access, uses server-side encryption, and has a lifecycle policy.
- ElastiCache is placed in private subnets and has a clear cache-aside or invalidation strategy for product and inventory reads.
- DAX is used only for DynamoDB eventually consistent read paths or is explicitly rejected with a reason.
- AWS DMS has a continuous replication design from Aurora to S3 with table mappings, monitoring, and recovery notes.
- API ingestion includes authentication, validation, throttling, and no direct client access to databases.
- The final design includes cleanup steps and cost controls for every billable resource.

Stretch Goals

- Add CloudFront with signed URLs or signed cookies for product image delivery without making the S3 bucket public.
- Implement a Lambda or stream consumer that invalidates ElastiCache keys after Aurora product updates.
- Add a synthetic load script that demonstrates database load reduction after cache warmup.
- Add Athena table definitions over the DMS S3 target and run a sample analytics query.
- Add multi-region disaster recovery notes for Aurora, DynamoDB global tables, S3 replication, and cache warmup.

Part II

Data Warehousing and Analytics

Amazon Redshift Architecture and Dimensional Modeling

Learning Objectives

- Explain Amazon Redshift’s massively parallel processing (MPP) architecture, the separation of leader and compute nodes, and the role of RA3 managed storage.
- Compare Redshift Serverless and provisioned RA3 clusters on pricing, scaling behavior, and workload fit.
- Load data with the COPY command and Auto-Copy from S3 using IAM role authentication, manifest files, and appropriate file formats.
- Apply Kimball dimensional modeling: declare the fact-table grain, choose among transaction, periodic snapshot, and accumulating snapshot facts, and design conformed dimensions.
- Implement slowly changing dimension (SCD) Type 1 and Type 2 logic in Redshift SQL, including MERGE-based upserts.
- Estimate monthly cost and recommend an architecture for a BI workload that is heavily utilized only during end-of-month reporting.

Cross-Cloud Equivalence

Week 5 introduces the cloud data warehouse. The MPP mental model—shared-nothing compute over distributed columnar storage—transfers directly to every other platform, even where the service hides it completely.

Capability	AWS	Azure	GCP
Cloud data warehouse	Amazon Redshift	Synapse dedicated SQL pools / Fabric Warehouse	BigQuery
Serverless warehouse	Redshift Serverless	Fabric capacity / Synapse serverless	BigQuery (default)
Managed storage layer	RA3 managed storage	ADLS-backed separation	Capacitor on Colossus
Bulk load command	COPY	COPY INTO / T-SQL bulk insert	LOAD DATA / bq load
Change-data upsert	MERGE	MERGE	MERGE
BI acceleration	Materialized views	Materialized views / semantic models	Materialized views, BI Engine
Pricing lever (provisioned)	Node-hours + managed storage TB	DWU-hours + storage	Flat-rate slots
Pricing lever (serverless)	RPU-hours	Capacity units (Fabric)	Bytes scanned / slots
Concurrency burst	Concurrency scaling	Scaling / workload groups	Autoscaling slots

On **Microsoft Fabric**, the warehouse and lakehouse share OneLake storage with Delta–Parquet tables, blurring the warehouse/lake boundary that Redshift and S3 keep explicit. **Snowflake** separates storage, virtual warehouses, and cloud services into three independently scaled layers—the closest conceptual cousin to RA3. **MongoDB** is an operational document store, not a warehouse; the comparison highlights why analytical star schemas live outside the OLTP system of record.

5.1 Week 5 Roadmap and Mental Model

Amazon Redshift is the analytical centerpiece of most AWS data platforms. Where S3 stores objects and DynamoDB serves millisecond key-value lookups, Redshift answers a different question: how do you aggregate, join, and scan terabytes of structured data with interactive SQL? The answer is a columnar, massively parallel processing engine whose architecture you must understand before you can model, load, or tune it [22].

The mental model for the week has three layers. First, the *physical layer*: a Redshift cluster or workgroup is a set of compute nodes that own slices of data and execute query fragments in parallel. Second, the *logical layer*: a dimensional star schema—fact tables at a declared grain surrounded by conformed dimensions—gives that engine a workload it is good at [92]. Third, the *economic layer*: Serverless versus provisioned RA3 is a capacity-purchasing decision that should follow your utilization curve, not fashion. Chapters 5 and 6 move through these layers in order; this chapter covers architecture, loading, and modeling, and Chapter 6 covers performance tuning.

Definition

Amazon Redshift is a fully managed, petabyte-scale cloud data warehouse. Data is stored in a compressed columnar format distributed across compute nodes; SQL queries are compiled into parallel execution plans whose fragments run on the nodes that hold the relevant data slices.

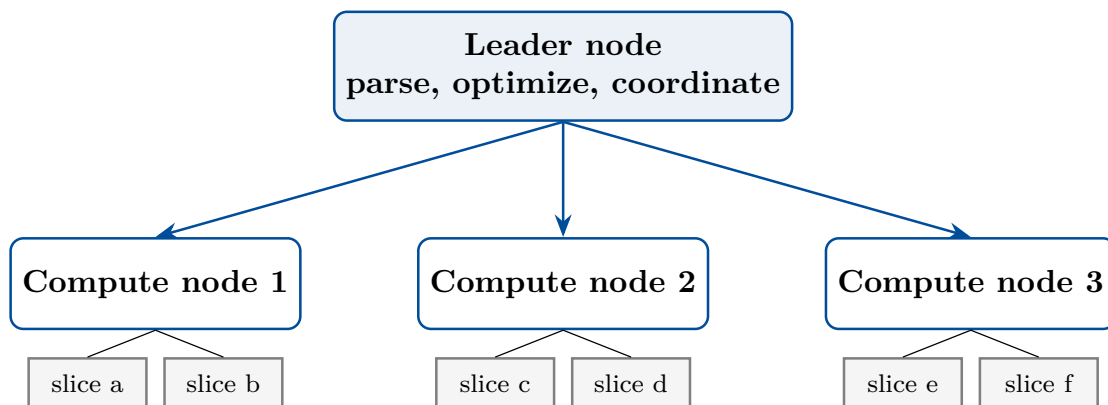
Key Terms

MPP: massively parallel processing; each node processes its local data slice in parallel with all other nodes. **Leader node**: plans and coordinates queries, returns results. **Compute node**: stores data slices and executes query fragments. **RA3**: node family with managed storage that scales independently of compute. **RPU**: Redshift Processing Unit, the capacity unit of Redshift Serverless. **COPY**: the bulk-load command that ingests files from S3 in parallel. **Grain**: the precise business definition of one row of a fact table. **SCD**: slowly changing dimension; Type 1 overwrites history, Type 2 preserves it with versioned rows.

5.2 Monday: Redshift MPP Architecture

5.2.1 Shared-Nothing Parallelism

Redshift is a shared-nothing MPP database. A table's rows are divided into *slices*, and each slice lives on one compute node. When you submit a query, the leader node parses and optimizes it, compiles an execution plan, and ships compiled code segments to the compute nodes. Each node runs its segments against its local slices, intermediate results flow across the cluster interconnect for joins and aggregations, and the leader assembles the final result [22]. The practical consequence: query speed is bounded by how evenly data is distributed and how little of it must move across the network—topics Chapter 6 develops in depth.



Each compute node owns data slices; the leader never stores user data.

Figure 5.1: Redshift MPP architecture: the leader node plans and coordinates; compute nodes own data slices and execute query fragments in parallel.

5.2.2 Columnar Storage and Zone Maps

Within a slice, Redshift stores each column separately in 1 MB blocks. Columnar layout means a query that touches three of a table's two hundred columns reads only the blocks for those three columns. Each block carries metadata—minimum and maximum values—that Redshift uses as a *zone map* to skip blocks that cannot contain matching rows. Compression encodings reduce storage and I/O further, and because I/O is usually the bottleneck in analytical scans, compression is effectively a performance feature, not merely a cost feature.

Tip

Run **ANALYZE** after large loads and let **AUTO** table properties stay enabled unless you have measured a reason to override them. Modern Redshift automates much of the encoding, sort-key, and vacuum work that older documentation describes as manual chores.

5.2.3 RA3 Nodes and Managed Storage

RA3 nodes decouple compute from storage. Hot data lives on local NVMe solid-state cache; the durable copy lives in Redshift Managed Storage, which is backed by S3 and scales to petabytes independently of the node count [22]. This matters for three reasons. First, you can scale compute without redistributing the entire dataset. Second, you pay for managed storage per terabyte-month, so keeping historical data in the warehouse is cheaper than on older dense-storage node types. Third, RA3 enables data sharing and cross-AZ relocation features that later chapters use.

5.3 Tuesday: Redshift Serverless Versus Provisioned

5.3.1 Two Ways to Buy Capacity

Provisioned Redshift bills per node-hour for a cluster you size, start, pause, and resize. Redshift Serverless bills in Redshift Processing Unit (RPU) hours, per second, with the workgroup scaling capacity up and down automatically between a base and maximum RPU setting [24]. Neither model is universally cheaper; the right choice is an arithmetic function of your utilization curve.

Definition

Redshift Serverless provisions capacity automatically: you set base and maximum RPUs, and the service scales compute to query demand, billing per RPU-second. **Provisioned RA3** allocates a fixed node count that you manage, billed per node-hour while running.

Dimension	Redshift Serverless	Provisioned RA3
Billing unit	RPU-hours, per-second	Node-hours + managed storage per TB-month
Best fit	Spiky, intermittent, or unpredictable workloads; fast onboarding	Steady 24/7 workloads at scale with predictable load
Scaling	Automatic between base and max RPUs	Manual resize, elastic resize, pause/resume
Watch-out	Base-RPU floor runs 24/7; unbounded max RPU spikes the bill	Idle capacity is paid for; resizing is an operational event
Controls	Usage limits and query monitoring rules	WLM queues, concurrency scaling limits, pause schedules

Table 5.1: Serverless versus provisioned Redshift as an economic and operational decision.

5.3.2 Reading the Utilization Curve

A workload that runs queries eight hours a month—end-of-month reporting, quarterly close, occasional audit extracts—cannot justify a cluster that bills 730 hours a month. A workload that keeps a cluster at 60–80% CPU all day every day usually pays less on provisioned capacity, because steady utilization amortizes the idle-capacity premium that Serverless charges for

elasticity. The Friday use case works this arithmetic end to end.

Common Pitfall

Leaving the Serverless maximum RPU unbounded. A runaway dashboard refresh or an unpartitioned cross join can scale the workgroup to its ceiling and burn the month’s budget in an afternoon. Always set a maximum RPU and usage limits, and alarm on RPU-hours in CloudWatch.

5.3.3 The Broader Analytical Pricing Landscape

Redshift is not the only way to run SQL over your data on AWS, and the warehouse decision should be made against the alternatives. Table 5.2 summarizes the four options this part of the book uses repeatedly.

Option	Billing model	Best for	Watch-outs
Redshift Serverless	RPU-hours, billed per second	Spiky or intermittent BI workloads, fast on-boarding	Base-RPU floor cost; unbounded concurrency spikes the bill
Redshift RA3 (provisioned)	Node-hours plus managed storage per TB-month	Steady, predictable 24/7 analytics at scale	Paying for idle capacity; re-size/pause planning needed
Athena	Per TB scanned	Ad-hoc SQL on S3 with zero infrastructure	Unpartitioned CSV scans explode cost; no indexes
Redshift Spectrum	Per TB scanned, plus a running Redshift cluster	Joining S3 lake data with Redshift tables in one query	Requires an active cluster; per-TB fees accumulate on wide scans

Table 5.2: Analytical engines on AWS compared by billing model and fit. Chapter 7 covers Athena and Spectrum in depth.

5.4 Wednesday: Loading Data and Kimball Dimensional Modeling

5.4.1 COPY: The Parallel Bulk Loader

The COPY command is the only sane way to load serious volume into Redshift. It reads files from S3 (or DynamoDB, EMR, or remote hosts via SSH) in parallel across compute nodes, so each node ingests the files assigned to its slices [69]. Three rules govern COPY performance. First, split input into multiple files—a single 100 GB file serializes the load on one node, while one hundred 1 GB files parallelize it. Second, prefer columnar or compressible formats: Parquet and ORC map directly onto Redshift’s columnar layout, and COPY can read JSON, CSV, and Avro when needed. Third, authenticate with an IAM role attached to the cluster or workgroup, never with embedded access keys.

```

1 COPY staging.sales_raw
2 FROM 's3://acme-analytics-raw/manifests/sales_2026_08.manifest'
3 IAM_ROLE 'arn:aws:iam::123456789012:role/RedshiftS3ReadRole'
4 FORMAT AS PARQUET
5 MANIFEST;
```

Listing 5.1: COPY with an IAM role, a manifest, and Parquet input.

5.4.2 Auto-Copy for Continuous Micro-Batches

COPY is a command you run; Auto-Copy is a job you declare. A copy job attached to an S3 prefix monitors for new objects and loads them automatically, turning micro-batch ingestion into a configuration rather than a scheduled script [69]. Auto-Copy suits continuously arriving files—vendor drops, CDC output from DMS (Chapter 11), Firehose delivery (Chapter 14)—where the alternative is an EventBridge-triggered Lambda that issues COPY on every upload. Reserve manual COPY for controlled bulk loads and backfills where you want explicit transaction boundaries.

Note

A manifest file is the safe way to COPY a precise set of objects. Prefix-based COPY picks up whatever matches at run time; a manifest pins the load to an explicit file list with optional mandatory flags, which makes retries and audits deterministic.

5.4.3 Declare the Grain First

Kimball dimensional modeling begins with a discipline that feels bureaucratic and pays for itself forever: before writing any DDL, state the grain of the fact table in one sentence [92]. “One row per order line item” and “one row per order” are different tables with different measures, different dimensions, and different correct answers to the same business question. Every column added to a fact table must be true at the declared grain; a column that is only meaningful at a coarser grain (order-level shipping cost on an order-line fact) invites double-counting.

5.4.4 Three Fact Table Types

- **Transaction facts** record one row per business event—a sale, a click, a swipe—at the lowest useful grain. They are the largest and most flexible facts, answering any question that aggregates events.
- **Periodic snapshot facts** record one row per entity per period—daily account balance, monthly inventory level. They answer “state at a point in time” questions that transaction facts answer awkwardly.
- **Accumulating snapshot facts** record one row per workflow instance—an order, a claim, a shipment—updated as the workflow passes milestones (ordered, shipped, delivered, returned). Multiple date foreign keys and overwritten milestone columns are normal here.

5.4.5 Conformed Dimensions and the Bus Matrix

A *conformed dimension* is shared, identical, and governed across data marts: `dim_customer` means the same thing in the sales mart and the support mart. Conformance is what makes cross-mart queries and enterprise consistency possible. The design tool is the bus matrix: rows are business processes (facts), columns are shared dimensions, and the matrix forces agreement on which dimensions each process uses before anyone writes SQL.

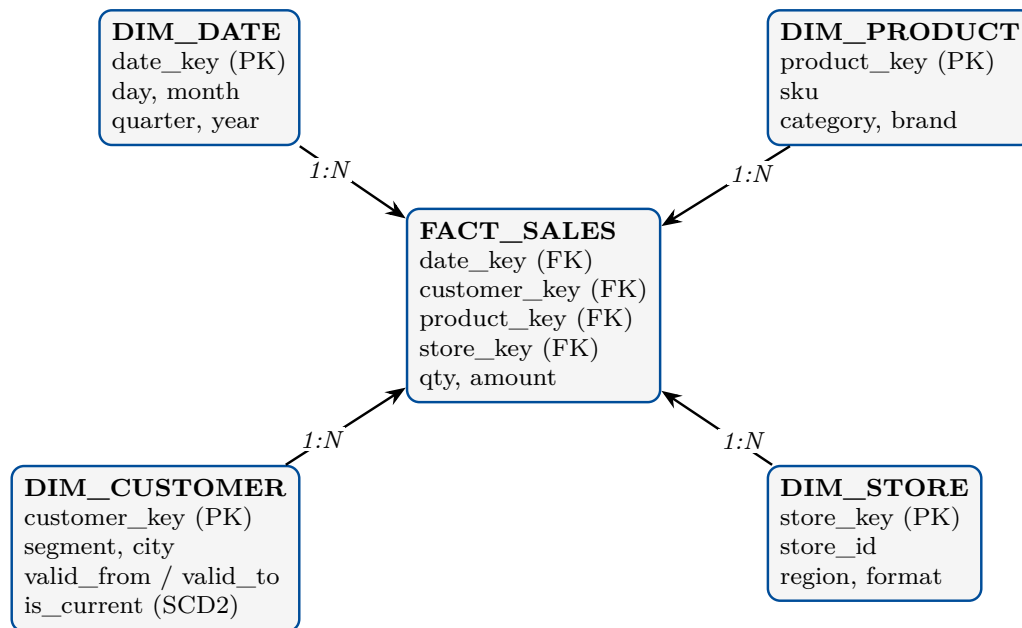


Figure 5.2: Star schema for the Thursday project: `fact_sales` at the order-line grain with four conformed dimensions.

5.4.6 Slowly Changing Dimensions

Dimension attributes change: customers move cities, products change categories. **SCD Type 1** overwrites the attribute—simple, but history is lost and last quarter’s report now restates under today’s values. **SCD Type 2** inserts a new row with effective dates (`valid_from`, `valid_to`) and an `is_current` flag, preserving exactly what the organization knew at any point in time. Choose per attribute, not per table: correcting a typo in a customer name is Type 1; tracking segment reassignment for commission calculation is Type 2.

Redshift’s `MERGE` statement makes Type 2 upserts expressible in two passes: close the rows whose tracked attributes changed, then insert the new current versions.

```

1  -- SCD Type 2 upsert in Redshift: close changed rows, then insert new versions
2  MERGE INTO dim_customer tgt
3  USING stg_customer src
4  ON tgt.customer_id = src.customer_id AND tgt.is_current = TRUE
5  WHEN MATCHED AND (tgt.segment <> src.segment OR tgt.city <> src.city)
6  THEN UPDATE SET valid_to = CURRENT_DATE, is_current = FALSE
7  WHEN NOT MATCHED
8  THEN INSERT (customer_id, segment, city, valid_from, valid_to, is_current)
9  VALUES (src.customer_id, src.segment, src.city,
10          CURRENT_DATE, DATE '9999-12-31', TRUE);
11
12 -- second pass: insert the new current versions of the rows just closed
13 INSERT INTO dim_customer (customer_id, segment, city, valid_from, valid_to,
14                             is_current)
15 SELECT src.customer_id, src.segment, src.city,
16        CURRENT_DATE, DATE '9999-12-31', TRUE
17 FROM stg_customer src
18 JOIN dim_customer tgt
19 ON tgt.customer_id = src.customer_id
20 AND tgt.is_current = FALSE
21 AND tgt.valid_to = CURRENT_DATE;

```

Listing 5.2: SCD Type 2 upsert in Redshift: close changed rows, then insert new versions.

Common Pitfall

SCD Type 2 on every attribute. Versioning a dimension on every change—including corrected typos—explodes row counts and complicates every query with effective-date joins. Reserve Type 2 for attributes whose history has analytical or regulatory value, and document the choice per attribute in the data dictionary.

5.5 Thursday Hands-on Project: Load a Star Schema into Redshift Serverless

This project builds the working core of a dimensional warehouse: a Redshift Serverless workgroup, a ten-million-row fact load from S3 with COPY, and three conformed dimensions around a declared grain.

Lab Objectives

- Provision a Redshift Serverless endpoint with an attached IAM role that can read an S3 staging bucket.
- Generate or obtain ten million rows of synthetic order-line JSON in S3, split across multiple files.
- Declare the grain of `fact_sales`, create the table plus `dim_customer`, `dim_product`, and `dim_date`, and load them with COPY.
- Practice the SCD Type 2 MERGE from listing 5.2 against a staged customer extract.

Procedure

Create the Serverless workgroup and namespace with the AWS CLI, then grant S3 read access through an IAM role. Replace every placeholder before running.

```

1 # PowerShell: set variables first, then run the CLI calls.
2 $Region      = "us-east-1"
3 $AccountId   = "123456789012"
4 $RoleName    = "RedshiftS3ReadRole"
5 $Bucket      = "acme-analytics-raw-123456789012"
6 $Namespace   = "week5-ns"
7 $Workgroup   = "week5-wg"
8
9 # Trust policy: allow Redshift to assume the role.
10 $TrustJson = @"
11 {
12   "Version": "2012-10-17",
13   "Statement": [{
14     "Effect": "Allow",
15     "Principal": { "Service": "redshift.amazonaws.com" },
16     "Action": "sts:AssumeRole"
17   }]
18 }
19 @"
20 $TrustJson | Set-Content -Path trust.json -Encoding ascii
21
22 aws iam create-role --role-name $RoleName `
23   --assume-role-policy-document file://trust.json
24
25 # Least-privilege read on the staging bucket only.
26 $PolicyJson = @"
27 {

```

```

28 "Version": "2012-10-17",
29 "Statement": [{
30     "Effect": "Allow",
31     "Action": ["s3:GetObject", "s3:ListBucket"],
32     "Resource": [
33         "arn:aws:s3:::$Bucket",
34         "arn:aws:s3:::$Bucket/*"
35     ]
36 }]
37 }
38 "@
39 $PolicyJson | Set-Content -Path s3-read.json -Encoding ascii
40
41 aws iam put-role-policy --role-name $RoleName '
42     --policy-name ReadAnalyticsRaw --policy-document file:///s3-read.json
43
44 aws redshift-serverless create-namespace '
45     --namespace-name $Namespace '
46     --iam-roles "arn:aws:iam::${AccountId}:role/$RoleName"
47
48 aws redshift-serverless create-workgroup '
49     --workgroup-name $Workgroup '
50     --namespace-name $Namespace '
51     --base-capacity 8 '
52     --max-capacity 32 '
53     --publicly-accessible

```

Listing 5.3: Provisioning a Redshift Serverless workgroup with an IAM role.

Generate the synthetic fact data locally and upload it as multiple files so COPY parallelizes. The generator below writes ten files of one million JSON lines each.

```

1 import json
2 import random
3 from datetime import date, timedelta
4
5 import boto3
6
7 BUCKET = "acme-analytics-raw-123456789012"
8 s3 = boto3.client("s3", region_name="us-east-1")
9
10 random.seed(42)
11 base = date(2026, 1, 1)
12
13 def make_row(i: int) -> dict:
14     return {
15         "order_line_id": i,
16         "order_date": str(base + timedelta(days=random.randint(0, 240))),
17         "customer_id": random.randint(1, 500_000),
18         "product_id": random.randint(1, 50_000),
19         "store_id": random.randint(1, 2_000),
20         "qty": random.randint(1, 5),
21         "amount": round(random.uniform(5.0, 500.0), 2),
22     }
23
24 row_id = 0
25 for part in range(10):
26     lines = []
27     for _ in range(1_000_000):
28         row_id += 1
29         lines.append(json.dumps(make_row(row_id)))
30     s3.put_object(
31         Bucket=BUCKET,
32         Key=f"raw/sales/part-{part:02d}.json",

```



```

33     Body="\n".join(lines).encode("utf-8"),
34 )
35 print(f"uploaded part {part}")

```

Listing 5.4: Generating ten million synthetic order-line rows and uploading to S3.

With the data staged, declare the schema. The grain statement is a design artifact: write it in a comment above the DDL so reviewers and future maintainers inherit it.

```

1  -- GRAIN: one row per order line item (order x product), as sold.
2  -- Every column below must be true at that grain.
3  CREATE TABLE dim_date (
4      date_key    INTEGER PRIMARY KEY,    -- yyyymmdd
5      day         SMALLINT,
6      month       SMALLINT,
7      quarter     SMALLINT,
8      year        SMALLINT
9  );
10
11 CREATE TABLE dim_customer (
12     customer_key BIGINT IDENTITY(1,1) PRIMARY KEY,
13     customer_id  BIGINT NOT NULL,        -- natural key from source
14     segment      VARCHAR(32),
15     city         VARCHAR(64),
16     valid_from   DATE,
17     valid_to     DATE,
18     is_current   BOOLEAN
19 );
20
21 CREATE TABLE dim_product (
22     product_key BIGINT IDENTITY(1,1) PRIMARY KEY,
23     product_id  BIGINT NOT NULL,
24     sku         VARCHAR(32),
25     category    VARCHAR(64),
26     brand       VARCHAR(64)
27 );
28
29 CREATE TABLE dim_store (
30     store_key BIGINT IDENTITY(1,1) PRIMARY KEY,
31     store_id  INTEGER NOT NULL,
32     region    VARCHAR(32),
33     format    VARCHAR(32)
34 );
35
36 CREATE TABLE fact_sales (
37     order_line_id BIGINT,
38     date_key       INTEGER REFERENCES dim_date(date_key),
39     customer_key   BIGINT REFERENCES dim_customer(customer_key),
40     product_key    BIGINT REFERENCES dim_product(product_key),
41     store_key      BIGINT REFERENCES dim_store(store_key),
42     qty            INTEGER,
43     amount         DECIMAL(12,2)
44 );

```

Listing 5.5: Star schema DDL with the grain declared as a design comment.

Load the staging table and dimensions, then the fact. COPY with JSONPaths handles the raw JSON; production loads should prefer Parquet.

```

1  CREATE TABLE staging.sales_raw (
2      order_line_id BIGINT, order_date DATE, customer_id BIGINT,
3      product_id BIGINT, store_id INTEGER, qty INTEGER, amount DECIMAL(12,2)
4  );
5

```

```

6 COPY staging.sales_raw
7 FROM 's3://acme-analytics-raw-123456789012/raw/sales/'
8 IAM_ROLE 'arn:aws:iam::123456789012:role/RedshiftS3ReadRole'
9 FORMAT AS JSON 'auto'
10 REGION 'us-east-1';
11
12 -- Load dimensions from curated sources (omitted), then resolve keys:
13 INSERT INTO fact_sales
14 SELECT s.order_line_id,
15        d.date_key,
16        c.customer_key,
17        p.product_key,
18        st.store_key,
19        s.qty,
20        s.amount
21 FROM staging.sales_raw s
22 JOIN dim_date d      ON d.date_key = TO_CHAR(s.order_date, 'YYYYMMDD')::INT
23 JOIN dim_customer c  ON c.customer_id = s.customer_id AND c.is_current
24 JOIN dim_product p   ON p.product_id = s.product_id
25 JOIN dim_store st    ON st.store_id = s.store_id;
26
27 ANALYZE fact_sales;

```

Listing 5.6: Loading the fact table from staged JSON with COPY.

Verification

- `SELECT COUNT(*) FROM fact_sales;` returns ten million.
- `SELECT COUNT(*) FROM fact_sales f LEFT JOIN dim_customer c ON f.customer_key = c.customer_key WHERE c.customer_key IS NULL;` returns zero—no orphan keys.
- `SELECT date_key, SUM(amount) FROM fact_sales GROUP BY 1 ORDER BY 1 LIMIT 5;` completes in seconds, demonstrating the MPP scan.
- Running the SCD Type 2 `MERGE` twice against the same staged extract is idempotent: the second run closes and inserts nothing.

Tip

Redshift Serverless bills RPU-seconds whenever the workgroup is active. Set the workgroup's idle timeout and a low base capacity for the lab, and check AWS/Billing or the Serverless usage dashboard after the load to see what ten million rows actually cost—usually pennies.

5.6 Friday Use Case: Costing a Month-End BI Workload

A retailer's BI platform runs heavy workloads only during the first three business days of each month, when 120 analysts refresh close-of-month dashboards and run allocation queries against a 4 TB warehouse. The rest of the month sees a handful of ad-hoc queries per day. The finance team asks for a cost estimate and an architecture recommendation: provisioned RA3 or Serverless?

5.6.1 Requirements and Assumptions

Assume the heavy window needs the equivalent of 32 RPUs for about 10 hours per day across 3 days (300 RPU-hours), and the quiet weeks need about 2 RPU-hours per day across 27 days

(54 RPU-hours). Managed storage holds 4 TB. Current published prices are illustrative; the point is the method, so substitute live pricing before committing to a decision.

5.6.2 The Arithmetic

Serverless. Compute is billed per RPU-second; at an illustrative \$0.375 per RPU-hour, the monthly compute is roughly $(300 + 54) \times \$0.375 \approx \133 , plus a base-RPU floor if the workgroup must stay warm for sub-second dashboard latency. Managed storage is billed per TB-month regardless of model: $4 \times \$24 \approx \96 at an illustrative rate. Total: roughly \$230–\$500 per month depending on the base capacity floor.

Provisioned RA3. The smallest configuration that comfortably serves the peak might be two ra3.xlplus nodes. At an illustrative \$1.086 per node-hour each, a month is $2 \times 730 \times \$1.086 \approx \$1,586$ plus the same managed storage. Pausing the cluster outside the heavy window reduces this, but pausing makes the warehouse unavailable for the ad-hoc queries, and resume takes minutes—a poor trade for a workload with daily light use.

Recommendation. Serverless wins for this utilization curve by a factor of roughly three to six, because the workload’s defining property is idle time. The answer flips when utilization climbs: a workload that sustains 32 RPUs around the clock bills about $32 \times 730 \times \$0.375 \approx \$8,760$ on Serverless, against roughly \$1,586 for the equivalent provisioned capacity.

Cost driver	Serverless estimate	Provisioned RA3 estimate
Peak capacity	32 RPUs, auto-scaled	$2 \times \text{ra3.xlplus}$
Monthly compute	~\$133 (354 RPU-hours)	~\$1,586 (always on)
Managed storage (4 TB)	~\$96	~\$96
Idle-time exposure	Base-RPU floor only	Full node-hours
Decision driver	Intermittent utilization	—

Table 5.3: Illustrative month-end BI cost comparison. Substitute live pricing and measured RPU consumption before committing.

5.6.3 Architecture Decisions

Capacity controls. Set base capacity to the minimum that keeps dashboard queries snappy (4–8 RPUs), maximum capacity to 32–48 RPUs to bound the month-end spike, and a usage limit with an alarm at 80% of the monthly RPU budget.

Data layout. The month-end dashboards aggregate the same dozen queries; Chapter 6’s materialized views and sort keys will cut the RPU consumption of those queries far more cheaply than any capacity choice.

Governance. Attach a query monitoring rule that aborts individual queries exceeding a cost threshold, so one unfiltered cross join cannot consume the month-end budget.

Common Pitfall

Recommending architecture from peak load alone. The correct inputs are the full utilization curve, the latency requirements during idle periods, and the operational cost of pause/resume or resize. A peak-driven recommendation over-provisions by the ratio of idle to busy hours.

5.7 Interview Q&A

5.7.1 Concepts and Trade-offs

1. **Q: Why must you declare the grain before adding columns to a fact table?**
A: Because every column and measure must be true at exactly one row of the fact. Without a declared grain, order-level and line-level measures mix, and every aggregate risks double-counting.
2. **Q: How do periodic snapshot and accumulating snapshot facts differ?**
A: A periodic snapshot records state at regular intervals (daily balances) and rows are never updated. An accumulating snapshot tracks one workflow instance (an order) and updates milestone columns as the process advances.
3. **Q: When is SCD Type 2 required instead of Type 1?**
A: When the analytical or regulatory question depends on what the attribute value was at the time of the event—commission by historical segment, pricing by historical category. Pure corrections use Type 1.
4. **Q: COPY versus Auto-Copy: which suits continuous micro-batch loads from S3?**
A: Auto-Copy. It monitors a prefix and loads new objects as they land, removing the scheduler. Manual COPY suits controlled bulk loads where you want explicit transactions and manifests.
5. **Q: When does Redshift Serverless beat provisioned RA3 on cost?**
A: When utilization is spiky or intermittent—idle hours dominate the month. Steady 24/7 workloads amortize provisioned node-hours better than per-second RPU billing.
6. **Q: What does the leader node do during a COPY?**
A: It plans the load and distributes input files across compute nodes; each node ingests its assigned files into its own slices in parallel. The leader never stores user data.

5.8 Further Reading

Start with the Amazon Redshift Database Developer Guide for architecture, COPY, and table design [22], and the Redshift Serverless documentation for RPU billing, base and maximum capacity, and usage limits [24]. The COPY reference covers formats, manifests, IAM authentication, and Auto-Copy jobs [69]. For dimensional modeling, Kimball and Ross remains the canonical text: grain declaration, fact types, conformed dimensions, and SCD handling are all treated in depth [92]. The AWS Well-Architected Analytics Lens frames the cost and reliability review questions used in the Friday use case [3].

Key Takeaways

- Redshift is a shared-nothing MPP, columnar warehouse: the leader plans, compute nodes own slices, and performance follows data distribution and scan minimization.
- RA3 decouples compute from managed storage; Serverless decouples you from cluster management at a per-RPU-second price.
- Serverless versus provisioned is an arithmetic decision over the utilization curve, with capacity limits as the safety control either way.

- COPY with multiple input files, columnar formats, manifests, and IAM roles is the correct bulk-load pattern; Auto-Copy automates micro-batch arrival.
- Declare the fact grain first, choose the fact type deliberately, conform dimensions across marts, and apply SCD Type 2 only where history has value.
- Month-end-style intermittent BI workloads are the canonical Serverless win; model cost before recommending.

5.9 Exercises

1. **(Conceptual)** Explain why a single 100 GB input file loads more slowly with COPY than one hundred 1 GB files on a multi-node cluster.
2. **(Conceptual)** A product's `list_price` is corrected after a typo, and its `category` is reassigned quarterly for reporting. Which attributes take SCD Type 1 and which Type 2, and why?
3. **(Conceptual)** Your CFO asks why the Serverless bill doubled while query count stayed flat. List three plausible causes and the CloudWatch metric you would check for each.
4. **(Hands-on)** Run the Thursday lab, then rerun the COPY command immediately. Inspect `stl_load_commits` and explain what happens to duplicate loads and how you would make the pipeline idempotent.
5. **(Hands-on)** Convert the ten million JSON rows to Parquet with a Glue job or local PyArrow script, reload with `FORMAT AS PARQUET`, and compare load times and storage.
6. **(Design)** Declare the grain and choose fact types for: (a) a ride-sharing trip record, (b) daily warehouse inventory, (c) an insurance claim lifecycle. Sketch the dimension lists.
7. **(Design)** Extend fig. 5.2 with a promotion dimension and a returns fact. State each new grain and identify which dimensions conform.
8. **(Architecture)** Rebuild the Friday cost model with live prices from the AWS pricing pages. At what sustained utilization does provisioned RA3 become cheaper than Serverless at your workload's peak?

5.10 Capstone Project: A Dimensional Retail Warehouse on Redshift Serverless

Capstone Project

Mission. Build a governed, cost-controlled dimensional warehouse for a retail chain on Redshift Serverless: land five years of order data from S3, model it as a star schema with declared grains, implement SCD Type 2 customers, and produce a defensible monthly cost model.

Estimated time. 4–6 hours in a sandbox AWS account, plus optional time for the cost model.

What you'll practice.

- Provisioning Redshift Serverless with base/maximum RPU controls and least-privilege IAM.
- Bulk loading with COPY, manifests, and columnar formats.
- Kimball modeling: grain declaration, transaction and snapshot facts, conformed di-

mensions, SCD Type 2.

- Building a Serverless-versus-RA3 cost model from a measured utilization curve.

Scenario

A retail chain with 2,000 stores is consolidating five years of order-line history—about ten million rows per year—into a warehouse for merchandising and finance analysts. Source systems drop nightly JSON extracts into an S3 landing bucket. The finance team wants month-end close dashboards, merchandising wants ad-hoc category analysis, and both want consistent customer and product definitions. Utilization is heavily month-end-weighted, so cost discipline is part of the deliverable.

Requirements

Functional requirements.

- Provision a Redshift Serverless namespace and workgroup with base capacity of 8 RPUs, maximum capacity of 64 RPUs, and an IAM role limited to the landing bucket.
- Land nightly extracts in `s3://<bucket>/raw/sales/` as multiple JSON or Parquet files and load them with COPY; document why Auto-Copy was or was not chosen for the nightly drop.
- Model `fact_sales` at the order-line grain with conformed `dim_date`, `dim_customer` (SCD Type 2), `dim_product`, and `dim_store`.
- Implement the SCD Type 2 MERGE for `dim_customer` and demonstrate idempotent re-runs.
- Provide a month-end reporting query set (five queries) that the cost model uses as its load profile.

Non-functional requirements.

- No credentials in code: IAM role authentication for COPY and workgroup access only.
- Usage limits and a CloudWatch alarm on RPU-hours at 80% of the monthly budget.
- Every fact table carries its grain declaration as a header comment; every SCD decision is documented per attribute.
- The cost model states assumptions, uses measured RPU consumption from the lab, and names the utilization level at which provisioned RA3 would win.

Reference Architecture

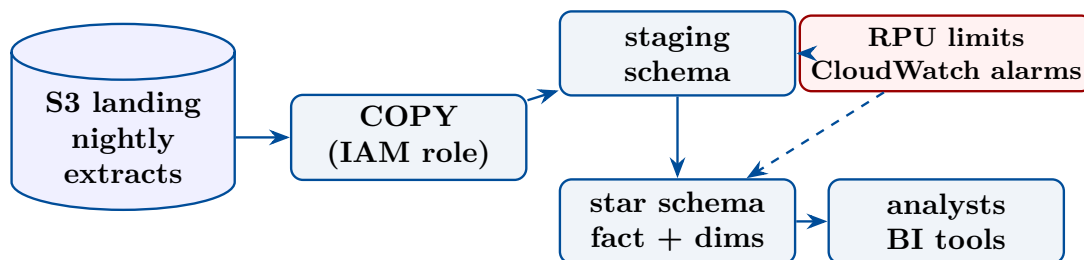


Figure 5.3: Capstone reference architecture: S3 landing, COPY into a staging schema, star-schema marts in Redshift Serverless, with RPU guardrails.

Implementation Steps

1. **Provision.** Create the namespace, workgroup, and IAM role from listing 5.3; record the workgroup endpoint.
2. **Land and stage.** Generate five years of synthetic order lines (reuse listing 5.4 with a wider date range) and upload as at least ten files per year.
3. **Model.** Write the DDL from listing 5.5 with grain comments; load `dim_date` with a calendar generator and the other dimensions from curated sources.
4. **Load and resolve.** COPY the extracts to staging, resolve surrogate keys into `fact_sales`, and run ANALYZE.
5. **Version customers.** Apply the SCD Type 2 MERGE from listing 5.2 to a changed-customer extract; verify closure and reinsertion.
6. **Measure.** Run the five month-end queries, record RPU-seconds from the Serverless usage dashboard, and extrapolate the monthly bill.
7. **Decide.** Write the one-page cost memo: Serverless versus RA3 at your measured utilization, with the crossover point.

Deliverables

Submit a concise engineering packet containing:

- A one-page architecture summary with the workgroup configuration, capacity limits, and the declared grain of every fact table.
- All DDL, the COPY commands, and the SCD Type 2 MERGE with evidence of an idempotent second run.
- Row-count and orphan-key verification output for every fact load.
- The five-query month-end workload and its measured RPU consumption.
- The cost memo with live pricing, assumptions, and the Serverless-to-RA3 crossover utilization.
- A cleanup plan for the workgroup, namespace, IAM role, and staging objects.

Acceptance Criteria

- The workgroup has explicit base and maximum RPU settings and a usage limit with an alarm.
- The IAM role used by COPY can read only the landing bucket, demonstrated by an access-denied test against another prefix.
- Every fact table carries a grain comment, and every column is defensible at that grain.
- `dim_customer` implements SCD Type 2 with `valid_from`, `valid_to`, and `is_current`, and a rerun of the MERGE changes nothing.
- Fact loads show zero orphan foreign keys after surrogate-key resolution.
- The cost memo compares Serverless and RA3 with measured—not assumed—RPU consumption and names the crossover utilization.
- No credentials or access keys appear in any submitted artifact.

Stretch Goals

- Convert the nightly extracts to partitioned Parquet and quantify the COPY time and scanned-bytes improvement.
- Add a periodic snapshot fact for daily store inventory and an accumulating snapshot for the order lifecycle; document the grain of each.
- Configure Auto-Copy on the landing prefix and compare its operational simplicity and latency against scheduled COPY.
- Add a bus matrix covering sales, returns, and inventory processes with conformed dimensions marked.
- Build the same schema on a small provisioned RA3 cluster, measure identical workloads on both, and validate the cost model empirically.
- Introduce a data-sharing datashare from this warehouse to a second Serverless workgroup and document the governance implications (preview of Chapter 8).

Redshift Optimization and Performance Modeling

Learning Objectives

- Choose distribution styles (EVEN, KEY, ALL, AUTO) that minimize data movement for join-heavy workloads.
- Distinguish compound and interleaved sort keys, select compression encodings, and reason about zone-map pruning.
- Configure workload management (WLM), concurrency scaling, and query monitoring rules for mixed BI workloads.
- Decide when a materialized view beats a plain view or a dashboard-level cache.
- Benchmark table designs empirically and diagnose CPU bottlenecks in daily aggregation workloads.
- Re-architect a poorly performing multi-terabyte analytics table using distribution, sorting, and pre-aggregation.

Cross-Cloud Equivalence

Every analytical engine faces the same physical problem—move less data, read less data, schedule work fairly—but each platform exposes different levers.

Capability	AWS	Azure	GCP
Data distribution	DISTSTYLE EVEN/KEY/ALL	Synapse hash/round-robin/replicated	Automatic (Big-Query)
Sort optimization	SORTKEY (compound/interleaved)	Clustered column-store index	Clustering columns
Result pre-computation	Materialized views	Materialized views / semantic models	Materialized views
Workload governance	WLM + query monitoring rules	Workload groups / classifiers	Reservations & slots
Burst concurrency	Concurrency scaling	Scale-out / serverless burst	Autoscaling slots
Manual tuning burden	Medium (AUTO helps)	Medium-high	Low (mostly automatic)
Skew diagnosis	SVV_TABLE_INFO	DMVs	Query plan explanation

Snowflake removes most physical design levers—micro-partition pruning is automatic, and clustering keys are optional—trading control for operational simplicity. **Fabric** warehouse and lakehouse inherit SQL Server-style distribution choices on dedicated capacity. **Mon-**

goDB offers a different instrument entirely: indexes and shard keys rather than distribution styles, because its workload is point lookups, not scans.

6.1 Week 6 Roadmap and Mental Model

Chapter 5 built a correctly modeled warehouse; this chapter makes it fast. Redshift performance work reduces to three questions. Where does each row physically sit—*distribution*? In what physical order is it stored—*sorting*? And who gets compute when demand exceeds supply—*workload management*? Everything else—compression, materialized views, concurrency scaling—is a refinement of those three [71].

The discipline that separates professional tuning from superstition is measurement. Every recommendation in this chapter is testable: create the table two ways, run the query both ways, compare `SVL_QUERY_SUMMARY` and the execution plan. The Thursday project institutionalizes that discipline with a controlled distribution-style experiment.

Definition

Physical design in Redshift is the choice of distribution style, distribution key, sort key, and compression encoding per table. These choices determine how much data moves across the cluster interconnect during joins, how many blocks zone maps can skip during scans, and how much I/O compression saves.

Key Terms

Distribution style: rule placing rows on slices (EVEN, KEY, ALL, AUTO). **Distribution skew**: uneven row counts across slices. **Sort key**: column list controlling on-disk row order; compound or interleaved. **Zone map**: per-block min/max metadata enabling block skipping. **WLM**: workload management; queues, service classes, and routing rules. **Concurrency scaling**: automatic transient capacity added for queued queries. **Materialized view**: precomputed query result stored as a table and refreshed on demand or automatically.

6.2 Monday: Distribution Styles

6.2.1 The Four Styles

Redshift places each row on one slice according to the table's distribution style [22]:

- **KEY**: rows are hashed on the distribution key column; equal keys land on the same slice. Two large tables distributed on their join key *collocate* the join—no data crosses the network.
- **EVEN**: rows round-robin across slices regardless of content. Balanced, but every join redistributes one or both sides.
- **ALL**: the entire table is copied to every node. Expensive in storage and load time, but joins against it never move data. Reserved for small, slowly changing dimensions.
- **AUTO**: Redshift picks and evolves the style based on table size and query patterns—usually ALL for small tables and EVEN or KEY as tables grow. The right default for most tables.

Definition

A **collocated join** joins two tables distributed on the same key, so matching rows already share a slice and no data crosses the cluster interconnect. Collocation is the single largest join-performance lever in Redshift.

6.2.2 Choosing the Distribution Key

Pick the column most frequently used to join the table to its largest partner—for `fact_sales`, typically `customer_key` or `date_key` depending on the dominant join. The key must also have high cardinality and even value distribution: distributing on a low-cardinality or skewed column (a boolean flag, a region with 90% of rows in one value) concentrates rows on a few slices, and the slowest slice determines query time. Check skew with `SVV_TABLE_INFO` and the `stv_blocklist`-based admin views.

Common Pitfall

Distributing on a timestamp or an ever-increasing key. High-cardinality does not imply good distribution if the column is never used in joins. The distribution key exists to collocate joins; if no join uses it, you have paid skew risk for nothing. Use `EVEN` instead.

Style	Use it when	Risk to watch
KEY	Large fact/dimension pairs join on the key	Skewed or low-cardinality keys; key never used in joins
EVEN	No dominant join column; staging tables	Every large join redistributes data
ALL	Small dimensions (thousands of rows) joined everywhere	Storage and load cost per node; slow updates
AUTO	Default; tables whose size or usage is evolving	Occasionally surprises after major growth; verify with the plan

Table 6.1: Distribution-style selection guide.

6.3 Tuesday: Sort Keys and Compression Encodings

6.3.1 Compound Versus Interleaved

A sort key orders rows on disk, which lets zone maps skip blocks when queries filter on leading sort columns. **Compound** sort keys behave like a phone book: excellent for filters on a prefix of the key (`date`, then `date+region`), useless for a filter that skips the leading column. They are the right choice for the common case of a date-ordered fact table. **Interleaved** sort keys give every column in the key roughly equal weight, helping workloads whose queries filter on unpredictable column combinations—at the cost of slower loads and vacuum operations. Interleaved keys are a legacy tool for genuinely varied access; start with compound.

Note

Modern Redshift with `SORTKEY AUTO` continuously learns the workload’s filter patterns and adjusts. Manual sort keys remain worth understanding for the exam and for tables whose access pattern you know better than the optimizer does—and because you must explain *why* a table is slow before you can trust automation to fix it.

6.3.2 Compression Encodings

Each column gets an encoding—`raw`, `az64`, `lz4`, `delta`, `runlength`, and dictionary-style encodings among them. Encoding reduces storage and, more importantly, I/O: scanning a compressed column reads fewer blocks. COPY applies automatic compression analysis on first load; `ANALYZE COMPRESSION` reports what it chose. Two rules matter in practice: never compress the first sort-key column so aggressively that zone-map range comparison suffers (historically the guidance was `raw` on the leading sort column; current `az64` handles this well), and let automation run unless measurements disagree.

6.4 Wednesday: WLM, Concurrency Scaling, and Materialized Views

6.4.1 Workload Management

WLM divides cluster capacity into queues (automatic WLM manages them as service classes) so that a runaway ad-hoc query cannot starve the executive dashboard [67]. Automatic WLM routes queries by user, role, or query group, assigns memory dynamically, and supports *query monitoring rules*—for example, abort any query scanning more than a set number of rows or running longer than a threshold. Short Query Acceleration (SQA) automatically bumps genuinely short queries past long-running ones.

Tip

Read WLM telemetry before changing WLM. Rising queue wait time means demand exceeds the capacity you have allocated to that workload class: either add concurrency scaling, raise the class's memory share, or make the queries cheaper. Guessing at queue counts without the wait-time data usually moves the bottleneck rather than removing it.

6.4.2 Concurrency Scaling

When queued queries pile up, concurrency scaling spins up transient, read-only capacity automatically, billed per second with a daily free allowance [67]. It exists for exactly the BI-dashboard pattern: fifty analysts hit refresh at 9:05 a.m. Enable it on the service classes that serve bursty read traffic, set usage limits so the burst is bounded, and remember that writes never use concurrency-scaling clusters.

6.4.3 Materialized Views

A materialized view (MV) stores a precomputed query result [62]. Unlike a plain view—which is re-executed on every reference—an MV trades storage and refresh cost for query speed. MVs shine for the repeated, expensive aggregations that power dashboards: define `mv_monthly_sales` once, refresh after each load, and let a hundred dashboard queries scan a tiny pre-aggregated table. Redshift supports automatic and incremental refresh and can rewrite eligible queries against MVs automatically.

```

1 CREATE MATERIALIZED VIEW mv_monthly_sales AS
2 SELECT d.year, d.month, p.category, st.region,
3        SUM(f.amount) AS revenue,
4        SUM(f.qty)    AS units
5 FROM fact_sales f
6 JOIN dim_date d   ON f.date_key = d.date_key
7 JOIN dim_product p ON f.product_key = p.product_key
8 JOIN dim_store st  ON f.store_key = st.store_key
9 GROUP BY 1, 2, 3, 4;
```

```

10
11 REFRESH MATERIALIZED VIEW mv_monthly_sales;

```

Listing 6.1: Materialized view for a repeated month-end aggregation.

Common Pitfall

A materialized view per dashboard. Each MV costs storage and refresh compute, and stale MVs answer yesterday's questions convincingly. Consolidate on a few shared, conformed MVs with a documented refresh schedule wired to the ETL calendar.

6.5 Thursday Hands-on Project: Benchmarking Distribution Styles

Theory says KEY distribution collocates joins and avoids redistribution. This project proves it—and measures the size of the effect—by creating three identical fact tables with different distribution styles and timing the same complex join against each.

Lab Objectives

- Create three copies of a large fact table with DISTSTYLE KEY, EVEN, and ALL.
- Run an identical join-heavy aggregation against each and record elapsed time.
- Inspect the query plan to confirm redistribution (DS_DIST_OUTER / broadcast steps) versus collocation.
- Measure storage and load-time cost of ALL distribution.

Procedure

Use the Chapter 5 workgroup and a dimension large enough that redistribution is visible—regenerate dim_customer with 500,000 rows if needed. Create the three experimental tables:

```

1 -- Baseline copy of the fact with KEY distribution on the join column
2 CREATE TABLE bench_fact_key DISTSTYLE KEY DISTKEY (customer_key) SORTKEY (
   date_key)
3 AS SELECT * FROM fact_sales;
4
5 CREATE TABLE bench_fact_even DISTSTYLE EVEN SORTKEY (date_key)
6 AS SELECT * FROM fact_sales;
7
8 -- ALL is only meaningful for the small side; demonstrate with dim_store
9 CREATE TABLE bench_store_all DISTSTYLE ALL AS SELECT * FROM dim_store;
10
11 ANALYZE bench_fact_key;
12 ANALYZE bench_fact_even;
13 ANALYZE bench_store_all;

```

Listing 6.2: Three identical tables, three distribution styles.

Run the benchmark query three times per table to warm caches, then record three timed runs. Wrap the timing in a script so results are comparable.

```

1 -- Time this identical aggregation against bench_fact_key and bench_fact_even
2 \timing on
3 SELECT c.segment, SUM(f.amount) AS revenue
4 FROM bench_fact_key f -- or bench_fact_even
5 JOIN dim_customer c ON f.customer_key = c.customer_key

```

```

6 WHERE c.is_current
7 GROUP BY 1 ORDER BY 2 DESC;
8
9 -- Inspect how data moved during execution:
10 EXPLAIN
11 SELECT c.segment, SUM(f.amount)
12 FROM bench_fact_even f
13 JOIN dim_customer c ON f.customer_key = c.customer_key
14 WHERE c.is_current
15 GROUP BY 1;
16
17 -- Compare skew and storage across the three tables:
18 SELECT "table", size, tbl_rows, skew_rows
19 FROM svv_table_info
20 WHERE "table" LIKE 'bench%' ORDER BY "table";

```

Listing 6.3: Benchmark query and plan inspection.

In the EVEN plan you should see redistribution steps (DS_DIST_OUTER or broadcast); the KEY plan joins collocated slices with no such step. Expect the KEY join to win by a margin that grows with table size—on small Serverless workgroups with ten million rows the difference may be modest; that is itself a finding worth recording.

Verification

- A results table records three timed runs per style with the mean and the plan's distribution steps.
- `svv_table_info` output shows roughly even slice distribution for the KEY and EVEN tables and the multiplied row count of the ALL table.
- The report states a rule, derived from the measurements, for when this workload should use each style.

Tip

Flush result caching between runs by varying an insignificant query comment or disabling the result cache for the session (`SET enable_result_cache_for_session = off;`), or your second run will measure the cache, not the engine.

6.6 Friday Use Case: Rescuing a Slow Multi-Petabyte Aggregation Table

A media company's central events table has grown past two petabytes of compressed storage. Daily dashboard aggregations that once took minutes now run for hours, pinning compute CPU near 100% every morning and queuing every other workload behind them. You are asked to re-architect the table and the workload.

6.6.1 Diagnosis First

The engagement starts with evidence, not redesign. `SVL_QUERY_SUMMARY` shows the daily aggregation scans the full table; `svv_table_info` shows 40% distribution skew on a `region` distribution key; the table has no sort key, so zone maps are useless; and the same aggregation runs 300 times a day from three dashboard tools—which no physical design will ever make cheap.

6.6.2 The Re-architecture

Distribution. The dominant join is to the customer dimension; redistribute the fact on `customer_key` to collocate it. The skew came from distributing on `region`, a four-value column—an error the diagnosis makes obvious in hindsight.

Sorting. Add a compound sort key on (`event_date`, `event_type`); every daily aggregation filters on `event_date`, and zone-map pruning now skips roughly 364/365 of the blocks.

Pre-aggregation. Replace the 300 identical daily scans with one materialized view refreshed after the nightly load. Dashboards query the MV; the two-petabyte scan happens once, not three hundred times.

Workload isolation. Route ad-hoc queries and dashboard queries to separate WLM service classes with a query monitoring rule that aborts unbounded scans, and enable concurrency scaling for the dashboard class so the morning burst stops queuing.

Finding	Fix	Expected effect
40% skew on <code>region</code> DIS-TKEY	Redistribute on <code>customer_key</code>	Slowest-slice time drops to the average
No sort key; full scans daily	Compound SORTKEY(date, type)	Zone maps skip ~99% of blocks
300 identical aggregations/day	One shared materialized view	Scan cost divided by ~300
Everything in one WLM class	Split classes + monitoring rule	Dashboards isolated from ad-hoc abuse
Morning queue spikes	Concurrency scaling on BI class	Burst absorbed without re-size

Table 6.2: From diagnosis to design: each fix maps to a measured symptom.

Common Pitfall

Optimizing the query when the workload is the problem. No distribution style makes scanning two petabytes three hundred times a day cheap. When the same expensive query recurs at scale, the architectural answer is pre-computation (materialized views, summary tables), not more tuning of the scan.

6.7 Interview Q&A

6.7.1 Concepts and Trade-offs

1. **Q: Which distribution style avoids data movement when joining two large tables?**

A: KEY distribution on the join column for both tables. Matching rows share a slice, so the join is collocated and nothing crosses the interconnect.

2. **Q: Compound versus interleaved sort keys: which suits varied filter-column combinations?**

A: Interleaved—each column in the key carries roughly equal weight. Compound keys only help filters on a prefix of the key, which suits the far more common date-ordered access pattern.

3. **Q: What problem does concurrency scaling solve for BI dashboards?**

A: Bursty read demand that exceeds steady capacity—fifty analysts refreshing at once.

Transient read-only clusters absorb the burst without a resize, billed per second.

4. **Q: When does a materialized view beat a plain view?**

A: When the same expensive aggregation is run repeatedly and results can tolerate the refresh cadence—classic dashboard workloads. A plain view re-executes the full query every time.

5. **Q: What does rising queue wait time in WLM tell you to change?**

A: Capacity for that workload class: enable concurrency scaling, adjust service-class memory, reduce query cost with sort keys or MVs, or throttle demand. Wait time is demand exceeding supply, not a query bug.

6. **Q: Why is ALL distribution dangerous for a large table?**

A: Every node stores a full copy: storage multiplies by node count, loads and updates rewrite every copy, and the benefit—collocated joins—rarely justifies that cost beyond small dimensions.

6.8 Further Reading

The Redshift performance tuning guide is the authoritative reference for distribution, sort keys, and query plan analysis [71]; the workload management documentation covers automatic WLM, query monitoring rules, and concurrency scaling [67]; and the materialized views documentation covers refresh strategies and automatic query rewrite [62]. The Database Developer Guide documents the system views used in this chapter’s diagnostics [22]. Kimball and Ross discuss aggregate tables—the dimensional ancestor of the materialized view—as a first-class design element [92].

Key Takeaways

- Distribution decides where rows sit: KEY collocates joins, EVEN balances scans, ALL replicates small dimensions, AUTO is the safe default.
- Distribution skew and a join-free DISTKEY are the two classic distribution mistakes; both are visible in `svv_table_info` and the query plan.
- Compound sort keys plus zone maps turn date-filtered scans into block-skipping exercises; interleaved keys are a niche tool for unpredictable filter combinations.
- WLM routes and protects workloads; concurrency scaling absorbs read bursts; monitoring rules stop runaway queries.
- Materialized views convert repeated expensive aggregation into refresh-once, read-many economics.
- Tune from measurement: benchmark competing designs, read the plan, and let each fix map to a diagnosed symptom.

6.9 Exercises

1. **(Conceptual)** Explain why the slowest slice determines total query time when distribution is skewed.
2. **(Conceptual)** A fact table joins a different dimension in nearly every query. Which distribution style do you choose, and why?

3. **(Hands-on)** Run the Thursday benchmark and add a fourth variant: KEY distribution on `date_key`. Predict the result before running it, then explain the measurement.
4. **(Hands-on)** Use EXPLAIN output to count redistribution steps for each benchmark table and correlate step count with elapsed time.
5. **(Hands-on)** Create the materialized view from listing 6.1, refresh it, and measure a dashboard-style query against it versus the base tables.
6. **(Design)** A table receives continuous small inserts and nightly large loads. Discuss how sort-key choice, vacuum strategy, and load order interact.
7. **(Design)** Write query monitoring rules for (a) the ad-hoc analyst class and (b) the nightly ETL user. Justify each threshold.
8. **(Architecture)** The Friday scenario's company grows to ten petabytes. Which of the five fixes scale, which need revisiting, and what new problem appears first?

6.10 Capstone Project: A Measured Performance Benchmark Harness for Redshift

Capstone Project

Mission. Build a repeatable benchmarking harness that creates multiple physical designs of the same star schema, executes a standardized query workload against each, and produces an evidence-backed physical-design recommendation for a realistic BI workload.

Estimated time. 3–5 hours in a sandbox AWS account.

What you'll practice.

- Distribution and sort-key selection driven by measurement rather than folklore.
- Plan reading: redistribution steps, zone-map elimination, and skew metrics.
- WLM configuration and concurrency-scaling observation under a burst.
- Materialized-view consolidation for repeated dashboard aggregations.

Scenario

Your Chapter 5 retail warehouse is a success—which is the problem. Analyst headcount tripled, the events table doubled, and the CFO wants to know whether next year's warehouse bill should grow. Before any capacity purchase, you must prove which physical design and workload-management configuration serves the current workload most cheaply.

Requirements

Functional requirements.

- Automate creation of at least four physical variants of the fact table: KEY on `customer_key`, KEY on `date_key`, EVEN, and AUTO, each with and without a compound sort key.
- Execute a standardized six-query workload (three joins, two aggregations, one filtered scan) three times per variant with result caching disabled.
- Record elapsed time, redistribution steps, and rows scanned per query per variant.
- Create one shared materialized view for the workload's repeated aggregation and measure its effect.

Non-functional requirements.

- The harness is rerunnable from a clean schema and produces a results table, not screenshots alone.
- A short written recommendation maps every design choice to a measured effect.
- Cost guardrails: a query monitoring rule aborts any benchmark query exceeding a stated row-scan threshold.

Implementation Steps

1. Script the variant DDL as parameterized SQL (psql variables or a Python driver) so variants differ only in physical design.
2. Load each variant from the same staging table with `INSERT ... SELECT`, then `ANALYZE` all variants.
3. Run the workload with `SET enable_result_cache_for_session = off;` and capture timings into a `bench_results` table.
4. Capture `EXPLAIN` plans per variant and tag redistribution steps.
5. Create and refresh the shared MV; rerun the aggregation queries against it.
6. Write the recommendation: winning design, expected monthly compute saving, and residual risks.

Deliverables

- The harness scripts, results table DDL, and the raw measurement data.
- A comparison table of variants with mean elapsed time and redistribution step counts.
- Annotated `EXPLAIN` excerpts showing collocation versus redistribution.
- The materialized-view effect analysis and the final written recommendation.

Acceptance Criteria

- Variants differ only in physical design; data volume and query text are identical.
- Every reported number is the mean of at least three uncached runs.
- The recommendation cites `svv_table_info` skew metrics and plan evidence.
- The MV analysis states refresh cost, not just query speedup.
- The cost guardrail fired at least once during testing and the event is documented.

Stretch Goals

- Repeat the benchmark at 10× data volume and identify which conclusions change.
- Simulate the morning dashboard burst with fifty concurrent clients (e.g., `pgbench` or a Python thread pool) and observe concurrency scaling in the console.
- Enable `AUTO` table optimization on all variants for a week and compare its choices with yours.
- Port the harness to compare Redshift Serverless base capacities of 8, 32, and 128 RPUs on price-performance.

Data Lakehouse and Amazon Athena

Learning Objectives

- Explain how Amazon Athena runs serverless SQL directly against S3 and how its per-byte-scanned pricing shapes table design.
- Catalog S3 data with AWS Glue crawlers, databases, and tables, and recognize where schema inference fails.
- Query the same catalog from Redshift Spectrum and decide when Spectrum is preferable to Athena.
- Convert raw CSV to partitioned Parquet and quantify the scan-cost reduction from columnar layout and partition pruning.
- Describe what Apache Iceberg adds beyond plain Parquet files: schema evolution, partition evolution, and ACID tables.
- Design a hybrid lakehouse architecture that queries 50 PB of S3 logs alongside core business data in Redshift.

Cross-Cloud Equivalence

The lakehouse pattern—open files on object storage, a shared catalog, multiple query engines—now exists on every cloud, and the week’s services map cleanly onto their counterparts.

Capability	AWS	Azure	GCP
Serverless SQL on the lake	Athena	Synapse/Fabric serverless SQL	BigQuery external tables
Shared metastore	Glue Data Catalog	Microsoft Purview / OneLake catalog	Dataplex / BigLake metastore
Schema inference	Glue crawlers	Synapse auto-discovery	BigQuery autodetect
Warehouse-to-lake query	Redshift Spectrum	Synapse/Fabric lakehouse shortcuts	BigLake tables
Table format	Iceberg / Hudi / Delta on S3	Delta (OneLake native)	BigLake Iceberg
Pricing lever	Per TB scanned	Capacity units	Bytes scanned / slots
Cost control	Partitioning + Parquet	Partitioning + Delta	Partitioning + clustering

Fabric is the most opinionated implementation: OneLake plus Delta–Parquet with *shortcuts* makes the warehouse and lake the same physical data. **Snowflake** reads external Iceberg tables while its native tables remain the managed counterpart of the same idea. **MongoDB** plays the source-system role here—Atlas Data Federation queries S3, but the

document model itself stays operational.

7.1 Week 7 Roadmap and Mental Model

Chapters 5 and 6 kept data inside the warehouse. This week opens the warehouse to the lake: the same SQL questions, asked against open files on S3, governed by a shared catalog, from whichever engine fits the query. The lakehouse thesis [85] is that object storage plus open table formats plus a metastore can deliver warehouse-grade management at lake economics.

Three services implement the pattern on AWS. **Athena** is serverless SQL (Trino/Presto heritage) over S3, billed per byte scanned. The **Glue Data Catalog** is the shared metastore that tells every engine what tables exist, where their files live, and what schema to apply. **Redshift Spectrum** lets a Redshift cluster read the same catalog and S3 files, so lake data and warehouse data join in one query. The engineering craft of the week is cost: on a per-byte-scanned engine, *table layout is the price list*.

Definition

Amazon Athena is a serverless, interactive query service that runs standard SQL against data in S3 using schemas from the Glue Data Catalog. There is no infrastructure to manage; you pay per terabyte of data scanned per query.

Key Terms

Crawler: a Glue job that scans S3 paths, infers schemas, and registers or updates catalog tables. **External table:** a catalog entry whose data lives as files in S3 rather than inside the engine. **Partition projection:** declaring partition rules in table properties instead of storing every partition in the catalog. **Spectrum:** Redshift’s ability to query catalog-registered S3 data via an external schema. **Table format** (Iceberg/Hudi/Delta): a metadata layer over Parquet files providing ACID transactions, schema evolution, and time travel.

7.2 Monday: Amazon Athena—Serverless SQL on S3

7.2.1 How Athena Executes

Athena needs no cluster. A query references catalog tables; Athena’s engine reads the relevant S3 objects, applies the table’s schema and SerDe, and executes the plan on transient compute. Results land in a query-results bucket you configure per workgroup. Because the unit of billing is bytes scanned—with a per-query minimum—the entire discipline of Athena cost control is making queries read fewer bytes: columnar formats, compression, partitioning, and projection of only needed columns [5].

Common Pitfall

SELECT * on raw CSV. An unpartitioned CSV table forces Athena to read every byte of every column of every file. On a 10 TB dataset that is a \$50 query run repeatedly by accident. Workgroup data-usage limits exist precisely to abort such scans; set them on day one.

7.2.2 Workgroups and Cost Guardrails

Workgroups isolate teams and enforce policy: per-query and per-period data-scanned limits, enforced encryption of results, and CloudWatch metrics per workgroup. In practice every production Athena deployment has at least an analyst workgroup with tight limits and an ETL workgroup with a higher ceiling and separate results bucket.

7.3 Tuesday: The Glue Data Catalog and Crawlers

7.3.1 A Shared Metastore

The Glue Data Catalog is a Hive-metastore-compatible service holding databases, tables, partitions, and schema versions [44]. Athena, Redshift Spectrum, EMR, and Glue ETL all resolve table names against it. This single-catalog design is what makes the lakehouse more than a pile of files: a table defined once is queryable from every engine with the same name, schema, and partition layout.

7.3.2 Crawlers: Inference and Its Limits

A crawler walks an S3 prefix, samples files, infers columns and types, detects partitions from `key=value` path segments, and creates or updates catalog tables [44]. Inference is a convenience, not a contract. Crawlers merge schemas when files in one prefix disagree, misclassify types from unlucky samples, and can create thousands of tiny tables if pointed at a prefix that mixes datasets. Production practice: crawl the *curated* zone where layout is uniform, define raw-zone tables by hand with explicit DDL, and version the catalog with schema-registry discipline (Chapter 11).

Tip

Prefer partition projection on tables with thousands of partitions (for example, `year=/month=/day=` over many years). It stores partition *rules* in table properties instead of millions of partition entries in the catalog, eliminating both catalog bloat and the `MSCK REPAIR TABLE` maintenance chore.

7.4 Wednesday: Redshift Spectrum and the Open Table Layer

7.4.1 Querying the Lake from the Warehouse

Redshift Spectrum lets Redshift SQL read catalog-registered S3 tables through an external schema, so a query can join `fact_sales` in the warehouse with `clickstream` in the lake in one statement [72]. Spectrum spins up an independent fleet to scan S3, billed per byte scanned; the Redshift cluster does the joins and aggregation.

```

1 CREATE EXTERNAL SCHEMA lakehouse FROM DATA CATALOG
2   DATABASE 'retail_lake'
3   IAM_ROLE 'arn:aws:iam::123456789012:role/SpectrumRole'
4   CREATE EXTERNAL DATABASE IF NOT EXISTS;
5
6 SELECT d.year, p.category, SUM(l.event_count) AS events
7 FROM lakehouse.clickstream_daily l
8 JOIN dim_date d      ON l.date_key = d.date_key
9 JOIN dim_product p   ON l.product_key = p.product_key
10 GROUP BY 1, 2;
```

Listing 7.1: Creating a Spectrum external schema over the Glue catalog.

Spectrum or Athena? Both read the same catalog and charge per byte scanned. Choose Spectrum when the query must join lake data with warehouse-resident tables in one statement, or when the organization already operates a Redshift cluster. Choose Athena when there is no cluster running, the query is lake-only, or the team wants zero infrastructure. The catalog makes this an execution-time decision, not a data-modeling decision.

7.4.2 From Files to Tables: Parquet and Iceberg

Plain Parquet gives columnar, compressed, splittable files—the foundation of lake economics. **Apache Iceberg** adds the table semantics files lack: atomic commits so concurrent writers never corrupt a table, schema evolution without rewriting data, partition evolution without rewriting data, hidden partitioning, and time travel over snapshots [82]. Athena, Glue, and EMR all read and write Iceberg tables registered in the Glue catalog, which is what turns an S3 prefix into a governed, mutable table.

Note

The catalog entry records the table format, not just the schema. An Iceberg table queried as if it were plain Parquet will silently read stale or duplicate file versions. Always register and query Iceberg tables through a format-aware engine.

7.5 Thursday Hands-on Project: Crawl, Query, and Federate the Same Data

One dataset, three access paths: catalog an S3 bucket of raw CSVs with a Glue crawler, aggregate it in Athena, then query the same catalog table from Redshift Spectrum—and finally measure what Parquet conversion buys.

Lab Objectives

- Run a Glue crawler over a raw CSV prefix and inspect the inferred schema.
- Write an Athena aggregation over the raw table and record bytes scanned.
- Convert the CSVs to partitioned Parquet, catalog the result, and rerun the aggregation.
- Create a Spectrum external schema and join the lake table to a Redshift dimension.

Procedure

Stage a CSV dataset (the Chapter 5 synthetic sales data exported as CSV works well) under `s3://<bucket>/raw/sales_csv/`. Create the crawler with the CLI:

```

1 $Region = "us-east-1"
2 $Bucket = "acme-analytics-raw-123456789012"
3 $Role    = "GlueCrawlerRole"    # trust: glue.amazonaws.com; policy: read $Bucket
4
5 aws glue create-database --database-input '{"Name":"week7_lake"}'
6
7 aws glue create-crawler '
8   --name week7-sales-crawler '
9   --role $Role '
10  --database-name week7_lake '
11  --targets '{"S3Targets":[{"Path":"s3://$Bucket/raw/sales_csv/"}]}'
12
13 aws glue start-crawler --name week7-sales-crawler
14 aws glue get-crawler --name week7-sales-crawler    # poll until State = READY

```

```
15 aws glue get-tables --database-name week7_lake
```

Listing 7.2: Creating and running a Glue crawler over raw CSVs.

Query the raw table in Athena and note the bytes scanned from the query statistics. Then convert to Parquet with a one-shot Athena CTAS (Create Table As Select)—Athena itself is the easiest converter:

```
1  -- Convert once: CTAS writes partitioned Parquet and registers a new table
2  CREATE TABLE week7_lake.sales_parquet
3  WITH (
4    format = 'PARQUET',
5    parquet_compression = 'SNAPPY',
6    external_location = 's3://acme-analytics-raw-123456789012/curated/
7    sales_parquet/',
8    partitioned_by = ARRAY['order_year', 'order_month']
9  ) AS
10 SELECT order_line_id, customer_id, product_id, store_id, qty, amount,
11        YEAR(order_date) AS order_year,
12        MONTH(order_date) AS order_month
13 FROM week7_lake.sales_csv;
14
15 -- Aggregation over raw CSV (record "Data scanned"):
16 SELECT customer_id, SUM(amount) FROM week7_lake.sales_csv
17 GROUP BY 1 ORDER BY 2 DESC LIMIT 100;
18
19 -- The same aggregation over Parquet (compare "Data scanned"):
20 SELECT customer_id, SUM(amount) FROM week7_lake.sales_parquet
21 WHERE order_year = 2026 AND order_month = 8
22 GROUP BY 1 ORDER BY 2 DESC LIMIT 100;
```

Listing 7.3: CTAS conversion from CSV to partitioned Parquet, then the same aggregation on both.

Finally, point Spectrum at the same catalog database from the Chapter 5 workgroup and join lake data to the warehouse dimension:

```
1  CREATE EXTERNAL SCHEMA week7 FROM DATA CATALOG
2    DATABASE 'week7_lake'
3    IAM_ROLE 'arn:aws:iam::123456789012:role/SpectrumRole'
4    CREATE EXTERNAL DATABASE IF NOT EXISTS;
5
6  SELECT c.segment, SUM(s.amount) AS revenue
7  FROM week7.sales_parquet s
8  JOIN dim_customer c ON s.customer_id = c.customer_id
9  WHERE s.order_year = 2026 AND c.is_current
10 GROUP BY 1;
```

Listing 7.4: Spectrum external schema joining the lake table to a Redshift dimension.

Verification

- The crawler creates one table with the expected columns; any schema merge surprises are documented.
- Athena query statistics show the Parquet aggregation scanning one to two orders of magnitude fewer bytes than the CSV equivalent; the date-filtered query prunes to one partition.
- The Spectrum query returns identical aggregates to the Athena query over the same table.
- A cost line computes the price of both Athena runs at \$5/TB and states the annualized saving for a daily-run query.

Common Pitfall

Crawler-created tables pointing at mixed prefixes. If the raw prefix contains more than one dataset shape, the crawler either creates several tables with cryptic names or merges incompatible schemas. Point crawlers at single-dataset prefixes and set a table-level configuration to enforce one table per prefix.

7.6 Friday Use Case: A 50 PB Media Log Lakehouse

A media company accumulates CDN, player, and ad-server logs at roughly 30 TB per day—about 50 PB retained—while its subscriber, content, and billing systems of record live in a provisioned Redshift cluster. Leadership wants analysts to query logs alongside business data without paying warehouse storage prices for 50 PB.

7.6.1 Architecture

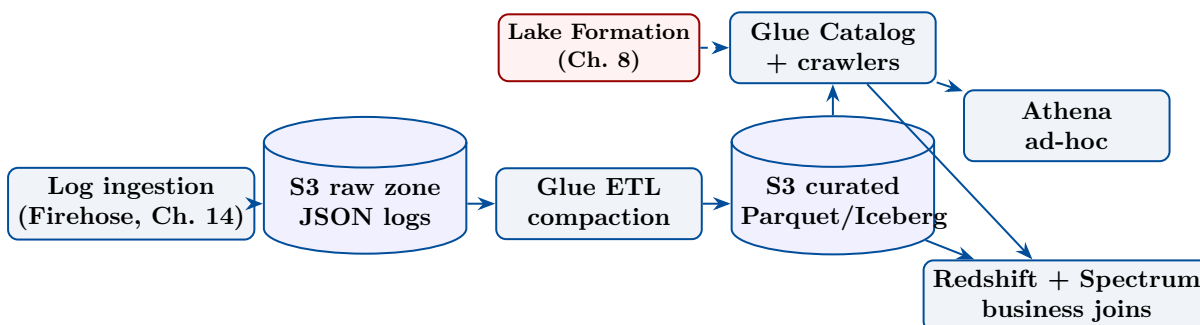


Figure 7.1: Media lakehouse: logs land raw, are compacted to partitioned Parquet/Iceberg, and are served through one catalog to Athena (ad-hoc) and Spectrum (joins with warehouse data).

7.6.2 Design Decisions

Layout is the budget. At \$5 per TB scanned, a full scan of 50 PB is a \$250,000 query. The curated zone is therefore partitioned by `day` and `log_source`, stored as Snappy-compressed Parquet, and compacted by a Glue job into ~1 GB files so engines do not drown in small-file overhead. Column projection (selecting five of eighty columns) and partition pruning (querying seven days, not five years) bring a typical analyst query down to low gigabytes—pennies.

Right engine per question. Ad-hoc log exploration runs in Athena with tight workgroup limits. Questions joining logs to subscribers and billing run through Spectrum inside Redshift, where warehouse dimensions already live. Hot aggregates that dashboards hit repeatedly are promoted into Redshift as materialized views over the external schema, so Spectrum pays the scan once per refresh, not once per dashboard load.

Cold history stays cold. Logs older than thirteen months transition to Glacier storage classes (Chapter 1) with an Iceberg snapshot retained for audit reproducibility; compliance retrieval is a restore operation, not a query.

Governance. All access flows through the Glue catalog with Lake Formation permissions (Chapter 8), so the same column masks apply whether the analyst arrives via Athena or Spectrum.

Common Pitfall

Letting the raw zone become the query surface. Raw JSON logs scanned interactively are the most expensive way imaginable to answer questions. The raw zone exists for durability and reprocessing; only compacted, partitioned, columnar curated data should be exposed to analysts.

7.7 Interview Q&A

7.7.1 Concepts and Trade-offs

1. **Q: Why does Parquet lower Athena cost compared with CSV?**
A: Athena bills per byte scanned. Parquet is columnar and compressed, so a query reads only the blocks of the columns it references—often one to two orders of magnitude fewer bytes than row-oriented, uncompressed CSV.
2. **Q: What does a Glue crawler infer, and when does it infer wrong?**
A: It infers columns, types, and partitions by sampling files. It fails when files in one prefix disagree in schema, when samples are unrepresentative, or when a prefix mixes datasets.
3. **Q: When is Redshift Spectrum preferable to Athena for the same S3 data?**
A: When the query must join lake data with warehouse-resident tables in one statement, or a Redshift cluster is already running and the team wants one SQL surface. For lake-only ad-hoc work with no cluster, Athena is simpler.
4. **Q: How does partition pruning cut bytes scanned?**
A: The engine reads partition metadata and skips every file outside the query's partition filter—a date-filtered query opens seven day-partitions instead of five years of files.
5. **Q: What makes Apache Iceberg a lakehouse format beyond plain Parquet files?**
A: A metadata layer providing atomic commits, schema and partition evolution, hidden partitioning, and snapshot time travel—table semantics that raw files on object storage lack.
6. **Q: Why run crawlers against the curated zone rather than the raw zone?**
A: Inference assumes uniform layout. The curated zone is compacted, typed, and partitioned, so inference succeeds; the raw zone is heterogeneous by design and deserves hand-written DDL.

7.8 Further Reading

The Athena User Guide covers workgroups, cost controls, CTAS, and partition projection [5]; the Glue documentation covers the Data Catalog and crawler configuration [44]; and the Spectrum documentation covers external schemas and the IAM role trust model [72]. For table formats, the Apache Iceberg site and specification explain snapshots, manifest files, and partition evolution [82]. Armbrust et al. provide the academic framing of the lakehouse itself [85].

Key Takeaways

- Athena turns S3 into a SQL database with no infrastructure; per-byte-scanned pricing makes table layout the cost model.
- The Glue Data Catalog is the shared metastore that lets Athena, Spectrum, EMR, and Glue ETL see one set of tables.

- Crawlers are inference conveniences: trust them in uniform curated zones, hand-write DDL for raw zones, and prefer partition projection at scale.
- Spectrum and Athena read the same catalog; choose per query by join needs and running infrastructure, not by data modeling.
- Parquet plus partitioning plus column projection is the cost-control trinity; Iceberg adds ACID, evolution, and time travel on top.
- In a 50 PB lakehouse, raw zones are for durability, curated zones are for queries, and warehouse materialization is for repeated questions.

7.9 Exercises

1. **(Conceptual)** Explain why “bytes scanned” and “bytes stored” diverge more for Parquet than for CSV.
2. **(Conceptual)** Why does a Spectrum query require an active Redshift cluster while an Athena query does not?
3. **(Hands-on)** Run the Thursday lab and produce a scanned-bytes table for CSV versus Parquet, with and without the date filter.
4. **(Hands-on)** Enable partition projection on `sales_parquet` and drop the catalog partitions; verify the same queries still prune correctly.
5. **(Hands-on)** Create an Athena workgroup with a 1 GB per-query limit and demonstrate it aborting a full-table CSV scan.
6. **(Design)** Design the S3 prefix scheme, partitioning, and crawler schedule for three log sources with different volumes and retention rules.
7. **(Design)** Decide, with reasons, which tables in the media lakehouse should be Iceberg and which can remain plain Parquet.
8. **(Architecture)** Extend fig. 7.1 with a third engine (EMR Spark, Chapter 9) reading the same catalog, and state what governs concurrent writes to an Iceberg table.

7.10 Capstone Project: A Cost-Measured Lakehouse Tier for Web Analytics

Capstone Project

Mission. Build a queryable lakehouse tier over raw web-analytics exports: catalog the raw data, convert it to partitioned Parquet, serve it through Athena and Spectrum, and prove the cost model with measured bytes scanned.

Estimated time. 3–5 hours in a sandbox AWS account.

What you'll practice.

- Crawler configuration, schema validation, and partition projection on the Glue catalog.
- CTAS conversion from CSV to partitioned, compressed Parquet.
- Serving one catalog through two engines and reconciling results.
- Building a scanned-bytes cost model with workgroup guardrails.

Scenario

A publisher’s web-analytics vendor drops hourly CSV exports into S3—about 2 GB per day, two years retained. Analysts currently download files locally; the data team wants governed SQL access, a predictable cost ceiling, and joins against the subscriber dimension already in Redshift.

Requirements

Functional requirements.

- Crawl the raw hourly exports into a `web_lake` database; validate the inferred schema against a hand-written contract and document discrepancies.
- Convert history to Snappy Parquet partitioned by `year/month/day`; register the curated table with partition projection.
- Serve a five-query analyst workload in Athena and the join-with-subscribers query through Spectrum.
- Set an analyst workgroup with a per-query scanned limit and an alarm on daily bytes scanned.

Non-functional requirements.

- All access resolves through the Glue catalog; no direct S3 paths in analyst queries.
- The cost model states price per query, per analyst-day, and per month at measured volumes.
- Raw and curated zones have distinct lifecycle policies consistent with Chapter 1.

Implementation Steps

1. Stage two years of synthetic hourly exports (adapt listing 5.4 to page-view events).
2. Create the database, crawler, and curated table per listings 7.2 and 7.3.
3. Run the analyst workload in Athena; record bytes scanned per query from the query statistics API.
4. Create the Spectrum external schema and run the subscriber-join query; reconcile results with Athena.
5. Configure the workgroup limit, trip it deliberately with a raw CSV scan, and capture the event.
6. Write the cost model and the one-paragraph engine-selection policy.

Deliverables

- Catalog inventory: databases, tables, partition counts, and the schema-validation report.
- Bytes-scanned measurements for CSV versus Parquet, with and without partition filters.
- The five-query workload, the Spectrum join, and reconciled results.
- The cost model and guardrail configuration with the tripped-alarm evidence.

Acceptance Criteria

- The curated table is partitioned, Parquet, Snappy, and uses partition projection.
- Parquet conversion reduces scanned bytes by at least 5× on the workload, documented per query.
- Athena and Spectrum return identical results on the shared table.
- The workgroup limit demonstrably aborts an unbounded query.
- The cost model is reproducible from the recorded measurements.

Stretch Goals

- Convert the curated table to Iceberg and demonstrate schema evolution (add a column) and time travel.
- Add a Glue ETL compaction job that merges small files nightly and measure its effect on query planning time.
- Materialize the two heaviest dashboard queries into Redshift and report the monthly Spectrum saving.
- Compare Athena engine v3 function coverage with the functions your workload actually uses; document any rewrites.

Data Sharing and Governance with Lake Formation

Learning Objectives

- Explain how AWS Lake Formation centralizes data-lake governance on top of the Glue Data Catalog, S3, and IAM.
- Configure row-level and column-level security, including column filters that hide PII from specific principals.
- Apply LF-Tag-based access control and contrast it with named-resource grants.
- Share live data across AWS accounts with Redshift data sharing without copying.
- Design a data clean room that merges CRM and third-party advertiser data without exposing raw PII to either party.
- Assemble the Milestone 2 enterprise lakehouse: Parquet zones, Glue catalog, Lake Formation security, and Spectrum-served dashboards.

Cross-Cloud Equivalence

Governance travels with the data, and every platform has converged on the same primitives: a catalog, tag- or attribute-based policies, fine-grained (row/column) controls, and clean-room collaboration.

Capability	AWS	Azure	GCP
Lake governance layer	Lake Formation	Microsoft Purview	Dataplex
Tag-based access	LF-Tags	Purview classifications	Dataplex aspects/policy tags
Row/column security	LF data filters + cell filters	Purview + SQL RLS/masking	BigQuery RLS + column ACLs
Live cross-account share	Redshift data sharing	Fabric OneLake sharing	Analytics Hub
Managed clean room	AWS Clean Rooms	Azure confidential clean rooms	BigQuery data clean rooms
Enforcement point	Catalog (engine-agnostic)	Per engine + Purview policies	BigQuery policy tags
Default stance	IAMAllowedPrincipals (revoke!)	Deny until granted	Deny until granted

Snowflake implements the same primitives natively—row access policies, masking policies, tags, and zero-copy shares/listings—and its clean-room offering parallels AWS Clean Rooms. **MongoDB** enforces field-level security at the document layer (client-side field-level

encryption), a different but comparable fine-grained posture.

8.1 Week 8 Roadmap and Mental Model

Week 7 built the lakehouse’s query surface; Week 8 governs it. The problem Lake Formation solves is that IAM and S3 bucket policies operate on *objects*, while analysts think in *tables*, *columns*, and *rows*. Lake Formation sits between: it registers S3 locations, extends the Glue Data Catalog with table-, column-, row-, and cell-level grants, and vends temporary, scoped credentials to query engines so that the same permissions apply whether the query comes from Athena, Spectrum, or Glue ETL [51].

The week’s second theme is sharing *without copying*. Redshift data sharing publishes live data from a producer cluster to consumer clusters—across accounts—with the producer owning storage and freshness. A data clean room pushes the idea to its privacy limit: two parties query combined data under cryptographic and policy constraints such that neither sees the other’s raw PII [37].

Definition

AWS Lake Formation is a governance layer over the data lake. It registers S3 data locations, manages permissions on catalog databases, tables, columns, and rows, and brokers scoped credentials to integrated engines so access policy is enforced uniformly at the catalog level.

Key Terms

Data lake administrator: principal allowed to manage Lake Formation settings and grants. **IAMAllowedPrincipals**: the default super-group that makes new catalog objects IAM-governed; revoked to enforce Lake Formation governance. **Named-resource grant**: permission on a specific database/table/column. **LF-Tag**: key–value attribute attached to catalog resources; grants reference tag expressions instead of resource names. **Data filter**: row-level and/or column-level restriction applied to a grant. **Datashare**: Redshift object publishing live tables to consumer clusters.

8.2 Monday: Lake Formation Centralized Governance

8.2.1 The Permission Stack

Lake Formation permissions sit *on top of* IAM, not instead of it. A query succeeds only when both layers allow it: IAM must permit the engine’s role to call Lake Formation and S3, and Lake Formation must grant the caller table access. When a table is governed, engines never use the caller’s S3 credentials directly; they request temporary, column-scoped credentials from Lake Formation, which is what makes column-level security enforceable at the storage layer.

8.2.2 Registering Locations and Revoking the Default

Governance begins with two administrative acts. First, *register* each S3 location with Lake Formation under a service role, so the service controls credential vending for that path. Second, *revoke the default*: new catalog databases and tables are created with a grant to **IAMAllowedPrincipals**, which makes them effectively IAM-governed and bypasses Lake Formation controls. Leaving that grant in place is the most common reason a “secured” lake leaks.

Common Pitfall

Forgetting IAMAllowedPrincipals. Every new database and table starts with this super-group grant. Until you revoke it—on the database and on each table—column filters and LF-Tag policies are decorative. Automate the revocation in the same deployment that creates the catalog.

Note

Apply least privilege in Lake Formation from day one: grant on named columns instead of *, and audit grants with CloudTrail Lake Formation data-plane events. A single overly broad GRANT silently defeats every column mask downstream.

8.3 Tuesday: Row-Level and Column-Level Security

8.3.1 Column-Level Grants

Column-level security restricts *which columns* a principal can select. The mechanism is the grant itself: instead of granting SELECT on a table, you grant SELECT on an explicit column list (or use a data filter that excludes columns). A marketing analyst can hold SELECT(customer_id, segment, city) on customers while pii_email and phone are absent from the grant; Athena then returns an authorization error if the analyst’s query references the excluded columns.

8.3.2 Row-Level Data Filters

A *data filter* restricts *which rows* a principal sees via a filter expression—for example, region = 'EU' for the European analyst role. Filters can also combine row and column restrictions, yielding cell-level control. The power and the danger are the same: the restriction is enforced by the engine at query time, so two analysts querying the same table can legitimately receive different result sets.

```

1 aws lakeformation create-data-cells-filter '
2   --table-data '{"DatabaseName': 'week7_lake',
3     'TableName': 'customers',
4     'Name': 'eu_marketing_view',
5     'ColumnNames': ['customer_id', 'segment', 'city', 'region'],
6     'RowFilter': {'FilterExpression': 'region = \'EU\''}}'
7   }"
8
9
10 aws lakeformation grant-permissions '
11   --principal DataLakePrincipalIdentifier=arn:aws:iam::123456789012:role/
12     MarketingRole '
13   --resource '{"DataCellsFilter': {'DatabaseName': 'week7_lake', 'TableName': 'customers',
14     'Name': 'eu_marketing_view'}}'
15   --permissions SELECT

```

Listing 8.1: Creating a row/column data filter and granting it to a role.

8.3.3 LF-Tags: Policy by Attribute

Named-resource grants scale linearly with tables; LF-Tags scale with *classes* of data. Assign tags like domain=sales, sensitivity=pii to databases, tables, or columns once, then grant SELECT to the marketing role *on the expression* domain=sales AND sensitivity<>pii. Every table tagged correctly thereafter inherits the policy with no new grant [51]. Tag-based control

is what makes a multi-hundred-table lake governable—and what makes tag taxonomy a design artifact worth reviewing like schema.

8.4 Wednesday: Redshift Data Sharing Across Accounts

8.4.1 Live Shares, No Copies

A Redshift datashare publishes selected objects from a producer cluster; consumer clusters—in the same or a different account—create a read-only database from the share and query live producer data through the producer’s managed storage [74]. There is no ETL copy to drift, no second storage bill, and no stale extract. The producer controls inclusion; consumers need only compute.

```

1 -- On the producer:
2 CREATE DATASHARE sales_share;
3 ALTER DATASHARE sales_share ADD SCHEMA analytics;
4 ALTER DATASHARE sales_share ADD ALL TABLES IN SCHEMA analytics;
5 GRANT USAGE ON DATASHARE sales_share TO ACCOUNT '210987654321';
6
7 -- On the consumer (account 210987654321):
8 CREATE DATABASE sales_shared
9 FROM DATASHARE sales_share OF ACCOUNT '123456789012';
10
11 SELECT region, SUM(amount) FROM sales_shared.analytics.fact_sales
12 GROUP BY 1;

```

Listing 8.2: Producer and consumer steps for a cross-account datashare.

Tip

Data sharing pairs naturally with Lake Formation for the lake side of the estate: warehouse tables flow through datashares, lake tables through catalog grants, and the consumer experience—one SQL surface—stays consistent. Decide deliberately which sharing mechanism owns which dataset, and document it in the data catalog of record.

8.5 Thursday Hands-on Project: Hiding the PII Column from Marketing

This project operationalizes Tuesday’s theory: register an S3 location with Lake Formation, revoke the default grant, and prove that a marketing role cannot see `pii_email` in Athena while an analytics role can.

Lab Objectives

- Register the lake location and designate a data lake administrator.
- Revoke `IAMAllowedPrincipals` on the database and table.
- Grant the marketing role `SELECT` on a named column list that excludes `pii_email`.
- Verify from both roles in Athena: marketing queries succeed on allowed columns and fail on the PII column.

Procedure

Build on the Chapter 7 `week7_lake` catalog; add a `customers` table including a `pii_email` column. Then configure governance:


```

1 $AccountId = "123456789012"
2 $Bucket    = "acme-analytics-raw-123456789012"
3 $AdminRole = "arn:aws:iam::${AccountId}:role/LakeFormationAdmin"
4
5 # 1. You (or the admin role) become a data lake administrator.
6 aws lakeformation put-data-lake-settings --data-lake-settings "{
7   'DataLakeAdmins': [{ 'DataLakePrincipalIdentifier': '$AdminRole' } ] }"
8
9 # 2. Register the S3 location under a service role.
10 aws lakeformation register-resource '
11   --resource-arn "arn:aws:s3:::$Bucket/curated/" '
12   --role-arn "arn:aws:iam::${AccountId}:role/LFServiceRole" '
13   --use-service-linked-role false
14
15 # 3. Revoke the default super-group on database and table.
16 aws lakeformation revoke-permissions '
17   --principal DataLakePrincipalIdentifier=IAM_ALLOWED_PRINCIPALS '
18   --resource "{ 'Database': { 'Name': 'week7_lake' } }" '
19   --permissions ALL
20
21 aws lakeformation revoke-permissions '
22   --principal DataLakePrincipalIdentifier=IAM_ALLOWED_PRINCIPALS '
23   --resource "{ 'Table': { 'DatabaseName': 'week7_lake', 'Name': 'customers'
24     } }" '
25   --permissions ALL
26
27 # 4. Column-scoped grant to marketing: no pii_email.
28 aws lakeformation grant-permissions '
29   --principal DataLakePrincipalIdentifier=arn:aws:iam::${AccountId}:role/
30   MarketingRole '
31   --resource "{ 'TableWithColumns': {
32     'DatabaseName': 'week7_lake', 'Name': 'customers',
33     'ColumnNames': [ 'customer_id', 'segment', 'city', 'region' ] } }" '
34   --permissions SELECT
35
36 # 5. Full-table grant to the analytics role for contrast.
37 aws lakeformation grant-permissions '
38   --principal DataLakePrincipalIdentifier=arn:aws:iam::${AccountId}:role/
39   AnalyticsRole '
40   --resource "{ 'Table': { 'DatabaseName': 'week7_lake', 'Name': 'customers'
41     } }" '
42   --permissions SELECT

```

Listing 8.3: Registering the location, revoking defaults, and granting column-scoped access.

Verification

Assume each role in turn (or use Athena workgroups mapped to roles) and run:

- **Marketing, allowed:** `SELECT customer_id, segment FROM week7_lake.customers LIMIT 10;` — succeeds.
- **Marketing, denied:** `SELECT pii_email FROM week7_lake.customers LIMIT 1;` — fails with an insufficient-permissions error naming the column.
- **Marketing, sneaky:** `SELECT * FROM week7_lake.customers LIMIT 1;` — also fails; `*` expands to include the excluded column.
- **Analytics:** both queries succeed.
- **Audit:** CloudTrail Lake Formation events show the grants; `aws lakeformation list-permissions` confirms the exact grant set.

Common Pitfall

Testing only the happy path. The security property is the *denied* query. Always demonstrate both the permitted projection and the denied PII access from the restricted principal, plus the `SELECT *` case—that is where column leaks usually hide.

8.6 Friday Use Case: A Data Clean Room for Ad-Tech

An advertising agency wants to measure campaign overlap between its clients' CRM data and a publisher's audience data. Neither party may see the other's raw customer records; both want aggregate answers like “how many users who saw campaign A later purchased?” and “what is the overlap audience size?”

8.6.1 Why Not CSV Exchange

The legacy pattern—export hashed emails to CSV, send by SFTP—fails on every axis: hashing emails is trivially reversible by dictionary attack, the receiving party holds a copy it controls, and there is no audit trail of what questions were asked. A clean room replaces data *movement* with query *governance*.

8.6.2 Architecture with AWS Clean Rooms

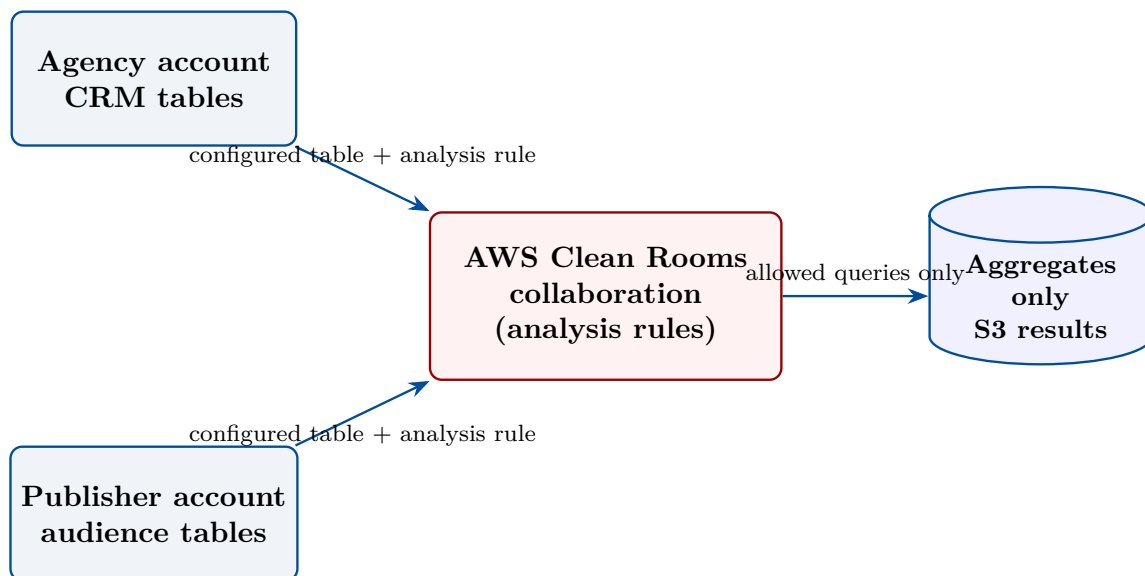


Figure 8.1: Clean-room collaboration: each party contributes a configured table behind analysis rules; only rule-compliant aggregate queries can run, and only aggregates leave.

Each party creates a *configured table* over its catalog data and attaches an *analysis rule*—typically an aggregation rule requiring `COUNT`, `SUM`, and friends with minimum group sizes, plus a list rule if overlap membership at the row level is contractually permitted. Queries can reference both configured tables only through the collaboration; results respect both rules; every query is logged to both parties. Differential-privacy options add mathematical noise where aggregate leakage is a concern.

8.6.3 Design Decisions

Join keys without disclosure. Overlap joins happen on the identifier column inside the collaboration; the aggregation rule prevents either side from selecting raw identifiers back out.

Minimum thresholds. Set minimum aggregation group sizes (for example, no group smaller than 100 users) so a cleverly sliced query cannot isolate an individual.

Governance evidence. Both parties retain query logs; the collaboration’s rules are the contract, and CloudTrail plus Clean Rooms logs are the audit trail that the contract was honored.

Common Pitfall

Hashing identifiers and calling it anonymization. A salted or unsalted hash of a low-entropy identifier (email, phone) is reversible by enumeration. If raw linkage must not be reconstructible, keep identifiers inside the collaboration and export only aggregates—or use tokenization services whose mapping table never leaves the owning account.

8.7 Interview Q&A

8.7.1 Concepts and Trade-offs

1. **Q: What does the default `IAMAllowedPrincipals` grant do, and why revoke it?**
A: It makes new catalog objects governed by IAM alone, bypassing Lake Formation. Until revoked on the database and each table, column filters and LF-Tag policies do not actually restrict anyone.
2. **Q: How do LF-Tags differ from named-resource grants?**
A: Named grants bind permissions to specific resources; LF-Tag grants bind them to tag expressions, so every correctly tagged table inherits policy automatically. Tags scale with data classes; names scale with table count.
3. **Q: What does column-level security protect that table grants cannot?**
A: Sensitive attributes on an otherwise shareable table—marketing can analyze segments and cities while `pii_email` is absent from their grant, with no view or copy required.
4. **Q: How does Redshift data sharing avoid copying data between accounts?**
A: The consumer queries the producer’s managed storage through the datashare: live, read-only access with the producer owning storage, freshness, and inclusion. No ETL, no duplicate storage bill, no drift.
5. **Q: Why is a clean room safer than exchanging raw CSV exports?**
A: No party receives the other’s records at all. Analysis rules restrict queries to approved aggregates with minimum group sizes, both sides get query logs, and raw identifiers never leave the owning account.
6. **Q: Why must both IAM and Lake Formation allow a lake query?**
A: Lake Formation grants are evaluated in addition to IAM. IAM governs API calls and service roles; Lake Formation governs catalog objects. The effective permission is the intersection.

8.8 Further Reading

The Lake Formation Developer Guide covers location registration, grants, LF-Tags, and data filters [51]. The Redshift data sharing documentation covers producer/consumer setup and

cross-account considerations [74]. The AWS Clean Rooms User Guide details collaborations, configured tables, and analysis rules [37]. For the governance review questions behind the Friday design, see the AWS Well-Architected Analytics Lens [3].

Key Takeaways

- Lake Formation adds table-, column-, row-, and cell-level governance on top of the Glue catalog, enforced uniformly across engines.
- Revoking `IAAllowedPrincipals` is the step that turns governance on; automate it.
- Column grants hide attributes; row filters hide records; LF-Tags make both scale with data classes instead of table counts.
- Redshift data sharing publishes live warehouse data across accounts with zero copies and producer-controlled freshness.
- Clean rooms replace risky data exchange with governed computation: analysis rules in, aggregates out, both sides audited.
- Fine-grained controls are only real when the denied case is tested, not just the permitted one.

8.9 Exercises

1. **(Conceptual)** Explain why Lake Formation permissions and IAM permissions are both evaluated, and which one can scope to a column.
2. **(Conceptual)** A team grants `SELECT` on all columns to “save time” and plans to add masks later. Describe two ways this fails before later arrives.
3. **(Hands-on)** Run the Thursday project, then add an LF-Tag `sensitivity=pii` on the `pii_email` column and re-express the marketing grant as a tag expression.
4. **(Hands-on)** Create a row filter limiting the marketing role to `region='EU'` and demonstrate two analysts receiving different row counts from the same query.
5. **(Hands-on)** Create the datashare from listing 8.2 between two sandbox accounts and query it from the consumer.
6. **(Design)** Design the LF-Tag taxonomy (keys, allowed values, ownership) for a lake with four domains and three sensitivity levels.
7. **(Design)** Specify the analysis rules for a clean-room query that measures campaign conversion overlap, including minimum group sizes and forbidden functions.
8. **(Architecture)** Draft the audit architecture for the clean room: which logs exist, who can see them, and how a disputed query is reconstructed.

8.10 Milestone 2 Capstone: Enterprise Data Lakehouse Build

Capstone Project

Mission. Deliver a governed enterprise lakehouse end to end: ingest five years of retail data into S3 as Parquet, catalog it with Glue, secure it with Lake Formation row/column policies per department, and serve executive dashboards through Redshift Spectrum materialized views.

Estimated time. 6–8 hours in a sandbox AWS account, plus review time.

What you'll practice.

- Zone design and Parquet conversion from Chapters 1 and 7.
- Catalog operations: crawlers, databases, partition projection.
- Lake Formation least-privilege: revoked defaults, named-column grants, LF-Tags, PII masks.
- Spectrum external schemas and materialized views for dashboard economics.

Scenario

A retail enterprise operates sales, inventory, and marketing domains. Five years of history (adapt the Chapter 5 generator) must become a governed analytical asset: raw and curated S3 zones, one catalog, department-scoped access (marketing never sees PII; finance sees everything), and executive dashboards cheap enough to refresh hourly.

Requirements

Functional requirements.

- Ingest five years of retail data into a raw zone; convert to curated Parquet partitioned by `year=/month=`.
- Crawl the curated zone; hand-write the raw-zone DDL; enable partition projection where partition counts warrant.
- Revoke `IAMAllowedPrincipals`; implement department policies with LF-Tags and named-column grants, including a mask on `pii_email`.
- Create a Spectrum external schema and at least two materialized views serving executive dashboard queries.

Non-functional requirements.

- The build is reproducible from a clean environment following a committed README.
- At least three automated data-quality checks (schema, null rate, row count) on the Parquet zones, with a failing-case demonstration.
- A crawler/job failure alarm wired to SNS, with evidence.
- A monthly cost estimate comparing Athena scanned bytes, Spectrum, and provisioned Redshift for the dashboard workload, with two optimization decisions documented.
- A security review: least-privilege grants, column masks on PII, no hardcoded credentials.

Reference Architecture

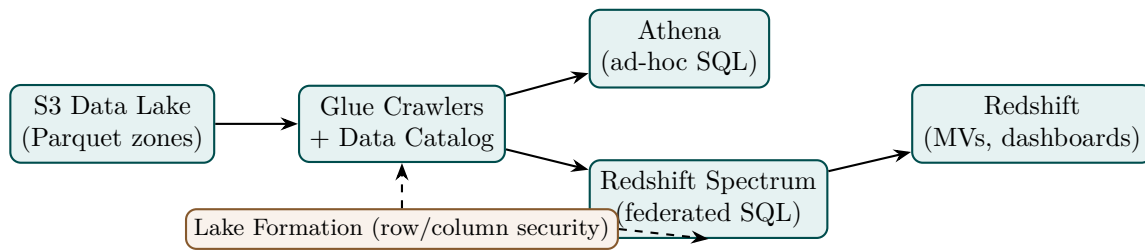


Figure 8.2: Milestone 2 reference architecture: governed enterprise lakehouse.

Implementation Steps

1. Land five years of retail data in the raw zone; convert to partitioned Parquet in the curated zone.
2. Run the crawler against the curated zone; validate schemas against the contract.
3. Register locations, revoke `IAMAllowedPrincipals`, and implement department policies with LF-Tags and named-column grants.
4. Create the Redshift external schema; build the two executive materialized views on top.
5. Wire the crawler failure alarm to SNS; add the three data-quality checks and demonstrate one failure.
6. Write the cost model and security review; tag all resources for cleanup.

Deliverables

- A committed architecture diagram covering S3, Glue Catalog, Lake Formation, and Redshift.
- The grant inventory (`list-permissions` export) proving department-scoped access and the PII mask.
- Data-quality check definitions with one failing-case run, and the SNS alarm evidence.
- The cost comparison (Athena vs. Spectrum vs. provisioned) with two documented optimizations.
- A README that rebuilds the lakehouse from a clean environment.

Acceptance Criteria

- A restricted department role demonstrably cannot select the PII column, including via `SELECT *`.
- Dashboard queries hit materialized views, not raw Spectrum scans; refresh is scheduled and logged.
- Every grant is expressible as least privilege: named columns or tag expressions, never `ALL` on a database.
- The quality gate halts the pipeline on failure and pages via SNS.
- The cost model uses measured bytes scanned, not assumptions.

- No credentials appear in any artifact; all access resolves through the catalog.

Stretch Goals

- Share the curated schema to a second account with a Redshift datashare and document the governance interplay.
- Add cell-level filters combining region row restrictions with column exclusions for a field-operations role.
- Convert the hottest curated table to Iceberg and demonstrate a schema-evolution event without breaking the crawler.
- Prototype the Friday clean room: two sandbox accounts, one configured table each, one aggregation-rule query.
- Add Storage Lens and Lake Formation audit dashboards to the monthly operations review.

Part III

Batch Processing and ETL Orchestration

Amazon EMR: Managed Hadoop and Spark

Learning Objectives

- Explain Amazon EMR cluster architecture: master, core, and task nodes, and which tiers can safely run on Spot capacity.
- Compare EMR on EC2, EMR Serverless, and EMR on EKS deployment modes and select per workload.
- Use EMRFS to treat S3 as the durable storage layer, decoupling compute lifetime from data lifetime.
- Submit and tune PySpark jobs that read from S3, compute distributed aggregations, and write Parquet back to S3.
- Design transient clusters for recurring batch work and justify them over long-running clusters.
- Plan a migration of a 100-node on-premises Cloudera cluster to AWS, prioritizing cost reduction and compute-storage decoupling.

Cross-Cloud Equivalence

Managed Hadoop/Spark exists on every cloud; the deployment-mode spectrum (full control to fully serverless) is the portable idea.

Capability	AWS	Azure	GCP
Managed Hadoop/Spark	EMR on EC2	HDInsight	Dataproc
Serverless Spark	EMR Serverless	Fabric Spark / Synapse Spark	Dataproc Serverless
Spark on Kubernetes	EMR on EKS	AKS-hosted Spark	Dataproc on GKE
Object-storage filesystem	EMRFS (S3)	ABFS (ADLS)	GCS connector
Cheap interruptible nodes	Spot task nodes	Spot node pools	Preemptible workers
Cluster lifetime model	Transient vs. long-running	Both	Both
Notebook integration	EMR Studio	Fabric notebooks	Vertex AI / Jupyter

Fabric folds Spark into its SaaS lakehouse: you get Spark pools and notebooks but never

node types. **Databricks** (not one of the six platforms, but the category leader) popularized the transient, job-scoped cluster pattern this chapter teaches. **Snowflake**'s Snowpark pushes Spark-like DataFrame code into the warehouse instead of a cluster. **MongoDB** stays the operational source that Spark jobs read from via the connector.

9.1 Week 9 Roadmap and Mental Model

AWS Glue (Chapter 10) covers most ETL needs serverlessly, but a class of workloads still wants the full open-source ecosystem: multi-framework clusters (Spark plus Hive plus Presto), custom libraries, long-running streaming jobs, or lift-and-shift migrations from on-premises Hadoop. Amazon EMR is that full-control option [13].

The mental shift from on-premises Hadoop is one sentence: *storage is S3; the cluster is ephemeral*. HDFS was the durable center of a Hadoop cluster; EMRFS makes S3 the durable center and reduces the cluster to rented compute that reads and writes objects. Once you believe that, transient clusters, Spot task nodes, and per-job sizing stop being exotic and become the obvious way to run batch analytics.

Definition

Amazon EMR is a managed platform for big-data frameworks—Spark, Hive, Presto, HBase, Flink, and others—on EC2, on EKS, or serverlessly. It provisions the framework stack, integrates it with S3 via EMRFS, and exposes steps, notebooks, and APIs for job submission.

Key Terms

Master node: runs cluster coordination (YARN ResourceManager, HDFS NameNode for local scratch). **Core node:** runs compute *and* HDFS DataNode duties; losing one risks local data. **Task node:** compute only; safe to lose, ideal for Spot. **EMRFS:** the S3 connector presenting object storage as the cluster's file system. **Bootstrap action:** script run on nodes at provisioning time for custom setup. **Step:** a unit of work (e.g., a Spark job) submitted to a cluster.

9.2 Monday: Cluster Architecture and Spot Capacity

9.2.1 Master, Core, and Task Nodes

An EMR on EC2 cluster has three node roles. The *master* node coordinates: it runs the resource manager and the HDFS NameNode for the cluster's local scratch space. *Core* nodes run compute tasks and host HDFS data, so losing a core node can lose in-flight intermediate state. *Task* nodes run compute only, hold no HDFS data, and exist purely to add parallel capacity [13]. This asymmetry drives the entire capacity strategy: **master and core on On-Demand or Reserved capacity; task nodes on Spot**.

9.2.2 Spot Instances for the Task Tier

Spot Instances sell spare EC2 capacity at steep discounts with two-minute interruption notice. Task nodes are the safe tier for Spot: when a task node vanishes, the framework reschedules its tasks onto surviving nodes and EMR provisions replacements. Losing Spot *core* nodes risks HDFS blocks and job failure; losing the master loses the cluster. Instance fleets (per-AZ, per-type weighting) increase the chance of fulfilling large Spot requests across pools.

Common Pitfall

Putting the whole cluster on Spot to save money. The discount is real and so is the interruption rate. Spot on the task tier commonly saves 60–80% of task-tier cost; Spot on core or master converts that saving into failed jobs and reprocessing. Scope Spot to the tier designed for interruption.

9.3 Tuesday: EMR on EC2 vs. EMR Serverless vs. EMR on EKS

9.3.1 Three Deployment Modes

- **EMR on EC2:** full control. You choose instances, frameworks, bootstrap actions, and cluster lifetime. Right for long-running clusters, exotic framework combinations, and migrations that replicate on-premises behavior.
- **EMR Serverless:** no clusters at all. You create an application (Spark or Hive), submit jobs, and pay per vCPU/GB-hour of actually used workers, which scale per job. Right for intermittent job fleets and teams that never want to size a node.
- **EMR on EKS:** EMR’s managed Spark runtime as pods on your Kubernetes cluster. Right when the platform team already runs EKS and wants data jobs to share its scheduling, quotas, and networking model.

Mode	You manage	Choose when
EMR on EC2	Nodes, scaling, lifetime, frameworks	Full control, multi-framework, lift-and-shift, steady load
EMR Serverless	Job config and code only	Intermittent jobs, no cluster expertise, per-job scaling
EMR on EKS	Kubernetes capacity; EMR manages runtime	EKS-standardized platform, shared compute with microservices

Table 9.1: EMR deployment modes compared.

Note

EMR Serverless still needs the architectural decisions of this chapter: worker sizing, Spark dynamic allocation, and S3 layout all remain yours. “Serverless” removes node management, not performance reasoning.

9.4 Wednesday: EMRFS—S3 as the Storage Layer

9.4.1 Decoupling Compute from Storage

EMRFS presents S3 as the cluster’s file system: Spark reads `s3://bucket/path` as if it were HDFS. The consequences are architectural. Clusters become disposable—terminate one, and the data is untouched. Multiple clusters read the same dataset concurrently. Storage scales to petabytes while compute scales to the job. And cluster lifetime becomes a cost decision: run a cluster for the two hours the job needs, not the 730 hours the month contains.

9.4.2 Transient Clusters and Bootstrap Actions

A *transient cluster* is created for a job or a day's schedule and terminated when the steps complete—typically launched by Step Functions, MWAA, or the EMR Steps API. Bootstrap actions run scripts on every node at provisioning (installing libraries, tuning configs), which is how transient clusters stay reproducible: the cluster definition is code, the data is in S3, and nothing of value lives on the cluster itself.

Tip

Keep local HDFS for what it is good at—shuffle scratch and intermediate spill within one job—and write every durable output to S3 via EMRFS. A job whose final output exists only in cluster HDFS is a job whose output dies with the cluster.

9.5 Thursday Hands-on Project: A PySpark Aggregation on EMR

Deploy an EMR cluster with Spark, submit a PySpark job that reads data from S3, performs a distributed aggregation, and writes Parquet back to S3.

Lab Objectives

- Launch an EMR on EC2 cluster with On-Demand core nodes and Spot task nodes via instance fleets.
- Submit a PySpark step against the Chapter 7 curated dataset.
- Verify Parquet output in S3 and inspect Spark UI evidence of the distributed aggregation.
- Terminate the cluster and confirm the data survives—the EMRFS lesson made physical.

Procedure

First, the job itself—an aggregation over the curated Parquet zone:

```

1 import sys
2
3 from pyspark.sql import SparkSession
4 from pyspark.sql.functions import col, sum as spark_sum
5
6 INPUT = sys.argv[1]    # e.g. s3://bucket/curated/sales_parquet/
7 OUTPUT = sys.argv[2]  # e.g. s3://bucket/analytics/monthly_revenue/
8
9 spark = SparkSession.builder.appName("week9-monthly-revenue").getOrCreate()
10
11 sales = spark.read.parquet(INPUT)
12
13 monthly = (
14     sales.groupBy("order_year", "order_month")
15         .agg(spark_sum("amount").alias("revenue"),
16             spark_sum("qty").alias("units"))
17     .orderBy("order_year", "order_month")
18 )
19
20 # coalesce keeps output file counts reasonable for a small dataset
21 monthly.coalesce(4).write.mode("overwrite").parquet(OUTPUT)
22

```

```
23 spark.stop()
```

Listing 9.1: PySpark job: distributed aggregation from S3 to Parquet.

Then create the cluster with a mixed capacity strategy and submit the step:

```
1 $Bucket = "acme-analytics-raw-123456789012"
2
3 aws emr create-cluster '
4   --name "week9-spark-lab" '
5   --release-label emr-7.5.0 '
6   --applications Name=Spark '
7   --use-default-roles '
8   --instance-fleets '
9     "InstanceFleetType=MASTER,TargetOnDemandCapacity=1,InstanceTypeConfigs=[{
10      InstanceType=m5.xlarge}]" '
11     "InstanceFleetType=CORE,TargetOnDemandCapacity=2,InstanceTypeConfigs=[{
12      InstanceType=m5.xlarge}]" '
13     "InstanceFleetType=TASK,TargetSpotCapacity=4,InstanceTypeConfigs=[{
14      InstanceType=m5.xlarge},{InstanceType=m5.2xlarge}]" '
15 --steps "[{'Name':'monthly-revenue','ActionOnFailure':'CONTINUE','
16   'Args':['spark-submit','
17     's3://$Bucket/jobs/monthly_revenue.py','
18     's3://$Bucket/curated/sales_parquet/','
19     's3://$Bucket/analytics/monthly_revenue/']}]" '
20 --auto-terminate
```

Listing 9.2: Creating an EMR cluster with Spot task nodes and submitting the Spark step.

Tip

`-auto-terminate` plus step submission makes this a transient cluster: it exists for exactly the job's lifetime. In production the same call is issued by Step Functions or MWAA per schedule, so cluster-hours track job-hours.

Verification

- The cluster reaches `WAITING/step COMPLETED` state; the step log shows the Spark application ID.
- `aws s3 ls s3://$Bucket/analytics/monthly_revenue/` shows Parquet part files.
- Reading the output back (`spark.read.parquet` or Athena over the prefix) shows one row per year-month with revenue and units.
- After `-auto-terminate` completes, the input and output prefixes are intact—storage outlived compute.
- The EMR console's Spot/on-demand split confirms only the task fleet used Spot.

Common Pitfall

Writing output to HDFS paths by habit. A job that ends with `.write.save("/data/out")` on a transient cluster writes to storage that is deleted minutes later. Parameterize output paths, require `s3://` prefixes in code review, and assert on them in job tests.

9.6 Friday Use Case: Migrating a 100-Node Cloudera Cluster

A financial-services firm runs a 100-node on-premises Cloudera cluster: 1.2 PB in HDFS, forty recurring Hive and Spark jobs, six petabytes of annual growth, and a hardware refresh quote that made leadership ask about the cloud. Priorities: reduce cost, decouple compute from storage, and keep the forty jobs’ business logic intact.

9.6.1 Migration Strategy

Phase 0 — inventory. Classify jobs by framework, schedule, data dependency, and SLA. In practice this inventory discovers that a third of the “cluster” is two heavy jobs and thirty-eight small ones—which drives the target split.

Phase 1 — data to S3. Use DataSync (Chapter 1) for the bulk HDFS-to-S3 copy, with incremental syncs until cutover. S3 becomes the system of record; the on-premises HDFS becomes the replica until decommission.

Phase 2 — compute mapping. The two heavy jobs get transient EMR on EC2 clusters sized per run. The thirty-eight small jobs, each intermittent, move to EMR Serverless—no cluster design at all. Hive-metastore-dependent jobs point at the Glue Data Catalog as the shared metastore (Chapter 7), which also opens the data to Athena.

Phase 3 — orchestration and cutover. Schedules move to MWAA (Chapter 12), which launches transient clusters per DAG run. Jobs cut over domain by domain with parallel-run reconciliation: old and new outputs compared for two cycles before the on-premises schedule is retired.

On-premises reality	AWS target	Why
100 always-on nodes	Transient clusters, 2–4 h/day	Pay per job-hour, not per month
HDFS 1.2 PB	S3 via EMRFS	Durable, decoupled, lifecycle-tiered
Hive metastore	Glue Data Catalog	Shared with Athena/Spectrum/Glue
One cluster for all jobs	Per-job sizing (EC2 + Serverless)	Heavy jobs no longer starve small ones
Fixed capacity	Spot task fleets	60–80% off the interruptible tier

Table 9.2: Cloudera-to-EMR migration mapping.

9.6.2 Cost Framing

The on-premises cluster’s cost is fixed regardless of utilization; inventory shows compute utilization under 25%. The AWS design bills for job-hours: roughly 100 node-hours per day across all forty jobs versus 2,400 node-hours per day provisioned on-premises. Even before the Spot discount on task capacity, compute cost drops by an order of magnitude; storage moves to lifecycle-tiered S3 at a fraction of SAN cost.

Common Pitfall

Replicating the on-premises shape in the cloud. A 100-node always-on EMR cluster is the refresh quote with a different logo. The migration’s value comes from changing the operating model—transient, per-job, decoupled—not from changing the vendor of the same architecture.

9.7 Interview Q&A

9.7.1 Concepts and Trade-offs

1. **Q: Core versus task nodes: which tier can safely run on Spot Instances?**
 A: Task nodes. They hold no HDFS data, so interruption costs only rescheduled tasks. Core nodes host HDFS blocks and the master coordinates the cluster; Spot on either risks job or cluster failure.
2. **Q: How does EMRFS decouple storage from compute?**
 A: It presents S3 as the cluster's file system, so durable data lives in object storage independent of any cluster. Clusters become disposable compute that reads and writes objects.
3. **Q: When would you pick EMR on EKS over EMR on EC2?**
 A: When the organization already runs Kubernetes as its compute standard and wants Spark jobs to share EKS scheduling, quotas, and networking—accepting Kubernetes operations in exchange.
4. **Q: What are bootstrap actions used for?**
 A: Running setup scripts on every node at provisioning—installing libraries, applying configuration—so transient clusters are reproducible from code.
5. **Q: Why prefer transient clusters for recurring batch jobs?**
 A: Cluster-hours then track job-hours. A two-hour daily job on a transient cluster bills two node-hours per day; an always-on cluster bills 24, mostly idle.
6. **Q: What stays on local HDFS in a well-designed EMR job?**
 A: Job-local scratch: shuffle and spill within one run. All durable inputs and outputs live in S3 via EMRFS.

9.8 Further Reading

The EMR Management Guide covers node types, instance fleets, Spot configuration, bootstrap actions, and the Steps API [13]; the Spark project documentation covers the DataFrame API used in the lab [84]; and Zaharia et al. provide the architectural background on Spark's unified engine [94]. For the storage-decoupling rationale, revisit the S3 foundations of Chapter 1 [33].

Key Takeaways

- EMR is the full-control big-data option: Spark, Hive, Presto, and friends on EC2, EKS, or serverless.
- Master and core nodes are precious; task nodes are disposable—that asymmetry is the Spot strategy.
- EMRFS makes S3 the durable layer, which makes transient, per-job clusters the default operating model.
- Choose deployment mode by organizational standard (EC2 control, Serverless simplicity, EKS consolidation), not by benchmark.
- Bootstrap actions and step submission turn clusters into reproducible code artifacts.
- Hadoop migrations create value by changing the operating model—decoupled, transient, per-job—not by relocating the same always-on cluster.

9.9 Exercises

1. **(Conceptual)** Explain why losing a core node is more dangerous than losing a task node during a running job.
2. **(Conceptual)** A team runs one always-on EMR cluster for forty small intermittent jobs. Describe the cost argument for EMR Serverless and one reason to stay on EC2.
3. **(Hands-on)** Run the Thursday lab, then rerun the step with Spot removed from the task fleet and compare step duration and interruption behavior.
4. **(Hands-on)** Add a bootstrap action installing a Python library, rebuild the cluster, and verify the library inside the job.
5. **(Hands-on)** Convert the lab to EMR Serverless: create an application, submit the same job, and compare time-to-first-task with cluster provisioning.
6. **(Design)** Given job runtimes of 10 min, 2 h, and 6 h, assign each to EC2, Serverless, or EKS with justification.
7. **(Design)** Design the reconciliation checks for the Friday migration's parallel-run phase: what proves the new output equals the old?
8. **(Architecture)** Sketch the MWAA integration for the migrated job fleet: DAG structure, cluster-per-run policy, and failure alerting (preview of Chapter 12).

9.10 Capstone Project: A Transient-Cluster Batch Platform

Capstone Project

Mission. Build a small but production-shaped batch platform: nightly transient EMR clusters process a multi-source dataset from S3, write curated Parquet, and publish to the Glue catalog—with Spot task capacity, reproducible bootstrap, and cost tracking per job.

Estimated time. 4–6 hours in a sandbox AWS account.

What you'll practice.

- Instance-fleet design with a Spot task tier.
- Parameterized PySpark jobs with S3-in/S3-out discipline.
- Transient cluster lifecycle driven by the Steps API.
- Per-job cost attribution for a batch estate.

Scenario

A logistics company runs three nightly jobs: a heavy route-optimization aggregation (2 h), a medium delivery-events join (30 min), and a light reference-data refresh (10 min). Today they share one always-on cluster. Rebuild the estate as transient, per-job clusters with a cost report that survives finance scrutiny.

Requirements

Functional requirements.

- Each job runs on its own transient cluster sized to its profile, with task-tier Spot capacity.
- All jobs read from and write to S3; curated outputs register in the Glue catalog.
- Bootstrap actions make every cluster reproducible; cluster definitions live in code.

- A daily cost report attributes cluster-hours (split On-Demand/Spot) to each job.

Non-functional requirements.

- No durable output ever lands on cluster-local storage.
- Job failure leaves no running cluster behind (auto-terminate on completion or failure).
- The cost report reconciles within 5% of the CUR or Cost Explorer figure for the tag set.

Implementation Steps

1. Parameterize the three jobs as listing 9.1 style scripts with input/output arguments.
2. Codify cluster creation per job (instance fleets, Spot task tier, bootstrap actions).
3. Drive the three runs from a scheduler (CLI cron for the lab; MWAA in the production sketch).
4. Register curated outputs with a crawler or explicit DDL; validate in Athena.
5. Tag every cluster with `job_name`; build the daily cost report from Cost Explorer by tag.
6. Write the one-page operating memo: what changed, what it costs, what can still break.

Deliverables

- Cluster-definition code and bootstrap scripts for all three jobs.
- Evidence of Spot usage confined to task fleets and of auto-termination.
- Athena query results over the curated outputs.
- The daily per-job cost report and the comparison against the always-on baseline.

Acceptance Criteria

- All durable data resides in S3; terminating any cluster loses nothing.
- Spot appears only on task tiers; interruptions are absorbed without job failure.
- Every job is reproducible from code with no console-only configuration.
- The cost report attributes at least 95% of EMR spend to named jobs.

Stretch Goals

- Move the light job to EMR Serverless and compare its cost and start latency.
- Add managed scaling to the heavy job's cluster and measure scale-down behavior mid-run.
- Introduce a Step Functions wrapper that retries a failed job on a fresh cluster with an SNS alert.
- Benchmark `coalesce` versus repartitioned output for Athena read performance.

Serverless ETL with AWS Glue

Learning Objectives

- Explain Glue’s serverless ETL model and the difference between DynamicFrames and Spark DataFrames.
- Build visual ETL pipelines in Glue Studio and profile data with Glue DataBrew.
- Make re-runs safe with job bookmarks and idempotent write patterns.
- Tune jobs with worker types, DPU sizing, and Flex execution, and reason about DPU billing.
- Evaluate Glue versus EMR for analyst-built daily reporting pipelines.
- Build a pipeline that joins two tables, drops nulls, transforms timestamps, and loads clean data into Redshift.

Cross-Cloud Equivalence

Serverless, catalog-integrated ETL is a universal pattern; only the branding differs.

Capability	AWS	Azure	GCP
Serverless ETL engine	AWS Glue (Spark)	Fabric Dataflow Gen2 / pipelines	Dataflow / Cloud Data Fusion
Visual ETL designer	Glue Studio	Data Factory designer	Data Fusion studio
Interactive data prep	Glue DataBrew	Power Query (Fabric)	Dataprep (Trifacta)
Shared catalog	Glue Data Catalog	OneLake catalog / Purview	Dataplex
Re-run state	Job bookmarks	Pipeline state / watermark columns	Job state / watermarking
Capacity unit	DPU-hour	Capacity units	vCPU-hr (Dataflow)
Batch spot discount	Flex execution	Low-priority pools	Preemptible/Spot workers

Snowflake approaches the same problems from inside the warehouse: streams, tasks, and Snowpark replace external ETL for in-warehouse transforms. **MongoDB** users meet this layer as the Atlas Data API or Spark connector feeding the lake. The Glue skill set—schema-aware transforms, idempotent re-runs, capacity tuning—transfers to all of them.

10.1 Week 10 Roadmap and Mental Model

Chapter 9 gave you full control of a big-data cluster; Week 10 asks when you should decline that control. AWS Glue is serverless Spark (and Python shell) with the Data Catalog built in:

you write the transformation, Glue provisions the workers, runs the job, and tears everything down [47]. For the large middle class of ETL work—daily joins, type fixes, format conversion, Redshift loads—Glue removes cluster operations entirely.

The week’s three engineering disciplines: understand the *abstraction* (DynamicFrames and when to drop to DataFrames), make every job *idempotent* (bookmarks plus overwrite-style writes), and manage *capacity economics* (DPUs, worker types, Flex).

Definition

AWS Glue is a serverless data integration service comprising an ETL engine (Spark and Python shell), a visual designer (Glue Studio), an interactive preparation tool (DataBrew), a scheduler and workflow engine, crawlers, and the Data Catalog shared with Athena, Spectrum, and EMR.

Key Terms

DynamicFrame: Glue’s schema-flexible distributed collection with resolve-choice semantics. **DPU**: data processing unit—4 vCPU + 16 GB; the billing and capacity unit. **Job bookmark**: persisted per-job state tracking processed inputs so re-runs process only new data. **Glue Studio**: the visual job designer that generates PySpark. **DataBrew**: the interactive, no-code profiling and cleaning tool. **Flex**: discounted spare-capacity execution for non-urgent jobs.

10.2 Monday: Glue ETL—DynamicFrames Versus Spark DataFrames

10.2.1 The DynamicFrame Abstraction

Glue jobs are Spark jobs with an extra abstraction. A Spark *DataFrame* has one rigid schema; a *DynamicFrame* is a collection of self-describing records designed for data whose schema is messy or evolving [47]. Its signature operations—`resolveChoice`, `relationalize`, `applyMapping`—handle the real-world cases: a field that is sometimes a string and sometimes a struct, nested JSON that must become relational tables.

```

1 import sys
2 from awsglue.context import GlueContext
3 from awsglue.dynamicframe import DynamicFrame
4 from awsglue.job import Job
5 from awsglue.utils import getResolvedOptions
6 from pyspark.context import SparkContext
7 from pyspark.sql.functions import col, to_timestamp
8
9 args = getResolvedOptions(sys.argv, ["JOB_NAME"])
10 sc = SparkContext()
11 glueContext = GlueContext(sc)
12 job = Job(glueContext)
13 job.init(args["JOB_NAME"], args)
14
15 orders = glueContext.create_dynamic_frame.from_catalog(
16     database="week7_lake", table_name="sales_parquet",
17     transformation_ctx="orders",
18 )
19 customers = glueContext.create_dynamic_frame.from_catalog(
20     database="week7_lake", table_name="customers",
21     transformation_ctx="customers",
22 )
23
24 # Drop to DataFrames for the join/filter/timestamp logic...
```

```

25 joined = (
26     orders.toDF()
27         .join(customers.toDF(), "customer_id")
28         .filter(col("customer_id").isNotNull() & col("amount").isNotNull())
29         .withColumn("order_ts", to_timestamp(col("order_date")))
30 )
31
32 # ...and back to a DynamicFrame for Glue's Redshift sink options.
33 out = DynamicFrame.fromDF(joined, glueContext, "out")
34
35 glueContext.write_dynamic_frame.from_options(
36     frame=out,
37     connection_type="redshift",
38     connection_options={
39         "url": "jdbc:redshift://week5-wg.123456789012.us-east-1.redshift-
serverless.amazonaws.com:5439/dev",
40         "dbtable": "analytics.sales_clean",
41         "redshiftTmpDir": "s3://acme-analytics-raw-123456789012/tmp/glue/",
42         "aws_iam_role": "arn:aws:iam::123456789012:role/RedshiftS3ReadRole",
43     },
44     transformation_ctx="redshift_sink",
45 )
46 job.commit()

```

Listing 10.1: Glue job skeleton: catalog in, DynamicFrame transforms, Redshift out.

When to use which: start from the catalog with DynamicFrames, convert to DataFrame (`.toDF()`) the moment you need the full Spark API—window functions, complex expressions, ML—and convert back for Glue-specific sinks. Most production jobs live mostly in DataFrame land; the DynamicFrame earns its keep at schema boundaries.

10.3 Tuesday: Glue Studio and DataBrew

10.3.1 Visual ETL for Working Teams

Glue Studio lets you compose sources, transforms, and targets on a canvas and generates the PySpark job underneath. Its honest role in a professional estate: fast prototyping, and a legible starting point for analysts. The generated code should graduate into version control, where engineers harden it—parameterize paths, add error handling, attach bookmarks.

10.3.2 DataBrew: Profiling Before Pipelines

Glue DataBrew is the interactive counterpart: point it at a dataset and it profiles columns (null rates, distributions, outliers) and offers 250+ no-code transformations as a visual “recipe” that can run on a schedule [46]. DataBrew’s highest-leverage use is *before* you write a Glue job—its profile tells you the null rates and type inconsistencies your job must handle. Chapter 12’s capstone uses DataBrew again as a data-quality gate.

Tip

DataBrew profile jobs scan the full dataset and bill accordingly. Profile a sampled S3 path during development; reserve full-dataset profiles for the pre-production validation run.

10.4 Wednesday: Bookmarks, DPUs, and Flex

10.4.1 Job Bookmarks and Idempotency

A job bookmark records what a job already processed—which S3 objects, which partition values—so a re-run processes only what is new [47]. Bookmarks solve the “what’s new” half of idempotency; the other half is the *write* side. A job that appends rows to the target will duplicate them on any re-run that reprocesses input. Production discipline: write with overwrite semantics scoped to the partition being rebuilt (or MERGE into Redshift), parameterize the run date, and keep bookmarks enabled so retries reprocess only new partitions.

Common Pitfall

Bookmark on, append write. Bookmarks prevent re-reading; they do nothing about re-writing. A mid-job failure retried with an append sink duplicates everything the first attempt wrote. Pair bookmarks with partition-overwrite or merge sinks, and make the pair a code-review checklist item.

10.4.2 Capacity and Cost: DPUs, Worker Types, Flex

A DPU is 4 vCPU and 16 GB of memory; jobs bill per DPU-second with a one-minute minimum [47]. Tuning levers: worker type (G.1X, G.2X and larger for memory-heavy shuffles), worker count (parallelism versus diminishing returns), and *Flex execution*, which runs the job on spare capacity at a discount in exchange for variable start times—ideal for non-urgent backfills and nightly jobs with slack in their window.

10.5 Thursday Hands-on Project: A Glue Studio Pipeline into Redshift

Build a visual ETL pipeline in Glue Studio that joins two tables, drops null values, transforms a timestamp, and loads the clean data into Redshift—then harden the generated script.

Lab Objectives

- Compose the pipeline visually: two catalog sources, a join, a null filter, a timestamp transform, a Redshift target.
- Inspect and parameterize the generated PySpark script.
- Enable bookmarks, run the job twice, and prove the second run loads nothing new.
- Introduce a bad record and observe the pipeline’s behavior.

Procedure

1. In Glue Studio, create a job from the visual canvas. Add two *AWS Glue Data Catalog* sources: `week7_lake.sales_parquet` and `week7_lake.customers`.
2. Add a *Join* transform on `customer_id`; a *Filter* transform dropping rows where `customer_id` IS NULL or `amount` IS NULL; a *Custom Transform* (or SQL) converting `order_date` to a true timestamp column `order_ts`.
3. Add an *Amazon Redshift* target writing `analytics.sales_clean` with the S3 temporary directory set to a dedicated `tmp/glue/` prefix.

4. Open the generated script tab. Replace hardcoded bucket and table literals with job parameters; save.
5. On the *Job details* tab: worker type G.1X, 4 workers, job bookmarks *Enabled*, Flex *off* for the timed first run.
6. Run the job; verify row counts in Redshift. Load one new day's files to S3, rerun, and verify only the new partition's rows arrive.

```

1  -- Row count and freshness
2  SELECT COUNT(*) AS rows_loaded,
3          MAX(order_ts) AS latest_order
4  FROM analytics.sales_clean;
5
6  -- After the second (bookmark) run, this delta must equal only new rows:
7  SELECT order_date::DATE, COUNT(*)
8  FROM analytics.sales_clean
9  GROUP BY 1
10 ORDER BY 1 DESC
11 LIMIT 5;
12
13 -- Null hygiene: both must return zero
14 SELECT COUNT(*) FROM analytics.sales_clean
15 WHERE customer_id IS NULL OR amount IS NULL;

```

Listing 10.2: Verification queries against the Redshift target.

Verification

- The Redshift table contains exactly the joined, filtered rows—no nulls in the enforced columns.
- The second run's Redshift delta equals the new data only, demonstrating bookmark behavior plus idempotent writes.
- The generated script is under version control with parameters externalized.
- A deliberately malformed record produces a documented, non-silent outcome (dropped by filter, or job failure—your stated choice).

10.6 Friday Use Case: Glue or EMR for the Analyst Team?

A team of eight business analysts must build and run daily reporting pipelines: join the day's extracts, clean types, load Redshift. None manage infrastructure; two know Python well. Their director asks whether to standardize on Glue or EMR.

10.6.1 The Evaluation

Skill fit. Glue Studio's visual canvas and generated code match analysts who think in transforms, not clusters. EMR's value—framework choice, instance tuning, long-running jobs—is precisely what this team does not want to operate.

Workload fit. The pipelines are daily, minutes-to-an-hour, schema-evolving, and catalog-centric: the Glue sweet spot. None are multi-framework, none run 24/7, none need exotic libraries beyond what custom Python wheel uploads provide.

Cost fit. Intermittent daily jobs bill only DPU-seconds used; there is no idle cluster. Flex execution discounts the overnight schedule further.

Where the answer would flip. Heavy continuous streaming, sub-second latency, framework requirements Glue lacks, or a platform team that already runs EKS—those point back to Chapter 9. State them in the recommendation so the decision is revisitable.

Criterion	AWS Glue	Amazon EMR
Operations burden	None (serverless)	Cluster lifecycle, even if transient
Skill required	SQL/Python, visual design	Spark/Hadoop operations
Workload shape	Batch, intermittent, catalog-centric	Continuous, multi-framework, exotic
Cost shape	DPU-seconds per run	Node-hours per cluster
Escape hatches	Drop to DataFrame; custom wheels	Anything the ecosystem offers

Table 10.1: Glue versus EMR for the analyst reporting estate.

Common Pitfall

Choosing the tool the builders cannot operate. The correct comparison is not feature lists; it is who owns a 2 a.m. failure. A serverless pipeline an analyst can rerun beats an elegant cluster only an absent engineer can restart.

10.7 Interview Q&A

10.7.1 Concepts and Trade-offs

- Q: What does a DynamicFrame offer that a Spark DataFrame does not?**
A: Self-describing records with schema-resolution operators—`resolveChoice`, `applyMapping`, `relationalize`—built for messy or evolving schemas and catalog-aware sinks. Convert to DataFrame when you need the full Spark API.
- Q: What do job bookmarks guarantee on a re-run?**
A: That inputs already processed are not reprocessed—new objects or partitions only. They do not make the sink idempotent; overwrite or merge writes must cover the output side.
- Q: When is Flex execution the cheaper choice?**
A: For non-urgent jobs—backfills, overnight batch with slack—that tolerate delayed starts. Flex runs on spare capacity at a discount; latency-sensitive or SLA-bound jobs stay on standard execution.
- Q: Why is Glue usually a better fit than EMR for analyst-built pipelines?**
A: No clusters to operate, a visual designer that generates real code, catalog integration, and per-second billing that matches intermittent daily runs.
- Q: What compute does one DPU represent for billing?**
A: 4 vCPU and 16 GB of memory, billed per second with a one-minute minimum. Worker type and count multiply from there.
- Q: What is DataBrew’s best use in a Glue estate?**
A: Interactive profiling and recipe prototyping before pipeline development—and, as Chapter 12 shows, a schedulable data-quality gate inside an orchestrated pipeline.

10.8 Further Reading

The Glue Developer Guide covers jobs, DynamicFrames, bookmarks, worker types, and Flex [47]; the DataBrew guide covers profiling and recipes [46]. Spark’s own documentation underpins everything the DataFrame half of a Glue job does [84, 94]. The Well-Architected Analytics Lens frames the Glue-versus-EMR operational trade [3].

Key Takeaways

- Glue is serverless Spark plus the catalog: the right default for intermittent, schema-evolving batch ETL.
- DynamicFrames handle messy schemas at the boundary; DataFrames carry the serious transform logic.
- Idempotency is a pair: bookmarks for the read side, overwrite-or-merge sinks for the write side, plus a run-date parameter.
- DPU sizing, worker types, and Flex execution are the cost levers; per-second billing rewards right-sizing.
- Glue Studio accelerates building; DataBrew accelerates understanding the data first.
- Tool selection is an operations decision: match the tool to the team that must run it at 2 a.m.

10.9 Exercises

1. **(Conceptual)** Explain why bookmarks alone do not prevent duplicate rows in Redshift, and name the two write-side patterns that complete idempotency.
2. **(Conceptual)** When would a Flex job be the wrong choice even if it is cheaper?
3. **(Hands-on)** Complete the Thursday lab; then modify the script to MERGE into Redshift by business key instead of inserting, and demonstrate a safe full rerun.
4. **(Hands-on)** Rerun the job on Flex three times and record start latency versus the standard runs.
5. **(Hands-on)** Profile the source dataset in DataBrew; list three findings that change the job’s transform logic.
6. **(Design)** A job’s runtime doubled after the source added ten columns. Walk through the DPU, worker-type, and layout checks you would run.
7. **(Design)** Design the parameter contract (run date, source prefix, target table, mode) for a reusable nightly Glue job template.
8. **(Architecture)** The analyst team now needs a five-minute micro-batch. Evaluate Glue streaming, EMR, and Kinesis/Flink (Chapters 13–15) for this requirement.

10.10 Capstone Project: An Idempotent Nightly ETL Fleet on Glue

Capstone Project

Mission. Convert three manual analyst workflows into a governed, idempotent nightly Glue fleet: catalog-driven sources, bookmarked incremental loads, Flex scheduling, and Redshift targets—with proof that failures rerun safely.

Estimated time. 4–6 hours in a sandbox AWS account.

What you'll practice.

- Graduating Glue Studio prototypes into version-controlled, parameterized jobs.
- Bookmark plus overwrite-sink idempotency, demonstrated under forced failure.
- Capacity tuning: worker types, DPU counts, Flex versus standard execution.
- Redshift sink hygiene: temporary directories, IAM roles, and merge semantics.

Scenario

Three analyst-built nightly pipelines currently run as manual notebook executions. They overlap in sources, drift in logic, and once a month someone reloads a full table by accident. Standardize them as Glue jobs with a shared template, and prove the failure story before finance relies on the numbers.

Requirements

Functional requirements.

- Three jobs from one parameterized template: run date, source, target, and mode as parameters.
- Bookmarks enabled; sinks use partition-overwrite or merge, never blind append.
- Nightly schedule on Flex where the window allows; alerting on failure to SNS.
- A forced mid-job failure followed by a rerun that produces zero duplicates.

Non-functional requirements.

- All job code in version control; no console-only configuration.
- DPU sizing justified with a before/after measurement for at least one job.
- A runbook entry per job: what it does, how to rerun safely, what the alarm means.

Implementation Steps

1. Extract the common template from the three workflows; externalize parameters.
2. Implement the sinks with overwrite/merge semantics; enable bookmarks.
3. Schedule on Flex; wire failures to SNS.
4. Kill one job mid-run (stop the run) and rerun; capture the zero-duplicate evidence.
5. Measure one job across two worker types and document the sizing decision.
6. Write the runbook entries and the template README.

Deliverables

- The template and three job configurations in version control.
- The forced-failure/rerun evidence with row-count comparisons.
- The sizing measurement and Flex-versus-standard latency notes.
- SNS alarm configuration evidence and the three runbook entries.

Acceptance Criteria

- Any job can be rerun for any date with no duplicates and no manual cleanup.
- Failure notifications arrive in SNS with job name and run ID.
- No pipeline step exists only in the console.
- Cost per nightly run is known and recorded per job.

Stretch Goals

- Add a Glue workflow chaining the three jobs with a shared failure path.
- Introduce a DataBrew profile job as a pre-load quality gate (preview of Chapter 12).
- Add Glue Data Quality rules to one job (deep coverage in Chapter 22).
- Parameterize the template enough to onboard a fourth pipeline in under an hour; demonstrate it.

Change Data Capture, Event-Driven Pipelines, and Data Contracts

Learning Objectives

- Configure AWS DMS change data capture (CDC) replication from a relational source to an S3 data lake.
- Monitor CDC health with the right CloudWatch metrics and detect silent lag.
- Build event-driven pipelines with Amazon EventBridge and lightweight AWS Lambda transformations.
- Define data contracts with the AWS Glue Schema Registry: Avro schemas, compatibility modes, and versioning.
- Distinguish breaking from non-breaking schema changes and assign contract ownership.
- Architect near-real-time synchronization from an on-premises Oracle database to Redshift for live BI.

Cross-Cloud Equivalence

CDC, event routing, and schema contracts are cloud-agnostic disciplines; each platform offers a managed shape of the same ideas.

Capability	AWS	Azure	GCP
Managed CDC replication	AWS DMS	Azure DMS / Data Factory CDC	Datastream
Event router	EventBridge	Event Grid	Eventarc
Serverless functions	Lambda	Azure Functions	Cloud Run functions
Schema registry	Glue Schema Registry	Azure Schema Registry	Managed Service for Schema Registry
Contract enforcement	In Glue jobs / registry checks	In pipelines	In Dataflow / registry checks
CDC lag metric	CDCLatencySource/Target	Replication latency metrics	Datastream freshness
Breaking-change policy	BACKWARD/FORWARD compatibility modes	Compatibility levels	Compatibility levels

Snowflake ingests CDC through Snowpipe and streams, applying the same contract ideas at the table boundary. **MongoDB** contributes change streams—the document-native CDC primitive that pairs naturally with everything in this chapter. **Fabric** routes database mirroring and event streams through OneLake, its version of the CDC-to-lake path.

11.1 Week 11 Roadmap and Mental Model

Batch windows are shrinking. The business question behind this week is: how fresh can analytical data be, and what breaks when it is continuous? The answer has two halves. The *plumbing* half is change data capture—reading the source database’s transaction log and replaying inserts, updates, and deletes into the lake continuously. The *contract* half is making sure the continuous stream cannot silently change shape: schemas are registered, compatibility is enforced, and producers cannot break consumers without an explicit versioned decision.

Weeks 1–10 mostly moved data on a schedule; this week moves it on *change*. The cadence changes the failure modes: instead of a failed nightly job, you get a silently lagging stream and a subtly drifting schema. The week’s tooling—DMS, EventBridge, Lambda, Schema Registry—maps onto those two failure modes directly.

Definition

Change data capture (CDC) is the technique of reading a source database’s transaction log to extract insert, update, and delete events as they happen, and applying them to a target—keeping the target continuously synchronized without re-scanning the source tables.

Key Terms

Full load + CDC: DMS task pattern that bulk-loads existing rows, then applies ongoing changes. **Replication instance:** the managed DMS compute that runs tasks. **CDCLatencySource / CDCLatencyTarget:** CloudWatch metrics measuring lag at the read and apply stages. **Event bus:** EventBridge’s routing fabric for events with pattern-matched rules. **Schema registry:** versioned store of schemas with enforced compatibility. **Compatibility mode:** BACKWARD, FORWARD, or FULL—which evolutions are permitted.

11.2 Monday: AWS DMS Architecture for Continuous Replication

11.2.1 Full Load Plus CDC

A DMS task has two phases. *Full load* copies existing table contents; *CDC* then tails the transaction log and applies changes as they arrive [42]. The pair matters separately: full load is a one-time bulk problem (parallelism, LOB handling, validation), while CDC is a forever problem (lag, log retention, source load). Tasks run on a replication instance you size like any compute: memory for cached changes during bursts, storage for spilled transactions.

11.2.2 CDC to the Lake

Replicating to S3, DMS writes full-load files and ongoing change files (with operation markers) under a partitioned prefix—commonly `table=/load-date=`. Downstream, a Glue job (Chapter 10) or a MERGE process applies the changes to a current-state table. This two-layer design—immutable change log in the lake, current-state table materialized downstream—preserves history, makes replays possible, and isolates consumers from replication mechanics.

Common Pitfall

Monitoring task status instead of lag. A DMS task shows *running* while minutes—or hours—behind the source. Alarm on `CDCLatencySource` and `CDCLatencyTarget`, and vali-

date that the source database retains enough transaction log history to survive a replication instance outage. Stale CDC with a “running” task is the classic silent failure.

11.3 Tuesday: Event-Driven Pipelines with EventBridge

11.3.1 Routing Events Instead of Polling

Amazon EventBridge is a serverless event bus: producers publish events, rules match patterns (source, detail-type, fields in the payload), and matched events fan out to targets—Lambda, Step Functions, SQS, Kinesis, and more [14]. For data engineering, its signature use is reacting to *arrival*: an S3 `ObjectCreated` event matching `raw/invoices/*.csv` starts the pipeline that validates and loads that file, replacing polling loops and fixed schedules with demand-driven execution.

11.3.2 Events as Pipeline Contracts

An event has a schema too. EventBridge schema discovery can infer and register event shapes, and mature teams treat event payloads like any other contract: versioned, documented, and owned by the producer. A rule pattern is a subscription to a contract; breaking the payload breaks every subscriber’s rule silently. That observation leads directly to Wednesday’s topic.

11.4 Wednesday: Lambda Transformations and Data Contracts

11.4.1 Lambda in the Data Path

AWS Lambda handles the transforms that are too small for a Spark cluster: validating a header row, enriching an event, splitting a batch file, calling an API per record [52]. The discipline is knowing the boundary: Lambda bills per millisecond with a 15-minute ceiling—perfect for per-file or per-event work, wrong for the 2 TB join. In this week’s architecture Lambda is the connective tissue: triggered by EventBridge, normalizing DMS output, enforcing schemas at ingestion.

11.4.2 Data Contracts with the Glue Schema Registry

A *data contract* is an enforceable agreement about a stream’s or table’s shape: field names, types, optionality, and evolution rules. The AWS Glue Schema Registry implements the machinery: Avro (or JSON Schema) definitions, versioned, with a *compatibility mode* that decides which evolutions are legal [48]:

- **BACKWARD**: new schema can read old data (consumers upgrade first). Deleting a field is allowed; adding a field without a default is not.
- **FORWARD**: old schema can read new data (producers upgrade first). Adding an optional field is allowed; deleting is not.
- **FULL**: both directions—the conservative production default. Only changes safe both ways (e.g., adding an optional field with a default) pass.

The contract taxonomy is worth memorizing: *adding an optional field* is non-breaking; *deleting a field, renaming a field, or changing a field’s type* breaks consumers. Ownership is the social half: the producer owns the contract, but consumers sign off on changes—the registry enforces the mechanics, the sign-off process enforces the intent.

```

1 aws glue create-registry --registry-name orders-registry
2
3 aws glue create-schema '
4   --registry-id RegistryName=orders-registry '
5   --schema-name order-events '
6   --data-format AVRO '
7   --compatibility FULL '
8   --schema-definition '{"type":"record","name":"Order","fields":[
9     {"name":"order_id","type":"string"},
10    {"name":"amount","type":"double"},
11    {"name":"coupon","type":["null","string"],"default":null}
12  ]}'

```

Listing 11.1: Registering an Avro schema with FULL compatibility.

Tip

Enforce contracts at *ingestion*, not at query time. A Glue job or Lambda that validates each batch against the registry rejects a breaking change before it poisons the lake; discovering the break at query time means the bad data is already persisted, versioned, and possibly replicated.

11.5 Thursday Hands-on Project: CDC from RDS PostgreSQL to the Lake

Configure a DMS task with CDC enabled to stream ongoing changes from an RDS PostgreSQL instance into an S3 data-lake bucket, then prove the stream is live and lag is observable.

Lab Objectives

- Prepare the PostgreSQL source for logical replication.
- Create a replication instance, source and target endpoints, and a full-load-plus-CDC task.
- Generate changes on the source and observe them arriving as change files in S3.
- Alarm on CDC lag metrics.

Procedure

1. **Enable logical replication** on the RDS instance: parameter group with `rds.logical_replication = 1`, reboot, and confirm `wal_level = logical`.
2. **Create the replication instance** (a `dms.t3.medium` suffices for the lab) in the same VPC as the database.
3. **Create endpoints**: a PostgreSQL source endpoint (with the master or a replication user) and an S3 target endpoint pointing at `s3://<bucket>/cdc/orders/`.
4. **Create the task** with migration type `full-load-and-cdc` and a table mapping over the `public.orders` table.

```

1 $Region = "us-east-1"
2
3 aws dms create-replication-instance '
4   --replication-instance-identifier week11-ri '
5   --replication-instance-class dms.t3.medium '
6   --allocated-storage 50
7

```

```

8 # (Endpoints omitted for brevity: source = RDS PostgreSQL, target = S3 prefix.)
9
10 $Mapping = @"
11 {
12   "rules": [{
13     "rule-type": "selection",
14     "rule-id": "1",
15     "rule-name": "orders",
16     "object-locator": { "schema-name": "public", "table-name": "orders" },
17     "rule-action": "include"
18   }]
19 }
20 "@
21 $Mapping | Set-Content -Path table-mappings.json -Encoding ascii
22
23 aws dms create-replication-task '
24   --replication-task-identifier week11-orders-cdc '
25   --source-endpoint-arn "arn:aws:dms:${Region}:123456789012:endpoint:SRC" '
26   --target-endpoint-arn "arn:aws:dms:${Region}:123456789012:endpoint:TGT" '
27   --replication-instance-arn "arn:aws:dms:${Region}:123456789012:rep:RI" '
28   --migration-type full-load-and-cdc '
29   --table-mappings file://table-mappings.json
30
31 aws dms start-replication-task '
32   --replication-task-arn "arn:aws:dms:${Region}:123456789012:task:TASK" '
33   --start-replication-task-type start-replication

```

Listing 11.2: Creating the DMS task with full load plus CDC.

Generate change traffic and verify:

```

1 INSERT INTO orders (order_id, amount, status) VALUES (90001, 129.99, 'NEW');
2 UPDATE orders SET status = 'PAID' WHERE order_id = 90001;
3 DELETE FROM orders WHERE order_id = 90001;

```

Listing 11.3: Source-side changes that should appear as CDC files.

Verification

- Full-load files appear under the S3 prefix before CDC files start.
- Each source change produces a change record with its operation marker (I/U/D) in the target prefix within the observed latency.
- CloudWatch shows `CDCLatencySource` and `CDCLatencyTarget` near zero under light load; an alarm exists on both.
- Stopping writes and querying the latest change files shows the delete as the final state of `order_id` 90001.

11.6 Friday Use Case: Near-Real-Time Oracle-to-Redshift Synchronization

A bank runs its core ledger on on-premises Oracle. The BI team needs Redshift dashboards reflecting ledger changes within five minutes; the nightly batch is no longer acceptable, and the Oracle estate cannot be modified beyond enabling supplemental logging.

11.6.1 Architecture

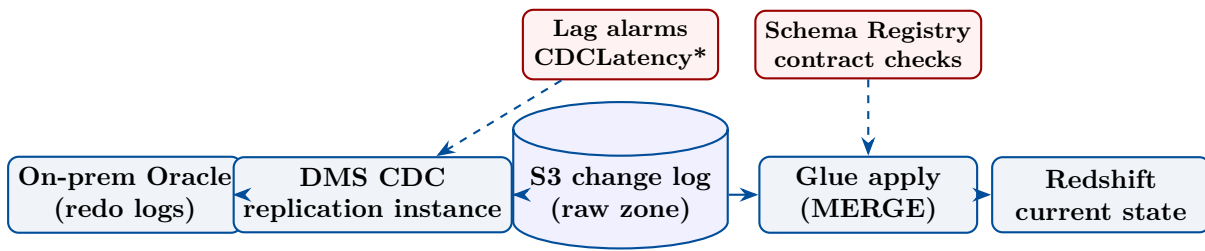


Figure 11.1: Near-real-time ledger sync: DMS tails Oracle redo into an S3 change log; a Glue MERGE maintains current state in Redshift; lag and contracts are monitored.

11.6.2 Design Decisions

Two targets, not one. DMS writes the raw change log to S3 (durable, replayable, auditable) and a second task—or a downstream Glue job reading the change log—applies MERGES into Redshift. Writing straight to Redshift couples the replication path to warehouse availability; the S3 stage decouples them and preserves the full history.

Freshness budget. The five-minute SLA is spent deliberately: DMS apply latency (sub-minute under normal load), S3 landing, a scheduled or event-driven Glue MERGE every two minutes, and Redshift commit time. Each stage gets a metric; the sum gets the alarm.

Log retention as an availability dependency. Oracle must retain enough redo/archive logs to cover the longest plausible replication outage. The runbook names the retention, the metric that watches it, and the re-snapshot procedure if it is ever exceeded.

Schema drift policy. Adding a nullable column flows: FULL compatibility permits it, the Glue apply adopts it, Redshift gains it via an expand-first migration (Chapter 24 formalizes this pattern). A type change or rename halts the pipeline and pages the owner—it is a contract violation, not a runtime event.

Common Pitfall

Treating the change log as disposable. The S3 change log is your replay capability and your audit trail. Lifecycle-expire it early and a Redshift-side corruption becomes unrecoverable without a new full load—which on a core ledger means an outage negotiation, not a retry.

11.7 Interview Q&A

11.7.1 Concepts and Trade-offs

1. **Q: Which CloudWatch metrics reveal CDC lag while a task shows “running”?**
A: `CDCLatencySource` (gap between the source commit and DMS reading it) and `CDCLatencyTarget` (gap between reading and applying at the target). Task status alone says nothing about freshness.
2. **Q: BACKWARD versus FULL compatibility: which permits deleting a field safely?**
A: BACKWARD (new schema reads old data, so consumers already tolerate the absence). FULL forbids deletions because old consumers could not read new data missing the field.

3. **Q: Who owns a data contract: producer, consumer, or both?**

A: The producer owns and versions the schema; consumers sign off on changes. The registry enforces mechanics; the sign-off workflow enforces intent.

4. **Q: Why enforce the schema at ingestion instead of at query time?**

A: By query time the bad data is already persisted—and possibly versioned, replicated, and transformed downstream. Ingestion enforcement rejects breaking changes before they become durable.

5. **Q: Name one breaking and one non-breaking schema change.**

A: Breaking: deleting a field, renaming it, or changing its type. Non-breaking: adding an optional field with a default.

6. **Q: Why stage DMS output in S3 before Redshift?**

A: The change log is replayable history and decouples replication from warehouse availability; MERGEs can be re-run, audited, and repointed without touching the source again.

11.8 Further Reading

The DMS User Guide and Best Practices cover replication instances, full-load-plus-CDC tasks, and source prerequisites [42, 59]. EventBridge and Lambda documentation cover event routing patterns and serverless transform limits [14, 52]. The Glue Schema Registry documentation details compatibility modes and versioning [48]. OpenLineage provides the emerging standard for tracing where contract-violating data flowed [93].

Key Takeaways

- CDC replays the transaction log; continuous freshness without re-scanning sources—and a new silent-failure mode called lag.
- Monitor `CDCLatencySource/CDCLatencyTarget`, and treat source log retention as an availability dependency.
- EventBridge replaces polling with routed arrival events; Lambda handles the per-event transforms too small for clusters.
- Data contracts make streams safe to evolve: registry, versions, and compatibility modes, owned by producers with consumer sign-off.
- Enforce contracts at ingestion; query-time discovery means the poison is already durable.
- Stage CDC through an S3 change log; current state is a rebuildable materialization, history is the asset.

11.9 Exercises

1. **(Conceptual)** Explain the difference between full load and CDC phases, and why each has distinct failure modes.
2. **(Conceptual)** Why does FULL compatibility permit adding an optional field but forbid deleting one?
3. **(Hands-on)** Complete the Thursday lab, then write a Glue job that MERGEs the change files into a current-state table; verify an update and a delete propagate.

4. **(Hands-on)** Register the `order-events` schema with FULL compatibility; attempt to register a version that deletes `amount` and capture the rejection.
5. **(Hands-on)** Build the EventBridge rule from arrival to action: S3 `ObjectCreated` on a prefix triggers a validating Lambda, with failures to an SQS DLQ.
6. **(Design)** Design the freshness monitoring for the Friday architecture: metrics per stage, alarm thresholds, and the five-minute budget arithmetic.
7. **(Design)** A producer must rename `amount` to `total_amount`. Write the multi-release contract-evolution plan that never breaks consumers.
8. **(Architecture)** Add OpenLineage emission to the Glue apply job and describe how it shortens the investigation of a contract violation.

11.10 Capstone Project: A Contract-Protected CDC Feed

Capstone Project

Mission. Stand up a production-shaped CDC feed: DMS full load plus CDC from PostgreSQL into an S3 change log, a contract-validating Glue apply into Redshift, EventBridge-driven arrival processing, and lag monitoring with alarms.

Estimated time. 5–7 hours in a sandbox AWS account.

What you'll practice.

- DMS replication design: instance sizing, endpoints, full-load-plus-CDC tasks.
- Contract enforcement at ingestion with the Glue Schema Registry.
- Event-driven apply pipelines with EventBridge and Lambda.
- Freshness observability: lag metrics, alarms, and re-snapshot runbooks.

Scenario

An order-management PostgreSQL database must feed Redshift analytics within ten minutes, continuously, with a guarantee that a schema change upstream can never silently corrupt the warehouse. The feed must survive a one-hour replication outage without data loss.

Requirements

Functional requirements.

- DMS full-load-plus-CDC task from RDS PostgreSQL to an S3 change log, partitioned by table and date.
- A Glue apply job that validates each batch against a FULL-compatibility registry schema before MERGEing into Redshift.
- EventBridge rules triggering validation on change-file arrival; failures route to a DLQ with an SNS alert.
- CloudWatch alarms on both CDC lag metrics and on apply-job failure.

Non-functional requirements.

- The feed tolerates a one-hour DMS outage with zero data loss (documented log-retention evidence).
- A deliberately breaking schema change is rejected at ingestion with the rejection logged.

- Rerunning the apply for any date is idempotent.

Implementation Steps

1. Prepare the source (logical replication) and create the DMS task per listing 11.2.
2. Register the contract per listing 11.1 and encode validation in the apply job.
3. Build the MERGE apply and prove idempotency by double application.
4. Wire EventBridge arrival events, the DLQ, and the SNS alerts.
5. Configure lag alarms; simulate an outage by stopping the task and document recovery.
6. Write the runbook: lag response, contract-violation response, re-snapshot procedure.

Deliverables

- Task configuration, table mappings, and endpoint definitions as code or exported JSON.
- The registry schema, its compatibility policy, and the validation code path.
- Evidence: change files, MERGE results, an injected breaking change rejected, alarm history from the outage drill.
- The freshness budget arithmetic and the runbook.

Acceptance Criteria

- End-to-end freshness meets the ten-minute target under normal load, measured.
- The one-hour outage drill recovers with no gap and no duplicate application.
- A breaking schema change cannot reach Redshift; the rejection is logged and alerted.
- Every stage of the freshness budget has a metric and an alarm.

Stretch Goals

- Add a second target task writing Parquet current-state snapshots for Athena access.
- Implement consumer sign-off as a pull-request workflow on schema versions.
- Replace scheduled apply with Kinesis Data Streams delivery (Chapter 13) and compare latency.
- Emit OpenLineage events from the apply job and trace one record end to end.

Pipeline Orchestration with Amazon MWAA and Step Functions

Learning Objectives

- Apply core Apache Airflow concepts: DAGs, operators, tasks, scheduling, retries, and XComs.
- Explain Amazon MWAA architecture and its operational model versus self-managed Airflow.
- Use AWS Step Functions for native serverless orchestration and know when it beats Airflow.
- Author a production-grade DAG that waits for S3 arrival, triggers a Glue job, runs an Athena validation, and alerts on failure.
- Design a resilient, self-healing orchestration layer for a 15-system banking reconciliation.
- Assemble the Milestone 3 automated batch ELT pipeline with a data-quality gate.

Cross-Cloud Equivalence

Orchestration splits into two families everywhere: general-purpose DAG schedulers and native state machines. Both exist on every cloud.

Capability	AWS	Azure	GCP
Managed Airflow	Amazon MWAA	Data Factory managed Airflow	Cloud Composer
Native state machine	Step Functions	Azure Logic Apps / Durable Functions	Workflows
File-arrival trigger	EventBridge + sensor	Event Grid trigger	Eventarc trigger
Failure alerting	SNS via operator/rule	Monitor alerts	Alerting to Pub/Sub
Inter-task data	XComs (small) / S3 (large)	Pipeline variables / ADLS	XComs / GCS
Retry semantics	Task retries + back-off	Activity retry policy	Step retry blocks

Fabric pipelines play the orchestrator role inside the SaaS boundary. **Snowflake** tasks and DAGs orchestrate in-warehouse work, and dbt (Chapter 24) orchestrates transformation graphs—the same dependency idea at the SQL layer. **MongoDB** triggers and Atlas schedulers cover the document-store corner.

12.1 Week 12 Roadmap and Mental Model

Weeks 9–11 built the engines; Week 12 conducts them. An orchestrator answers four questions a scheduler cron entry cannot: What depends on what? What happens when a step fails? What data does a run produce, and which run produced it? And who gets paged? This week’s two instruments approach those questions from different histories: **Airflow** (via Amazon MWAA) is the general-purpose, code-first DAG platform the industry standardized on [80]; **Step Functions** is AWS’s native, serverless state machine with deep service integrations and per-transition billing [57].

The professional skill is not picking one—it is knowing the boundary. Rule of thumb this chapter defends: Step Functions for event-driven, high-rate, AWS-native task graphs; Airflow for schedule-driven, dependency-heavy data estates with cross-day logic and a Python-native team.

Definition

Orchestration is the managed execution of a directed acyclic graph (DAG) of data tasks with dependency resolution, retry and alerting policy, run history, and parameterized scheduling—distinct from the *computation* the tasks themselves perform.

Key Terms

DAG: the code-defined graph of tasks and dependencies. **Operator**: a task template (run Glue job, run Athena query). **Sensor**: a polling operator that waits for a condition (file exists). **XCom**: Airflow’s small inter-task message channel. **catchup**: whether Airflow backfills missed schedules on deploy. **State machine**: Step Functions’ Amazon States Language definition of task flow. **Execution role / task role**: MWAA’s separation between the environment’s identity and per-task AWS permissions.

12.2 Monday: Airflow Concepts

12.2.1 DAGs, Operators, Tasks, and Scheduling

A DAG is Python code declaring tasks and their dependency edges; a *DAG run* is one scheduled or triggered execution; a *task instance* is one task within one run. Operators wrap actions—`GlueJobOperator`, `AthenaOperator`, `SnsPublishOperator`—and sensors poll for external conditions like file arrival. Scheduling is cron or timetable-based, with two settings that bite newcomers: **catchup** (if true, deploying a DAG with a past `start_date` instantly schedules every missed run) and retries with backoff, which convert transient failures—throttling, a brief S3 inconsistency—into eventual success instead of pages [80].

Common Pitfall

Deploying with `catchup=True` and a past `start_date`. The scheduler will cheerfully launch months of backfill runs against production systems the moment the DAG file lands. Set `catchup=False` unless you have designed the backfill, and design the backfill deliberately when you need one.

12.2.2 XComs and the Data Rule

XComs pass small values between tasks—a row count, an S3 key, a status. They are stored in the metadata database, so they are *not* a data channel: the rule is **data flows through S3, signals**

flow through XComs. A task that XComs a dataframe will bloat the metadata database and fail mysteriously at scale; a task that writes S3 and XComs the key scales indefinitely.

12.3 Tuesday: Amazon MWAA Architecture

12.3.1 Managed Airflow, Honest Boundaries

Amazon MWAA runs Airflow for you: a managed scheduler and web server, with workers autoscaling on ECS Fargate, all inside your VPC [19]. You choose an environment class, a worker range, and a requirements file. DAGs deploy by syncing an S3 bucket—the *dags* prefix is the deployment surface, so CI/CD for DAGs is “merge to main, sync to S3” (Chapter 24 formalizes this).

The operational realities: environment updates cause brief scheduler restarts; worker scaling lags sudden parallelism spikes; and every AWS call from a DAG uses the execution role, so least-privilege means that role can call exactly the Glue jobs, Athena workgroups, and SNS topics the DAGs need—no more.

Tip

Wire MWAA and Glue failures to CloudWatch alarms that publish to an SNS topic subscribed by the on-call runbook. Alarm on **FailedTasks** and DAG-duration anomalies—never rely on someone noticing a red box in the Airflow UI. Every alert must link to a runbook entry with triage steps.

12.4 Wednesday: AWS Step Functions

12.4.1 Native Serverless Orchestration

Step Functions defines workflows in Amazon States Language (JSON/YAML): Task states call AWS services (with optimized integrations that handle polling and callbacks), Choice states branch on payload fields, Map states fan out, and built-in retry/catch blocks implement per-state failure policy [57]. Standard workflows run up to a year with full execution history; Express workflows handle high-rate, sub-five-minute event processing.

Step Functions’ signature advantage for data pipelines is the *service integrations*: triggering a Glue job, running an EMR Serverless job, or executing an Athena query is a one-line Task state with IAM scoped per state. Its signature constraint is the 256 KB payload limit between states—the same “S3 for data, signals for state” rule as XComs, enforced by the platform.

12.4.2 Choosing Between MWAA and Step Functions

Criterion	MWAA (Airflow)	Step Functions
Sweet spot	Schedule-driven estates, cross-day dependencies, rich backfill	Event-driven, high-rate, AWS-service-centric flows
Programming model	Python DAGs, huge operator ecosystem	ASL JSON/YAML, visual designer
Billing	Environment-hours (always on)	Per state transition (per request for Express)
Data passing	XComs (discipline required)	Payloads capped at 256 KB (enforced)
Team fit	Python-native data engineers	Platform/app teams, event architectures

Table 12.1: Choosing the orchestrator: MWAA versus Step Functions.

12.5 Thursday Hands-on Project: A Production-Shaped MWAA DAG

Author and deploy a DAG that waits for a file in S3, triggers a Glue job, validates the load with Athena, and publishes an SNS alert on any failure.

Lab Objectives

- Deploy the DAG to an MWAA environment's S3 DAG bucket.
- Configure the execution role with least-privilege access to exactly the four services used.
- Prove the failure path: force the Glue job to fail and confirm the SNS alert fires.
- Explain every scheduling-related setting in the DAG header.

Procedure

The DAG below is the lab's complete artifact. Read it once for the task graph and once for the operational settings: `catchup=False`, retries with delay, a tuned sensor, and a failure-triggered alert.

```

1 from datetime import datetime
2 from airflow import DAG
3 from airflow.providers.amazon.aws.sensors.s3 import S3KeySensor
4 from airflow.providers.amazon.aws.operators.glue import GlueJobOperator
5 from airflow.providers.amazon.aws.operators.athena import AthenaOperator
6 from airflow.providers.amazon.aws.operators.sns import SnsPublishOperator
7
8 with DAG(
9     dag_id="daily_recon_pipeline",
10    schedule="@daily",
11    start_date=datetime(2026, 1, 1),
12    catchup=False,
13    default_args={"retries": 2, "retry_delay": 300},
14 ) as dag:
15     wait_for_file = S3KeySensor(
16         task_id="wait_for_file",
17         bucket_name="bank-raw-ingest",
18         bucket_key="recon/{{ ds }}/transactions.csv",
19         poke_interval=300, timeout=3600,

```

```

20 )
21 run_glue = GlueJobOperator(
22     task_id="run_glue", job_name="recon-etl", iam_role_name="GlueExecRole",
23 )
24 validate = AthenaOperator(
25     task_id="validate", query="SELECT COUNT(*) FROM recon.daily_load",
26     database="recon", output_location="s3://bank-athena-results/",
27 )
28 alert = SnsPublishOperator(
29     task_id="alert", target_arn="arn:aws:sns:us-east-1:123456789012:data-
30     alerts",
31     message="Recon pipeline failed for {{ ds }}", trigger_rule="one_failed"
32 )
33 wait_for_file >> run_glue >> validate >> alert

```

Listing 12.1: The daily reconciliation DAG for MWAA.

Deploy by uploading to the environment's DAG prefix:

```

1 $MwaaEnv = "week12-mwaa"
2 $DagBucket = (aws mwaa get-environment --name $MwaaEnv '
3     --query 'Environment.DagS3Path' --output text) -replace '/dags$', ''
4
5 aws s3 cp daily_recon_pipeline.py "s3://$DagBucket/dags/"
6
7 # The scheduler picks it up within a minute or two:
8 aws mwaa get-environment --name $MwaaEnv --query 'Environment.Status'

```

Listing 12.2: Deploying the DAG and watching it appear.

Design notes worth being able to defend in review:

- S3KeySensor with `poke_interval=300` trades freshness against S3 LIST request cost; `timeout=3600` bounds the wait so a missing file fails visibly rather than hanging the run forever.
- The alert task uses `trigger_rule="one_failed"` so it runs precisely on the failure path—without it, the alert would be skipped when its upstream is skipped.
- The `{{ ds }}` template makes runs partition-aligned and rerunnable for any logical date—idempotency at the orchestration layer, matching Chapter 10's job-layer discipline.

Verification

- Uploading a matching file triggers a run that completes all four tasks.
- A run with a deliberately broken Glue job ends with the SNS alert published and the Airflow UI showing the failure path.
- A run with no file fails after one hour with a clear sensor timeout, not an indefinite hang.
- Rerunning a past logical date reprocesses exactly that date's partition (job-layer book-marks honored).

12.6 Friday Use Case: Self-Healing Banking Reconciliation Across Fifteen Systems

A bank reconciles positions daily across fifteen systems—core banking, card processor, SWIFT gateway, general ledger, and eleven more. Files arrive at different times with different formats;

the close cannot complete until all fifteen reconcile; and yesterday’s manual rerun procedure takes an engineer four hours. Design the orchestration.

12.6.1 Architecture

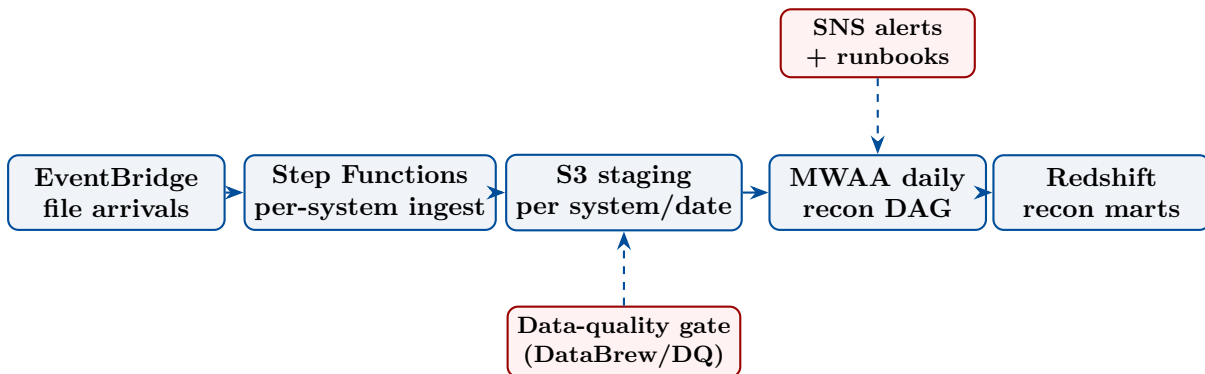


Figure 12.1: Two-tier orchestration: Step Functions handles per-arrival, per-system ingestion; MWAA owns the cross-system daily reconciliation with dependencies, retries, and alerting.

12.6.2 Design Decisions

Two tiers, each at its strength. Step Functions runs fifteen per-system ingestion state machines, triggered by EventBridge on each file’s arrival—event-driven, high-rate, AWS-native. MWAA owns the daily reconciliation DAG: one task per system reading that system’s staged partition, a barrier task requiring all fifteen, the cross-system match, and the exception report. Step Functions is wrong for the cross-day dependency logic; Airflow is wrong for fifteen independent arrival streams. The boundary is the staged S3 partition.

Self-healing by construction. Every task is idempotent and partition-scoped (`system/date`), so any rerun is safe. Retries with backoff absorb transient failures. A sensor per system replaces human polling, with a timeout that escalates to the on-call with the system name and expected SLA. The “manual rerun procedure” becomes “click *Clear* on the failed task”—the four-hour incident becomes a five-minute one.

Blast-radius controls. Each system’s task is isolated: a broken card-processor file fails that branch and alerts, while the other fourteen stage normally and the barrier reports exactly which system blocks the close. Partial close with flagged exceptions is a business decision; silent partial data is not an option.

Common Pitfall

A single mega-DAG that recomputes everything on any failure. If the failure of one of fifteen inputs forces reprocessing all fifteen, every incident is maximal. Partition-scoped, idempotent tasks with a barrier are what make reruns cheap enough to be the recovery strategy.

12.7 Interview Q&A

12.7.1 Concepts and Trade-offs

1. **Q: What does `catchup=False` prevent when you deploy a DAG?**

A: Backfill storms. With `catchup` enabled and a past `start_date`, the scheduler imme-

diately launches every missed historical run on deploy—often against production systems that did not expect it.

2. **Q: When are Step Functions a better orchestrator than Airflow?**

A: For event-driven, high-rate workflows composed of AWS service calls—per-file ingestion, per-order processing—where per-transition billing and native service integrations beat running an always-on scheduler.

3. **Q: What do retries with backoff protect a pipeline against?**

A: Transient failures—throttling, brief network or consistency hiccups—converting them into eventual success instead of pages. They do not fix deterministic failures; those need the failure path.

4. **Q: Why tune an S3KeySensor's poke_interval?**

A: Each poll is S3 LIST requests billed per thousand. A 300-second interval for a daily file balances freshness against request cost; a 5-second interval buys nothing and pays continuously.

5. **Q: When should data move through S3 instead of XComs?**

A: Always, for datasets. XComs live in the metadata database and suit small signals—keys, counts, statuses. Step Functions enforces the same rule with its 256 KB payload limit.

6. **Q: Why does the alert task need trigger_rule="one_failed"?**

A: Default trigger rules skip a task when its upstream is skipped. Without the explicit rule, the alert itself would be skipped on exactly the runs it exists for.

12.8 Further Reading

The Airflow documentation is the reference for DAG semantics, scheduling, sensors, and trigger rules [80]; the MWAA User Guide covers environment classes, worker scaling, and the S3 deployment model [19]; and the Step Functions Developer Guide covers ASL, service integrations, and Standard versus Express workflows [57]. The EventBridge guide covers the arrival-trigger half of the Friday architecture [14].

Key Takeaways

- Orchestration is dependency management, failure policy, and observability—computation belongs to the engines.
- Airflow's sharp edges are scheduling semantics: catchup, start dates, trigger rules, and retry policy.
- MWAA makes Airflow managed, not magic: execution-role least privilege and S3-based deployment are the operational core.
- Step Functions excels at event-driven AWS-native flows; its 256 KB payload limit enforces the S3-for-data rule.
- Sensors replace human polling; idempotent, partition-scoped tasks replace manual rerun procedures.
- Two-tier orchestration—native state machines for arrival, a DAG platform for cross-system logic—serves the 15-system reconciliation better than either tool alone.

12.9 Exercises

1. **(Conceptual)** Explain the difference between a DAG, a DAG run, and a task instance, and which one a “rerun for yesterday” operates on.
2. **(Conceptual)** Why is an always-on MWAA environment the wrong economic shape for a workload of pure per-arrival event reactions?
3. **(Hands-on)** Deploy the Thursday DAG, then add a second sensor for a `control/` file and make the Glue task require both arrivals.
4. **(Hands-on)** Force each failure mode—missing file, Glue failure, Athena failure—and record the alert behavior for each.
5. **(Hands-on)** Reimplement the DAG’s happy path as a Step Functions state machine and compare the operational evidence each tool gives you.
6. **(Design)** Design the barrier task for the fifteen-system recon: how does it report exactly which systems are missing, and what happens at the close deadline?
7. **(Design)** Write the MWAA execution-role policy for the Thursday DAG as least privilege; justify each statement.
8. **(Architecture)** Extend the Friday architecture with a cross-region failover posture: what must exist in the second region, and what state is hard to replicate?

12.10 Milestone 3 Capstone: Automated Batch ELT Pipeline

Capstone Project

Mission. Build an end-to-end automated batch ELT pipeline: an EventBridge rule fires on S3 upload, a Step Functions state machine runs a DataBrew data-quality check that gates an EMR Serverless Spark feature job, and the curated output loads into Redshift—with MWAA handling cross-day scheduling dependencies.

Estimated time. 6–8 hours in a sandbox AWS account.

What you'll practice.

- Event-driven triggering: S3 notifications to EventBridge to Step Functions.
- Data-quality gating inside a state machine with a Choice state.
- EMR Serverless job submission from orchestration.
- Failure routing to SNS with runbook-linked alerts.

Scenario

The bank’s analytics team receives daily feature extracts from an upstream vendor. Files arrive irregularly; quality varies; and the ML feature pipeline must never consume a failed-quality file. Automate arrival-to-Redshift with a hard quality gate and full failure visibility.

Requirements

Functional requirements.

- S3 event notifications to EventBridge; a rule matching the raw prefix starts the state machine with the object key in the payload.
- The state machine runs a DataBrew (or Glue Data Quality) check; a Choice state halts and alerts on failure.

- On pass, an EMR Serverless Spark job computes features; output is COPYed into Redshift.
- MWAA orchestrates the cross-day dependencies (yesterday's recon must finish before today's features publish).
- Failures at any state publish to SNS with the execution ARN.

Non-functional requirements.

- End-to-end runs from a clean environment per the README: upload a file, pipeline completes.
- At least three automated data-quality checks with a failing-case demonstration that halts the load.
- CloudWatch alarms on `ExecutionsFailed` and on MWAA task failures.
- A monthly cost estimate (EMR Serverless vCPU-hours, Glue DPUs, Step Functions transitions) with two optimization decisions documented.
- Task-scoped IAM roles per pipeline stage; secrets in Secrets Manager; no hardcoded credentials anywhere.

Reference Architecture

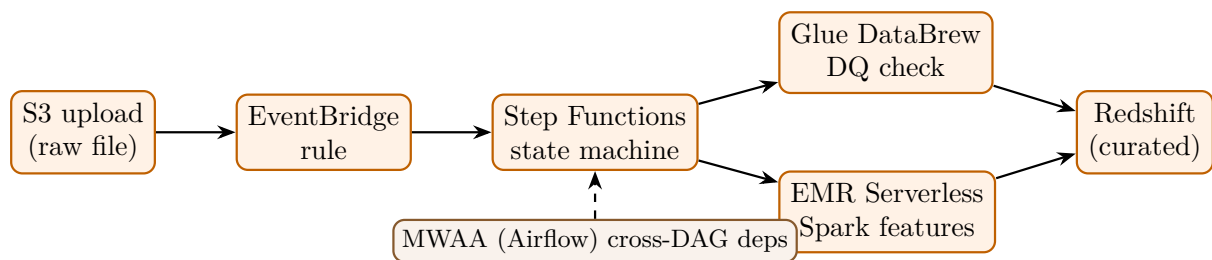


Figure 12.2: Milestone 3 reference architecture: automated batch ELT pipeline with a data-quality gate.

Implementation Steps

1. Enable S3 event notifications to EventBridge and create the prefix-matched rule targeting the state machine.
2. Define the state machine: DataBrew job Task, the Choice gate, EMR Serverless Task, Redshift COPY Task, SNS failure catches.
3. Create the EMR Serverless application and feature job; pass S3 keys—never data—between states (the 256 KB rule).
4. Build the MWAA DAG that sequences the cross-day dependency.
5. Wire the `ExecutionsFailed` alarm and MWAA failure alerts to SNS with runbook links.
6. Write the cost model and the security review; tag resources for cleanup.

The hardest state is the quality gate; the ASL Choice state is small but load-bearing:

```

1 "CheckDataQuality": {
2   "Type": "Choice",
3   "Choices": [{

```

```

4     "Variable": "$dq_status",
5     "StringEquals": "FAILED",
6     "Next": "AlertAndHalt"
7   }],
8   "Default": "RunSparkFeatures"
9 }

```

Listing 12.3: The data-quality gate as a Choice state.

Deliverables

- The state-machine definition (ASL), the DAG, and the EMR job code, all in version control.
- Evidence of the happy path (upload to Redshift rows) and of the DQ gate halting a bad file with an SNS alert.
- Alarm configurations and one triggered-alarm record.
- The cost estimate with two documented optimizations and the IAM/security review.

Acceptance Criteria

- A file upload triggers the pipeline with no manual step; the object key flows through as payload.
- A quality failure halts the load before any Spark or Redshift work and pages with context.
- No state passes bulk data through payloads; all data moves through S3 keys.
- Each pipeline stage runs under its own least-privilege role.
- The MWAA dependency demonstrably blocks today's publish until yesterday's recon succeeds.

Stretch Goals

- Add a Map state to fan out per-file processing when a manifest lists multiple files.
- Convert the DataBrew check to Glue Data Quality rules and compare rule expressiveness.
- Add Step Functions redrive policies and test recovery from a throttled EMR submission.
- Emit pipeline metrics (arrival-to-curated latency) to CloudWatch and dashboard the trend.

Part IV

Streaming and Real-Time Analytics

Message Queuing and Streaming Ingestion with Amazon Kinesis

Learning Objectives

- Decouple producers from consumers with Amazon SQS queues and SNS fan-out patterns.
- Size Kinesis Data Streams from shard throughput limits and design partition keys for per-entity ordering.
- Write retrying Boto3 producers and idempotent consumers, and interpret `IteratorAgeMilliseconds`.
- Compare Kinesis with Amazon MSK (managed Kafka) on operations, pricing, and ecosystem fit.
- Reason honestly about exactly-once processing on top of at-least-once delivery.
- Architect a global telemetry ingestion point for five million mobile-game clients.

Cross-Cloud Equivalence

Queues, pub/sub, and append-only logs are the three messaging primitives; every cloud ships all three.

Capability	AWS	Azure	GCP
Pull queue	Amazon SQS	Azure Queue Storage / Service Bus	Pub/Sub (pull)
Pub/sub fan-out	Amazon SNS	Event Grid / Service Bus topics	Pub/Sub
Durable streaming log	Kinesis Data Streams	Event Hubs	Pub/Sub (ordered)
Managed Kafka	Amazon MSK	Event Hubs for Kafka	Managed Service for Apache Kafka
Serverless stream processor	Managed Flink / Lambda	Stream Analytics	Dataflow
Scaling unit	Shard (1 MB/s in, 2 MB/s out)	Throughput units / partitions	Throughput quotas
Ordering scope	Per partition key	Per partition key	Per ordering key
Retention	24 h–365 days	1–90 days	Up to 31 days

Fabric Real-Time Intelligence wraps KQL databases and Eventstreams around the same log concept. **Snowflake** consumes streams via Snowpipe Streaming. **MongoDB** change streams are the database-native analog—a log of changes, replayable and resumable.

13.1 Week 13 Roadmap and Mental Model

Everything so far moved data in files or tables. Week 13 moves it as *events*: small, ordered, continuous, and consumed the moment they exist. The week’s core distinction is between a **queue**—a work-distribution channel where each message is processed once and deleted—and a **stream**—a durable, replayable log where many consumers read the same events at their own pace [15].

Three services cover the spectrum. **SQS** is the queue: decoupling, buffering, and per-message deletion. **SNS** is the fan-out: one publish, many subscribers. **Kinesis Data Streams (KDS)** is the log: shards, ordering keys, retention, and replay. And **Amazon MSK** is the ecosystem answer for organizations standardized on Apache Kafka. Choosing among them is Wednesday’s work; building correct producers and consumers on the log is Thursday’s.

Definition

A **streaming data platform** durably records an ordered, append-only sequence of events (the log), lets multiple independent consumers read it at their own pace, and guarantees per-key ordering and at-least-once delivery. Throughput is provisioned in shards (Kinesis) or partitions (Kafka).

Key Terms

Shard: Kinesis throughput unit—1 MB/s or 1,000 records/s in, 2 MB/s out. **Partition key**: the hashing input that routes a record to a shard and defines the ordering scope. **Sequence number**: a record’s position in its shard. **Iterator**: a consumer’s cursor into a shard. **IteratorAgeMilliseconds**: how far behind the newest record a consumer is. **DLQ**: dead-letter queue for poison messages. **Fan-out**: one event delivered to many independent consumers (SNS, or enhanced fan-out in KDS).

13.2 Monday: Decoupling with SQS and SNS

13.2.1 SQS: The Work Queue

Amazon SQS is a fully managed queue: producers send messages, consumers poll and delete them, and a visibility timeout hides in-flight messages from other consumers until processing completes or fails [32]. Standard queues offer at-least-once delivery and best-effort ordering at virtually unlimited throughput; FIFO queues offer exactly-once *delivery* within a deduplication window and strict per-group ordering, capped by throughput quotas per message group. Dead-letter queues capture messages that fail repeatedly, so one poison message cannot block a fleet.

13.2.2 SNS: The Fan-Out

Amazon SNS is push-based pub/sub: publishers send to a topic, and every subscription—SQS queues, Lambda functions, HTTPS endpoints, email—receives each message [31]. The canonical pattern is *fan-out to queues*: one SNS topic fronting several SQS queues, each queue owned by an independent consumer team. SNS handles distribution; SQS gives each consumer its own buffering, retry, and DLQ semantics. Message filtering lets subscribers receive only the subset they declare.

Tip

Never subscribe a Lambda directly to a high-volume topic when you may need replay or backpressure control. SNS-to-SQS-to-consumer preserves a durable buffer you can drain, inspect, and replay; a direct subscription drops events your consumer could not keep up with.

13.3 Tuesday: Kinesis Data Streams—Shards, Keys, and Scaling

13.3.1 The Shard Model

A Kinesis stream is a sequence of shards; each shard is an ordered, append-only log with fixed capacity: 1 MB/s or 1,000 records per second of ingress, 2 MB/s of egress per consumer (or a dedicated 2 MB/s per consumer with enhanced fan-out) [15]. Records carry a *partition key*; Kinesis hashes it to a shard, so all records with the same key land in the same shard in order. Retention runs from 24 hours to 365 days, during which any consumer can re-read—the property that makes streams replayable and queues not.

13.3.2 Sizing and Hot Shards

Sizing is arithmetic. Ingress: $\text{expected events/second} \times \text{average record size}$, divided by 1 MB/s and by 1,000 records/s; the shard count is the maximum of the two, plus headroom for growth and skew. The subtle failure is the *hot shard*: if one partition key dominates—a celebrity user, a single tenant—its shard saturates while others idle, and you get `ProvisionedThroughputExceeded` on a stream that is “mostly empty.” Defenses: choose high-cardinality keys, add entropy to hot keys (and handle the ordering consequences), and scale by split/merge or on-demand mode.

Common Pitfall

Partition keys with low cardinality or skew. Ordering scope and parallelism are the same choice: the partition key. A key with ten values pins your stream to ten hot spots forever. Choose keys that are high-cardinality *and* the unit of ordering you actually need—usually an entity ID like `sensor-17` or `user-92341`.

13.3.3 Consumers, Checkpointing, and Iterator Age

Consumers read via shard iterators and checkpoint their position—the Kinesis Client Library (KCL) does this in DynamoDB, tracking which sequence numbers each worker has processed. The health metric of the whole arrangement is `IteratorAgeMilliseconds`: the age of the last record the consumer read. Rising iterator age means consumers are falling behind; because retention is finite (24 hours by default), falling far enough behind is data loss. Alarm on it.

Common Pitfall

Blocking on poison records. Kinesis delivers at-least-once and in order per shard: a record your consumer cannot process will be redelivered forever, pinning the shard. Design consumers to be idempotent—deduplicate on a record ID (DynamoDB conditional put, Redis)—and route repeatedly failing records to a dead-letter location (SQS DLQ or S3 error prefix) instead of blocking the iterator.

13.4 Wednesday: Amazon MSK Versus Kinesis

13.4.1 Same Log, Different Contract

Amazon MSK is managed Apache Kafka: real brokers, real ZooKeeper/KRaft, the real Kafka API, in your VPC [18, 83]. The comparison with Kinesis is not “which is better” but “which operating model you are buying”:

Dimension	Kinesis Data Streams	Amazon MSK
API	AWS-native, IAM-integrated	Full Kafka protocol and ecosystem
Operations	Nearly none; shard or on-demand scaling	Broker fleet you size, patch windows, version upgrades
Pricing unit	Shard-hours + payload (or on-demand)	Broker instance-hours + storage
Portability	Low (AWS API)	High (standard Kafka clients move anywhere)
Retention	24 h–365 days, per stream	Configurable, disk-bounded per broker
Native integrations	Lambda triggers, Firehose, Flink, KCL	Kafka Connect, MirrorMaker, any Kafka client

Table 13.1: Kinesis versus MSK: an operating-model decision.

Choose Kinesis when the estate is AWS-native, the team is small, and IAM-integrated access matters. Choose MSK when Kafka compatibility is a requirement—existing clients, Connect connectors, ksqlDB, multi-cloud posture—or when retention and partitioning semantics must be Kafka’s exactly.

13.5 Thursday Hands-on Project: An IoT Temperature Stream

Create a Kinesis data stream, write a Python producer streaming mock IoT temperature readings with per-sensor ordering, and a consumer that prints what it reads.

Lab Objectives

- Create a stream and compute the shard count from first principles.
- Produce batches with `put_records`, handling partial failures with backoff.
- Consume with shard iterators and checkpoint mentally (KCL in production).
- Watch `IteratorAgeMilliseconds` move as the consumer lags and catches up.

Procedure

Fifty sensors reporting once per second at roughly 150 bytes each is ~ 7.5 KB/s and 50 records/s—one shard is comfortably sufficient; we use two to make the sharding visible.

```
1 aws kinesis create-stream --stream-name iot-temperature-stream --shard-count 2
2 aws kinesis describe-stream-summary --stream-name iot-temperature-stream
```

Listing 13.1: Creating the stream.

```
1 import json, random, time
2 import boto3
```

```

3
4 kinesis = boto3.client("kinesis", region_name="us-east-1")
5 STREAM = "iot-temperature-stream"
6
7 def make_reading(sensor_id: int) -> dict:
8     return {
9         "sensor_id": f"sensor-{sensor_id}",
10        "ts": int(time.time()),
11        "temp_c": round(random.uniform(15.0, 35.0), 2),
12        "battery_pct": random.randint(20, 100),
13    }
14
15 while True:
16     records = []
17     for sensor_id in range(1, 51):
18         records.append({
19             "Data": json.dumps(make_reading(sensor_id)).encode("utf-8"),
20             "PartitionKey": f"sensor-{sensor_id}", # per-sensor ordering
21         })
22     resp = kinesis.put_records(StreamName=STREAM, Records=records)
23     if resp["FailedRecordCount"]:
24         # retry only failed records with backoff (ProvisionedThroughputExceeded
25         )
26         failed = [r for r, o in zip(records, resp["Records"]) if "ErrorCode" in
27                     o]
28         time.sleep(0.5)
29         kinesis.put_records(StreamName=STREAM, Records=failed)
30         time.sleep(1)

```

Listing 13.2: Boto3 producer with per-sensor partition keys and partial-failure retry.

```

1 import json
2 import boto3
3
4 kinesis = boto3.client("kinesis", region_name="us-east-1")
5 STREAM = "iot-temperature-stream"
6
7 shards = kinesis.list_shards(StreamName=STREAM)["Shards"]
8 iterators = [
9     kinesis.get_shard_iterator(
10         StreamName=STREAM,
11         ShardId=s["ShardId"],
12         ShardIteratorType="LATEST",
13     )["ShardIterator"]
14     for s in shards
15 ]
16
17 while True:
18     for i, it in enumerate(iterators):
19         resp = kinesis.get_records(ShardIterator=it, Limit=100)
20         iterators[i] = resp["NextShardIterator"]
21         for rec in resp["Records"]:
22             reading = json.loads(rec["Data"])
23             print(f"shard {i}: {reading['sensor_id']} "
24                   f"{reading['temp_c']}C seq={rec['SequenceNumber']}")

```

Listing 13.3: Minimal shard-iterator consumer (KCL handles this machinery in production).

Verification

- Records from one sensor arrive in timestamp order within their shard—the partition-key guarantee made visible.

- Halting the consumer and restarting from `TRIM_HORIZON` replays retained records—the `stream-is-a-log` property.
- Forcing throughput past the shard limit (raise the sensor count) produces `ProvisionedThroughputExceeded` and exercises the retry path.
- CloudWatch shows `IteratorAgeMilliseconds` climbing while the consumer sleeps and falling when it resumes.

13.6 Friday Use Case: Global Telemetry for Five Million Mobile Games

A games publisher must ingest telemetry from five million concurrent mobile clients: session events, purchases, and crash reports, feeding live dashboards, anti-cheat analytics, and a data lake. The requirements state “exactly-once processing and strict message ordering.” Your job includes negotiating those requirements into something buildable.

13.6.1 Unpacking the Requirements

Ordering is only definable per entity: events for player p-123 must be ordered; player A’s events against player B’s need no order. That is exactly the partition-key guarantee—so ordering costs nothing if scoped per player, and is impossible (or ruinously serializing) if scoped globally.

Exactly-once splits into delivery and effect. Kinesis delivers at-least-once; exactly-once *effects* come from idempotent consumers: deduplicate on a unique event ID (the client generates a UUID per event; the consumer does a DynamoDB conditional put before writing), or make downstream writes naturally idempotent (upsert by event ID). Where the sink supports transactions (Chapter 15’s two-phase commit), end-to-end exactly-once is real; everywhere else, “exactly-once” is a contract about effects, not wires.

13.6.2 Architecture

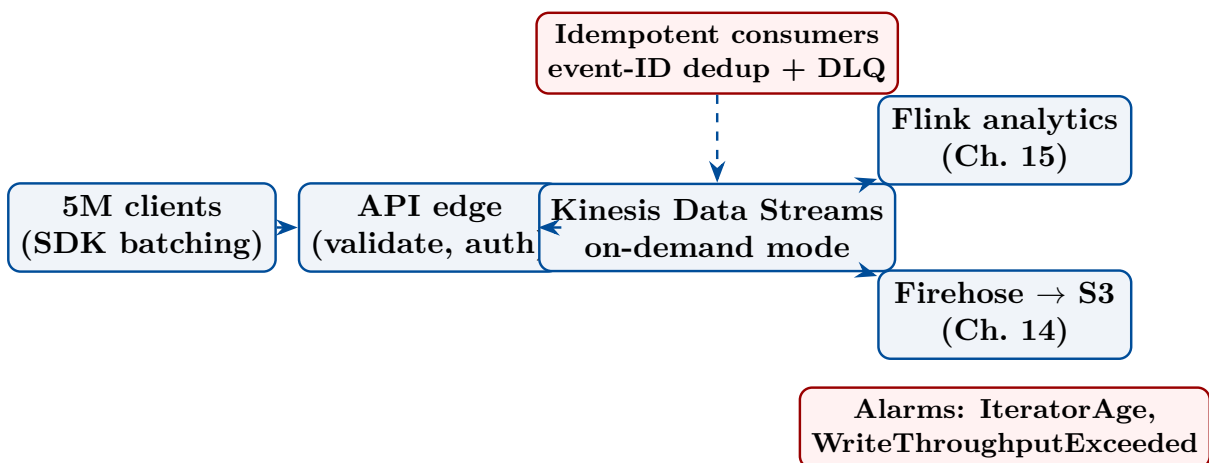


Figure 13.1: Global telemetry: edge validation, per-player partition keys, on-demand shards, independent consumers, idempotent writes.

13.6.3 Design Decisions

Client discipline. Clients batch events (10–50 per request) and tag each with a UUID and monotonically increasing per-session sequence. Batching converts $5\text{M} \times 1 \text{ event/s}$ into a manageable record rate; UUIDs make dedup possible.

Capacity. Peak estimate: $200,000 \text{ records/second} \times 300 \text{ bytes} \approx 60 \text{ MB/s ingress} \Rightarrow$ roughly 200 shards with headroom (records/s bound). On-demand capacity mode absorbs the daily player-count swings; alarms on write-throughput rejections catch the edge cases.

Regional strategy. Streams are regional; clients write to their nearest region’s stream, and cross-region replication into the analytics region consolidates. Global ordering is neither needed nor attempted; per-player ordering survives because a player is pinned to a region per session.

Common Pitfall

Accepting “exactly-once, globally ordered” as a literal requirement. Global order destroys parallelism; exactly-once delivery does not exist on unreliable networks—only exactly-once effects do. The professional move is to negotiate to per-entity ordering plus idempotent consumers, and write the dedup contract into the API.

13.7 Interview Q&A

13.7.1 Concepts and Trade-offs

1. **Q: Which two throughput limits determine how many shards a stream needs?**
A: Per-shard ingress of 1 MB/s *and* 1,000 records/s—the shard count is the max of the byte-based and record-based computations, plus headroom.
2. **Q: Why does partition-key choice control per-entity ordering?**
A: Kinesis hashes the key to a shard; a shard is the ordered unit. Equal keys share a shard, so per-key order is guaranteed—and a skewed key concentrates load on one shard.
3. **Q: What does a rising `IteratorAgeMilliseconds` warn about?**
A: Consumers falling behind. Since retention is finite (default 24 h), a consumer that stays behind long enough loses data; alarm before the retention horizon.
4. **Q: How do you build exactly-once effects on at-least-once delivery?**
A: Idempotent consumers: deduplicate on a unique event ID (conditional put in DynamoDB) or write idempotently (upsert by key), and route poison records to a DLQ.
5. **Q: Kinesis versus MSK: which is billed per shard-hour and record volume?**
A: Kinesis. MSK bills broker instance-hours and storage—you pay for a fleet; Kinesis bills for log capacity.
6. **Q: When does SQS FIFO suffice and Kinesis is overkill?**
A: When each message has exactly one consumer, no replay is needed, and throughput fits FIFO quotas. Streams earn their cost when multiple consumers read the same events or replay matters.

13.8 Further Reading

The Kinesis Data Streams Developer Guide covers shards, partition keys, iterators, KCL checkpointing, and on-demand mode [15]. The SQS and SNS guides cover visibility timeouts, DLQs, FIFO, and fan-out [32, 31]. The MSK Developer Guide and the Kafka documentation cover

the broker-side model [18, 83]. Streaming Systems provides the delivery-semantics and ordering theory behind Friday’s negotiation [1].

Key Takeaways

- Queues distribute work (consumed once); streams record a replayable log (read by many). Pick per consumer topology, not habit.
- Shard sizing is byte-and-record arithmetic; hot shards are a key-design failure, not a capacity failure.
- Partition keys simultaneously define ordering scope and parallelism—choose them as high-cardinality entity IDs.
- `IteratorAgeMilliseconds` against the retention window is the stream’s existential alarm.
- At-least-once delivery plus idempotent consumers plus DLQs is how exactly-once *effects* are actually built.
- Kinesis buys AWS-native simplicity; MSK buys Kafka compatibility and portability.

13.9 Exercises

1. **(Conceptual)** Explain the difference between message deletion in SQS and retention in Kinesis, and what each implies for adding a new consumer next month.
2. **(Conceptual)** Why does enhanced fan-out exist, given shards already have 2 MB/s of egress?
3. **(Hands-on)** Run the Thursday lab; then set the consumer’s iterator to `TRIM_HORIZON` after ten minutes and document what replays.
4. **(Hands-on)** Force a hot shard: send 80% of records with one partition key and observe which metrics move.
5. **(Hands-on)** Build the SNS-to-SQS fan-out: one topic, two queues, message filtering per queue, and a DLQ with a redrive policy.
6. **(Design)** Size the shard count for 50,000 events/second at 2 KB average, with 3× burst headroom. Show the arithmetic both ways.
7. **(Design)** Design the dedup contract for the Friday telemetry case: where the UUID is generated, where dedup state lives, and what the eviction window must cover.
8. **(Architecture)** The publisher must now serve European players from EU-resident infrastructure. Sketch the regional design and the consolidation path.

13.10 Capstone Project: A Resilient Telemetry Ingestion Tier

Capstone Project

Mission. Build a production-shaped telemetry ingestion tier: a sized and alarmed Kinesis stream, a retrying producer, an idempotent consumer with checkpointing and a DLQ, and the evidence that ordering, replay, and failure semantics all behave as designed.

Estimated time. 4–6 hours in a sandbox AWS account.

What you'll practice.

- Shard sizing arithmetic and on-demand versus provisioned mode selection.
- Producer partial-failure handling and partition-key design.
- Idempotent consumption with dedup state and poison-record routing.
- Stream observability: iterator age, throughput rejections, DLQ depth.

Scenario

The publisher from the Friday use case is piloting with 50,000 concurrent clients. Build the ingestion tier they will scale from, and prove its failure semantics before the pilot—not during the launch spike.

Requirements

Functional requirements.

- A stream sized from a documented workload estimate, with the arithmetic committed.
- A producer handling partial `put_records` failures with bounded backoff.
- A consumer that deduplicates on event ID, checkpoints progress, and routes three-time failures to a DLQ.
- Alarms on iterator age, write-throughput rejections, and DLQ depth, wired to SNS.

Non-functional requirements.

- A killed consumer resumes from its checkpoint with no duplicates downstream.
- A hot-key fault injection is detectable from metrics alone.
- The DLQ is inspectable and a redrive procedure is documented.

Implementation Steps

1. Write the sizing memo and create the stream accordingly.
2. Implement the producer (extend listing 13.2 with event UUIDs).
3. Implement the idempotent consumer with DynamoDB conditional-put dedup and checkpointing.
4. Inject faults: consumer kill, poison record, hot key; capture metric and DLQ evidence for each.
5. Configure the three alarms and demonstrate one firing.
6. Write the redrive runbook and the pilot go/no-go checklist.

Deliverables

- The sizing memo, stream configuration, and alarm definitions.
- Producer and consumer code with the dedup and DLQ paths.
- Fault-injection evidence: kill/resume without duplicates, poison record in the DLQ, hot-key metrics.
- The redrive runbook and go/no-go checklist.

Acceptance Criteria

- Per-player ordering holds under the full test load, verified programmatically.
- Replay from TRIM_HORIZON produces zero duplicate downstream effects.
- Poison records never block a shard longer than the retry policy allows.
- All three alarms have fired at least once during testing and are documented.

Stretch Goals

- Add a Lambda consumer triggered by the stream with a bisect-batch-on-error policy and compare its failure behavior.
- Attach a Firehose delivery stream as a second consumer (preview of Chapter 14).
- Replatform the consumer onto KCL with multi-instance rebalancing and document the lease table.
- Repeat the pilot sizing for MSK with the same workload and compare monthly cost.

Real-Time Delivery with Amazon Data Firehose

Learning Objectives

- Explain Firehose delivery-stream architecture: sources, buffering, transformation, and destinations.
- Configure inline Lambda transformations and reason about their latency and cost.
- Convert JSON to Parquet or ORC in flight and explain the downstream cost impact.
- Design delivery reliability: source-record backup, retry behavior, and failure destinations.
- Build an end-to-end pipeline from Kinesis Data Streams through Firehose to Parquet on S3.
- Architect a real-time anonymization pipeline for healthcare HL7 messages destined for Redshift.

Cross-Cloud Equivalence

Managed stream-to-sink delivery—buffer, transform, convert, land—exists in every platform, usually as a feature of the streaming service rather than a separate product.

Capability	AWS	Azure	GCP
Managed stream delivery	Data Firehose	Event Hubs Capture	Pub/Sub export subscriptions / Dataflow templates
Inline transformation	Lambda in Firehose	Capture (no transform) / Functions pattern	Dataflow / Cloud Run functions
In-flight format conversion	JSON → Parquet/ORC	Via downstream Synapse/Fabric	Via Dataflow
Warehouse delivery	Firehose → Redshift	Event Hubs → Synapse	Pub/Sub → BigQuery (Storage Write API)
Search delivery	Firehose → OpenSearch	Event Hubs → Elastic	Pub/Sub → Elasticsearch
Buffering model	Size + interval hints	Time/size capture windows	Subscription batching
Failure handling	S3 backup + error prefixes	Dead-lettering	Dead-letter topics

Snowflake Snowpipe Streaming is the same idea aimed at one destination: the warehouse

table. **Fabric** Eventstreams route to lakehouse tables with optional transformation. **Mon-goDB** sits on the receiving end via sink connectors or trigger-based loads.

14.1 Week 14 Roadmap and Mental Model

Week 13 ended at the stream: ordered, durable, replayable—and useless until the data lands somewhere analysts can touch. Amazon Data Firehose (formerly Kinesis Data Firehose) is the managed answer to the most common streaming question: *get these records from the stream into S3/Redshift/OpenSearch/HTTP, batched sensibly, optionally transformed, without me running any consumers* [9].

Firehose is deliberately not a stream processor. There are no windows, no joins, no state (that is Chapter 15). What it offers is the *delivery* concerns done right: buffering by size and time, inline Lambda transformation, JSON-to-Parquet conversion, compression, encryption, retries, and a backup copy of the raw records. The week’s craft is knowing exactly where its ceiling is—and never asking it to do Chapter 15’s job.

Definition

Amazon Data Firehose is a fully managed delivery service: it consumes records (from Kinesis Data Streams, MSK, or direct puts), buffers them by size and interval, optionally transforms them with Lambda and converts their format, and delivers the batch to a destination with retry and raw-backup handling.

Key Terms

Delivery stream: the Firehose resource binding a source to a destination with buffering and transform configuration. **Buffer hints:** size (1–128 MB) and interval (60–900 s for S3) controlling batch formation. **Inline transformation:** a Lambda invocation per buffer batch that can modify, enrich, or drop records. **Data format conversion:** schema-driven JSON-to-Parquet/ORC conversion using the Glue catalog. **Source-record backup:** a raw copy of incoming records to a separate S3 prefix, independent of delivery success.

14.2 Monday: Firehose Architecture and Destinations

14.2.1 The Delivery Pipeline

A delivery stream’s lifecycle per batch: read records from the source, invoke the transformation Lambda if configured, convert format if configured, buffer to the size or interval hint, compress and encrypt, write to the destination, and retry per destination-specific policy. Every stage is observable via CloudWatch metrics—`DeliveryToS3.Success`, `ExecuteProcessing.Success`, and friends—which is where your alarms live.

Destinations include S3 (the lake landing zone), Redshift (via an intermediate S3 COPY—Firehose stages, then issues COPY), OpenSearch, Splunk, and generic HTTP endpoints for third-party services. The Redshift path’s S3 staging detail matters operationally: failed or retried deliveries accumulate in the intermediate bucket unless lifecycle rules (Chapter 1) clean them.

14.2.2 Buffering Is a Latency–Efficiency Trade

Buffer hints trade delivery latency against file economics. A 60-second / 1 MB buffer lands data fast but writes many small files—which Chapter 7 taught us to fear at query time. A 15-minute / 128 MB buffer writes few large, Parquet-friendly files at the cost of freshness. There is no correct setting, only a stated freshness budget per destination: dashboards get tight buffers; the data lake gets fat ones.

Tip

When freshness and file economics conflict, split the stream: two delivery streams from the same Kinesis source—one with tight buffers to a “hot” prefix for near-real-time reads, one with fat buffers to the curated lake. Streams are multi-consumer by design; use it.

14.3 Tuesday: Inline Transformation with Lambda

14.3.1 Per-Batch Record Surgery

Attach a Lambda function and Firehose passes each buffered batch through it; the function returns every record marked `Ok` (deliver), `Dropped` (discard), or `ProcessingFailed` (retry, then route to the error prefix) [9, 52]. This is the right tool for per-record work: uppercasing fields, enriching from a lookup, masking a column, validating shape, splitting one record into two. It is the wrong tool for anything requiring state across records—dedup windows, joins, aggregations all belong in Flink (Chapter 15).

14.3.2 Latency and Cost Accounting

Each transformation adds the Lambda’s execution time to the batch’s path and bills Lambda GB-seconds plus Firehose’s transformation surcharge per GB. At high volume the discipline is: keep the function small (no network calls per record—batch lookups), keep memory modest, and set the retry duration so a flapping dependency cannot stall delivery indefinitely. Source-record backup (Wednesday) is what makes aggressive transformation safe: the untouched original always exists.

Common Pitfall

Per-record external API calls inside the transformation. A Lambda that calls an enrichment API per record multiplies a 10,000-record batch into 10,000 sequential or throttled calls, blowing the batch timeout and backing up the stream. Cache lookups in the function’s memory, batch the API, or move enrichment to a stateful stream processor.

14.4 Wednesday: In-Flight Format Conversion

14.4.1 JSON In, Parquet Out

Firehose can convert JSON records to Parquet or ORC during delivery, using a table definition in the Glue Data Catalog as the schema of record. The payoff compounds downstream: Chapter 7’s scanned-bytes economics apply immediately, no conversion job required. The constraints: incoming records must match the catalog schema (mismatches land in the error prefix), and the schema lives in one place—which is exactly the contract discipline of Chapter 11.

Note

Conversion failures and transformation failures write to different error prefixes under your S3 error output (`processing-failed/`, `format-conversion-failed/`). Alarm on both; a misconfigured schema version can silently divert an entire day's records into the error path while the main prefix stays green on *zero delivered*—a metric worth alarming on too.

14.4.2 Source-Record Backup

Enabling source-record backup writes every incoming record—pre-transformation—to a separate S3 prefix, regardless of delivery outcome. This is your replay capability and your audit trail: when the transformation has a bug, when Redshift rejects a load, when a regulator asks what *actually* arrived, the raw prefix answers. The cost is one extra S3 write path; the alternative is reconstructing history from a log that rolled past retention.

14.5 Thursday Hands-on Project: Stream to Parquet via Firehose

Connect a Firehose delivery stream to the Week 13 Kinesis stream, transform records with Lambda, convert to Parquet, and land in S3—with raw backup enabled.

Lab Objectives

- Create a delivery stream sourcing from `iot-temperature-stream`.
- Attach a Lambda that uppercases all incoming string values.
- Enable JSON-to-Parquet conversion against a Glue catalog table.
- Verify curated Parquet output, error prefixes, and the raw backup prefix.

Procedure

First the transformation function:

```

1 import base64
2 import json
3
4 def lambda_handler(event, context):
5     output = []
6     for record in event["records"]:
7         payload = json.loads(base64.b64decode(record["data"]))
8         transformed = {
9             k: (v.upper() if isinstance(v, str) else v)
10             for k, v in payload.items()
11         }
12         output.append({
13             "recordId": record["recordId"],
14             "result": "Ok",
15             "data": base64.b64encode(
16                 (json.dumps(transformed) + "\n").encode("utf-8")
17             ).decode("utf-8"),
18         })
19     return {"records": output}

```

Listing 14.1: Firehose transformation Lambda: uppercase every string field.

Register the schema the converter will use (matching the transformed record shape), then create the delivery stream:

```

1 $Bucket = "acme-analytics-raw-123456789012"
2
3 # Schema of record for JSON -> Parquet conversion
4 aws glue create-table --database-name week7_lake --table-input "{
5   "Name": "iot_readings",
6   "StorageDescriptor": {
7     "Columns": [
8       { "Name": "sensor_id", "Type": "string",
9       { "Name": "ts", "Type": "bigint",
10      { "Name": "temp_c", "Type": "double",
11      { "Name": "battery_pct", "Type": "int" } ],
12     "Location": "s3://$Bucket/curated/iot/",
13     "InputFormat": "org.apache.hadoop.hive ql.io.parquet.
MapredParquetInputFormat",
14     "OutputFormat": "org.apache.hadoop.hive ql.io.parquet.
MapredParquetOutputFormat",
15     "SerdeInfo": { "SerializationLibrary":
16       "org.apache.hadoop.hive ql.io.parquet.serde.ParquetHiveSerDe" } } } }
17
18 # Delivery stream: KDS source, Lambda transform, Parquet to S3, raw backup on
19 aws firehose create-delivery-stream --cli-input-json "{
20   "DeliveryStreamName": "iot-to-lake",
21   "DeliveryStreamType": "KinesisStreamAsSource",
22   "KinesisStreamSourceConfiguration": {
23     "KinesisStreamARN": "arn:aws:kinesis:us-east-1:123456789012:stream/iot-
temperature-stream",
24     "RoleARN": "arn:aws:iam::123456789012:role/FirehoseRole",
25   "ExtendedS3DestinationConfiguration": {
26     "BucketARN": "arn:aws:s3:::$Bucket",
27     "Prefix": "curated/iot/",
28     "ErrorOutputPrefix": "errors/iot/",
29     "BufferingHints": { "SizeInMBs": 64, "IntervalInSeconds": 300 },
30     "CompressionFormat": "UNCOMPRESSED",
31     "RoleARN": "arn:aws:iam::123456789012:role/FirehoseRole",
32     "ProcessingConfiguration": { "Enabled": true, "Processors": [
33       { "Type": "Lambda",
34       "Parameters": [ { "ParameterName": "LambdaArn",
35       "ParameterValue": "arn:aws:lambda:us-east-1:123456789012:function:
UppercaseTransform" } ] } ] },
36   "DataFormatConversionConfiguration": {
37     "Enabled": true,
38     "InputFormatConfiguration": { "Deserializer": {
39       "OpenXJsonSerDe": { } } },
40     "OutputFormatConfiguration": { "Serializer": {
41       "ParquetSerDe": { } } },
42     "SchemaConfiguration": {
43       "DatabaseName": "week7_lake", "TableName": "iot_readings",
44       "RoleARN": "arn:aws:iam::123456789012:role/FirehoseRole",
45       "CatalogId": "123456789012" } },
46   "S3BackupMode": "Enabled",
47   "S3BackupConfiguration": {
48     "BucketARN": "arn:aws:s3:::$Bucket",
49     "Prefix": "raw-backup/iot/",
50     "RoleARN": "arn:aws:iam::123456789012:role/FirehoseRole" } } } }

```

Listing 14.2: Glue table for schema-of-record, then the delivery stream.

(The listing keeps Firehose’s snake-case JSON verbatim inside PowerShell strings; in real deployments this configuration belongs in Terraform—Chapter 24.)

Verification

- Parquet files appear under `curated/iot/` within one buffer interval; Athena reads them via the Glue table and shows uppercased `sensor_id` values.
- The `raw-backup/iot/` prefix contains the pre-transformation JSON.
- A record violating the schema (drop a field from the producer) lands under `errors/iot/format-conversi` while valid records continue.
- CloudWatch shows `DeliveryToS3.Success` at 1.0 and non-zero `DeliveryToS3.DataFreshness` under the buffer interval.

14.6 Friday Use Case: Real-Time HL7 Anonymization into Redshift

A health network streams HL7 admission/discharge/transfer messages from hospital systems. Analysts need near-real-time occupancy and flow dashboards in Redshift, but messages contain patient identifiers that must never reach the warehouse. Design the anonymization pipeline.

14.6.1 Architecture

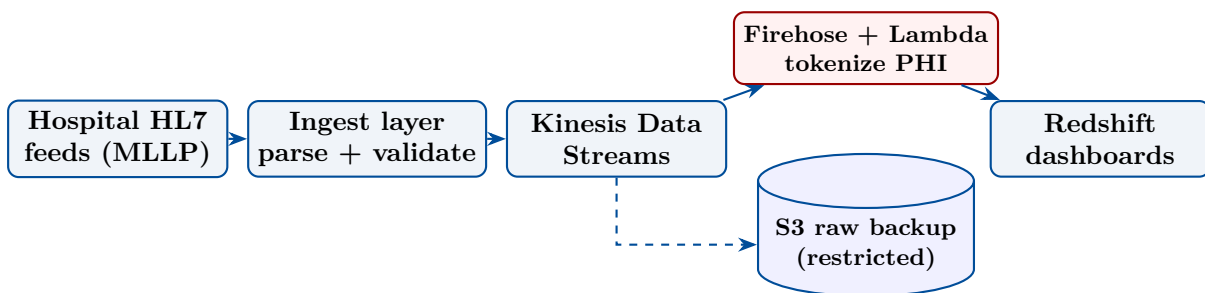


Figure 14.1: HL7 anonymization pipeline: tokenization happens in the delivery path; only de-identified records reach Redshift; the restricted raw backup preserves originals under separate controls.

14.6.2 Design Decisions

Tokenize, don’t mask. The Lambda transformation replaces the patient identifier with a token from a dedicated tokenization store (deterministic, so the same patient links across messages; mapping table in a separate account with separate keys). Irreversible masking would break longitudinal analysis; raw identifiers would break the compliance boundary. Chapter 23 covers the erasure implications.

Two destinations, two trust levels. The Redshift destination receives only tokenized records via Firehose’s COPY path. The raw backup—which contains PHI—lands in a bucket with Object Lock, separate KMS keys, and Lake Formation grants limited to the privacy team. The backup exists for replay and audit, not for analytics.

Failure discipline. `ProcessingFailed` records route to the error prefix with alarmed depth; a failing tokenizer must never fail-open to delivering raw PHI. The Lambda’s retry budget is short and the alarm is loud, because the alternative to latency here is not risk—it is compliance exposure.

Freshness. Occupancy dashboards tolerate five minutes; the buffer is 5 minutes / 16 MB, and the intermediate Redshift staging prefix has a lifecycle rule because failed COPY attempts accumulate there silently.

Common Pitfall

Anonymizing after the warehouse. Any design that lands PHI in Redshift “temporarily” has already failed the requirement—Redshift snapshots, logs, and Spectrum exports all inherit the copy. De-identification belongs in the delivery path, before the first analytical write.

14.7 Interview Q&A

14.7.1 Concepts and Trade-offs

1. **Q: Name three native Firehose destinations.**

A: S3, Redshift (via an intermediate S3 COPY), and OpenSearch—plus Splunk and generic HTTP endpoints.

2. **Q: Why enable source-record backup to S3 on a delivery stream?**

A: It preserves the pre-transformation originals regardless of delivery success: replay after a transformation bug, audit of what actually arrived, and recovery when a destination rejects records.

3. **Q: What do inline Lambda transformations add to latency and cost?**

A: The function’s execution time per batch plus Lambda GB-seconds and Firehose’s per-GB transformation surcharge. Small, stateless functions keep both modest; per-record network calls explode them.

4. **Q: Why convert JSON to Parquet during Firehose delivery?**

A: It eliminates a separate conversion job and makes the lake columnar at landing—the scanned-bytes cost savings (Chapter 7) apply from the first query.

5. **Q: Where does data land when Redshift delivery keeps failing?**

A: In the intermediate S3 staging prefix Firehose uses for its COPY—which is why that prefix needs lifecycle rules and the `DeliveryToRedshift.Success` metric needs an alarm.

6. **Q: What can Firehose’s Lambda transformation never correctly do?**

A: Anything requiring state across records: dedup windows, joins, aggregations. Those belong in a stream processor (Chapter 15).

14.8 Further Reading

The Firehose Developer Guide covers buffering, transformation, format conversion, backup, and per-destination retry behavior [9]; the Lambda guide covers the batch-response contract and execution model [52]. The Glue catalog’s schema-of-record role is documented with the Data Catalog [44], and the privacy patterns build on Chapter 1’s encryption and Chapter 8’s governance [50, 51].

Key Takeaways

- Firehose is managed delivery, not stream processing: buffer, transform per record, convert format, land, retry, back up.

- Buffer hints are a stated freshness–efficiency trade per destination; split streams when one stream needs both ends.
- Inline Lambda is for per-record surgery; state belongs in Flink; per-record API calls are the classic scaling bug.
- In-flight Parquet conversion makes the lake columnar at landing, with the Glue catalog as schema of record.
- Source-record backup is the replay and audit floor—enable it before you need it.
- De-identification belongs in the delivery path; a warehouse that ever held raw PHI inherits that copy everywhere.

14.9 Exercises

1. **(Conceptual)** Explain why Firehose-to-Redshift is a two-hop delivery and what operational duty the hop creates.
2. **(Conceptual)** A team wants exactly-once delivery into S3 from Firehose. Explain what Firehose actually guarantees and how to get idempotent effects downstream.
3. **(Hands-on)** Complete the Thursday lab; then change the buffer to 1 MB / 60 s and quantify the small-file explosion with an Athena query-time comparison.
4. **(Hands-on)** Break the transformation Lambda (raise an exception) and document where records go and which metrics move.
5. **(Hands-on)** Point a second delivery stream at the same Kinesis stream with different buffer hints and compare the two prefixes.
6. **(Design)** Design the tokenization store for the HL7 case: consistency requirements, latency budget, key management, and the re-identification process for authorized care use.
7. **(Design)** A vendor needs a subset of the stream via HTTPS. Design the HTTP-endpoint delivery with its retry and failure posture.
8. **(Architecture)** Add Macie (Chapter 23) scanning of the raw-backup prefix and describe the finding workflow for PHI discovered where none should persist.

14.10 Capstone Project: A Governed Streaming Delivery Layer

Capstone Project

Mission. Build the delivery layer for a streaming platform: one Kinesis stream feeding governed delivery to a Parquet lake and a Redshift serving tier, with inline transformation, in-flight conversion, raw backup, and full failure-path evidence.

Estimated time. 4–6 hours in a sandbox AWS account.

What you'll practice.

- Multi-destination delivery design from a single stream.
- Transformation and format-conversion configuration with error-path observability.
- Buffer-hint tuning against a stated freshness budget.
- Backup, lifecycle, and alarm hygiene for every prefix Firehose touches.

Scenario

An e-commerce company streams order events. Analysts query the curated lake (Parquet, five-minute freshness acceptable); the operations dashboard reads Redshift (one-minute freshness desired); compliance requires a tamper-evident raw archive. One stream, three requirements.

Requirements

Functional requirements.

- One Kinesis stream; two delivery streams (lake and Redshift) plus raw backup on both.
- The lake path converts to Parquet against a catalog schema; the Redshift path stages through a lifecycle-managed prefix.
- An inline transformation normalizes currency codes and drops internal-only fields before any analytical landing.
- Alarms on delivery success, transformation failures, error-prefix depth, and data freshness.

Non-functional requirements.

- Every prefix Firehose writes has a lifecycle policy and an owner.
- The internal-only fields are demonstrably absent from lake and warehouse, verified by query and by Macie-style scan.
- A destination outage drill shows records preserved and recoverable.

Implementation Steps

1. Create the stream and register the catalog schema for conversion.
2. Build the two delivery streams with their buffer postures.
3. Implement and test the transformation Lambda, including the **Dropped** path for internal fields.
4. Configure backup prefixes with Object Lock on the compliance archive.
5. Run the outage drill: disable the Redshift endpoint, observe, restore, recover.
6. Write the freshness budget table and the alarm rationale.

Deliverables

- Delivery-stream configurations as code or exported JSON.
- Query evidence: Parquet lake and Redshift both populated, internal fields absent.
- The outage-drill write-up with metric screenshots and recovery steps.
- Alarm definitions and the freshness budget.

Acceptance Criteria

- Curated Parquet is queryable within the stated lake freshness budget.
- Redshift dashboards see data within their budget, with staging-prefix lifecycle hygiene proven.
- Raw backup contains pre-transformation records and is access-separated.

- Transformation failure routes records to error prefixes and pages—never silently drops.

Stretch Goals

- Add an OpenSearch destination for operational search over the same stream.
- Replace direct conversion with an Iceberg-aware landing using a Flink job (Chapter 15) and compare trade-offs.
- Add per-record PII detection in the Lambda using a managed service and measure the latency/cost impact.
- Build the cost model: Firehose GB, Lambda GB-s, conversion surcharge, versus a self-run consumer fleet.

Stream Processing with Apache Flink on AWS

Learning Objectives

- Explain Managed Service for Apache Flink and its operational model versus self-managed Flink.
- Distinguish event time from processing time and apply watermarks to bound lateness.
- Implement tumbling, sliding, and session windows and stream-stream joins in Flink SQL.
- Manage keyed state: the RocksDB state backend, state TTL, checkpoints versus savepoints.
- Deliver end-to-end exactly-once with transactional or idempotent sinks—and know when at-least-once plus dedup is the wiser design.
- Troubleshoot inaccurate aggregations caused by late-arriving mobile data.

Cross-Cloud Equivalence

Stateful stream processing is the deepest discipline in this book, and the concepts—event time, watermarks, windows, state, checkpoints—are identical everywhere; only the managed packaging differs.

Capability	AWS	Azure	GCP
Managed Flink	Managed Service for Apache Flink	HDInsight/AKS-hosted Flink	Self-managed on GKE / Dataflow (Beam model)
Native stream SQL	Flink SQL	Stream Analytics query language	Dataflow SQL / Beam
Event-time windows	Flink SQL TUM-BLE/HOP/SESSION	ASA windowing	Beam windows
State backend	RocksDB (managed)	Per platform	Per runner
Exactly-once sink	Two-phase commit connectors	Idempotent sinks	Exactly-once in Dataflow
Savepoint story	App snapshots + savepoints	Platform-specific	Job drain/update
Kinesis connector	Native	Via Kafka bridge	Via Kafka/Pub/Sub IO

Fabric Real-Time Intelligence handles many windowed-analytics cases with KQL, trading Flink’s control for simplicity. **Snowflake** approaches streaming through Snowpipe Streaming plus dynamic tables. **MongoDB** stream processing (Atlas) applies the same windowed

ideas to change streams. Learn Flink’s model once; everywhere else is a dialect.

15.1 Week 15 Roadmap and Mental Model

Weeks 13–14 moved events and delivered them. Week 15 *computes* over them: continuously, correctly, and at scale. Stream processing’s hard questions are all about time and state. *When* did this event happen versus when did we see it? *How long* do we wait for stragglers? *What* must we remember between events, and for how long? And when the job crashes mid-computation, *what exactly* is recovered?

Apache Flink is the industry’s reference implementation of the answers [81, 1]. Amazon Managed Service for Apache Flink runs Flink applications for you—provisioning, scaling, checkpoint storage, and application snapshots—so the week’s attention goes to the model, not the cluster [17]. Thursday builds a windowed aggregation in Flink SQL; Friday confronts the scenario that separates people who have run streams from people who have read about them: a dashboard that is wrong because phones went offline.

Definition

Stream processing is continuous computation over unbounded event sequences, maintaining state across events and producing results with explicit semantics for time (event time versus processing time), lateness (watermarks and allowed lateness), and failure (checkpointed, exactly-once state).

Key Terms

Event time: when the event happened (its embedded timestamp). **Processing time:** when the engine sees it. **Watermark:** a heuristic assertion that no event older than time t will arrive; drives window firing and state cleanup. **Window:** tumbling (fixed, disjoint), sliding (fixed, overlapping), or session (gap-closed). **Keyed state:** per-key memory the engine checkpoints. **Checkpoint:** automatic, periodic, for failure recovery. **Savepoint:** manual, consistent, for upgrades and redeploys. **Two-phase commit:** transactional sink protocol for end-to-end exactly-once.

15.2 Monday: Managed Service for Apache Flink

15.2.1 What the Service Manages

Managed Service for Apache Flink runs your Flink application (Java/Scala via the DataStream API, Python via PyFlink, or SQL via Studio notebooks and the SQL gateway pattern) on managed compute, with automatic checkpointing to service-managed storage, snapshots on demand, and autoscaling. You bring the application, the connector configuration, and the parallelism; the service brings the runtime, the durability of state, and the restart machinery [17].

The billing unit is the KPU (1 vCPU, 4 GB memory, 50 GB storage) per hour, including one for orchestration. Autoscaling adjusts KPUs to load—which makes *backpressure* the metric to watch: an under-provisioned application falls behind, iterator age on the source stream climbs (Chapter 13’s alarm), and the retention window becomes a deadline.

Tip

Treat snapshot discipline as part of deployment, not an afterthought: stop the application with a snapshot/savepoint, never with a plain cancel, and configure the next version to restore from it. Checkpoint state cannot survive code changes that alter the operator graph—a missed savepoint before redeploy means reprocessing or lost state.

15.3 Tuesday: Event Time, Processing Time, and Watermarks

15.3.1 Two Clocks

Processing-time windows are trivially correct—the engine groups by its own clock—and trivially wrong for anything involving mobile devices, retries, or cross-region buffering, because arrival order is not event order. Event-time windows group by the timestamp inside the event, which is the time the business cares about. The cost of event time is the question processing time never asks: *how long do we wait?*

15.3.2 Watermarks: The Lateness Contract

A watermark is the engine’s running estimate of event-time completeness: “no event with timestamp earlier than W will arrive.” Windows fire when the watermark passes their end; state for a window is dropped once the watermark passes its end plus allowed lateness. Declaring a watermark of `event_ts - INTERVAL '10' SECOND` says: we expect events up to ten seconds late, and we accept the risk of anything later being dropped (or routed to a side output). Choosing the bound is a business decision wearing a technical costume—ten seconds of dashboard latency versus the accuracy of late data [81, 1].

Common Pitfall

Watermarks declared but never advancing. In Kinesis sources, a watermark is per-shard; one idle shard (no traffic) can hold the global watermark back and stall every window in the job. Use idleness detection on the source and verify watermark metrics, not just throughput, when windows stop firing.

15.4 Wednesday: Windows, Joins, and State Management

15.4.1 Windowing Strategies

- **Tumbling:** fixed-size, disjoint—“clicks per 5 minutes.” Each event belongs to exactly one window; state is bounded.
- **Sliding:** fixed-size, overlapping—“cross-city card use within any 10-minute span, evaluated every minute.” Events belong to several windows; state multiplies by slide ratio.
- **Session:** closed by inactivity gaps—“a user’s session of activity.” Windows vary in length; state depends on the gap timeout.

15.4.2 Stream–Stream Joins

Joining two unbounded streams requires bounding *how long* an event waits for its partner. An interval join states it explicitly: clicks match impressions seen up to 15 minutes prior, and the watermarks on both inputs let Flink discard join state older than the interval. Without the

bound, join state grows forever—which is the general law of streaming state: **unbounded state eventually kills every streaming job**, by checkpoint size, by restore time, or by heap.

```

1 -- Watermarks declared in DDL on both input streams
2 CREATE TABLE impressions (
3   ad_id STRING, imp_ts TIMESTAMP(3),
4   WATERMARK FOR imp_ts AS imp_ts - INTERVAL '10' SECOND
5 ) WITH ('connector' = 'kinesis', 'stream' = 'imps' /*, ... */);
6
7 CREATE TABLE clicks (
8   user_id STRING, ad_id STRING, click_ts TIMESTAMP(3),
9   WATERMARK FOR click_ts AS click_ts - INTERVAL '5' SECOND
10 ) WITH ('connector' = 'kinesis', 'stream' = 'clicks' /*, ... */);
11
12 -- Interval join: attribute clicks to impressions seen up to 15 min prior.
13 -- Watermarks let Flink drop state older than the join interval.
14 SELECT i.ad_id, c.user_id, c.click_ts
15 FROM impressions i
16 JOIN clicks c ON i.ad_id = c.ad_id
17 WHERE c.click_ts BETWEEN i.imp_ts AND i.imp_ts + INTERVAL '15' MINUTE;
```

Listing 15.1: Watermarks declared in DDL and an interval join bounding state retention.

15.4.3 State Backends, TTL, Checkpoints, and Savepoints

Keyed state lives in the configured state backend; at scale that is RocksDB—state on local disk with spill, checkpointed incrementally to durable storage. **State TTL** expires keys idle beyond a threshold, the finishing touch that turns “bounded in theory” into “bounded in production.” **Checkpoints** are the automatic, periodic consistency barriers that make failure recovery exactly-once *within* the job: on restart, state rolls back to the last completed checkpoint and sources replay from the corresponding offsets. **Savepoints** are the same mechanism triggered manually with a stable identifier—the tool for upgrades, because a savepoint is a contract with your future code.

15.4.4 End-to-End Exactly-Once

Internal exactly-once is table stakes; *end-to-end* exactly-once requires the sink to participate: either a transactional sink that commits only on checkpoint completion (two-phase commit to Kafka, or Flink’s S3 file sink committing files on checkpoint), or an idempotent sink (upsert by event ID) that makes duplicate writes harmless. When neither is available, at-least-once delivery plus downstream dedup (Chapter 13’s pattern) is the honest architecture—often the right choice anyway when the sink cannot transact.

15.5 Thursday Hands-on Project: A Flink SQL Tumbling-Window Counter

Write a Flink SQL application that reads clicks from a Kinesis stream, counts them in 5-minute tumbling windows, and writes results to a second stream.

Lab Objectives

- Create source and sink streams and declare event-time tables over them.
- Implement the tumbling-window count with a declared watermark.
- Observe window firing behavior against the watermark, not the wall clock.
- Demonstrate late-event handling within and beyond the watermark bound.

Procedure

Using a Studio notebook or a SQL application in Managed Service for Apache Flink:

```

1 CREATE TABLE clicks (
2   user_id    STRING,
3   page       STRING,
4   click_ts   TIMESTAMP(3),
5   WATERMARK FOR click_ts AS click_ts - INTERVAL '10' SECOND
6 ) WITH (
7   'connector' = 'kinesis',
8   'stream'    = 'user-clicks',
9   'aws.region' = 'us-east-1',
10  'format'     = 'json',
11  'scan.init.pos' = 'LATEST'
12 );
13
14 CREATE TABLE click_counts (
15   window_start TIMESTAMP(3),
16   window_end   TIMESTAMP(3),
17   page         STRING,
18   clicks       BIGINT
19 ) WITH (
20   'connector' = 'kinesis',
21   'stream'    = 'click-counts',
22   'aws.region' = 'us-east-1',
23   'format'     = 'json'
24 );
25
26 INSERT INTO click_counts
27 SELECT window_start, window_end, page, COUNT(*) AS clicks
28 FROM TABLE(
29   TUMBLE(TABLE clicks, DESCRIPTOR(click_ts), INTERVAL '5' MINUTES))
30 GROUP BY window_start, window_end, page;

```

Listing 15.2: Tumbling-window click counter in Flink SQL.

Drive it with an adapted Week 13 producer emitting click events with controllable timestamps, including deliberately late ones.

Verification

- On-time events appear in the correct 5-minute window; the window fires when the watermark passes its end, measurably after wall-clock end.
- An event 5 seconds late still lands in its correct window; an event 30 seconds late does not—the watermark contract made observable.
- Application metrics show checkpoint completion and growing-then-cleaned window state.
- A restart mid-window resumes without double-counting: internal exactly-once demonstrated.

15.6 Friday Use Case: The Dashboard That Lied About Late Mobile Data

A gaming company’s daily-engagement dashboard under-reports by up to 8% every morning and “catches up” over the day. Investigation: mobile clients buffer events while offline and flush them hours later. The current job uses processing-time tumbling windows. Walk the failure and the fix.

15.6.1 The Failure Chain

Processing-time windows file a flushed event in the window of *arrival*, not occurrence. An offline phone’s afternoon activity arrives at midnight and counts as midnight activity. Worse, the downstream daily aggregation reads each window once, so late data changes nothing already reported. The dashboard is not wrong because of a bug; it is wrong because the time semantics of the job do not match the question.

15.6.2 The Fix

- **Event time.** Rebuild windows on the client-side event timestamp with a watermark reflecting realistic delivery (say, `- INTERVAL '2' MINUTE` for the online majority).
- **Allowed lateness.** Permit windows to accept late events for a bounded period (e.g., 24 hours), re-firing with corrected counts—downstream must therefore handle *updates*, which pushes the sink toward upsert-by-window-key rather than append.
- **Side output.** Events later than the allowed lateness route to a side output (S3 prefix), where a daily batch correction reconciles them into the warehouse—the streaming job stays honest about its bound, and the batch layer owns the long tail.
- **Metric honesty.** Publish a “lateness distribution” metric so the business sees how much of yesterday arrived late, instead of silently believing the first number.

15.6.3 Exactly-Once, End to End

The corrected job writes window results keyed by (`window_start`, `window_end`, `page`). With an upsert sink, re-fires are safe and exactly-once effects are achieved idempotently; with a transactional sink (two-phase commit to Kafka or the S3 file sink), results appear only on checkpoint completion. The team chose idempotent upserts: their sink (Redshift via a merge step) could not transact with Flink, and dedup-by-key made the pipeline correct *and* simpler.

Common Pitfall

Refiring windows into an append-only sink. The moment allowed lateness lets a window emit twice, an append-only downstream has duplicates forever. Late-correcting streams require sinks that can update—upserts, transactions, or a reconciling batch layer. Choose the sink semantics before enabling refires.

15.7 Interview Q&A

15.7.1 Concepts and Trade-offs

1. **Q: Why does unbounded keyed state eventually kill a streaming job?**
A: State grows until checkpoints are too large to complete, restores take minutes to hours, and RocksDB spills beyond local disk. Every stateful operator needs a bound: window end, join interval, or TTL.
2. **Q: Checkpoint versus savepoint: which do you trigger before a redeployment, and why?**
A: A savepoint. Checkpoints are for automatic failure recovery and may be discarded or incompatible with a changed operator graph; a savepoint is a manual, stable, portable snapshot your new code restores from.

3. **Q: In a stream–stream interval join, what bounds how long join state is retained?**

A: The interval predicate plus the watermarks on both inputs: once the watermark passes the interval’s reach, the state can never match again and is dropped.

4. **Q: Which two components combine for end-to-end exactly-once delivery to Kafka or S3?**

A: Checkpointing (internal state consistency) plus a transactional or two-phase-commit sink that only makes results visible on checkpoint completion—or an idempotent sink achieving the same effect by key.

5. **Q: When is at-least-once plus downstream dedup preferred over exactly-once?**

A: When the sink cannot participate in transactions, when throughput outweighs strictness, or when a dedup key already exists naturally. It is simpler, often faster, and honest about its semantics.

6. **Q: What does a watermark actually assert?**

A: “No event older than this timestamp will arrive”—a heuristic contract that drives window firing and state cleanup, trading result latency against tolerance for late data.

15.8 Further Reading

The Apache Flink documentation covers the DataStream API, Flink SQL, windows, watermarks, and state backends [81]; the Managed Service for Apache Flink guide covers deployment, snapshots, autoscaling, and connectors [17]. Akidau, Chernyak, and Lax’s *Streaming Systems* is the definitive treatment of the what/where/when/how model this chapter operationalizes [1].

Key Takeaways

- Stream processing is computation with explicit time and state semantics; the model matters more than the framework dialect.
- Event time answers business questions; processing time answers arrival questions; confusing them produces dashboards that lie politely.
- Watermarks are the lateness contract—window firing, join bounds, and state cleanup all follow from them.
- All state must be bounded: by window, by join interval, or by TTL. Unbounded state is a time bomb.
- Checkpoints recover from failures; savepoints survive redeployments. Snapshot discipline is deployment discipline.
- End-to-end exactly-once is a property of the whole path: checkpointed state plus transactional or idempotent sinks, with dedup as the honest fallback.

15.9 Exercises

1. **(Conceptual)** Explain why a processing-time window is “never late” and why that is precisely its flaw.
2. **(Conceptual)** Why does a sliding window multiply state compared with a tumbling window of the same size?

3. **(Hands-on)** Run the Thursday application; then set the watermark to `- INTERVAL '60' SECOND` and measure the change in window-fire latency.
4. **(Hands-on)** Add `ALLOWED_LATENESS` (via the DataStream API or SQL equivalent) and an upsert sink; demonstrate a corrected re-fire.
5. **(Hands-on)** Implement the interval join from listing 15.1 with a synthetic click/impression producer and verify attribution counts.
6. **(Design)** Design watermark and lateness policy for three event classes with different delivery characteristics (online clicks, offline mobile batches, server logs).
7. **(Design)** Specify the side-output and batch-reconciliation contract for events later than the allowed lateness.
8. **(Architecture)** The job must now survive a region evacuation. Which components (state, source offsets, sink commits) make cross-region continuity hard, and what is the design?

15.10 Capstone Project: A Late-Data-Tolerant Sessionization Pipeline

Capstone Project

Mission. Build a production-shaped sessionization pipeline: event-time session windows over a clickstream, watermark and lateness policy for offline mobile clients, bounded state with TTL, savepoint-disciplined deployment, and an idempotent sink—with metrics that tell the business how late the data is.

Estimated time. 6–8 hours in a sandbox AWS account.

What you'll practice.

- Session windows keyed per user with realistic watermarks.
- Allowed lateness with an update-capable sink, plus a side output for the long tail.
- State TTL and RocksDB configuration, observed through checkpoint metrics.
- Savepoint-based redeployment without state loss.

Scenario

The gaming company from Friday wants user sessions—gap-closed at 30 minutes—computed continuously for engagement analytics, tolerant of phones going offline for hours, and safe to redeploy weekly. The sink is Redshift, so updates and merges are on you.

Requirements

Functional requirements.

- Event-time session windows (30-minute gap) over a Kinesis clickstream, keyed by user.
- A documented watermark policy and allowed lateness of 24 hours with an upsert sink into Redshift.
- A side output for events beyond allowed lateness and a batch reconciliation sketch.
- A lateness-distribution metric published to CloudWatch.

Non-functional requirements.

- State is provably bounded: TTL configured, checkpoint size stable over a 24-hour soak.

- A redeployment drill restores from a savepoint with zero loss and zero duplicates downstream.
- Iterator age on the source stays bounded under a $3\times$ load test.

Implementation Steps

1. Implement the sessionization job (DataStream API or SQL) with the watermark policy.
2. Implement the Redshift upsert sink keyed by (`user_id`, `session_start`).
3. Add allowed lateness, the side output, and the lateness metric.
4. Configure RocksDB, TTL, and checkpointing; run the soak test.
5. Perform the savepoint redeployment drill and document every step.
6. Write the operations runbook: watermark stalls, backpressure, savepoint procedure.

Deliverables

- The application code, connector configuration, and watermark/lateness rationale.
- Soak-test evidence: checkpoint size and iterator age over 24 hours.
- The redeployment drill record with savepoint ARNs and downstream dedup verification.
- The runbook and the business-facing lateness dashboard.

Acceptance Criteria

- Sessions are correct under injected offline-device lateness, verified against a batch reference computation.
- Window corrections update Redshift in place—no duplicate sessions after refires.
- State size plateaus during the soak; TTL expiry is visible in metrics.
- The redeployment drill completes with documented proof of no loss and no duplicates.

Stretch Goals

- Replace the upsert sink with a two-phase-commit path to Kafka and compare latency and complexity.
- Add a second stream and implement the attribution interval join in the same application.
- Simulate an idle shard and implement watermark idleness handling; document the before/after.
- Autoscale under a load ramp and characterize KPU scaling lag against iterator age.

Business Intelligence with QuickSight and Reverse ETL

Learning Objectives

- Explain Amazon QuickSight architecture and the SPICE in-memory engine's role and limits.
- Connect QuickSight to Redshift, Athena, and S3 with appropriate refresh and direct-query trade-offs.
- Enforce row-level security on shared datasets and apply ML Insights for anomaly detection.
- Implement reverse ETL: Redshift UNLOAD to S3, Lambda push to a CRM/marketing API, with identity resolution upstream.
- Reason about sync-frequency trade-offs: freshness versus API rate limits and cost.
- Design the BI access layer for 500 analysts with region-scoped visibility, and assemble the Milestone 4 real-time fraud capstone.

Cross-Cloud Equivalence

The serving layer—dashboards for humans, operational pushes back to business systems—closes the analytics loop on every platform.

Capability	AWS	Azure	GCP
Managed BI	Amazon QuickSight	Power BI (Fabric)	Looker / Looker Studio
In-memory acceleration	SPICE	Power BI import/semantic models	BI Engine
Row-level security	QuickSight RLS rules	Power BI RLS	Looker access filters
ML-assisted insights	QuickSight ML Insights	Copilot / Smart visuals	Gemini in Looker
Warehouse export	Redshift UNLOAD	COPY INTO / Fabric export	BigQuery EXPORT DATA
Reverse-ETL glue	Lambda + API calls	Logic Apps / Functions	Cloud Run functions
Reader pricing	Per-reader session	Per-user / capacity	Per-user
Sharing unit	Dashboard / analysis	Workspace app / semantic model	Look / dashboard

Snowflake hosts dashboards in Snowsight and streams reverse ETL through external func-

tions; the ecosystem (Census, Hightouch) serves all platforms. **MongoDB Charts** covers operational BI directly on collections.

16.1 Week 16 Roadmap and Mental Model

Everything before this week moved data toward analytical truth; Week 16 moves it toward *decisions and actions*. That happens in two directions. Outward to humans: dashboards, reports, and governed self-service—Amazon QuickSight’s territory. And outward to systems: pushing curated segments and scores back into the operational tools where work happens—reverse ETL.

The engineering shift is from correctness to *governance at scale*. A dashboard seen by 500 analysts with different entitlements is an access-control system with charts attached. A reverse-ETL pipeline writing to a CRM is a production integration with rate limits, identity ambiguity, and business consequences. This week treats both with the same rigor as the ingestion side [21].

Definition

Reverse ETL is the practice of syncing curated warehouse data—segments, scores, aggregates—back into operational systems (CRMs, marketing platforms, support tools) so that analytics directly drives business processes.

Key Terms

SPICE: QuickSight’s in-memory calculation engine; datasets imported into SPICE serve fast, cheap dashboard queries. **Direct query**: QuickSight querying the source (Redshift/Athena) live instead of SPICE. **RLS**: row-level security via a rules dataset mapping users/groups to allowed dimension values. **ML Insights**: QuickSight’s built-in anomaly detection, forecasting, and auto-narratives. **UNLOAD**: Redshift’s parallel export command to S3. **Identity resolution**: matching records for the same person across systems before syncing.

16.2 Monday: QuickSight Architecture and SPICE

16.2.1 The Serving Layer, Managed

QuickSight is serverless BI: no servers, per-reader pricing, browser-based authoring, and embedding APIs for application dashboards. A *dataset* defines a source and its preparation; an *analysis* is the author-facing canvas; a *dashboard* is the published, shared, read-only product. Because it is serverless, the capacity question moves from “how many nodes” to “SPICE or direct query?”

16.2.2 SPICE: The Acceleration Decision

SPICE (Super-fast, Parallel, In-memory Calculation Engine) imports dataset snapshots into managed memory. Dashboards against SPICE respond in sub-seconds without touching the source, which is both a performance and a cost story: a thousand dashboard loads do not become a thousand Redshift queries. The limits are the dataset capacity per account and the *refresh* cadence—SPICE data is only as fresh as its last refresh, so freshness-hungry dashboards either schedule tight refreshes (which do hit the source) or use direct query and pay the source for every interaction [21].

Criterion	SPICE import	Direct query
Freshness	As of last refresh	Live
Dashboard latency	Sub-second, independent	load- Source-dependent
Load on source	Refresh schedule only	Every interaction
Capacity limit	SPICE allocation per account	Source concurrency
Best for	Shared executive/operational dashboards	Live operational monitoring

Table 16.1: SPICE versus direct query.

Common Pitfall

Defaulting everything to direct query. Five hundred analysts on live Redshift queries re-creates the Chapter 6 concurrency problem at the BI layer. Default to SPICE for shared dashboards; reserve direct query for the few views whose freshness requirement is measured in minutes.

16.3 Tuesday: Connecting QuickSight to Redshift, Athena, and S3

QuickSight connects natively to the week’s familiar sources. **Redshift** connections support both SPICE and direct query and can target Serverless workgroups. **Athena** connections query the Glue catalog directly—the governed lake from Chapters 7–8 becomes a dashboard source with no data movement, and Lake Formation column masks apply through the Athena path. **S3** manifests allow file-set datasets for simple cases. Cross-account patterns matter here: dashboards in a central BI account can query Redshift datashares (Chapter 8) and catalogs shared via RAM, keeping data in domain accounts while the BI layer centralizes.

Tip

Give QuickSight its own Redshift user or IAM-identity-mapped role and a dedicated WLM service class. Then the Chapter 6 telemetry answers “what did the BI layer do to the warehouse at 9:05 a.m.?” without archaeology.

16.4 Wednesday: Row-Level Security, ML Insights, and Reverse ETL

16.4.1 Row-Level Security on Shared Datasets

QuickSight RLS attaches a *rules dataset* to a shared dataset: rows map users or groups to allowed values of a column (`region = 'EU'` for the EU managers group). Every visual, filter, and export from that dataset applies the restriction. The alternative—duplicating datasets per region—is a maintenance failure waiting for its second region. RLS keeps one dataset, one definition of the metrics, and per-viewer data [21].

16.4.2 ML Insights

QuickSight’s built-in ML features—anomaly detection on time series, forecasting, and auto-generated narratives—run on SPICE datasets without a modeling pipeline. For data engineers

they are the fastest way to put a drift tripwire on a business metric: configure anomaly detection on daily revenue, and the first day the pipeline breaks, the *business* metric alarms too, complementing the infrastructure alarms from earlier chapters.

16.4.3 Reverse ETL: Closing the Loop

Reverse ETL pushes warehouse truth back to operational tools. The AWS-native shape: **UNLOAD** a curated segment query from Redshift to S3 [75], an S3 event triggers a **Lambda** that batches records to the CRM/marketing API with rate-limit-aware throttling and dead-letter capture, and a run-log table records what was synced when.

Two disciplines separate a pipeline from a liability. **Identity resolution** comes first: the warehouse row must map to the right person in the CRM—matching on a stable identity key established upstream, not on email strings that differ in case and era. And **sync frequency** is a budget: hourly syncs against a rate-limited API can cost more in calls than the value of the freshness; state the freshness requirement, then engineer down to it.

```

1 UNLOAD ($$
2   SELECT customer_id, churn_score, segment
3   FROM analytics.customer_scores
4   WHERE score_date = CURRENT_DATE AND churn_score > 0.7
5 $$)
6 TO 's3://acme-analytics-raw-123456789012/reverse-etl/churn/'
7 IAM_ROLE 'arn:aws:iam::123456789012:role/RedshiftS3ReadRole'
8 FORMAT AS CSV
9 HEADER
10 PARALLEL OFF;
```

Listing 16.1: UNLOAD a churn-risk segment for reverse ETL.

Common Pitfall

Syncing duplicates to systems of engagement. A warehouse join bug that double-counts is embarrassing; the same bug reverse-ETL'd emails a customer twice or double-books a discount. Treat reverse ETL as a production integration: idempotency keys on the push, dry-run modes, and reconciliation counts between UNLOAD output and API acknowledgments.

16.5 Thursday Hands-on Project: A Monitored Sales Dashboard

Connect QuickSight to the Redshift warehouse from Part 2, build a sales monitoring dashboard, apply a filter, and configure an ML anomaly-detection alert on the revenue trend.

Lab Objectives

- Create a QuickSight dataset from Redshift over **fact_sales** and its dimensions.
- Build a dashboard with a KPI, a trend line, and a working region filter.
- Configure anomaly detection on daily revenue with an alert to email/SNS.
- Compare SPICE import and direct query on the same dataset.

Procedure

1. In QuickSight, create a dataset from the Redshift Serverless workgroup: a join of `fact_sales` to `dim_date`, `dim_product`, and `dim_store`.
2. Choose SPICE import; schedule a daily refresh after the ETL window.
3. Build the analysis: KPI of month-to-date revenue, line chart of daily revenue, bar of revenue by category, and a `region` filter control affecting all visuals.
4. Add an anomaly-detection insight on daily revenue and configure a threshold alert.
5. Duplicate the dataset as direct query; compare dashboard interaction latency and observe the Redshift query log entries per interaction.
6. Publish as a dashboard shared with a test reader group.

Verification

- The region filter restricts every visual; exported data respects it.
- The anomaly alert fires when a synthetic revenue dip is injected into the fact table and the refresh runs.
- The SPICE dashboard is materially faster than direct query under ten concurrent viewers; the direct-query variant shows per-interaction queries in `SVL_QUERY_SUMMARY`.
- Refresh timestamps on the dashboard match the scheduled cadence.

16.6 Friday Use Case: Governed BI for 500 Analysts

An enterprise rolls out QuickSight to 500 analysts across five regions. Every analyst queries centralized warehouse data; regional sales managers must see only their region's rows. Data engineering owns the datasets; analysts own their analyses. Design the access layer.

16.6.1 Design Decisions

One governed dataset per domain. Curated, documented datasets (sales, inventory, marketing) are built by data engineering on conformed warehouse marts; analysts build analyses on top but cannot redefine the metrics layer. This is the semantic-layer bargain: consistency in exchange for a backlog.

RLS from the identity source. The rules dataset maps IAM-Identity-Center groups (`sales-eu`, `sales-na`, ...) to region values. Group membership is managed in the identity provider, so access reviews and joiner/mover/leaver processes inherit the existing HR workflow—no BI-side access list to drift.

SPICE with staggered refreshes. All shared datasets use SPICE, refreshed after the nightly ETL and again midday for the operational domain. Direct query is an exception requiring a freshness justification, protecting the warehouse from the 9:05 a.m. refresh storm.

Cost visibility. Per-reader session pricing makes adoption cheap to start; the platform team dashboards monthly reader-session counts per department so finance sees BI cost as a function of actual usage.

Requirement	Mechanism	Failure it prevents
Region-scoped visibility	RLS rules dataset by IdP group	Per-region dataset forks
Consistent metric definitions	Governed datasets only	Competing revenue numbers
Warehouse protection	SPICE default, staggered refresh	9:05 a.m. query storms
Access lifecycle	Group membership via IdP	Orphaned access after transfers
Adoption cost control	Per-reader session reporting	Silent license sprawl

Table 16.2: The 500-analyst BI access layer.

Common Pitfall

RLS rules managed by hand in a spreadsheet. The rules dataset *is* the authorization system for the BI layer. Derive it from the identity provider’s groups with a small pipeline, review its diffs like code, and alert when a user appears in no rule (fail-closed: they see nothing) versus in a wildcard rule (fail-open: they see everything).

16.7 Interview Q&A

16.7.1 Concepts and Trade-offs

- Q: What does SPICE accelerate, and what is its capacity limit?**
A: Dataset queries for dashboards—imported in-memory snapshots answer in sub-seconds without touching the source. Capacity is bounded by the SPICE allocation per account, and freshness by the refresh schedule.
- Q: How does QuickSight row-level security filter a shared dataset?**
A: A rules dataset maps users/groups to allowed column values; every visual and export applies the restriction at query time, so one dataset serves many audiences.
- Q: What is reverse ETL, and which operational systems does it feed?**
A: Syncing curated warehouse results—segments, scores—back into operational tools: CRMs, marketing platforms, support systems. Analytics becomes action inside the systems where work happens.
- Q: Why must identity resolution happen before syncing audiences to a CRM?**
A: The warehouse key must map to the correct person in the target system; syncing on uncontrolled identifiers creates duplicate or mis-targeted outreach with real business consequences.
- Q: Hourly versus daily sync: which trade-offs decide the frequency?**
A: API rate limits and call cost against the business value of freshness. State the freshness requirement first; then engineer to the loosest cadence that meets it.
- Q: When is direct query justified over SPICE?**
A: When freshness requirements are measured in minutes and the source can bear per-interaction load—typically operational monitors with small audiences, not company-wide dashboards.

16.8 Further Reading

The QuickSight User Guide covers SPICE, data sources, RLS rules datasets, ML Insights, and embedding [21]. The UNLOAD reference covers parallel export options for reverse ETL [75]; the Lambda guide covers the throttled-push pattern to external APIs [52]. The Well-Architected Analytics Lens frames the governed-self-service balance [3].

Key Takeaways

- QuickSight makes BI serverless; the real capacity decision is SPICE versus direct query per dataset.
- Governed datasets plus RLS rules let one semantic layer serve every audience safely.
- ML Insights puts anomaly tripwires on business metrics—complementing, not replacing, infrastructure alarms.
- Reverse ETL turns the warehouse into an action system; identity resolution and sync budgets make it safe.
- Treat CRM pushes as production integrations: idempotency, dry runs, reconciliation.
- The BI layer’s access controls derive from the identity provider, not from dashboard-side lists.

16.9 Exercises

1. **(Conceptual)** Explain why SPICE both reduces warehouse load and creates a freshness contract with the business.
2. **(Conceptual)** Why is fail-closed the correct default for RLS rule misses, and how do you detect them?
3. **(Hands-on)** Complete the Thursday project; add an RLS rules dataset restricting a test user to one region and verify exports.
4. **(Hands-on)** Build the reverse-ETL loop: listing 16.1 plus a Lambda pushing to a mock CRM endpoint, with a reconciliation count.
5. **(Hands-on)** Inject an anomaly into daily revenue and capture the ML Insights alert end to end.
6. **(Design)** Design the identity-resolution strategy for customers who exist in web, app, and store systems with different keys.
7. **(Design)** Budget a reverse-ETL sync: 200,000 members, API limit 100 calls/s, 100 records/call, business wants 30-minute freshness. Is it feasible? Show the math.
8. **(Architecture)** Design the BI layer for a holding company where each subsidiary’s data must remain in its own account.

16.10 Milestone 4 Capstone: Real-Time Fraud Detection

Capstone Project

Mission. Build an end-to-end real-time fraud-detection platform: simulated card swipes stream into Kinesis, a Managed Flink application flags any card used in two cities within ten minutes (sliding windows, event time), alerts flow through Firehose into Redshift, and a QuickSight dashboard displays live fraud alerts.

Estimated time. 8–10 hours in a sandbox AWS account.

What you'll practice.

- Producer design with per-card partition keys (Chapter 13).
- Event-time sliding-window detection in Flink SQL (Chapter 15).
- Governed delivery with Firehose, raw backup, and idempotent writes (Chapter 14).
- A live QuickSight dashboard over the alert stream (this week).

Scenario

A card network wants same-minute detection of an impossible-travel pattern: one physical card used in two different cities within ten minutes. Swipes arrive as a stream; flagged alerts must reach a fraud-operations dashboard within a minute, with raw events durably archived. The card PAN must never be persisted in plain text anywhere in the analytical estate.

Requirements

Functional requirements.

- A Python producer simulating swipes—including scripted impossible-travel scenarios—with `card_id` as the partition key and PANs tokenized at the edge.
- A Flink application on event time using a sliding window to detect cross-city use within ten minutes.
- Firehose delivery of alerts to Redshift with raw backup to S3; deduplication on `txn_id`.
- A QuickSight dashboard over the alerts table, refreshing at least every minute.

Non-functional requirements.

- At least three automated tests: window-logic unit test, late-event handling test, DLQ routing test with a failing case.
- Alarms on `IteratorAgeMilliseconds` and Flink restarts, wired to SNS.
- A monthly cost estimate (shard-hours versus MSK, Flink KPIs, Firehose volume) with two documented optimizations.
- Security review: KMS-encrypted streams, least-privilege consumers, PAN never persisted in plain text.

Reference Architecture

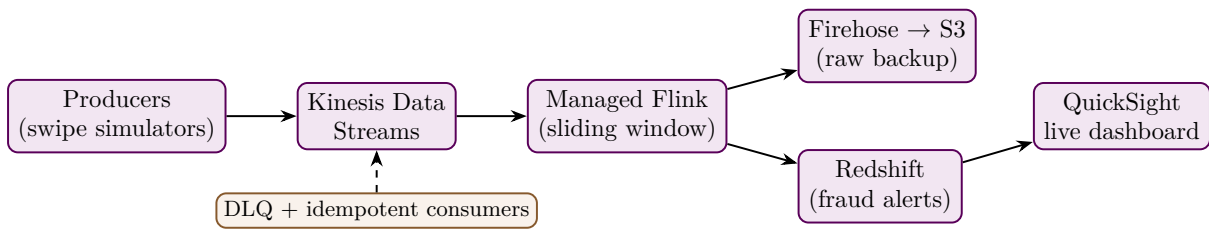


Figure 16.1: Milestone 4 reference architecture: real-time fraud detection from swipe simulation to live dashboard.

Implementation Steps

1. Create the stream (on-demand capacity) and the tokenizing producer with scripted fraud scenarios.
2. Build the Flink SQL application: watermarks on `event_ts`, the sliding-window detection query below, alert output to a results stream.
3. Connect Firehose from the results stream to Redshift with S3 raw backup; dedupe alerts on `txn_id`.
4. Build the QuickSight dashboard on the alerts table with a one-minute refresh and an anomaly insight on alert rate.
5. Write the three tests, wire the two alarms, run the security review, and build the cost model.

The detection query is the heart of the capstone:

```

1 SELECT card_id, window_start, window_end,
2        COUNT(DISTINCT city) AS city_count
3 FROM TABLE(HOP(TABLE swipes, DESCRIPTOR(event_ts),
4              INTERVAL '10' MINUTES, INTERVAL '1' MINUTES))
5 GROUP BY card_id, window_start, window_end
6 HAVING COUNT(DISTINCT city) > 1;

```

Listing 16.2: Two-cities-in-ten-minutes detection with a hopping window.

Deliverables

- Producer, Flink application, Firehose configuration, and dashboard definition, all in version control.
- The three automated tests with the failing-case demonstration.
- Alarm evidence and a screen recording or screenshots of a scripted fraud event reaching the dashboard.
- The cost model with two optimizations and the security review (tokenization verified).

Acceptance Criteria

- A scripted cross-city pair flags within the freshness budget; a scripted same-city pair does not.

- Late events within the watermark bound attribute correctly; events beyond it route to the side output.
- No plain-text PAN exists in any S3 prefix, stream record, or Redshift table—verified by scan.
- Alarms fire on injected consumer lag and on a forced Flink restart.
- Alert rows in Redshift show no duplicates under producer retries.

Stretch Goals

- Add a second detection rule (amount spike versus per-card baseline) as a keyed-process function with per-card state.
- Replatform detection on MSK with Kafka consumers and compare operational burden and cost.
- Add QuickSight RLS so regional fraud teams see only their region's alerts.
- Build the batch reconciliation job that refines streaming alerts with next-day settled data.

Part V

Machine Learning and MLOps

In-Database Machine Learning with Redshift ML

Learning Objectives

- Explain Redshift ML architecture: SQL in front, SageMaker training behind, model functions in the warehouse.
- Create models for regression, binary and multi-class classification, and K-Means clustering with `CREATE MODEL`.
- Select problem types and objectives, and justify metric choice (e.g., F1 over accuracy) for imbalanced data.
- Score live data with model functions directly in SQL and reason about data movement avoided.
- Judge when a problem has outgrown Redshift ML and should graduate to full SageMaker.
- Architect SQL-only daily inventory forecasting for 1,000 retail stores.

Cross-Cloud Equivalence

Bringing the model to the data—rather than shipping data to a modeling platform—is a pattern every warehouse now offers.

Capability	AWS	Azure	GCP
SQL-based ML	Redshift ML (<code>CREATE MODEL</code>)	Fabric SynapseML / T-SQL <code>PREDICT</code>	BigQuery ML (<code>CREATE MODEL</code>)
AutoML behind the scenes	SageMaker Autopilot	Azure ML AutoML	Vertex AI AutoML
In-warehouse scoring	SQL function per model	T-SQL <code>PREDICT</code>	ML. <code>PREDICT</code>
Custom code path	BYO model via SageMaker	Synapse Spark MLlib	BigQuery remote models / Vertex
Clustering	K-Means problem type	MLlib KMeans	ML. <code>KMEANS</code>
Training data leaves warehouse?	Yes (to SageMaker, in-account)	Varies	No (BigQuery-native)
Cost driver	SageMaker training + S3 staging	Capacity units	Bytes processed + training

Snowflake ML offers `CREATE MODEL` and Snowpark ML with warehouse-side scoring—the closest analog. **MongoDB** has no in-database ML; Atlas Vector Search addresses the adjacent similarity problem instead.

17.1 Week 17 Roadmap and Mental Model

The next four weeks build machine learning into the data platform. Week 17 starts where the leverage is highest for a data engineer: *inside the warehouse you already run*. Redshift ML lets you train a model with a SQL statement and call it like a function in a query—no notebook, no endpoint, no feature pipeline [23].

The mental model: `CREATE MODEL` is a compiler directive, not magic. Redshift exports the training data to S3, delegates training to Amazon SageMaker (Autopilot for automatic mode, or a specified algorithm), and compiles the winning model into a scalar function available in SQL. Knowing that architecture explains everything else in the week: the IAM role, the S3 staging bucket, the billing (SageMaker training time plus Redshift inference), and the limits—you get a good model with zero infrastructure, not arbitrary control over the modeling process.

Definition

Redshift ML is a warehouse feature that trains machine-learning models via `CREATE MODEL` SQL statements—delegating training to SageMaker—and exposes the resulting model as a SQL function for in-database inference.

Key Terms

CREATE MODEL: the DDL statement defining training data, target, problem type, and objective. **FUNCTION clause**: names the SQL function that scores rows. **Problem type**: regression, binary/multiclass classification, or K-Means. **Objective**: the metric Autopilot optimizes (F1, AUC, MSE, accuracy). **BYO model**: importing an existing SageMaker model for in-warehouse scoring. **Training/export bucket**: the S3 staging area where training data and artifacts land.

17.2 Monday: Redshift ML Architecture

17.2.1 SQL in Front, SageMaker Behind

When `CREATE MODEL` runs, Redshift unloads the training query’s result to the designated S3 bucket, calls SageMaker Autopilot (in automatic mode) to explore candidates, selects the best model against your objective, and deploys it as a function invocable in SQL [23]. Inference happens inside the warehouse—no per-row API calls, no endpoint to manage—which is the entire value proposition: **data never leaves your account’s analytical plane for scoring, and the model travels to the data**.

The IAM role attached to `CREATE MODEL` is the security boundary: it needs S3 access to the staging bucket and SageMaker permissions, and it should be scoped to exactly those—the role becomes the delegation point auditors ask about first.

Note

Redshift ML pricing has two meters: SageMaker training (charged per training resource-hour) and the S3 staging of training data and artifacts. Inference itself runs in the warehouse and consumes cluster compute. For iterative experimentation, cap Autopilot’s candidates (`MAX_CANDIDATES`) and runtime (`MAX_RUNTIME`) in the model settings.

17.3 Tuesday: Creating Models—Regression, Classification, K-Means

17.3.1 The Three Problem Types

Regression predicts a continuous number (tomorrow’s units sold). **Binary and multiclass classification** predict a label (churn/not-churn; product category). **K-Means** clusters unlabeled rows into k groups (customer micro-segments)—unsupervised, so there is no target column. Automatic mode infers the problem type from the target column; production practice is to *declare* `PROBLEM_TYPE` and `OBJECTIVE` explicitly so the model’s contract is in the DDL and reviewed like any schema.

The training query is the feature-engineering surface. Everything conventional wisdom says about ML data applies: dedupe before training, exclude leaky columns (anything only knowable after the outcome), and treat the `SELECT` as a versioned artifact—the model is exactly as good as that query.

Common Pitfall

Target leakage through the training query. If `days_to_churn` or `winback_offer_sent` sneaks into the feature list, the model “learns” the answer and collapses in production. Review training queries for leakage with the same suspicion you review grants for over-permission.

17.4 Wednesday: Evaluation and Scoring in SQL

17.4.1 Evaluation Where the Data Lives

Model metadata and validation metrics are queryable (`SHOW MODEL`, system views), so evaluation happens beside the data. The metric choice deserves the same care as the modeling: for imbalanced classification—5% churn—*accuracy* is a trap (predicting “no churn” always scores 95%), while **F1** balances precision and recall on the minority class, and AUC summarizes ranking quality across thresholds. Setting `OBJECTIVE 'F1'` in the DDL is not a detail; it is the statement of what the business is optimizing.

17.4.2 Scoring Live Data

Once the model state is `READY`, the function scores any row set in SQL—including live, current tables:

```

1 CREATE MODEL customer_churn_model
2 FROM (
3     SELECT tenure_months, monthly_spend, support_tickets,
4           avg_session_min, is_churned
5     FROM analytics.customers
6 )
7 TARGET is_churned
8 FUNCTION fn_predict_churn
9 IAM_ROLE 'arn:aws:iam::123456789012:role/RedshiftMLRole'
10 SETTINGS (
11     S3_BUCKET 'ml-artifacts-bucket',
12     PROBLEM_TYPE 'binary_classification',
13     OBJECTIVE 'F1'
14 );
15
16 -- score live customers once the model state is READY

```

```

17 SELECT customer_id, churn_prob
18 FROM (
19     SELECT customer_id,
20            fn_predict_churn(tenure_months, monthly_spend,
21                           support_tickets, avg_session_min) AS churn_prob
22     FROM analytics.customers_current
23 )
24 WHERE churn_prob > 0.7;

```

Listing 17.1: Training a churn model and scoring live customers in SQL.

The pattern to notice: scoring is a *query*, so it composes with everything SQL already does—joins, filters, scheduled inserts into a risk-segment table that Chapter 16’s reverse ETL can push to the CRM. That composability is why in-database ML punches above its modeling sophistication.

17.5 Thursday Hands-on Project: Churn Prediction in Pure SQL

Train a logistic-regression-style churn classifier on sample customer data and predict churn on live data—without leaving SQL.

Lab Objectives

- Create the IAM role and staging bucket for Redshift ML.
- Build a training dataset with a leaky column deliberately excluded (and documented).
- Train with an explicit problem type and F1 objective; inspect validation metrics.
- Score the current customer base and persist a risk table for downstream use.

Procedure

1. Create `RedshiftMLRole` trusted by `redshift.amazonaws.com`, with S3 access to `ml-artifacts-bucket` and the SageMaker permissions Redshift ML requires; attach it to the workgroup/namespace.
2. Build `analytics.customers` with features and the `is_churned` label (synthetic is fine—five columns, 100k rows, roughly 10% positive).
3. Run listing 17.1’s `CREATE MODEL`; poll `SHOW MODEL customer_churn_model` until state is `READY`; record the validation F1.
4. Score `analytics.customers_current`; persist high-risk rows into `analytics.churn_risk_daily` with the score date—the table Chapter 16’s reverse ETL consumes.
5. Retrain on a second synthetic batch and compare F1 to feel for data sensitivity.

Verification

- `SHOW MODEL` reports `READY` with a validation F1 substantially above the majority-class baseline.
- The risk table’s score distribution is plausible (a minority above 0.7), and re-scoring is idempotent by score date.
- The training query is version-controlled with its leakage exclusions noted.
- Cost evidence: the SageMaker training line item for the run is captured and stated.

17.6 Friday Use Case: SQL-Only Inventory Forecasting for 1,000 Stores

A retailer needs daily unit-demand forecasts per store per category for 1,000 stores, feeding the replenishment system. The data platform team is SQL-strong and ML-tooling-light; leadership wants minimal data movement. Design it on Redshift ML.

17.6.1 Architecture Decisions

Problem shape. Daily store-category demand is a regression problem with strong seasonality. Features come from the warehouse itself: trailing 7/28-day sales means, day-of-week and holiday flags, promotion indicators, price—all SQL-computable from `fact_sales`. No feature leaves the warehouse; the training query is the feature pipeline.

Model topology. One global model with store and category features usually beats 1,000 per-store models in both accuracy and operability—one `CREATE MODEL`, one function, one monitoring surface. Where a category behaves wildly differently (fresh food versus hardware), a handful of per-segment models is the middle path.

Scoring and serving. A nightly scheduled query scores tomorrow’s demand for every store-category into `forecast_daily(store, category, date, forecast, model_version)`. The replenishment system reads that table; the warehouse remains the only integration point. Chapter 16’s reverse-ETL pattern applies if replenishment lives in a SaaS tool.

When to graduate. The trigger list for moving to full SageMaker (Chapters 18–20): you need custom architectures (deep sequence models), per-model hyperparameter control, explainability reports the business requires, or latency-critical real-time inference. Daily batch forecasting from warehouse features hits none of those—which is precisely why Redshift ML is the right scope here.

Common Pitfall

Using in-database ML to avoid building ML discipline. Redshift ML removes infrastructure, not rigor. You still need train/validation honesty, a champion-challenger re-training cadence, forecast-accuracy monitoring (MAPE by store), and a documented owner for the training query. The SQL is easy; the lifecycle is the job.

17.7 Interview Q&A

17.7.1 Concepts and Trade-offs

1. **Q: What part of training does Redshift ML delegate to SageMaker?**

A: Everything past the export: Redshift unloads the training query to S3, SageMaker Autopilot (or a specified algorithm) trains and selects the model, and the winner is compiled back into a SQL function.

2. **Q: Why is in-database ML attractive for SQL-only forecasting pipelines?**

A: Features, training, and scoring all live where the data already is. No extraction pipelines, no endpoints, no new operational surface—the training query doubles as the feature pipeline.

3. **Q: What does the FUNCTION clause of CREATE MODEL define?**

A: The name and signature of the SQL function that scores rows, mapping argument order to the training query’s feature columns.

4. **Q: Why choose F1 over accuracy for imbalanced churn data?**

A: With 5% positives, always-negative scores 95% accuracy while detecting nothing. F1 balances precision and recall on the minority class, aligning the objective with the business cost of missed churners.

5. **Q: When should you graduate from Redshift ML to full SageMaker?**

A: When you need custom architectures, fine hyperparameter control, formal explainability, or real-time low-latency inference—or when the team owns an ML platform practice that wants full lifecycle control.

6. **Q: What is target leakage and how do you catch it?**

A: A feature unknowable at prediction time contaminating training. Catch it by reviewing the training query column-by-column against the question “is this knowable before the outcome?”

17.8 Further Reading

The Redshift ML documentation covers `CREATE MODEL` settings, problem types, objectives, BYO models, and cost controls [23]. The SageMaker guide provides the delegated-training context [27]. Kimball and Ross discuss the warehouse as the natural home for scored, conformed analytical outputs [92].

Key Takeaways

- Redshift ML is SQL front-end, SageMaker engine, in-warehouse inference—model comes to the data.
- Declare `PROBLEM_TYPE` and `OBJECTIVE` explicitly; the DDL is the model’s contract.
- The training query is the feature pipeline: version it, review it for leakage, own it.
- Metric choice is business policy: F1 for imbalanced classification, never default accuracy.
- Scoring as SQL composes with scheduled tables and reverse ETL—the compounding advantage.
- Graduate to SageMaker for control and lifecycle needs, not for fashion; daily batch forecasting is Redshift ML’s home turf.

17.9 Exercises

1. **(Conceptual)** Explain the two billing meters of Redshift ML and how `MAX_CANDIDATES` interacts with them.
2. **(Conceptual)** Why does a global model with store features usually beat 1,000 per-store models?
3. **(Hands-on)** Run the Thursday project; then retrain with `OBJECTIVE 'Accuracy'` on the imbalanced data and compare confusion matrices.
4. **(Hands-on)** Add a leaky column to the training query, observe the inflated validation metric, and write the review note that catches it.
5. **(Hands-on)** Build `forecast_daily` for ten synthetic stores and schedule its nightly refresh.

6. **(Design)** Design the champion–challenger process: how does a retrained model earn its way into the scoring path?
7. **(Design)** Specify the monitoring for the forecasting system: which accuracy metric, which granularity, which alarm thresholds.
8. **(Architecture)** The retailer adds a requirement for per-request pricing inference under 100 ms. Redesign the serving layer and justify leaving Redshift ML for that path.

17.10 Capstone Project: A Warehouse-Native Demand Forecasting Pipeline

Capstone Project

Mission. Deliver the Friday scenario as a working system: feature engineering in SQL, a trained demand model, nightly batch forecasts into a served table, accuracy monitoring, and a documented retraining cadence—entirely within the warehouse plane.

Estimated time. 5–7 hours in a sandbox AWS account.

What you'll practice.

- Feature queries as versioned artifacts with leakage review.
- Model DDL with explicit problem type, objective, and cost caps.
- Nightly scoring into a contract-stable forecast table.
- Forecast-accuracy monitoring as a first-class deliverable.

Scenario

A 50-store pilot of the Friday scenario: daily forecasts per store-category for the top 20 categories, consumed by a replenishment prototype reading one table. Success means the pilot's forecast accuracy beats the incumbent naive (last-7-days-mean) baseline.

Requirements

Functional requirements.

- A versioned training query producing the feature set from `fact_sales` history, with a leakage review note.
- A trained regression model with capped Autopilot runtime, and its validation metrics recorded.
- A nightly scoring job writing `forecast_daily` with model version and score date.
- An accuracy report comparing model versus naive baseline by category.

Non-functional requirements.

- The IAM role is scoped to the staging bucket and required SageMaker actions only.
- Training cost per run is measured and recorded; reruns are idempotent by date.
- The forecast table's schema is a documented contract with the replenishment consumer.

Implementation Steps

1. Generate two years of synthetic store-category daily sales with seasonality and promotions.
2. Write and review the feature query; build the training view.

3. Train with capped settings; record validation metrics and cost.
4. Implement nightly scoring and the accuracy report against held-out days.
5. Wire the report to a weekly scheduled query; document the retraining trigger.
6. Write the consumer contract for `forecast_daily`.

Deliverables

- The feature query, model DDL, and scoring job in version control.
- Validation metrics, training-cost record, and the model-versus-baseline accuracy report.
- The consumer contract and the retraining runbook.
- IAM policy evidence for the scoped role.

Acceptance Criteria

- The model beats the naive baseline on held-out data by a stated, measured margin.
- Nightly scoring is idempotent and lands before the replenishment read window.
- The training query has a documented leakage review.
- Every claim in the accuracy report is reproducible from recorded queries.

Stretch Goals

- Add a per-segment model for the worst-forecast category and measure the lift.
- Feed forecast versus actuals into a QuickSight dashboard with an anomaly alert.
- Push stockout-risk segments to a mock replenishment API via the Chapter 16 reverse-ETL pattern.
- Rebuild the same model in SageMaker Autopilot directly (Chapter 18) and compare control, cost, and accuracy.

Amazon SageMaker Foundations for Data Engineers

Learning Objectives

- Navigate SageMaker's surfaces: Studio, notebook instances, and Data Wrangler for interactive data preparation.
- Choose between built-in algorithms, SageMaker Autopilot, and custom training containers.
- Select the right inference shape: real-time, serverless, asynchronous, or Batch Transform.
- Clean an S3 dataset in Data Wrangler, train XGBoost via Autopilot, and deploy to a serverless endpoint.
- Design secure training-data access from a governed S3 lake through Lake Formation.
- Architect a GPU training workflow for data scientists working on governed data.

Cross-Cloud Equivalence

Managed ML platforms share a shape: a studio, managed training, a model registry, and deployment targets. The AWS names map onto their peers directly.

Capability	AWS	Azure	GCP
ML studio	SageMaker Studio	Azure ML studio	Vertex AI Workbench
Interactive data prep	Data Wrangler	Fabric Data Wrangler / Power Query	BigQuery DataFrames / Colab
AutoML	SageMaker Autopilot	Azure ML AutoML	Vertex AI AutoML
Managed training	SageMaker Training Jobs	Azure ML jobs	Vertex AI Training
Real-time inference	Real-time endpoints	Managed online endpoints	Vertex AI endpoints
Batch inference	Batch Transform	Batch endpoints	Batch prediction
Custom code	BYO container (any framework)	Custom environments	Custom containers
GPU access	ml.g/ml.p instances	GPU compute clusters	GPU nodes/Vertex

Fabric data science runs in-notebook with SynapseML against OneLake. **Snowflake** ML covers training and serving inside the warehouse boundary. **MongoDB** feeds these platforms as a source; its vector search serves retrieval rather than training.

18.1 Week 18 Roadmap and Mental Model

Week 17 stayed inside the warehouse; Week 18 steps onto the full ML platform. SageMaker is not one service but a workbench: Studio for interactive development, Data Wrangler for visual data preparation, Training Jobs for managed compute, Autopilot for automated modeling, a registry for versions, and four shapes of inference infrastructure [27].

The data engineer’s perspective on SageMaker is distinctive: your concerns are *data access* (how does training read governed data lawfully?), *data movement* (what is copied where, in what format?), and *cost shape* (GPUs and always-on endpoints are where ML budgets go to die). This chapter builds the platform literacy that Week 19’s pipelines and Week 20’s AI services assume.

Definition

Amazon SageMaker is a managed machine-learning platform covering the model lifecycle: data preparation (Data Wrangler), experimentation (Studio notebooks), automated and custom training (Autopilot, Training Jobs, custom containers), and deployment (real-time, serverless, asynchronous, and batch inference).

Key Terms

Studio: the web-based IDE unifying SageMaker tools. **Data Wrangler**: visual data-prep tool that exports repeatable flows. **Autopilot**: automated model exploration producing candidate notebooks and a best model. **Built-in algorithm**: SageMaker-maintained implementation (XGBoost, BlazingText) run from a managed container. **Training job**: managed, ephemeral compute that reads training data, runs the container, and writes model artifacts to S3. **Endpoint**: managed inference infrastructure—real-time (persistent), serverless (scale-to-zero), or async; **Batch Transform**: offline scoring over a dataset.

18.2 Monday: Studio, Notebooks, and Data Wrangler

18.2.1 The Working Surfaces

SageMaker Studio is the hub: hosted notebooks, the Data Wrangler canvas, Autopilot launches, pipeline visualization, and endpoint management—all under one IAM-governed domain. The data engineer’s first question is never “which notebook instance” but “what data can this domain’s execution role reach?”—because the answer determines whether governance (Chapter 8) survives contact with the data science team.

18.2.2 Data Wrangler: Visual Prep That Exports

Data Wrangler is the interactive cleaning surface: connect S3, Athena, or Redshift; sample the data; apply from 300+ built-in transforms; see distributions and a quick model’s feature importances as you work [27]. Its professional significance is the *export*: an interactive session compiles to a repeatable artifact—a Processing Job script, a SageMaker Pipelines step, or a notebook—so the cleaning you did by hand becomes code that reruns. Interactive exploration that cannot be replayed is technical debt; Wrangler’s export is how exploration graduates.

Common Pitfall

Hand-editing data in a notebook and calling the result a pipeline. If cleaning exists only as executed cells in someone’s session, it is unreproducible by definition. Export

the Wrangler flow or write the transforms in a job; the notebook is the lab bench, not the factory.

18.3 Tuesday: Built-In Algorithms, Autopilot, and Custom Containers

18.3.1 Three Ladders of Control

Built-in algorithms—XGBoost, BlazingText, linear learner, DeepAR, and others—are managed containers: you provide data in the expected format and hyperparameters; SageMaker runs a maintained, scalable implementation. XGBoost on tabular data remains the default first answer for good reason.

Autopilot automates candidate generation: point it at a tabular dataset and a target, and it explores feature preprocessors, algorithms, and hyperparameters, returning a best model *plus* the candidate notebooks—so you can read what it did. Autopilot is Week 17’s engine exposed directly, with more control and more visibility.

Custom containers are the escape hatch: bring any Docker image (PyTorch, XGBoost with custom objectives, domain libraries), and SageMaker Training runs it on managed instances with the same data-channel, artifact, and IAM conventions. The training contract stays constant; only the code is yours.

Tip

Stay on the lowest rung of the control ladder that meets the requirement. Built-ins are patched and tuned by AWS; Autopilot adds exploration you can inspect; custom containers add a supply chain you now own—base-image CVEs, dependency updates, and rebuilds are yours forever.

18.4 Wednesday: Inference Shapes—Real-Time, Serverless, Batch

18.4.1 Four Ways to Serve

- **Real-time endpoints:** persistent instances behind an HTTPS endpoint, millisecond latency, billed while running. For interactive, SLA-bound traffic.
- **Serverless endpoints:** scale to zero, cold starts in seconds, billed per request compute. For sporadic or unpredictable traffic.
- **Asynchronous endpoints:** queue-backed, large payloads, up to hour-scale processing. For document/video-class payloads with relaxed latency.
- **Batch Transform:** no endpoint at all—score a dataset from S3 to S3 on ephemeral compute. For nightly scoring of everything.

Shape	Latency	Cost shape	Best fit
Real-time	Milliseconds	Always-on instance-hours	Interactive, SLA-bound
Serverless	Seconds (cold start)	Per-request compute	Sporadic traffic
Asynchronous	Minutes–hours	Instance-hours while processing	Large payloads
Batch Transform	Dataset-scale	Ephemeral job-hours	Periodic bulk scoring

Table 18.1: Choosing an inference shape.

The data engineer’s rule of thumb: if the consumer is a dashboard or nightly process, you want Batch Transform and a scored table (the Week 17 pattern at platform scale); if the consumer is an application with users waiting, you want an endpoint, and the serverless variant until traffic justifies always-on.

Common Pitfall

An always-on endpoint for a model scored once a day. A forgotten `ml.m5.xlarge` endpoint runs 730 hours a month to serve one batch—often dwarfing the entire training bill. Match the inference shape to the traffic shape, and audit for orphaned endpoints monthly.

18.5 Thursday Hands-on Project: Wrangler to Serverless Endpoint

Clean an S3 dataset in Data Wrangler, train an XGBoost model with Autopilot, deploy to a serverless endpoint, and send a test payload.

Lab Objectives

- Prepare a governed S3 dataset and verify the Studio domain’s access path.
- Build a Data Wrangler flow (type fixes, null handling, one-hot encoding) and export it.
- Launch an Autopilot experiment (XGBoost candidates included) on the cleaned data.
- Deploy the best model to a serverless endpoint and score a payload.

Procedure

1. Stage the churn dataset from Week 17 (CSV) in a governed bucket; confirm the Studio execution role reads it through the Lake Formation grant, not a broad S3 policy.
2. In Data Wrangler: import the dataset, fix types, impute nulls in `avg_session_min`, one-hot encode `segment`, and export the flow as a Processing Job.
3. Run the exported job to produce the cleaned dataset—this, not the interactive session, is the pipeline artifact.
4. Create an Autopilot experiment on the cleaned data with `is_churned` as target; inspect the candidate list and the best model’s metrics.
5. Deploy the best candidate as a **serverless** endpoint.

```
1 import json
2 import boto3
3
```

```

4 runtime = boto3.client("sagemaker-runtime", region_name="us-east-1")
5
6 payload = "12,49.90,2,38.5,segment_b" # CSV feature row for the model
7 resp = runtime.invoke_endpoint(
8     EndpointName="churn-serverless-endpoint",
9     ContentType="text/csv",
10    Body=payload,
11 )
12 probability = float(resp["Body"].read().decode())
13 print({"churn_probability": probability})

```

Listing 18.1: Scoring a test payload against the serverless endpoint.

Verification

- The exported Wrangler flow reruns as a job and reproduces the cleaned dataset byte-comparably.
- The Autopilot leaderboard is inspectable and the chosen model's validation metrics are recorded.
- The endpoint returns a sane probability for the test payload; cold-start latency is measured and acceptable.
- The endpoint scales to zero after idleness—verify zero instance-hours accrue overnight.
- The Studio domain's Lake Formation-scoped access is demonstrated by a denied read of an unrelated PII table.

18.6 Friday Use Case: GPU Training on a Governed Lake

A retailer's data science team trains custom PyTorch demand models on GPU instances. The training data lives in the governed S3 lake from Part 2, including customer-level features behind Lake Formation column masks. Security requires that scientists never receive standing broad S3 access. Design the workflow.

18.6.1 Architecture

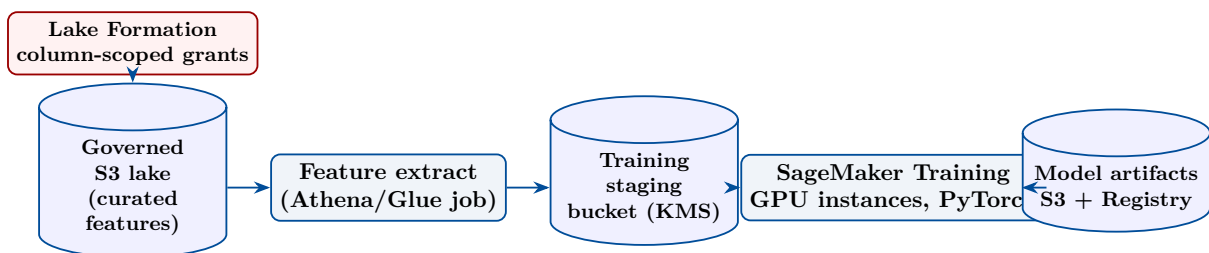


Figure 18.1: Governed GPU training: a catalog-scoped extraction job produces the training set; scientists' roles never touch the governed lake directly.

18.6.2 Design Decisions

Access through the catalog, not around it. Scientists submit feature requests; an approved extraction job (Athena or Glue, running under a role with the Lake Formation grants) materializes the training set into a KMS-encrypted staging bucket. Column masks apply at

extraction, so PII columns never reach the staging area. The scientists’ execution roles read only staging. Governance is enforced by architecture, not policy documents.

GPU discipline. Training jobs are ephemeral and billed per second: right-size (a `ml.g5.xlarge` before an `ml.p4d.24xlarge`), use managed Spot training for fault-tolerant runs at a large discount with checkpointing to S3, and enforce `max_runtime` so a hung job stops billing. Every job’s input is a versioned S3 prefix and its output a versioned artifact—the training run is reproducible or it is not done.

Network containment. Training runs in the platform VPC with no internet route and VPC endpoints for S3, ECR, and logs (Chapter 21’s perimeter applies here). Exfiltration resistance is a property of the network, not of the notebook policy.

Common Pitfall

Granting the Studio role `s3:*` “temporarily.” The governed lake’s value is exactly the access boundary. One broad role on the domain and every column mask in Chapter 8 becomes decorative for the ML team. The extraction-job pattern costs an afternoon to build and protects the boundary permanently.

18.7 Interview Q&A

18.7.1 Concepts and Trade-offs

1. **Q: Real-time versus serverless versus Batch Transform: which fits sporadic traffic?**
A: Serverless endpoints—scale to zero between requests, pay per request compute, accept seconds of cold start. Real-time is for steady SLA-bound traffic; Batch Transform for bulk offline scoring.
2. **Q: What does SageMaker Data Wrangler export after interactive cleaning?**
A: A repeatable artifact: a Processing Job script, a pipeline step, or a notebook—so interactive prep becomes rerunnable code rather than a one-off session.
3. **Q: When do you need a custom container instead of built-in XGBoost?**
A: When the framework, architecture, or dependencies have no managed equivalent—custom PyTorch models, specialized libraries, or pinned research stacks. Otherwise stay on maintained images.
4. **Q: How does Autopilot pick its final model?**
A: It explores preprocessors, algorithms, and hyperparameters as candidates, evaluates them against the objective metric on validation splits, and ranks a best model—with candidate notebooks exposed for inspection.
5. **Q: Why route training-data access through Lake Formation instead of direct S3?**
A: So column masks and grants apply to the extraction: PII never reaches the staging bucket, scientists never hold broad lake access, and every training set is a governed, reproducible artifact.
6. **Q: What is managed Spot training and what does it require of your code?**
A: Training on spare capacity at a steep discount with possible interruption—requiring checkpointing to S3 so an interrupted job resumes rather than restarts.

18.8 Further Reading

The SageMaker Developer Guide covers Studio, Data Wrangler, Autopilot, Training Jobs, and all four inference shapes [27]. The Lake Formation guide’s credential-vending model underpins the Friday architecture [51], and the KMS guide covers the staging-bucket encryption posture [50]. The Well-Architected Machine Learning Lens extends the analytics lens used throughout [3].

Key Takeaways

- SageMaker is a workbench: Studio, Wrangler, Autopilot, Training, Registry, and four inference shapes.
- Interactive data prep graduates only through export—Wrangler flows become jobs, or the pipeline is fiction.
- Use the lowest control rung that fits: built-in, then Autopilot, then custom container with its supply-chain cost.
- Match inference shape to traffic shape; the orphaned always-on endpoint is the classic ML budget leak.
- Training data access goes through governed extraction, not broad roles; masks apply at extraction time.
- GPU economics are instance-rightsizing, managed Spot with checkpoints, and hard run-time caps.

18.9 Exercises

1. **(Conceptual)** Explain why Batch Transform often beats a real-time endpoint for data-engineering consumers.
2. **(Conceptual)** What does Autopilot expose that Redshift ML (Week 17) does not, and why does that matter for governance reviews?
3. **(Hands-on)** Run the Thursday project; then redeploy the same model as Batch Transform over the full dataset and compare cost.
4. **(Hands-on)** Measure serverless cold-start latency across the first five invocations and report the distribution.
5. **(Hands-on)** Export the Wrangler flow as a Pipelines step (preview of Week 19) and execute it standalone.
6. **(Design)** Design the feature-request workflow for the Friday scenario: request, approval, extraction, staging TTL, and audit trail.
7. **(Design)** Write the GPU instance selection policy for the team: when g5, when p4d, when CPU is actually correct.
8. **(Architecture)** Extend the Friday architecture for two data-science teams with separate data domains and one shared model registry.

18.10 Capstone Project: A Governed ML Training Service

Capstone Project

Mission. Productize the Friday architecture: a self-service, governed path from feature request to trained model—catalog-scoped extraction, reproducible training on right-sized compute, artifacts in the registry, and inference shaped to the consumer.

Estimated time. 6–8 hours in a sandbox AWS account.

What you'll practice.

- The extraction-job pattern enforcing Lake Formation masks for ML consumers.
- Reproducible training: versioned inputs, managed Spot, runtime caps, checkpointing.
- Inference-shape selection with cost evidence per shape.
- Endpoint lifecycle hygiene: scale-to-zero defaults and orphan audits.

Scenario

The retailer's data platform team productizes ML access for two data scientists. Every model trained must be reproducible (inputs, code, config versioned), every training dataset must be a governed extraction, and every endpoint must justify its shape against a traffic estimate.

Requirements

Functional requirements.

- A parameterized extraction job (Athena or Glue) that materializes approved feature sets into a KMS-encrypted staging bucket with TTL.
- A training template (PyTorch or XGBoost) with versioned input prefixes, S3 checkpointing, and `max_runtime`.
- Deployment to the shape justified by a documented traffic estimate; serverless is the default.
- A registry entry per model with metrics and lineage to the training-set version.

Non-functional requirements.

- No scientist role can read the governed lake directly; the denial is demonstrated.
- Managed Spot is used where the framework's checkpointing allows; interruption recovery is demonstrated.
- A monthly orphan-endpoint audit exists and has run at least once.

Implementation Steps

1. Build the extraction job and its approval-light workflow (a reviewed SQL file per feature set).
2. Implement the training template with checkpointing; run an interruption drill.
3. Register the model with metrics and input lineage.
4. Deploy serverless; document the traffic estimate and the shape decision.
5. Implement the orphan-endpoint audit as a scheduled Lambda report.
6. Write the team's operating guide: request to endpoint in five documented steps.

Deliverables

- The extraction job, training template, and deployment configuration in version control.
- The interruption-drill evidence and the Spot savings estimate.
- The registry entry with lineage and the shape-decision memo.
- The access-denial evidence and the first orphan-audit report.

Acceptance Criteria

- A new model can go from approved feature set to endpoint using only the documented path.
- Training resumes from checkpoint after a forced Spot interruption.
- Every artifact (dataset, code, config, model) is version-linked in the registry.
- Endpoint spend is attributable per model and per team.

Stretch Goals

- Wrap the training template as a SageMaker Pipeline (preview of Week 19).
- Add Autopilot as a baseline candidate that custom models must beat to register.
- Implement data-quality checks on extractions (preview of Chapter 22) that block training on bad inputs.
- Cost-compare the whole workflow against the Redshift ML path from Week 17 for the same problem.

MLOps: Feature Stores, Pipelines, and Model Monitoring

Learning Objectives

- Operate SageMaker Feature Store with online and offline stores and reason about training/serving skew.
- Build SageMaker Pipelines: Preprocess → Train → Evaluate → Register, with quality gates.
- Gate deployments through the Model Registry with approval states.
- Configure Model Monitor baselines and schedules to detect data drift and quality violations.
- Architect a drift-triggered retraining loop with human approval before redeployment.
- Trace lineage across features, training jobs, model versions, and endpoints.

Cross-Cloud Equivalence

MLOps primitives—feature stores, pipeline orchestration, registries, drift monitoring—are platform-standard; the AWS names map one-for-one.

Capability	AWS		Azure	GCP		
Feature store	SageMaker Store	Feature	Fabric feature store / Feathr	Vertex Store	AI	Feature
ML pipelines	SageMaker Pipelines	Pipelines	Azure ML pipelines	Vertex (KFP)	AI	Pipelines
Model registry	SageMaker Registry	Model	Azure ML registry	Vertex Registry	AI	Model
Drift monitoring	SageMaker Monitor	Model	Azure ML monitoring	Vertex Monitoring	AI	Model
Retrain trigger	EventBridge pipeline	+	Logic Apps/Functions + pipeline	Eventarc	+ pipeline	
Online store backing	DynamoDB-backed records		Cosmos-backed	Bigtable-backed		
Lineage	SageMaker + OpenLineage	Lineage	MLflow tracking	Vertex ML Metadata		

Snowflake's Feature Store and Model Registry bring the same lifecycle inside the warehouse. **Databricks** MLflow is the open-source reference many of these services interoperate with. **MongoDB** typically appears as the online-serving store behind low-latency feature lookups in self-managed stacks.

19.1 Week 19 Roadmap and Mental Model

Week 18 trained and deployed a model. Week 19 makes that a *lifecycle*: models are retrained on fresh features, gated by quality thresholds, registered as immutable versions, monitored in production, and rolled back when reality disagrees with them. The discipline is called MLOps, and its artifacts will feel familiar: pipelines, registries, gates, monitors, runbooks—the data-engineering practices of Part 3 applied to models [30].

Three systems carry the week. **Feature Store** makes features shared, versioned, and consistent between training and serving. **SageMaker Pipelines** makes the path from data to registered model a code artifact. **Model Monitor** watches production traffic against a baseline and turns drift into events—which EventBridge routes back into the pipeline, closing the loop.

Definition

MLOps is the application of CI/CD and observability discipline to machine learning: versioned features, reproducible training pipelines, gated model registration, monitored production behavior, and controlled retraining and rollback.

Key Terms

Feature group: a Feature Store table of related features keyed by record ID and event time. **Online store**: low-latency lookup for inference. **Offline store**: S3-backed history for training. **Pipeline**: a DAG of steps (processing, training, evaluation, registration) defined in Python. **Model Registry**: versioned model packages with approval states. **Baseline**: the training-data statistics and constraints Model Monitor compares production traffic against. **Data drift**: input distribution change; **concept drift**: the input-to-outcome relationship changing.

19.2 Monday: SageMaker Feature Store—Online and Offline

19.2.1 One Definition, Two Stores

A feature group declares feature names, types, a record identifier, and an event-time column. Writes flow to two stores [28]: the *online store* (millisecond **GetRecord** lookups for inference) and the *offline store* (an S3-backed, catalog-registered table for training-set construction and point-in-time-correct joins). The same definition serves both, which is the entire point: **the feature a model trained on is the feature the endpoint receives**—same computation, same values, no hand-ported transformation.

19.2.2 Training/Serving Skew and Point-in-Time Correctness

The classic ML production bug is skew: features computed one way in a training notebook and another way in the serving path. Feature Store’s answer is structural—one ingestion path feeding both stores. Its second answer is *point-in-time correctness*: offline queries can reconstruct *as of* timestamps, so training examples join features as they were when the label was known, preventing the future from leaking into training.

Common Pitfall

Computing features twice. When transforms are coded once for training and again for serving, they will drift—different null handling, different rounding, different time zones. If you cannot use Feature Store, put the transformation in one shared library both paths call.

The mechanism is negotiable; the single definition is not.

19.3 Tuesday: SageMaker Pipelines

19.3.1 The Pipeline as the Product

A SageMaker Pipeline defines ML workflow steps in Python—`ProcessingStep` (feature prep), `TrainingStep`, evaluation, `ConditionStep` gates, and `RegisterModel`—executed on managed infrastructure with per-step caching and full lineage [30]. Parameters (input S3 URI, instance types, thresholds) make one definition serve experimentation and production. Caching means rerunning with unchanged inputs reuses prior outputs—iteration is cheap; correctness is enforced.

```

1 cond = ConditionStep(
2     name="QualityGate",
3     conditions=[ConditionGreaterThanOrEqualTo(
4         left=JsonGet(step=eval_step, property_file=metrics,
5                       json_path="classification.f1"),
6         right=0.80)],
7     if_steps=[register_step],
8     else_steps=[FailStep(name="RejectModel",
9                          error_message="F1 below 0.80")],
10 )

```

Listing 19.1: The quality gate: register only when F1 clears the bar.

The `ConditionStep` is where engineering judgment becomes policy: no metric, no registration. Note what this implies—every pipeline needs an evaluation step that produces a machine-readable metrics file, which forces the team to state its acceptance criteria in code.

19.3.2 Model Registry: Versions and Approvals

Registered models are immutable versioned packages: artifact location, metrics, approval status (`PendingManualApproval` → `Approved`), and lineage back to the training job and dataset. Deployment automation keys on approval: the CD system (Chapter 24) deploys only `Approved` versions, so “who approved this model in production?” always has an auditable answer.

19.4 Wednesday: Model Monitor—Drift as an Event

19.4.1 Baselines and Schedules

Model Monitor learns a baseline from the training data—per-feature distributions and constraints—then runs scheduled jobs comparing captured production traffic against it [29]. Violations (a feature’s distribution shifted, nulls appeared, types changed) emit events. Two monitoring modes matter: *data quality* (inputs versus baseline) and *model quality* (predictions versus ground truth, when labels arrive later).

Data drift versus concept drift. Data drift: the inputs changed (a new traffic source, a broken upstream feed emitting nulls). Concept drift: the relationship between inputs and outcomes changed (customer behavior shifted after a pricing change). Data drift is detectable immediately from features alone; concept drift requires delayed ground truth. Both are events to respond to, not curiosities to report.

Common Pitfall

A baseline computed from stale training data. If the baseline is six months old, Monitor reports drift against a world that no longer exists, and the team learns to ignore alerts—the worst possible outcome. Refresh baselines on a cadence tied to retraining, and treat alert fatigue as a monitoring bug.

Tip

Treat drift alerts as incidents, not curiosities: violations publish to SNS with a runbook that defines who acknowledges, what the rollback path is (previous Model Registry version), and when retraining auto-triggers. Gate automatic retraining behind a human approval step—drift can also mean broken upstream ingestion, and retraining on corrupt data makes things worse.

19.5 Thursday Hands-on Project: A SageMaker Pipeline, End to End

Build and execute a SageMaker Pipeline—Preprocess → Train → Evaluate → Register—from a notebook, with a quality gate.

Lab Objectives

- Define the four-step pipeline with parameters for input data and thresholds.
- Execute from a notebook and inspect per-step outputs and lineage.
- Demonstrate the gate: one run passes, one fails on an injected quality drop.
- Approve the registered version in the Registry.

Procedure

The pipeline definition (abridged to its skeleton; the full script lives in the lab repository):

```

1 from sagemaker.workflow.pipeline import Pipeline
2 from sagemaker.workflow.parameters import ParameterString, ParameterFloat
3 from sagemaker.workflow.steps import ProcessingStep, TrainingStep
4 from sagemaker.workflow.conditions import ConditionGreaterThanOrEqualTo
5 from sagemaker.workflow.condition_step import ConditionStep
6 from sagemaker.workflow.fail_step import FailStep
7 from sagemaker.workflow.properties import PropertyFile
8
9 input_data = ParameterString(name="InputData")
10 f1_threshold = ParameterFloat(name="F1Threshold", default_value=0.80)
11
12 # preprocess_step: ProcessingStep running the exported Wrangler/job script
13 # train_step:      TrainingStep (XGBoost) consuming preprocess output
14 # eval_step:      ProcessingStep producing evaluation.json
15
16 metrics = PropertyFile(name="Metrics", step_name="Evaluate",
17                        path="evaluation.json")
18
19 register_step = ... # RegisterModel with metrics + approval "
20                     PendingManualApproval"
21
22 gate = ConditionStep(
23     name="QualityGate",
24     conditions=[ConditionGreaterThanOrEqualTo(

```

```

24     left=PropertyFile(...), # JsonGet on classification.f1
25     right=f1_threshold)],
26     if_steps=[register_step],
27     else_steps=[FailStep(name="RejectModel", error_message="F1 below gate")],
28 )
29
30 pipeline = Pipeline(
31     name="churn-pipeline",
32     parameters=[input_data, f1_threshold],
33     steps=[preprocess_step, train_step, eval_step, gate],
34 )
35 pipeline.upsert(role_arn="arn:aws:iam::123456789012:role/SageMakerExecRole")
36 execution = pipeline.start(parameters={"InputData": "s3://.../curated/churn/"})

```

Listing 19.2: Pipeline skeleton: four steps and a gate.

Verification

- A run on good data registers a model version in `PendingManualApproval` with metrics attached.
- A run on the corrupted dataset fails at `QualityGate` with the reject reason visible.
- Rerunning with unchanged inputs reuses cached step outputs—visible in execution time and step status.
- Approving the version is a recorded, attributable action; the lineage graph links endpoint → version → training job → dataset.

19.6 Friday Use Case: Drift-Triggered Retraining for a Recommendation Engine

A recommendation engine serves from a real-time endpoint. Traffic patterns shift seasonally and after marketing campaigns. The team wants drift in production to trigger retraining automatically—but never to deploy automatically without a human decision. Architect the loop.

19.6.1 Architecture

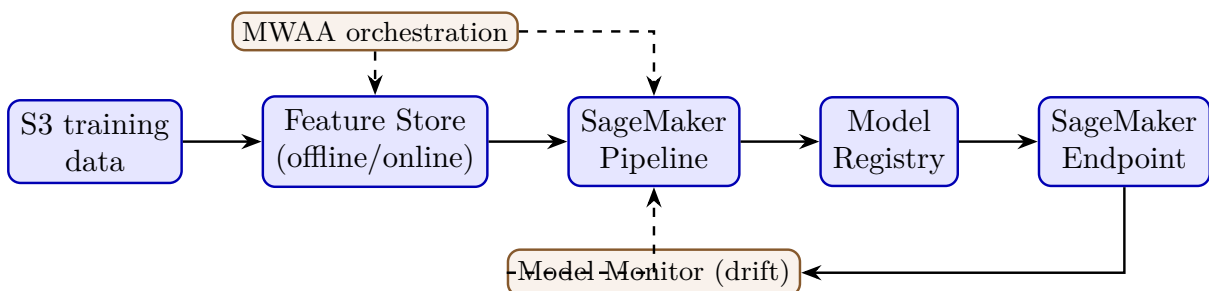


Figure 19.1: The MLOps loop: monitored endpoint traffic feeds drift detection; violations route through EventBridge to a retraining pipeline with a human approval gate before redeploy.

19.6.2 Design Decisions

The loop. Endpoint traffic is captured (data capture config) → Model Monitor’s scheduled comparison against the baseline → violations publish to EventBridge → a rule starts the retraining pipeline with fresh Feature Store data → the quality gate and a *human approval* step stand between a candidate and the registry → the CD system deploys approved versions (blue/green, with rollback to the previous approved version).

Why the human stays in. Drift has two causes: the world changed (retrain is right) or the pipeline broke (retraining on broken data is doubly wrong). The monitor cannot tell these apart; the runbook can. The approval step’s job is the triage: check the ingestion alarms from Chapters 11–14 before approving anything.

Skew control. All training features come from the offline store; all serving lookups from the online store; one ingestion path defines both. The pipeline’s preprocess step reads the offline store by point-in-time join, so retraining on “last 90 days” is one parameter change.

Rollback as a first-class path. Every endpoint change is a registry-versioned deployment; the previous approved version stays warm enough to restore within minutes. “Rollback” is a deployment of an older artifact, not an emergency procedure.

Common Pitfall

Online/offline feature skew. If a campaign-feature transform is coded in the training job and again in the serving Lambda, the two will diverge—silently. Ingest through one path into Feature Store, or share one transformation library between paths, and test equivalence in CI.

19.7 Interview Q&A

19.7.1 Concepts and Trade-offs

1. **Q: Online versus offline Feature Store: which serves training, which inference?**
A: Offline (S3-backed, point-in-time joins) builds training sets; online (millisecond `GetRecord`) serves inference. One feature-group definition feeds both to prevent skew.
2. **Q: What does the Model Registry gate before deployment?**
A: Approval. Deployment automation keys on the **Approved** state of a versioned, metrics-carrying package—so production changes are deliberate and attributable.
3. **Q: Data drift versus concept drift: what is the difference?**
A: Data drift is a change in input distributions, detectable from features alone. Concept drift is a change in the input-to-outcome relationship, detectable only when delayed ground truth arrives.
4. **Q: Why gate automatic retraining behind human approval?**
A: Drift can mean the world changed *or* upstream ingestion broke. Automatic retraining on corrupt data automates the production of worse models; the human triages the cause.
5. **Q: What baseline does Model Monitor need to detect violations?**
A: Per-feature statistics and constraints learned from the training data, refreshed on a cadence tied to retraining so “normal” stays current.
6. **Q: What does pipeline step caching buy you?**
A: Reruns with unchanged inputs reuse prior outputs: cheaper, faster iteration with correctness enforced by input fingerprinting.

19.8 Further Reading

The SageMaker documentation covers Feature Store [28], Pipelines [30], and Model Monitor [29] in depth. OpenLineage provides the open lineage standard for cross-platform tracing [93]. The Airflow documentation covers the MWAA side of the loop [80].

Key Takeaways

- Feature Store is the skew-control mechanism: one definition, two stores, point-in-time correctness.
- Pipelines make the path from data to registry a code artifact; the ConditionStep is policy in code.
- The Registry’s approval state is the deployment boundary; lineage makes every production model answerable.
- Model Monitor turns drift into events; baselines must stay fresh or alerts become noise.
- The retraining loop is EventBridge-routed and human-gated; rollback is a deployment, not a panic.
- MLOps is data-engineering discipline—versioning, gates, monitors, runbooks—applied to models.

19.9 Exercises

1. **(Conceptual)** Explain why point-in-time joins prevent label leakage even when feature data arrives late.
2. **(Conceptual)** Why is an ignored drift alert worse than no monitoring at all?
3. **(Hands-on)** Run the Thursday pipeline; then lower the gate threshold, rerun, and observe the registry outcome change.
4. **(Hands-on)** Create a feature group, ingest a week of synthetic features, and build a training set via point-in-time query.
5. **(Hands-on)** Configure Model Monitor on a deployed endpoint; inject a shifted feature and capture the violation event.
6. **(Design)** Write the drift runbook: alert payload, triage steps, approval criteria, rollback steps, and retraining triggers.
7. **(Design)** Design the label-arrival path for model-quality monitoring when outcomes are known 30 days late.
8. **(Architecture)** Extend the loop for two models sharing one feature group; where does shared infrastructure help and where does it couple failure?

19.10 Capstone Project: A Drift-Guarded Recommendation Lifecycle

Capstone Project

Mission. Operate the Friday loop for real: a Feature-Store-backed pipeline, a monitored endpoint, drift events routed to retraining with human approval, and rollback demonstrated—the complete MLOps lifecycle with evidence at every stage.

Estimated time. 8–10 hours in a sandbox AWS account.

What you'll practice.

- Feature Store ingestion with online and offline stores.
- Pipeline construction with evaluation gates and registry integration.
- Monitor baselines, schedules, and violation-to-EventBridge routing.
- Approval-gated deployment and versioned rollback.

Scenario

The recommendation engine's owner wants one monthly touch: review the drift report, approve or reject the candidate model. Everything else—features, training, evaluation, registration, monitoring—runs itself and produces the evidence for that decision.

Requirements

Functional requirements.

- A feature group with daily ingestion; training sets built by point-in-time offline queries.
- The four-step pipeline with an F1 gate and registry integration.
- A monitored endpoint (data capture on, baseline, daily schedule) with violations to SNS via EventBridge.
- A retraining trigger from violations to the pipeline, ending at human approval.

Non-functional requirements.

- A rollback drill: deploy candidate, detect regression, restore previous approved version in minutes.
- Skew evidence: the same feature values served online as used in training, tested.
- Every monthly decision is reconstructable: drift report, candidate metrics, approver, deployment record.

Implementation Steps

1. Create the feature group and the ingestion job; verify online/offline parity.
2. Build and gate the pipeline; register the baseline model.
3. Deploy the monitored endpoint; establish the baseline and schedule.
4. Wire violations through EventBridge to the retraining pipeline with the approval step.
5. Run the drift drill (shifted feature) and the rollback drill; record both.
6. Produce the first monthly decision packet end to end.

Deliverables

- All code: ingestion, pipeline, monitor configuration, EventBridge rules.
- Drill evidence: drift detection to approval, and rollback timing.
- The skew test and the lineage graph for the production model.
- The monthly decision packet template, completed once.

Acceptance Criteria

- A drift violation demonstrably reaches a human decision with full context attached.
- No candidate deploys without approval; the gate is tested with a rejected candidate.
- Rollback restores the prior version within the stated time, with evidence.
- Training and serving features are provably identical for a sample of records.

Stretch Goals

- Add model-quality monitoring with a simulated 30-day label delay.
- Emit OpenLineage events and trace one production prediction to its training data.
- Blue/green deploy with automatic rollback on endpoint metric regression.
- Add a challenger model in shadow mode and compare live predictions before promotion.

Unstructured Data and Managed AI Services

Learning Objectives

- Extract text, key-value pairs, and tables with Amazon Textract, and label image/video content with Amazon Rekognition.
- Apply Amazon Comprehend (NLP) and Amazon Translate to document text at pipeline scale.
- Integrate AI APIs into Lambda and Glue pipelines with the synchronous/asynchronous boundary respected.
- Index extracted metadata into OpenSearch for discovery.
- Manage PII risk in persisted OCR output.
- Architect a terabyte-scale image pipeline: OCR, object tagging, searchable metadata—and deliver the Milestone 5 end-to-end MLOps capstone.

Cross-Cloud Equivalence

Pre-trained AI APIs—vision, OCR, NLP, translation—are commodity building blocks on every cloud; the skill is pipeline integration, not model building.

Capability	AWS	Azure	GCP
OCR / document parsing	Textract	Document Intelligence	Document AI
Image/video labeling	Rekognition	Azure AI Vision	Vision AI / Video AI
NLP (entities, sentiment, PII)	Comprehend	Azure AI Language	Natural Language AI
Translation	Translate	Translator	Translation API
Search over metadata	OpenSearch	AI Search	Vertex AI Search
Async document jobs	Textract async APIs	Batch APIs	Batch processing
PII detection	Comprehend PII + Macie	Language PII + Purview	Sensitive Data Protection
Custom models	Rekognition Custom Labels, Comprehend custom	Custom Vision	Vertex custom training

Fabric exposes Azure AI services natively in notebooks and pipelines. **Snowflake** offers Document AI and Cortex functions—extraction as SQL. **MongoDB** stores the extracted

JSON naturally; its vector search serves the retrieval side of the same estate.

20.1 Week 20 Roadmap and Mental Model

So far, “data” meant rows. Week 20 opens the other 80% of enterprise information: scanned forms, PDFs, images, video, free text. The week’s services are *pre-trained APIs*—you do not train Textract or Rekognition; you integrate them. That shifts the engineering content entirely to the pipeline: event-driven invocation, sync/async boundaries, cost per thousand pages, error handling and retries, result persistence, indexing, and the privacy posture of everything the APIs emit [34, 25].

The mental model: an AI API is a *per-record enrichment transform* with a network boundary. Everything learned in Part 4 about Lambda transforms applies; everything in Part 2 about catalogs and governance applies to the outputs.

Definition

Managed AI services are pre-trained, API-accessible models—vision, OCR, NLP, translation—billed per unit of input (page, image, character block). They convert unstructured content into structured, catalog-able, indexable data without any model training.

Key Terms

Textract: OCR plus structure—key-value pairs, tables, forms; synchronous for single pages, asynchronous for multi-page documents. **Rekognition:** label, face, text, and moderation detection for images and video. **Comprehend:** entities, sentiment, key phrases, PII detection, custom classification. **Translate:** neural machine translation. **OpenSearch:** managed search/analytics engine for the extracted metadata. **Confidence score:** every API result carries one; thresholds are a pipeline design input.

20.2 Monday: Rekognition and Textract

20.2.1 Vision as an API

Rekognition answers “what is in this image/video?”: labels with confidence scores, detected text, faces (analysis, and recognition against a collection you manage), and content moderation categories [25]. For data pipelines the pattern is uniform: S3 object in, JSON result out, confidence thresholds applied downstream.

20.2.2 Textract: Structure from Documents

Textract goes beyond raw OCR: `AnalyzeDocument` returns forms (key-value pairs) and tables as structured blocks, and `DetectDocumentText` returns lines and words [34]. The critical operational boundary: the synchronous APIs accept single-page documents up to a few MB; multi-page PDFs and TIFFs require the *asynchronous* API (`StartDocumentAnalysis`), which runs a job and publishes completion to SNS. Every document pipeline of consequence is therefore asynchronous—which means it is an event pipeline (S3 upload → job → SNS → result handling), and Chapter 12’s orchestration discipline applies.

Common Pitfall

Calling the synchronous Textract API on multi-page PDFs. It fails or truncates—and teams then split PDFs into single pages and call sync APIs per page, paying per-page latency and building a distributed fan-out by accident. Use the async API with SNS completion; that is what it exists for.

20.3 Tuesday: Comprehend and Translate

20.3.1 NLP as Enrichment

Comprehend turns text into structured signal: entities (people, places, organizations), sentiment, key phrases, language, and—crucially for this book’s governance thread—*PII detection* with entity types and offsets [8]. Custom classifiers and entity recognizers cover domain-specific needs when the general model is not enough. Translate handles language conversion at character-billed volume. Both integrate the same way: text in, JSON out, per-unit pricing—and both are happiest called in batches from a Lambda or Glue job rather than per row from a query engine.

Tip

Comprehend’s PII detection and Macie (Chapter 23) overlap but serve different moments: Comprehend is *in-pipeline* (redact or route before persistence); Macie is *at-rest discovery* (find what already landed). Mature designs use both.

20.4 Wednesday: Integrating AI APIs into Data Pipelines

20.4.1 The Integration Pattern

The canonical shape: **S3 event** → **Lambda (or Glue job)** → **AI API** → **structured JSON to S3** → **catalog + index**. Lambda suits per-object, sub-15-minute work; Glue suits bulk backfills and joins against existing tables. Three disciplines matter at scale:

Idempotency and cost. API calls are billed per unit; a retried pipeline that re-analyzes every object pays twice. Persist results keyed by object etag/version and skip already-processed inputs—Chapter 10’s bookmark discipline applied to APIs.

Result persistence as a dataset. Extraction results are data: write them as versioned JSON under a governed prefix, register them in the catalog, and treat their schema as a contract (Chapter 11). “The JSON from Textract” becomes a table analysts join against everything else.

Indexing for discovery. Extracted metadata earns its keep when searchable: an OpenSearch domain indexed from the results stream (via Lambda or Firehose, Chapter 14) turns ten million processed documents into a queryable corpus [20].

20.4.2 PII in OCR Output

OCR output is faithful: if the scanned form contained a national ID number, the JSON contains it too. Persisting raw extraction JSON is persisting PII. The pipeline must decide per field: keep (governed zone, masked grants), redact (Comprehend PII + substitution), or route to quarantine (Chapter 23’s workflow). The default for raw outputs is the restricted zone with a TTL; curated, reviewed extracts are what analysts see.

20.5 Thursday Hands-on Project: S3-Triggered Textract Extraction

Write a Lambda function triggered by an S3 upload that sends a scanned PDF to Textract, extracts key-value pairs, and saves them as JSON back to S3.

Lab Objectives

- Wire an S3 event notification to Lambda on a `raw/documents/` prefix.
- Call Textract (async for multi-page), parse the block structure into key-value JSON.
- Persist results idempotently under `extracted/documents/`.
- Handle a failure path visibly.

Procedure

For the lab's single-page forms the synchronous API suffices; the code notes where the async boundary sits.

```

1 import json
2 import os
3 from urllib.parse import unquote_plus
4
5 import boto3
6
7 s3 = boto3.client("s3")
8 textract = boto3.client("textract")
9 OUT_PREFIX = os.environ.get("OUT_PREFIX", "extracted/documents/")
10
11 def lambda_handler(event, context):
12     for rec in event["Records"]:
13         bucket = rec["s3"]["bucket"]["name"]
14         key = unquote_plus(rec["s3"]["object"]["key"])
15
16         # Idempotency: skip if already extracted for this exact object version.
17         out_key = f"{OUT_PREFIX}{key.rsplit('/', 1)[-1]}.json"
18         head = s3.head_object(Bucket=bucket, Key=key)
19         etag = head["ETag"].strip('"')
20         try:
21             existing = s3.head_object(Bucket=bucket, Key=out_key)
22             if existing.get("Metadata", {}).get("source-etag") == etag:
23                 continue
24         except s3.exceptions.ClientError:
25             pass
26
27         # Sync API: single-page documents only. Multi-page ->
28         # StartDocumentAnalysis
29         # with SNS completion and a second Lambda handling the result.
30         resp = textract.analyze_document(
31             Document={"S3Object": {"Bucket": bucket, "Name": key}},
32             FeatureTypes=["FORMS"],
33         )
34
35         # Reconstruct key-value pairs from the block graph.
36         blocks = {b["Id"]: b for b in resp["Blocks"]}
37         kv = {}
38         for b in resp["Blocks"]:
39             if b["BlockType"] == "KEY_VALUE_SET" and "KEY" in b["EntityTypes"]:
40                 key_text = _text_of(b, blocks)
41                 val_text = ""

```

```

41         for rel in b.get("Relationships", []):
42             if rel["Type"] == "VALUE":
43                 for vid in rel["Ids"]:
44                     vb = blocks[vid]
45                     val_text = _text_of(vb, blocks)
46             if key_text:
47                 kv[key_text] = val_text
48
49     s3.put_object(
50         Bucket=bucket, Key=out_key,
51         Body=json.dumps({"source": key, "fields": kv}, indent=2).encode(),
52         ContentType="application/json",
53         Metadata={"source-etag": etag},
54     )
55
56 def _text_of(block, blocks):
57     parts = []
58     for rel in block.get("Relationships", []):
59         if rel["Type"] == "CHILD":
60             for cid in rel["Ids"]:
61                 child = blocks[cid]
62                 if child["BlockType"] == "WORD":
63                     parts.append(child["Text"])
64     return " ".join(parts)

```

Listing 20.1: S3-triggered Lambda extracting key-value pairs with Textract.

Verification

- Uploading a sample form produces a JSON file with the expected key-value pairs.
- Re-uploading the identical object does not re-call Textract (idempotency via the etag metadata check).
- A corrupted file produces a visible failure (Lambda error metric, retry, then a documented dead-letter outcome).
- The output prefix is registered in the catalog and queryable in Athena.

20.6 Friday Use Case: Terabyte-Scale Image Understanding with Search

A photo-sharing platform receives terabytes of user images daily. Product wants every image searchable by its content (“receipts,” “dogs on beaches,” “documents with signatures”), and trust-and-safety wants moderation at upload. Design the pipeline.

20.6.1 Architecture

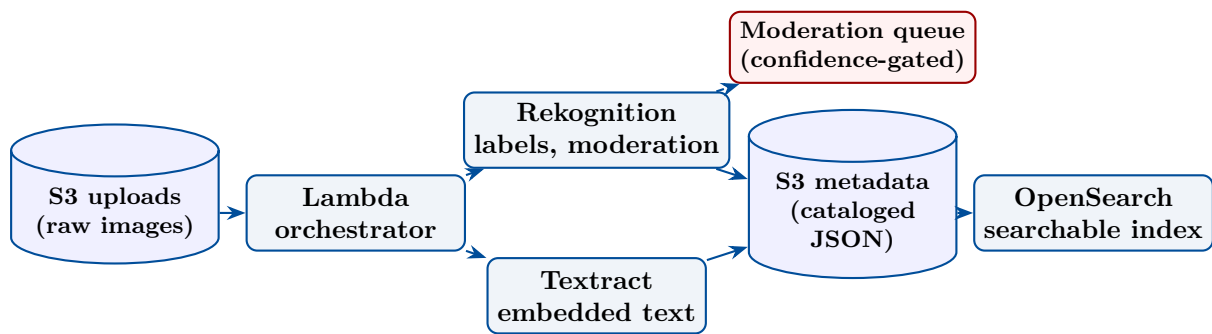


Figure 20.1: Image understanding pipeline: parallel enrichment by Rekognition and Textract, cataloged JSON metadata, OpenSearch indexing, and a confidence-gated moderation path.

20.6.2 Design Decisions

Fan-out per image. One S3 event triggers a Lambda that invokes Rekognition (labels + moderation) and, only for images with detected text density, Textract—conditional invocation is a cost decision at terabyte scale. Results converge into one metadata document per image.

Confidence thresholds as policy. Labels below 80% confidence are stored but not searchable; moderation scores above the action threshold route to a human-review queue with the image reference—never auto-deletion on a model’s say-so. Thresholds are configuration, versioned, and tuned from review-outcome statistics.

The metadata is the product. Cataloged JSON (labels, OCR text, moderation verdict, EXIF) in the governed lake feeds analytics; an OpenSearch index fed from the same documents serves interactive search. Two consumers, one extraction: the write-once, read-many discipline of the whole book applied to AI output.

Cost shape. Per-image Rekognition calls dominate; Textract runs on a filtered subset; OpenSearch sizing follows index volume and query rate. The monthly model states per-1,000-image costs and the levers (label-only tiers, text-density gating, index refresh interval).

Common Pitfall

Persisting OCR text without a PII posture. User images contain documents, ID cards, and screenshotted messages. The extracted text is PII-bearing data the moment it lands: govern the raw outputs, redact in the searchable index, and let Chapter 23’s Macie scanning verify the posture continuously.

20.7 Interview Q&A

20.7.1 Concepts and Trade-offs

1. **Q: Textract versus Comprehend: which extracts key–value pairs from scanned forms?**

A: Textract—`AnalyzeDocument` with `FORMS` returns the form’s key–value structure. Comprehend operates on already-digital text (entities, sentiment, PII), often as Textract’s downstream.

2. **Q: Why trigger AI APIs from S3 events via Lambda?**

A: Arrival-driven processing with no polling and per-object scale-out; the event carries the object identity, and Lambda's concurrency model matches spiky upload patterns.

3. **Q: When must you switch from synchronous to asynchronous Textract jobs?**

A: For multi-page PDFs/TIFFs or large documents. The async API runs a job and signals completion via SNS; the sync API is single-page only.

4. **Q: How do you make extracted document metadata searchable?**

A: Persist normalized JSON results, then index them into OpenSearch (via Lambda or Firehose) while also registering the results in the catalog for SQL analytics.

5. **Q: What PII risks exist when persisting raw OCR output?**

A: OCR is faithful—IDs, account numbers, and addresses in the document appear in the JSON. Raw outputs need a restricted zone, field-level redaction for curated zones, and at-rest discovery scanning.

6. **Q: How do you keep retried pipelines from re-billing API calls?**

A: Idempotency keyed on object identity: persist results with the source etag/version and skip already-processed inputs.

20.8 Further Reading

The Textract and Rekognition developer guides cover block structure, async jobs, and confidence semantics [34, 25]; the Comprehend guide covers NLP and PII detection [8]; the OpenSearch guide covers indexing patterns [20]. The event and Lambda foundations are in [14, 52].

Key Takeaways

- Managed AI APIs are per-record enrichment transforms with a network boundary; pipeline discipline is the engineering content.
- The sync/async boundary (single-page versus multi-page) determines pipeline shape; async means event-driven completion.
- Persist extraction results as governed, cataloged datasets with idempotency keyed on object identity.
- Confidence thresholds are versioned policy, and moderation decisions stay human-gated.
- Raw OCR output is PII-bearing data: restrict it, redact for curated use, and scan continuously.
- One extraction, many consumers: catalog for analytics, OpenSearch for interactive discovery.

20.9 Exercises

1. **(Conceptual)** Explain why an asynchronous document API forces an event-driven pipeline design.
2. **(Conceptual)** Why store low-confidence labels but exclude them from search, rather than discarding them?
3. **(Hands-on)** Run the Thursday lab; then convert a multi-page PDF path to the async API with SNS completion.

4. **(Hands-on)** Add Comprehend PII detection to the extraction Lambda and redact detected spans in the curated output.
5. **(Hands-on)** Index the extracted JSON into a small OpenSearch domain and query it.
6. **(Design)** Design the moderation-review workflow: queueing, reviewer tooling inputs, decision persistence, and threshold tuning feedback.
7. **(Design)** Build the monthly cost model for the Friday pipeline at 50 million images/day; identify the top two cost levers.
8. **(Architecture)** Extend the architecture for video uploads: which Rekognition APIs apply, and what changes about latency and cost?

20.10 Milestone 5 Capstone: End-to-End MLOps Pipeline

Capstone Project

Mission. Deliver the full MLOps lifecycle orchestrated by MWAA: Athena extracts features into SageMaker Feature Store, a custom training job produces a model, the Model Registry gates it, and Model Monitor watches the serving endpoint for training/serving skew—with MWAA owning the schedule and the alerts.

Estimated time. 8–10 hours in a sandbox AWS account.

What you'll practice.

- Feature engineering from the lake (Athena) into Feature Store (Weeks 18–19).
- Custom SageMaker training with versioned inputs.
- Registry gating, monitored deployment, and drift feedback.
- Airflow orchestration of the whole lifecycle (Week 12).

Scenario

The retailer's personalization team ships a propensity model monthly. The platform team must turn that into infrastructure: one DAG run produces features, a trained model, an evaluated registry entry, and a monitored deployment—with serving skew watched continuously.

Requirements

Functional requirements.

- An Athena feature extraction writing to a feature group (offline for training, online for serving).
- A SageMaker Pipeline (preprocess → train → evaluate → register) with a metric gate.
- MWAA orchestration from feature extraction through pipeline execution.
- A monitored endpoint with a skew/violation alarm to SNS and an MWAA failure alert.

Non-functional requirements.

- End-to-end runs from a clean environment per the README.
- At least three automated tests: feature validation, a model-quality gate, and a drift-injection test demonstrating a Monitor violation.
- A monthly cost estimate (training instance-hours, endpoint versus Batch Transform, Feature Store reads/writes) with two documented optimizations.

- Security review: training access scoped via Lake Formation, artifacts in a KMS-encrypted bucket, no credentials in notebooks.

Reference Architecture

The pipeline follows fig. 19.1: Athena extraction → Feature Store → Pipeline → Registry → monitored endpoint, with MWAA orchestrating and drift violations feeding back through EventBridge to retraining with human approval.

Implementation Steps

1. Create the feature group and the Athena-to-Feature-Store ingestion step.
2. Define the pipeline with the quality gate ($F1 \geq 0.80$) and registry integration.
3. Author the MWAA DAG: extraction → pipeline start → status watch → notify.
4. Deploy the approved model with data capture; build the baseline and monitoring schedule.
5. Implement the three tests, including drift injection producing a real violation.
6. Write the cost model and the security review.

Deliverables

- The DAG, pipeline definition, and training code in version control.
- Test evidence, including the injected drift violation and the gate rejection of a weak model.
- Alarm configurations and one fired alert of each kind.
- The cost model with two optimizations and the security review.

Acceptance Criteria

- One DAG run carries a dataset from extraction to a gated, registered, deployable model.
- A weak model is rejected at the gate; a drift event reaches SNS with context.
- Training data access is demonstrably catalog-governed; artifacts are KMS-encrypted.
- Endpoint shape is justified against the traffic estimate (serverless or batch by default).

Stretch Goals

- Add a champion/challenger shadow deployment with live comparison metrics.
- Wire the Week 20 enrichment outputs as additional features and measure lift.
- Add lineage emission (OpenLineage) from the DAG and pipeline.
- Automate monthly baseline refresh tied to the retraining cadence.

Part VI

Security, Governance, and Exam Prep

Security, IAM, and Networking for Data Platforms

Learning Objectives

- Apply least-privilege IAM: roles, policies, permissions boundaries, and service control policies (SCPs).
- Keep data traffic off the public internet with gateway and interface VPC endpoints and PrivateLink.
- Manage encryption keys with AWS KMS and application secrets with Secrets Manager, including rotation.
- Build and verify an S3 gateway endpoint and access a bucket from EC2 without an internet gateway.
- Design an anti-exfiltration data perimeter that holds even when IAM credentials are compromised.
- Reason about the cost of the perimeter itself: endpoint-hours, KMS requests, and budget alarms.

Cross-Cloud Equivalence

Identity, private networking, and key management form the security substrate of every cloud; the concepts are identical, the acronyms differ.

Capability	AWS	Azure	GCP
Identity and access	IAM roles/policies	Entra ID + Azure RBAC	Cloud IAM
Org-wide guardrails	SCPs (Organizations)	Azure Policy	Organization Policy
Permissions ceiling	Permissions boundaries	Management groups + deny	IAM deny policies
Private service access	VPC endpoints / PrivateLink	Private Endpoints / Private Link	Private Service Connect
Key management	AWS KMS	Azure Key Vault	Cloud KMS
Secrets management	Secrets Manager	Key Vault secrets	Secret Manager
Free S3-class endpoint	Gateway endpoint	Service endpoints	Private Google Access
Anti-exfiltration primitive	Endpoint policy + SCP + aws:PrincipalOrgID	Private Link + policy	VPC Service Controls

Snowflake adds account-level network policies and external-private-endpoint rules; **MongoDB Atlas** uses private endpoints and IP access lists; **Fabric** centralizes on Entra conditional access and tenant-level perimeter settings. The pattern—identity plus network plus keys—is universal.

21.1 Week 21 Roadmap and Mental Model

Every chapter so far ended with an implicit question: who is allowed to touch this, and over what network path? Week 21 makes the answers explicit. The week’s stack has three layers: **identity** (IAM roles and policies, bounded by permissions boundaries and SCPs), **network** (VPC endpoints and PrivateLink so traffic never leaves the AWS backbone), and **cryptography** (KMS keys and Secrets Manager so access requires more than a stolen credential).

The mental model that organizes all three is the *data perimeter*: a set of controls ensuring that data can only be accessed by expected identities, from expected networks, within the expected organization [49]. A data platform that is fast, cheap, and exfiltratable is a liability; this week is how you make it merely fast and cheap.

Definition

A data perimeter is the combination of identity, network, and organization-boundary controls that constrains data access to expected principals, expected networks, and the expected organization—designed to hold even if a credential is stolen.

Key Terms

Role: an assumable identity with temporary credentials—the default way workloads authenticate. **Permissions boundary**: a maximum-permissions ceiling on a role, regardless of its policies. **SCP**: an Organizations-level guardrail bounding what accounts may do. **Gateway endpoint**: free VPC route-table entry for S3/DynamoDB. **Interface endpoint**: PrivateLink-backed ENI for most other services, billed per AZ-hour plus data. **KMS**: managed key service with envelope encryption and key policies. **Secrets Manager**: secrets storage with automatic rotation.

21.2 Monday: IAM Deep Dive—Roles, Boundaries, and SCPs

21.2.1 Least Privilege as an Engineering Practice

The unit of AWS authorization is the *role*: workloads assume roles and receive temporary credentials—never long-lived access keys. Least privilege is an iterative practice, not a one-time design: start from the actions the workload provably needs (from CloudTrail or Access Analyzer), scope resources by ARN and prefix, add conditions (`aws:SourceVpc`, `aws:PrincipalTag`), and re-review on a cadence [49].

21.2.2 Ceilings: Permissions Boundaries and SCPs

Two mechanisms bound what even a misconfigured policy can grant. A *permissions boundary* caps a role’s maximum permissions—delegated role-creation (developers create their own Glue job roles) is safe only because the boundary prevents those roles from exceeding their lane. An *SCP* applies at the AWS Organizations level and cannot be overridden by any account administrator: “deny all S3 access from outside our VPC endpoints,” “deny leaving the organization,” “restrict regions.” Boundaries protect role hygiene; SCPs protect the organization.

Common Pitfall

An allow without a ceiling. IAM evaluates identity policies *and* boundaries *and* SCPs—the effective permission is the intersection. Teams that only write identity policies leave both ceiling mechanisms empty, so any future over-broad grant is instantly effective. Deploy the ceilings first; they are cheap and invisible until the day they save you.

21.3 Tuesday: VPC Endpoints and PrivateLink

21.3.1 Gateway Versus Interface

A *gateway endpoint* (S3 and DynamoDB only) adds a route-table entry directing service traffic over the AWS backbone—no ENI, no hourly charge, and an endpoint policy that can restrict which buckets are reachable. An *interface endpoint* places an elastic network interface in your subnet backed by AWS PrivateLink, available for most AWS services and for third-party/private services—billed per AZ-hour plus per-GB processed [53].

The data-platform consequence: with an S3 gateway endpoint, a VPC with *no internet gateway at all* can still reach the lake—which is exactly the posture regulated workloads want. Glue, Redshift, Athena, KMS, and the rest get interface endpoints; traffic to AWS APIs never touches the public internet.

Note

Cost guardrail: interface endpoints bill per AZ-hour plus per-GB. For high-volume S3 traffic always prefer the free gateway endpoint, and consolidate interface endpoints (one shared-services VPC) instead of one per workload VPC. Set AWS Budgets alerts per account *before* the hardening project, so the perimeter itself never produces a surprise bill.

21.3.2 Endpoint Policies: The Network Half of the Perimeter

An endpoint policy restricts what the endpoint will carry: “this S3 endpoint serves only these buckets, only for principals in our organization.” Endpoint policies are how the network layer enforces the data perimeter independently of IAM—a stolen credential used from a laptop cannot route through your endpoint, and your endpoint will not carry data to a stranger’s bucket.

21.4 Wednesday: KMS and Secrets Manager

21.4.1 KMS: Keys as Policy Objects

AWS KMS provides envelope encryption: data keys encrypt data; KMS customer master keys encrypt data keys [50]. For the data engineer, the essentials are: customer-managed keys (CMKs) when you need key policy control, rotation evidence, and cross-account sharing (the lake’s key must allow the analytics account to decrypt); per-domain keys to bound blast radius; and CloudTrail logging of every key use, which makes “who decrypted what” an answerable question. Grants and key policies together decide whether a stolen credential can decrypt anything—with a per-service, per-bucket key policy, often it cannot.

21.4.2 Secrets Manager: No More Static Credentials

Database passwords, API keys, and JDBC strings live in AWS Secrets Manager, retrieved at run-time by IAM-authenticated callers, and—the point—*rotated automatically* on a schedule with

Lambda rotators that update both the secret and the target database [55]. Static credentials in environment variables, DAG code, or `.env` files are the most common real-world data-platform breach vector; rotation converts a leaked credential from a breach into an incident ticket.

Tip

Redshift, RDS, and document databases have native Secrets Manager rotation integrations. For anything else, the pattern is: secret in Secrets Manager, application fetches at startup and caches briefly, rotation Lambda updates both sides, and CloudTrail alarms on unusual `GetSecretValue` patterns.

21.5 Thursday Hands-on Project: Private S3 Access with No Internet Route

Create a VPC endpoint for S3, then access a bucket from an EC2 instance in a VPC with no internet gateway—proving the private path works and the public path does not exist.

Lab Objectives

- Build a VPC with a private subnet and no internet gateway.
- Add an S3 gateway endpoint with a restrictive endpoint policy.
- From EC2 (via Systems Manager Session Manager, since there is no inbound path), copy a file to the bucket.
- Demonstrate that the endpoint policy blocks access to a non-authorized bucket.

Procedure

```

1 # 1. VPC with a private subnet; deliberately no IGW is created or attached.
2 $VpcId = (aws ec2 create-vpc --cidr-block 10.21.0.0/16 '
3   --query Vpc.VpcId --output text)
4 $SubnetId = (aws ec2 create-subnet --vpc-id $VpcId '
5   --cidr-block 10.21.1.0/24 --query Subnet.SubnetId --output text)
6 $RtId = (aws ec2 describe-route-tables --filters Name=vpc-id,Values=$VpcId '
7   --query 'RouteTables[0].RouteTableId' --output text)
8
9 # 2. Gateway endpoint for S3, scoped by endpoint policy to one bucket.
10 $Policy = @"
11 {
12   "Version": "2012-10-17",
13   "Statement": [{
14     "Effect": "Allow",
15     "Principal": "*",
16     "Action": ["s3:GetObject", "s3:PutObject", "s3:ListBucket"],
17     "Resource": [
18       "arn:aws:s3:::week21-private-bucket-123456789012",
19       "arn:aws:s3:::week21-private-bucket-123456789012/*"
20     ]
21   }]
22 }
23 "@
24 $Policy | Set-Content -Path endpoint-policy.json -Encoding ascii
25
26 aws ec2 create-vpc-endpoint --vpc-id $VpcId '
27   --service-name com.amazonaws.us-east-1.s3 '
28   --route-table-ids $RtId '
29   --policy-document file://endpoint-policy.json

```

```

30
31 # 3. EC2 in the private subnet (SSM-enabled role; no public IP, no IGW).
32 aws ec2 run-instances --image-id ami-0abcdef1234567890 '
33   --instance-type t3.micro --subnet-id $SubnetId '
34   --iam-instance-profile Name=Week21SSMRole --no-associate-public-ip-address
35
36 # 4. From the instance (via Session Manager):
37 #   aws s3 cp test.txt s3://week21-private-bucket-123456789012/ # succeeds
38 #   aws s3 ls s3://some-other-bucket/ # denied by
    endpoint policy

```

Listing 21.1: Private VPC, gateway endpoint, and the proof.

Verification

- The copy to the authorized bucket succeeds with no internet gateway attached—traffic used the gateway endpoint.
- Access to any other bucket fails with an explicit endpoint-policy denial.
- VPC flow logs (or S3 server access logs with `vpce` source) evidence the private path.
- A Route53/hosted check confirms no public DNS egress path exists from the subnet.

21.6 Friday Use Case: A Healthcare Data Perimeter That Survives Credential Theft

A healthcare provider’s threat model states: *assume an IAM credential will eventually be compromised; no data may leave to an outside AWS account even then*. Design the perimeter for the analytics platform.

21.6.1 The Layered Design

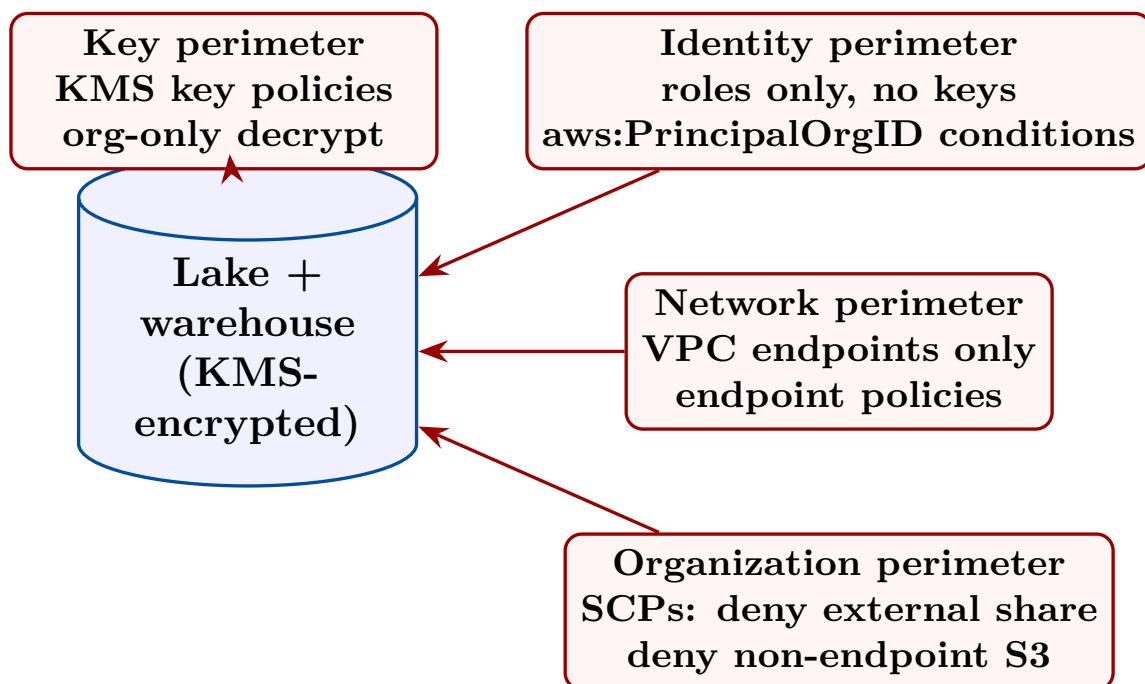


Figure 21.1: The four perimeters: identity, network, organization, and keys. A stolen credential must defeat all four to exfiltrate data.

Identity perimeter. All access via roles with conditions requiring the organization (`aws:PrincipalOrgID`) and, where applicable, the expected network (`aws:SourceVpce`). No IAM users with long-lived keys exist to steal in the first place; workforce access is federated.

Network perimeter. Data services are reachable only through VPC endpoints whose policies allow only the organization’s buckets and only expected actions. There is no internet gateway in data subnets. A credential used from an attacker’s laptop satisfies no network condition.

Organization perimeter (SCP). An SCP denies `s3:PutObject` to any bucket where the destination account is outside the organization, denies disabling CloudTrail, and confines regions. Even a fully compromised account admin cannot exceed an SCP.

Key perimeter. Bucket-default KMS keys whose policies allow decrypt only for organization principals. Data copied out—if every other layer somehow failed—arrives as ciphertext without a usable key.

```

1 {
2   "Effect": "Deny",
3   "Action": ["s3:PutObject", "s3:PutObjectAcl"],
4   "Resource": "arn:aws:s3:::/*/*",
5   "Condition": {
6     "StringNotEquals": {
7       "aws:PrincipalAccount": ["111111111111", "222222222222"]
8     }
9   }
10 }
```

Listing 21.2: The anti-exfiltration SCP: deny S3 writes to non-organization accounts.

21.6.2 Verification and Operations

The perimeter is tested like code: a red-team drill attempts exfiltration with a deliberately “leaked” sandbox credential and must fail at a named layer, with the CloudTrail event captured. Continuously, Config rules check for internet gateways in data VPCs and for buckets without org-scoped key policies, and CloudTrail Lake Formation/S3 data events alarm on access patterns outside the endpoint set.

Common Pitfall

A perimeter that exists only in diagrams. Each layer must have a test that would fail if the layer were removed. Untested SCPs have typos; untested endpoint policies have wildcards; untested key policies allow *. The red-team drill is the unit test of the perimeter.

21.7 Interview Q&A

21.7.1 Concepts and Trade-offs

1. **Q: Gateway versus interface VPC endpoint: which is free for S3 traffic?**
A: The gateway endpoint—a route-table entry for S3 and DynamoDB with no hourly or per-GB charge. Interface endpoints (PrivateLink) cover other services and bill per AZ-hour plus data.
2. **Q: What does a permissions boundary limit on a role?**
A: The role’s maximum permissions regardless of its identity policies—the intersection of policy and boundary is effective. It makes delegated role creation safe.

3. **Q: Why is automatic Secrets Manager rotation better than static keys?**

A: Rotation bounds the value of any leaked credential to the rotation window and updates the target system automatically; static keys leak forever and rotate never.

4. **Q: How do SCPs differ from IAM policies?**

A: SCPs apply at the Organizations level and cap what any account—including its administrators—may do. IAM policies grant within an account; SCPs bound the account itself.

5. **Q: What exfiltration path does PrivateLink close that security groups cannot?**

A: Security groups control instance traffic but not which *service endpoints* data flows to. Endpoint policies on PrivateLink/gateway endpoints restrict the destinations themselves—your network cannot be used to write to a stranger’s bucket.

6. **Q: Why does KMS complete the perimeter?**

A: Key policies can restrict decryption to organization principals, so data copied past every other control arrives as unusable ciphertext.

21.8 Further Reading

The IAM User Guide covers roles, boundaries, and policy evaluation [49]; the PrivateLink guide covers both endpoint types and endpoint policies [53]; KMS and Secrets Manager documentation cover envelope encryption, key policies, and rotation [50, 55]. The Config and CloudTrail guides support the verification layer [41, 39].

Key Takeaways

- Roles everywhere, users never; least privilege iterated from observed usage.
- Boundaries and SCPs are ceilings that make every future grant safer; deploy them first.
- Gateway endpoints make private S3 free; interface endpoints privatize everything else—with a price to watch.
- Endpoint policies are the network half of the perimeter, independent of IAM.
- KMS key policies are the final layer: ciphertext without an org key is not exfiltration.
- Perimeters are tested by drills, not diagrams; every layer needs a test that would fail without it.

21.9 Exercises

1. **(Conceptual)** Walk the IAM evaluation logic for a role with an allow policy, a boundary, and an SCP. What wins in a conflict?
2. **(Conceptual)** Why does the identity-perimeter condition on `aws:SourceVpce` defeat a stolen credential used from an attacker’s machine?
3. **(Hands-on)** Run the Thursday lab; then tighten the endpoint policy to read-only and demonstrate the write failing.
4. **(Hands-on)** Create a Secrets Manager secret for a database, wire automatic rotation, and verify a rotation cycle.
5. **(Hands-on)** Use IAM Access Analyzer to generate a least-privilege policy from a role’s CloudTrail history and diff it against the hand-written policy.

6. **(Design)** Write the SCP set for a three-account data estate: region restriction, CloudTrail protection, and exfiltration denial.
7. **(Design)** Price the perimeter for the Friday scenario: endpoints per AZ, KMS request volume, Config rules, and CloudTrail data events.
8. **(Architecture)** Design break-glass access for the perimeter: who can bypass, how is it alarmed, and how does it expire?

21.10 Capstone Project: A Tested Data Perimeter for the Analytics Estate

Capstone Project

Mission. Wrap the course’s analytics estate in a tested perimeter: org-scoped SCPs, endpoint-privatized networking, org-scoped KMS keys, rotated secrets—and prove it with a red-team drill that fails at a named layer.

Estimated time. 6–8 hours in a sandbox organization.

What you'll practice.

- SCP authoring and attachment with safe rollout.
- Gateway and interface endpoint design with endpoint policies.
- KMS key policy design for cross-account analytics.
- Evidence-driven verification: drills, Config rules, CloudTrail alarms.

Scenario

The healthcare provider from Friday approves a proof-of-concept on two sandbox accounts (data and analytics). Your job: the estate must hold against a simulated credential leak, and every control must come with its test.

Requirements

Functional requirements.

- SCPs: region restriction, anti-exfiltration (listing 21.2 generalized to `aws:PrincipalOrgID`), CloudTrail protection.
- S3 gateway endpoint plus interface endpoints for the services in use, with bucket-scoped endpoint policies.
- Bucket-default KMS with organization-scoped decrypt; Secrets Manager with rotation for the database credential.
- The red-team drill: a “leaked” sandbox credential attempting console and API exfiltration.

Non-functional requirements.

- Every layer has a test that demonstrably fails when the control is removed (in the sandbox).
- Config rules and CloudTrail alarms watch the perimeter continuously.
- A cost statement covers the perimeter’s own monthly bill.

Implementation Steps

1. Stand up the two-account organization; attach SCPs progressively and test each.
2. Build the private networking per listing 21.1, extended to the used services.
3. Re-key the buckets to org-scoped CMKs; migrate the database credential into Secrets Manager with rotation.
4. Author the drill script and run it; capture the layer and CloudTrail event of each failure.
5. Deploy the Config rules and alarms; document the cost statement.

Deliverables

- SCPs, endpoint policies, and key policies as code.
- The drill report: attempted actions, blocking layer, evidence events.
- Config rule set and alarm definitions with one fired example.
- The perimeter cost statement and the operations runbook.

Acceptance Criteria

- The drill's exfiltration attempts all fail at named, evidenced layers.
- Cross-account analytics (the lake read from the analytics account) still works—the perimeter permits the intended path.
- Rotation demonstrably invalidates the previous database credential.
- Removing any single control in the sandbox is detected by its test.

Stretch Goals

- Add Macie (Chapter 23) to close the loop on what the perimeter protects.
- Implement network-perimeter exceptions as time-boxed, alarmed break-glass roles.
- Extend the drill to an insider scenario: an authorized analyst exceeding their normal pattern.
- Codify the whole perimeter in Terraform (Chapter 24) with policy tests in CI.

Data Governance and Data Quality at Scale

Learning Objectives

- Share catalog data across accounts with Lake Formation and manage access with LF-Tags.
- Author AWS Glue Data Quality rulesets (DQDL) with thresholds, and route failures to alerts.
- Use lineage—SageMaker ML Lineage and open standards—to trace quality failures to their upstream source.
- Implement a quality gate that halts a pipeline when a required column exceeds 5% nulls.
- Distinguish rules that catch schema drift from rules that catch content degradation.
- Architect a five-domain data mesh with centralized governance and decentralized data engineering.

Cross-Cloud Equivalence

Quality rules, lineage, and mesh organization are the governance layer every platform now ships; the vocabulary converged faster here than anywhere else in the stack.

Capability	AWS	Azure	GCP
Quality rules engine	Glue Data Quality (DQDL)	Fabric/Purview data quality	Dataplex data quality
Cross-account sharing	Lake Formation + RAM	Purview + Fabric sharing	Analytics Hub / Dataplex
Tag-based governance	LF-Tags	Purview classifications	Policy tags
Lineage	SageMaker Lineage, OpenLineage	Purview lineage	Dataplex lineage
Catalog of record	Glue Data Catalog	Microsoft Purview	Dataplex
Open-source alternative	Great Expectations, DataHub	Same (cloud-agnostic)	Same
Quality gate pattern	In-job evaluation + SNS	Pipeline gates	Dataflow/Dataplex gates

Snowflake implements quality as data metric functions and governance as Horizon Catalog. **MongoDB** enforces shape at write time with schema validation—the closest operational-store analog to a DQ ruleset. **Fabric** ties quality and lineage into OneLake’s unified gov-

ernance story.

22.1 Week 22 Roadmap and Mental Model

Weeks 5–20 built a platform that moves and computes data correctly when everything works. Week 22 asks the harder questions: how do you *know* the data is right, and who is accountable when it is not? Governance at scale has two technical pillars—a catalog with fine-grained, tag-based access (Chapter 8 extended across accounts) and automated data quality (rules as code, evaluated in pipelines)—and one organizational pillar: ownership that scales, which leads to the data mesh.

The connecting idea: *quality and access are properties of data products, enforced in pipelines, traceable through lineage* [45, 93]. A dashboard that silently consumed a half-null feed is a governance failure before it is an engineering one.

Definition

Data quality engineering is the practice of expressing expectations about data (completeness, uniqueness, validity, freshness) as versioned, executable rules evaluated inside pipelines—with failures that halt, alert, and trace to root cause.

Key Terms

DQDL: Data Quality Definition Language—Glue’s rule syntax (`IsComplete`, `ColumnValues`, ...). **Ruleset**: a named, versioned set of rules evaluated against a dataset or frame. **Threshold**: a soft-fail level allowing observed-versus-expected comparison. **Lineage**: the recorded graph of what produced what. **Data mesh**: domain-owned data products with federated governance. **Data product**: a dataset with an owner, a contract, quality SLAs, and discoverability.

22.2 Monday: Cross-Account Sharing and Tag-Based Access

22.2.1 Sharing the Catalog, Not the Data

Chapter 8 governed one account; the enterprise reality is many. Lake Formation shares catalog databases and tables across accounts (via AWS RAM invitations), and the receiving account sees shared objects in its own catalog with its own grant structure on top [51]. The data stays in the producer’s S3; the consumer’s engines read it under Lake Formation credential vending. Column masks and row filters apply across the boundary—a shared table with a masked `pii_email` arrives masked.

22.2.2 Why Tags Win at Scale

In a multi-account estate, named-resource grants multiply into an unmanageable matrix. LF-Tags invert the work: producers tag resources (`domain=finance`, `sensitivity=confidential`), consumers’ roles hold grants on tag expressions, and new datasets inherit correct policy by being tagged correctly at creation. The tag taxonomy—its keys, allowed values, and who may assign them—becomes the governance contract reviewed by the data council, with tag assignment enforced in the pipeline code that registers tables.

Common Pitfall

Tag taxonomy by convention instead of control. LF-Tag matching is exact and case-sensitive: PII and pii are different worlds, and a mistyped tag silently changes who can read the table. Constrain tag values at assignment time (CI validation on the registration code) and audit tag coverage monthly—untagged tables are ungoverned tables.

22.3 Tuesday: AWS Glue Data Quality

22.3.1 RULES as Code

Glue Data Quality evaluates rulesets written in DQDL against catalog tables or in-flight DynamicFrames, publishing results to CloudWatch and—in ETL jobs—letting you branch on the outcome [45]:

```

1 Rules = [
2     IsComplete "order_id",
3     IsUnique "order_id",
4     ColumnValues "order_amount" >= 0,
5     ColumnLength "email" between 5 and 254,
6     Completeness "email" >= 0.95 with threshold 0.9
7 ]

```

Listing 22.1: A DQDL ruleset for the orders feed.

Each rule is a falsifiable claim about the data: `IsComplete` catches nulls, `IsUnique` catches key duplication, `ColumnValues` catches out-of-domain values, `ColumnExists` catches schema drift (a renamed or dropped source column), and `Completeness` with a threshold distinguishes “some nulls” from “the feed is degrading.” Rules live in version control next to the pipeline that enforces them; a rule change is a pull request, reviewed like the contract change it is.

22.3.2 Rule Selection: Drift Versus Degradation

Two failure families need different instruments. *Schema drift*—structure changed—is caught by `ColumnExists`, `ColumnCount`, and type assertions (and prevented upstream by Chapter 11’s schema registry). *Content degradation*—structure intact, values rotting—is caught by completeness, uniqueness, distribution, and freshness rules. A ruleset covering only one family leaves the other invisible.

22.4 Wednesday: Lineage—Tracing Failures Upstream

22.4.1 Why Lineage Is an Operational Tool

When Tuesday’s `Completeness "email"` rule fails, the questions arrive instantly: which downstream tables and dashboards consumed the bad batch, and which upstream change introduced it? Lineage answers both. SageMaker ML Lineage tracks artifacts and their relationships across the ML estate; OpenLineage emits a portable event standard from Airflow, Spark, and dbt that platforms like DataHub render as the full graph [93, 86]. For the data engineer, the discipline is emission: every job you write should record its inputs, outputs, and run identity.

Tip

Combine DQ rule results with lineage: publish the failing rule, the run ID, and the dataset version into the lineage graph so a quality failure can be traced to the exact upstream table that introduced it. A red dashboard with a one-click path to the offending upstream

commit is the difference between a ten-minute and a two-day incident.

22.5 Thursday Hands-on Project: A Quality Gate That Halts the Pipeline

Implement a Glue Data Quality ruleset inside an ETL job that fails the pipeline if a required column is more than 5% null, routing failure alerts to SNS.

Lab Objectives

- Attach a DQDL ruleset to an existing Glue job's DynamicFrame.
- Branch on the evaluation outcome: pass writes; fail publishes and raises.
- Inject a violating batch and observe the halt and the alert.
- Record rule outcomes to CloudWatch for trending.

Procedure

The evaluation lives inside the job, between transform and sink:

```

1 import boto3
2 from awsglue.context import GlueContext
3 from awsglue.dynamicframe import DynamicFrame
4 from awsgluedq.transforms import EvaluateDataQuality
5
6 RULESET = """
7 Rules = [
8     IsComplete "order_id",
9     IsUnique "order_id",
10    ColumnValues "order_amount" >= 0,
11    ColumnLength "email" between 5 and 254,
12    Completeness "email" >= 0.95 with threshold 0.9
13 ]
14 """
15
16 ALERTS_TOPIC = "arn:aws:sns:us-east-1:123456789012:data-quality-alerts"
17
18 # ... earlier: build 'frame' (a DynamicFrame) from catalog sources ...
19 dq_results = EvaluateDataQuality().process_rows(
20     frame=frame,
21     ruleset=RULESET,
22     publishing_options={
23         "cloudwatch_metrics_enabled": True,
24         "results_s3_prefix": "s3://acme-analytics-raw-123456789012/dq-results/"
25     },
26     additional_options={"performanceTuning.caching": "CACHE_NOHING"},
27 )
28
29 outcome = dq_results[EvaluateDataQuality.DATA_QUALITY_RULES_OUTCOME]
30 overall = outcome.toDF().agg({"DataQualityEvaluationResult": "max"}) \
31     .first()[0]
32
33 if overall == "Failed":
34     sns = boto3.client("sns")
35     sns.publish(
36         TopicArn=ALERTS_TOPIC,
37         Subject="DQ gate failed: orders feed",

```

```

38     Message=f"Quality gate failed for run. "
39         f"Results: s3://...dq-results/ (see rule outcomes).",
40     )
41     raise RuntimeError("DQ gate failed - load halted")
42
43 # gate passed: proceed to the Redshift/lake sink

```

Listing 22.2: In-job data-quality evaluation with an SNS-routed hard gate.

Verification

- A clean batch passes the gate and lands in the target.
- A batch with 8% null `email` halts the job before any write; the SNS message arrives; rule outcomes are in S3 and CloudWatch.
- The threshold rule reports “observed 0.92 vs. required 0.95” granularity, not a bare pass/fail.
- A rerun with a fixed batch succeeds without manual target cleanup (idempotency from Chapter 10).

Common Pitfall

Warning-only quality checks in production pipelines. A rule that logs and loads anyway trains the organization that quality is advisory. Decide per rule: hard gates (halt the load) for contract-level guarantees, soft monitors (alert and load) for informational trends—and never confuse the two in the same list without labels.

22.6 Friday Use Case: A Five-Domain Data Mesh

A global conglomerate has five independent business domains—retail, logistics, finance, marketing, manufacturing—each with its own data engineers, its own AWS account, and its own release cadence. The CDO wants decentralized delivery with centralized governance: one policy language, enforced everywhere, without a central bottleneck team. Architect the mesh.

22.6.1 Operating Model

Domains own data products. Each domain publishes datasets as products: an owner, a schema contract (Chapter 11), a DQ ruleset (this week), an SLO (freshness, completeness), and catalog registration with the standard tag set. “Who owns the quality of a data product?” has one answer: the domain that publishes it.

The center owns the platform and the policy. A central governance account defines the LF-Tag taxonomy, the cross-account sharing patterns, the DQ rule libraries, and the audit aggregation (CloudTrail, DQ outcomes). It grants by tag expression—`domain=finance AND sensitivity<=internal`—so policy scales without a ticket queue.

22.6.2 Architecture

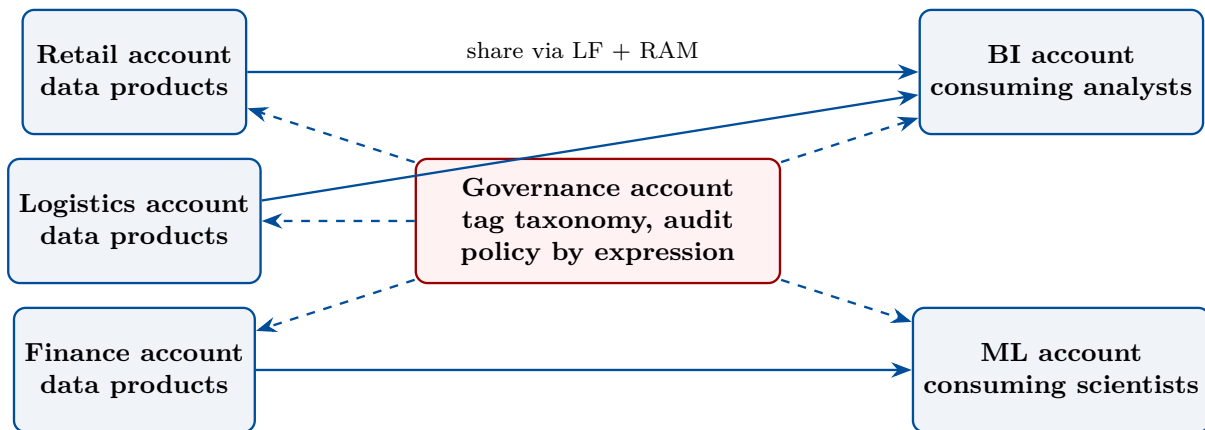


Figure 22.1: Data mesh on AWS: domains publish governed products from their own accounts; the center owns taxonomy and audit; consumers read through tag-expression grants.

22.6.3 Failure Modes the Design Anticipates

Ungoverned product drift. A domain stops tagging or stops running its DQ ruleset. Countermeasure: the governance account aggregates per-domain compliance signals (tag coverage, DQ pass rates, contract freshness) into a published scorecard—governance by transparency.

Central bottleneck relapse. If every share needs central approval, the mesh dies. Countermeasure: grants on tag expressions are pre-authorized policy; shares matching policy need no ticket.

Semantic divergence. Five domains define “customer” five ways. Countermeasure: the conformed-dimension practice of Chapter 5 survives the mesh—shared dimensions are themselves data products with named owners.

Common Pitfall

A mesh of pipelines instead of products. Sharing raw tables with no owner, contract, or SLO recreates the swamp with extra accounts. The unit of the mesh is the data product—owned, contracted, quality-gated, discoverable—not the table.

22.7 Interview Q&A

22.7.1 Concepts and Trade-offs

1. **Q: How do LF-Tags simplify cross-account data sharing?**

A: Grants bind to tag expressions instead of resource names, so any correctly tagged shared table inherits policy in every account automatically—policy scales with data classes, not table count.

2. **Q: Why consult lineage when a data-quality check fails?**

A: Lineage answers the incident’s two questions instantly: what consumed the bad batch downstream, and which upstream change introduced it—turning a two-day hunt into a directed fix.

3. **Q: ColumnExists versus Completeness: which catches schema drift?**

A: `ColumnExists`—structure. `Completeness` catches content degradation within an intact schema. A production ruleset needs both families.

4. **Q: In a data mesh, who owns the quality of a data product?**

A: The publishing domain. The center provides the platform, taxonomy, and audit—accountability for the product’s contract and SLO stays with its owner.

5. **Q: What does the threshold clause on a DQ rule buy you?**

A: Nuance: `Completeness >= 0.95` with threshold 0.9 distinguishes a mild dip (report) from a cliff (fail), so rules can monitor without halting on noise.

6. **Q: Why enforce schema contracts at registration time, not query time?**

A: Registration is where the producer’s intent is declared; query time is where violations have already become durable bad data—and possibly replicated downstream.

22.8 Further Reading

The Glue Data Quality documentation covers DQDL, in-job evaluation, and CloudWatch publication [45]; the Lake Formation guide covers cross-account sharing and LF-Tags [51]. Great Expectations and DataHub are the open-source anchors for rules and metadata respectively [90, 86], and OpenLineage is the lineage event standard [93].

Key Takeaways

- Cross-account governance shares catalogs, not copies; masks and filters survive the boundary.
- Tags are the scaling mechanism for policy; the taxonomy is a controlled artifact, validated in CI.
- DQDL rules are falsifiable, versioned claims about data, evaluated inside pipelines.
- Cover both failure families: schema drift (`ColumnExists`) and content degradation (`Completeness` and friends).
- Lineage makes quality failures traceable; emission is a discipline every job must practice.
- The mesh unit is the data product: owner, contract, SLO, discoverability—governance by transparency, not by ticket queue.

22.9 Exercises

1. **(Conceptual)** Explain why named-resource grants fail operationally past a few hundred tables, and how tag expressions fix it.
2. **(Conceptual)** Distinguish hard gates from soft monitors and give one rule that belongs in each category.
3. **(Hands-on)** Run the Thursday lab; add a `ColumnExists "email"` rule and demonstrate it catching a renamed source column.
4. **(Hands-on)** Share a catalog table to a second sandbox account via Lake Formation and RAM; verify the column mask survives.

5. **(Hands-on)** Emit OpenLineage events from the DQ gate job and visualize one failure path in DataHub.
6. **(Design)** Write the tag taxonomy (keys, values, assignment owners) for the five-domain mesh.
7. **(Design)** Design the governance scorecard: which signals, which cadence, who acts on it.
8. **(Architecture)** A domain must leave the mesh (divestiture). Design the un-sharing and data-custody process.

22.10 Capstone Project: A Governed Two-Domain Mesh Slice

Capstone Project

Mission. Prove the mesh pattern at small scale: two domain accounts publishing quality-gated, tag-governed data products, one governance account owning taxonomy and audit, and one consumer account reading through expression-based grants—with a deliberately broken feed demonstrating the full failure path.

Estimated time. 6–8 hours across three sandbox accounts.

What you'll practice.

- Cross-account Lake Formation sharing with tag-expression grants.
- Pipeline-embedded DQ gates with SNS alerting and S3 result evidence.
- Tag taxonomy enforcement in registration code.
- Governance telemetry: tag coverage and DQ pass rates per domain.

Scenario

Retail and logistics publish; analytics consumes; governance oversees. The pilot's success criterion: a consumer gains correct access to a new product with zero tickets, and a bad feed is halted, alerted, and traced without human intervention.

Requirements

Functional requirements.

- Each domain publishes one product: cataloged, tagged per taxonomy, quality-gated in its pipeline.
- The consumer account reads via LF-Tag expression grants; PII columns arrive masked.
- The governance account aggregates tag coverage and DQ outcomes into a scorecard.
- The broken-feed drill: halt, alert, and trace to the upstream change.

Non-functional requirements.

- New-product onboarding requires no grant ticket—only correct tags, enforced in CI.
- Every DQ failure leaves durable evidence (S3 results, CloudWatch metrics, SNS alert).
- Lineage connects each product to its source pipeline run.

Implementation Steps

1. Stand up the taxonomy in the governance account; codify allowed values.
2. Build each domain's pipeline with the DQ gate and tagged registration.

3. Share the products; grant the consumer by tag expression; verify masks.
4. Build the scorecard from CloudWatch and `list-permissions` exports.
5. Run the broken-feed drill and the zero-ticket onboarding drill; document both.

Deliverables

- Taxonomy definition, pipeline code, and grant configuration as code.
- Scorecard output for both domains.
- Drill evidence: halted pipeline, alert, lineage trace, and the ticketless onboarding record.
- The operating memo: what the center owns, what domains own, what breaks next.

Acceptance Criteria

- A mistyped tag value fails CI; an untagged table fails the scorecard.
- The consumer cannot reach an untagged or out-of-policy product, demonstrated.
- The broken feed never reaches the consumer; the evidence chain is complete.
- Onboarding a third (synthetic) product takes under an hour using only the documented path.

Stretch Goals

- Add Great Expectations in one domain and reconcile its results with DQDL reporting.
- Add a data contract (Chapter 11 registry) to one product and show the two layers cooperating.
- Publish the scorecard to QuickSight with per-domain drill-down.
- Simulate the divestiture exercise: un-share a domain and produce its custody report.

Compliance and Data Privacy with Amazon Macie

Learning Objectives

- Discover and classify PII in S3 with Amazon Macie, including custom data identifiers.
- Build non-destructive, auditable quarantine workflows for suspect files.
- Choose among masking, tokenization, and cryptographic hashing in pipelines.
- Propagate GDPR-style erasure requests across lake zones, warehouses, streams, caches, and backups—or render them moot with crypto-shredding.
- Audit data access with CloudTrail data events and configuration posture with AWS Config.
- Architect a third-party data onboarding pipeline that identifies, tokenizes, and safely warehouses PII.

Cross-Cloud Equivalence

Sensitive-data discovery, de-identification, and erasure are regulatory constants; each cloud ships a discovery engine and the same architectural answers.

Capability	AWS	Azure	GCP
PII discovery at rest	Amazon Macie	Microsoft Purview scanning	Sensitive Data Protection (DLP)
Custom identifiers	Macie custom data identifiers	Custom classifiers	Custom infoTypes
Audit trail	CloudTrail (incl. data events)	Azure Monitor / audit logs	Cloud Audit Logs
Config posture	AWS Config rules	Azure Policy	Security Command Center
Crypto-shredding	Per-subject KMS keys, then delete key	Per-subject KEKs in Key Vault	Per-subject keys in Cloud KMS
Quarantine pattern	Object Lock bucket + Lambda	Immutability policies + Functions	Bucket lock + functions
Erasure across copies	Enumerate zones/snapshots/streamspattern	Same problem, same pattern	Same problem, same pattern

Snowflake offers object-tagging-driven masking and supports erasure via row deletion within its retention windows—with Time Travel/Fail-safe explicitly in scope. **MongoDB** supports client-side field-level encryption, which makes per-subject crypto-shredding natural. **Fabric** centralizes discovery in Purview over OneLake.

23.1 Week 23 Roadmap and Mental Model

The platform is built; now it must answer to regulators, customers, and counsel. Week 23’s three disciplines are *discovery* (you cannot protect PII you have not found—Macie), *de-identification* (masking, tokenization, hashing—and knowing which is reversible), and *erasure* (the right to be forgotten against an architecture that was designed, all book long, to never forget anything: versioning, replication, snapshots, streams, backups) [16].

The week’s uncomfortable thesis: **deletion is an architecture, not an API call**. Every durability feature from Chapters 1–16 is an erasure obstacle, and the designs that survive audit are the ones that planned for forgetting at the same time they planned for durability—per-subject encryption keys being the standout pattern.

Definition

Amazon Macie is a managed sensitive-data discovery service for S3: it inventories buckets, samples and scans objects with managed and custom data identifiers, and produces findings—typed, located, and severity-scored—that drive quarantine and remediation workflows.

Key Terms

Managed data identifier: Macie’s built-in detectors (credit cards, national IDs, names, credentials). **Custom data identifier**: your regex-plus-context detector for domain-specific identifiers. **Finding**: a located, typed, severity-scored discovery. **Masking**: irreversible redaction for display. **Tokenization**: reversible substitution via a mapping under control. **Crypto-shredding**: deleting a subject’s encryption key to render all copies unrecoverable. **Data events**: CloudTrail’s per-object API logging.

23.2 Monday: Amazon Macie—Automated PII Discovery

23.2.1 What Macie Actually Does

Macie maintains a continuous inventory of your S3 estate and runs discovery jobs (or automated discovery) that sample objects, apply managed data identifiers (credit cards with Luhn validation, national identifiers, credentials, contact data), evaluate your custom data identifiers, and emit findings: which object, which identifiers, how many occurrences, how severe [16]. Findings route to EventBridge—which is what turns discovery into workflow.

Beyond regex, Macie’s detectors use proximity keywords, checksums, and format validation to suppress false positives—a 16-digit number that fails Luhn is not a credit-card finding. That is why managed identifiers outperform hand-rolled regex scanning on both noise and recall.

23.2.2 Custom Data Identifiers and the Noise Budget

Custom identifiers (order numbers with a known prefix, employee IDs, claim numbers) extend discovery to your domain. Their engineering constraint is the *noise budget*: an over-broad regex generates thousands of findings and trains the team to ignore real PII alerts. Tune custom identifiers against a representative sample before rollout—set proximity keywords, match thresholds, and exclusions—and review finding precision monthly.

Common Pitfall

Enabling Macie without a findings workflow. Discovery without response is liability documentation: you now have written proof that PII sat unprotected and nobody acted. Wire findings to triage (EventBridge → SNS/Lambda) before the first job runs, and define severity SLAs.

23.3 Tuesday: De-Identification and the Erasure Problem

23.3.1 Masking, Tokenization, Hashing

Three transformations, three different guarantees. **Masking** (****-****-****-1234) is irreversible display redaction—right for analyst-facing columns. **Tokenization** substitutes a value with a token whose mapping lives in a controlled store—reversible under authorization, so the same customer links across systems while raw values stay in the vault. **Cryptographic hashing** (salted, e.g., HMAC) is one-way: good for join keys and dedup, useless for re-identification—which is the point—but remember that low-entropy identifiers hash to a dictionary-attackable space.

23.3.2 Right to Erasure Against a Durable Architecture

A GDPR-style erasure request must reach every copy: raw S3 lake zones (all *versions*—versioning retains deleted objects), curated zones, Redshift tables (and their snapshots and automated backups until expiry), Kinesis streams (records persist for the full 24-hour-to-365-day retention window), downstream derived tables and aggregates, caches, and cross-region replicas [70, 73].

Pattern A: crypto-shredding. Encrypt each data subject’s records under a subject-specific KMS data key at ingestion. Erasure deletes the key; every copy—backups, replicas, Glacier archives, warehouse snapshots—becomes permanently unrecoverable in one auditable operation [50]. The cost is key-management cardinality (one key per subject) and ingestion-time discipline.

Pattern B: enumerated deletion. Maintain a per-subject copy register; on erasure, delete versioned objects, issue warehouse deletes (with CDC replay downstream), wait out stream retention, and schedule snapshot expiry—then *verify* with a Macie re-scan that no residual PII remains. The cost is a long, testable runbook.

The legal-hold exception. An S3 Object Lock legal hold overrides retention and blocks deletion until released. Erasure jobs must check hold status and route held records to an exception queue—failing or silently skipping them both violate the process.

Common Pitfall

The naive DELETE. On AWS, DELETE does not erase: S3 versioning retains prior versions, Redshift snapshots keep deleted rows until expiry, Kinesis records persist for the retention window, and Glacier archives are immutable until retention ends. Erasure runbooks must enumerate every copy location—then crypto-shred the subject key or schedule versioned deletion plus snapshot expiry, and verify with a Macie re-scan.

23.4 Wednesday: CloudTrail and Config—Auditing Access and Posture

23.4.1 CloudTrail: The Access Ledger

CloudTrail records API activity. Management events (who created the bucket) are on by default; *data events* (who read which S3 object, who invoked which Lambda) are opt-in per bucket/function and are exactly what “audit data access” means for a lake [39]. Deliver trails to a dedicated log-archive account with Object Lock, so the audit trail survives compromise of the account being audited. CloudTrail Lake turns the ledger queryable: “show every `GetObject` by this role against the PII prefix in March.”

23.4.2 Config: Posture as Code

AWS Config records resource configuration over time and evaluates rules: bucket public access disabled, KMS rotation enabled, CloudTrail logging on, endpoint policies present [41]. For the privacy program, Config is the continuous control: not “we configured it right in March,” but “it is configured correctly right now, and here is the evidence trail of every change since March.”

23.5 Thursday Hands-on Project: Macie Discovery and Automated Quarantine

Enable Macie on a bucket, upload a CSV containing mock credit card numbers, review the findings, and trigger a Lambda that quarantines the file.

Lab Objectives

- Enable Macie and run a one-time discovery job against a test prefix.
- Upload a synthetic CSV with mock card numbers and observe the finding.
- Implement the quarantine path: EventBridge rule → Lambda → move to an Object-Locked quarantine bucket.
- Produce the audit record linking the finding ID to the quarantine action.

Procedure

```

1 aws macie2 enable-macie
2
3 $Job = @"
4 {
5   "name": "week23-discovery",
6   "jobType": "ONE_TIME",
7   "s3JobDefinition": {
8     "bucketDefinitions": [{
9       "accountId": "123456789012",
10      "buckets": ["week23-ingest-123456789012"]
11    }],
12   "sampling": { "samplingType": "NONE" }
13 }
14 @"
15
16 $Job | Set-Content -Path macie-job.json -Encoding ascii
17 aws macie2 create-classification-job --cli-input-json file://macie-job.json
18

```



```

19 # Synthetic test data (never real PANs):
20 Set-Content cards.csv "name,card'nAlice,4111111111111111'nBob,5500005555555559"
21 aws s3 cp cards.csv s3://week23-ingest-123456789012/incoming/cards.csv
22 aws macie2 list-findings --finding-criteria file://severity-high.json

```

Listing 23.1: Enabling Macie and running a discovery job.

```

1 import json
2 import os
3
4 import boto3
5
6 s3 = boto3.client("s3")
7 QUARANTINE = os.environ["QUARANTINE_BUCKET"] # Object Lock enabled
8
9 def lambda_handler(event, context):
10     detail = event["detail"]
11     finding_id = detail["id"]
12     res = detail["resourcesAffected"]["s3Object"]
13     bucket, key = res["bucketName"], res["key"]
14
15     # Move, do not delete: quarantine preserves evidence under Object Lock.
16     dest_key = f"quarantined/{finding_id}/{key.split('/')[1]}"
17     s3.copy_object(
18         Bucket=QUARANTINE, Key=dest_key,
19         CopySource={"Bucket": bucket, "Key": key},
20         MetadataDirective="REPLACE",
21         Metadata={"macie-finding-id": finding_id, "source-bucket": bucket},
22     )
23     s3.delete_object(Bucket=bucket, Key=key)
24
25     # The action itself is CloudTrail-logged; the finding ID in metadata
26     # and this log line are the audit linkage.
27     print(json.dumps({"action": "quarantine", "finding": finding_id,
28                       "from": f"{bucket}/{key}", "to": dest_key}))
29

```

Listing 23.2: Quarantine Lambda: non-destructive, audited.

Verification

- The discovery job reports findings on the test file with the credit-card identifier type and occurrence locations.
- The EventBridge rule fires on the finding; the object moves to the quarantine bucket with the finding ID in its metadata.
- The quarantine bucket's Object Lock prevents deletion of the quarantined object; CloudTrail shows the copy, the delete, and the finding publication.
- A clean control file produces no finding and remains in place—the false-positive check.

Common Pitfall

Quarantining by deletion. Deleting the suspect file destroys the evidence and makes the incident unanswerable. Move to an Object-Locked quarantine bucket, log the Macie finding ID with every action to CloudTrail, and let retention policy—not panic—govern final disposal.

23.6 Friday Use Case: Tokenizing Third-Party Customer Data Onboarding

A retailer buys third-party customer datasets monthly: CSVs with names, emails, phone numbers, and purchase histories. The warehouse needs the analytics value; compliance requires that raw PII never persists in the analytical estate. Design the onboarding pipeline.

23.6.1 Architecture

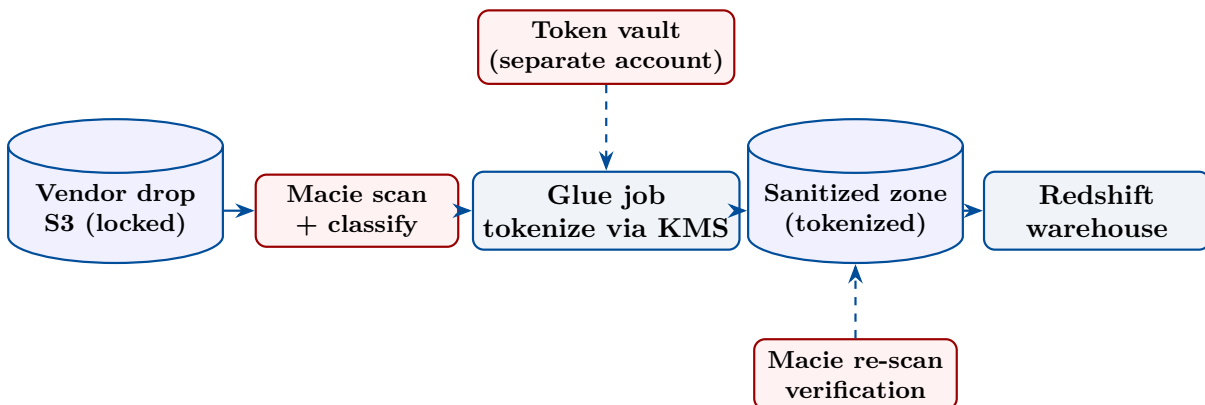


Figure 23.1: Third-party PII onboarding: discovery, KMS-anchored tokenization, a sanitized zone, warehouse load, and verification by re-scan.

23.6.2 Design Decisions

Discovery before processing. Every drop is Macie-scanned on arrival: the scan both classifies the file (which identifier types are present—driving which columns get tokenized) and catches schema surprises (a vendor silently adding a national-ID column produces a new finding type, not a silent warehouse column).

Tokenization with KMS anchoring. The Glue job tokenizes PII columns through a vault in a separate account, with the vault’s keys in KMS and access via a tightly scoped role. The token is deterministic per subject, so customer linkage works in the warehouse; the mapping never leaves the vault account. The choice of tokenization over hashing is deliberate: fraud investigations need authorized re-identification, which hashing forbids.

The raw drop’s lifecycle. The vendor prefix is Object-Locked, access-restricted, and short-retained: long enough to reprocess on pipeline failure, then expired by lifecycle rule. The sanitized zone is the system of record for analytics; the raw zone is a processing stage, not an asset.

Verification as a pipeline stage. After the load, a Macie re-scan of the sanitized zone must find zero PII before the batch is marked complete; a finding halts publication and pages. Trust, then verify, then scan.

Common Pitfall

Tokenizing the warehouse but not the pipeline. Staging files, Glue temporary directories, error outputs, and logs all held the raw values on the way through. The PII posture must cover the whole path: encrypted staging with short lifecycles, no PII in job logs (log field names, never values), and the re-scan covering intermediate prefixes too.

23.7 Interview Q&A

23.7.1 Concepts and Trade-offs

1. **Q: What does Macie detect beyond regex-matched patterns?**
 A: Managed identifiers use checksums (Luhn for cards), proximity keywords, and format validation; Macie also inventories bucket posture (public, unencrypted, shared) alongside content findings.
2. **Q: Why quarantine suspect files with Object Lock instead of deleting them?**
 A: Quarantine preserves evidence immutably for investigation and audit; deletion destroys the artifact the finding refers to and can itself violate retention obligations.
3. **Q: Tokenization versus hashing: which transformation is reversible?**
 A: Tokenization—via the controlled mapping vault. Hashing is one-way; and for low-entropy identifiers, hash dictionaries make it effectively reversible to attackers anyway, so choose tokenization with a real vault when re-identification is a requirement.
4. **Q: What do CloudTrail data events record for S3?**
 A: Object-level API activity—`GetObject`, `PutObject`, `DeleteObject`—per principal, per object: the access ledger for the lake, delivered to an independent log-archive account.
5. **Q: Why tune Macie custom data identifiers before rollout?**
 A: False-positive economics: an over-broad identifier floods findings, and teams trained to ignore alerts miss real PII. Tune on representative samples and review precision monthly.
6. **Q: What makes crypto-shredding superior to enumerated deletion for erasure?**
 A: One key deletion renders every copy—versions, replicas, snapshots, archives—unrecoverable in a single auditable act, instead of a long runbook that must find and delete every copy and then prove it did.

23.8 Further Reading

The Macie User Guide covers managed and custom identifiers, jobs, and finding structure [16]; the KMS guide covers per-subject key patterns for crypto-shredding [50]; CloudTrail and Config documentation cover the audit and posture layers [39, 41]. The Object Lock and replication references ground the erasure analysis [70, 73].

Key Takeaways

- Discovery precedes protection: Macie finds the PII your architecture forgot it had.
- Findings without a workflow are liability documentation; wire triage before the first scan.
- Masking hides, hashing deduplicates, tokenization links under control—choose by reversibility requirement.
- Erasure is architecture: enumerate every copy, or crypto-shred and let copies become ciphertext.
- Legal holds are first-class erasure exceptions; route them, never silently skip them.
- Verify the posture continuously: Config rules for configuration, CloudTrail data events for access, Macie re-scans for content.

23.9 Exercises

1. **(Conceptual)** Explain why a hash of an email address is not anonymization.
2. **(Conceptual)** Walk the erasure checklist for a subject whose data lives in a versioned bucket, a Redshift table with snapshots, and a Kinesis stream with 7-day retention.
3. **(Hands-on)** Run the Thursday project; add a custom data identifier for your synthetic employee-ID format and tune it to zero false positives on a control set.
4. **(Hands-on)** Implement per-subject KMS data keys on a small dataset and demonstrate crypto-shredding one subject.
5. **(Hands-on)** Write the CloudTrail Lake query reconstructing all access to a quarantined object.
6. **(Design)** Design the exception queue for erasure requests blocked by legal holds: states, owners, SLAs, and audit records.
7. **(Design)** Choose masking, tokenization, or hashing for: analyst dashboards, cross-system join keys, fraud investigation, and marketing audiences. Justify each.
8. **(Architecture)** Extend the Friday architecture for weekly deltas from the same vendor: what changes in discovery, tokenization, and verification?

23.10 Capstone Project: An Auditable PII Onboarding and Erasure Service

Capstone Project

Mission. Productize the Friday pipeline: third-party datasets are discovered, classified, tokenized, and warehoused—with a working erasure service that handles a deletion request end to end, including the legal-hold exception and the verification re-scan.

Estimated time. 8–10 hours in sandbox accounts.

What you'll practice.

- Macie-driven classification as a pipeline stage, not a report.
- Vault-backed tokenization with KMS anchoring and separate-account custody.
- Erasure runbooks (enumerated) and crypto-shredding (key-based), compared in practice.
- Continuous verification: Config, CloudTrail data events, and re-scans.

Scenario

The retailer's privacy office requires evidence, not assurances: the onboarding service must pass a quarterly audit in which auditors pick a subject and a batch and ask you to prove the subject's PII exists nowhere it should not—and, for one exercised request, nowhere at all.

Requirements

Functional requirements.

- Onboarding per fig. 23.1, with classification-driven tokenization mapping.
- An erasure runbook covering versions, snapshots, streams, and replicas, executed end to end on a test subject.

- The legal-hold exception path: held records queued, not deleted, with audit records.
- Post-erasure Macie re-scan evidence attached to the request record.

Non-functional requirements.

- The token vault lives in a separate account; re-identification is possible, logged, and alarming.
- Raw staging prefixes expire by lifecycle; pipeline logs contain no PII values.
- CloudTrail data events cover all PII prefixes; Config rules cover the posture controls.

Implementation Steps

1. Build the onboarding pipeline with the scan-classify-tokenize-load stages.
2. Implement the vault integration and the re-identification procedure with its alarm.
3. Author the erasure runbook; implement crypto-shredding for one domain as the comparison.
4. Run the full erasure drill including the held-record exception.
5. Produce the audit packet: findings, actions, re-scan evidence, access ledger extracts.

Deliverables

- Pipeline code, vault policies, and the classification-to-tokenization mapping.
- The erasure runbook and its drill evidence, including the exception queue.
- The crypto-shredding proof (key deletion, unreadable copies) and the comparison memo.
- The quarterly-audit packet, produced end to end.

Acceptance Criteria

- The auditor's sample subject is provably absent from every location post-erasure, with the re-scan attached.
- A held record survives the drill in the exception queue with its audit trail, not in a failed job.
- Re-identification is possible for an authorized role and alarms on use.
- No PII value appears in any pipeline log—verified by scan of the log groups.

Stretch Goals

- Extend crypto-shredding across all domains and retire the enumerated runbook.
- Automate the audit packet as a scheduled report.
- Add DSAR intake (EventBridge + Step Functions workflow) with SLA tracking.
- Build the data-minimization review: which collected fields earn their privacy cost?

Production DataOps and Certification Preparation

Learning Objectives

- Provision data infrastructure as code with Terraform: S3, Redshift Serverless, Glue jobs, and IAM roles, against a remote locked state backend.
- Build tested, documented dbt models on Redshift with sources, `ref()` dependencies, and environment targets.
- Implement CI/CD for data workloads: GitHub Actions PR checks, OIDC federation, and Dev→Prod promotion.
- Execute zero-downtime schema evolution with the expand → migrate → contract pattern.
- Review the AWS Certified Data Engineer – Associate domains, identify weak areas, and plan the final assessment.
- Deliver the Milestone 6 capstone: a secure, governed, multi-account data mesh.

Cross-Cloud Equivalence

DataOps—IaC, transformation-as-code, CI/CD—is the most portable week in the book: the tools are deliberately cloud-agnostic.

Capability	AWS	Azure	GCP
IaC (multi-cloud)	Terraform provider	AWS Terraform azurearm	Terraform google
IaC (native)	CloudFormation / CDK	Bicep / ARM	Deployment Manager
Transformation as code	dbt-redshift	dbt-fabric / dbt-synapse	dbt-bigquery
CI/CD	GitHub Actions / CodePipeline	GitHub Actions / Azure DevOps	GitHub Actions / Cloud Build
Remote state locking	S3 + DynamoDB	Azure Blob leases	GCS + state locking
CI authentication	OIDC to IAM role	OIDC to Entra workload identity	OIDC to Workload Identity
Zero-downtime DDL	Expand-contract + view shims	Same pattern	Same pattern

Snowflake (dbt-snowflake, Terraform provider) and **MongoDB** (atlas provider, schema-versioned migrations) plug into the same toolchain—this week’s skills transfer without translation.

24.1 Week 24 Roadmap and Mental Model

The final week turns everything into an operation. Twenty-three weeks of architecture earn their keep only if the platform is *reproducible* (infrastructure as code), *reviewable* (transformations as tested code), and *changeable without downtime* (CI/CD with expand-contract migrations). That is DataOps [91, 87, 89].

Then the week pivots to you: the certification review compresses the whole program into the AWS Certified Data Engineer – Associate exam domains, so you can see the platform you built the way the exam will ask about it [36]. The Milestone 6 capstone—a multi-account, governed, privacy-protected data mesh—is the program’s final integration and your portfolio’s centerpiece.

Definition

DataOps applies DevOps discipline to data platforms: infrastructure as code, transformations as tested and versioned code, continuous integration with automated quality gates, staged environment promotion, and zero-downtime change management.

Key Terms

IaC: infrastructure defined in declarative code and applied by tooling (Terraform, CloudFormation, CDK). **Remote state**: Terraform’s shared state store (S3 backend) with locking (DynamoDB). **dbt model**: a versioned SQL transformation; **source** and **ref()** declare the dependency graph. **dbt test**: an executable assertion on model output. **OIDC federation**: CI authenticating to AWS via short-lived tokens, no stored keys. **Expand-contract**: zero-downtime migration pattern—add new, dual-write/migrate, then remove old only after consumers verify.

24.2 Monday: Terraform for Data Infrastructure

24.2.1 Everything You Clicked, as Code

Every resource from this book—buckets and their lifecycle rules, the Serverless workgroup, Glue jobs, IAM roles, endpoints—exists as Terraform resources [91]. The discipline that makes IaC an asset rather than a liability: *remote state in S3 with DynamoDB locking* (so two engineers cannot apply divergent states and state survives a laptop), *modules* for repeated patterns (a governed bucket module bundling encryption, lifecycle, and logging), and *CI-only applies*: engineers run **plan** in pull requests; only the pipeline runs **apply**.

```

1 resource "aws_s3_bucket" "data_lake" {
2   bucket = "acme-data-lake-prod"
3   tags   = { Environment = "prod", Owner = "data-platform" }
4 }
5
6 resource "aws_s3_bucket_versioning" "data_lake" {
7   bucket = aws_s3_bucket.data_lake.id
8   versioning_configuration { status = "Enabled" }
9 }
10
11 resource "aws_s3_bucket_lifecycle_configuration" "data_lake" {
12   bucket = aws_s3_bucket.data_lake.id
13
14   rule {
15     id      = "tier-raw-to-glacier"
16     status  = "Enabled"
17     filter { prefix = "raw/" }
18   }

```

```

19   transition {
20     days          = 30
21     storage_class = "GLACIER"
22   }
23   noncurrent_version_expiration { noncurrent_days = 90 }
24 }
25 }

```

Listing 24.1: A governed data-lake bucket as Terraform.

Common Pitfall

Local state and laptop applies. Local `terraform.tfstate` is a single point of divergence, loss, and secret leakage (state contains attribute values). Remote state with locking, plus apply restricted to CI, is the difference between infrastructure as code and infrastructure as folklore.

24.3 Tuesday: dbt on Redshift

24.3.1 Transformations as a Tested DAG

dbt turns the warehouse’s transformation layer into software: *models* are SQL files materialized as tables/views, *sources* declare raw inputs with freshness checks, `ref()` builds the dependency graph, and *tests*—generic (unique, not-null, accepted values, relationships) or custom SQL—assert the output’s contract [87]. Documentation and lineage are generated from the same code, so the DAG visualization your reviewers read is the DAG that runs.

The operating pattern on Redshift: staging models over the raw schema (renames, type casts, nothing clever), intermediate models for reusable logic, marts for consumption—with tests at every boundary. dbt build runs models and tests in dependency order; a failing test stops downstream models, which is exactly the behavior Chapter 22’s gates taught you to want.

```

1 -- models/marts/fct_orders.sql
2 SELECT
3     order_id,
4     customer_id,
5     order_date,
6     amount
7 FROM {{ ref('stg_orders') }}
8 WHERE amount >= 0

```

Listing 24.2: A dbt model with tests, in the house style.

```

1 # models/marts/schema.yml
2 models:
3   - name: fct_orders
4     columns:
5       - name: order_id
6         tests: [unique, not_null]
7       - name: customer_id
8         tests:
9           - not_null
10          - relationships:
11              to: ref('dim_customers')
12              field: customer_id
13       - name: amount
14         tests:
15           - dbt_utils.accepted_range:
16               min_value: 0

```

Listing 24.3: Schema tests beside the model.

Tip

dbt tests validate what schemas cannot: uniqueness, referential integrity, value ranges, and freshness of sources. A table can be perfectly typed and completely wrong; the test layer is where “the pipeline ran green” becomes “the data is right.”

24.4 Wednesday: CI/CD and Zero-Downtime Schema Evolution

24.4.1 The Pipeline for Pipelines

The reference workflow: a pull request triggers GitHub Actions to run `terraform plan`, `dbt build` against a CI schema, and policy/lint checks; merge to `main` promotes through Dev to Prod with an approval environment [89]. Two controls deserve emphasis. **OIDC federation**: the workflow assumes an IAM role via GitHub’s OIDC token—no long-lived AWS keys in repository secrets. **Environment promotion**: dbt targets and Terraform workspaces separate Dev from Prod, so every change is rehearsed against a real environment before production [49].

24.4.2 Expand → Migrate → Contract

Renaming `order_amount` to `total_amount` in one release breaks every dashboard mid-deploy. The zero-downtime pattern phases it: **expand** (add the new nullable column; old column untouched), **migrate** (dual-write new writes, backfill history; reads still on the old column; a backward-compatible view shim for consumers), **contract** (switch reads to the new column, verify all consumers, and only then drop the old column in a later release). Each phase is its own dbt model change and its own deployment.

Common Pitfall

Shipping the contract phase with the migration. One team shipped the rename and the QuickSight dataset change in a single release: the migration applied minutes before the dashboard refresh, and executive dashboards errored for an hour because nothing pointed at the old column during the gap. Contract-phase drops go in a separate pull request, merged no sooner than one full deploy cycle after all consumers verify—with the compatibility view shim kept until then.

24.5 Thursday Hands-on Project: The Repository Is the Platform

Create a GitHub repository containing a Terraform module that provisions an S3 data-lake bucket and a Glue job, plus a dbt project with three models and five tests against Redshift, and a GitHub Actions workflow that runs dbt tests on every pull request and deploys on merge to `main`.

Lab Objectives

- Structure the repo: `terraform/` (modules + environments), `dbt/`, `.github/workflows/`.
- Configure the S3+DynamoDB remote backend and OIDC-to-IAM federation.
- Author three dbt models (staging, intermediate, mart) and five tests.
- Prove the gates: a failing test blocks merge; merge deploys to Dev; Prod needs approval.

Procedure

The workflow is the deliverable's heart:

```

1 name: data-platform
2 on:
3   pull_request:
4   push:
5     branches: [main]
6
7 permissions:
8   id-token: write    # OIDC federation
9   contents: read
10
11 jobs:
12   test:
13     runs-on: ubuntu-latest
14     steps:
15       - uses: actions/checkout@v4
16       - uses: aws-actions/configure-aws-credentials@v4
17       with:
18         role-to-assume: arn:aws:iam::123456789012:role/GitHubActionsDataRole
19         aws-region: us-east-1
20       - name: Terraform plan
21         run: terraform -chdir=terraform/environments/dev plan -input=false
22       - name: dbt build (CI schema)
23         run: |
24           cd dbt
25           dbt deps
26           dbt build --target ci
27   deploy-dev:
28     needs: test
29     if: github.ref == 'refs/heads/main'
30     runs-on: ubuntu-latest
31     steps:
32       - run: terraform -chdir=terraform/environments/dev apply -auto-approve
33       - run: cd dbt && dbt build --target dev
34   deploy-prod:
35     needs: deploy-dev
36     runs-on: ubuntu-latest
37     environment: production    # required reviewers gate this job
38     steps:
39       - run: terraform -chdir=terraform/environments/prod apply -auto-approve
40       - run: cd dbt && dbt build --target prod

```

Listing 24.4: .github/workflows/data-platform.yml (abridged).

Verification

- A PR with a deliberately failing dbt test is red and unmergeable.
- No AWS keys exist in repository secrets; the OIDC role's trust policy restricts to this repo and branch set.
- Merge deploys to Dev automatically; the Prod job waits for named approval.
- `terraform plan` on a second engineer's machine against the remote backend shows no drift.

24.6 Friday: Certification Preparation—Data Engineer Associate

The AWS Certified Data Engineer – Associate exam is organized into domains that map almost one-to-one onto this book [36]:

Exam domain	Weight	This book’s chapters
Data Ingestion and Transformation	~34%	Ch. 1, 9–15 (S3, EMR, Glue, DMS, Kinesis, Firehose, Flink)
Data Store Management	~26%	Ch. 1–8 (S3, RDS/Aurora, DynamoDB, Redshift, lakehouse)
Data Operations and Support	~22%	Ch. 12, 16, 22, 24 (orchestration, quality, DataOps)
Data Security and Governance	~18%	Ch. 8, 21–23 (Lake Formation, IAM, Macie)

Table 24.1: Exam domains mapped to the book. Weights are indicative of the published guide.

24.6.1 The Review Protocol

1. **Self-assess per domain.** For each domain, re-read the chapter’s Key Takeaways and attempt the exercises cold. Mark each objective *confident*, *shaky*, or *blank*.
2. **Repair the blanks first.** A shaky area you have used (Kinesis shard math) recovers by re-reading; a blank area you never built (configure one Lake Formation grant by hand) recovers only by doing the lab.
3. **Practice scenario questions.** The exam’s character is “which service/design for this constraint set”—the Friday use cases in this book are exactly that shape. Re-derive five of them from the constraint lists alone.
4. **Schedule two full-length mock exams** for the weekend after the program, spaced two days apart, reviewing every wrong answer to a root cause.

Tip

Exam heuristics that recur across domains: prefer managed/serverless when the stem emphasizes operational simplicity; prefer per-byte cost answers (Parquet, partitioning) when it emphasizes cost; choose the answer with least-privilege IAM and KMS when it emphasizes security; and when two answers both “work,” the differentiator is usually operational overhead or cost model, not features.

24.7 Interview Q&A

24.7.1 Concepts and Trade-offs

1. **Q: Why store Terraform state remotely with locking?**
A: Shared state is the source of truth for what exists; locking prevents two applies from diverging it. Local state diverges, gets lost with laptops, and leaks attribute values.
2. **Q: What do dbt tests validate that table schemas alone cannot?**
A: Semantics: uniqueness, referential integrity, accepted ranges, source freshness. A schema guarantees shape; tests guarantee the data is right.

3. **Q: Why promote dbt runs Dev→Prod instead of sharing one workspace?**

A: Every change is rehearsed against a real environment with real data shapes before production. One shared workspace means production is the rehearsal.

4. **Q: Why prefer OIDC federation over long-lived AWS keys in CI?**

A: OIDC issues short-lived, audience-scoped credentials per workflow run—nothing stored to leak, nothing to rotate. Stored keys in CI secrets are a standing breach waiting for a log line.

5. **Q: What must the PR-check workflow prove before merge to main?**

A: That the plan is reviewable (`terraform plan`), the transformations build and pass tests against CI data (`dbt build`), and policy checks pass—so `main` is always deployable.

6. **Q: Summarize expand-contract in one breath.**

A: Add the new schema alongside the old, dual-write and backfill while reads stay old, switch reads behind a compatibility shim, verify all consumers, and drop the old column in a later release.

24.8 Further Reading

The Terraform AWS provider registry documents every resource used in this book [91]; dbt’s documentation covers models, tests, sources, and targets [87]; GitHub Actions documentation covers OIDC and environments [89]. The exam guide lists the official domains and sample questions [36]. Revisit the Well-Architected Analytics Lens as a final review scaffold [3].

Key Takeaways

- The platform is the repository: IaC for infrastructure, dbt for transformations, CI for gates, CD for promotion.
- Remote locked state and CI-only applies turn Terraform from folklore into infrastructure.
- dbt tests are the warehouse’s contract layer; a green build means correct data, not just completed SQL.
- OIDC federation eliminates stored cloud credentials from CI entirely.
- Expand-contract makes schema change a non-event; the contract phase is a separate, later release.
- Certification review is gap analysis against domains—repair blanks by doing, not reading.

24.9 Exercises

1. **(Conceptual)** Explain why `terraform plan` output belongs in the PR while `apply` belongs in the pipeline.
2. **(Conceptual)** A teammate proposes running Dev and Prod dbt in one schema “to save time.” Enumerate the failure modes.
3. **(Hands-on)** Complete the Thursday project; then add a sixth custom test (a SQL assertion on daily row counts) and watch it gate a PR.
4. **(Hands-on)** Implement the expand-contract rename of one column across three releases; document each phase’s consumer evidence.

5. **(Hands-on)** Convert the OIDC role's trust to require a specific branch and environment; show a fork failing to assume it.
6. **(Design)** Design the dbt project structure (staging/intermediate/marts) for the Chapter 5 star schema with the Chapter 22 tests.
7. **(Design)** Write the exam-prep plan for a teammate whose blank domains are streaming and security: which chapters, which labs, in what order.
8. **(Architecture)** The platform must now support two teams deploying independently. Adapt the repo, state, and promotion design for monorepo versus polyrepo, and choose.

24.10 Milestone 6 Capstone: A Secure, Governed, Multi-Account Data Mesh

Capstone Project

Mission. Deliver the program's final integration: a three-account architecture under AWS Organizations—Account A ingests streaming PII and tokenizes it with Macie/Lambda, Account B hosts Redshift Serverless analytics, Account C governs via Lake Formation LF-Tags—with VPC endpoints locking all traffic off the public internet, all provisioned with Terraform and deployed by CI/CD.

Estimated time. 10–14 hours across sandbox accounts, plus review.

What you'll practice.

- Everything: ingestion (Part 4), warehousing (Part 2), governance (Weeks 8, 22), privacy (Week 23), perimeter (Week 21), and DataOps (this week).

Scenario

The conglomerate from Week 22's mesh commissions its first production slice. Streaming customer events arrive continuously; PII must be tokenized before any analytical persistence; analytics runs in its own account; governance is centralized; and the whole estate must redeploy from the repository after an imagined account loss.

Requirements

Functional requirements.

- AWS Organizations with the three accounts, OU structure, and the anti-exfiltration SCP attached.
- Account A: Kinesis ingestion, Macie classification, Lambda tokenization into a sanitized S3 zone.
- Account B: Redshift Serverless consuming the governed catalog; dashboards for analysts.
- Account C: Lake Formation LF-Tags, cross-account catalog sharing, CloudTrail aggregation.
- VPC endpoints (gateway + interface) in every data subnet; no internet path.

Non-functional requirements.

- Entire estate deploys from Terraform via the CI/CD pipeline (OIDC, Dev→Prod promotion).

- At least three automated policy/quality tests with failing-case demonstrations: PII column inaccessible cross-account, Macie quarantine trigger fires, DQ ruleset halts a bad feed.
- Alarms: CloudTrail data-event alarms and Macie findings to SNS.
- Monthly cost estimate (Macie volume, Redshift Serverless RPU, endpoint-hours) with two optimizations documented.
- Security review: SCPs attached, tag-based least privilege, no hardcoded secrets, tokenization verified by re-scan.

Reference Architecture

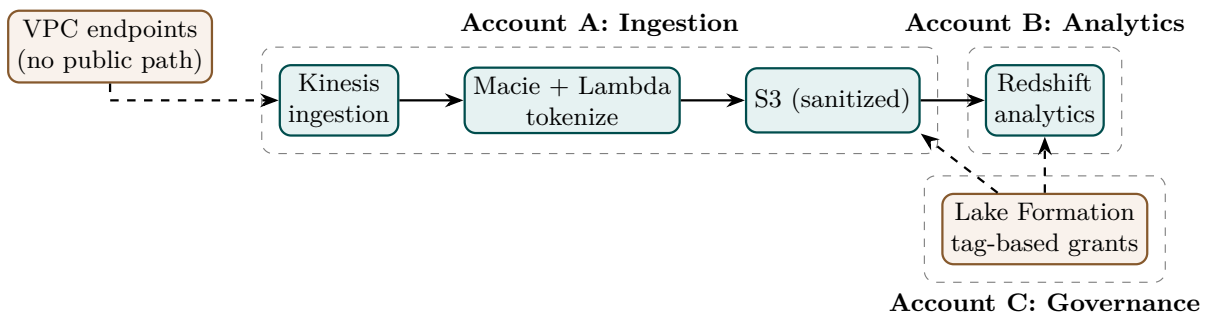


Figure 24.1: Milestone 6 reference architecture: the multi-account governed data mesh.

Implementation Steps

1. Terraform the organization: OUs, accounts, SCPs, and the per-account baselines.
2. Build Account A’s ingestion and tokenization path with the Macie classification stage.
3. Share the catalog via Lake Formation/RAM; define LF-Tags and expression grants in Account C.
4. Stand up Redshift Serverless in Account B against the shared catalog; build the analyst datasets.
5. Privatize networking end to end; verify no public path remains.
6. Implement the three tests, the alarms, the cost model, and the security review; run the account-loss redeploy drill.

Deliverables

- The repository: Terraform modules, environments, dbt models, CI workflows, README that rebuilds everything.
- The three policy/quality tests with failing-case evidence.
- Alarm evidence and the redeploy-drill record (time-to-restore measured).
- The cost model with two optimizations and the security review with the re-scan evidence.

Acceptance Criteria

- A clean-environment deploy reproduces the estate with no console-only step.

- PII is tokenized before analytical persistence; a cross-account PII read is denied and the denial is evidenced.
- The Macie quarantine trigger and the DQ gate both fire on injected faults.
- All data-plane traffic routes through endpoints; flow-log evidence included.
- The estate’s monthly cost is modeled and the two optimizations are measurable.

Stretch Goals

- Add a fourth consumer account onboarded with zero tickets via tag-expression grants.
- Implement crypto-shredding for the ingested subjects end to end.
- Add the QuickSight layer with RLS from Week 16 in Account B.
- Publish the estate’s lineage (OpenLineage) and scorecard (Week 22) to the governance account.

Closing Note

Twenty-four weeks ago the deliverable was a bucket with a lifecycle rule. Today it is a governed, private, self-testing, multi-account platform that deploys from a repository. That arc—storage, warehousing, batch, streaming, ML, governance, operations—is the AWS data engineering profession, and it is now yours to practice.

Cross-Cloud Service Equivalence

This appendix is a living reference that maps conceptually equivalent services and ideas across the seven platforms studied in this book series: Amazon Web Services (AWS), Microsoft Azure, Microsoft Fabric, Google Cloud Platform (GCP), MongoDB, Snowflake, and Databricks. The names differ, but the underlying data-engineering concepts—durable object storage, tiered life-cycle economics, immutability, managed relational engines, elastic compute, open table formats, and disciplined data organization—recur everywhere.

Note

This appendix grows with each chapter. Every new chapter contributes the equivalence tables introduced in its cross-cloud callout, so by the end of the book you will hold a single consolidated Rosetta Stone for moving fluently between clouds. Where a clean one-to-one mapping does not exist, the prose notes the closest analog and the caveat.

A.1 Object and Data-Lake Storage

The foundational layer covered in Chapter 1. Every platform offers durable, HTTP-addressable object storage with tiered pricing, lifecycle transitions, versioning, and write-once-read-many (WORM) controls.

Capability	AWS	Azure	GCP
Object storage	Amazon S3	ADLS Gen2 / Blob	Cloud Storage
Hot tier	S3 Standard	Hot	Standard
Infrequent access	S3 Standard-IA	Cool	Nearline
Archive (instant)	Glacier Instant Re-trieval	Cold	Coldline
Deep archive	Glacier Deep Archive	Archive	Archive
Lifecycle rules	Lifecycle policies	Lifecycle management	Object Lifecycle Mgmt
Versioning	S3 Versioning	Blob versioning	Object Versioning
Immutability (WORM)	Object Lock	Immutable blob storage	Bucket Lock
Cross-region copy	CRR / SRR	Object replication	Dual/multi-region + Transfer
Encryption at rest	SSE-S3 / SSE-KMS	SSE + CMK (Key Vault)	Google-managed / CMEK

The remaining platforms approach storage differently. **Microsoft Fabric** exposes a single logical lake, *OneLake*, and uses *shortcuts* to virtualize S3, ADLS Gen2, and Google Cloud Storage in place without copying. **Snowflake** abstracts storage entirely: table data lives in cloud object storage as immutable, columnar *micro-partitions* that the service manages on

your behalf. **MongoDB** persists documents through the *WiredTiger* storage engine, balancing an internal cache against the operating-system page cache rather than exposing storage tiers. **Databricks** sells no object storage at all: it layers Delta Lake tables and Unity Catalog *volumes* over the same S3, ADLS Gen2, or GCS buckets you already own, so the storage conversation is identical to the AWS one—plus a table format on top.

A.2 Managed Relational Databases

The focus of Chapter 2 for the three hyperscalers. Each provides a fully managed relational engine, a cloud-native high-performance variant, high availability, read replicas, and a migration service.

Capability	AWS	Azure	GCP
Managed relational	Amazon RDS	Azure SQL Database	Cloud SQL
Cloud-native engine	Amazon Aurora	SQL Hyperscale	AlloyDB
Managed PostgreSQL	RDS / Aurora PostgreSQL	Azure DB for PostgreSQL	Cloud SQL / AlloyDB
High availability	Multi-AZ	Zone-redundant HA	Regional HA
Read scaling	Read Replicas	Read replicas / geo-replicas	Read Replicas
Global / geo	Aurora Global Database	Active geo-replication	Cross-region replicas
Serverless compute	Aurora Serverless v2	Azure SQL Serverless	AlloyDB (elastic)
Migration service	DMS + SCT	Azure Database Migration Svc	Database Migration Svc

Snowflake, **Fabric**, **Databricks**, and **MongoDB** are not managed relational engines, but they solve overlapping problems: Snowflake and Fabric provide SQL analytics over warehouse/lakehouse storage, Databricks consumes OLTP sources through Lakeflow Connect, JDBC federation, and CDC landing into Delta tables, while MongoDB provides a distributed document store whose schema design (Chapter 2) replaces rigid relational normalization with intentional, access-driven modeling.

A.3 Elastic Compute and Analytical Warehouses

Snowflake’s Chapter 2 topic—separating compute from storage and scaling it elastically—has a counterpart in every analytics platform.

Concept	Snowflake	AWS	Azure / GCP
Compute unit	Virtual Warehouse	Redshift RA3 / Serverless	Synapse pool / Big-Query slots
Elastic concurrency	Multi-cluster warehouse	Concurrency scaling	Autoscale / on-demand
Pause when idle	Auto-suspend / resume	Pause / resume	Auto-pause / serverless
Billing unit	Credits	RPU / node-hours	DWU / slot-hours
Result reuse	Result cache	Result cache	Result / BI cache

Microsoft Fabric meters all workloads against a shared pool of Capacity Units (CU) on an F-SKU, an approach closer to a single elastic budget than to per-engine warehouses. **Databricks** meters compute in Databricks Units (DBUs) layered over the underlying cloud VM cost: SQL warehouses (serverless, pro, or classic) play the virtual-warehouse role with auto-stop and multi-cluster load balancing, while job clusters give per-run isolation. **MongoDB Atlas** scales through cluster tiers with optional auto-scaling of compute and storage.

A.4 Data Organization and the Medallion Pattern

Fabric’s Chapter 2 topic—the Bronze/Silver/Gold medallion—is a portable design pattern rather than a product. Every platform implements the same progressive-refinement idea with its own primitives.

Layer	Fabric	Data (AWS/Azure/GCP)	lake	Snowflake
Bronze (raw)	Bronze Lakehouse	raw/ (S3/ADLS/GCS)	prefix	RAW schema / stage
Silver (validated)	Silver Lakehouse	validated/	prefix	STAGING schema
Gold (curated)	Gold Lakehouse	curated/	prefix	ANALYTICS schema
Storage format	Delta-Parquet	Parquet / Delta / Ice- berg		Micro-partitions

Databricks is the medallion pattern’s native home: **bronze**, **silver**, and **gold** schemas of Delta tables governed by Unity Catalog, with expectations and quarantine as the quality gates between layers. **MongoDB** realizes the same separation with distinct collections or databases (for example **raw_events**, **validated**, **curated**) governed by schema-design patterns rather than a lake layout.

A.5 Document Data Modeling

MongoDB’s Chapter 2 schema-design patterns generalize to every document and wide-column store.

Concept	MongoDB	Amazon namoDB	Dy-	Azure Cosmos DB
Primary access unit	Document / collection	Item / table		Item / container
Partitioning	Shard key	Partition key		Partition key
One-table modeling	Embedding + patterns	Single-table design		Embedding + parti- tioning
Time-series grouping	Bucket Pattern	Composite sort keys		Bucketing by key
Optional attributes	Attribute Pattern	Sparse attributes		Sparse properties

Google Cloud’s closest analogs are **Firestore** and **Bigtable**. In **Snowflake** and **Fabric**, semi-structured documents are stored in **VARIANT** (Snowflake) or JSON columns and queried with path expressions rather than modeled as collections. **Databricks** likewise has no document store; it ingests MongoDB and Cosmos DB through Spark connectors and stores documents as Delta rows with **VARIANT**-style handling.

A.6 Globally Distributed and NoSQL Databases

Chapter 3 branches by platform into high-scale NoSQL (AWS DynamoDB, Azure Cosmos DB), globally consistent NewSQL (GCP Cloud Spanner), and advanced document modeling (MongoDB). They all answer the same question: how to scale a database horizontally across partitions and regions.

Concept	DynamoDB	Cosmos DB	Cloud Spanner	MongoDB
Model	Key-value / doc	Multi-model	Distributed SQL	Document
Partitioning	Partition key	Partition key	Split by key range	Shard key
Throughput unit	RCU / WCU	Request Units (RU)	Nodes / processing units	Cluster tier
Secondary indexes	GSI / LSI	Automatic indexing	Secondary indexes	Indexes
Change stream	DynamoDB Streams	Change Feed	Change streams (manual)	Change streams
Expiry	TTL	TTL		TTL index
Multi-region	Global Tables	Multi-region writes	Multi-region (synchronous)	Zones / global clusters
Consistency	Eventual / strong	5 tunable levels	External (True-Time)	Tunable read/write concern

Snowflake and **Fabric** are analytical platforms rather than OLTP key-value stores; they ingest from these operational databases and store semi-structured payloads in VARIANT/JSON columns. Google’s dedicated NoSQL analogs are **Firestore** (document) and **Bigtable** (wide-column). **Databricks** is likewise analytical-only: it consumes DynamoDB, Cosmos DB, and Bigtable through connectors into Delta rather than offering a request-serving store of its own.

A.7 Managed Apache Spark

Microsoft Fabric’s Chapter 3 topic—managed Spark pools and notebooks—has a first-class equivalent on every cloud.

Capability	Fabric	AWS	Azure / GCP
Managed Spark	Starter / Custom pools	Glue / EMR / EMR Serverless	Synapse Spark / Databricks; Dataproc
Notebook UX	Fabric Notebooks	Glue Studio / EMR Studio	Synapse / Databricks; Vertex AI
Serverless option	Autoscaling pools	Glue / EMR Serverless	Synapse serverless; Dataproc Serverless
Lake format	Delta on OneLake	Delta / Iceberg / Hudi	Delta; Delta / Iceberg

Databricks is the reference implementation this table generalizes: the Databricks Runtime (with the Photon vectorized engine) over autoscaling or serverless clusters, with Delta as the default table format. **Snowflake** offers **Snowpark** for DataFrame-style processing, and **MongoDB** integrates through the Spark Connector.

A.8 Query Acceleration and Materialized Views

Snowflake’s Chapter 3 acceleration tools—materialized views, the Search Optimization Service, and transient/temporary tables—map to comparable features in every warehouse.

Feature	Snowflake	AWS Redshift	Azure Synapse / GCP BigQuery
Materialized views	Materialized Views	Materialized views	Materialized views
Point-lookup accel.	Search Optimization Svc	Sort keys / zone maps	(indexing) / search indexes
Result reuse	Result cache	Result cache	Result-set cache / BI Engine
Ephemeral tables	Transient / temporary	Temp tables	Temp tables

Fabric accelerates reads through Direct Lake and its Warehouse engine, **Databricks** through materialized views, Photon, file-level statistics with bloom filters, and predictive optimization

over Delta tables, while **MongoDB** relies on secondary and compound indexes plus the in-memory working set.

A.9 In-Memory Caching and Hybrid Ingestion

The AWS and Azure Chapter 4 topic—low-latency caching in front of a durable store, fed by hybrid (edge-to-cloud) ingestion—has a managed Redis-compatible equivalent on every cloud.

Capability	AWS	Azure	GCP / others
Managed cache	ElastiCache (Redis/Memcached)	Cache for Redis	Memorystore (Redis/Memcached)
Real-time API layer	AppSync / API Gateway	API Management / SignalR	API Gateway / Apigee
Hybrid ingestion	DataSync / Snow family / IoT Core	Data Box / IoT Hub / Arc	Transfer Appliance / IoT Core
Streaming intake	Kinesis / MSK	Event Hubs	Pub/Sub

Snowflake and **Fabric** rely on result and Direct Lake caches rather than a standalone Redis tier, **Databricks** relies on its warehouse result cache and the cluster-local disk cache rather than a general-purpose cache service, while **MongoDB** serves hot data from its in-memory working set and Atlas can pair with an external cache.

A.10 Wide-Column and Key-Value NoSQL

GCP’s Chapter 4 topic—Cloud Bigtable—is a wide-column, high-throughput, low-latency NoSQL store. Its peers span the managed and open-source ecosystems.

Characteristic	GCP Bigtable	AWS	Azure / open source
Data model	Wide-column	DynamoDB (key-value/doc)	Cosmos DB (multi-model)
HBase-compatible	HBase API	Keyspaces (Cassandra)	Managed Instance for Cassandra
Scaling unit	Nodes / clusters	On-demand / provisioned	RU/s (Cosmos)
Best fit	Time-series, IoT, ad-tech	Serverless key-value	Global multi-model

Snowflake and **Fabric** address these workloads analytically rather than as operational key-value stores; **Databricks** reads Cassandra- and HBase-family sources with Spark connectors for analytical offload; **MongoDB** covers the document end of the same NoSQL spectrum.

A.11 Data Protection: Cloning, Time Travel, and Migration

Snowflake’s Chapter 4 topic—zero-copy cloning, Time Travel, and Fail-safe—plus MongoDB’s relational-to-document migration map to data-protection and migration tooling across the clouds.

Capability	Snowflake	AWS / Azure	GCP / Fabric
Instant clone	Zero-copy clone	Aurora clone / SQL DB copy	BigQuery table clone; OneLake shortcut
Point-in-time recovery	Time Travel	PITR (RDS/SQL DB backups)	PITR; Delta time travel
Retention safety	Fail-safe (7 days)	Automated backups	Backups; Delta VACUUM window
Schema migration	—	DMS / SCT	Database Migration Service

MongoDB migrations use **Relational Migrator** plus the embed-vs-reference patterns from Chapter 4, **Delta Lake** exposes historical versions through **VERSION AS OF** time-travel queries, and **Databricks** rounds out the set with **RESTORE**, **VACUUM**, and zero-copy *shallow clones* of Delta tables—the closest analog to Snowflake’s zero-copy cloning outside the warehouse.

A.12 Account and Environment Setup

Chapter 0’s first-hour pattern—free offer, secured administrative identity, CLI, cost guardrails, verification—repeats on every platform, with different defaults for what a “free” environment means.

Setup step	AWS	Azure	GCP	Databricks
Free offer	Free Tier (12 mo + always free)	Free account (\$200/30 days)	Free trial (\$300/90 days)	14-day trial / Free Edition
Admin identity	Root (MFA, then never)	Global Admin (avoid)	Organization Owner (avoid)	Account admin (first user)
Workforce identity	IAM Identity Center	Microsoft Entra ID	Cloud Identity	SSO + SCIM from IdP
CLI	AWS CLI v2 (aws configure)	Azure CLI (az login)	gcloud (gcloud init)	databricks configure
Cost guardrail	AWS Budgets + alerts	Cost Management budgets	Budgets & alerts	Budget policies + serverless guardrails
Browser shell	CloudShell	Cloud Shell	Cloud Shell	Web terminal / notebooks

Fabric starts from a Microsoft 365/Power BI work account plus a trial capacity (no card, org tenant); **Snowflake** uses a 30-day/\$400 trial guarded by Resource Monitors; **MongoDB Atlas** offers a forever-free M0 cluster secured by database users and an IP allowlist.

A.13 Cloud Data Warehouses and MPP Architecture

Chapter 5 introduces the cloud data warehouse. The MPP mental model—shared-nothing compute over distributed columnar storage—transfers to every platform, even where the service hides it.

Capability	AWS	Azure	GCP	Databricks
Cloud data warehouse	Amazon Redshift	Synapse dedicated pools / Fabric Warehouse	BigQuery	Databricks SQL warehouses
Serverless option	Redshift Serverless	Fabric capacity / Synapse serverless	BigQuery (default)	Serverless SQL warehouses
Managed storage layer	RA3 managed storage	ADLS-backed separation	Capacitor on Colossus	Delta on your object storage
Bulk load	COPY	COPY INTO / bulk insert	LOAD DATA / bq load	COPY INTO / Auto Loader
Change-data upsert	MERGE	MERGE	MERGE	MERGE INTO
BI acceleration	Materialized views	Materialized views / semantic models	MVs, BI Engine	Materialized views, Photon
Pricing lever (provisioned)	Node-hours + storage TB	DWU-hours + storage	Flat-rate slots	DBU + VM cost (classic)
Pricing lever (serverless)	RPU-hours	Capacity units	Bytes scanned / slots	Serverless DBU-hours
Concurrency burst	Concurrency scaling	Workload groups	Autoscaling slots	Multi-cluster load balancing

Fabric blurs the warehouse/lake boundary: warehouse and lakehouse share OneLake Delta—

Parquet storage. **Snowflake** separates storage, virtual warehouses, and cloud services into three independently scaled layers—the closest conceptual cousin to RA3. **MongoDB** is an operational document store, not a warehouse; the contrast explains why star schemas live outside the system of record.

A.14 Warehouse Performance and Physical Design

Chapter 6’s physical-design levers—move less data, read less data, schedule work fairly—exist on every analytical engine; each platform exposes a different subset.

Capability	AWS	Azure	GCP	Databricks
Data distribution	DISTSTYLE EVEN/KEY/ALL	Synapse hash/round-robin/replicated	Automatic	Automatic (partitioning optional)
Sort optimization	SORTKEY (compound/interleaved)	Clustered column-store index	Clustering columns	Z-ORDER / liquid clustering
Pre-computation	Materialized views	Materialized views	Materialized views	Materialized views
Workload governance	WLM + QMRs	Workload groups / classifiers	Reservations & slots	SQL warehouse clusters + queries API
Burst concurrency	Concurrency scaling	Scale-out / serverless burst	Autoscaling slots	Multi-cluster autoscaling
Manual tuning burden	Medium (AUTO helps)	Medium-high	Low	Low (predictive optimization)
Skew diagnosis	SVV_TABLE_INFO	DMVs	Query plan explanation	Query profile, AQE skew split

Snowflake removes most physical levers—micro-partition pruning is automatic and clustering keys optional—trading control for operational simplicity. **Fabric** inherits SQL Server-style distribution choices on dedicated capacity. **MongoDB** offers indexes and shard keys rather than distribution styles, because its workload is point lookups, not scans.

A.15 Lakehouse Query Layer and Open Table Formats

Chapter 7’s lakehouse pattern—open files on object storage, a shared catalog, multiple query engines—exists on every cloud, with Databricks as its original author.

Capability	AWS	Azure	GCP	Databricks
Serverless SQL on the lake	Athena	Synapse/Fabric serverless SQL	BigQuery external tables	SQL warehouses over Delta
Shared metastore	Glue Data Catalog	Purview / OneLake catalog	Dataplex / BigLake	Unity Catalog
Schema inference	Glue crawlers	Synapse auto-discovery	BigQuery autodeTECT	Auto Loader inference + hints
Warehouse-to-lake query	Redshift Spectrum	Lakehouse shortcuts	BigLake tables	Native (same tables)
Table format	Iceberg / Hudi / Delta on S3	Delta (OneLake native)	BigLake Iceberg	Delta (UniForm for Iceberg/Hudi reads)
Pricing lever	Per TB scanned	Capacity units	Bytes scanned / slots	DBU-hours
Cost control	Partitioning + Parquet	Partitioning + Delta	Partitioning + clustering	Liquid clustering + OPTIMIZE

Fabric is the most opinionated implementation: OneLake plus shortcuts makes the warehouse and lake the same physical data. **Snowflake** reads external Iceberg tables while its native tables remain the managed counterpart. **MongoDB** plays the source-system role; Atlas Data Federation queries S3 but the document model stays operational.

A.16 Lake Governance, Sharing, and Clean Rooms

Chapter 8’s governance primitives—catalog, tag-based policy, fine-grained controls, clean-room collaboration—have converged across platforms.

Capability	AWS	Azure	GCP	Databricks
Lake governance layer	Lake Formation	Microsoft Purview	Dataplex	Unity Catalog
Tag-based access	LF-Tags	Purview classifications	Policy tags	UC tags + attribute-based policies
Row/column security	LF data filters + cell filters	SQL RLS/masking	BigQuery RLS + column ACLs	Row filters + column masks
Live cross-account share	Redshift data sharing	Fabric OneLake sharing	Analytics Hub	Delta Sharing (open protocol)
Managed clean room	AWS Clean Rooms	Azure confidential clean rooms	BigQuery data clean rooms	Databricks Clean Rooms
Enforcement point	Catalog (engine-agnostic)	Per engine + Purview	BigQuery policy tags	UC (all engines on platform)
Default stance	IAMAllowedPrincipals (revoke!)	Deny until granted	Deny until granted	Deny until granted

Snowflake implements the same primitives natively—row access policies, masking policies, tags, zero-copy shares/listings—with a clean-room offering parallel to AWS Clean Rooms. **MongoDB** enforces field-level security at the document layer (client-side field-level encryption). Delta Sharing’s distinction is the *open* protocol: recipients read with pandas, Spark, or Power BI without any Databricks account.

A.17 Managed Hadoop and Spark Clusters

Chapter 9’s deployment-mode spectrum—full-control clusters to fully serverless Spark—is the portable idea; every cloud occupies several points on it.

Capability	AWS	Azure	GCP	Databricks
Managed Hadoop/Spark	EMR on EC2	HDInsight	Dataproc	All-purpose clusters (no YARN)
Serverless Spark	EMR Serverless	Fabric / Synapse Spark	Dataproc Serverless	Serverless compute
Spark on Kubernetes	EMR on EKS	AKS-hosted Spark	Dataproc on GKE	(control plane managed)
Object-storage filesystem	EMRFS (S3)	ABFS (ADLS)	GCS connector	DBFS mounts / UC volumes
Cheap interruptible nodes	Spot task nodes	Spot node pools	Preemptible workers	Spot with fallback policy
Cluster lifetime model	Transient vs. long-running	Both	Both	All-purpose vs. job clusters
Notebook integration	EMR Studio	Fabric notebooks	Vertex AI / Jupyter	Databricks notebooks (native)

Fabric folds Spark into its SaaS lakehouse—pools and notebooks, never node types. **Databricks** popularized the transient, job-scoped cluster pattern this chapter teaches; cluster policies replace EMR’s instance-fleet discipline. **Snowflake**’s Snowpark pushes DataFrame code into the warehouse instead of a cluster. **MongoDB** stays the operational source Spark jobs read via the connector.

A.18 Serverless, Catalog-Integrated ETL

Chapter 10’s serverless ETL pattern—schema-aware transforms over a shared catalog, billed per run—is universal; only the branding differs.

Capability	AWS	Azure	GCP	Databricks
Serverless ETL engine	AWS Glue (Spark)	Fabric Dataflow Gen2 / pipelines	Dataflow / Data Fusion	Lakeflow Declarative Pipelines (DLT)
Visual designer	Glue Studio	Data Factory designer	Data Fusion studio	DLT graph editor / notebooks
Interactive data prep	Glue DataBrew	Power Query (Fabric)	Dataprep	Databricks notebooks + Assistant
Shared catalog	Glue Data Catalog	OneLake catalog / Purview	Dataplex	Unity Catalog
Re-run state	Job bookmarks	Pipeline state / watermarks	Job state / water-marking	Checkpoints + pipeline state
Capacity unit	DPU-hour	Capacity units	vCPU-hr	DBU-hour (serverless or classic)
Spot discount	Flex execution	Low-priority pools	Preemptible/Spot workers	Spot instances with fallback

Snowflake approaches the same problems from inside the warehouse: streams, tasks, and Snowpark replace external ETL for in-warehouse transforms. **MongoDB** users meet this layer as the Atlas Data API or Spark connector feeding the lake. DLT’s distinguishing move is *declarative* pipelines with expectations (warn/drop/fail) as first-class quality gates.

A.19 Change Data Capture, Event Routing, and Schema Contracts

Chapter 11’s disciplines—CDC replication, event routing, and schema contracts—are cloud-agnostic; each platform offers a managed shape of the same ideas.

Capability	AWS	Azure	GCP	Databricks
Managed CDC replication	AWS DMS	Azure DMS / Data Factory CDC	Datastream	Lakeflow Connect (managed connectors)
Event router	EventBridge	Event Grid	Eventarc	(cloud-native router triggers jobs)
Serverless functions	Lambda	Azure Functions	Cloud Run functions	(SQL warehouses + jobs; cloud functions alongside)
Schema registry	Glue Schema Registry	Azure Schema Registry	Managed Service for Schema Registry	Schema inference + UC schema enforcement
Contract enforcement	In Glue jobs / registry checks	In pipelines	In Dataflow / registry checks	Delta constraints + DLT expectations
CDC lag metric	CDCLatencySource/Target	Replication latency	Datastream freshness	Pipeline event log freshness
CDC apply	Glue/EMR MERGE	T-SQL MERGE	BigQuery MERGE	AUTO CDC / MERGE INTO

Snowflake ingests CDC through Snowpipe and streams, applying contracts at the table boundary. **MongoDB** contributes change streams—the document-native CDC primitive that pairs with everything here. **Fabric** routes database mirroring and event streams through OneLake, its version of the CDC-to-lake path.

A.20 Orchestration: DAG Schedulers and State Machines

Chapter 12 splits orchestration into two families—general-purpose DAG schedulers and native state machines; both exist on every cloud.

Capability	AWS	Azure	GCP	Databricks
Managed Airflow	Amazon MWAA	Data Factory managed Airflow	Cloud Composer	(external Airflow via Jobs API)
Native state machine	Step Functions	Logic Apps / Durable Functions	Workflows	Workflows (jobs + tasks)
File-arrival trigger	EventBridge + sensor	Event Grid trigger	Eventarc trigger	File-arrival triggers
Failure alerting	SNS via operator/rule	Monitor alerts	Alerting to Pub/Sub	Job notifications + system tables
Inter-task data	XComs (small) / S3 (large)	Pipeline variables / ADLS	XComs / GCS	Task values / Delta tables
Partial recovery	Redrive / Airflow retries	Rerun from activity	Step retry blocks	Repair run (failed + downstream)

Fabric pipelines play the orchestrator role inside the SaaS boundary. **Snowflake** tasks and DAGs orchestrate in-warehouse work, and dbt orchestrates transformation graphs at the SQL layer. **MongoDB** triggers and Atlas schedulers cover the document-store corner. Workflows’ native advantage is colocation: clusters, catalog, and lineage in one pane.

A.21 Messaging and Streaming Primitives

Chapter 13’s three primitives—queues, pub/sub, append-only logs—ship on every cloud.

Capability	AWS	Azure	GCP	Databricks
Pull queue	Amazon SQS	Queue Storage / Service Bus	Pub/Sub (pull)	(none; consumes external queues)
Pub/sub fan-out	Amazon SNS	Event Grid / Service Bus topics	Pub/Sub	(none; consumes external buses)
Durable streaming log	Kinesis Streams	Data Event Hubs	Pub/Sub (ordered)	Kafka sources into Structured Streaming
Managed Kafka	Amazon MSK	Event Hubs for Kafka	Managed Service for Apache Kafka	(Consumes MSK/Confluent/Event Hubs)
Serverless stream processor	Managed Flink / Lambda	Stream Analytics	Dataflow	Structured Streaming (serverless option)
Scaling unit	Shard (1 MB/s in, 2 MB/s out)	Throughput units / partitions	Throughput quotas	Kafka partitions / trigger rate
Ordering scope	Per partition key	Per partition key	Per ordering key	Per Kafka partition
Retention	24 h–365 days	1–90 days	Up to 31 days	Source-side (Kafka/Kinesis retention)

Fabric Real-Time Intelligence wraps KQL databases and Eventstreams around the same log concept. **Snowflake** consumes streams via Snowpipe Streaming. **MongoDB** change streams are the database-native analog—replayable, resumable. Databricks deliberately ships no bus: it is the consumer/processor of record for all of them.

A.22 Managed Stream-to-Sink Delivery

Chapter 14’s buffer–transform–convert–land delivery pattern exists everywhere, usually as a feature of the streaming service rather than a separate product.

Capability	AWS	Azure	GCP	Databricks
Managed stream delivery	Data Firehose	Event Hubs Capture	Pub/Sub export / Dataflow templates	Auto Loader + Structured Streaming
Inline transformation	Lambda in Firehose	Functions pattern	Dataflow / Cloud Run functions	Streaming DataFrame transforms
In-flight format conversion	JSON → Parquet/ORC	Via downstream Synapse/Fabric	Via Dataflow	Native Delta writes
Warehouse delivery	Firehose → Redshift	Event Hubs → Synapse	Pub/Sub → BigQuery	Streaming → Delta tables
Search delivery	Firehose → OpenSearch	Event Hubs → Elastic	Pub/Sub → Elasticsearch	Sink connectors / foreachBatch
Buffering model	Size + interval hints	Time/size capture windows	Subscription batching	Micro-batch triggers
Failure handling	S3 backup + error prefixes	Dead-lettering	Dead-letter topics	<code>_rescued_data</code> + checkpoint replay

Snowflake Snowpipe Streaming is the same idea aimed at one destination: the warehouse table. **Fabric** Eventstreams route to lakehouse tables with optional transformation. **MongoDB** sits on the receiving end via sink connectors or trigger-based loads. On Databricks the “delivery service” is just a streaming job—more control, more to operate.

A.23 Stateful Stream Processing

Chapter 15 is the deepest discipline in the book: event time, watermarks, windows, state, and checkpoints are identical everywhere; only the managed packaging differs.

Capability	AWS	Azure	GCP	Databricks
Engine	Managed Service for Apache Flink	HDInsight/AKS-hosted Flink	Dataflow (Beam model)	Structured Streaming (Spark)
Execution model	True streaming	True streaming	Unified batch+stream	Micro-batch (continuous optional)
Native stream SQL	Flink SQL	Stream Analytics query language	Dataflow SQL / Beam	Spark SQL over streams
Event-time windows	TUMBLE/HOP/SESSION	ASA windowing	Beam windows	<code>window()</code> , session windows
Lateness control	Watermarks + allowed lateness	ASA late-arrival policy	Beam watermarks	<code>withWatermark</code>
Custom state	Keyed process functions	Per platform	Beam stateful DoFns	<code>flatMapGroupsWithState</code>
State backend	RocksDB (managed)	Per platform	Per runner	HDFS-compatible / RocksDB store
Exactly-once sink	Two-phase commit connectors	Idempotent sinks	Exactly-once in Dataflow	Checkpoint + idempotent Delta sink

Fabric Real-Time Intelligence covers many windowed cases with KQL, trading control for simplicity. **Snowflake** approaches streaming with Snowpipe Streaming plus dynamic tables. **MongoDB** stream processing (Atlas) applies the same windowed ideas to change streams. Learn Flink’s model once; everywhere else is a dialect—Spark’s dialect is the micro-batch trigger and the checkpoint directory.

A.24 Real-Time Serving, BI, and Reverse ETL

Chapter 16’s serving layer—dashboards for humans, operational pushes back to business systems—closes the analytics loop on every platform.

Capability	AWS	Azure	GCP	Databricks
Managed BI	Amazon QuickSight	Power BI (Fabric)	Looker / Looker Studio	AI/BI dashboards
In-memory acceleration	SPICE	Import / semantic models	BI Engine	SQL warehouse result cache
NL analytics	Amazon Q in QuickSight	Copilot (Power BI)	Gemini in Looker	Genie spaces
Row-level security	QuickSight RLS	Power BI RLS	Looker access filters	UC row filters (inherited)
Warehouse export	Redshift UNLOAD	COPY INTO / export	BigQuery EXPORT DATA	COPY INTO / Delta Sharing
Reverse-ETL glue	Lambda + API calls	Logic Apps / Functions	Cloud Run functions	Jobs + webhooks / partner tools
Reader pricing	Per-reader session	Per-user / capacity	Per-user	Included in platform

Snowflake hosts dashboards in Snowsight and streams reverse ETL through external functions; the ecosystem (Census, Hightouch) serves all platforms. **MongoDB Charts** covers operational BI directly on collections. AI/BI’s differentiator: tiles read governed Delta tables in place and inherit Unity Catalog lineage back to the producing pipeline.

A.25 In-Warehouse Machine Learning

Chapter 17’s pattern—bringing the model to the data instead of shipping data to a modeling platform—is now offered by every warehouse.

Capability	AWS	Azure	GCP	Databricks
SQL-based ML	Redshift ML (CREATE MODEL)	Fabric SynapseML / T-SQL PREDICT	BigQuery (CREATE MODEL)	SQL functions over served models (<code>ai_query</code>)
AutoML behind the scenes	SageMaker Autopilot	Azure ML AutoML	Vertex AI AutoML	AutoML (glass-box notebooks)
In-warehouse scoring	SQL function per model	T-SQL PREDICT	ML.PREDICT	Batch scoring UDFs from registry
Custom code path	BYO model via SageMaker	Synapse Spark MLlib	Remote models / Vertex	Notebooks + MLflow (native)
Clustering	K-Means problem type	MLlib KMeans	ML.KMEANS	MLlib / scikit-learn on Spark
Training data leaves warehouse?	Yes (to SageMaker, in-account)	Varies	No (BigQuery-native)	No (Delta tables stay governed)
Cost driver	SageMaker training + S3 staging	Capacity units	Bytes processed + training	Cluster DBUs

Snowflake ML offers `CREATE MODEL` and Snowpark ML with warehouse-side scoring—the closest analog. **MongoDB** has no in-database ML; Atlas Vector Search addresses the adjacent similarity problem. Databricks inverts the direction: the lakehouse *is* the modeling platform, so “in-warehouse ML” is the default rather than a feature.

A.26 Managed ML Platforms

Chapter 18’s managed-ML shape—a studio, managed training, a registry, deployment targets—maps name-for-name across platforms.

Capability	AWS	Azure	GCP	Databricks
ML studio	SageMaker Studio	Azure ML studio	Vertex AI Workbench	Databricks workspace + notebooks
Interactive data prep	Data Wrangler	Fabric Data Wrangler / Power Query	BigQuery DataFrames / Colab	Notebooks + Assistant
AutoML	SageMaker Autopilot	Azure ML AutoML	Vertex AI AutoML	Databricks AutoML
Managed training	SageMaker Training Jobs	Azure ML jobs	Vertex AI Training	Jobs on ML runtimes
Real-time inference	Real-time endpoints	Managed online endpoints	Vertex AI endpoints	Model Serving (serverless)
Batch inference	Batch Transform	Batch endpoints	Batch prediction	Spark UDF from registry + jobs
Custom code	BYO container (any framework)	Custom environments	Custom containers	Custom images / libraries
GPU access	ml.g/ml.p instances	GPU compute clusters	GPU nodes/Vertex	GPU clusters (ML runtime)

Fabric data science runs in-notebook with SynapseML against OneLake. **Snowflake** ML covers training and serving inside the warehouse boundary. **MongoDB** feeds these platforms as a source; its vector search serves retrieval rather than training.

A.27 MLOps: Feature Stores, Registries, and Monitoring

Chapter 19’s MLOps primitives—feature stores, pipeline orchestration, registries, drift monitoring—are platform-standard.

Capability	AWS	Azure	GCP	Databricks
Feature store	SageMaker Feature Store	Fabric feature store / Feathr	Vertex AI Feature Store	Feature Engineering in UC
ML pipelines	SageMaker Pipelines	Azure ML pipelines	Vertex AI Pipelines (KFP)	Workflows + DLT / MLflow Projects
Model registry	SageMaker Model Registry	Azure ML registry	Vertex AI Model Registry	Unity Catalog model registry
Drift monitoring	SageMaker Model Monitor	Azure ML monitoring	Vertex AI Model Monitoring	Lakehouse Monitoring
Retrain trigger	EventBridge + pipeline	Logic Apps + pipeline	Eventarc + pipeline	Job triggers + webhooks
Online store backing	DynamoDB-backed records	Cosmos-backed	Bigtable-backed	Feature serving endpoints
Lineage	SageMaker Lineage + OpenLineage	MLflow tracking	Vertex ML Meta-data	Automatic UC lineage (data → model)

Snowflake’s Feature Store and Model Registry bring the same lifecycle inside the warehouse. **MongoDB** typically appears as the online-serving store behind low-latency feature lookups in self-managed stacks. **MLflow**—born at Databricks—is the open-source reference most of these registries interoperate with.

A.28 Pre-trained AI Services and Unstructured Data

Chapter 20’s commodity building blocks—vision, OCR, NLP, translation—differ only in packaging; the skill is pipeline integration, not model building.

Capability	AWS	Azure	GCP	Databricks
OCR / document parsing	Textract	Document Intelligence	Document AI	<code>ai_parse_document</code> / partner parsers
Image/video labeling	Rekognition	Azure AI Vision	Vision AI / Video AI	Foundation Model APIs (vision models)
NLP (entities, sentiment, PII)	Comprehend	Azure AI Language	Natural Language AI	<code>ai_query</code> with LLM endpoints
Translation	Translate	Translator	Translation API	<code>ai_query</code> / external models
Foundation models	Amazon Bedrock	Azure OpenAI	Vertex AI	Foundation Model APIs (pay-per-token)
Vector index / search	OpenSearch / Kendra	AI Search	Vertex AI Vector Search	Mosaic AI Vector Search
PII detection	Comprehend PII + Macie	Language PII + Purview	Sensitive Data Protection	UC discovery + <code>ai_query</code> redaction
RAG orchestration	Bedrock Knowledge Bases	Azure AI RAG pattern	Vertex RAG Engine	LangChain + Vector Search + MLflow

Fabric exposes Azure AI services natively in notebooks and pipelines. **Snowflake** offers Document AI and Cortex functions—extraction as SQL. **MongoDB** stores the extracted JSON naturally; Atlas Vector Search serves the retrieval side of the same estate.

A.29 Identity, Networking, and Key Management

Chapter 21’s security substrate—identity, private networking, key management—is identical in concept on every cloud; only the acronyms differ.

Capability	AWS	Azure	GCP	Databricks
Identity and access	IAM roles/policies	Entra ID + Azure RBAC	Cloud IAM	UC grants + SSO/SCIM
Org-wide guardrails	SCPs (Organizations)	Azure Policy	Organization Policy	Account-level policies + workspaces
Permissions ceiling	Permissions boundaries	Management groups + deny	IAM deny policies	Cluster policies + UC securables
Private service access	VPC endpoints / PrivateLink	Private Endpoints / Private Link	Private Service Connect	Private connectivity (control + data plane)
Key management	AWS KMS	Azure Key Vault	Cloud KMS	Cloud KMS (CMK on workspace storage)
Secrets management	Secrets Manager	Key Vault secrets	Secret Manager	Secret scopes (DB- or vault-backed)
Free S3-class endpoint	Gateway endpoint	Service endpoints	Private Google Access	(via cloud provider endpoint)
Anti-exfiltration primitive	Endpoint policy + SCP + org ID	Private Link + policy	VPC Service Controls	Egress controls + storage credentials

Snowflake adds account-level network policies and external-private-endpoint rules; **MongoDB** Atlas uses private endpoints and IP access lists; **Fabric** centralizes on Entra conditional access and tenant-level perimeter settings. The pattern—identity plus network plus keys—is universal.

A.30 Data Quality, Lineage, and Mesh Governance

Chapter 22’s governance layer—quality rules, lineage, mesh organization—converged faster here than anywhere else in the stack.

Capability	AWS	Azure	GCP	Databricks
Quality rules engine	Glue Data Quality (DQDL)	Fabric/Purview data quality	Dataplex data quality	DLT expectations (warn/drop/fail)
Cross-account sharing	Lake Formation + RAM	Purview + Fabric sharing	Analytics Hub / Dataplex	Delta Sharing
Tag-based governance	LF-Tags	Purview classifications	Policy tags	UC tags + governed tags
Lineage	SageMaker Lineage, OpenLineage	Purview lineage	Dataplex lineage	Automatic UC table/column lineage
Catalog of record	Glue Data Catalog	Microsoft Purview	Dataplex	Unity Catalog
Open-source alternative	Great Expectations, DataHub	Same (cloud-agnostic)	Same	Same (+ MLflow for models)
Quality gate pattern	In-job evaluation + SNS	Pipeline gates	Dataflow/Dataplex gates	Expectation fail → pipeline halt

Snowflake implements quality as data metric functions and governance as the Horizon Catalog. **MongoDB** enforces shape at write time with schema validation—the closest operational-store analog to a DQ ruleset. **Fabric** ties quality and lineage into OneLake’s unified governance story.

A.31 Sensitive-Data Discovery and Erasure

Chapter 23’s regulatory constants—discovery, de-identification, erasure—get the same architectural answers on every cloud.

Capability	AWS	Azure	GCP	Databricks
PII discovery at rest	Amazon Macie	Microsoft Purview scanning	Sensitive Data Protection (DLP)	UC discovery (classification)
Custom identifiers	Macie custom data identifiers	Custom classifiers	Custom infoTypes	Custom classifiers / patterns
Audit trail	CloudTrail (incl. data events)	Azure Monitor / audit logs	Cloud Audit Logs	<code>system.access.audit</code>
Config posture	AWS Config rules	Azure Policy	Security Command Center	Account compliance + policies
Crypto-shredding	Per-subject keys, then delete key	Per-subject KEKs in Key Vault	Per-subject keys in Cloud KMS	Per-subject keys via cloud KMS
Quarantine pattern	Object Lock bucket + Lambda	Immutability + Functions	Bucket lock + functions	Quarantine schema + UC grants
Erasure across copies	Enumerate zones/snapshots/streamsa	Same problem, same pattern	Same problem, same pattern	Delete + VACUUM within retention window

Snowflake offers object-tagging-driven masking and supports erasure via row deletion within retention windows—with Time Travel/Fail-safe explicitly in scope. **MongoDB** supports client-side field-level encryption, which makes per-subject crypto-shredding natural. **Fabric** centralizes discovery in Purview over OneLake. On Databricks, erasure must account for Delta history: the row delete and the **VACUUM** that destroys old versions are one operation from the auditor’s perspective.

A.32 DataOps: IaC, Transformation-as-Code, and CI/CD

Chapter 24 is the most portable week in the book: the tools are deliberately cloud-agnostic, so the table maps providers rather than products.

Capability	AWS	Azure	GCP	Databricks
IaC (multi-cloud)	Terraform provider	Terraform azurerm	Terraform google	Terraform databricks provider
IaC (native)	CloudFormation / CDK	Bicep / ARM	Deployment Manager	Asset Bundles (<code>databricks.yml</code>)
Transformation as code	dbt-redshift	dbt-fabric / dbt-synapse	dbt-bigquery	dbt-databricks
CI/CD	GitHub Actions / CodePipeline	GitHub Actions / Azure DevOps	GitHub Actions / Cloud Build	GitHub Actions + bundle deploy
Remote state locking	S3 + DynamoDB	Azure Blob leases	GCS + state locking	(cloud backend of choice)
CI authentication	OIDC to IAM role	OIDC to Entra workload identity	OIDC to Workload Identity	Service principal + OAuth M2M
Cost telemetry	CUR 2.0	Cost Management	Billing export	<code>system.billing.usage</code>
Zero-downtime DDL	Expand-contract + view shims	Same pattern	Same pattern	Same pattern (+ Delta evolution)

Snowflake (dbt-snowflake, Terraform provider) and **MongoDB** (Atlas provider, schema-versioned migrations) plug into the same toolchain—this week’s skills transfer without translation. The lesson that holds on all seven platforms: *the merge is the deploy, and the pipeline is the only writer to production.*

Tip

Use this appendix as a study aid for the book’s lockstep, cross-cloud approach: when you learn a concept on one platform, immediately locate its equivalent here and predict how the other six would express the same idea. That habit turns seven separate vocabularies into one mental model.

Environment Setup Reference

This appendix consolidates Chapter 0 into a single reference you can return to before any lab: account and identity, command-line tooling, SDKs, infrastructure-as-code, and cost guardrails, followed by a per-chapter table of the tools each week's hands-on project requires. Everything here assumes the Windows workstation and PowerShell conventions established in Chapter 0.

Note

Use a personal learning account or an organization-approved sandbox for every lab in this book. Do not run training commands in a production account unless your employer has explicitly approved the account, Region, budget, and cleanup process.

B.1 Account and Identity

The account setup sequence is fixed; do it once, in order:

1. **Create the account** with a long-term email address, a non-sensitive account name (for example `data-engineering-sandbox`), a payment card, and a reachable phone number.
2. **Enable MFA on the root user** before building anything. Sign out and back in once to confirm the second factor is required, then stop using root for daily work.
3. **Create a daily administrative identity**, preferring **IAM Identity Center** (workforce sign-in with temporary credentials) over a long-lived IAM admin user. Never create access keys for the root user.
4. **Set the IAM password policy** (minimum length 14, all character classes) and add billing and security contacts.
5. **Record the account ID** and store root credentials, MFA recovery notes, and the admin profile name in a password manager—never in notes, screenshots, or source files.

Common Pitfall

Working as root or leaving root without MFA. An unprotected root sign-in is a high-risk target: it can change billing, close the account, and lock you out. Root is the owner, not the operator.

B.2 AWS CLI v2 and Profiles

Install AWS CLI v2 from the official MSI [40], open a fresh PowerShell window, and verify with `aws --version`. Then configure access through one of two paths:

- **IAM Identity Center (preferred):** `aws configure sso` captures the SSO start URL and Region; `aws sso login --profile sandbox` opens the browser flow and issues short-lived credentials. No long-lived keys touch the laptop.
- **Access keys (legacy-compatible):** `aws configure --profile sandbox` stores an IAM user's access key under a named profile. Rotate keys periodically and delete unused ones.

Named profiles keep sandbox and any employer credentials strictly separate; every command in this book accepts `--profile <name>`. Set the default Region (`us-east-1` unless the chapter says otherwise) in the profile so it travels with the credentials. **AWS CloudShell** is the zero-install fallback: a browser shell in the console, pre-authenticated to your console identity, with the CLI and common SDKs already present.

Verification before any lab:

```
1 aws sts get-caller-identity --profile sandbox
2 aws s3 ls --profile sandbox
```

Listing B.1: Environment smoke test used throughout the book.

The first command proves authentication and shows which account and identity you are acting as; run it *every* time you switch profiles. The second proves S3 access in the expected Region.

B.3 SDKs: Boto3 and Friends

Python scripts in this book use Boto3, the AWS SDK for Python [61]:

```
1 pip install boto3 pandas pyarrow redis
```

Listing B.2: Installing the Python tooling used across chapters.

Boto3 reads the same profile chain as the CLI: `boto3.Session(profile_name="sandbox")` or the `AWS_PROFILE` environment variable. The recurring pattern is `session.client("s3")` for operation-level APIs and `session.resource("s3")` where an object-oriented interface reads better. Streaming chapters add `redshift-data` (execute SQL without a persistent connection), and the ML chapters add `sagemaker` and `pyspark` where noted.

B.4 Infrastructure as Code: CDK and Terraform

Chapters 12 and 24 move from CLI commands to infrastructure as code. Two toolchains appear:

- **AWS CDK** (Chapter 12, orchestration environments): install Node.js, then `npm install -g aws-cdk`, and bootstrap each account/Region pair once with `cdk bootstrap`. CDK synthesizes CloudFormation, so `cdk synth` is the review-before-deploy step [38].
- **Terraform** (Chapter 24, DataOps): the AWS provider with a remote S3 backend and DynamoDB state locking; `terraform plan` is the review-before-deploy step. CI authenticates via OIDC to an IAM role—never stored long-lived keys.

Both coexist with the CLI: the CLI is for exploration and smoke tests, IaC is for anything you intend to keep, repeat, or hand to a teammate.

B.5 Cost Guardrails

Set every guardrail in Chapter 0 before the first lab, and keep them on for the whole program:

- **AWS Budgets:** a monthly cost budget with alerts at 50%, 80%, and 100% of a small threshold (for example \$10), plus a forecasted-spend alert.
- **Free Tier usage alerts:** enabled in Billing preferences so approaching an allowance emails you before it is exceeded.
- **Cost Explorer:** reviewed weekly, grouped by service, to catch forgotten resources—idle NAT gateways, unattached volumes, running clusters, and Serverless workgroups left at high base capacity are the recurring offenders.
- **Cleanup habit:** every lab chapter ends with deletion commands; run them. Tag all lab resources (`Environment=Training, Project=<chapter>`) so `aws resourcegroupstaggingapi` can enumerate what a lab created.

Common Pitfall

Assuming Free Tier means no bill. Free Tier is a discount schedule with limits. Resources outside the offer, usage above the allowance, idle billable infrastructure, data egress, and forgotten resources all create charges.

B.6 Per-Chapter Tool Requirements

The table below distills each chapter's Thursday lab prerequisites. "CLI" always means AWS CLI v2 with a configured profile; "PS" means Windows PowerShell. Everything builds on the Chapter 0 sandbox: secured account, named profile, budget alerts.

Ch.	Services	Local tooling and prerequisites
0	IAM, STS, S3, Budgets	CLI v2, CloudShell, password manager
1	S3, SQS	CLI; globally unique bucket names; Python + Boto3 for the checker script
2	RDS/Aurora, VPC, Secrets Manager	CLI; small ACU ranges; restrictive security groups
3	DynamoDB	CLI + Boto3; on-demand capacity
4	ElastiCache (Redis), RDS	Python <code>redis</code> client; sandbox VPC with two private subnets; existing RDS/Aurora endpoint
5	Redshift Serverless, S3, IAM	CLI; Python data generator; query editor v2
6	Redshift Serverless	Chapter 5 workgroup; EXPLAIN for plan inspection
7	Glue crawlers, Athena, Spectrum	Chapter 5 dataset exported as CSV; Athena workgroup with results bucket
8	Lake Formation, Athena	Chapter 7 catalog; two IAM roles (marketing, analytics)
9	EMR (EC2 + Spot fleets), S3	CLI; PySpark job from Chapter 7 curated zone
10	Glue Studio, Redshift	Chapter 7 catalog tables; Redshift workgroup from Chapter 5
11	DMS, RDS PostgreSQL, EventBridge	Parameter group with <code>rds.logical_replication=1</code> ; CloudWatch alarms
12	MWAA, Step Functions, SNS	S3 DAG bucket; least-privilege execution role; optional CDK for the environment
13	Kinesis Data Streams	Python + Boto3 producer/consumer
14	Data Firehose, Lambda, Glue table	Chapter 13 stream; Glue table for Parquet conversion
15	Managed Service for Apache Flink	Source and sink Kinesis streams; Studio notebook
16	QuickSight, Redshift, SNS	Chapter 5 warehouse; QuickSight account (free trial tier suffices)
17	Redshift ML, SageMaker, S3	RedshiftML IAM role; <code>ml-artifacts</code> staging bucket
18	SageMaker Studio, Data Wrangler, Autopilot	Studio domain; governed bucket read through Lake Formation grant
19	SageMaker Pipelines, Feature Store, Registry, Model Monitor	Chapter 18 domain; injected-bad-batch dataset for the gate demo
20	Textract, Comprehend, Lambda, OpenSearch	S3 event notification on <code>raw/documents/</code> ; sample scanned PDFs
21	VPC, endpoints, KMS, EC2 + SSM	VPC with no internet gateway; Session Manager for instance access
22	Glue Data Quality, SNS, LF-Tags	Existing Glue job (Chapter 10); deliberately violating batch
23	Macie, EventBridge, Lambda, Object Lock	Object-Locked quarantine bucket; synthetic PII CSV (never real data)
24	Terraform, dbt-redshift, GitHub Actions	GitHub account; S3+DynamoDB backend; OIDC-to-IAM role; Dev/Prod environments

Tip

When a lab fails on an unfamiliar error, check the three usual suspects in order: wrong profile (`aws sts get-caller-identity`), wrong Region (resources are Regional; the console and CLI must agree), and a missing Chapter 0 guardrail (budget alerts off, tagging skipped). These three account for most first-hour failures in every chapter.

AWS CLI, SDK, and SQL Quick Reference

This appendix collects the commands used most often across the book, grouped by service, with a one-line purpose for each. All CLI commands accept `--profile` and `--region`; all are shown without them for readability. Where a chapter prefers the console or an SDK, the table says so. Full syntax lives in the AWS CLI Command Reference [40] and each service’s developer guide (see the chapter’s Further Reading).

C.1 Amazon S3 (Chapters 1, 7, 21)

Command	Purpose
<code>aws s3 mb s3://<bucket></code>	Create a bucket (name must be globally unique)
<code>aws s3 ls [-recursive] s3://<bucket>/<prefix></code>	List buckets or objects
<code>aws s3 cp <src> <dst></code>	Copy one file to/from S3
<code>aws s3 sync <dir> s3://<bucket>/<prefix></code>	Reconcile a local tree with a prefix (uploads only deltas)
<code>aws s3 rm s3://<bucket>/<prefix> --recursive</code>	Delete objects (add <code>--exclude</code> / <code>--include</code> to scope)
<code>aws s3api put-bucket-versioning --bucket B --versioning-configuration Status=Enabled</code>	Turn on versioning
<code>aws s3api put-bucket-lifecycle-configuration --bucket B --lifecycle-configuration file://lc.json</code>	Apply lifecycle transitions/expiration
<code>aws s3api put-bucket-notification-configuration ...</code>	Wire S3 events to SQS/SNS/Lambda
<code>aws s3api put-object-lock-configuration ...</code>	Enable default WORM retention (bucket creation option)
<code>aws s3 presign s3://<bucket>/<key></code>	Generate a time-limited download URL
<code>aws s3api head-object --bucket B --key K</code>	Read object metadata without downloading

Boto3: `s3.upload_file` / `download_file` for managed transfers, `list_objects_v2` with a paginator for listings, `put_object` for generated content, `get_object_attributes` for inspection.

C.2 Amazon Athena (Chapter 7)

Command / SQL	Purpose
<code>aws athena start-query-execution</code>	Submit a query; returns an execution ID
<code>--query-string "..."</code> <code>--work-group W</code>	
<code>aws athena get-query-execution</code>	Poll status; read bytes-scanned for cost
<code>--query-execution-id ID</code>	
<code>aws athena get-query-results</code>	Fetch result rows
<code>--query-execution-id ID</code>	
<code>CREATE EXTERNAL TABLE ... LOCATION</code> <code>'s3://...'</code>	Define a table over lake files
<code>MSCK REPAIR TABLE t;</code>	Register new partitions after files land
<code>CREATE TABLE x WITH (format='PARQUET',</code> <code>partitioned_by=...) AS SELECT ...</code>	CTAS: convert CSV to partitioned Parquet (the chapter's cost fix)
<code>UNLOAD (SELECT ...) TO 's3://...' WITH</code> <code>(format='PARQUET')</code>	Export query results as files

C.3 Amazon Redshift and Redshift Serverless (Chapters 5, 6, 17)

Command / SQL	Purpose
<code>aws redshift-serverless create-namespace</code>	Create the Serverless namespace with an attached role
<code>--namespace-name N --iam-roles ...</code>	
<code>aws redshift-serverless create-workgroup</code>	Create the endpoint; base/max RPU bound cost
<code>--base-capacity 8 ...</code>	
<code>aws redshift-data execute-statement</code>	Run SQL without a persistent connection
<code>--workgroup-name W --sql "..."</code>	
<code>aws redshift-data get-statement-result --id</code> <code>ID</code>	Fetch rows for a finished statement
<code>COPY t FROM 's3://...' IAM_ROLE '...'</code> <code>FORMAT AS JSON 'auto';</code>	Parallel bulk load; one input file per slice ideal
<code>MERGE INTO target USING stage ON ...</code>	Change-data upsert / SCD Type 2 maintenance
<code>UNLOAD ('SELECT ...') TO 's3://...'</code> <code>IAM_ROLE '...' PARQUET;</code>	Export warehouse data to the lake
<code>CREATE MATERIALIZED VIEW mv AS SELECT ...</code>	Pre-compute a repeated aggregation
<code>EXPLAIN <query></code>	Inspect distribution moves (<code>DS_DIST_*</code>) before benchmarking
<code>SELECT * FROM SVV_TABLE_INFO;</code>	Diagnose skew, sort-key effectiveness, table size
<code>CREATE MODEL m FROM (SELECT ...) TARGET y</code> <code>FUNCTION f ... PROBLEM_TYPE ... OBJECTIVE</code> <code>'f1';</code>	Redshift ML: train via SageMaker Autopilot from SQL

C.4 AWS Glue (Chapters 7, 9, 10, 22)

Command / code	Purpose
<code>aws glue create-crawler --name C --role R --database-name D --targets '...'</code>	Define a crawler over an S3 prefix
<code>aws glue start-crawler --name C</code>	Run inference; populate the Data Catalog
<code>aws glue get-tables --database-name D</code>	Inspect inferred schemas and partitions
<code>aws glue create-job --name J --role R --command '...' --glue-version "4.0"</code>	Register an ETL job (script in S3)
<code>aws glue start-job-run --job-name J [--arguments '...']</code>	Run the job; pass parameterized arguments
<code>aws glue get-job-run --job-name J --run-id ID</code>	Check status and error detail
<code>glueContext.create_dynamic_frame.from_catalog(Job)script: read a catalog table with bookmark support</code>	
<code>.resolveChoice(...) / .applyMapping(...)</code>	DynamicFrame schema-resolution operators for messy data
<code>EvaluateDataQuality.apply(frame, ruleset)</code>	In-job DQ gate (DQDL rules; Chapter 22)
<code>job.init(...) / job.commit()</code>	Enable job bookmarks around the script body

C.5 Amazon EMR (Chapter 9)

Command	Purpose
<code>aws emr create-cluster --release-label emr-7.x --instance-fleets file://f.json ...</code>	Launch a cluster; instance fleets mix On-Demand core + Spot task nodes
<code>aws emr add-steps --cluster-id J-X --steps file://step.json</code>	Submit a Spark step (e.g., <code>spark-submit s3://.../job.py</code>)
<code>aws emr list-clusters --active</code>	See what is running (and billing)
<code>aws emr terminate-clusters --cluster-ids J-X</code>	Tear down; EMRFS means the S3 data survives
<code>aws emr-serverless create-application / start-job-run</code>	Same Spark job without managing nodes

C.6 Amazon Kinesis Data Streams (Chapter 13)

Command / code	Purpose
<code>aws kinesis create-stream --stream-name S --shard-count N</code>	Create a stream (shard math from Chapter 13)
<code>aws kinesis put-records --stream-name S --records file://batch.json</code>	Batch write; inspect <code>FailedRecordCount</code> and retry with backoff
<code>aws kinesis get-shard-iterator --shard-id ... --shard-iterator-type TRIM_HORIZON</code>	Position a consumer on a shard
<code>aws kinesis get-records --shard-iterator I</code>	Poll records; loop with the returned <code>NextShardIterator</code>
<code>aws kinesis update-shard-count --scaling-type UNIFORM_SCALING ...</code>	Reshard when throughput outgrows capacity

Watch: `IteratorAgeMilliseconds` (consumer lag) in CloudWatch; alarm well before the retention horizon. Production consumers use KCL, not raw iterators.

C.7 Amazon MSK (Chapter 14)

Command	Purpose
<code>aws kafka create-cluster --cluster-name C --broker-node-group-info file://b.json ...</code>	Provision a managed Kafka cluster
<code>aws kafka get-bootstrap-brokers --cluster-arn ARN</code>	Retrieve broker endpoints for producers/consumers
<code>kafka-topics.sh --create --topic T --partitions N</code>	Topic administration from a client host
<code>aws kafka update-configuration / update-broker-count</code>	Change config or scale brokers in place

C.8 Amazon Data Firehose (Chapter 14)

Command	Purpose
<code>aws firehose create-delivery-stream --delivery-stream-name D ...</code>	Create the stream (source, transform, destination in one definition)
<code>aws firehose put-record-batch --delivery-stream-name D ...</code>	Direct-write test records
<code>aws firehose update-destination ...</code>	Change buffering, conversion, or backup settings

Enable the S3 source-record backup prefix on every stream; JSON→Parquet conversion requires a Glue catalog table describing the schema.

C.9 AWS Lambda (Chapters 14, 15, 20)

Command	Purpose
<code>aws lambda create-function --function-name F --runtime python3.12 --role R --handler h.lambda_handler --zip-file fileb://z.zip</code>	Deploy a function
<code>aws lambda update-function-code --function-name F --zip-file fileb://z.zip</code>	Ship a new version of the code
<code>aws lambda invoke --function-name F out.json</code>	Synchronous test invocation
<code>aws lambda create-event-source-mapping --event-source-arn <kinesis-arn> --function-name F</code>	Attach a stream trigger (batch size, failure destination here)

C.10 Managed Flink (Chapter 15)

Command	Purpose
<code>aws kinesisanalyticsv2 create-application --runtime-environment FLINK-1_20 ...</code>	Create the Flink application
<code>aws kinesisanalyticsv2 start-application / stop-application</code>	Run or pause (savepoint on stop)
<code>aws kinesisanalyticsv2 create-application-snapshot</code>	Manual savepoint before a redeploy

Flink SQL from Chapter 15: declare stream tables with `CREATE TABLE ... WITH ('connector'='kinesis', ...)` and a `WATERMARK FOR click_ts AS click_ts - INTERVAL '1' MINUTE`, then `GROUP BY TUMBLE(click_ts, INTERVAL '5' MINUTE)`.

C.11 Step Functions and Amazon MWAA (Chapter 12)

Command	Purpose
<code>aws stepfunctions create-state-machine</code> <code>--name M --definition file://asl.json</code> <code>--role-arn R</code>	Deploy a state machine (ASL definition)
<code>aws stepfunctions start-execution</code> <code>--state-machine-arn ARN --input '...'</code>	Trigger a run
<code>aws stepfunctions describe-execution</code> <code>--execution-arn ARN</code>	Inspect state-by-state results and failures
<code>aws mwaa create-environment --dag-s3-path</code> <code>dags/ ...</code>	Provision a managed Airflow environment
<code>aws s3 cp dag.py s3://<mwaa-bucket>/dags/</code> <code>airflow dags trigger <dag_id> (via CLI plugin)</code>	Deploy a DAG (the S3 sync is the deploy) Manual run for testing

C.12 Amazon SageMaker (Chapters 17–20)

Command / call	Purpose
<code>aws sagemaker create-training-job</code> <code>--training-job-name J ...</code>	Launch managed training (<code>EnableManagedSpotTraining</code> for the discount)
<code>aws sagemaker create-auto-ml-job-v2 ...</code>	Autopilot experiment over an S3 dataset
<code>aws sagemaker create-endpoint-config /</code> <code>create-endpoint</code>	Deploy a model (<code>ServerlessConfig</code> for sporadic traffic)
<code>aws sagemaker-runtime invoke-endpoint</code> <code>--endpoint-name E --body '...' out.json</code>	Score a payload
<code>aws sagemaker create-pipeline /</code> <code>start-pipeline-execution</code>	Register and run a Preprocess→Train→Evaluate→Register pipeline
<code>aws sagemaker-featurestore-runtime</code> <code>put-record / get-record</code>	Write to the offline/online store; millisecond reads at inference
<code>aws sagemaker update-model-package-approval-status</code>	Registry gate before deployment automation picks a version up

C.13 Amazon Textract, Comprehend, and Macie (Chapters 20, 23)

Command	Purpose
<code>aws textract analyze-document --document</code> <code>'...' --feature-types ['FORMS']</code>	Sync key-value extraction (single-page)
<code>aws textract start-document-analysis /</code> <code>get-document-analysis</code>	Async jobs for multi-page PDFs; SNS signals completion
<code>aws comprehend detect-pii-entities --text</code> <code>"..." --language-code en</code>	PII entity detection over extracted text
<code>aws macie2 enable-macie</code>	Turn on discovery for the account/Region
<code>aws macie2 create-classification-job</code> <code>--job-type ONE_TIME ...</code>	Scan a bucket/prefix for sensitive data
<code>aws macie2 list-findings --finding-criteria</code> <code>'...'</code>	Review findings (IDs feed the quarantine automation)

C.14 Lake Formation and Governance (Chapters 8, 21, 22)

Command	Purpose
<code>aws lakeformation put-data-lake-settings --data-lake-admins '[...]</code>	Designate data lake administrators
<code>aws lakeformation register-resource --resource-arn arn:aws:s3:::B/prefix/ --role-arn R</code>	Register a lake location for governed access
<code>aws lakeformation revoke-permissions ... IAMAllowedPrincipals</code>	Remove the default IAM-only grant so LF policies actually bind
<code>aws lakeformation grant-permissions --principal '...' --resource '...' --permissions SELECT</code>	Grant on databases, tables, or named column lists
<code>aws lakeformation create-lf-tag / add-lf-tags-to-resource</code>	Define tags and tag resources for expression-based policy
<code>aws lakeformation get-effective-permissions-for-path</code>	Debug who can read a location and why

C.15 Cross-Cutting: STS, IAM, Budgets (Chapter 0)

Command	Purpose
<code>aws sts get-caller-identity</code>	Prove which identity/account a profile acts as
<code>aws configure sso / aws sso login --profile P</code>	Identity Center setup and sign-in
<code>aws iam create-role --role-name R --assume-role-policy-document file://trust.json</code>	Create a service role (trust policy)
<code>aws iam put-role-policy --role-name R --policy-name P --policy-document file://p.json</code>	Attach least-privilege permissions
<code>aws budgets create-budget --account-id ID --budget file://b.json ...</code>	Cost guardrail with alert thresholds
<code>aws ce get-cost-and-usage --time-period ... --granularity MONTHLY --metrics UnblendedCost</code>	Cost Explorer from the CLI

Tip

Three commands debug most lab failures: `aws sts get-caller-identity` (am I the identity I think?), `aws configure list` (which profile and Region?), and the service's `describe-*` call (does the resource actually exist, in this Region, with the name I typed?).

Lab Solutions and Acceptance Guidance

This appendix provides reference solutions for every chapter’s Thursday hands-on project (Chapters 0–24) and selected end-of-chapter exercises. Each entry gives the approach, the key commands or code fragments, the evidence that satisfies the chapter’s acceptance criteria, and the failure modes that actually occur. Use it to unblock yourself after a genuine attempt—not instead of one: the labs are the curriculum, and the AWS Certified Data Engineer – Associate exam measures exactly the judgment they build [36].

Note

A solution counts only when the chapter’s acceptance criteria are met with evidence: row counts, metric screenshots, ledger rows, drill outputs. “It ran once on my machine” is the starting point of each lab, never the end. Replace every placeholder—bucket names, account IDs, Regions, role ARNs—with your own before running anything.

D.1 Chapter 0: Environment Verification

D.1.1 Project: Account, Profile, Guardrails, and the S3 Smoke Test

Approach. Secure root (MFA), create the Identity Center admin, configure the `sandbox` CLI profile, create a budget with alerts, then verify:

```
aws sts get-caller-identity --profile sandbox
aws s3 mb s3://<unique-bucket> ; aws s3 ls ; aws s3 rb s3://<unique-bucket>
```

Critical evidence. Caller identity shows the admin (never root); the budget exists with three thresholds; bucket create/list/delete succeed in the intended Region. **Common failure.** SSO token expired mid-lab—re-run `aws sso login`; or working in the console’s default Region while the CLI profile points elsewhere.

D.2 Part I: Core Data Infrastructure and Storage (Chapters 1–4)

D.2.1 Chapter 1: Bucket, Lifecycle, and Events

Approach. CLI-first: create the bucket, enable versioning, apply a 30-day transition to Glacier Flexible Retrieval, create the SQS queue, attach the queue policy, register the S3 event notifi-

cation, upload objects, and read the events back. **Key fragment.**

```
aws s3api put-bucket-versioning --bucket $B --versioning-configuration Status=
  Enabled
aws s3api put-bucket-lifecycle-configuration --bucket $B --lifecycle-
  configuration file://lc.json
aws s3api put-bucket-notification-configuration --bucket $B '
  --notification-configuration file://events.json
aws sqs receive-message --queue-url $Q --max-number-of-messages 10
```

Critical evidence. The Python checker confirms versioning, lifecycle rules, and the SQS binding; an uploaded object produces a readable S3 ObjectCreated event. **Common failure.** The SQS queue policy missing the `Service: s3.amazonaws.com` principal, so the notification configuration is rejected; or bucket names colliding with the global namespace—append initials plus the account ID.

D.2.2 Chapter 2: Aurora Serverless v2 and Regional Failover

Approach. Create an Aurora PostgreSQL Serverless v2 cluster with a small ACU range, attach it to an Aurora Global Database with a secondary Region, then rehearse the failover: switchover, verify the writer moved, switch back. **Key fragment.**

```
aws rds create-db-cluster --engine aurora-postgresql --engine-mode provisioned
  ,
  --serverless-v2-scaling-configuration MinCapacity=0.5,MaxCapacity=4 ...
aws rds switchover-global-cluster --global-cluster-identifier $G '
  --target-db-cluster-identifier $SecondaryArn
```

Critical evidence. The secondary cluster becomes the writer; a write/read round-trip succeeds against the new endpoint; the cluster and secrets are deleted afterward. **Common failure.** Forgetting that global-cluster teardown is ordered (instances, then clusters, then the global cluster), or leaving the secondary Region’s cluster running and billing.

D.2.3 Chapter 3: Forum Single-Table Schema and GSI

Approach. Design access patterns first, then create one table holding users, forums, threads, and posts with generic PK/SK attributes, plus a GSI for user-centric queries. Insert representative items and run every declared access pattern as a query. **Key fragment.**

```
aws dynamodb create-table --table-name forum-single-table-demo '
  --attribute-definitions AttributeName=PK,AttributeType=S ... '
  --key-schema AttributeName=PK,KeyType=HASH AttributeName=SK,KeyType=RANGE '
  --global-secondary-indexes file://gsi.json --billing-mode PAY_PER_REQUEST
```

Critical evidence. Each access pattern answered by Query (not Scan); the GSI returns user-centric ordering; item sizes stay small. **Common failure.** Modeling separate tables per entity (relational habits) and then trying to join client-side; or choosing a low-cardinality partition key that concentrates hot traffic.

D.2.4 Chapter 4: Redis Cache-Aside for RDS Queries

Approach. Deploy a small ElastiCache Redis replication group in the sandbox VPC, then a Python cache-aside layer: check cache, on miss query RDS and store with a TTL. Measure latency before/after and deliberately invalidate. **Key fragment.**

```
cached = r.get(key)
if cached is None:
    rows = cur.execute(sql).fetchall()
    r.setex(key, 300, json.dumps(rows)) # TTL bounds staleness
```

Critical evidence. Second identical query served from cache with sub-millisecond reads; after a source write, the invalidation path (or TTL expiry) makes the change visible. **Common failure.** Security-group mismatch (Lambda/app SG not allowed to the cache SG on 6379); or caching without TTL so staleness is unbounded.

D.3 Part II: Data Warehousing and Analytics (Chapters 5–8)

D.3.1 Chapter 5: Load a Star Schema into Redshift Serverless

Approach. Create namespace + workgroup with an attached least-privilege role, generate ten million synthetic order-line rows as ten JSON files (so COPY parallelizes across slices), declare the grain, create `fact_sales` plus three conformed dimensions, COPY in, then practice the SCD Type 2 MERGE against a staged customer extract. **Critical evidence.** `SELECT COUNT(*)` on the fact equals the generated row count; dimension lookups join cleanly; the SCD2 MERGE run twice changes nothing on the second run. **Common failure.** The role trust policy missing `redshift.amazonaws.com`, so COPY fails authorization; or generating one giant file so COPY serializes on the leader’s file assignment instead of parallelizing.

D.3.2 Chapter 6: Benchmarking Distribution Styles

Approach. Create three copies of the fact table—`DISTSTYLE KEY` on the join column, `EVEN`, and `ALL`—then run one join-heavy aggregation against each, record elapsed time, and read `EXPLAIN` for `DS_DIST_OUTER`/broadcast steps. **Critical evidence.** The completed timing table; the `KEY` plan showing a colocated join; the `ALL` plan’s storage and load-time cost quantified. **Common failure.** Benchmarking on data too small for redistribution to matter (regenerate a larger dimension); or comparing cold versus cached runs—clear the result cache or vary the query text.

D.3.3 Chapter 7: Crawl, Query, and Federate the Same Data

Approach. Run a Glue crawler over raw CSVs, aggregate in Athena recording bytes scanned, CTAS to partitioned Parquet, catalog the result, rerun (bytes scanned collapses), then create a Spectrum external schema and join lake to warehouse. **Key fragment.**

```
CREATE TABLE sales_parquet WITH (format='PARQUET',
    external_location='s3://.../curated/sales/', partitioned_by=ARRAY['dt'])
AS SELECT *, CAST(order_date AS date) AS dt FROM raw_sales_csv;
```

Critical evidence. The bytes-scanned before/after pair (expect one to two orders of magnitude); the Spectrum join returning correct counts. **Common failure.** Crawling a prefix that mixes datasets so the crawler creates spurious tables; or forgetting the Athena workgroup results bucket.

D.3.4 Chapter 8: Hiding the PII Column from Marketing

Approach. Register the S3 location with Lake Formation, designate the admin, revoke `IAMAllowedPrincipals` on database and table, grant the marketing role `SELECT` on a column list excluding `pii_email`, then verify from both roles in Athena. **Critical evidence.** Marketing query on allowed columns succeeds; the same query with `pii_email` fails with a Lake Formation error; the analytics role still sees everything. **Common failure.** Testing as the lake admin—every grant passes trivially; or revoking `IAMAllowedPrincipals` on the database but not the table, so the filter never binds.

D.4 Part III: Batch Processing and ETL Orchestration (Chapters 9–12)

D.4.1 Chapter 9: A PySpark Aggregation on EMR

Approach. Launch EMR on EC2 with instance fleets (On-Demand core, Spot task), submit a PySpark step aggregating the Chapter 7 curated Parquet, write results back to S3, inspect the Spark UI, terminate, and prove the data survived. **Critical evidence.** Output Parquet in S3 after cluster termination; Spark UI shows the distributed aggregation (stages, tasks across executors); Spot interruptions rescheduled tasks without job failure. **Common failure.** Putting durable output on cluster HDFS (lost at termination); or Spot on core nodes risking HDFS loss—only task nodes are Spot-safe.

D.4.2 Chapter 10: A Glue Studio Pipeline into Redshift

Approach. Visual canvas: two catalog sources (`week7_lake.sales_parquet`, `week7_lake.customers`), a join, a null filter, a timestamp transform, a Redshift target. Then harden the generated PySpark: parameterize, enable bookmarks, run twice, and inject one bad record. **Critical evidence.** Second run with bookmarks processes nothing new; the bad record is visibly handled (dropped or quarantined, per the transform); row counts reconcile between source and Redshift. **Common failure.** Bookmarks enabled but the sink not idempotent—re-runs still duplicate rows; or forgetting the Redshift connection’s VPC/subnet placement so the job cannot reach the endpoint.

D.4.3 Chapter 11: CDC from RDS PostgreSQL to the Lake

Approach. Enable logical replication (`rds.logical_replication=1` in the parameter group, reboot, confirm `wal_level=logical`), create a DMS replication instance, source and target endpoints, and a full-load-plus-CDC task. Generate source changes, watch change files land in S3, alarm on lag. **Critical evidence.** INSERT/UPDATE/DELETE on the source appear as S3 change files; `CDCLatencySource`/`CDCLatencyTarget` stay near zero under light load and the CloudWatch alarm exists. **Common failure.** Skipping the parameter-group reboot, so the task fails at CDC start; or WAL retention too low so a paused task cannot resume and forces a reload.

D.4.4 Chapter 12: A Production-Shaped MWAA DAG

Approach. Deploy the daily-reconciliation DAG to the MWAA DAG bucket: `S3KeySensor` → `GlueJobOperator` → Athena validation → SNS alert, with `catchup=False`, retries with delay, a 300-second `poke_interval`, and `trigger_rule="one_failed"` on the alert. **Critical evidence.** A forced Glue failure fires the SNS alert with the runbook link; a normal run lands one ledger row; the execution role is least-privilege to exactly the four services used. **Common failure.** The alert task skipped because the default trigger rule skips on upstream skip—exactly the runs it exists for; or `catchup=True` with a past `start_date` backfilling against production on deploy.

D.5 Part IV: Streaming and Real-Time Analytics (Chapters 13–16)

D.5.1 Chapter 13: An IoT Temperature Stream

Approach. Shard math first ($50 \text{ sensors} \times 150 \text{ B} \approx 7.5 \text{ KB/s}$, $50 \text{ records/s} \rightarrow$ one shard suffices; two used to make sharding visible). Python producer with `put_records`, partial-failure handling with backoff, per-sensor partition keys for ordering; a raw-iterator consumer printing records. **Critical evidence.** Records grouped per sensor arrive in order per shard; `IteratorAgeMilliseconds` rises when the consumer pauses and falls to zero on catch-up. **Common failure.** Ignoring `FailedRecordCount` (individual records fail inside a 200 batch); or using the sensor *type* as the partition key and hot-sharding one key.

D.5.2 Chapter 14: Stream to Parquet via Firehose

Approach. Delivery stream sourcing from the Chapter 13 Kinesis stream, a Lambda that uppercases string values, JSON→Parquet conversion against a Glue table, S3 destination with error prefixes and raw backup enabled. **Critical evidence.** Curated Parquet files under the date-partitioned prefix; original JSON in the backup prefix; a malformed record in the error prefix. **Common failure.** The Glue table schema not matching the transformed JSON (conversion fails silently into the error prefix); or Lambda timeouts from per-record work that belongs in batch.

D.5.3 Chapter 15: A Flink SQL Tumbling-Window Counter

Approach. Source and sink Kinesis streams; Flink SQL tables with a declared watermark; a 5-minute TUMBLE count; then deliberately send on-time, late-within-watermark, and late-beyond-watermark events and observe. **Key fragment.**

```
CREATE TABLE clicks (user_id STRING, page STRING, click_ts TIMESTAMP(3),
  WATERMARK FOR click_ts AS click_ts - INTERVAL '1' MINUTE) WITH (...);
SELECT TUMBLE_START(click_ts, INTERVAL '5' MINUTE) AS w, COUNT(*) AS n
FROM clicks GROUP BY TUMBLE(click_ts, INTERVAL '5' MINUTE);
```

Critical evidence. Windows fire on the watermark, not the wall clock; the beyond-bound late event is dropped (or side-output, if configured) and that behavior is documented. **Common failure.** Judging lateness by ingestion time—the table must declare event time plus watermark or every “late” test is meaningless.

D.5.4 Chapter 16: A Monitored Sales Dashboard

Approach. QuickSight dataset over `fact_sales` joined to its dimensions; KPI + trend line + working region filter; SPICE import with a refresh schedule; ML anomaly detection on daily revenue alerting to email/SNS; the same dataset in direct query for comparison. **Critical evidence.** The dashboard filtered per region; the anomaly alert fires on an injected dip; SPICE vs. direct-query latency and freshness notes recorded. **Common failure.** Publishing before RLS rules are tested with a non-author identity; or scheduling SPICE refresh slower than the freshness requirement the dashboard promises.

D.6 Part V: Machine Learning and MLOps (Chapters 17–20)

D.6.1 Chapter 17: Churn Prediction in Pure SQL

Approach. RedshiftMLRole trusted by Redshift with S3 staging and SageMaker permissions; training query with the leaky column deliberately excluded (and documented); `CREATE MODEL` with explicit problem type and F1 objective; score the customer base and persist a risk table. **Critical evidence.** Validation metrics show F1 (not accuracy) as the objective; the risk table exists and a second scoring run is idempotent; the excluded leaky column is named in the write-up. **Common failure.** Training on a feature only knowable after the outcome (target leakage) producing a “too good” model; or forgetting the staging bucket grant so the export step fails.

D.6.2 Chapter 18: Wrangler to Serverless Endpoint

Approach. Governed S3 dataset read through the Lake Formation grant; Data Wrangler flow (type fixes, null imputation, one-hot) exported as a Processing Job; Autopilot experiment including XGBoost candidates; best model to a serverless endpoint; test payload. **Critical evidence.** The exported flow reruns without edits; the endpoint scales to zero and answers a payload; the candidate leaderboard was inspected, not just accepted. **Common failure.** The Studio execution role bypassing Lake Formation with broad S3 access—the lab’s governance point is then untested; or leaving a real-time endpoint running when the lab called for serverless.

D.6.3 Chapter 19: A SageMaker Pipeline, End to End

Approach. Four-step pipeline—Preprocess → Train → Evaluate → Register—with parameters for input data and thresholds, a condition step gating registration on the metric, executed twice: once passing, once with an injected quality drop. **Critical evidence.** One run registers, one run halts at the gate; lineage shows the registered version’s data and training inputs; the approved version is attributable in the Registry. **Common failure.** Step caching masking a real change (inputs changed but fingerprints did not); or the condition comparing the wrong metric name so the gate never fires.

D.6.4 Chapter 20: S3-Triggered Textract Extraction

Approach. S3 event notification on `raw/documents/` invoking Lambda; sync `AnalyzeDocument` with FORMS for single pages (async + SNS for multi-page); block structure parsed into key-value JSON; results persisted under `extracted/documents/` keyed by source etag for idempotency. **Critical evidence.** Re-uploading the same file does not re-bill (etag check skips); a deliberately corrupt upload lands in the visible failure path; extracted JSON matches the form’s fields. **Common failure.** Sending a multi-page PDF to the sync API (it is single-page only); or persisting raw OCR output into the curated zone—it faithfully contains the document’s PII.

D.7 Part VI: Security, Governance, and Exam Prep (Chapters 21–24)

D.7.1 Chapter 21: Private S3 Access with No Internet Route

Approach. VPC with a private subnet and deliberately no internet gateway; S3 gateway endpoint with a restrictive endpoint policy; EC2 reached via Session Manager; copy a file to the authorized bucket, then prove a non-authorized bucket is blocked. **Critical evidence.** The upload succeeds with no public route in the route table; the endpoint policy denies the other bucket; `aws s3 ls` against the unauthorized bucket fails with an explicit deny. **Common failure.** An

interface endpoint where a gateway endpoint was meant (hourly cost plus DNS confusion); or Session Manager failing because the instance profile lacks `AmazonSSMManagedInstanceCore`.

D.7.2 Chapter 22: A Quality Gate That Halts the Pipeline

Approach. DQDL ruleset evaluated inside the Glue job between transform and sink: `ColumnExists` for structure plus `Completeness "col" >= 0.95` for content. Pass writes; fail publishes to SNS and raises. Inject a violating batch. **Critical evidence.** The violating batch halts the job, publishes nothing downstream, and pages via SNS; outcomes recorded to CloudWatch for trending; the next clean batch passes. **Common failure.** The gate evaluated but not wired—results logged while the write proceeds anyway (aspirational quality control); or thresholdless rules paging on noise until the team learns to ignore them.

D.7.3 Chapter 23: Macie Discovery and Automated Quarantine

Approach. Enable Macie, run a one-time classification job over a test prefix, upload a synthetic CSV with mock card numbers, then wire `EventBridge Macie Finding` → `Lambda` → move the object to an Object-Locked quarantine bucket and write an audit record linking finding ID to action. **Critical evidence.** The finding appears with the managed identifier match; the file moves to quarantine immutably; the audit record resolves `finding` → `object` → `action`. **Common failure.** Deleting instead of quarantining (destroys the evidence the finding refers to); or testing with real PII—synthetic fixtures only, always.

D.7.4 Chapter 24: The Repository Is the Platform

Approach. One GitHub repository: `terraform/` (module + Dev/Prod environments over an S3+DynamoDB backend), `dbt/` (three models—staging, intermediate, mart—and five tests against Redshift), and `.github/workflows/` with OIDC-to-IAM federation. **Critical evidence.** A failing dbt test blocks merge; merge deploys to Dev; Prod waits on named approval; `terraform plan` on a second machine against the remote backend shows no drift. **Common failure.** Long-lived AWS keys stored in CI secrets instead of OIDC; or state committed locally so two engineers diverge the same environment.

D.8 Selected End-of-Chapter Exercises

D.8.1 Storage and Warehousing

- (Ch. 1, design) **Lifecycle for mixed-size objects.** Compute per-object transition overhead against the actual size histogram first; exclude objects below 128 KB from Glacier transitions and let them expire in Standard-IA instead. The correct answer is usually a *filter*, not a fancier rule.
- (Ch. 5, design) **Choose fact types for an order pipeline.** Order *line* is the transaction fact; daily inventory is the periodic snapshot; the order itself (placed → shipped → delivered) is the accumulating snapshot. Grains differ; never merge them.
- (Ch. 6, tuning) **Diagnose a slow dashboard without touching SQL first.** Check WLM queue wait time: if waits dominate, the fix is capacity or concurrency scaling, not query rewriting. Only when execution time dominates do sort keys, MVs, and distribution enter.

D.8.2 ETL and Streaming

- **(Ch. 10, debugging) A bookmarked job still duplicates rows.** Bookmarks only deduplicate the *source* side. The sink must be idempotent: overwrite the target partition or MERGE by business key.
- **(Ch. 13, design) Replay requirement chooses the service.** Multiple consumers and replay rule out SQS FIFO; one consumer, no replay rules out Kinesis. State both requirements before naming a service.
- **(Ch. 15, theory) Exactly-once to S3.** Checkpointing plus a two-phase-commit (or idempotent-by-key) sink; checkpointing alone only makes *internal* state consistent.

D.8.3 ML, Security, and DataOps

- **(Ch. 18, design) Endpoint choice for sporadic traffic.** Serverless endpoints: scale to zero, pay per request, accept cold starts. Real-time only for steady SLA-bound traffic; Batch Transform for bulk offline scoring.
- **(Ch. 21, design) Close the exfiltration path.** Security groups alone cannot stop writes to a stranger's bucket: endpoint policies restrict destinations, SCPs cap the account, KMS key policies make copied data unreadable. All three layers.
- **(Ch. 24, design) Exam-prep plan for a teammate blank on streaming and security.** Rebuild, do not re-read: the Kinesis shard-math and Flink watermark labs (Chapters 13, 15), then the Lake Formation grant and VPC endpoint labs (Chapters 8, 21), each followed by the chapter exercises attempted cold.

Tip

When your solution diverges from this appendix, the tie-breaker is the acceptance criteria in the chapter, not the reference here: labs admit many correct implementations, but only one definition of done.

Production-Readiness and Governance Checklists

This appendix distills the chapters' production notes, acceptance criteria, and operational guidance into pre-launch checklists, one per domain. They are written as review artifacts: walk the relevant list with the team before a workload is declared production, and tick items with evidence, not intention. Chapter references point to the source material.

E.1 Storage and Data Lake (S3, Chapter 1)

- ☐ Versioning enabled on every bucket that holds data you cannot regenerate.
- ☐ Lifecycle rules transition and expire per the data's retention class; the cost model was validated with the actual object-size distribution (small objects can cost *more* after a Glacier transition).
- ☐ Object Lock mode chosen deliberately: governance mode only when an administrative bypass process is documented; compliance mode only when retention is certain.
- ☐ Default encryption is SSE-KMS with a key policy that names the allowed principals; public access blocked at the account level.
- ☐ Replication (CRR/SRR) configured where the RPO demands it, with a replication-time alarm.
- ☐ Zone naming convention (**raw/**, **validated/**, **curated/**) enforced by bucket or prefix policy, and every prefix has an owner tag.
- ☐ Event notifications (SQS/SNS/EventBridge) tested end-to-end with a poison-message path.
- ☐ Deletion runbook exists and has been rehearsed; nothing relies on “we will remember.”

E.2 Warehousing and Databases (RDS/Aurora, DynamoDB, Redshift; Chapters 2, 3, 5, 6)

- ☐ Multi-AZ for the primary; failover rehearsed, not just configured (Chapter 2 lab pattern).
- ☐ PITR window matches the recovery objective; cross-region posture (Aurora Global Database, global tables) justified by a written RPO/RTO.

- ☐ DynamoDB: access patterns enumerated *before* keys were chosen; on-demand vs. provisioned decided from measured traffic; GSIs sparse where possible; TTL understood as asynchronous.
- ☐ Redshift: grain declared for every fact table; distribution and sort keys validated with `EXPLAIN` and `SVV_TABLE_INFO` against production-shaped data volumes.
- ☐ WLM configured with queue monitoring rules; concurrency scaling enabled for bursty BI.
- ☐ COPY loads use manifests and multiple input files; MERGE-based upserts are idempotent under re-run.
- ☐ Serverless capacity bounds (max RPU, max ACU) set so a runaway query cannot run away with the budget.
- ☐ Materialized views have an owner and a refresh cadence; stale-MV staleness is visible to consumers.

E.3 ETL and Orchestration (Glue, EMR, MWAA, Step Functions; Chapters 9–12)

- ☐ Every job re-runnable without duplicates: bookmarks, watermarks, or idempotent MERGE writes.
- ☐ EMR clusters are transient for recurring batch; Spot only on task nodes; durable state lives in S3 via EMRFS, never on cluster HDFS.
- ☐ Glue jobs parameterized (no hardcoded dates or bucket names); Flex execution for non-urgent jobs.
- ☐ DAGs deployed with `catchup=False` unless a backfill is intended; every alert task has `trigger_rule="one_failed"` (or the state-machine equivalent).
- ☐ Sensors tuned: `poke_interval` sized to the expected arrival cadence.
- ☐ Large data moves through S3, never through XComs or state-machine payloads.
- ☐ Failure path proven, not assumed: a forced failure fires the alert and pages the right queue.
- ☐ Schema contracts enforced at ingestion (registry compatibility checks), not at query time.
- ☐ CDC lag monitored on `CDCLatencySource/CDCLatencyTarget`, not on task status.

E.4 Streaming (Kinesis, Firehose, MSK, Flink, Lambda; Chapters 13–15)

- ☐ Shard/partition count derived from both throughput limits (bytes *and* records per second) plus headroom; hot-key risk assessed in partition-key choice.
- ☐ `IteratorAgeMilliseconds` (or consumer-lag equivalent) alarmed well before retention expiry.
- ☐ Consumers idempotent: dedup on a unique event ID or upsert by key; poison records route to a DLQ that someone watches.
- ☐ Firehose streams have source-record backup enabled and an alarm on `DeliveryToRedshift.Success` (or destination equivalent); the intermediate S3 prefix has lifecycle rules.

- ☐ Lambda transforms are stateless; anything needing windows, dedup, or joins runs in Flink, not in a delivery transform.
- ☐ Flink jobs: every stateful operator bounded (window end, join interval, TTL); savepoint taken before any redeploy; RocksDB state size trended.
- ☐ Watermark policy stated in one sentence (“no event older than X”) and agreed with downstream consumers.
- ☐ Replay drill performed: reprocess from a checkpoint/savepoint without double-landing output.

E.5 Machine Learning and MLOps (Chapters 16–20)

- ☐ Training datasets built through governed access (Lake Formation grants), so PII masks apply to extraction; no broad S3 policies on ML roles.
- ☐ Training query reviewed column-by-column for target leakage (“is this knowable before the outcome?”).
- ☐ Objective metric matches the business cost (e.g., F1 for imbalanced churn), not accuracy by default.
- ☐ One feature-group definition feeds both offline training and online serving—the skew defense.
- ☐ Deployment gated on Model Registry approval; every production model is versioned, attributable, and rollback-ready.
- ☐ Model Monitor baselines refreshed on a cadence tied to retraining; drift alerts triaged by a human before any automatic retrain.
- ☐ Managed Spot training used with checkpointing to S3 so interruption resumes, not restarts.
- ☐ AI-API pipelines (Textract/Comprehend) idempotent on object identity so retries never re-bill; raw OCR/PII output lives in a restricted zone with field-level redaction downstream.

E.6 Security, Governance, and Compliance (Chapters 8, 21–23)

- ☐ Root user: MFA on, no access keys, no daily use; all operations through Identity Center or roles.
- ☐ `IAMAllowedPrincipals` revoked on every governed database and table; Lake Formation grants verified from a consumer role, not the admin role.
- ☐ Permissions boundaries on delegated role creation; SCPs cap what any account—including its admins—may do.
- ☐ Private connectivity: gateway endpoints for S3/DynamoDB, interface endpoints with endpoint policies elsewhere; no public route for data-plane traffic.
- ☐ KMS key policies restrict decryption to organization principals, closing the exfiltration path that survives network controls.
- ☐ CloudTrail data events enabled for lake buckets and delivered to an independent log-archive account.

- ☐ Secrets in Secrets Manager with automatic rotation; zero long-lived keys in CI (OIDC federation instead).
- ☐ Data-quality gates halt the pipeline on hard failures and report on thresholds (**Completeness** `>= 0.95 with threshold 0.9` pattern); quality metrics trended in CloudWatch.
- ☐ Lineage consultable per dataset: given a bad batch, you can name its consumers and its origin change in minutes.
- ☐ Macie findings feed an automated quarantine (Object-Locked bucket), with an audit record linking finding ID to action; custom identifiers tuned for precision before rollout.
- ☐ Erasure designed, not improvised: crypto-shredding keys per subject where re-identification is not required, or an enumerated-deletion runbook that includes replicas, snapshots, and streams.

E.7 Cost and Operations (Chapters 0, 23, 24)

- ☐ Budgets with 50/80/100% and forecast alerts on every account that can spend money.
- ☐ Every resource tagged (**Environment, Project, Owner**); untagged spend reviewed weekly.
- ☐ Per-TB-scanned workloads (Athena, Spectrum) enforced through partitioning + Parquet; bytes-scanned reported per query in reviews.
- ☐ Idle-cost audit automated: running clusters, attached-but-stopped databases, NAT gateways, old snapshots, and high base-capacity Serverless endpoints.
- ☐ Terraform state remote (S3 + DynamoDB locking); **main** always deployable; PR checks prove **plan** + **dbt build** before merge; Prod deploys require named approval.
- ☐ Schema changes follow expand-contract with a compatibility shim and a deferred drop.

Tip

A checklist item is done when you can point to the evidence—a config output, a metric graph, a drill log. If the honest answer is “we believe so,” the box stays empty.

Appendix F

Exam Mapping: AWS Certified Data Engineer – Associate

This appendix maps the AWS Certified Data Engineer – Associate exam domains and objectives onto the book’s chapters and sections [36], then gives the study strategy and a self-assessment checklist. Weights are indicative of the published exam guide; always confirm against the current guide before booking.

F.1 Domain-to-Chapter Map

Exam domain	Weight	Book coverage
1. Data Ingestion and Transformation	~34%	Ch. 1 (S3 events, transfer), 9 (EMR), 10 (Glue ETL), 11 (DMS CDC, contracts), 13 (Kinesis), 14 (Firehose, MSK), 15 (Flink), 20 (Textract/Comprehend)
2. Data Store Management	~26%	Ch. 1 (S3 classes, lifecycle), 2 (RDS/Aurora), 3 (DynamoDB), 4 (ElastiCache/DAX), 5–6 (Redshift), 7 (lakehouse, Athena, Spectrum)
3. Data Operations and Support	~22%	Ch. 12 (MWAA, Step Functions), 16 (serving, monitoring), 19 (pipelines, drift), 22 (data quality), 24 (DataOps, CI/CD)
4. Data Security and Governance	~18%	Ch. 8 (Lake Formation, sharing), 21 (IAM, VPC endpoints, KMS), 22 (lineage, mesh), 23 (Macie, erasure)

Table F.1: Exam domains mapped to the book (same mapping as Chapter 24, expanded below).

F.2 Objective-Level Breakdown

F.2.1 Domain 1: Data Ingestion and Transformation

- **Ingest data from sources:** Kinesis producer/shard design (Ch. 13, Monday–Wednesday); Firehose delivery, transformation, backup (Ch. 14); DMS full-load-plus-CDC (Ch. 11); S3 event notification patterns (Ch. 1 Thursday, Ch. 20).
- **Transform and process data:** Glue Studio and PySpark jobs, DynamicFrames, bookmarks (Ch. 10); EMR deployment modes (Ch. 9); Flink SQL windows, watermarks, exactly-once (Ch. 15); Lambda event processing (Ch. 14–15, 20).

- **Orchestrate pipelines:** MWAA DAGs, sensors, retries, trigger rules; Step Functions state machines (Ch. 12).

F.2.2 Domain 2: Data Store Management

- **Choose a data store:** the decision framing of Ch. 2 (relational), Ch. 3 (key-value), Ch. 4 (cache), Ch. 5 (warehouse), Ch. 7 (lakehouse)—the exam’s “which service for these constraints” questions live here.
- **Design data models and schemas:** DynamoDB single-table design and GSIs (Ch. 3); Kimball grain, fact types, SCD (Ch. 5); distribution and sort keys (Ch. 6); schema evolution and compatibility (Ch. 11).
- **Manage the data lifecycle:** S3 storage classes and lifecycle rules (Ch. 1); retention, versioning, time travel (Ch. 1, 7); TTL semantics (Ch. 3).

F.2.3 Domain 3: Data Operations and Support

- **Automate processing:** idempotent re-runs, bookmarks, manifests (Ch. 10–12); IaC with Terraform/CDK and CI/CD gates (Ch. 24).
- **Monitor and troubleshoot:** CloudWatch metrics that matter—CDC lag (Ch. 11), `IteratorAgeMilliseconds` (Ch. 13), WLM queue wait (Ch. 6), delivery success (Ch. 14); pipeline failure alerting (Ch. 12, 22).
- **Maintain data quality:** Glue Data Quality rulesets and thresholds (Ch. 22); contract enforcement (Ch. 11); dbt tests (Ch. 24).

F.2.4 Domain 4: Data Security and Governance

- **Authentication and authorization:** IAM roles, policies, boundaries, SCPs (Ch. 21); Lake Formation grants, LF-Tags, column-level security (Ch. 8); QuickSight RLS (Ch. 16).
- **Encryption and data protection:** KMS key policies, Secrets Manager rotation (Ch. 21); S3 Object Lock and SSE options (Ch. 1); crypto-shredding (Ch. 23).
- **Logging, auditing, and governance:** CloudTrail including data events (Ch. 23); catalog and lineage (Ch. 7, 22); Macie discovery and quarantine (Ch. 23); clean rooms and data sharing (Ch. 8).

F.3 Study Strategy

1. **Self-assess per domain.** For each domain, re-read the chapter Key Takeaways and attempt the exercises cold. Mark each objective *confident*, *shaky*, or *blank*.
2. **Repair blanks by doing.** A shaky area you have used recovers by re-reading; a blank area recovers only by running the lab (Appendix D has the reference solutions).
3. **Practice scenario questions.** The exam’s character is “which service/design for this constraint set”—the Friday use cases train exactly that muscle.
4. **Two full-length mock exams,** spaced two days apart, in the final week; review every wrong answer back to its chapter. Target: 80%+ overall, no domain below 70%.
5. **Use the cross-cloud appendix defensively.** Questions sometimes name a non-AWS concept (Kafka, Airflow, Delta); knowing the AWS equivalent (MSK, MWAA, Glue/EMR) converts them into free points.

Tip

Exam heuristics that recur across domains: prefer managed/serverless when the stem emphasizes operational simplicity; prefer provisioned when it emphasizes steady-state cost or fine control; prefer the answer that encrypts, logs, and least-privileges by default; distrust any answer whose durability story is “a job remembers to clean up.”

F.4 Self-Assessment Checklist

Tick an item only when you can do it unaided—build it or explain it—not merely recognize it.

Domain 1: Ingestion and Transformation

- ☐ Compute Kinesis shard counts from byte and record rates; choose partition keys; interpret `IteratorAgeMilliseconds`.
- ☐ Build a Firehose stream with Lambda transform, Parquet conversion, and raw backup.
- ☐ Configure DMS full-load-plus-CDC and read its lag metrics.
- ☐ Write a Glue job with bookmarks and explain what bookmarks do *not* guarantee.
- ☐ Write Flink SQL with event-time windows and a declared watermark.
- ☐ Author an MWAA DAG with sensors, retries, and a failure alert that cannot be skipped.

Domain 2: Data Store Management

- ☐ Choose between S3 classes and write a lifecycle policy for a stated retention requirement.
- ☐ Design a DynamoDB table from an access-pattern list, including GSIs and TTL.
- ☐ Explain Multi-AZ versus read replicas, and when Aurora Serverless v2 wins.
- ☐ Declare a fact-table grain; choose distribution and sort keys; read an `EXPLAIN` plan.
- ☐ Explain why Parquet + partitioning collapses Athena bytes scanned.

Domain 3: Operations and Support

- ☐ Make any pipeline in this book re-runnable without duplicates and say exactly how.
- ☐ Wire a data-quality gate that halts a pipeline and pages a human.
- ☐ Read a failed Step Functions execution and locate the failing state.
- ☐ Explain step caching in SageMaker Pipelines and when it masks a real change.

Domain 4: Security and Governance

- ☐ Revoke `IAMAllowedPrincipals` and grant column-level access with Lake Formation.
- ☐ Design private S3 access with no internet route and an endpoint policy.
- ☐ Explain why KMS key policies complete the network perimeter.
- ☐ Trace a bad batch to its consumers using lineage.
- ☐ Design an erasure mechanism (crypto-shredding or enumerated deletion) for a stated regulation.

Interview Question Bank

This appendix consolidates the Interview Q&A sections of Chapters 1–24 into one bank, grouped by domain and tagged by difficulty: **(F)** foundational, **(I)** intermediate, **(A)** advanced. Answers are abbreviated to the key points an interviewer listens for; the chapter reference points to the full discussion. Practice out loud—a correct answer you cannot say in ninety seconds is not yet an interview answer.

G.1 Storage and the Data Lake

1. **(F) Why is S3 a better data-lake foundation than cluster-attached disks?** Decouples durable storage from ephemeral compute; many engines read the same data; lifecycle, replication, encryption, and policy are independent of any cluster. (Ch. 1)
2. **(F) When would you avoid One Zone-IA?** For the only copy of critical or regulated data—single-AZ storage suits re-creatable derivatives and secondary copies. (Ch. 1)
3. **(I) What does S3 strong consistency solve—and not solve?** Immediate visibility of completed writes and lists. It is not multi-object transactions and does not repair partial job output. (Ch. 1)
4. **(I) Object Lock: governance or compliance mode?** Governance when a documented administrative bypass exists; compliance for certain, immutable WORM retention. (Ch. 1)
5. **(I) DataSync or Snowball for 10 PB?** Snow Family for the initial bulk transfer; DataSync for ongoing deltas. (Ch. 1)

G.2 Databases and Caching

1. **(F) RDS Multi-AZ versus a Read Replica?** Multi-AZ: synchronous standby for HA/failover. Read replica: asynchronous, for read scaling and migration staging; can lag. (Ch. 2)
2. **(I) Why can Aurora fail over faster than traditional databases?** Compute separates from a shared, replicated cluster volume—promotion needs no volume copy. (Ch. 2)
3. **(I) When is Aurora Global Database justified?** Low-latency cross-region reads or regional DR beyond backups; still needs application failover planning. (Ch. 2)
4. **(I) What does Aurora Serverless v2 not solve?** Schema tuning, connection management, transaction design, maximum-capacity planning. (Ch. 2)

5. **(I) SCT's role in an Oracle→Aurora PostgreSQL migration?** Assess and convert schema; flag manual remediation for incompatible SQL, procedures, types. (Ch. 2)
6. **(A) Why does CDC enable near-zero-downtime migration?** The target stays current while the source serves traffic; cutover waits only for lag and validation thresholds. (Ch. 2)
7. **(F) How is DynamoDB different from a relational database?** Managed key-value/document store modeled around known access patterns; no joins or ad hoc relational queries. (Ch. 3)
8. **(F) What makes a good DynamoDB partition key?** High cardinality, even traffic distribution, groups items queried together; avoids hot keys. (Ch. 3)
9. **(F) When choose on-demand capacity?** Unknown, bursty, early-stage traffic; provisioned + autoscaling for predictable steady load. (Ch. 3)
10. **(I) LSI versus GSI?** LSI shares the partition key, alternate sort key, created with the table. GSI has its own key schema, can be added later, supports new access patterns. (Ch. 3)
11. **(I) Why are sparse GSIs useful?** Only items with the index key appear—efficient queues, status views, entity-type slices. (Ch. 3)
12. **(F) What are DynamoDB Streams for?** Item-level change feed: cache invalidation, materialized views, search indexing, audit, lake ingestion. (Ch. 3)
13. **(I) DynamoDB TTL caveat?** Expiration is asynchronous—fine for cleanup, wrong for exact-time deletion deadlines. (Ch. 3)
14. **(A) What extra design do Global Tables require?** Conflict-aware write rules, regional observability, idempotent consumers, multi-region-aware routing. (Ch. 3)
15. **(F) What problem does ElastiCache solve?** Serves repeated reads from memory so databases/APIs see fewer requests at lower latency. (Ch. 4)
16. **(I) Redis-compatible ElastiCache versus Memcached?** Redis: rich structures, replication, clustering, persistence. Memcached: simple ephemeral key-value caching. (Ch. 4)
17. **(F) What is cache-aside?** Check cache, on miss load from the database, store with TTL; the database remains the source of truth. (Ch. 4)
18. **(A) Why is cache invalidation hard?** Must track every write affecting a key and race readers, writers, retries, and TTL expiry. (Ch. 4)
19. **(I) DAX versus ElastiCache?** DAX: DynamoDB-specific, API-compatible read cache. ElastiCache: general Redis/Valkey/Memcached for many sources. (Ch. 4)
20. **(I) Can DAX serve strongly consistent reads?** No—strong reads bypass DAX to DynamoDB; design read-after-write paths explicitly. (Ch. 4)
21. **(I) AppSync versus API Gateway?** AppSync: GraphQL with typed schemas and subscriptions. API Gateway: REST/HTTP/webhook ingestion. (Ch. 4)
22. **(I) Which metrics show cache health?** Hit ratio, latency, evictions, memory pressure, CPU, connections, replication lag, error rate, downstream database load. (Ch. 4)

G.3 Warehousing and Lakehouse Analytics

1. **(I) Why declare the grain before adding fact columns?** Every measure must be true at exactly one row of the fact; mixed grains double-count. (Ch. 5)
2. **(I) Periodic versus accumulating snapshot facts?** Periodic: state at intervals, never updated. Accumulating: one workflow instance, milestone columns updated as it advances. (Ch. 5)
3. **(F) When is SCD Type 2 required over Type 1?** When the question depends on the attribute value at event time; pure corrections use Type 1. (Ch. 5)
4. **(I) COPY versus Auto-Copy for micro-batch loads?** Auto-Copy monitors a prefix and loads on arrival; manual COPY for controlled, manifest-driven bulk loads. (Ch. 5)
5. **(I) When does Redshift Serverless beat provisioned RA3 on cost?** Spiky/intermittent utilization where idle hours dominate; steady 24/7 amortizes provisioned better. (Ch. 5)
6. **(A) What does the leader node do during COPY?** Plans and assigns input files to compute nodes, which ingest into their slices in parallel; stores no user data. (Ch. 5)
7. **(I) Which distribution style avoids data movement on large joins?** KEY on the join column for both tables—collocated join, nothing crosses the interconnect. (Ch. 6)
8. **(A) Compound versus interleaved sort keys?** Interleaved for varied filter-column combinations; compound for prefix-ordered (typically date-first) access. (Ch. 6)
9. **(I) What does concurrency scaling solve?** Bursty read demand (dashboard refresh storms) with transient read-only clusters, billed per second. (Ch. 6)
10. **(F) When does a materialized view beat a plain view?** Repeated expensive aggregations tolerating a refresh cadence—classic dashboards. (Ch. 6)
11. **(A) Rising WLM queue wait time tells you to change what?** Capacity: concurrency scaling, service-class memory, cheaper queries, or throttled demand—wait time is demand exceeding supply, not a query bug. (Ch. 6)
12. **(I) Why is ALL distribution dangerous for large tables?** Full copy per node: storage multiplies, loads rewrite every copy; justified only for small dimensions. (Ch. 6)
13. **(I) Why does Parquet lower Athena cost versus CSV?** Per-byte-scanned billing; columnar + compressed reads only referenced column blocks—often 10–100× fewer bytes. (Ch. 7)
14. **(I) What does a Glue crawler infer, and when does it infer wrong?** Columns, types, partitions by sampling. Fails on mixed schemas in one prefix or unrepresentative samples. (Ch. 7)
15. **(I) Spectrum versus Athena over the same S3 data?** Spectrum when joining lake with warehouse tables in one statement; Athena for lake-only ad hoc with no cluster. (Ch. 7)
16. **(F) How does partition pruning cut bytes scanned?** Engine skips files outside the partition filter—seven day-partitions instead of five years. (Ch. 7)
17. **(A) What makes Iceberg a lakehouse format beyond Parquet files?** Metadata layer: atomic commits, schema/partition evolution, hidden partitioning, snapshot time travel. (Ch. 7)

18. **(I) Why crawl the curated zone, not the raw zone?** Inference assumes uniform layout; curated is compacted/typed/partitioned. Raw deserves hand-written DDL. (Ch. 7)
19. **(I) What does IAMAllowedPrincipals do, and why revoke it?** Makes new catalog objects IAM-governed, bypassing Lake Formation; until revoked, LF filters do not bind. (Ch. 8)
20. **(I) LF-Tags versus named-resource grants?** Tags bind policy to expressions—every correctly tagged table inherits it; names scale with table count, tags with data classes. (Ch. 8)
21. **(I) What does column-level security protect that table grants cannot?** Sensitive attributes on shareable tables—segments visible, `pii_email` absent, no view or copy. (Ch. 8)
22. **(A) How does Redshift data sharing avoid copies?** Consumers query producer managed storage live and read-only—no ETL, no duplicate storage, no drift. (Ch. 8)
23. **(I) Why is a clean room safer than exchanging CSV exports?** No party receives raw records; approved aggregates only, minimum group sizes, full query logs. (Ch. 8)
24. **(A) Why must both IAM and Lake Formation allow a lake query?** Grants are additive gates: IAM governs API calls, LF governs catalog objects; effective permission is the intersection. (Ch. 8)
25. **(F) What does SPICE accelerate, and its limits?** Dashboard dataset queries via in-memory snapshots; bounded by SPICE allocation and refresh schedule. (Ch. 16)
26. **(I) How does QuickSight row-level security work?** A rules dataset maps users/groups to allowed values, applied at query time—one dataset, many audiences. (Ch. 16)
27. **(F) What is reverse ETL and what does it feed?** Syncs curated warehouse results (segments, scores) back into CRMs, marketing, support systems. (Ch. 16)
28. **(A) Why must identity resolution precede CRM audience syncs?** Warehouse keys must map to the right person; uncontrolled identifiers create mis-targeted outreach. (Ch. 16)
29. **(I) Hourly versus daily reverse-ETL sync?** Weigh API rate limits and cost against freshness value; engineer to the loosest cadence that meets the stated requirement. (Ch. 16)
30. **(I) When is direct query justified over SPICE?** Freshness in minutes and a source that bears per-interaction load—operational monitors, not company-wide dashboards. (Ch. 16)

G.4 ETL, CDC, and Orchestration

1. **(I) Core versus task nodes: which can run on Spot?** Task nodes—no HDFS data, interruption only reschedules tasks. Core/master on Spot risks job or cluster failure. (Ch. 9)
2. **(F) How does EMRFS decouple storage from compute?** S3 as the cluster file system; clusters become disposable compute over durable objects. (Ch. 9)
3. **(I) When EMR on EKS over EMR on EC2?** When Kubernetes is the org's compute standard and Spark should share its scheduling, quotas, networking. (Ch. 9)

4. **(F) What are bootstrap actions for?** Setup scripts on every node at provisioning—reproducible transient clusters from code. (Ch. 9)
5. **(I) Why prefer transient clusters for recurring batch?** Cluster-hours track job-hours; a 2-hour daily job bills 2 node-hours/day, not 24. (Ch. 9)
6. **(I) What stays on local HDFS in a well-designed EMR job?** Job-local scratch (shuffle, spill); all durable inputs/outputs live in S3. (Ch. 9)
7. **(A) What does a DynamicFrame offer over a DataFrame?** Self-describing records with `resolveChoice/applyMapping/relationalize` for messy, evolving schemas and catalog-aware sinks. (Ch. 10)
8. **(I) What do job bookmarks guarantee?** Inputs already processed are not reprocessed. They do not make the sink idempotent. (Ch. 10)
9. **(I) When is Flex execution cheaper?** Non-urgent jobs (backfills, slack-heavy overnight batch) that tolerate delayed starts. (Ch. 10)
10. **(F) Why Glue over EMR for analyst-built pipelines?** No clusters, visual designer generating real code, catalog integration, per-second billing. (Ch. 10)
11. **(F) What compute is one DPU?** 4 vCPU + 16 GB, billed per second with a one-minute minimum. (Ch. 10)
12. **(I) DataBrew's best use?** Interactive profiling and recipe prototyping; also a schedulable quality gate inside a pipeline. (Ch. 10)
13. **(I) Which CloudWatch metrics reveal CDC lag on a "running" task?** `CDCLatencySource` and `CDCLatencyTarget`; task status says nothing about freshness. (Ch. 11)
14. **(A) BACKWARD versus FULL compatibility: which permits field deletion?** BACKWARD (new reads old). FULL forbids deletions old consumers cannot tolerate. (Ch. 11)
15. **(I) Who owns a data contract?** Producer owns and versions the schema; consumers sign off; the registry enforces mechanics, the workflow enforces intent. (Ch. 11)
16. **(I) Why enforce schema at ingestion, not query time?** By query time bad data is durable—and possibly replicated and transformed downstream. (Ch. 11)
17. **(F) One breaking and one non-breaking schema change?** Breaking: delete/rename/retype a field. Non-breaking: add an optional field with a default. (Ch. 11)
18. **(A) Why stage DMS output in S3 before Redshift?** Replayable change log decouples replication from warehouse availability; MERGEs re-runnable and auditable. (Ch. 11)
19. **(I) What does `catchup=False` prevent?** Backfill storms—every missed historical run launching on deploy against production. (Ch. 12)
20. **(I) When are Step Functions better than Airflow?** Event-driven, high-rate workflows of AWS service calls where per-transition billing beats an always-on scheduler. (Ch. 12)
21. **(F) What do retries with backoff protect against?** Transient failures (throttling, brief hiccups). Deterministic failures need the failure path. (Ch. 12)
22. **(I) Why tune a sensor's `poke_interval`?** Each poll is billed LIST requests; match the interval to the expected arrival cadence. (Ch. 12)

23. **(I) When should data move through S3 instead of XComs?** Always, for datasets; XComs (and the 256 KB state-machine payload) suit small signals only. (Ch. 12)
24. **(A) Why does the alert task need `trigger_rule="one_failed"`?** Default rules skip the alert on upstream skip—exactly the runs it exists for. (Ch. 12)

G.5 Streaming

1. **(I) Which two throughput limits size a Kinesis stream?** Per-shard 1 MB/s ingress and 1,000 records/s; shard count is the max plus headroom. (Ch. 13)
2. **(I) Why does partition-key choice control per-entity ordering?** The key hashes to a shard; a shard is the ordered unit. Skewed keys concentrate load. (Ch. 13)
3. **(I) What does rising `IteratorAgeMilliseconds` warn of?** Consumers falling behind finite retention—alarm before the retention horizon. (Ch. 13)
4. **(A) How do you build exactly-once effects on at-least-once delivery?** Idempotent consumers: dedup on event ID or upsert by key; poison records to a DLQ. (Ch. 13)
5. **(I) Kinesis versus MSK billing models?** Kinesis: shard-hours + record volume (capacity). MSK: broker instance-hours + storage (a fleet). (Ch. 13)
6. **(I) When does SQS FIFO suffice and Kinesis is overkill?** One consumer per message, no replay, throughput within FIFO quotas. (Ch. 13)
7. **(F) Name three native Firehose destinations.** S3, Redshift (via intermediate S3 COPY), OpenSearch—plus Splunk and HTTP endpoints. (Ch. 14)
8. **(I) Why enable source-record backup on a delivery stream?** Pre-transformation originals preserved regardless of delivery success: replay, audit, recovery. (Ch. 14)
9. **(I) What do inline Lambda transformations add?** Execution latency per batch plus Lambda GB-seconds and the per-GB surcharge; per-record network calls explode both. (Ch. 14)
10. **(I) Why convert JSON to Parquet during Firehose delivery?** No separate conversion job; the lake is columnar at landing, so scanned-bytes savings apply from the first query. (Ch. 14)
11. **(I) Where does data land when Redshift delivery keeps failing?** The intermediate S3 staging prefix—needs lifecycle rules and a `DeliveryToRedshift.Success` alarm. (Ch. 14)
12. **(A) What can a Firehose Lambda transform never correctly do?** Cross-record state: dedup windows, joins, aggregations—stream-processor work. (Ch. 14)
13. **(A) Why does unbounded keyed state kill a streaming job?** Checkpoints grow until they cannot complete; restores take hours; RocksDB spills. Every stateful operator needs a bound. (Ch. 15)
14. **(I) Checkpoint versus savepoint before redeployment?** Savepoint—manual, stable, portable across graph changes; checkpoints serve automatic recovery. (Ch. 15)
15. **(A) What bounds join-state retention in a stream-stream interval join?** The interval predicate plus watermarks on both inputs; past the watermark's reach, state is dropped. (Ch. 15)

16. **(A) Which two components give end-to-end exactly-once to Kafka or S3?** Checkpointing plus a two-phase-commit sink—or an idempotent-by-key sink achieving the same effect. (Ch. 15)
17. **(I) When prefer at-least-once plus downstream dedup?** Non-transactional sinks, throughput over strictness, or a natural dedup key—simpler, faster, honest. (Ch. 15)
18. **(I) What does a watermark assert?** “No event older than this timestamp will arrive”—a heuristic trading result latency against late-data tolerance. (Ch. 15)

G.6 Machine Learning and MLOps

1. **(I) What does Redshift ML delegate to SageMaker?** Everything past export: unload to S3, Autopilot training/selection, winner compiled back into a SQL function. (Ch. 17)
2. **(F) Why is in-database ML attractive for SQL-only pipelines?** Features, training, scoring live where the data is—no extraction pipelines, no endpoints, no new operational surface. (Ch. 17)
3. **(I) What does CREATE MODEL’s FUNCTION clause define?** The scoring function’s name and signature, mapping arguments to the training query’s feature columns. (Ch. 17)
4. **(I) Why F1 over accuracy for imbalanced churn?** 5% positives: always-negative scores 95% while detecting nothing; F1 balances precision and recall on the minority class. (Ch. 17)
5. **(A) When graduate from Redshift ML to full SageMaker?** Custom architectures, fine hyperparameter control, formal explainability, low-latency real-time inference. (Ch. 17)
6. **(A) What is target leakage and how do you catch it?** A feature unknowable at prediction time in training; review the training query column-by-column against “knowable before the outcome?” (Ch. 17)
7. **(I) Real-time vs. serverless vs. Batch Transform for sporadic traffic?** Serverless: scale to zero, pay per request, accept cold starts. Real-time for steady SLAs; Batch Transform for bulk offline. (Ch. 18)
8. **(F) What does Data Wrangler export?** A repeatable artifact—Processing Job script, pipeline step, or notebook—not a one-off session. (Ch. 18)
9. **(I) When do you need a custom container over built-in XGBoost?** Framework/architecture/dependencies with no managed equivalent; otherwise stay on maintained images. (Ch. 18)
10. **(I) How does Autopilot pick its final model?** Explores preprocessors, algorithms, hyperparameters; evaluates on validation splits against the objective; ranks a winner with inspectable candidate notebooks. (Ch. 18)
11. **(A) Why route training-data access through Lake Formation?** Column masks apply to extraction; PII never reaches staging; every training set is governed and reproducible. (Ch. 18)
12. **(I) What is managed Spot training and its code requirement?** Spare capacity at a steep discount with interruption—requires checkpointing to S3 to resume rather than restart. (Ch. 18)

13. **(I) Online versus offline Feature Store?** Offline (S3, point-in-time joins) builds training sets; online (millisecond `GetRecord`) serves inference; one definition feeds both—the skew defense. (Ch. 19)
14. **(F) What does the Model Registry gate?** Approval: deployment keys on the `Approved` state of a versioned, metrics-carrying package. (Ch. 19)
15. **(A) Data drift versus concept drift?** Data: input distributions change (features alone detect). Concept: input→outcome relationship changes (needs delayed ground truth). (Ch. 19)
16. **(A) Why gate automatic retraining behind human approval?** Drift may mean the world changed *or* ingestion broke; auto-retraining on corrupt data automates worse models. (Ch. 19)
17. **(I) What baseline does Model Monitor need?** Per-feature statistics and constraints from training data, refreshed on a retraining cadence. (Ch. 19)
18. **(I) What does pipeline step caching buy?** Reruns with unchanged inputs reuse outputs—cheaper iteration with input-fingerprint correctness. (Ch. 19)
19. **(F) Textract versus Comprehend for scanned-form key–value pairs?** `Textract AnalyzeDocument FORMS`; `Comprehend` works on already-digital text downstream. (Ch. 20)
20. **(F) Why trigger AI APIs from S3 events via Lambda?** Arrival-driven, no polling, per-object scale-out matching spiky uploads. (Ch. 20)
21. **(I) When switch from sync to async Textract?** Multi-page PDFs/TIFFs; async signals completion via SNS. (Ch. 20)
22. **(I) How make extracted document metadata searchable?** Persist normalized JSON, index into `OpenSearch`, register results in the catalog for SQL. (Ch. 20)
23. **(A) PII risks of persisting raw OCR output?** OCR is faithful—IDs and addresses land in the JSON; restricted zone, field-level redaction, at-rest discovery scanning. (Ch. 20)
24. **(I) How keep retried pipelines from re-billing API calls?** Idempotency keyed on object identity (etag/version); skip already-processed inputs. (Ch. 20)

G.7 Security, Governance, and Compliance

1. **(I) Gateway versus interface VPC endpoint for S3?** Gateway: free route-table entry for S3/DynamoDB. Interface (`PrivateLink`): other services, billed per AZ-hour plus data. (Ch. 21)
2. **(I) What does a permissions boundary limit?** A role’s maximum permissions regardless of identity policies—the intersection is effective; makes delegated role creation safe. (Ch. 21)
3. **(F) Why is automatic Secrets Manager rotation better than static keys?** A leaked credential’s value is bounded to the rotation window; static keys leak forever. (Ch. 21)
4. **(I) How do SCPs differ from IAM policies?** SCPs cap what any account—including its admins—may do; IAM policies grant within an account. (Ch. 21)

5. **(A) What exfiltration path does PrivateLink close that security groups cannot?** Endpoint policies restrict the *service destinations* themselves—your network cannot write to a stranger’s bucket. (Ch. 21)
6. **(A) Why does KMS complete the perimeter?** Key policies restrict decryption to org principals; copied data arrives as unusable ciphertext. (Ch. 21)
7. **(I) How do LF-Tags simplify cross-account sharing?** Grants bind to tag expressions; correctly tagged shared tables inherit policy in every account. (Ch. 22)
8. **(I) Why consult lineage when a quality check fails?** Names what consumed the bad batch and which upstream change introduced it—a directed fix, not a two-day hunt. (Ch. 22)
9. **(I) ColumnExists versus Completeness?** Structure versus content; a production ruleset needs both families. (Ch. 22)
10. **(F) In a data mesh, who owns a data product’s quality?** The publishing domain; the center provides platform, taxonomy, audit. (Ch. 22)
11. **(I) What does a DQ rule’s threshold clause buy?** Distinguishes a mild dip (report) from a cliff (fail)—monitoring without halting on noise. (Ch. 22)
12. **(A) Why enforce contracts at registration, not query time?** Registration declares producer intent; query time means violations are already durable bad data. (Ch. 22)
13. **(I) What does Macie detect beyond regex patterns?** Checksums (Luhn), proximity keywords, format validation; also bucket posture inventory. (Ch. 23)
14. **(I) Why quarantine suspect files with Object Lock instead of deleting?** Preserves immutable evidence for investigation and audit; deletion can violate retention. (Ch. 23)
15. **(A) Tokenization versus hashing: which is reversible?** Tokenization, via the controlled vault. Low-entropy hashes are effectively reversible to attackers—use real tokenization when re-identification is required. (Ch. 23)
16. **(I) What do CloudTrail data events record for S3?** Object-level API activity per principal and object—the lake’s access ledger, delivered to an independent log-archive account. (Ch. 23)
17. **(I) Why tune Macie custom identifiers before rollout?** False-positive economics: over-broad identifiers train teams to ignore alerts. Tune on samples; review precision monthly. (Ch. 23)
18. **(A) Why is crypto-shredding superior to enumerated deletion?** One key deletion renders every copy—versions, replicas, snapshots, archives—unrecoverable in a single auditable act. (Ch. 23)

G.8 DataOps and Delivery

1. **(I) Why store Terraform state remotely with locking?** Shared state is the source of truth; locking prevents divergent applies; local state is lost with laptops and leaks attribute values. (Ch. 24)
2. **(I) What do dbt tests validate that schemas cannot?** Semantics: uniqueness, referential integrity, accepted ranges, freshness. Schema guarantees shape; tests guarantee the data is right. (Ch. 24)

3. **(I) Why promote dbt runs Dev→Prod instead of one shared workspace?** Every change is rehearsed against a real environment; one shared workspace makes production the rehearsal. (Ch. 24)
4. **(A) Why prefer OIDC federation over long-lived AWS keys in CI?** Short-lived, audience-scoped credentials per run—nothing stored to leak, nothing to rotate. (Ch. 24)
5. **(I) What must the PR-check workflow prove before merge?** Reviewable terraform plan, dbt build passing against CI data, policy checks green—main is always deployable. (Ch. 24)
6. **(A) Summarize expand-contract in one breath.** Add new alongside old; dual-write and backfill while reads stay old; switch reads behind a shim; verify consumers; drop the old column in a later release. (Ch. 24)

Tip

Interviewers score the *reasoning*, not the product name. Answer in the pattern this book teaches: state the constraint, name the trade-off, then choose—and mention what you would monitor to know the choice was wrong.

Glossary

This glossary grows with each chapter and collects the cross-cloud data-engineering terms introduced so far. Definitions are platform-neutral unless a platform is named explicitly.

Access tier A storage pricing class that trades retrieval cost and latency for lower at-rest price (for example Azure Hot, Cool, Cold, and Archive). See also *storage class*.

AlloyDB Google Cloud’s PostgreSQL-compatible engine with a columnar accelerator for analytical queries, positioned as the cloud-native tier above Cloud SQL.

Apache Spark A distributed data-processing engine; Microsoft Fabric runs it on Starter or Custom pools, with counterparts in AWS Glue/EMR, Azure Synapse/Databricks, and GCP Dataproc.

Aurora Amazon’s cloud-native relational engine with distributed storage, Global Databases, and a serverless (v2) compute option.

Availability SLA The contractual uptime a provider guarantees for a service, distinct from *durability*, which concerns data loss rather than reachability.

Bigtable Google Cloud’s wide-column, high-throughput, low-latency NoSQL store for time-series, IoT, and ad-tech workloads; exposes the HBase API and scales by nodes and clusters.

Bucket (storage) The top-level container for objects in AWS S3 and Google Cloud Storage; Azure calls the equivalent a *container*.

Bucket Pattern A MongoDB schema-design pattern that groups many small time-series readings into a single document to cut document count and metadata overhead.

Capped collection A fixed-size MongoDB collection that overwrites its oldest documents when full, useful for high-volume logs and bounded activity feeds.

Change Feed / Streams An ordered log of item-level changes used for event-driven processing; called Change Feed in Cosmos DB, DynamoDB Streams in DynamoDB, and change streams in MongoDB.

Checkpoint In WiredTiger, a periodic flush of a consistent snapshot of cached data to disk, bounding crash-recovery time together with the journal.

Clustering key A user-defined column set that Snowflake uses to co-locate related rows within micro-partitions, improving partition pruning on selective queries.

CMEK / CMK Customer-managed encryption keys (GCP CMEK; Azure customer-managed keys via Key Vault), giving the customer control over the key that encrypts data at rest.

Cloud Spanner Google Cloud’s horizontally scalable, globally distributed relational database offering external (strong) consistency backed by TrueTime.

Consistency level A tunable guarantee about read recency; Azure Cosmos DB offers five (Strong, Bounded Staleness, Session, Consistent Prefix, Eventual).

Cosmos DB Azure’s globally distributed multi-model NoSQL database, billed in Request Units (RU) and supporting multiple APIs (NoSQL, MongoDB, Cassandra).

Cross-Region Replication (CRR) Asynchronous copying of objects to a bucket in another region for disaster recovery; Azure’s equivalent user-configured feature is *object replication*.

DynamoDB Amazon’s managed key-value and document NoSQL database, with capacity measured in read/write capacity units (RCU/WCU) or on-demand, and Global Tables for multi-region.

Durability The probability that stored data is not lost, commonly quoted as “eleven nines” (99.99999999%) for cloud object storage.

ElastiCache Amazon’s managed in-memory cache (Redis- and Memcached-compatible) placed in front of a durable store to serve hot data at sub-millisecond latency; Azure’s analog is Cache for Redis and GCP’s is Memorystore.

Embedding vs referencing The MongoDB modeling choice between nesting related data in one document (embedding) or linking documents by reference (and joining with `lookup`).

Fail-safe Snowflake’s non-configurable seven-day recovery window, available only to Snowflake support, that follows the Time Travel retention period as a last-resort safety net.

Global Secondary Index (GSI) A DynamoDB index with its own partition/sort key enabling queries on non-primary attributes; a Local Secondary Index (LSI) shares the table’s partition key.

Hierarchical namespace (HNS) The Azure Data Lake Storage Gen2 feature that adds true directories and atomic directory operations on top of Blob storage.

Journal (WAL) The write-ahead log that records changes before they are checkpointed, enabling crash recovery in MongoDB’s WiredTiger engine.

JSON Schema validation MongoDB’s rule mechanism (via `jsonSchema` in `createCollection`) that enforces required fields, data types, and value constraints.

Interleaved table A Cloud Spanner layout that physically co-locates child rows with their parent row to make parent-child joins fast.

Lakehouse An architecture that stores open table formats (such as Delta) in a data lake while offering warehouse-like SQL and governance; the core storage abstraction in Microsoft Fabric.

Lifecycle policy A rule set that automatically transitions or expires objects across storage tiers based on age or other conditions.

Medallion architecture A progressive-refinement data layout with Bronze (raw), Silver (validated), and Gold (curated/enriched) layers.

Materialized view A precomputed, automatically maintained query result stored for fast reads, trading maintenance/storage cost for lower query latency (Snowflake, Redshift, Synapse, BigQuery).

Memorystore Google Cloud’s managed in-memory cache service (Redis and Memcached), the GCP counterpart to AWS ElastiCache and Azure Cache for Redis.

- Micro-partition** Snowflake's internal storage unit: an immutable, compressed, columnar file of roughly 50–500 MB of uncompressed data, annotated with per-column metadata for pruning.
- Multi-AZ** An AWS deployment spanning multiple Availability Zones for high availability; Azure's analog is zone-redundant HA and GCP's is regional HA.
- Multi-cluster warehouse** A Snowflake virtual warehouse that adds and removes compute clusters automatically to absorb concurrent query load.
- Object Lock** The AWS S3 WORM feature (governance mode, compliance mode, and legal hold) that prevents object deletion or overwrite for a retention period.
- OneLake** Microsoft Fabric's single, tenant-wide logical data lake shared by all workspaces.
- Partition pruning** Skipping storage units (micro-partitions, files, or partitions) that cannot match a query's predicates, reducing scanned data.
- Point-in-time recovery (PITR)** Restoring a database or table to any moment within a retention window, using continuous backups or transaction logs; realized as Snowflake Time Travel, RDS/SQL Database PITR, and Delta Lake time travel.
- Read replica** A read-only copy of a database that offloads read traffic and can serve as a failover or geo-local endpoint.
- Relational Migrator** MongoDB's tool for migrating relational schemas and data to document collections, applying embed-versus-reference decisions from schema-design patterns.
- Request Unit (RU)** Azure Cosmos DB's normalized currency for throughput, abstracting the CPU, memory, and IOPS consumed by a database operation.
- Search Optimization Service (SOS)** A Snowflake feature that builds a persistent search access path to accelerate selective point-lookup queries on high-cardinality columns.
- Shortcut (Fabric)** A OneLake reference that virtualizes external data (S3, ADLS Gen2, GCS) in place so it can be queried without copying.
- Storage class** The AWS/GCP term for a pricing tier of object storage (for example S3 Standard, Standard-IA, Glacier; GCP Standard, Nearline, Coldline, Archive).
- SSE-KMS** Server-side encryption in AWS S3 using keys managed in AWS Key Management Service.
- Time Travel** Snowflake's ability to query, clone, or restore data as it existed at an earlier point within a retention period (up to 90 days on higher editions); Delta Lake offers an analogous `VERSION AS OF / TIMESTAMP AS OF` capability.
- Transient table** A Snowflake table that persists across sessions but has no Fail-safe period, reducing storage cost for reproducible staging data; a temporary table lives only for one session.
- TrueTime** Google's globally synchronized clock (with bounded uncertainty) that lets Cloud Spanner assign commit timestamps and provide external consistency.
- TTL (Time to Live)** An expiry mechanism that automatically deletes items or documents after a set time (DynamoDB TTL, Cosmos DB TTL, MongoDB TTL index).
- Versioning** Retaining prior versions of an object or blob so that overwrites and deletes are recoverable.

Virtual warehouse Snowflake's unit of elastic compute, sized XS–6XL and billed in credits, fully decoupled from storage.

Wide-column store A NoSQL data model that stores rows as sparse, dynamically wide sets of column families keyed by a row key (Bigtable, HBase, Cassandra), optimized for high write throughput and range scans.

WiredTiger MongoDB's default storage engine, using a configurable internal cache alongside the operating-system page cache, with journaling and checkpointing.

WORM Write-once-read-many storage that prevents modification or deletion for a retention period; implemented as Object Lock (AWS), immutable blob storage (Azure), or Bucket Lock (GCP).

Z-ordering A Delta Lake optimization that co-locates related information in the same files by multi-dimensional clustering, improving data skipping; applied via `OPTIMIZE ... ZORDER BY` and complemented by `VACUUM` to reclaim obsolete files.

Zero-copy clone Snowflake's instant, storage-free copy of a table, schema, or database that shares underlying micro-partitions until either copy is modified (copy-on-write).

Bibliography

- [1] Tyler Akidau, Slava Chernyak, and Reuven Lax. *Streaming Systems: The What, Where, When, and How of Large-Scale Data Processing*. O'Reilly Media, 2018.
- [2] Amazon Web Services. Amazon s3 strong consistency. <https://aws.amazon.com/s3/consistency/>, 2020. AWS feature announcement and documentation.
- [3] Amazon Web Services. Aws well-architected framework: Analytics lens. <https://docs.aws.amazon.com/wellarchitected/latest/analytics-lens/analytics-lens.html>, 2024. AWS Well-Architected guidance.
- [4] Amazon Web Services. Amazon api gateway developer guide. <https://docs.aws.amazon.com/apigateway/latest/developerguide/welcome.html>, 2026. Accessed 2026-07-31.
- [5] Amazon Web Services. Amazon athena user guide. <https://docs.aws.amazon.com/athena/latest/ug/what-is.html>, 2026. Accessed 2026-08-31.
- [6] Amazon Web Services. Amazon aurora storage and reliability. <https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/Aurora.Overview.StorageReliability.html>, 2026. Accessed 2026-07-31.
- [7] Amazon Web Services. Amazon aurora user guide. https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/CHAP_AuroraOverview.html, 2026. Accessed 2026-07-31.
- [8] Amazon Web Services. Amazon comprehend developer guide. <https://docs.aws.amazon.com/comprehend/latest/dg/what-is.html>, 2026. Accessed 2026-08-31.
- [9] Amazon Web Services. Amazon data firehose developer guide. <https://docs.aws.amazon.com/firehose/latest/dev/what-is-this-service.html>, 2026. Accessed 2026-08-31.
- [10] Amazon Web Services. Amazon dynamodb accelerator developer guide. <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/DAX.html>, 2026. Accessed 2026-07-31.
- [11] Amazon Web Services. Amazon dynamodb developer guide. <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Introduction.html>, 2026. Accessed 2026-07-31.
- [12] Amazon Web Services. Amazon elasticache user guide. <https://docs.aws.amazon.com/AmazonElastiCache/latest/dg/WhatIs.html>, 2026. Accessed 2026-07-31.
- [13] Amazon Web Services. Amazon emr management guide. <https://docs.aws.amazon.com/emr/latest/ManagementGuide/emr-what-is-emr.html>, 2026. Accessed 2026-08-31.
- [14] Amazon Web Services. Amazon eventbridge user guide. <https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-what-is.html>, 2026. Accessed 2026-08-31.

- [15] Amazon Web Services. Amazon kinesis data streams developer guide. <https://docs.aws.amazon.com/kinesis/latest/dev/introduction.html>, 2026. Accessed 2026-08-31.
- [16] Amazon Web Services. Amazon macie user guide. <https://docs.aws.amazon.com/macie/latest/user/what-is-macie.html>, 2026. Accessed 2026-08-31.
- [17] Amazon Web Services. Amazon managed service for apache flink developer guide. <https://docs.aws.amazon.com/managed-flink/latest/java/what-is.html>, 2026. Accessed 2026-08-31.
- [18] Amazon Web Services. Amazon managed streaming for apache kafka developer guide. <https://docs.aws.amazon.com/msk/latest/developerguide/what-is-msk.html>, 2026. Accessed 2026-08-31.
- [19] Amazon Web Services. Amazon managed workflows for apache airflow user guide. <https://docs.aws.amazon.com/mwaa/latest/userguide/what-is-mwaa.html>, 2026. Accessed 2026-08-31.
- [20] Amazon Web Services. Amazon opensearch service developer guide. <https://docs.aws.amazon.com/opensearch-service/latest/developerguide/what-is.html>, 2026. Accessed 2026-08-31.
- [21] Amazon Web Services. Amazon quicksight user guide. <https://docs.aws.amazon.com/quicksight/latest/user/welcome.html>, 2026. Accessed 2026-08-31.
- [22] Amazon Web Services. Amazon redshift database developer guide. <https://docs.aws.amazon.com/redshift/latest/dg/welcome.html>, 2026. Accessed 2026-08-31.
- [23] Amazon Web Services. Amazon redshift ml. <https://docs.aws.amazon.com/redshift/latest/dg/machine-learning.html>, 2026. Accessed 2026-08-31.
- [24] Amazon Web Services. Amazon redshift serverless. <https://docs.aws.amazon.com/redshift/latest/mgmt/serverless-what-is.html>, 2026. Accessed 2026-08-31.
- [25] Amazon Web Services. Amazon rekognition developer guide. <https://docs.aws.amazon.com/rekognition/latest/dg/what-is.html>, 2026. Accessed 2026-08-31.
- [26] Amazon Web Services. Amazon relational database service user guide. <https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Welcome.html>, 2026. Accessed 2026-07-31.
- [27] Amazon Web Services. Amazon sagemaker developer guide. <https://docs.aws.amazon.com/sagemaker/latest/dg/whatis.html>, 2026. Accessed 2026-08-31.
- [28] Amazon Web Services. Amazon sagemaker feature store. <https://docs.aws.amazon.com/sagemaker/latest/dg/feature-store.html>, 2026. Accessed 2026-08-31.
- [29] Amazon Web Services. Amazon sagemaker model monitor. <https://docs.aws.amazon.com/sagemaker/latest/dg/model-monitor.html>, 2026. Accessed 2026-08-31.
- [30] Amazon Web Services. Amazon sagemaker pipelines. <https://docs.aws.amazon.com/sagemaker/latest/dg/pipelines.html>, 2026. Accessed 2026-08-31.
- [31] Amazon Web Services. Amazon simple notification service developer guide. <https://docs.aws.amazon.com/sns/latest/dg/welcome.html>, 2026. Accessed 2026-08-31.
- [32] Amazon Web Services. Amazon simple queue service developer guide. <https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/welcome.html>, 2026. Accessed 2026-08-31.

- [33] Amazon Web Services. Amazon simple storage service user guide. <https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html>, 2026. Accessed 2026-07-30.
- [34] Amazon Web Services. Amazon textract developer guide. <https://docs.aws.amazon.com/textract/latest/dg/what-is.html>, 2026. Accessed 2026-08-31.
- [35] Amazon Web Services. Aws appsync developer guide. <https://docs.aws.amazon.com/appsync/latest/devguide/what-is-appsync.html>, 2026. Accessed 2026-07-31.
- [36] Amazon Web Services. Aws certified data engineer – associate exam guide. <https://aws.amazon.com/certification/certified-data-engineer-associate/>, 2026. Accessed 2026-08-31.
- [37] Amazon Web Services. Aws clean rooms user guide. <https://docs.aws.amazon.com/clean-rooms/latest/ug/what-is.html>, 2026. Accessed 2026-08-31.
- [38] Amazon Web Services. Aws cloud development kit (aws cdk) developer guide. <https://docs.aws.amazon.com/cdk/v2/guide/home.html>, 2026. Accessed 2026-08-31.
- [39] Amazon Web Services. Aws cloudtrail user guide. <https://docs.aws.amazon.com/awscloudtrail/latest/userguide/cloudtrail-user-guide.html>, 2026. Accessed 2026-08-31.
- [40] Amazon Web Services. Aws command line interface version 2 user guide. <https://docs.aws.amazon.com/cli/latest/userguide/cli-chap-welcome.html>, 2026. Accessed 2026-08-31.
- [41] Amazon Web Services. Aws config developer guide. <https://docs.aws.amazon.com/config/latest/developerguide/WhatIsConfig.html>, 2026. Accessed 2026-08-31.
- [42] Amazon Web Services. Aws database migration service user guide. <https://docs.aws.amazon.com/dms/latest/userguide/Welcome.html>, 2026. Accessed 2026-07-31.
- [43] Amazon Web Services. Aws datasync user guide. <https://docs.aws.amazon.com/datasync/latest/userguide/what-is-datasync.html>, 2026. Accessed 2026-07-30.
- [44] Amazon Web Services. Aws glue data catalog and crawlers. <https://docs.aws.amazon.com/glue/latest/dg/catalog-and-crawler.html>, 2026. Accessed 2026-08-31.
- [45] Amazon Web Services. Aws glue data quality. <https://docs.aws.amazon.com/glue/latest/dg/glue-data-quality.html>, 2026. Accessed 2026-08-31.
- [46] Amazon Web Services. Aws glue databrew developer guide. <https://docs.aws.amazon.com/databrew/latest/dg/what-is.html>, 2026. Accessed 2026-08-31.
- [47] Amazon Web Services. Aws glue developer guide. <https://docs.aws.amazon.com/glue/latest/dg/what-is-glue.html>, 2026. Accessed 2026-08-31.
- [48] Amazon Web Services. Aws glue schema registry. <https://docs.aws.amazon.com/glue/latest/dg/schema-registry.html>, 2026. Accessed 2026-08-31.
- [49] Amazon Web Services. Aws identity and access management user guide. <https://docs.aws.amazon.com/IAM/latest/UserGuide/introduction.html>, 2026. Accessed 2026-08-31.
- [50] Amazon Web Services. Aws key management service developer guide. <https://docs.aws.amazon.com/kms/latest/developerguide/overview.html>, 2026. Accessed 2026-08-31.

- [51] Amazon Web Services. Aws lake formation developer guide. <https://docs.aws.amazon.com/lake-formation/latest/dg/what-is-lake-formation.html>, 2026. Accessed 2026-08-31.
- [52] Amazon Web Services. Aws lambda developer guide. <https://docs.aws.amazon.com/lambda/latest/dg/welcome.html>, 2026. Accessed 2026-08-31.
- [53] Amazon Web Services. Aws privatelink guide. <https://docs.aws.amazon.com/vpc/latest/privatelink/what-is-privatelink.html>, 2026. Accessed 2026-08-31.
- [54] Amazon Web Services. Aws schema conversion tool user guide. https://docs.aws.amazon.com/SchemaConversionTool/latest/userguide/CHAP_Welcome.html, 2026. Accessed 2026-07-31.
- [55] Amazon Web Services. Aws secrets manager user guide. <https://docs.aws.amazon.com/secretsmanager/latest/userguide/intro.html>, 2026. Accessed 2026-08-31.
- [56] Amazon Web Services. Aws snow family documentation. <https://docs.aws.amazon.com/snowball/latest/developer-guide/whatisedge.html>, 2026. Accessed 2026-07-30.
- [57] Amazon Web Services. Aws step functions developer guide. <https://docs.aws.amazon.com/step-functions/latest/dg/welcome.html>, 2026. Accessed 2026-08-31.
- [58] Amazon Web Services. Aws storage gateway user guide. <https://docs.aws.amazon.com/storagegateway/latest/userguide/WhatIsStorageGateway.html>, 2026. Accessed 2026-07-30.
- [59] Amazon Web Services. Best practices for aws database migration service. https://docs.aws.amazon.com/dms/latest/userguide/CHAP_BestPractices.html, 2026. Accessed 2026-07-31.
- [60] Amazon Web Services. Best practices for designing and architecting with dynamodb. <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/best-practices.html>, 2026. Accessed 2026-07-31.
- [61] Amazon Web Services. Boto3 documentation: The aws sdk for python. <https://boto3.amazonaws.com/v1/documentation/api/latest/index.html>, 2026. Accessed 2026-08-31.
- [62] Amazon Web Services. Creating materialized views in amazon redshift. <https://docs.aws.amazon.com/redshift/latest/dg/materialized-view-overview.html>, 2026. Accessed 2026-08-31.
- [63] Amazon Web Services. Dynamodb global tables. <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/GlobalTables.html>, 2026. Accessed 2026-07-31.
- [64] Amazon Web Services. Dynamodb read and write capacity mode. <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.ReadWriteCapacityMode.html>, 2026. Accessed 2026-07-31.
- [65] Amazon Web Services. Dynamodb streams and time to live. <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Streams.html>, 2026. Accessed 2026-07-31.
- [66] Amazon Web Services. High availability for amazon rds db instances. <https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZ.html>, 2026. Accessed 2026-07-31.

- [67] Amazon Web Services. Implementing workload management in amazon redshift. <https://docs.aws.amazon.com/redshift/latest/dg/cm-c-implementing-workload-management.html>, 2026. Accessed 2026-08-31.
- [68] Amazon Web Services. Improving data access with secondary indexes in dynamodb. <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/SecondaryIndexes.html>, 2026. Accessed 2026-07-31.
- [69] Amazon Web Services. Loading data with the copy command. https://docs.aws.amazon.com/redshift/latest/dg/r_COPY.html, 2026. Accessed 2026-08-31.
- [70] Amazon Web Services. Locking objects with s3 object lock. <https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lock.html>, 2026. Accessed 2026-07-30.
- [71] Amazon Web Services. Optimizing query performance in amazon redshift. <https://docs.aws.amazon.com/redshift/latest/dg/c-optimizing-query-performance.html>, 2026. Accessed 2026-08-31.
- [72] Amazon Web Services. Querying external data with amazon redshift spectrum. <https://docs.aws.amazon.com/redshift/latest/dg/c-using-spectrum.html>, 2026. Accessed 2026-08-31.
- [73] Amazon Web Services. Replicating objects. <https://docs.aws.amazon.com/AmazonS3/latest/userguide/replication.html>, 2026. Accessed 2026-07-30.
- [74] Amazon Web Services. Sharing data across clusters in amazon redshift. <https://docs.aws.amazon.com/redshift/latest/dg/datashare-overview.html>, 2026. Accessed 2026-08-31.
- [75] Amazon Web Services. Unloading data with the unload command. https://docs.aws.amazon.com/redshift/latest/dg/r_UNLOAD.html, 2026. Accessed 2026-08-31.
- [76] Amazon Web Services. Using amazon aurora global database. <https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/aurora-global-database.html>, 2026. Accessed 2026-07-31.
- [77] Amazon Web Services. Using amazon aurora serverless v2. <https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/aurora-serverless-v2.html>, 2026. Accessed 2026-07-31.
- [78] Amazon Web Services. Using amazon s3 storage classes. <https://docs.aws.amazon.com/AmazonS3/latest/userguide/storage-class-intro.html>, 2026. Accessed 2026-07-30.
- [79] Amazon Web Services. Working with db instance read replicas. https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_ReadRepl.html, 2026. Accessed 2026-07-31.
- [80] Apache Software Foundation. Apache airflow documentation. <https://airflow.apache.org/docs/>, 2026. Accessed 2026-08-31.
- [81] Apache Software Foundation. Apache flink documentation. <https://nightlies.apache.org/flink/flink-docs-stable/>, 2026. Accessed 2026-08-31.
- [82] Apache Software Foundation. Apache iceberg: The definitive guide. <https://iceberg.apache.org/>, 2026. Accessed 2026-08-31.
- [83] Apache Software Foundation. Apache kafka documentation. <https://kafka.apache.org/documentation/>, 2026. Accessed 2026-08-31.

- [84] Apache Software Foundation. Apache spark: Unified analytics engine. <https://spark.apache.org/>, 2026. Accessed 2026-08-31.
- [85] Michael Armbrust, Ali Ghodsi, Reynold Xin, and Matei Zaharia. Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics. In *Conference on Innovative Data Systems Research (CIDR)*, 2021.
- [86] DataHub Project. Datahub: Open source metadata platform. <https://datahubproject.io/>, 2026. Accessed 2026-08-31.
- [87] dbt Labs. dbt core documentation. <https://docs.getdbt.com/>, 2026. Accessed 2026-08-31.
- [88] Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati, Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Voss, and Werner Vogels. Dynamo: Amazon’s highly available key-value store. In *Proceedings of Twenty-First ACM SIGOPS Symposium on Operating Systems Principles*, pages 205–220, 2007.
- [89] GitHub. Github actions documentation. <https://docs.github.com/actions>, 2026. Accessed 2026-08-31.
- [90] Great Expectations Project. Great expectations: Data quality framework. <https://greatexpectations.io/>, 2026. Accessed 2026-08-31.
- [91] HashiCorp. Terraform aws provider documentation. <https://registry.terraform.io/providers/hashicorp/aws/latest/docs>, 2026. Accessed 2026-08-31.
- [92] Ralph Kimball and Margy Ross. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. Wiley, 3 edition, 2013.
- [93] OpenLineage Project. Openlineage: An open standard for metadata and lineage collection. <https://openlineage.io/>, 2026. Accessed 2026-08-31.
- [94] Matei Zaharia, Reynold S. Xin, Patrick Wendell, Tathagata Das, Michael Armbrust, Ankur Dave, Xiangrui Meng, Josh Rosen, Shivaram Venkatesan, Michael J. Franklin, Ali Ghodsi, Joseph Gonzalez, Scott Shenker, and Ion Stoica. Apache spark: A unified engine for big data processing. *Communications of the ACM*, 59(11):56–65, 2016.

Index

- acceptance criteria, 316
- access pattern, 21, 59
- account setup
 - equivalence, 295
- accumulating snapshot, 95
- active-active, 63
- adaptive capacity, 61
- AI services
 - equivalence, 303
- Airflow
 - equivalence, 299
- ALL distribution, 108
- allowed lateness, 199
- Amazon API Gateway, 73, 77
- Amazon Athena, 117
- Amazon Aurora, 40
- Amazon Comprehend, 243
- Amazon Data Firehose, 187
- Amazon DynamoDB, 58
- Amazon ElastiCache, 73
- Amazon EMR, 139
- Amazon EventBridge, 157
- Amazon Macie, 271
- Amazon MSK, 177
- Amazon MWAA, 166
- Amazon OpenSearch, 243
- Amazon QuickSight, 206
- Amazon RDS, 40
- Amazon Redshift, 90
 - performance, 107
- Amazon Rekognition, 243
- Amazon S3, 17
- Amazon SageMaker, 225
- Amazon SNS, 177
- Amazon SQS, 177
- Amazon States Language, 168
- Amazon Textract, 243
- Amazon Translate, 243
- ANALYZE COMPRESSION, 110
- anomaly detection, 206, 209
- anonymization, 187
- Apache Airflow, 166
- Apache Flink, 197
- Apache Iceberg, 117, 120
- Apache Kafka, 177
- Apache Parquet, 117
- Apache Spark, 139
 - equivalence, 293
- API Gateway cache, 80
- API throttling, 78
- AppSync subscription, 80
- asynchronous jobs, 244
- asynchronous replication, 43
- Athena
 - capstone, 124
 - commands, 311
- Athena data source, 208
- Athena workgroup, 118
- attribute, 60
- attribute-based access control, 130
- Aurora cache, 75
- Aurora Capacity Unit, 45
- Aurora cluster volume, 44
- Aurora Global Database, 40, 44
 - capstone, 51
- Aurora PostgreSQL, 46, 48
 - capstone, 51
- Aurora Serverless v2, 40, 45, 83
- AUTO distribution, 108
- Auto-Copy, 95
- automated backups, 42
- Autopilot, 227
- availability, 17, 20
- Availability Zone, 20, 42
- AWS account, 1
- AWS AppSync, 73, 77
- AWS Budgets, 1, 308
- AWS CDK, 307
- AWS Certified Data Engineer, 328
- AWS Clean Rooms, 127
- AWS CLI, 1, 24, 46, 63, 78, 306
 - commands, 310
 - profiles, 307
- AWS CloudShell, 1

- AWS CloudTrail, 271
- AWS Config, 271, 274
- AWS Database Migration Service, 40
- AWS DataSync, 23
- AWS DMS, 45, 83, 157
- AWS Glue, 148
 - capstone, 155
- AWS Glue Data Catalog, 117
- AWS IAM, 253
- AWS KMS, 253
- AWS Lake Formation, 127
- AWS Lambda, 157
- AWS PrivateLink, 253
- AWS RAM, 263
- AWS Schema Conversion Tool, 40
- AWS SCT, 45
- AWS Step Functions, 166
- az64, 110

- backup retention, 42
- baseline, 236
- Batch Transform, 225, 227
- BI
 - equivalence, 301
- Bigtable
 - equivalence, 294
- bootstrap action, 142
- Boto3, 46, 63, 306, 307
- bucket, 18
- buffer hints, 188
- buffering, 189
- bus matrix, 95
- bytes scanned, 118

- cache hit ratio, 76
- cache invalidation, 75
- cache warming, 77
- cache-aside, 73
- cache-aside pattern, 75
- caching, 73
 - equivalence, 294
- capstone project, 30, 51, 67, 103, 115, 124, 135, 146, 155, 164, 173, 184, 194, 204, 213, 223, 232, 241, 250, 260, 269, 278, 287
- case studies, 13
- catchup, 167
- CDC, 158
 - capstone, 164
 - equivalence, 298
- certification, 280
 - AWS Data Engineer Associate, 285
 - exam mapping, 328
- change data capture, 45, 62, 157
- checklist
 - cost, 327
 - ETL, 325
 - ML, 326
 - security, 326
 - storage, 324
 - streaming, 325
 - warehousing, 324
- checklists, 324
- checkpoint, 197, 200
- checkpointing, 179
- churn prediction, 217
- CI/CD, 280
- class imbalance, 219
- classification, 219
- clean room
 - equivalence, 297
- cloning
 - equivalence, 294
- CloudShell, 307
- CloudTrail, 274
- collocated join, 108
- column-level security, 127, 129
- columnar storage, 92
- community resources, 13
- compatibility mode, 159
- compliance mode, 22
- composite key, 61
- compound sort key, 109
- Comprehend, 245
- compression encoding, 92, 107, 110
- compute node, 92
- concept drift, 234
- concurrency scaling, 107, 110
- conflict resolution, 63
- conformed dimension, 95
- connection pooling, 45
- consistency, 20
- COPY, 90, 94
- core node, 140
- Cost Explorer, 1, 308
- cost guardrails, 306
 - equivalence, 295
- crawler, 119
- crawlers, 117
- CREATE MODEL, 217, 218
- cross-account BI, 208
- cross-account sharing, 262, 263
- cross-cloud equivalence, 290
- Cross-Region Replication, 22

- capstone, 30
- cross-region replication, 44
- crypto-shredding, 271, 273
- custom container, 227
- custom data identifier, 272
- customer-managed key, 255
- cutover, 48
- DAG, 166
- data classification, 271
- data clean room, 127
- data contract, 158
- data contracts, 157
 - capstone, 164
- data delivery, 187
- data drift, 234
- data events, 274
- data filter, 129
- data governance, 262
- data lake, 19
- data lakehouse, 117
- data lineage, 262
- data mesh, 262
 - capstone, 269, 287
- data perimeter, 253
 - capstone, 260
- data profiling, 150
- data quality, 262
 - equivalence, 304
- data residency, 19
- data retention, 62
- data skew, 109
- data warehouse, 90
 - equivalence, 295
- Data Wrangler, 225, 226
- DataFrame conversion, 150
- DataHub, 264
- DataOps, 280
 - equivalence, 305
- DataSync, 17
- DAX, 64, 73, 76
 - capstone, 83
- DAX cluster, 77
- DB instance, 42
- dbt, 280
 - equivalence, 305
- dead-letter queue, 178
- decoupled storage, 141
- delete marker, 21
- delivery stream, 188
- denormalization, 61
- DICOM, 27
- dimensional modeling, 90
 - capstone, 103
- disaster recovery, 27
- distributed databases
 - equivalence, 292
- distributed storage, 44
- distribution key, 109
- distribution style, 107, 108
 - equivalence, 296
- DMS
 - capstone, 51
- DMS endpoint, 45
- DMS to S3, 158
- document database, 58
- document modeling
 - equivalence, 292
- DPU, 148, 151
- DQDL, 264
- durability, 17, 20
- DynamicFrame, 148, 149
- DynamoDB
 - capstone, 67
- DynamoDB Accelerator, 73, 76
- DynamoDB cache, 76
- DynamoDB Streams, 58, 62
 - capstone, 67
- DynamoDB table, 60
- e-commerce backend, 83
- edge compute, 23
- effective dating, 96
- elastic compute
 - equivalence, 291
- ElastiCache
 - capstone, 83
- ElastiCache for Redis, 75
- EMR
 - capstone, 146
 - commands, 312
 - equivalence, 297
- EMR on EKS, 139, 141
- EMR Serverless, 139, 141
- EMRFS, 139–141
- encryption, 27
- endpoint policy, 255
- enhanced fan-out, 179
- Enhanced Monitoring, 42
- envelope encryption, 255
- environment promotion, 283
- environment setup, 306
- erasure
 - capstone, 278

- equivalence, 304
- ETL
 - equivalence, 298
- EVEN distribution, 108
- event bus, 159
- event time, 197, 199
- event-driven architecture, 59
- EventBridge, 159
- eventual consistency, 60, 76
- eviction, 76
- exactly-once, 197, 200
- exam domains, 328
- exercises
 - solutions, 322
- expand and contract, 280
- expand-contract, 283
- F1 score, 219
- fact table, 95
- failover, 42
- failover drill, 46
- fallback behavior, 77
- fan-out pattern, 178
- feature group, 235
- Feature Store, 234
- feature store
 - equivalence, 302
- File Gateway, 23
- Firehose
 - capstone, 194
 - commands, 313
 - equivalence, 300
- Flex execution, 151
- Flink
 - capstone, 204
 - commands, 313
 - equivalence, 300
- format conversion, 187, 189
- forum schema, 63
- fraud detection, 213
- Free Tier, 1, 308
- full load, 45, 158
- gaming leaderboard, 64
 - capstone, 67
- gateway endpoint, 255
- GitHub Actions, 280, 283
- Glacier Flexible Retrieval, 24
- global secondary index, 58, 62
- Global Tables, 58, 63
 - capstone, 67
- glossary, 341
- Glue
 - commands, 312
- Glue Data Catalog, 119
- Glue Data Quality, 262
- Glue DataBrew, 148, 150
- Glue Schema Registry, 157, 159
- Glue Studio, 148, 150
- governance
 - checklist, 324
- governance mode, 22
- grain, 95
- GraphQL, 77
- Hadoop, 139
- hands-on project
 - distribution-style benchmark, 111
 - DMS CDC to S3, 160
 - EMR PySpark job, 142
 - Firehose transformation, 190
 - Flink SQL windows, 200
 - Glue crawler, Athena, and Spectrum, 120
 - Glue Data Quality gate, 265
 - Glue Studio pipeline, 151
 - Kinesis producer and consumer, 180
 - Lake Formation column security, 130
 - Macie quarantine, 274
 - MWAA DAG, 169
 - QuickSight dashboard, 209
 - Redshift ML churn, 220
 - Redshift Serverless star schema, 97
 - SageMaker Autopilot, 228
 - SageMaker Pipeline, 237
 - solutions, 316
 - Terraform, dbt, and GitHub Actions, 283
 - Textract extraction, 246
 - VPC endpoint for S3, 256
- hashing, 273
- healthcare, 27
- heterogeneous migration, 45
- HIPAA, 27
- Hive metastore, 119
- hot path, 74
- hot shard, 179
- HTTP API, 77
- hybrid cloud, 23
- hybrid ingestion, 73
- IAM
 - equivalence, 303
- IAM Identity Center, 1, 306
- IAM role, 94, 254
- idempotency, 151

- identity resolution, 206, 209
- in-database machine learning, 217
- in-database scoring, 220
- in-memory cache
 - equivalence, 294
- in-warehouse ML
 - equivalence, 301
- incremental transfer, 23
- index projection, 62
- infrastructure as code, 280
- inline transformation, 187, 189
- instance fleets, 140
- interface endpoint, 255
- interleaved sort key, 109
- interval join, 200
- interview questions, 331
- ISO-8601 timestamp, 61
- item, 60
- item collection, 61
- IteratorAgeMilliseconds, 179
- job bookmark, 151
- job bookmarks, 148
- K-Means, 219
- KCL, 179
- KEY distribution, 108
- key-value database, 58
- Kimball, 95
- Kinesis
 - capstone, 184
 - commands, 312
- Kinesis Data Firehose, 187
- Kinesis Data Streams, 177
- KMS, 19, 255
- lab prerequisites, 308
- lab solutions, 316
- Lake Formation
 - capstone, 135
 - commands, 315
- lake governance
 - equivalence, 297
- lakehouse, 118
 - capstone, 124, 135
 - equivalence, 296
- Lambda, 159
 - commands, 313
- Lambda cost, 189
- Lambda trigger, 62
- latency, 74
- leader node, 92
- leaderboard architecture, 64
- least privilege, 254
- legal hold, 22, 273
- LF-Tags, 127, 130, 263
- lifecycle policies, 17
- lifecycle policy, 21
- lineage, 234, 264
 - equivalence, 304
- list consistency, 20
- LOB migration, 45
- local secondary index, 58, 62
- Macie, 272
 - commands, 314
- managed data identifier, 272
- managed database, 41
- Managed Service for Apache Flink, 197, 198
- managed Spark
 - equivalence, 297
- managed storage, 93
- manifest, 94
- masking, 273
- master node, 140
- materialized view, 62, 78
- materialized views, 107
 - equivalence, 293
- medallion architecture
 - equivalence, 292
- medical imaging data lake, 30
- Memcached, 73, 75
- messaging
 - equivalence, 299
- metadata, 18
- metastore, 118
- micro-batch, 95
- Milestone 1 capstone, 83
- Milestone 2 capstone, 135
- Milestone 3 capstone, 173
- Milestone 4 capstone, 213
- Milestone 5 capstone, 250
- Milestone 6 capstone, 287
- minimum storage duration, 21
- ML Insights, 209
- ML platform
 - equivalence, 302
- MLOps, 234
 - capstone, 241, 250
 - equivalence, 302
- mobile data API, 80
- mobile ingestion, 78
- Model Monitor, 234, 236
- Model Registry, 234, 236

- modules, 281
- MPP, 90
 - equivalence, 295
- MSK
 - commands, 313
- multi-account architecture, 287
- Multi-AZ, 40
- Multi-AZ deployment, 42
- multi-cloud, 290
- multi-factor authentication, 1, 306
- multi-region, 63
- multipart upload, 21
- MWAA
 - commands, 314
- MWAA execution role, 168
- noncurrent version, 21
- NoSQL, 58
 - equivalence, 292
- object key, 18
- Object Lock, 17, 22
- object storage, 17
 - equivalence, 290
- OCR, 243
 - equivalence, 303
- offline migration, 23
- offline store, 235
- OIDC federation, 283
- OLTP, 41
- on-demand capacity, 61
- online migration, 23
- online store, 235
- OpenLineage, 264
- OpenSearch, 245
- option group, 42
- Oracle ERP, 48
- Oracle ERP migration
 - capstone, 51
- Oracle migration, 40
- orchestration, 166
 - equivalence, 299
- parameter group, 42
- Parquet, 94
- Parquet conversion, 189
- partition key, 58, 177, 179
- partition pruning, 117
- Performance Insights, 42
- performance tuning
 - equivalence, 296
- periodic snapshot, 95
- permissions boundary, 253, 254
- physical partition, 60
- PII
 - capstone, 278
- PII discovery, 271
 - equivalence, 304
- point-in-time join, 235
- point-in-time recovery, 42
 - equivalence, 294
- portfolio projects, 13
- POSIX, 19
- prefix, 19
- private networking
 - equivalence, 303
- PrivateLink, 255
- processing time, 197, 199
- production readiness, 324
- provisioned capacity, 61
- ProvisionedThroughputExceeded, 179
- query acceleration
 - equivalence, 293
- query monitoring rules, 110
- query-first design, 61
- question bank, 331
- quick reference, 310
- QuickSight, 207
- quorum, 44
- RA3, 93
- RA3 nodes, 90
- RDS cache, 75
- RDS query cache, 78
- read amplification, 74
- read capacity unit, 60
- Read Replica, 43
- Read Replicas, 40
- read scaling, 43
- read-after-write consistency, 20
- reader endpoint, 44
- real-time endpoint, 227
- recovery, 21
- Redis, 73
- Redis cluster, 78
- redis-py, 78
- Redshift
 - capstone, 103
 - commands, 311
 - performance capstone, 115
- Redshift data sharing, 127
- Redshift ML, 217
 - capstone, 223

- equivalence, 301
- Redshift Serverless, 90
- Redshift Spectrum, 117
- Region, 1, 19
- regional failover, 44
- regression, 219
- relational databases, 40
 - equivalence, 291
- remote state, 281
- replica lag, 43
- replication, 17
- replication instance, 45, 158
- replication lag, 22, 76
- resolveChoice, 149
- resolver, 78
- REST API, 77
- retrieval fee, 21
- retry, 167
- reverse ETL, 206, 209
 - equivalence, 301
- right to erasure, 271, 273
- RocksDB state backend, 200
- root user, 1, 306
- rotation, 256
- row-level security, 127, 129, 206, 208
- RPO, 22, 44
- RTO, 44
- ruleset, 264
- S3
 - commands, 310
- S3 encryption, 83
- S3 lifecycle, 83
- S3 Object Lock
 - capstone, 30
- SageMaker
 - capstone, 232
 - commands, 314
 - equivalence, 302
- SageMaker Autopilot, 217, 225
- SageMaker endpoints, 225
- SageMaker Pipelines, 234
- SageMaker Studio, 225, 226
- Same-Region Replication, 22
- savepoint, 197, 200
- SCD Type 1, 96
- SCD Type 2, 96
- schema drift, 262
- schema evolution, 157
- schema inference, 119
- schema registry
 - equivalence, 298
- SDK, 307
- Secrets Manager, 253, 256
- security group, 76
- self-assessment, 330
- sensor, 167
- sensors, 166
- serverless database, 45
- serverless endpoint, 227
- serverless ETL, 148
 - equivalence, 298
- service control policy, 253, 254
- service mapping, 290
- session window, 199
- shard, 177, 179
- shared-nothing, 92
- short query acceleration, 110
- single-table design, 59
- single-table schema, 63
- slice, 92
- sliding window, 199
- slowly changing dimensions, 90
- Snow Family, 17
- Snowball, 23
- Snowcone, 23
- Snowmobile, 23
- SNS, 178
- sort key, 58, 107, 109
- sort-key prefix, 61
- source of truth, 74
- source-record backup, 190
- sparse index, 62
- Spectrum versus Athena, 120
- SPICE, 206, 207
- sports analytics, 80
- Spot Instances, 139, 140
- SQL
 - reference, 310
- SQS, 178
- SQS event notification, 24
 - capstone, 30
- star schema, 90
- state TTL, 200
- Step Functions, 168
 - capstone, 173
 - commands, 314
- storage classes, 17
 - equivalence, 290
- Storage Gateway, 17, 23
- stream delivery
 - equivalence, 300
- stream processing

- equivalence, 300
- stream-stream join, 200
- streaming
 - equivalence, 299
- strong consistency, 60
- STS
 - commands, 315
- study strategy, 329
- supplemental logging, 45
- sync frequency, 209
- synchronous replication, 42
- table format, 120
 - equivalence, 296
- table mapping, 45
- tagging, 1
- tags, 18
- Tape Gateway, 23
- task node, 140
- Terraform, 280, 281, 307
 - equivalence, 305
- Textract, 244
 - commands, 314
- throttling, 61
- time to live, 62
- time travel, 120
 - equivalence, 294
- tokenization, 271, 273
- traffic spike, 80
- training-serving skew, 235
- transaction fact, 95
- transaction processing, 41
- transient cluster, 140, 142
- Translate, 245
- TTL, 62, 75
- tumbling window, 199
- two-phase commit, 200
- UNLOAD, 206
- unstructured data, 243
- use case
 - banking reconciliation orchestration, 170
 - data clean room, 132
 - data mesh, 266
 - data perimeter, 257
 - drift-triggered retraining, 238
 - enterprise BI access layer, 210
 - Glue versus EMR, 152
 - governed GPU training, 229
 - Hadoop migration, 144
 - HL7 anonymization, 192
 - image processing pipeline, 247
 - inventory forecasting, 221
 - late mobile events, 201
 - media log lakehouse, 122
 - mobile game telemetry, 182
 - month-end BI cost model, 100
 - Oracle to Redshift CDC, 161
 - third-party PII onboarding, 276
 - warehouse performance rescue, 112
- Valkey, 75
- versioning, 17, 21
- virtual warehouse
 - equivalence, 291
- visibility timeout, 178
- Volume Gateway, 23
- VPC endpoint, 253
- watermark, 199
- watermarks, 197
- WebSocket API, 77
- wide-column
 - equivalence, 294
- windowing, 197
- WLM, 110
- worker type, 151
- workgroup limits, 119
- workload management, 107, 110
- WORM, 22
- write capacity unit, 60
- write sharding, 64
- write-through cache, 73, 75
- writer endpoint, 44
- XCom, 166
- XGBoost, 225
- zero-downtime migration, 48, 283
- zone map, 92, 109